

Associate Architect 양성과정 아키텍처 설계



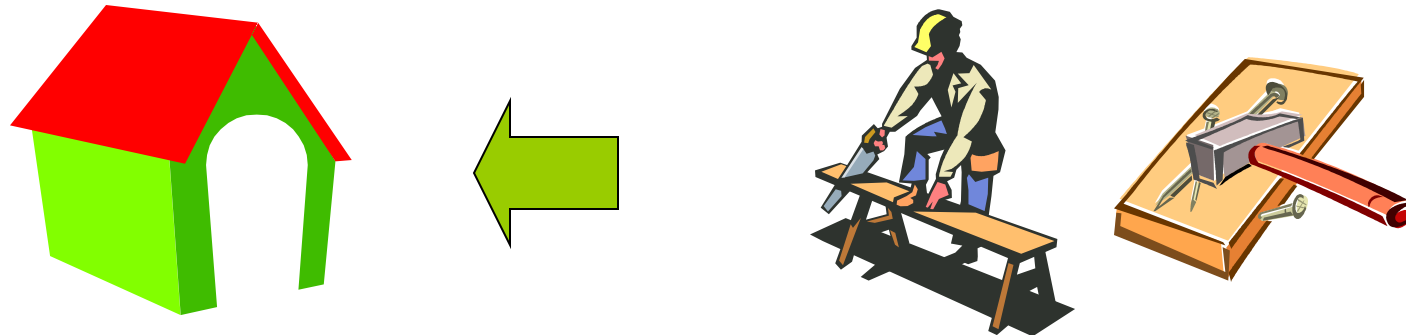
Course Introduction

Topics

- ❑ **Introduction**
 - Background
 - Software architecture definition
- ❑ **Why is Software Architecture Important?**
 - Early design decisions
 - Transferable abstraction of a system
 - Communication among stakeholders
- ❑ **About this course**
 - Course objectives
 - Architectural styles topics
 - Architectural views topics
 - Architectural tactics topics

Background

- Think about a situation where we are trying to build a new dog house.
- we can build a new one simply by cutting the wood and driving a nail into the wood

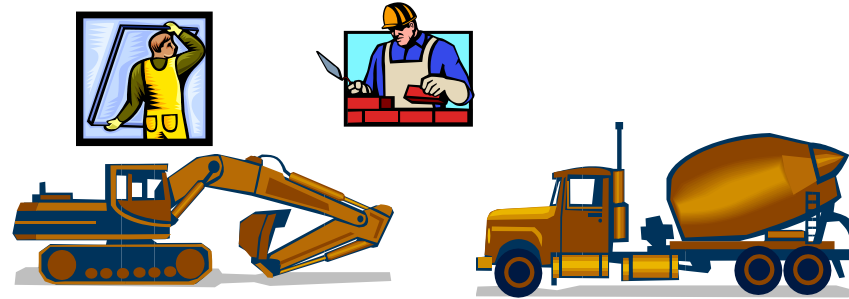


Background

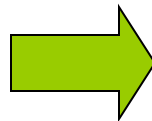
□ How about a situation where we have to build a two-storied house?

■ We need

- More workers
- More instruments



■ We may need to draw a blue-print before constructing the house

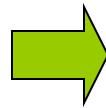


Background

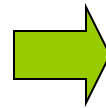
- What if we want to construct a high-rise building
 - Many specialists are involved
 - The high-rise building cannot be built without careful planning and design
 - **Architectures** are the key to the successful construction of the building



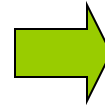
planning



design



construction



Background

□ Analogy

- As the size and complexity of software systems increases, **the design of overall system structure-software architecture-** become more significant issues than the choice of algorithms and data structures of computation.
- **Structural issues include**
 - The organization of a system as a composition of components
 - The protocols for communications
 - The assignment of functionality to design elements
 - Physical distribution, etc.

Software Architecture Definition

□ Definition

- **“The software architecture of a program or computing system is the structure or structures of the system, which comprise**
 - software elements,
 - the **externally visible properties** of those elements
 - the relationships among those elements”
- **“Externally visible property”**
 - Assumptions other elements can make of an element
 - **Example**
 - The services or performance characteristics of an element

Implications on the Definition

- ❑ **Every computing system with software has a software architecture**
- ❑ **Systems can comprise more than one structure**
 - Module structure, run time structure, allocation structure
- ❑ **Architecture is an abstraction**
 - local information about elements are suppressed
- ❑ **Architecture includes behavior**
 - The behavior of each element is part of the architecture insofar as that behavior can be discerned from the point of view of another element

Why is Software Architecture Important?

- ❑ **Inhibiting or enabling a system's quality attributes**
- ❑ **Transferable abstraction of a system**
- ❑ **Communication among stakeholders**

Architecture inhibiting or enabling a system's quality attributes

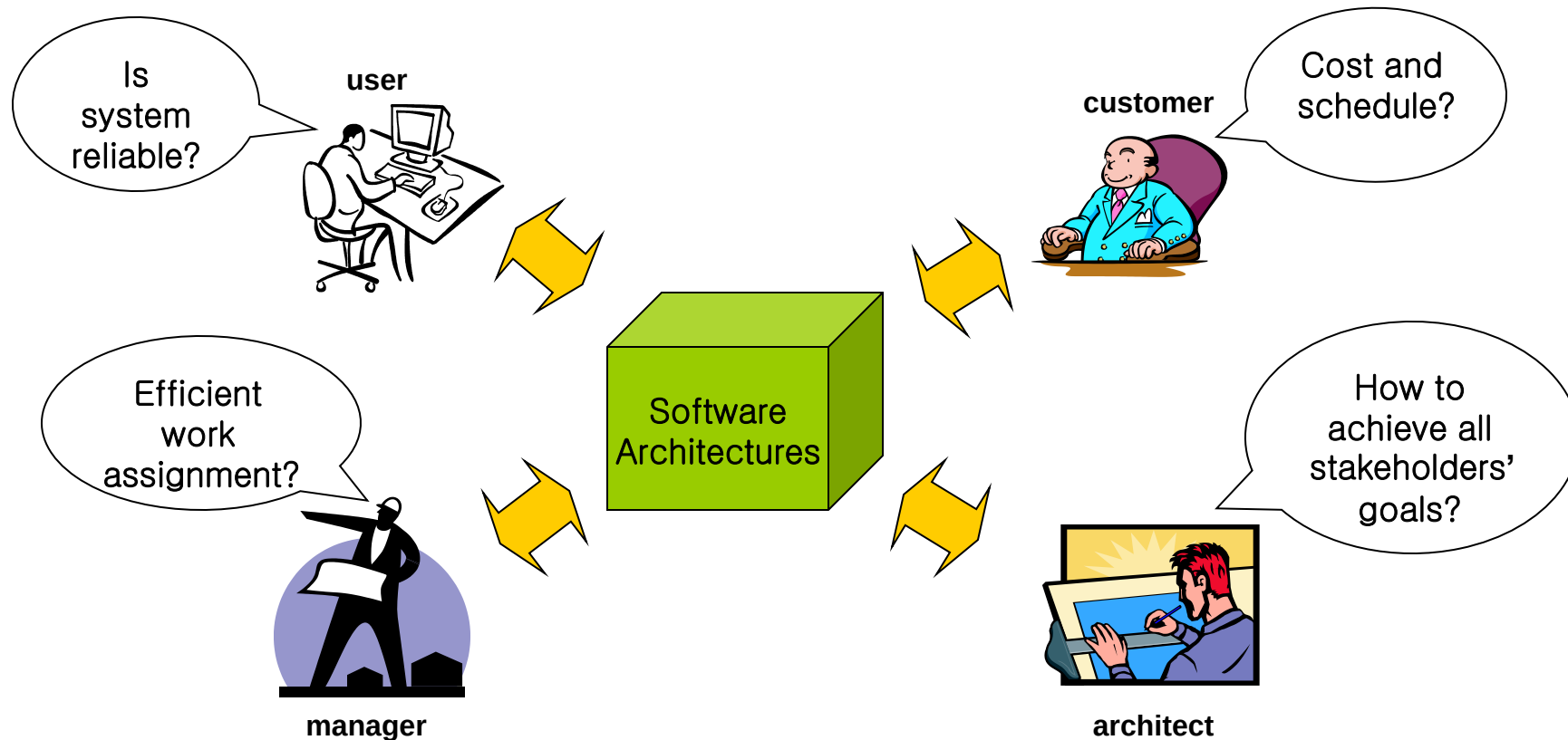
- ❑ **The architecture may substantially determine whether a system will be able to achieve its desired quality attributes**
 - If your system requires **high performance**, then you need to pay attention to managing the time-based behavior of elements
 - If **modifiability is important**, then you need to pay careful attention to assigning responsibilities to elements so that the majority of changes to the system will affect a small number of those elements
- ❑ **Achieving quality attributes must be considered throughout design, implementation, and deployment**
 - **Quality attributes must be considered from the beginning of architectural design**
 - Earliest architectural decisions are hardest to change later

Architecture as a transferable, reusable model

- ❑ When an architecture can be reused across multiple systems, all of the early architectural decisions are also transferred and reused.
- ❑ Design Concepts for Architectural reuse
 - **Styles** – recurring system structures and patterns
 - **Tactics** – common solutions used for achieving specific quality attributes
 - **Reference architecture** - a standardized model that outlines architectural guidelines for building a specific type of applications
 - **Framework** – a structured set of software elements that provide a foundation of building applications
 - **Product Line Architecture** – reusable architecture for a set of products in a software product line

Architecture is the vehicle for stakeholder communication

□ Architecture supports communication between stakeholders



Course Objectives

- ❑ Understand the basic concept of software architecture in software development
- ❑ Recognize some **major architectural styles and patterns** and analyze their implications on quality attributes
- ❑ Present architecture in **multiple viewpoints**
- ❑ Understand **various tactics that satisfy specific quality attributes**
- ❑ Generate architectural alternatives for a problem and choose among them with rationale

Topics in this course

□ **Architectural Styles**

- Each of the 23 major architectural styles is described in terms of configurations, examples, strengths, and weaknesses

□ **Architectural Views**

- For each architectural view from various perspectives, the characteristics, usage, how to create views, examples of how to create views, etc. are explained.

□ **Architectural Tactics**

- Description of 10 categories of tactics for achieving quality attributes
- A best practice for architectural design with patterns and tactics

Architectural style topics

□ Category of Architectural Styles

Category	Style
Data Flow	Batch Sequential
	Pipe and Filter
	Process Control
Data Centered	Shared Repository
	Blackboard
Implicit Invocation	Event Based
	Message Based
Interaction-Oriented	Model-View-Controller
	Presentation-Abstraction-Control

Category	Style	Sub-style
Hierarchical	Main-Subroutine	
	Master-Slave	
	Layered	Virtual Machine
		Microkernel
Distributed	Multi-tier	Client-Server
	Broker	Dispatcher
	Service-Oriented	Microservice
Deployment	Container-Based	
	Cloud-Based	
	Edge Based	

Updates From the Previous Version

□ Architectural Style Topics

- **Adding examples of style application in AI-related systems**
 - Batch Sequential Style, Pipe and Filter Style, Blackboard Style, Event-Based Style
- **Adding examples of style application in cloud based systems**
 - Virtual Machine Style, Dispatcher Style
- **Revise the concept and examples of MVC related Style**
- **Adding Deployment Architecture Styles**
 - Container-Based, Cloud-Based, Edge-Based Style

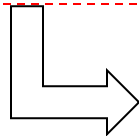
Architectural view topics

□ Introduction to Architectural views

- 4+1 View Model
- Siemens View Model
- SEI View Model

□ Documenting Architecture

- Documenting Architecture using SEI View Model
- Documenting Architecture using Hierarchical Multi-View Model
- Documenting Architecture using C4 Model



Added in the current(4th) version

Architecture tactic topics

- ❑ Introduction to Architectural Tactics

- ❑ Architectural Tactics

 - Availability Tactics

 - Deployability Tactics

 - Energy Efficiency Tactics

 - integrability Tactics

 - Modifiability Tactics

 - Performance Tactics

 - Safety Tactics

 - Security Tactics

 - Testability Tactics

 - Usability Tactics

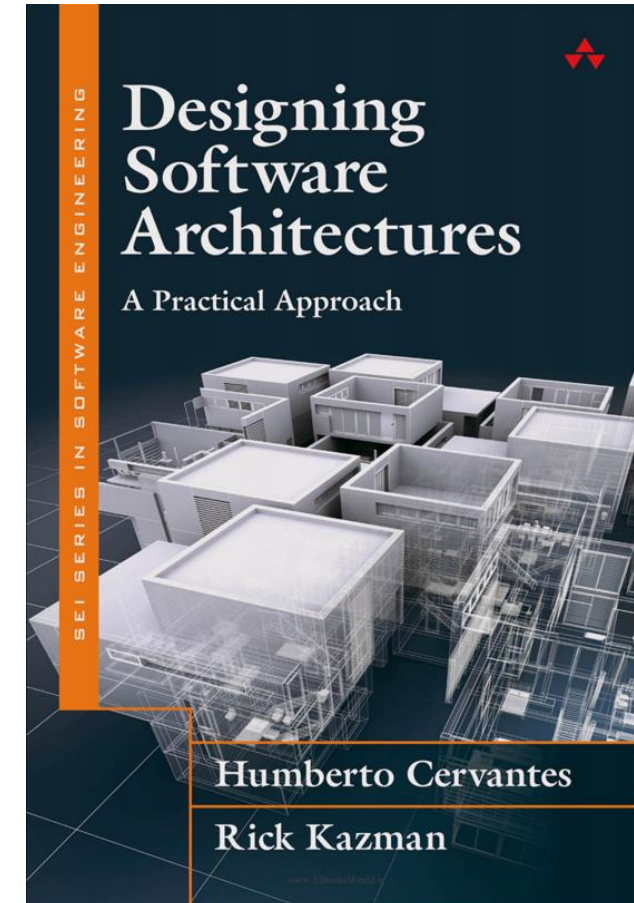
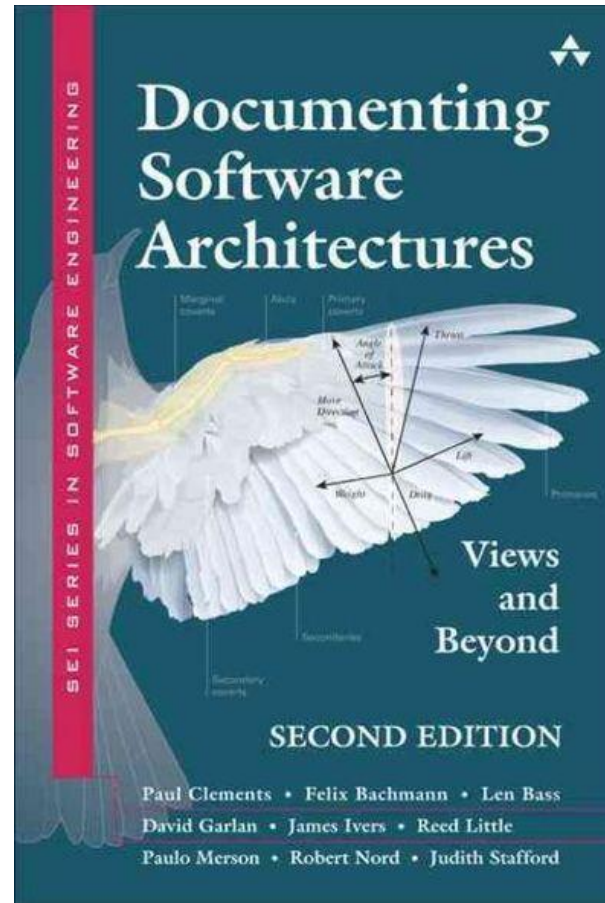
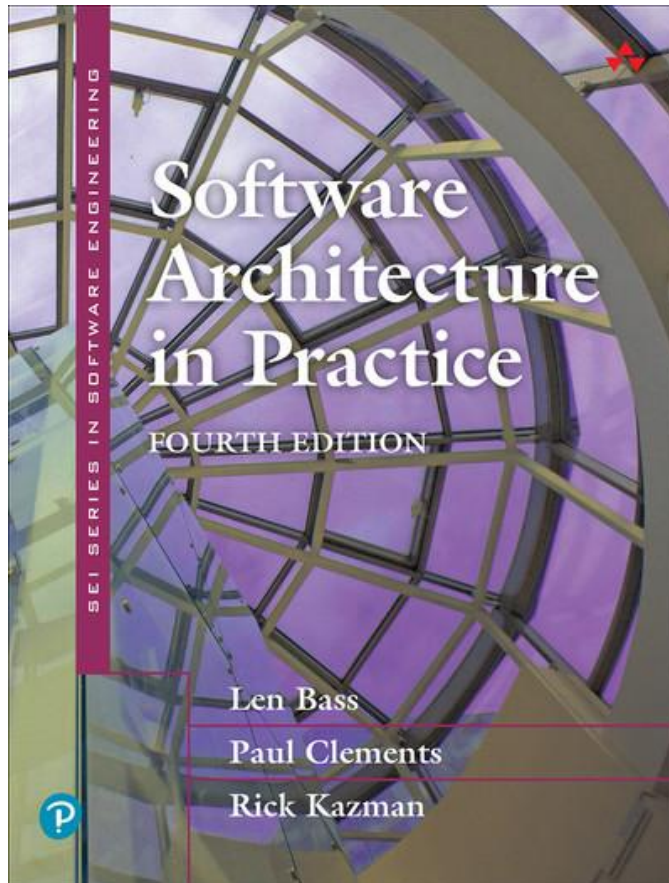
- ❑ Tactics and Architectural Patterns

- ❑ Architecture Design with Tactics and Patterns



Added in the current(4th) version

References



Associate Architect 양성과정

아키텍처 설계

Architecture Styles

Topics

- ❑ **Introduction to Architecture Style**
- ❑ **Category of Architecture Styles**
 - **Data Flow Architecture**
 - **Data Centered Architecture**
 - **Implicit Invocation Architecture**
 - **Interaction-Oriented Architecture**
 - **Hierarchical Architecture**
 - **Distributed Architecture**
 - **Deployment Architecture**
- ❑ **Summary**

Architecture Styles

INTRODUCTION TO ARCHITECTURE STYLE

Architectural Styles (Gothic Style)



뽕족한 첨탑

Architectural Styles (Gothic Style)



스테인드 글래스

Architectural Styles (Gothic Style)



높고 둥근
아치형 천장

Architectural Styles (Byzantine Style)



정사각형 벽위에
둥근 돔 설치

Architectural Styles (Byzantine Style)



모자이크 내부장식



Architectural Styles (Church Building)

□ Gothic Style

- 뾰족한 탑
- 높고 둥근 아치형 천장
- 스테인드 글라스



세인트 패트릭스
대성당, 뉴욕



밀라노 두오모, 밀라노

□ Byzantine Style

- 정사각형 벽위에 둥근 돔 설치
- 모자이크 내부 장식



성 마르크 성당, 베니스

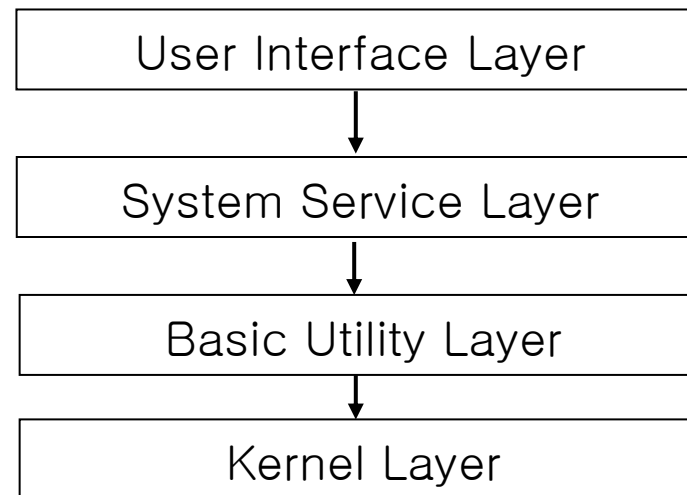
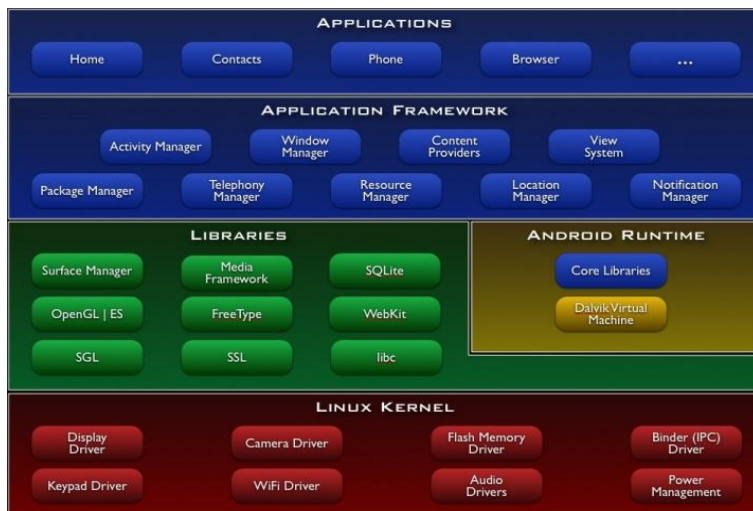


성 소피아 성당, 이스탄불

Architectural Styles (Software)

□ An architecture style

- describes a class of software architectures, which is found repeatedly in practice
- is an abstraction of structural features of the software system
- is a named collection of design decisions (e.g., element types, relationships between the elements, and various constraints)

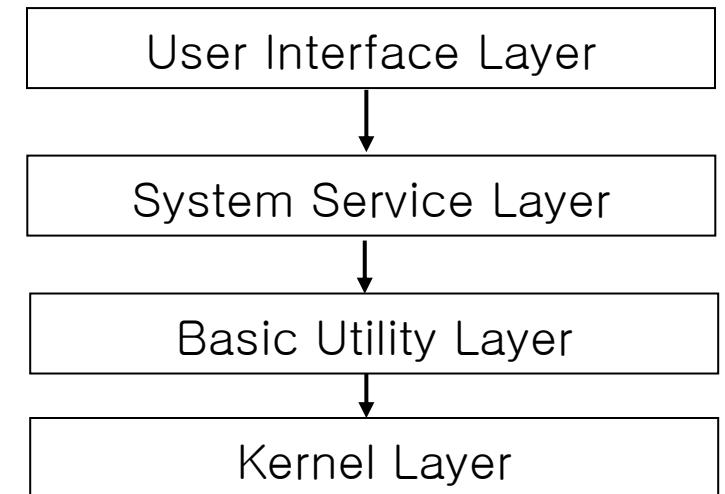


**Layered
Architecture**

Architectural Styles (Software)

- An architectural style is determined by:
 - a set of **element types**
 - a set of **relationships** between the elements
 - a set of **constraints** on the topology and behavior

- Layered Style
 - Elements
 - **Layer** (a kind of module)
 - Relations
 - **Allowed to use** (a specialization of a more generic depends-on relation)
 - Constraints
 - **The allowed-to-use relations should not be circular**



Why Architectural Styles

- An architectural style is **the abstraction** of similar software architectures
- Architectural styles help software engineers **reason about quality attributes**



Quality Attributes

Availability	The degree to which a system is in a specified operable state at the start of a mission, when the mission is called for at any time.
Modifiability	the degree of ease at which changes can be made to a system
Performance	the time it takes for the system to perform certain actions when events occur.
Security	the ability of the system to reduce the likelihood of malicious actions as well as the possibility of theft or loss of information
Testability	The easy with which software can be made to demonstrate its faults through testing
Usability	how easy it is for the user to accomplish a desired task

Quality Attributes

□ Availability

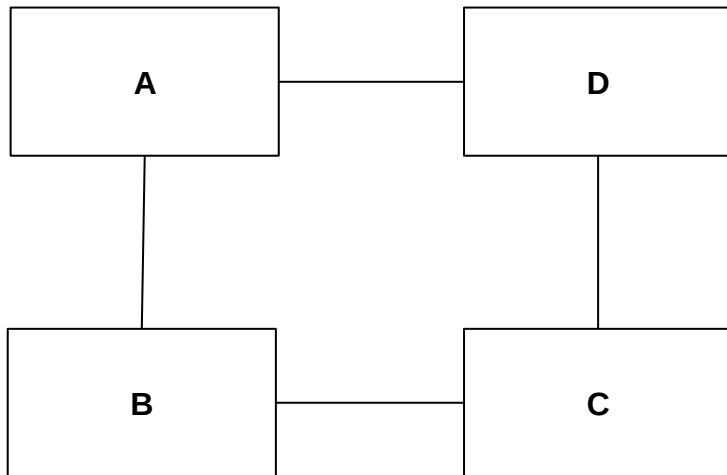
- The degree to which a **system is in a specified operable state at the start of a mission, when the mission is called for at any time**



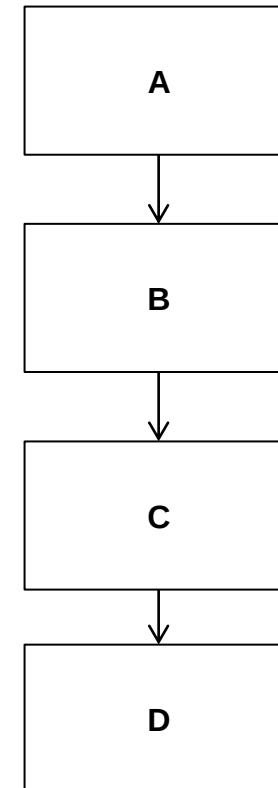
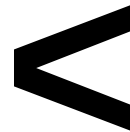
Quality Attributes

□ Modifiability

- the degree of ease at which changes can be made to a system.



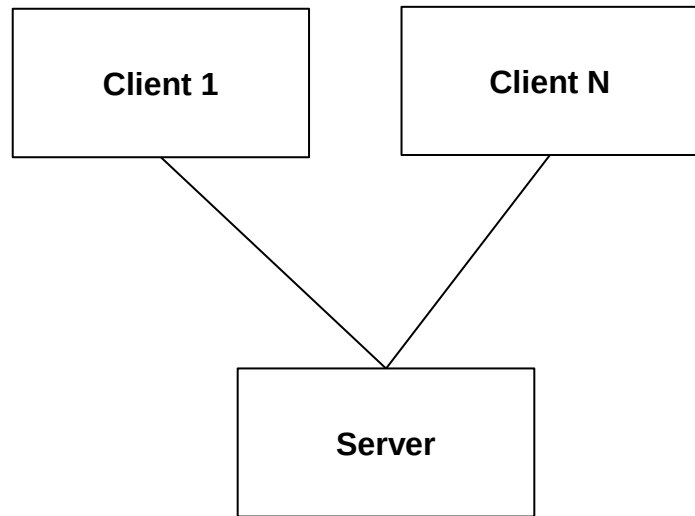
Modifiability



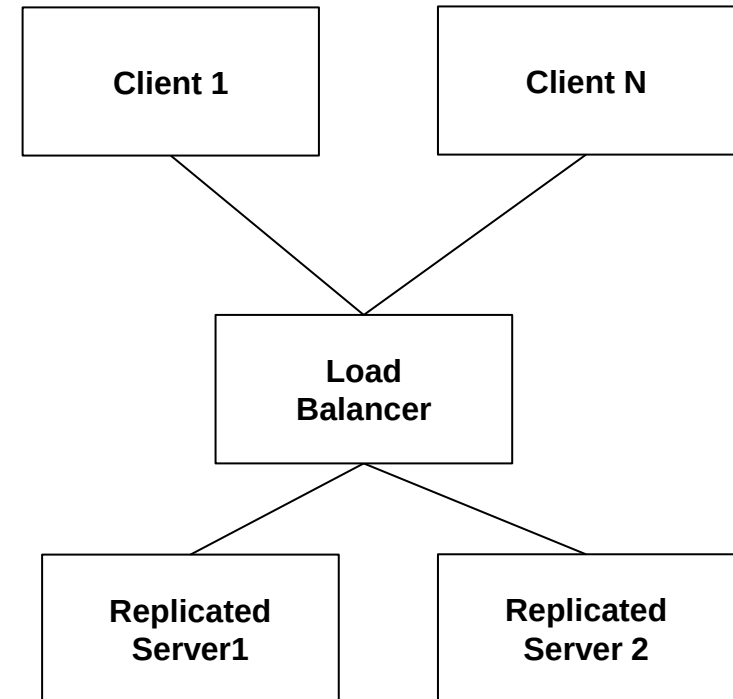
Quality Attributes

□ Performance

- the time it takes for the system to perform certain actions when events occur.



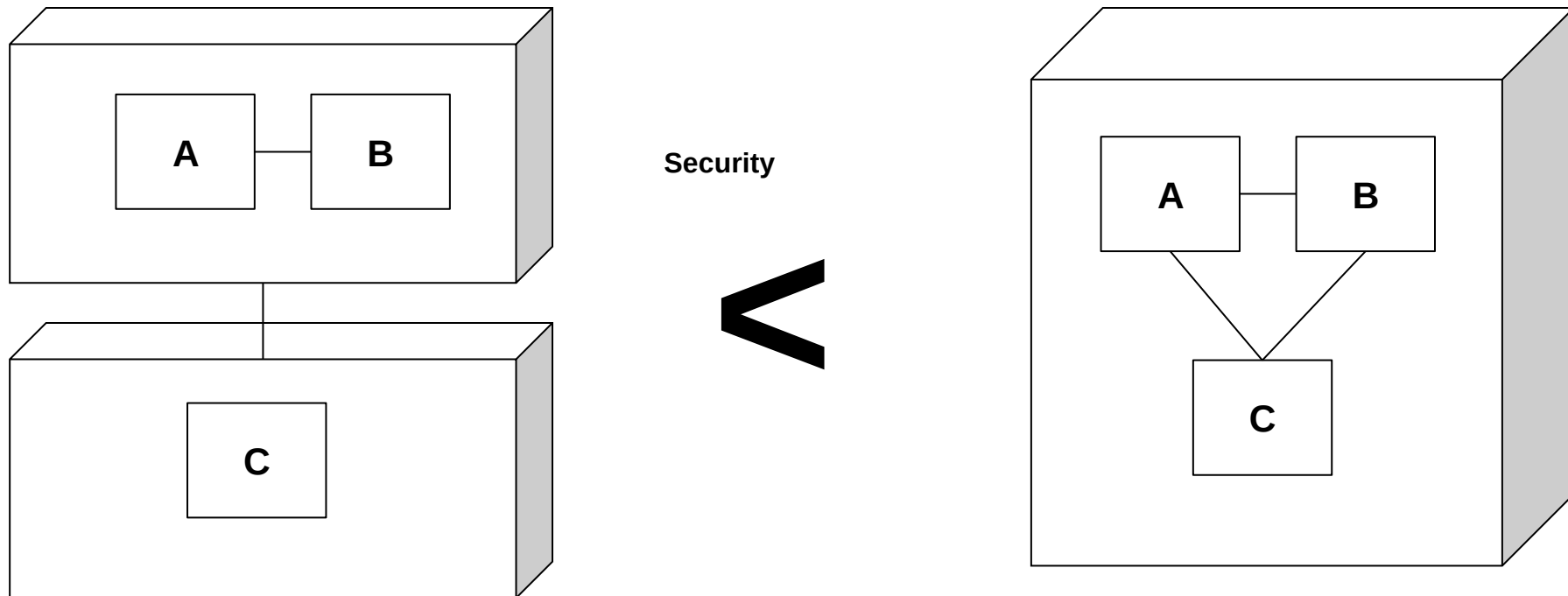
Performance



Quality Attributes

□ Security

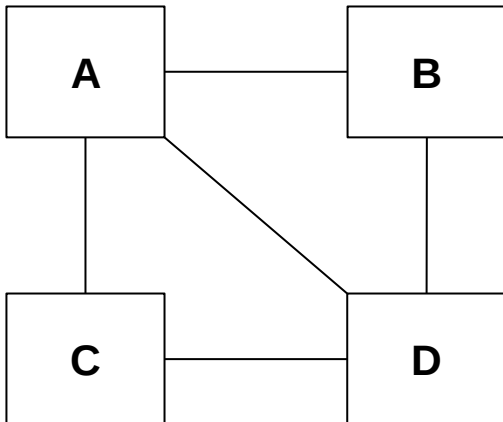
- the ability of the system to reduce the likelihood of malicious actions as well as the possibility of theft or loss of information.



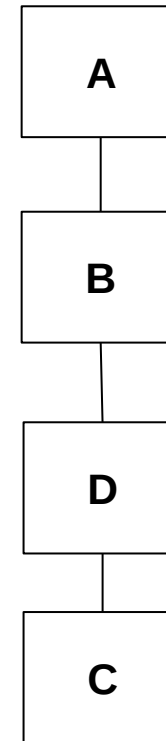
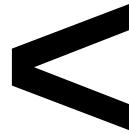
Quality Attributes

□ Testability

- The easy with which software can be made to demonstrate its faults through testing.



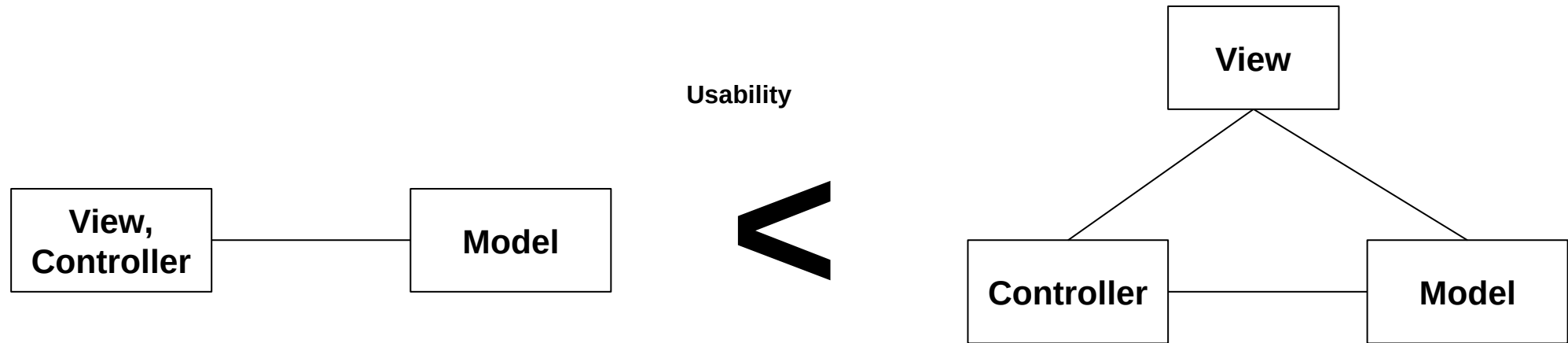
Testability



Quality Attributes

□ Usability

- how easy it is for the user to accomplish a desired task.



Architectural Styles Catalog

❑ **Garlan and Shaw compiled a catalog of architectural styles in 1995**

❑ **Others have augmented this list**

- there is **no complete, unique, non-overlapping list** of architectural styles
- systems exhibit multiple styles at once
- pure architectures are rarely found in non-trivial systems

- **Data flow**
 - batch sequential
 - pipes and filters
 - process control
- **Call-return**
 - main program-subroutine
 - object-oriented
 - component-based
 - peer-to-peer
 - service-oriented
 - N-tiered
- **Event-based**
 - asynchronous messaging
 - publish-subscribe
 - Implicit invocation
 - Data-triggered
- **Data-centered**
 - repository
 - blackboard

A Catalog of Architectural Styles

Category	Style
Data Flow	Batch Sequential
	Pipe and Filter
	Process Control
Data Centered	Shared Repository
	Blackboard
Implicit Invocation	Event Based
	Message Based
Interaction-Oriented	Model-View-Controller
	Presentation-Abstraction-Control

Category	Style	Sub-style
Hierarchical	Main-Subroutine	
	Master-Slave	
	Layered	Virtual Machine
		Microkernel
Distributed	Multi-tier	Client-Server
	Broker	Dispatcher
	Service-Oriented	Microservice
Deployment	Container-Based	
	Cloud-Based	
	Edge Based	

What is a Pattern?

- ❑ A pattern **is a recurring solution** to a recurring problem.
 - The solution schema is specified by describing
 - ❑ (Structure) Its constituent components, their responsibilities and relationships
 - ❑ (Dynamics) the way in which they collaborate
- ❑ Pattern Categories
 - Architectural patterns (architecture-level)
 - Design patterns (component-level)
 - Idioms (implementation-level)

What Makes a Pattern

□ Pattern Name

■ Context

- Design situation giving rise to a design problem

■ Problem

- Set of forces repeatedly arising in the context
 - Forces: requirements, constraints, desirable properties

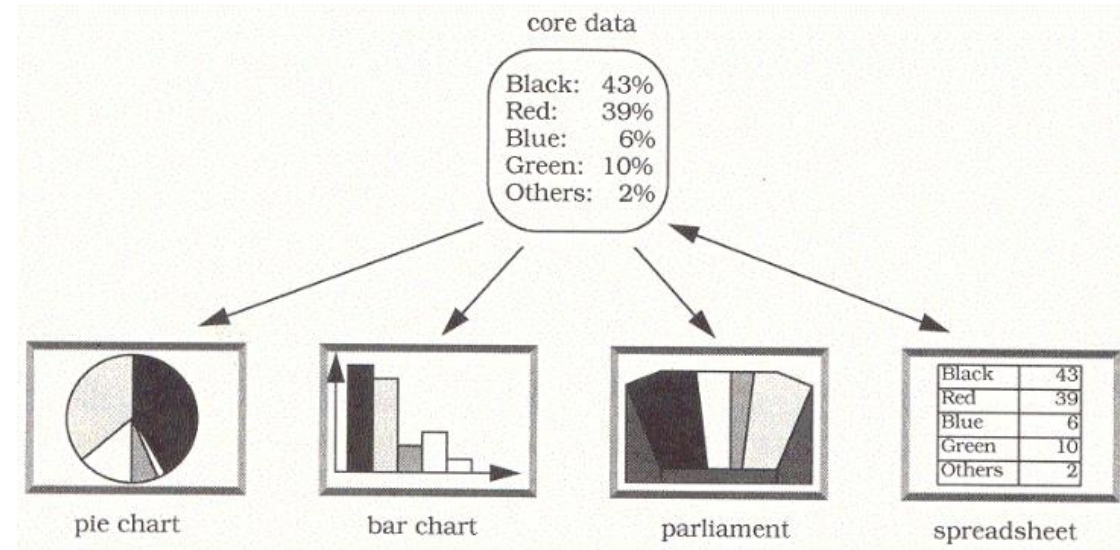
■ Solution

- Configuration to balance the forces
 - Structure with component and relationships
 - Run-time behavior

Example of a Pattern (MVC)

□ Context

- User Interface of a long-lived interactive system displays core data in multiple ways

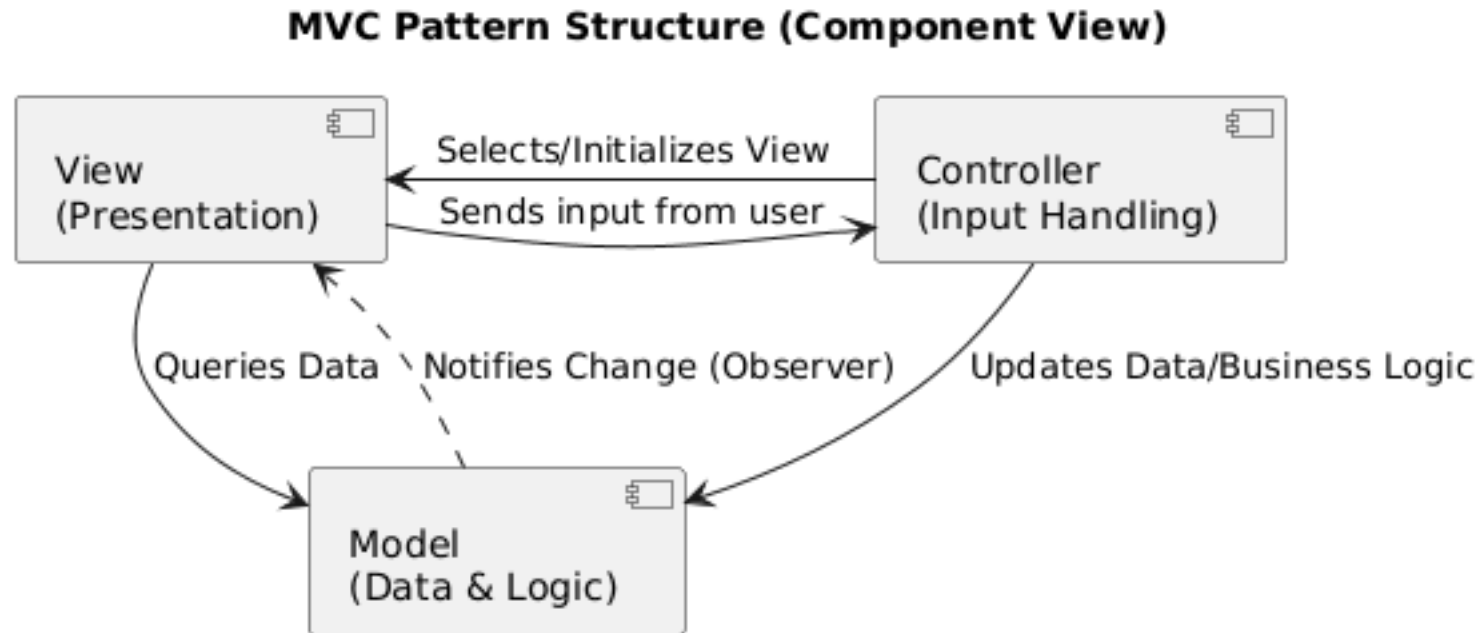


□ Problem

- Changes to the user interface should be easy, and possible at run time
- Adapting or porting the user interface should not impact code in the functional core of the application

Example of a Pattern (MVC)

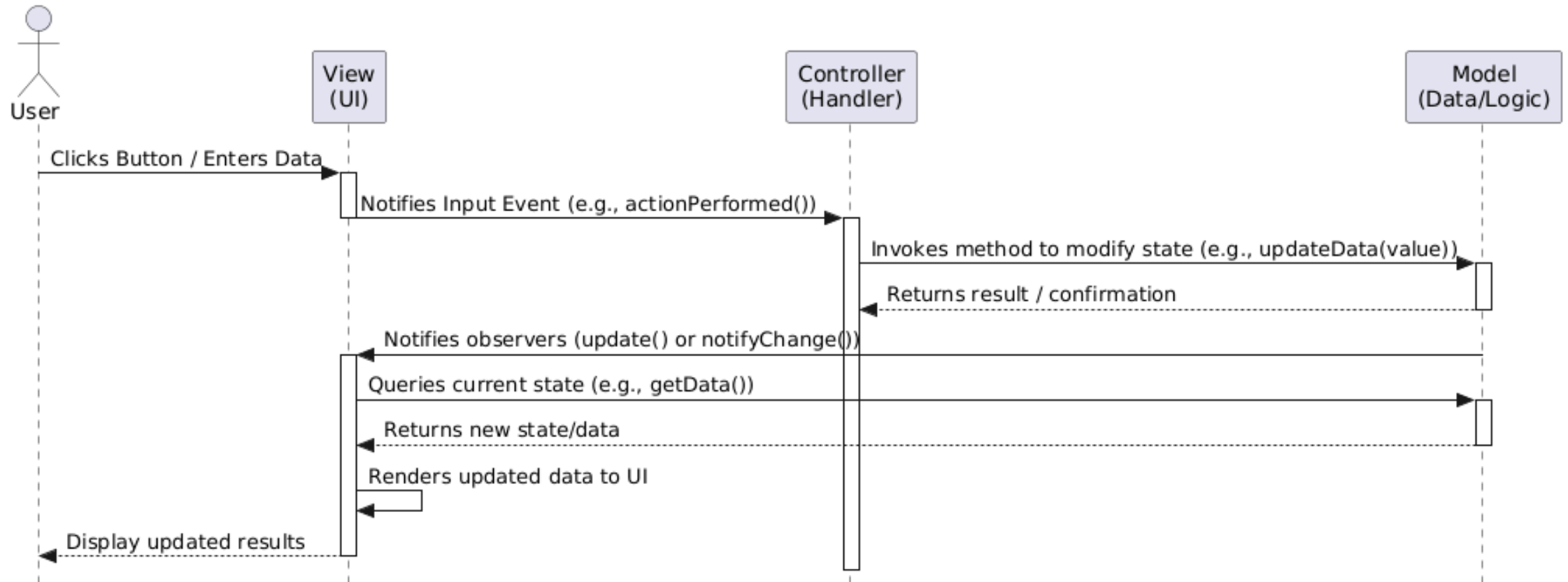
▣ Solution (Structure)



Example of a Pattern (MVC)

▣ Solution (Dynamics: Initialization Process)

MVC Pattern Dynamics (Sequence View)



Architectural Pattern vs. Architectural Style

- ❑ **Architectural patterns** focus on **problem-oriented architectural solution**, whereas **Architectural style** focus on **structural or behavioral characteristics** of related software architectures.
- ❑ Because the concept of architecture styles is very similar to the concept of patterns, in the field of architecture you can **use the terms “styles” and “patterns” synonymously**.

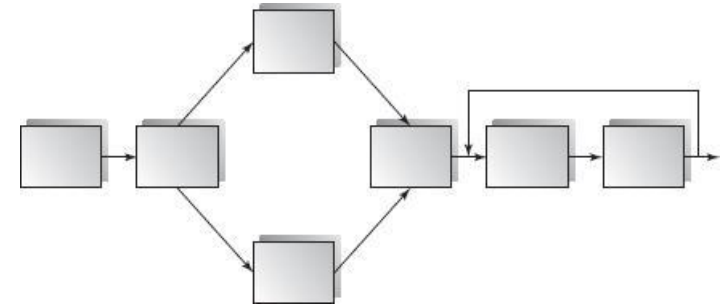
Architecture Styles

DATA FLOW ARCHITECTURE

Data Flow Architecture

❑ In a Data Flow system

- The design is characterized by **a series of transformations on successive pieces of input data**
- The pattern of data flow is explicit
- No other interactions between processes

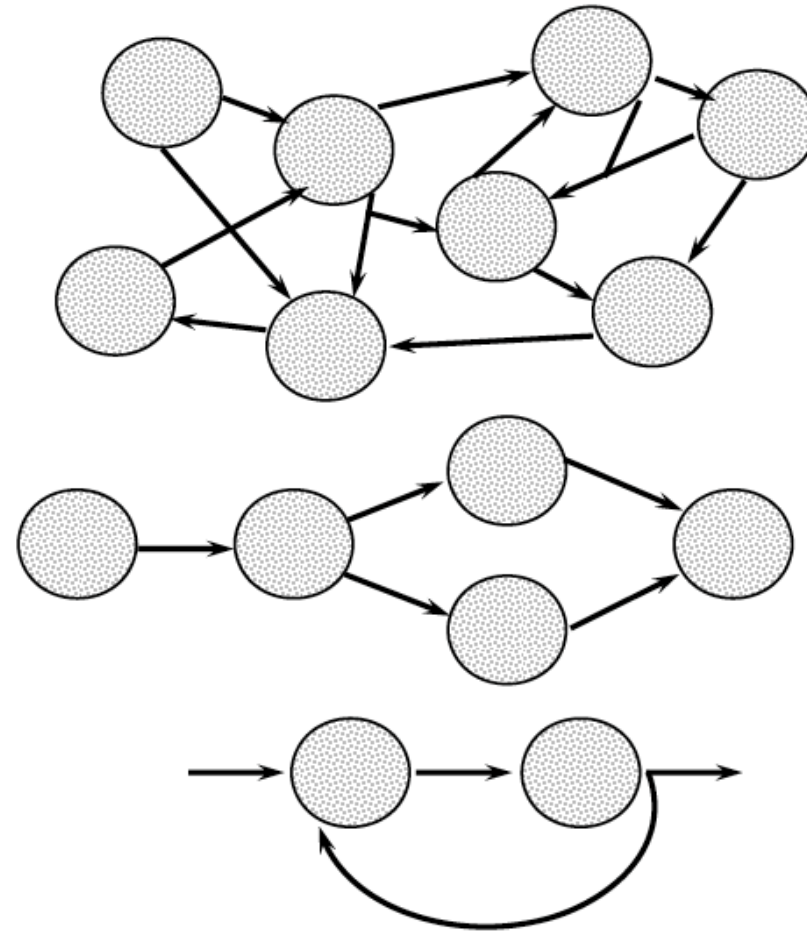


❑ Structure

- **Component (Data Transformer)**
 - ❑ Transforms its input data into corresponding output data
- **Connector (Data Channel)**
 - ❑ May be implemented as I/O streams, I/O files, buffers, etc.

Kinds of Data Flow Systems

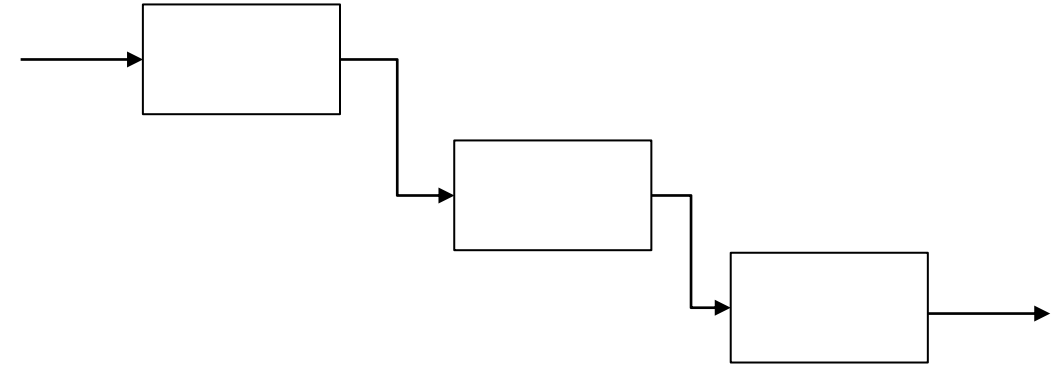
- ❑ In general, data can flow in arbitrary patterns
- ❑ Often, we are primarily interested in nearly linear data flow systems
- ❑ or in very simple, highly constrained cyclic structures



Styles of Data Flow Architecture

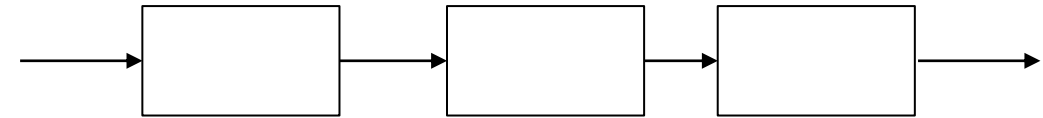
❑ Batch Sequential

- **Sequential processing** steps, run to completion



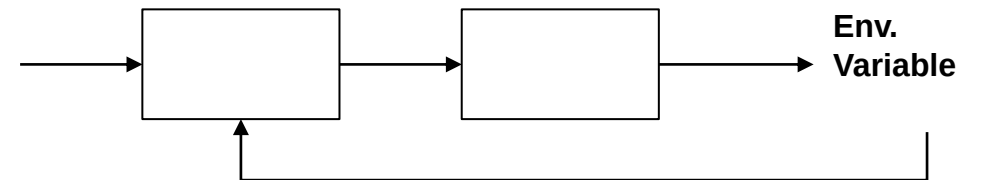
❑ Pipe-and-Filter

- **Incremental transformation** of streams



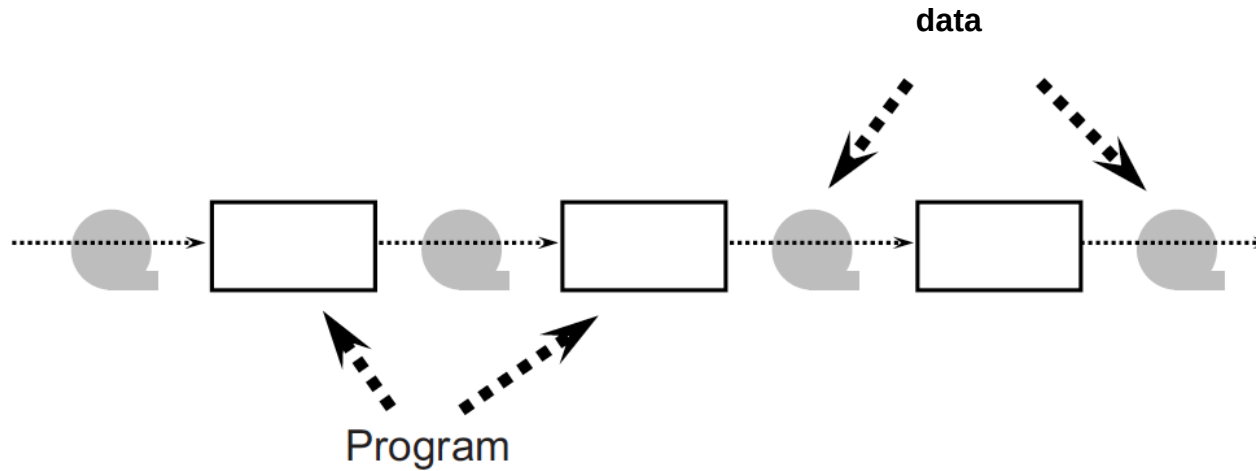
❑ Process Control

- **Looping structure** to control environment variables



Batch Sequential Styles

- The main feature of this architecture is as follows:
 - Separate programs are executed in order
 - Data is passed as an aggregate from one program to the next.



Batch Sequential Styles

□ Elements

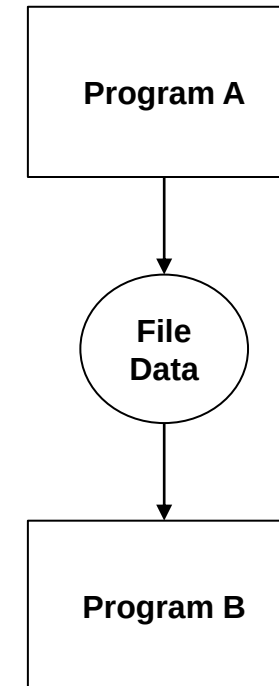
- Components: **independent programs**
- Connectors: **files**
 - Data is transmitted as a whole between programs, most likely in the form of a file.

□ Relations

- The file produced by one program serves as input to the next program

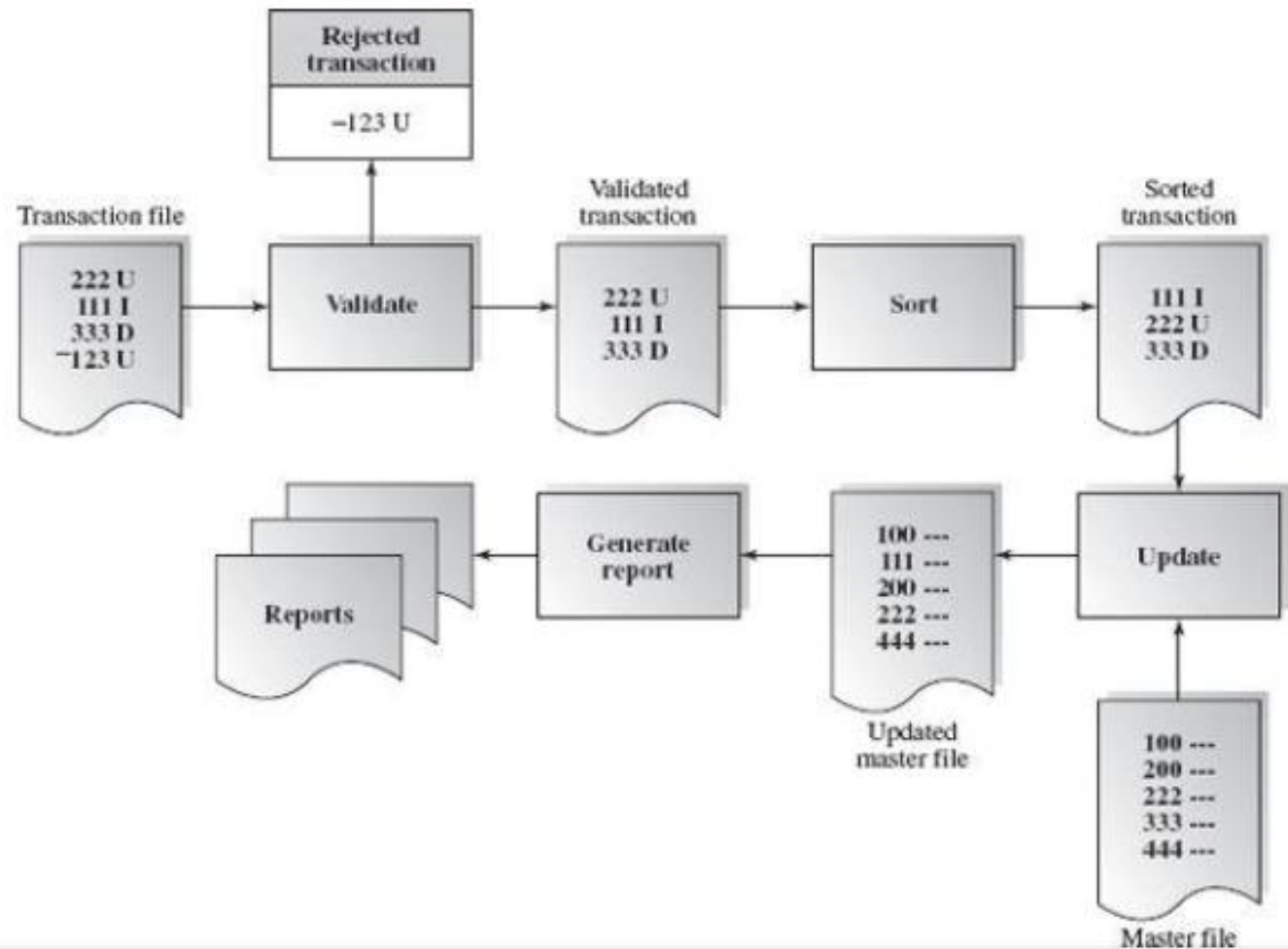
□ Constraint

- Each program runs to completion before the next program starts



Examples

Information Processing Architecture

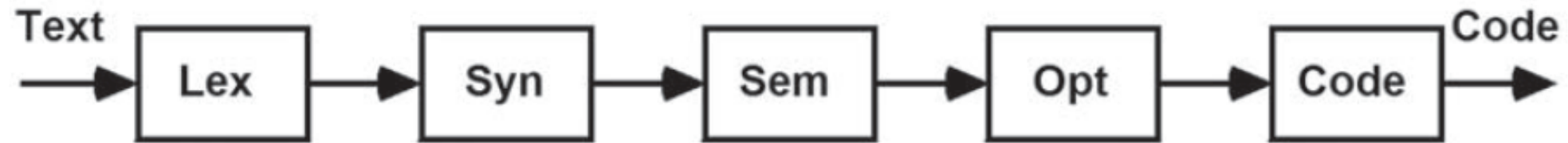


Examples - Programming Level

```
public class batch_sequential {  
    ...  
    public static void main() {  
        searching(kwd, inputFile, matchedFile);  
        counting(matchedFile, countedFile);  
        sorting (countedFile, reportFile);  
    }  
    public static void searching(String kwd, String inFile, String outFile) {...}  
  
    public static void counting(String inFile, String outFile)  
    {...}  
  
    public static void sorting(String inFile, String outFile)  
    {...} }
```

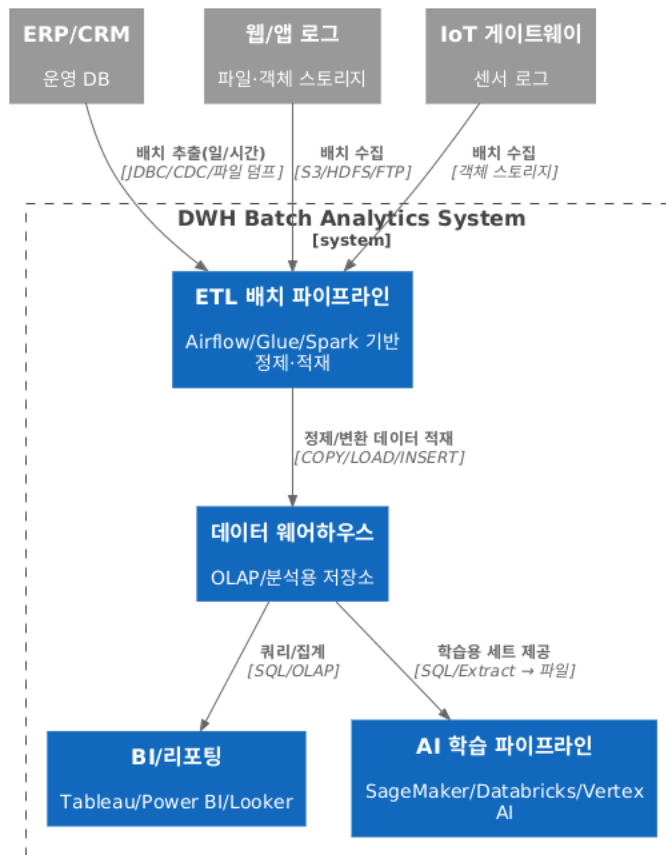
Examples

Compiler Architecture



Examples

데이터웨어하우스 기반 배치 처리 아키텍처



Consequences

□ Benefits

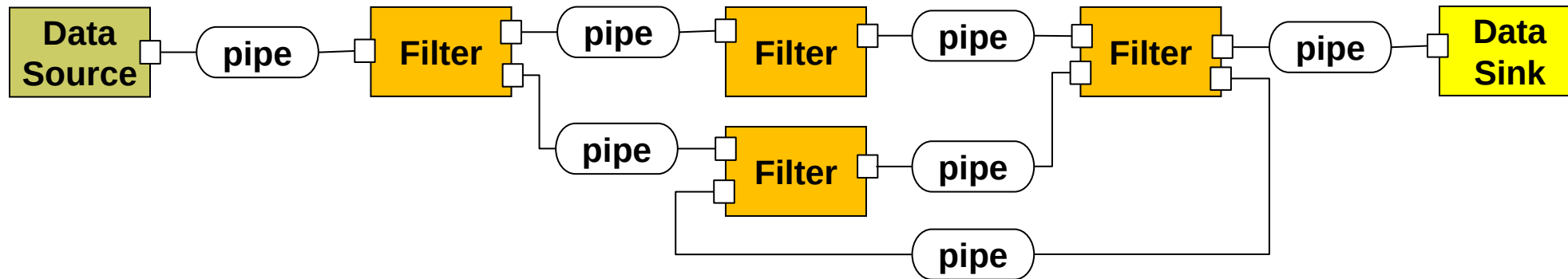
- Simple composition
- Efficiency in large-scale data processing
- Reuse of processing steps
- Easy to replace processing steps, if and only if I/O data match

□ Weakness

- Non-interactive
- No concurrency
- Increased latency and low throughput
- Limited availability

Pipe and Filter Style

- ❑ The main feature of this architecture is **its concurrent and incremented execution.**
 - A loop is allowed



Pipe and Filter Style

□ Elements

■ Component: **Filters**

- **Incrementally transform some of the source data** to sink data

■ Connector: **Pipes**

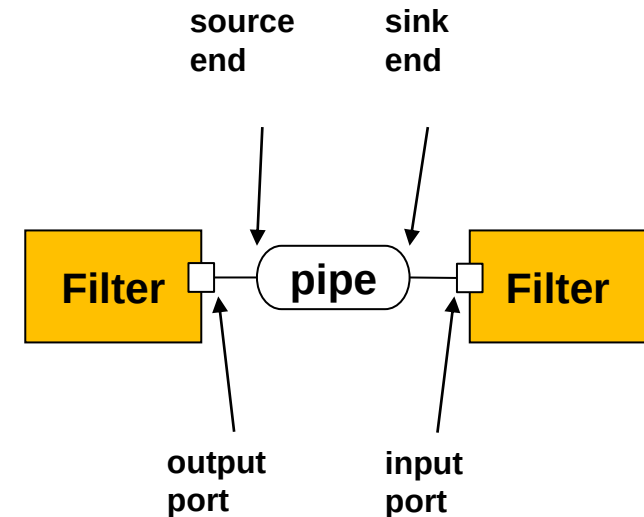
- One-way flow, order-preserving, data-preserving

□ Relations

- The source end of a pipe is connected to a filter's output port
- The sink end of a pipe is connected to a filter's input port.

□ Constraints

- Filters must be independent entities



Pipe and Filter Style

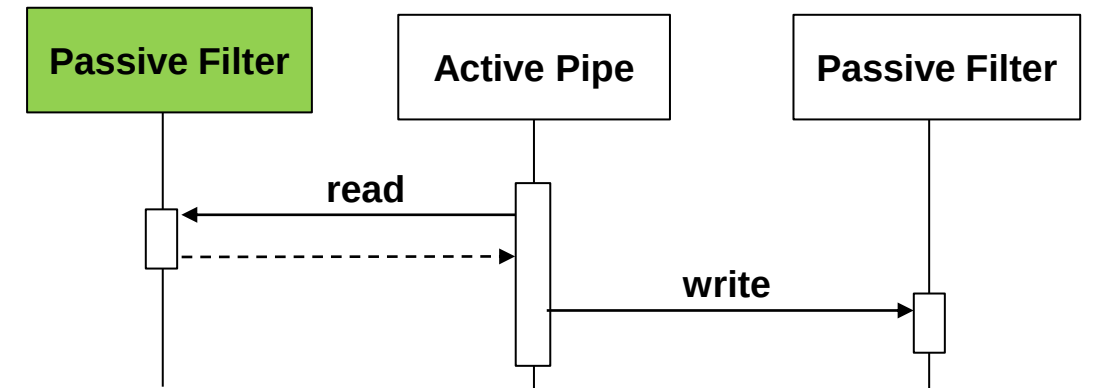
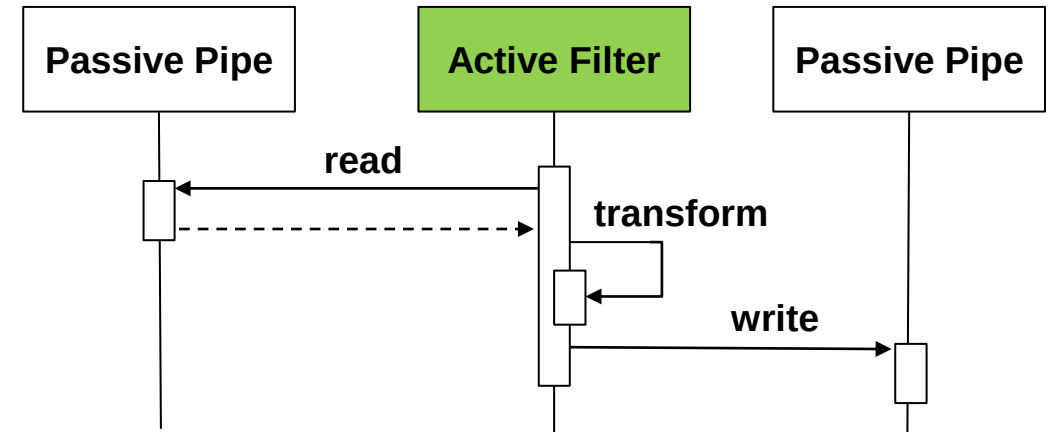
□ Two types of **filters**

■ Active filter

- Pulls in data and pushes out the transformed data
- It works with a passive pipe that provides read/write mechanisms for pulling and pushing.

■ Passive filter

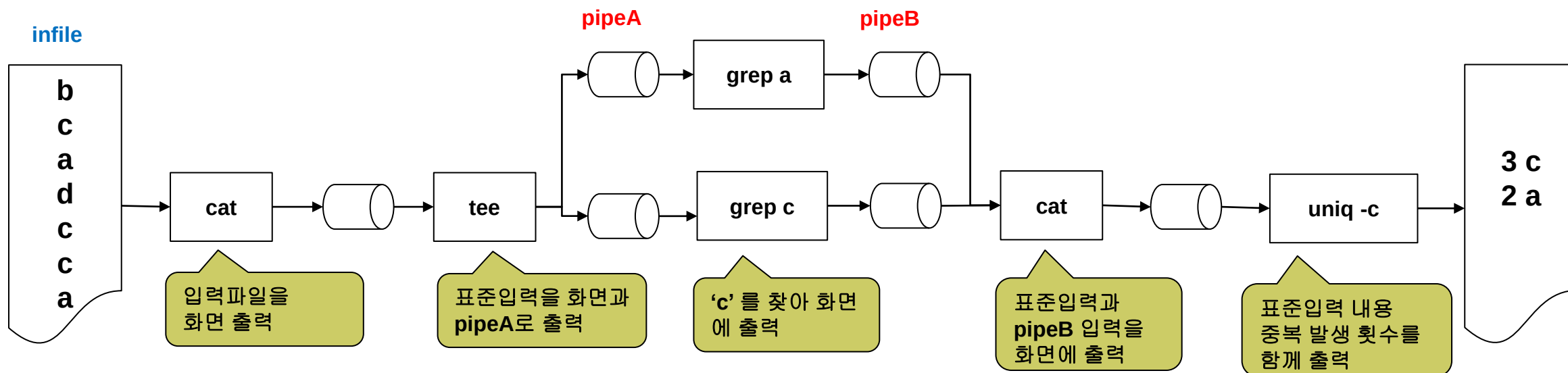
- It works with active pipes that pull data out from a filter and push data into the next filter.



Examples – Pipes and Filters in Unix

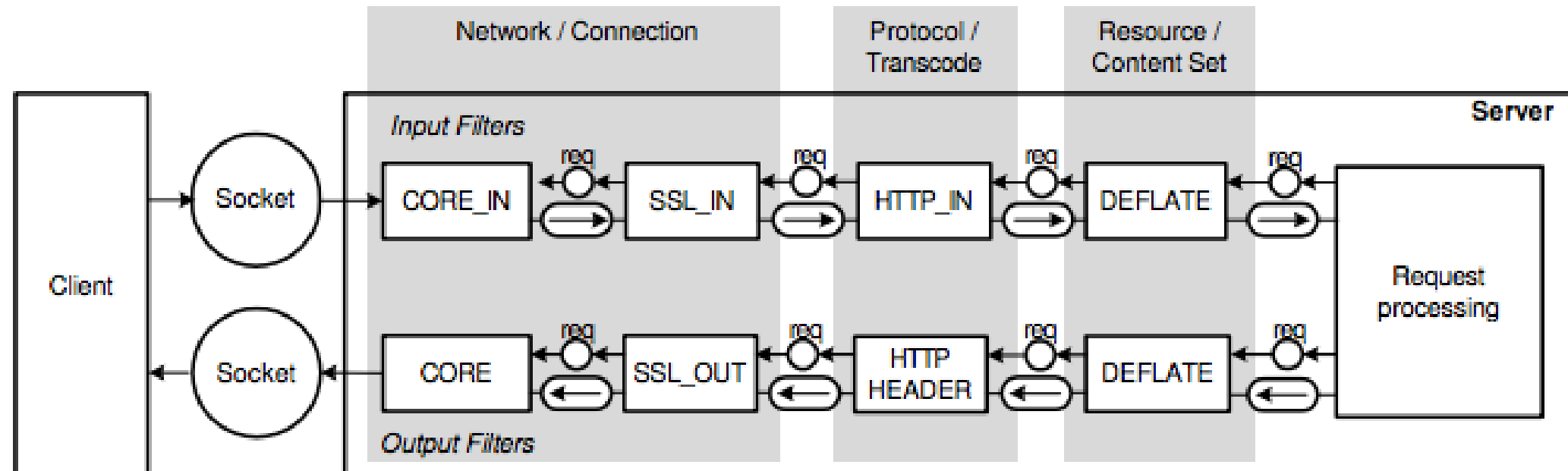
Named pipe between forward and backward processes

```
$ mkfifo pipeA
$ mkfifo pipeB
$ grep a < pipeA > pipeB &
$ cat infile | tee pipeA | grep c | cat - pipeB | uniq -c
```



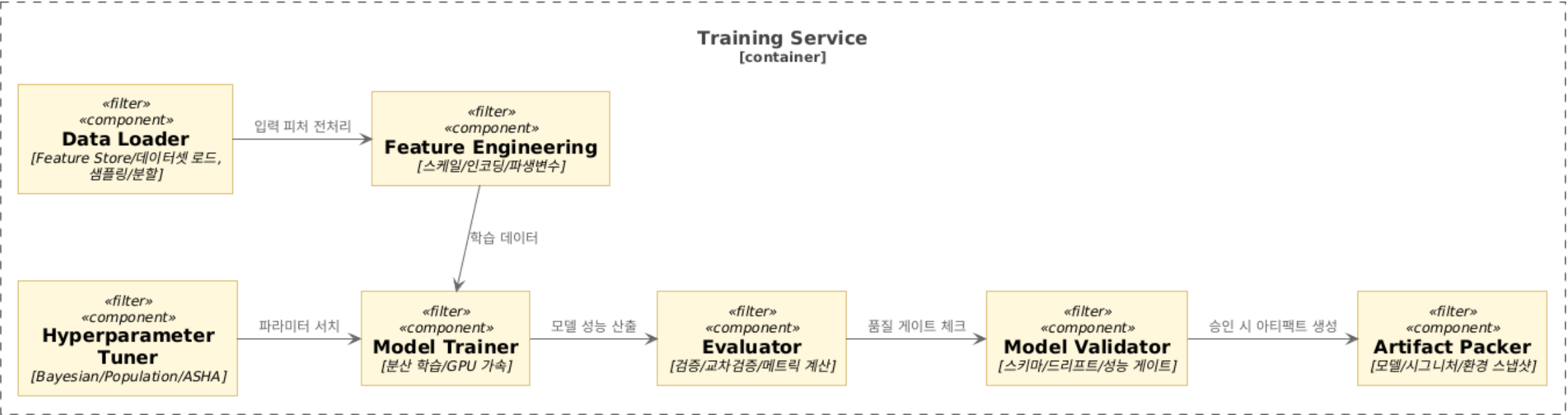
Examples - Apache Filters

□ Apache Filters



Examples

AI Training Pipeline Architecture



Consequences

□ Benefits

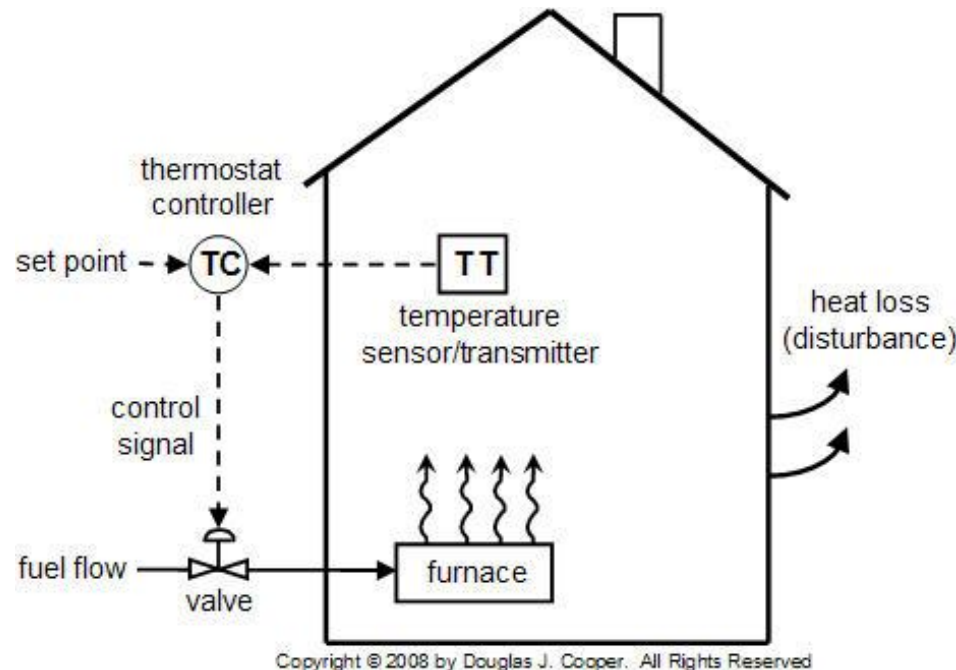
- Simple composition
- Reuse of filters
- Easy to maintain and enhance systems
- Enhancing performance through the distributed parallel processing of filters

□ Weakness

- Not good at handling interactive I/O
- Having large numbers of independent filters can add substantial amounts of computational overhead
- Pipe-and-filter systems may not be appropriate for long-running computations
- Low fault tolerance

Process Control Style

- **Process Control Architecture** is a type of Data Flow Architecture, where the **flow of data comes from a set of variables** which **controls the execution of process to maintain desired process outcomes**.



Process Control Style

□ Elements

■ Components

□ Control Unit

□ Processing Unit

■ Connectors: a set of variables

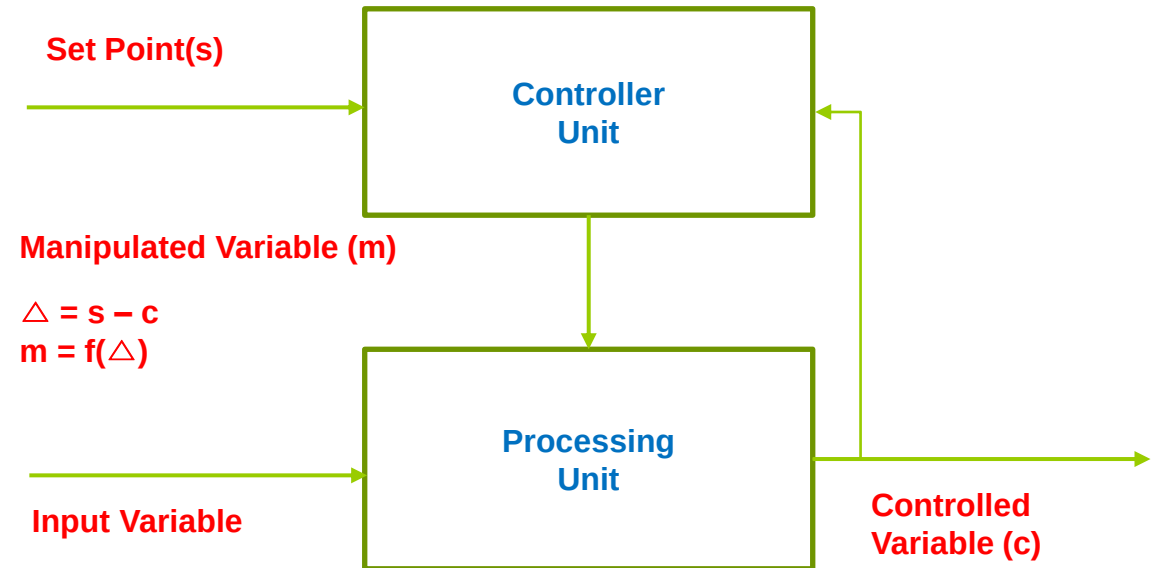
□ Set Point, Manipulated Variable,
Input Variable, Controlled Variable

□ Relations

■ See the figure on the right

□ Constraints

■ The value of **Controlled Variable** should be maintained as much as the value of **Set Point**.



Process Control Style

❑ Set Point

- It is the desired value for a controlled variable.

❑ Manipulated Variable

- Manipulated Variable value is adjusted or changed by the controller.

❑ Input Variable

- an input data to the processing unit

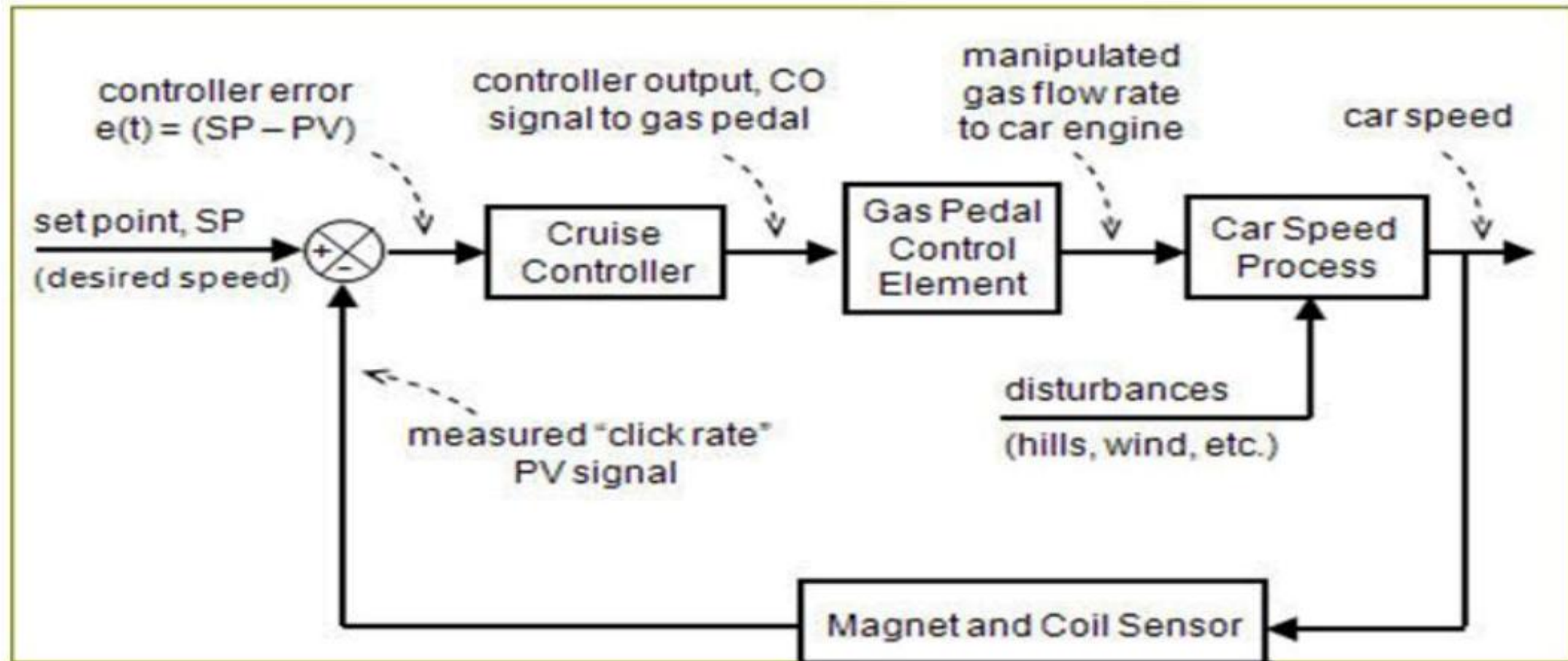
❑ Controlled Variable

- Controlled Variable provides values for the underlying system and should be measured by sensors.

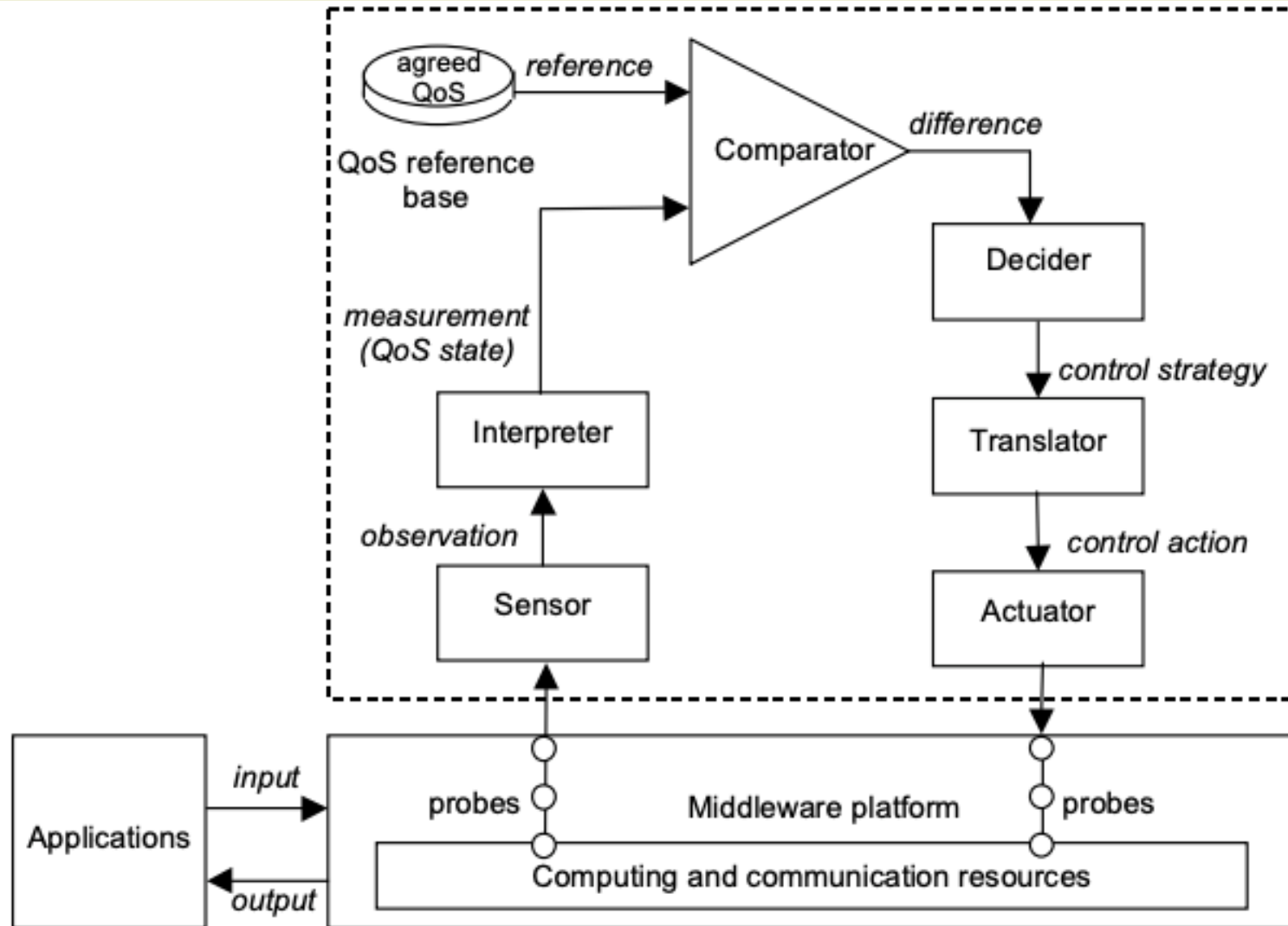
Application Area

- ❑ **Process control architecture is suitable for systems that need to maintain an output data at a stable level.**
 - **Cruise Control System**
 - **QoS Control System**

Example – Cruise Control System



Example – A QoS-Control Architecture for Object Middleware



Consequences

□ Benefits

- Maintaining optimal process conditions
- Enhancing reliability and robustness of processing

□ Weaknesses

- Complexity in design
- Any failures, malfunctions, or cybersecurity attacks in the processing and control units can potentially disrupt the control system

Comparisons of Data Flow Architectures

- ❑ If your problem can be decomposed into sequential states,
 - Consider **batch sequential** or **pipeline architectures**
 - If in addition each stage is incremental, so that later stages can begin before earlier stage finish
 - ❑ Consider **a pipeline architecture**
- ❑ If your problem involves transformation on continuous streams of data
 - Consider **a pipe and filter architecture**
- ❑ If your system involves controlling action which is subject to unpredictable external disturbances
 - Consider **a closed loop process control architecture**

Architecture Style

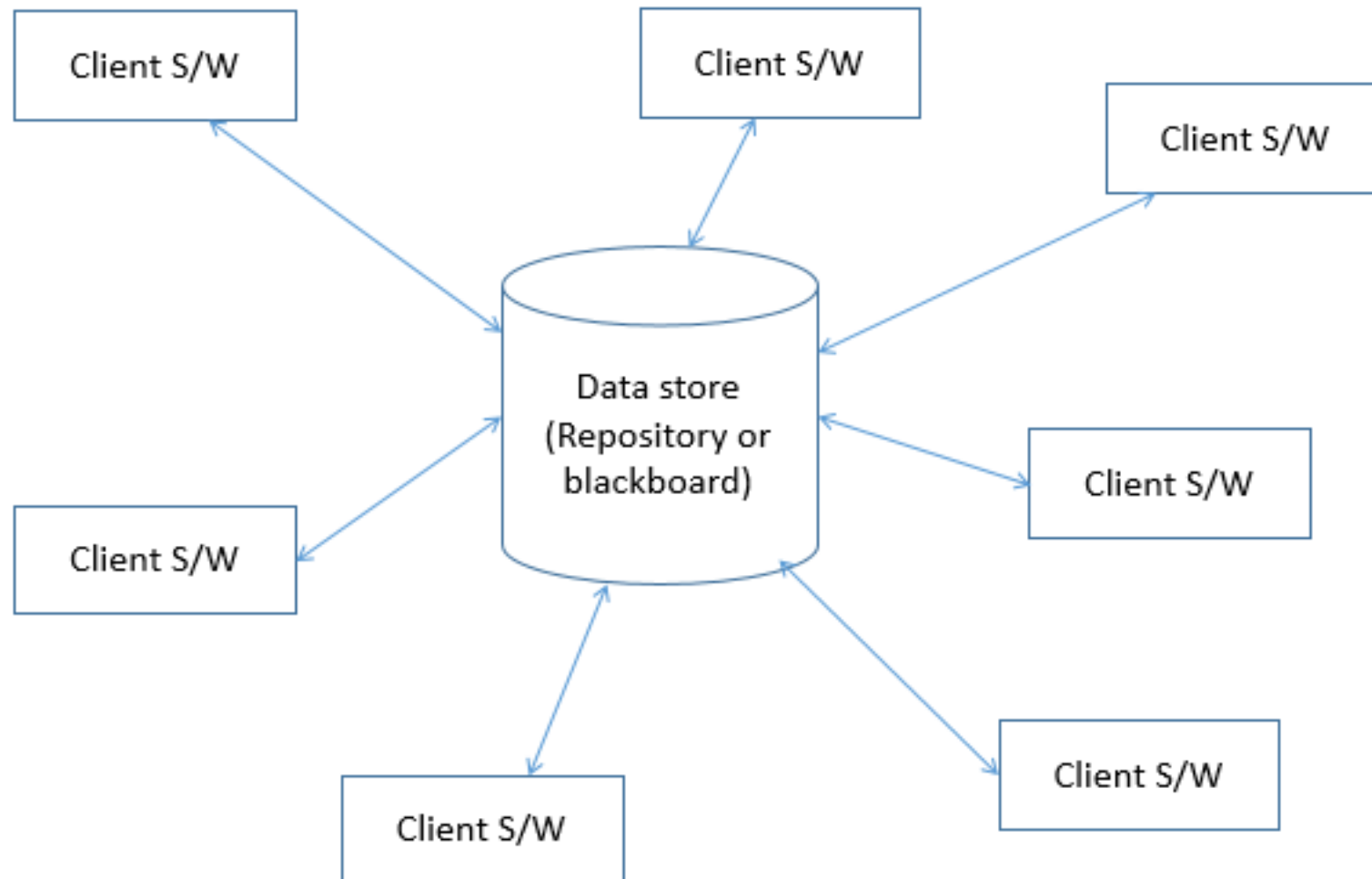
DATA CENTERED ARCHITECTURE

Data-Centered Architecture

- ❑ **Data-centered architecture** is characterized by a centralized data store that is shared by all surrounding software components.

- ❑ **Structure**
 - Decomposed into two major partitions:
 - ❑ **data store**
 - ❑ **data client** (independent software components or agents)
 - Interaction or **communication between the data clients is only through the data store**. The data is the only means of communication among them.

Example



Styles of Data-Centric Architecture

□ Repository

- The **data store** is **passive** , and its **clients** are **active**
- **Examples**
 - DBMS, Information System,

□ Blackboard

- The **data store** is **active** , and its **clients** are **passive**
 - The **clients of a blackboard** are called **knowledge sources, listeners, or subscribers**
- **Examples**
 - Knowledge based AI systems, Voice and image recognition systems

Repository Style

□ Elements

■ Components

- **One memory** (data store)
- Many **computational processes** (data client)

■ Connectors

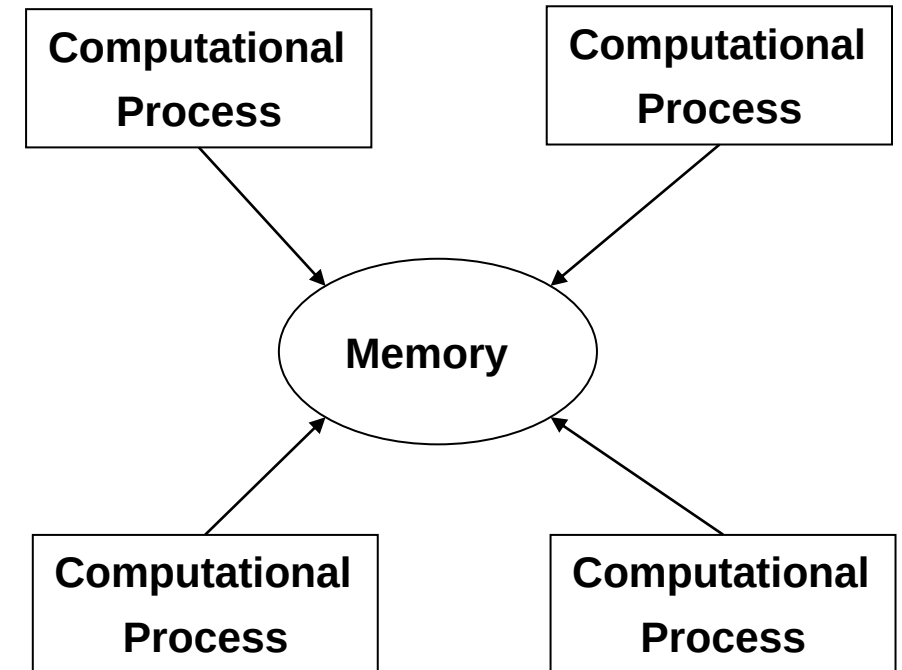
- Interaction between computational units and memory by directly **data access** or **procedure calls**

□ Relations

- See the figure on the right

□ Constraints

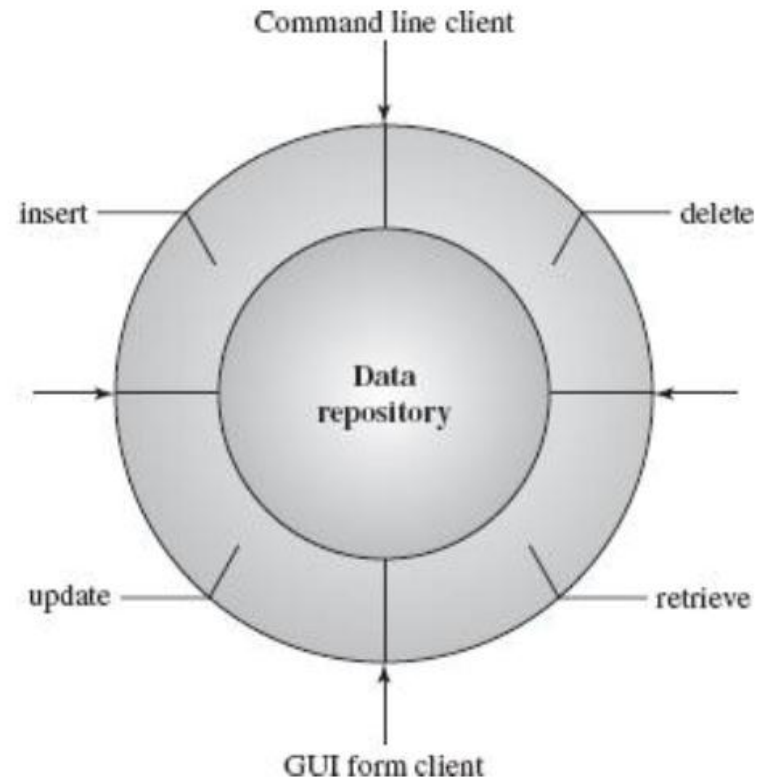
- Memory (data store) is **passively activated**
- No direct interactions among data clients



→ Data access

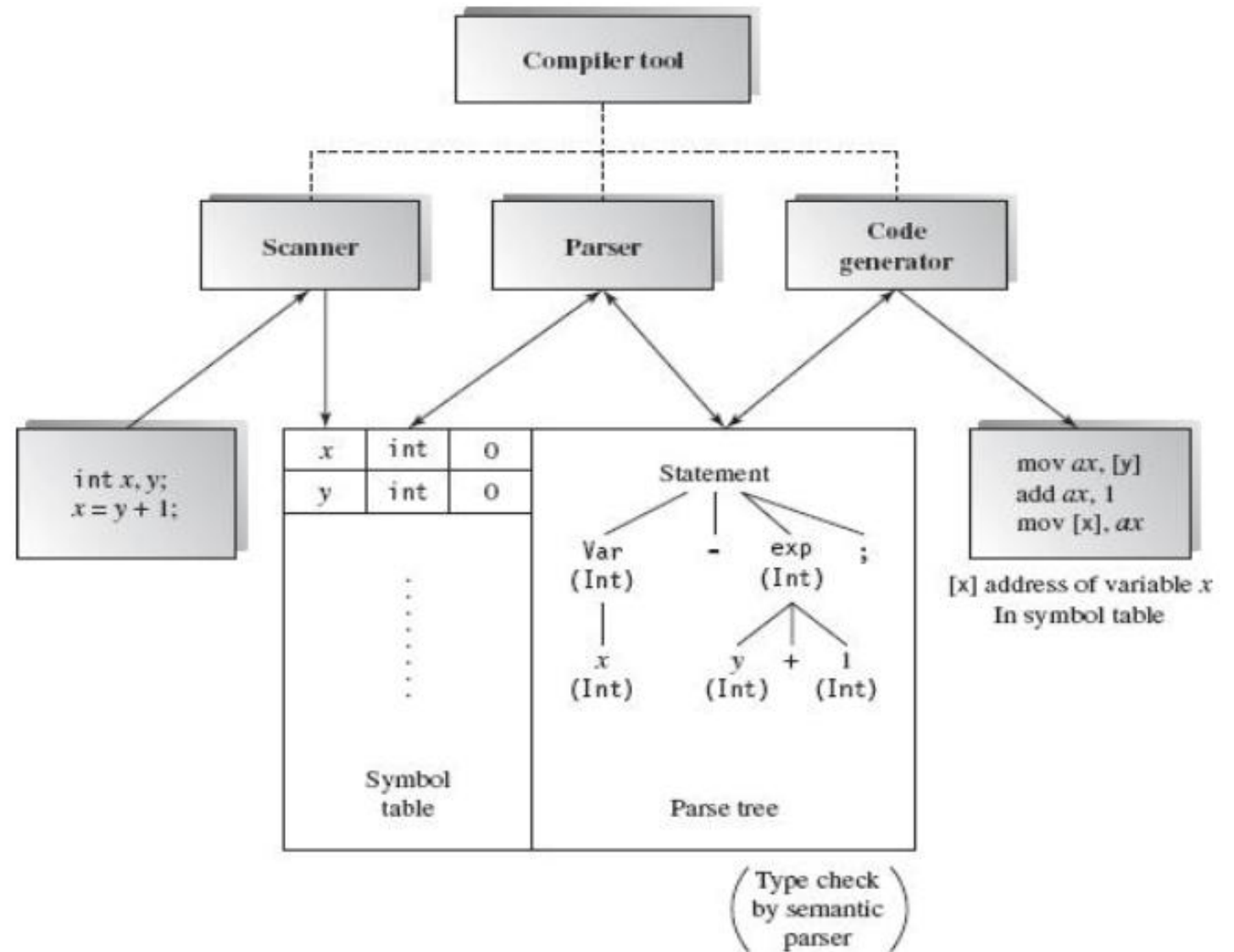
Example

□ Relational database management system(RDBMS)



Example

□ Compiler System



Consequences

□ Benefits

- Easy to add new data clients/data store (Scalability)
- Modifiable with respect to changing the functionality of any particular data client
- Data clients are reusable
- Reduces overhead of transient data between data clients.

□ Weaknesses

- Data store failures/attacks reduce the overall system availability/security
- High dependency between data structure of data store and its clients
- Cost of moving data on network, if data is distributed

Blackboard Style

□ Elements

■ Components

- **knowledge source** (data client)
- A **blackboard** component act as a central data repository
- A **controller** monitors the state of the blackboard and triggers the processing of **knowledge sources**.

■ Connectors

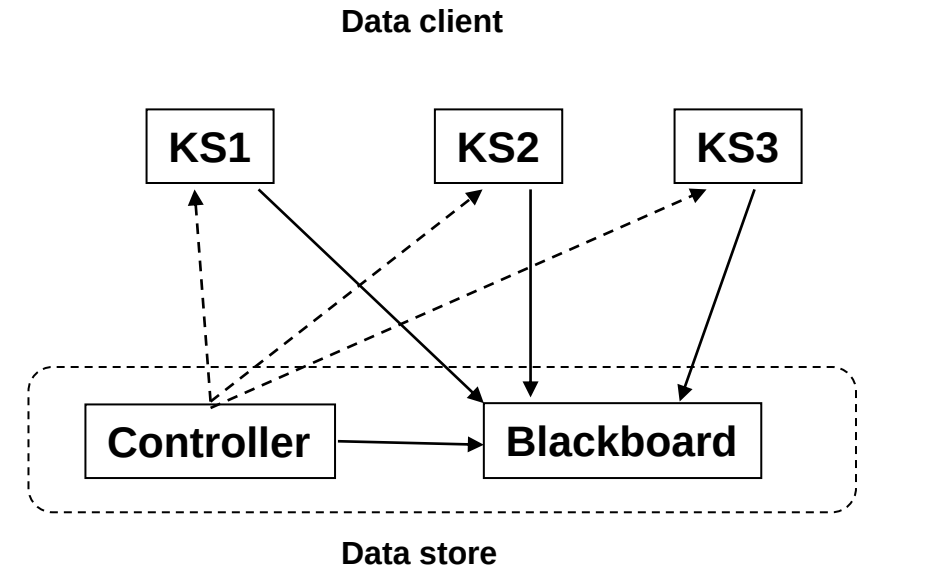
- **data access** or **procedure calls**

□ Relations

- See the figure on the right

□ Constraints

- the data store (i.e., **Controller** and **Blackboard**) is **active**, and its clients (i.e., **Knowledge source**) are **passive**



-----> Procedure calls (control)

————> Data access

Blackboard Pattern

□ **Components**

■ **Blackboard**

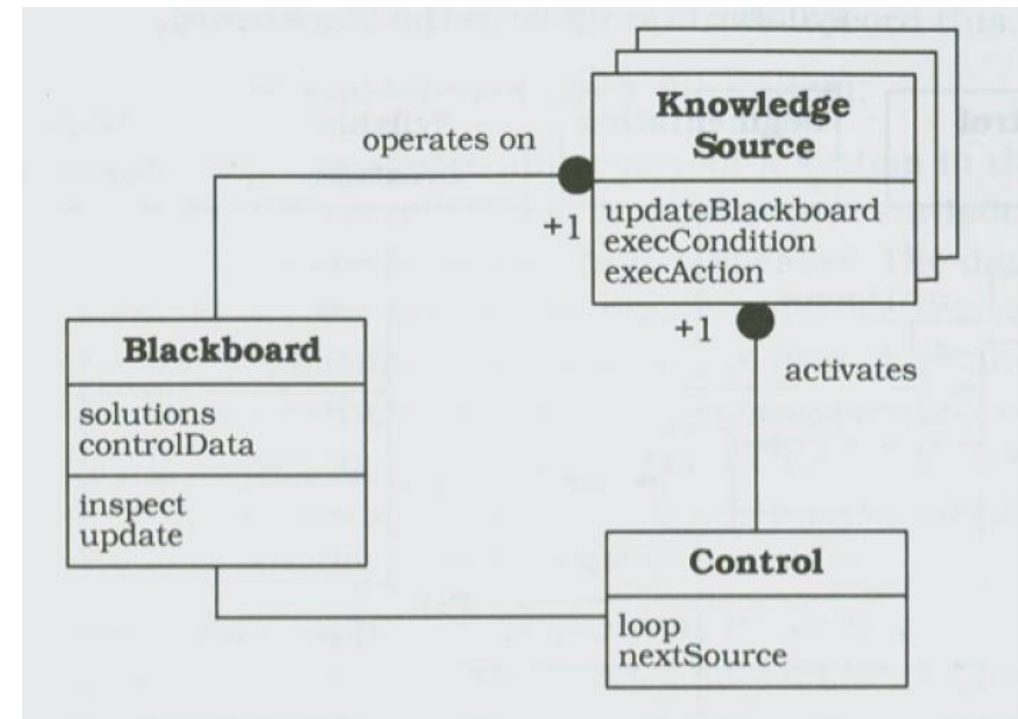
- **update** is used to make changes to the data on the blackboard
- **inspect** is used to check the current solutions on the blackboard

■ **Knowledge Source**

- Updates the blackboard
- Evaluates its own applicability
- Computes a result

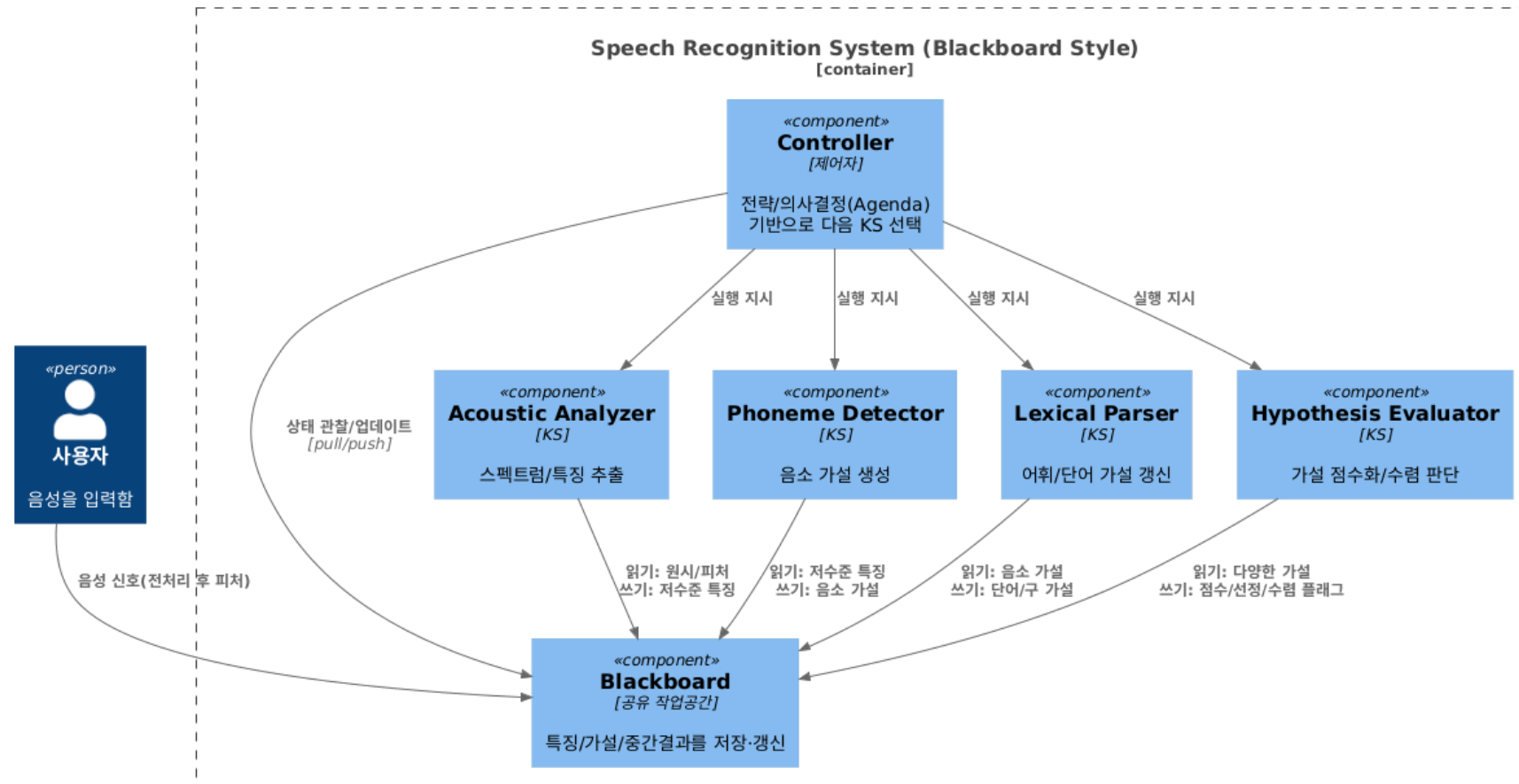
■ **Control**

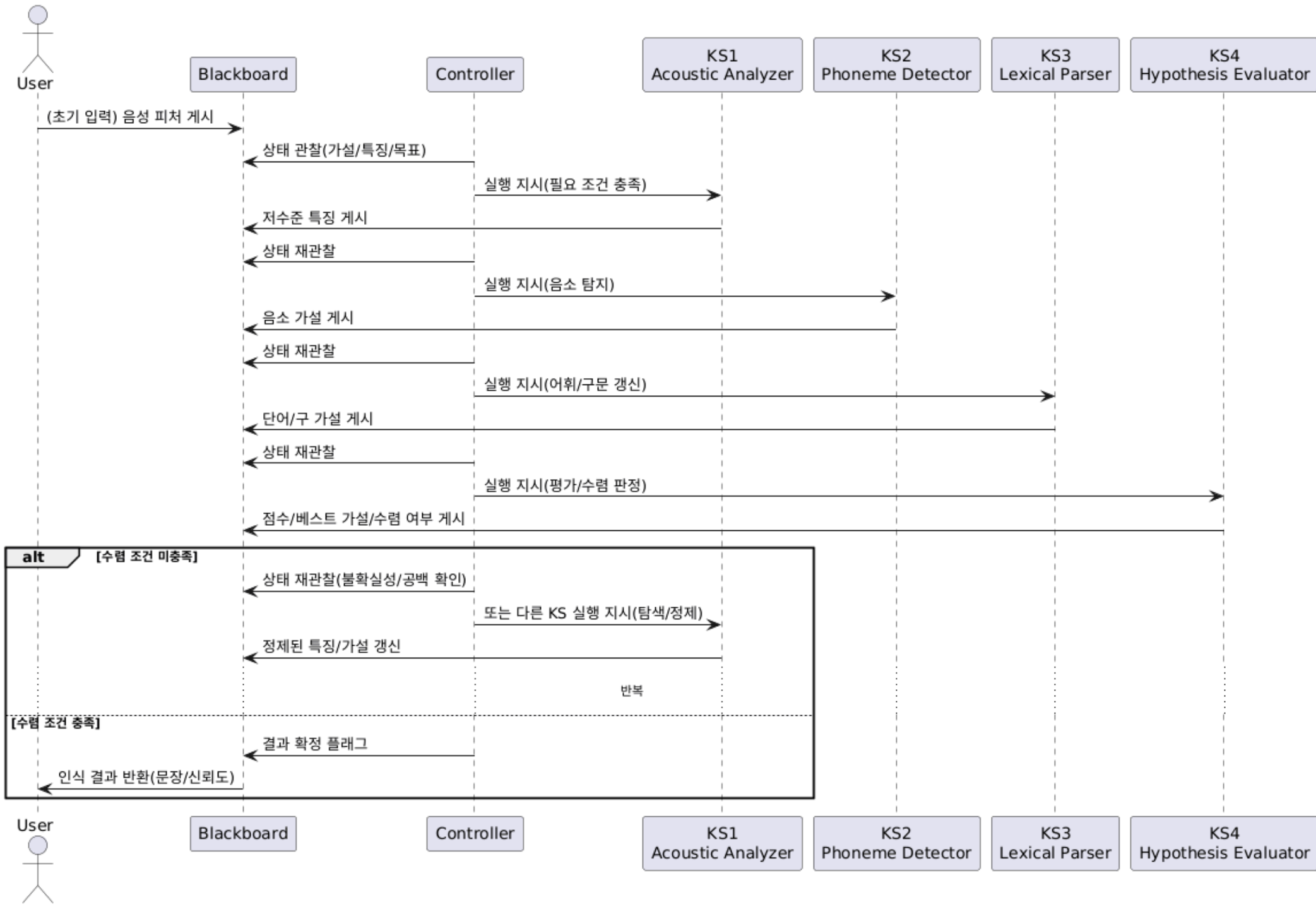
- **Runs a loop that monitors changes on the blackboard and decides what actions to take next**



Examples

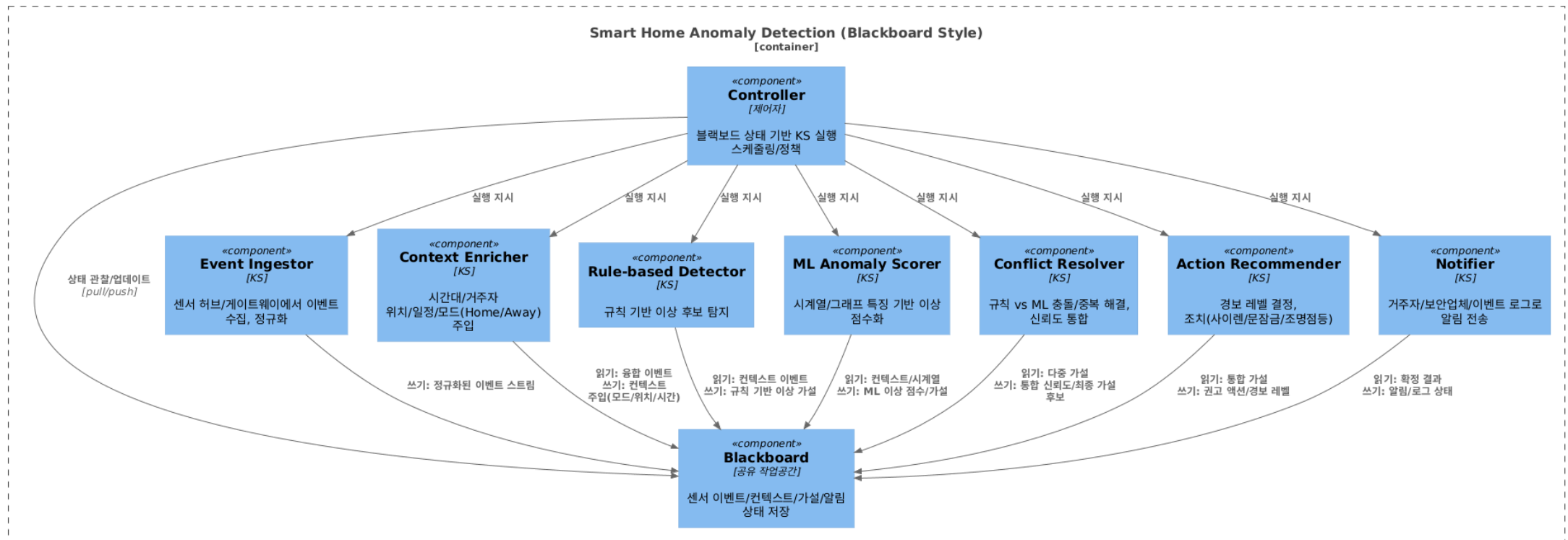
□ Speech Recognition System





Examples

Smart Home Anomaly Detection System



Consequences

□ Benefits

- Easy to add new data clients/data store (Scalability)
- Modifiable with respect to changing the functionality of any particular data client
- Data clients are reusable
- **Provides concurrency that allows all knowledge sources to work in parallel as they are independent of each other.**
- **Supports experimentation for various hypotheses.**

□ Weaknesses

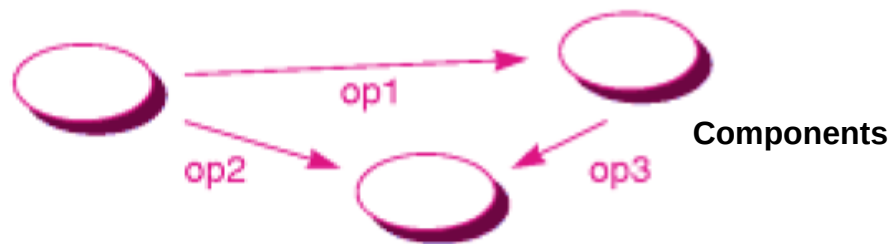
- Data store failures/attacks reduce the overall system availability/security
- High dependency between data structure of data store and its clients
- **It can be difficult to decide when to terminate the reasoning as only approximate solution is expected.**
- **Difficulty of testing.**

Architecture Style

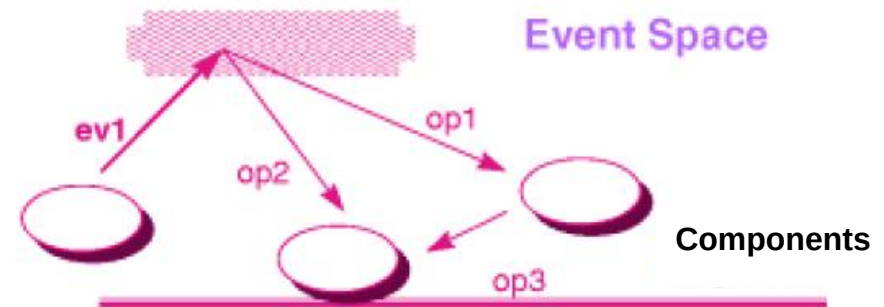
IMPLICIT INVOCATION ARCHITECTURE

Implicit Invocation

- **Implicit invocation** is that instead of invoking a procedure directly, event announcement *implicitly* causes the invocation of procedures in other modules.



Explicit Invocation



Implicit Invocation

Implicit Invocation Architecture

- ❑ Implicit invocation architecture is characterized by **loosely coupled communication between software components**.
- ❑ Structure
 - break the system into two parties
 - ❑ one party, called **a publisher, producer, sender, or source**
 - ❑ the other, called **a subscriber, consumer, receiver, or target**.
 - The communication between these two parties can be **synchronous** or **asynchronous** in one of the following mode
 - ❑ one-to-one
 - ❑ one-to-many
 - ❑ many-to-one

Styles of Implicit Invocation Architecture

- Non-buffered **Event-Based Style**

- Observer Pattern (one-to-many, many-to-one)

- Buffered **Message-Based Style**

- Point-to-Point Message Architecture (one-to-one)
 - Publish-Subscribe Messaging Architecture(one-to-many)

Non-buffered Event-Based Style

□ Elements

■ Event Producer

- detect, generate, or emit events when a specific condition or state change occurs

■ Event Channel

- A lightweight communication medium that directly connects producers and consumers
- No buffer

■ Event Consumer

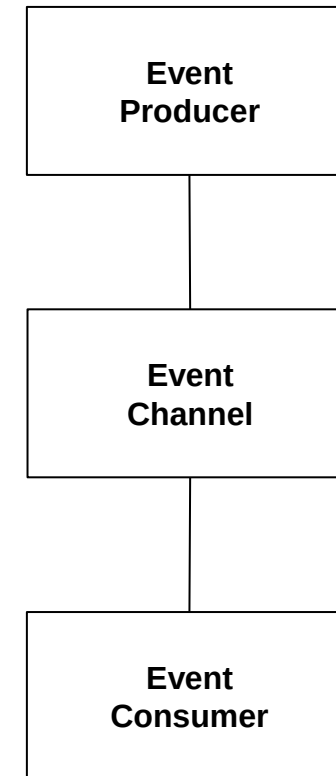
- receive and process events immediately after they are emitted

□ Relations

- See the figure on the right

□ Constraints

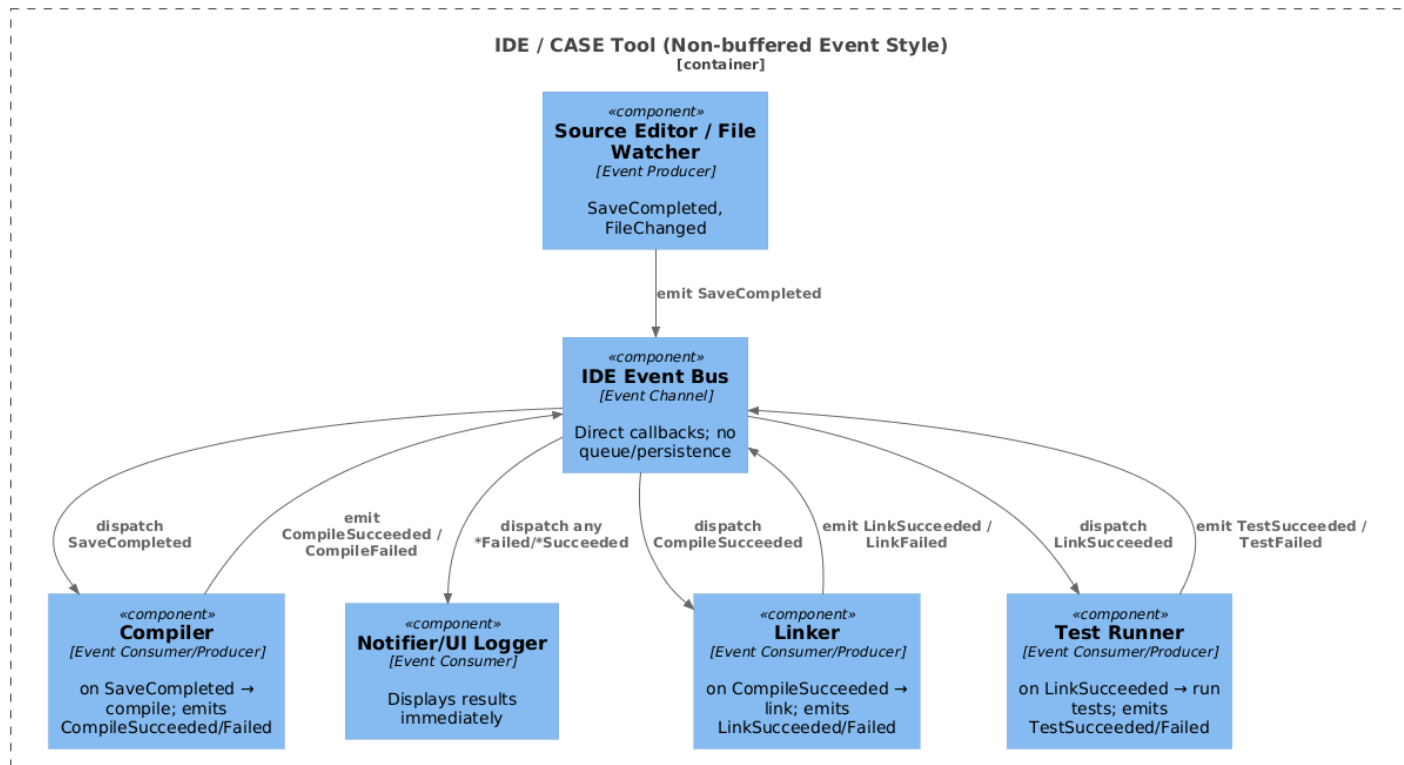
- Events must be delivered to consumers as soon as they are generated
- Producers and consumers remain independent of each other



Application Examples

□ Many IDE for CASE tools

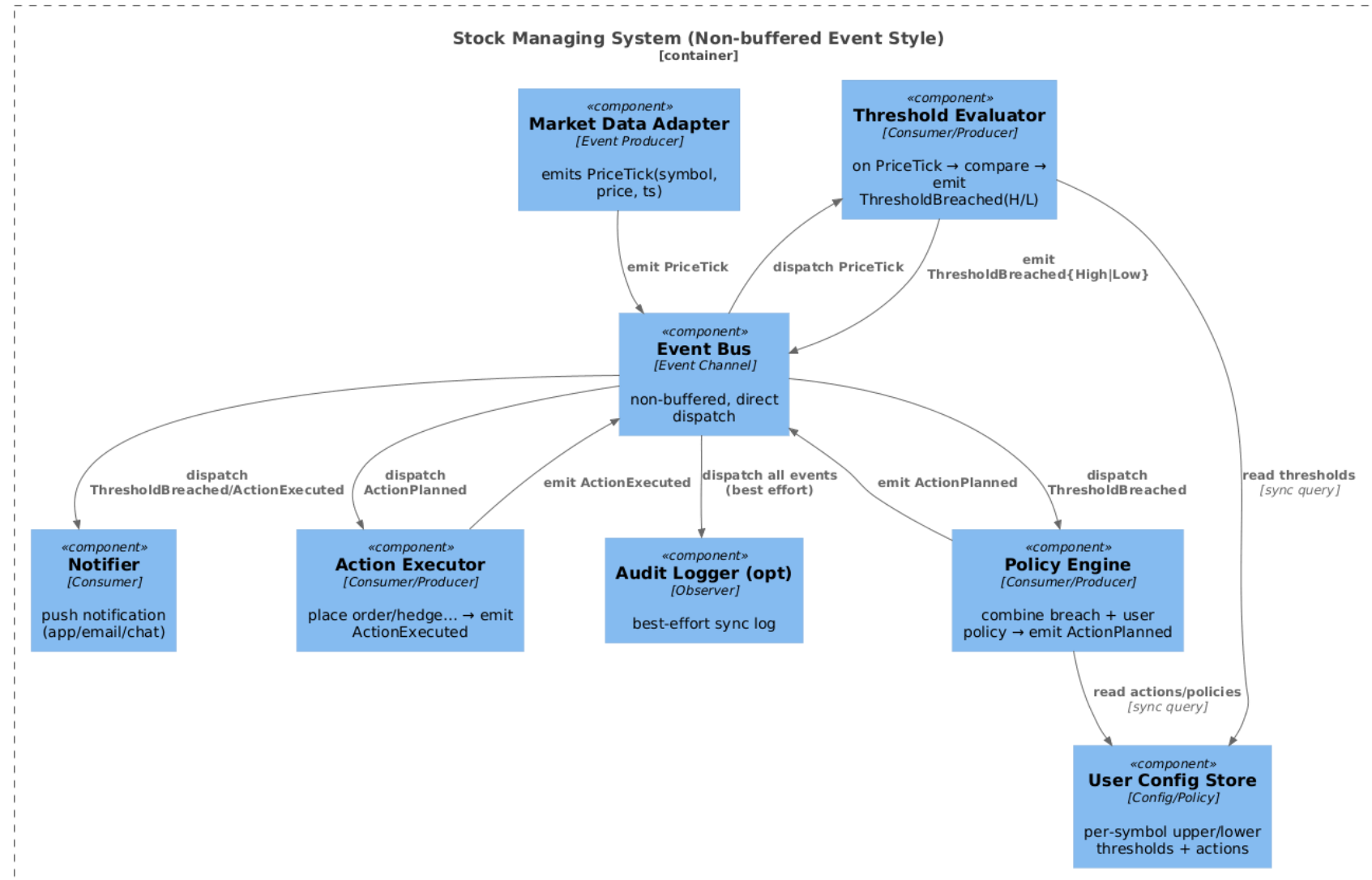
- The completion of the editing of a source file may trigger its compilation, or a new recompilation; this in turn triggers the relink processing, and so on.



Application Examples

□ Stock managing system

- a user can set upper and lower threshold levels of a specific stock.
- When the stock value goes either too high or too low, the user will be notified automatically by the system and action will be taken automatically which can be configured by the user in advance



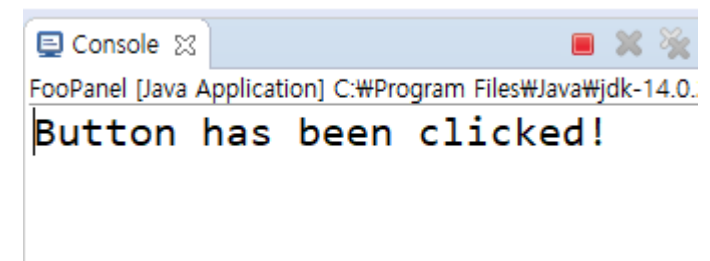
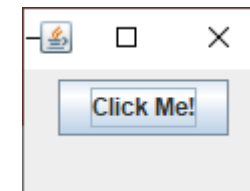
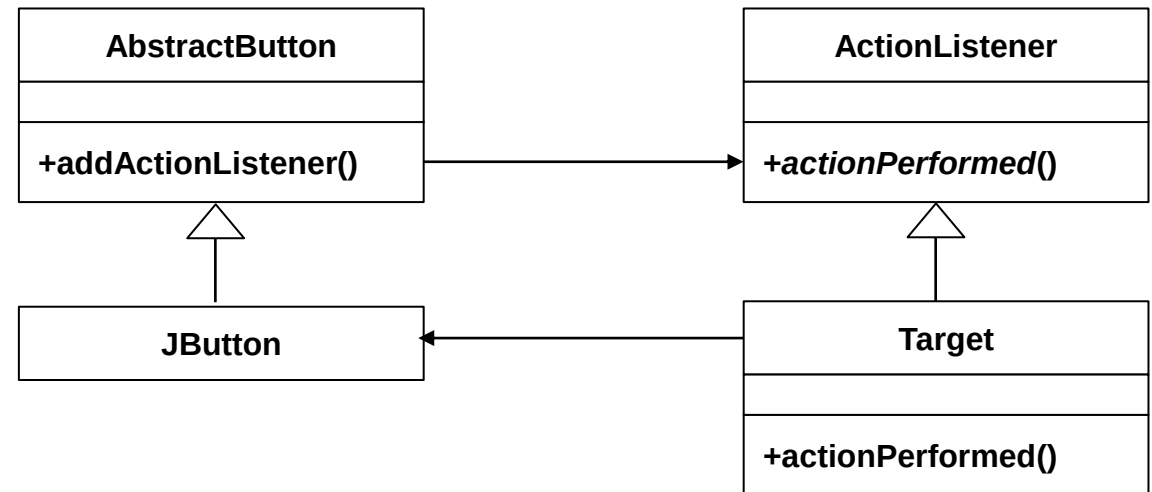
Examples – Java Swing

```

public class FooPanel extends JPanel{
    public FooPanel() {
        super();
        JButton btn = new JButton("Click Me!");
        btn.addActionListener(new Target());
        this.add(btn);
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setSize(100,100);
        frame.add(new FooPanel());
        frame.setVisible(true);
    }
}

class Target implements ActionListener {
    public void actionPerformed(ActionEvent ae) {
        System.out.println("Button has been clicked!");
    }
}

```



Consequences

□ **Benefits**

■ **Framework availability**

- Many vendor APIs such as Java AWT and Swing components are available.

■ **Reusability of components**

- It is easy to plug in existing event listeners without affecting the rest of the system.

■ **System maintenance and evolution**

- Both event sources and event listeners are easy to update.

■ **Flexible connectivity**

- Dynamic registration and deregistration can be done dynamically at runtime.

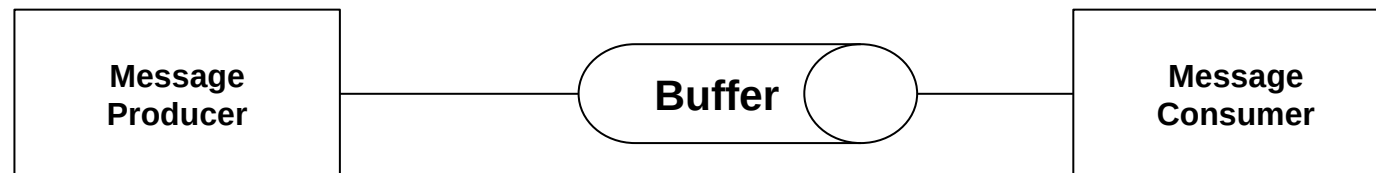
Consequences

□ Weaknesses

- **It is difficult to test and debug the system** since it is hard to predict and verify responses and the order of responses from the listeners.
- **Overhead of indirect invocations** may be an issue.
- There is **tighter coupling between event sources and their listeners than in message-based implicit invocation.**

Buffered Message-Based Style

- ❑ The buffered message-based software architecture breaks the software system into two partitions:
 - Message producers
 - ❑ generates a message and sends it to a buffer
 - ❑ **do not know the consumers of the message**
 - Message consumers
 - ❑ receive the message from the buffer asynchronously
- ❑ They are connected asynchronously by **a buffer** (either **a message queue** or **a pub-sub connector**).

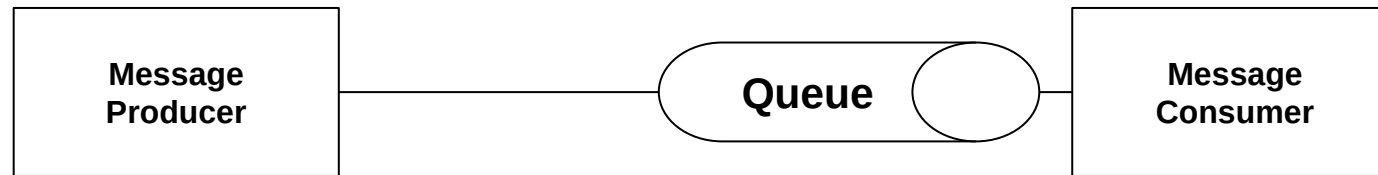


Point-to-Point Messaging Architecture

□ Elements

- **Message Producer** (component)
 - sends messages to a destination (a specific queue)
- **Message Consumer** (component)
 - consumers extract messages from these queues
- **Queue** (connector)
 - retains all messages it receives either until the messages are consumed or until the messages expire

□ Relations



□ Constraints

- Message-based communication is typically **asynchronous**, meaning publishers and subscribers do not need to be active simultaneously

Publish-Subscribe Messaging Architecture

□ Elements

■ Publisher (component)

- sends messages to the pub-sub connector

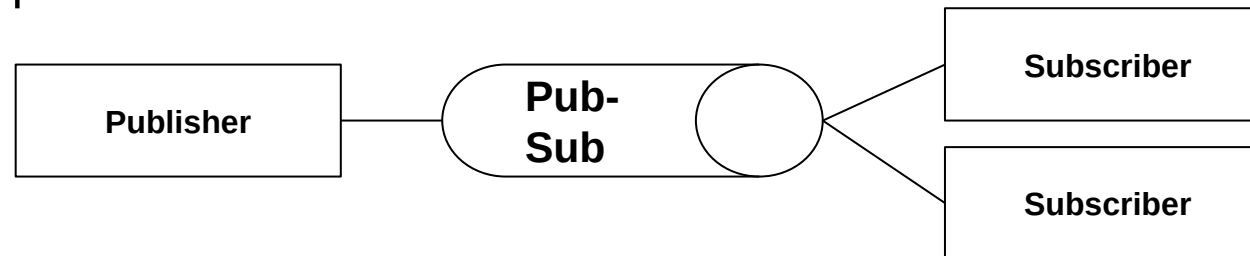
■ Subscriber (component)

- subscribes to specific types of messages it is interested in
- receives the subscribed messages from the connector

■ Pub-Sub (connector)

- ensures that messages published by publishers are delivered to the relevant subscribers based on their subscriptions

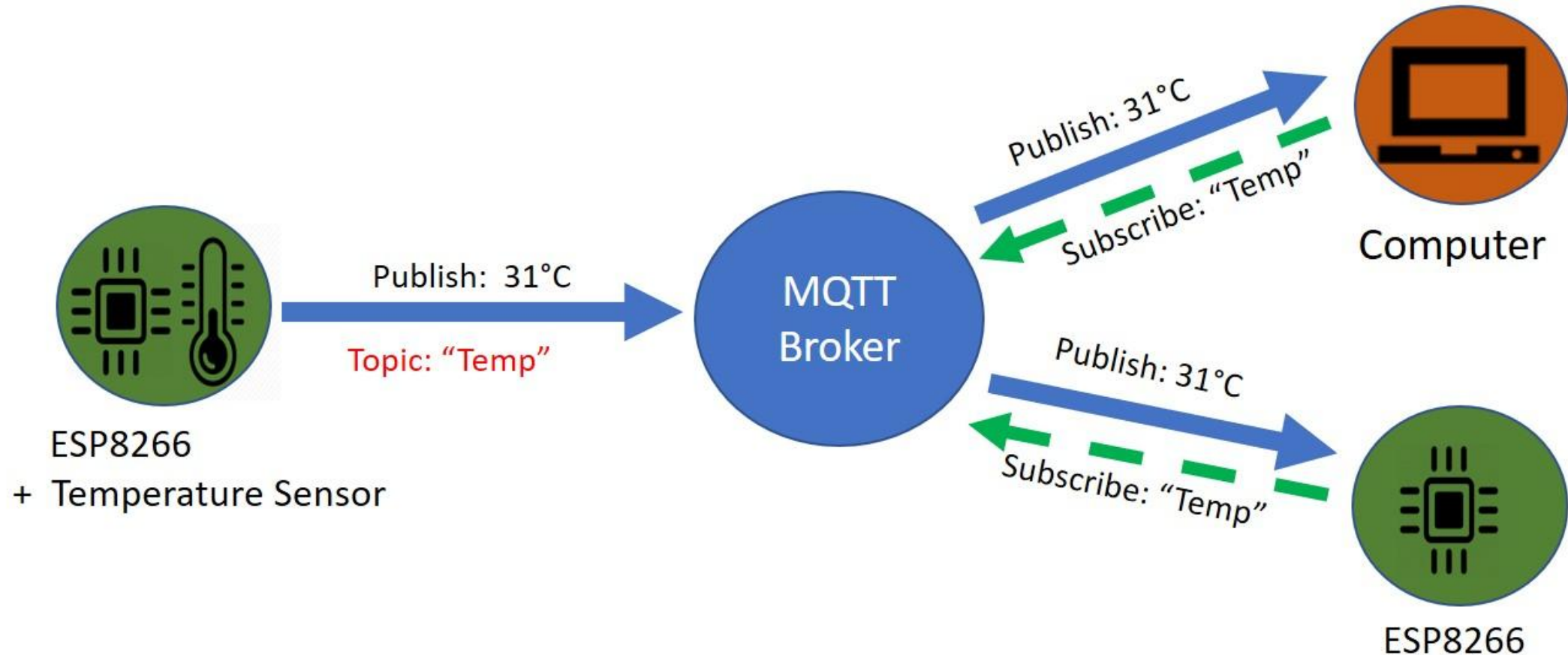
□ Relations



□ Constraint

- Asynchronous message communication
- The system delivers the messages to all of its multiple subscribers

Examples - MQTT Broker



Consequences

□ Benefits

■ Anonymity

- The message consumer does not know
 - who produced the message (user independence)
 - where the producer lives on the network (location independence)
 - When the message was produced (time independence)

■ Concurrency

- Concurrency both among consumers and between producer and consumers

■ Reliability of message delivery

- Control level setting of message acknowledgement
- Message priority level setting
- Message expiration setting

Consequences

□ Weaknesses

■ Capacity limit of message queue

- This is not an inherent limitation but an implementation issue that can be minimized if the queue is implemented as a dynamic data structure (e.g., linked lists).
- an absolute limit based on available memory

■ Overhead, indirection complexity, and difficulty in debugging

Architecture Style

INTERACTION-ORIENTED ARCHITECTURE

Interaction Oriented Architecture

□ Objective

- to **separate the interaction of user** from data abstraction and business data processing.

□ Structure

- The interaction-oriented software architecture decomposes the system into three major partitions
 - **Data module** – Data module provides the data abstraction and all business logic.
 - **Control module** - Control module manages the flow of control and system configuration actions.
 - **View presentation module** - View presentation module is responsible for **visual or audio presentation of data output** and it also provides **an interface for user input**.

Styles of Interaction-Oriented Architecture

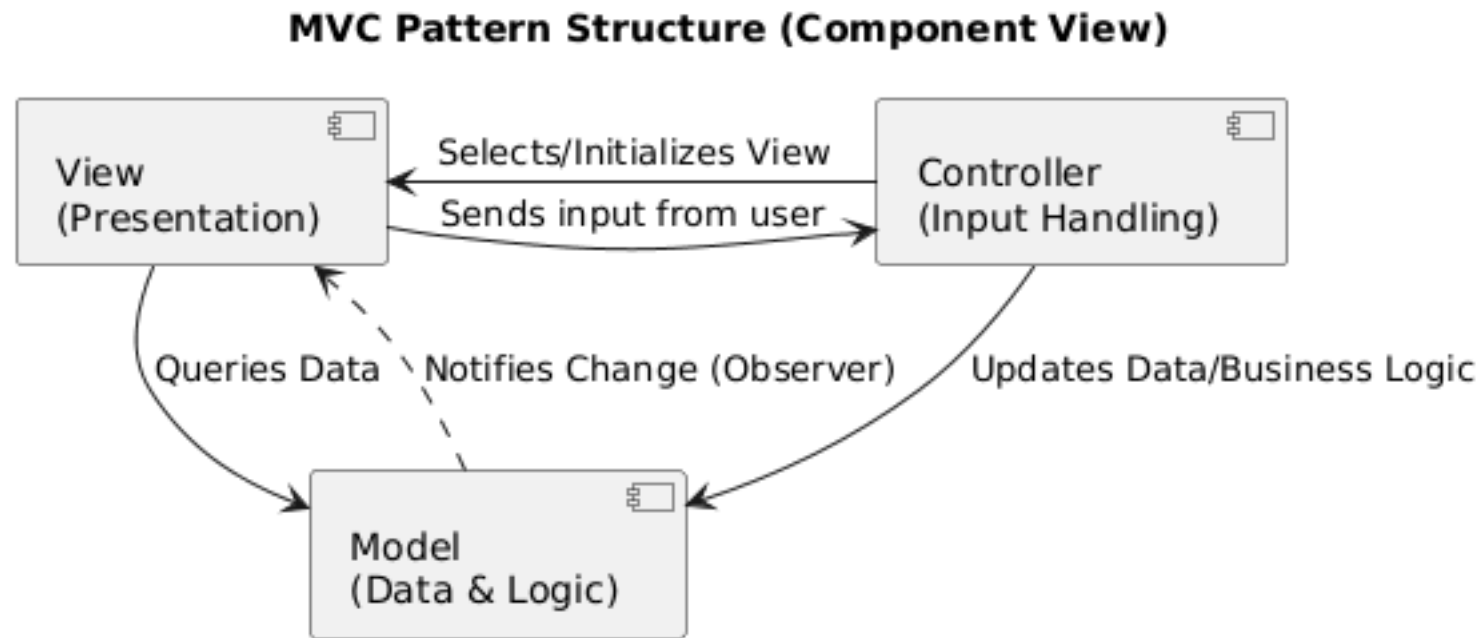
- Two major style
 - **Model-View-Controller (MVC)**
 - **Presentation-Abstraction-Control (PAC)**
- Both **MVC** and **PAC** propose three components decomposition and are used for interactive applications such as web applications.
- They are different in their flow of control and organization.
 - **PAC** is an **agent-based** hierarchical architecture but **MVC** does not have a clear hierarchical structure.

Model-View-Controller Style

- ❑ **MVC** is a software architecture style commonly used in designing interactive applications.
 - separating the internal representations of information from the information presented to or accepted from the user.
 - A change propagation mechanism ensure consistency between the user interface and the model
- ❑ **Elements**
 - **Model**
 - ❑ encapsulates of the underlying data and business logic
 - ❑ does not depend on the **View** or the **Controller**
 - **View**
 - ❑ displays information to the user
 - ❑ Does not contain application logic
 - **Controller**
 - ❑ receives user input from the **View** and translates it into actions on the **Model**
 - ❑ contains application logic and manages the flow of data between the **Model** and the **View**

Model-View-Controller Style

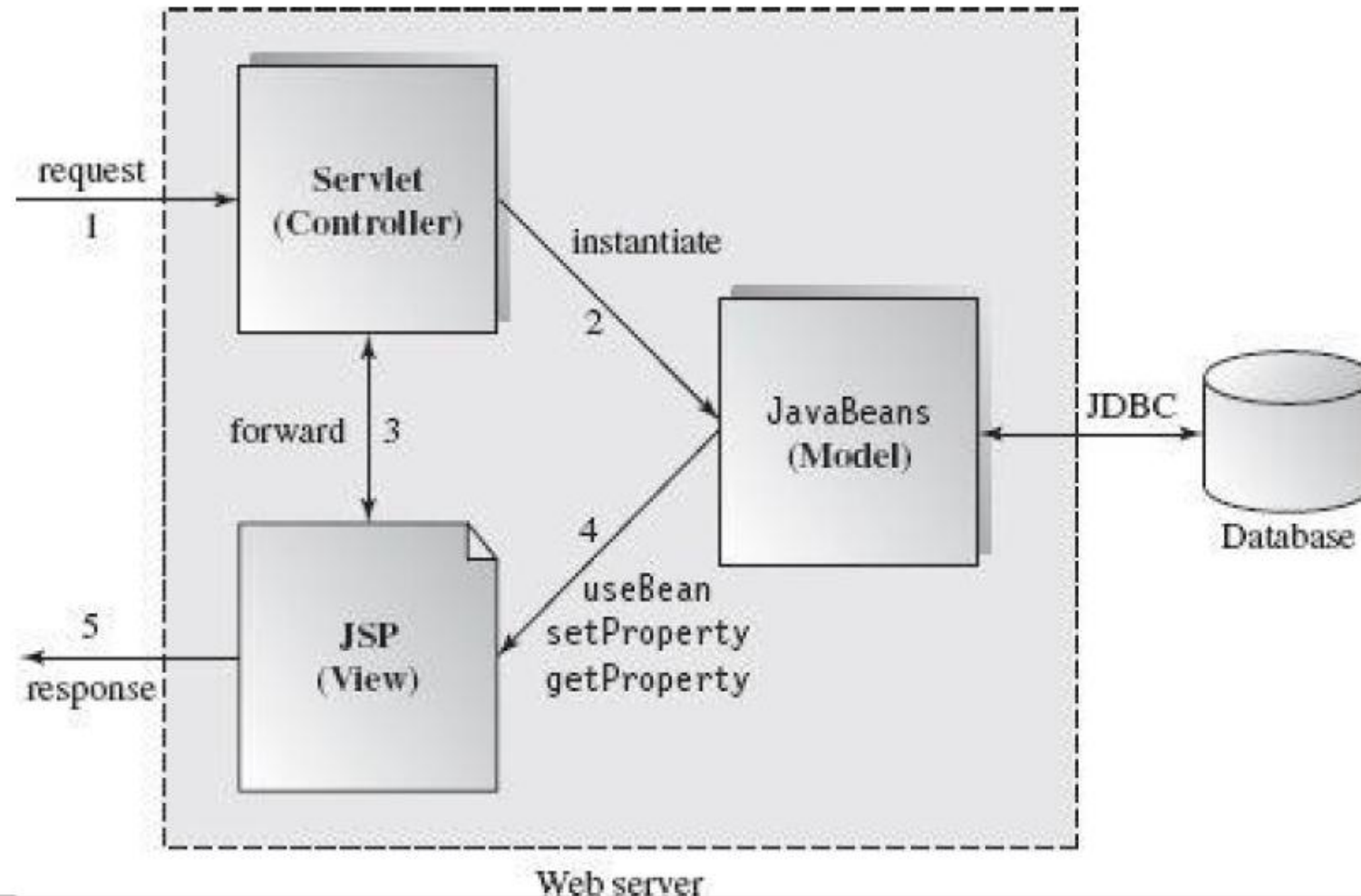
▣ Relations (Structure)



Applicable Domain

- ❑ **MVC** applications are effective for interactive applications where multiple views are needed for a single data model and easy to plug-in a new interface view.
- ❑ **MVC** applications are suitable for applications where there are clear divisions between the modules so that different professionals can be assigned to work on different aspects of such applications concurrently.
 - Example
 - ❑ graphics expertise professionals, programming professionals, and data base development professionals are working in a designed project team.

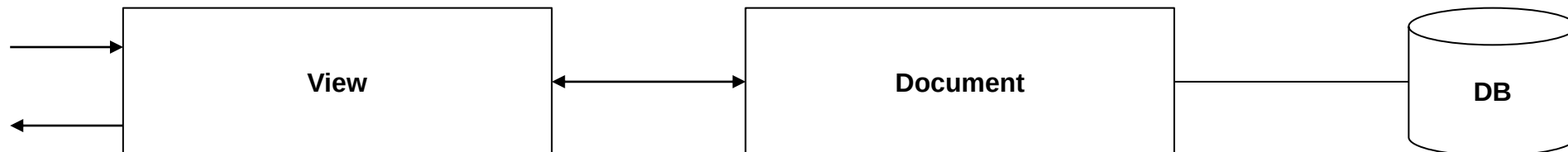
Example – Spring MVC Framework



Variants

□ Document-View

- the system is divided into two sub-systems:
 - **The View (Controller-View)** : the view acts as input /output interface and processing is done.
 - **The Document (Model)** : the document provides all the data and domain services.
- The connection between the **View** and the **Document** can be designed through the **Observer pattern**
 - The **View** subscribes to the **Document** and the **Document** notifies the **View** of any changes



Variants

□ Model-View-Presenter

■ Model :

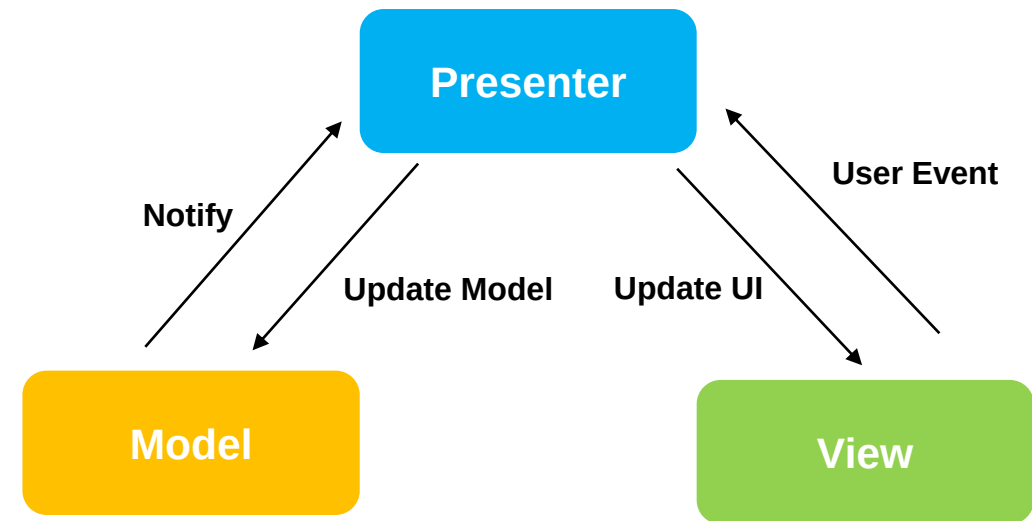
- Contains the application's data and business logic

■ View

- Handles the display of data to the user.
- Forwards user input event (like button clicks) to the Presenter

■ Presenter

- Receives user actions from the View and translates them into commands for the Model.
- Receives data from the Model and formats it for display in the View



In MVC, the Controller manages user input and may directly update the View, while in MVP, the Presenter completely controls the View through an interface, making the View passive and easier to test.

Variants

□ Model-View-ViewModel

■ Model :

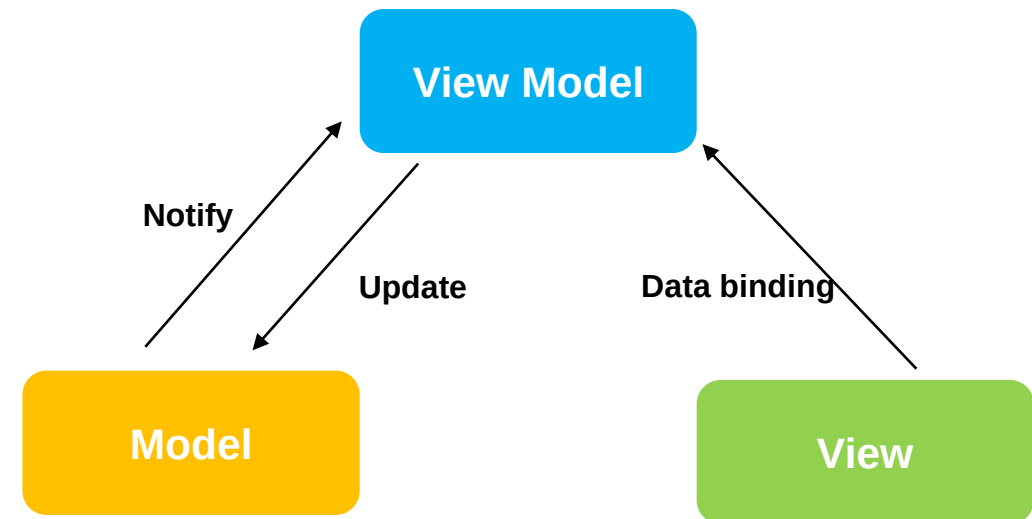
- Contains the application's data and business logic

■ View

- Handles the user interface and displays data, but contains no business logic

■ ViewModel

- It retrieves data from the Model and reformats it so it can be easily used by the View through **data bindings**



In MVC, the Controller handles user input and directly updates the View, while in MVVM, the ViewModel exposes observable data and commands that the View automatically binds to, without any direct reference.

Consequences

□ Benefits

■ Synchronized views

- The change-propagation mechanism of the model ensures that **all attached observers are notified of changes to the application's data at the correct time.**

■ 'Pluggable' views and controllers

- The view and controller object of a model can even be substituted at run-time

■ Exchangeability of 'look and feel'

- **Porting an application to a new platform** does not affect the functional core but **need suitable implementations of view and controller components** for each platform

Consequences

❑ Weaknesses

■ Increased complexity

- ❑ Following the MVC structure strictly is not always the best way to build an application with simple UI.

■ Potential for excessive number of updates

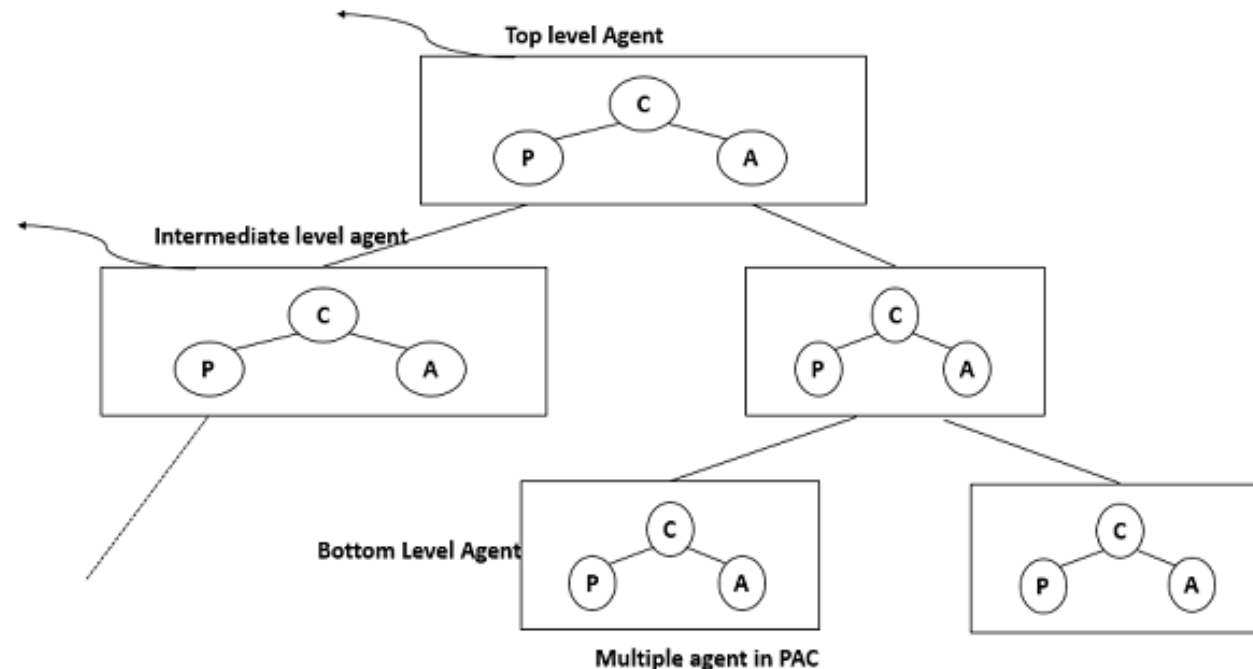
■ Inefficiency of data access in view

- ❑ Depending on the interface of the model (i.e. push vs. pull), a view may need to make multiple calls to obtain all its display data

■ The model, view, and controller abstractions may not be good fits for some user interface toolkits

Presentation-Abstraction-Control Style

- ❑ **PAC** structures the interactive application as **a tree-like hierarchy of PAC agents**, each of which is responsible **for specific aspects of the application's functionality** and consists of three components: **Presentation, Abstraction, and Control**.



The top-level agent orchestrates the middle-layer agents

Intermediate-level agents combine the functionalities of related lower-level agents

Bottom-level agents are responsible for specific functionalities

Presentation-Abstraction-Control Style

□ Elements

■ Presentation

- takes care of the user interface

■ Abstraction

- maintains application data and the logic that modifies it

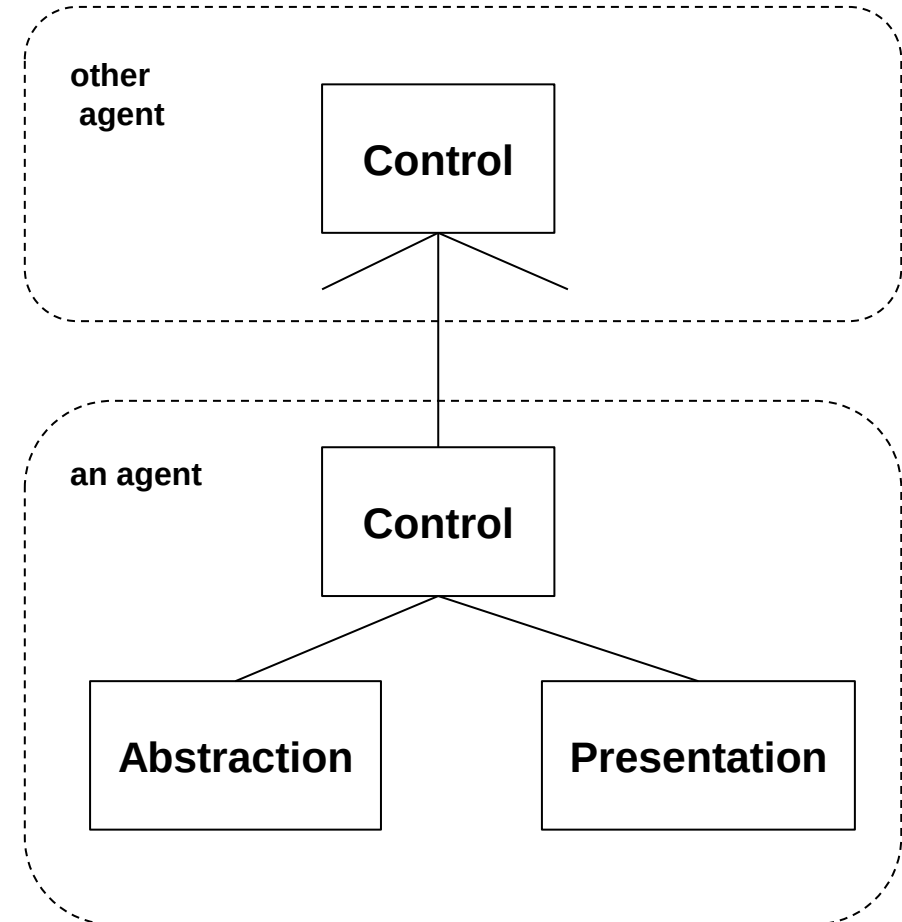
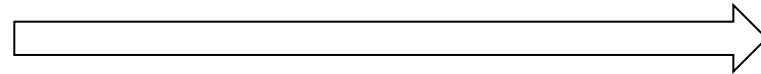
■ Control

- intermediates between the **Presentation** and the **Abstraction** and handles all communication with the **Controls of other Agents**

□ Relations

□ Constraints

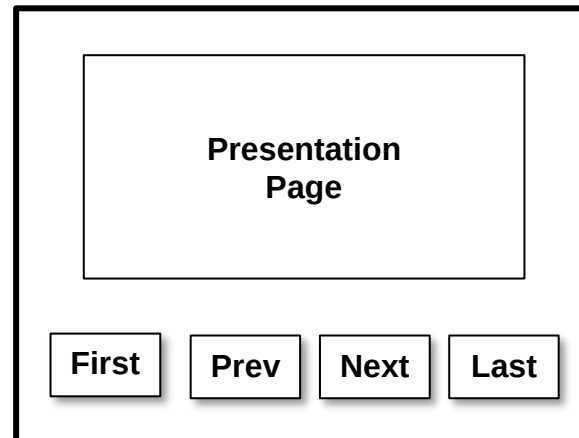
- no direct connections between **Abstraction** component and **Presentation** component



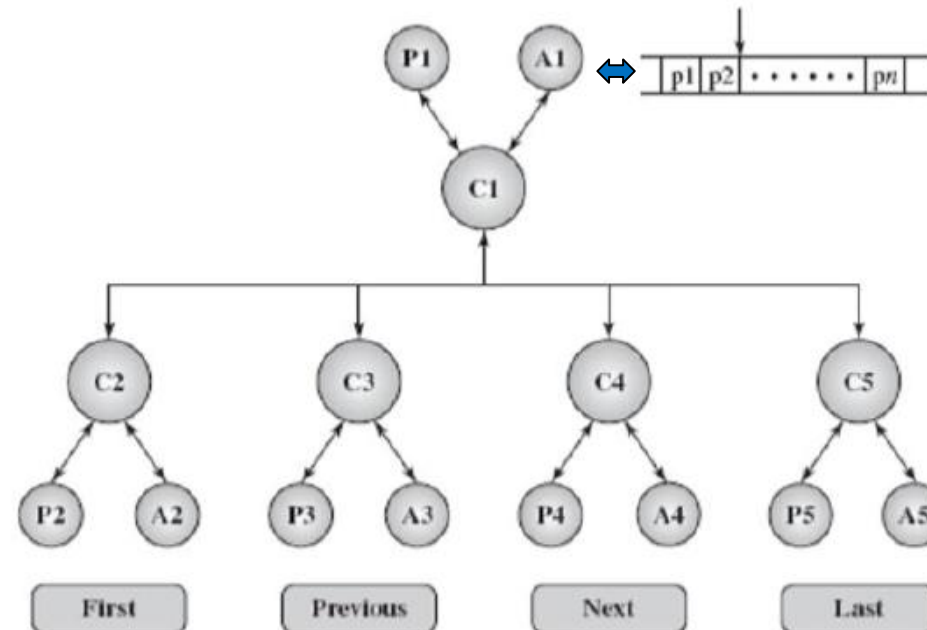
Example

□ a desktop presentation application

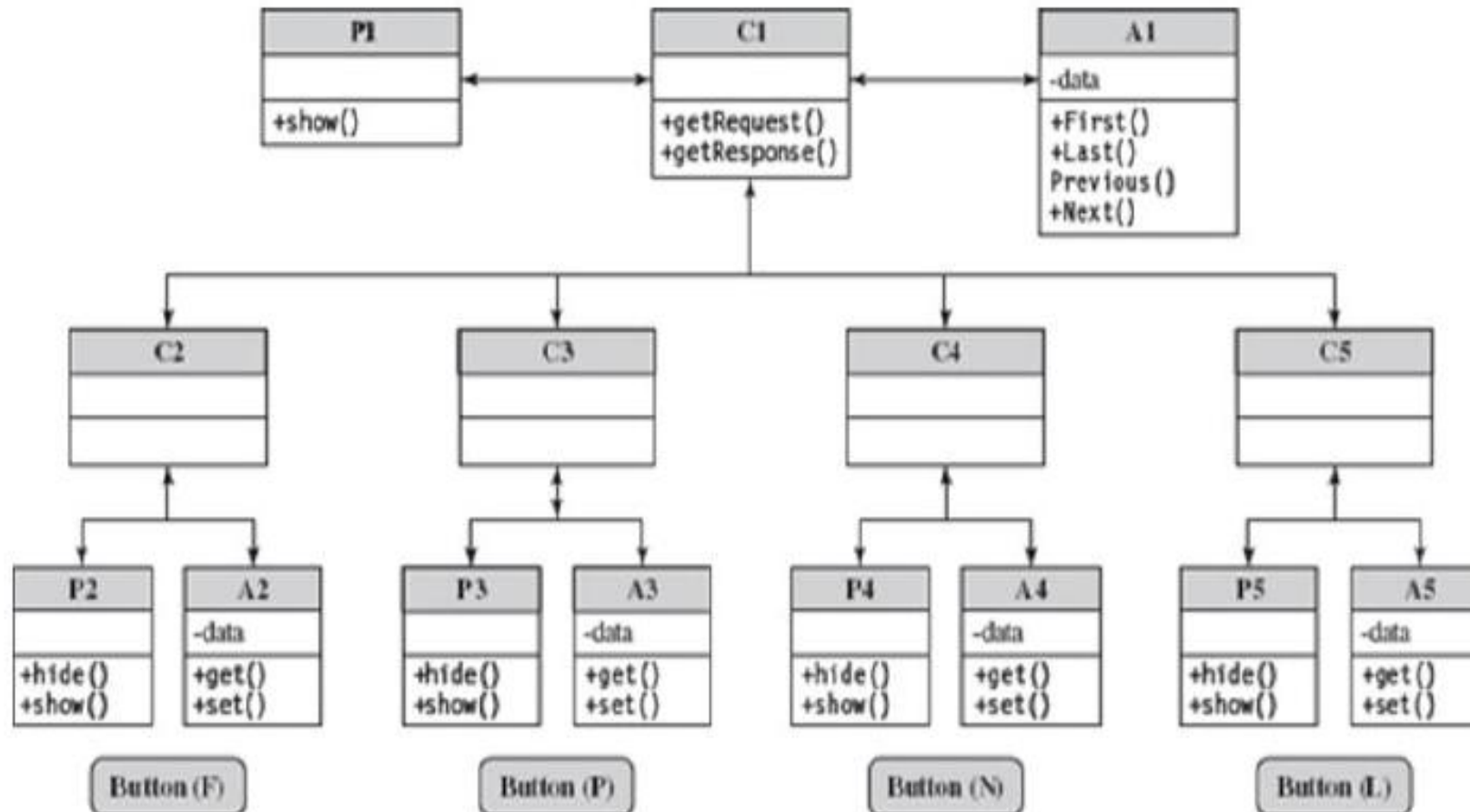
- To browse and display presentation pages in a graphic document at a time, the application provides four buttons (**First**, **Last**, **Previous**, **Next**).
- The agent **A1** has a connection to the data repository, which is the document file



desktop presentation application



Example (cont'd)



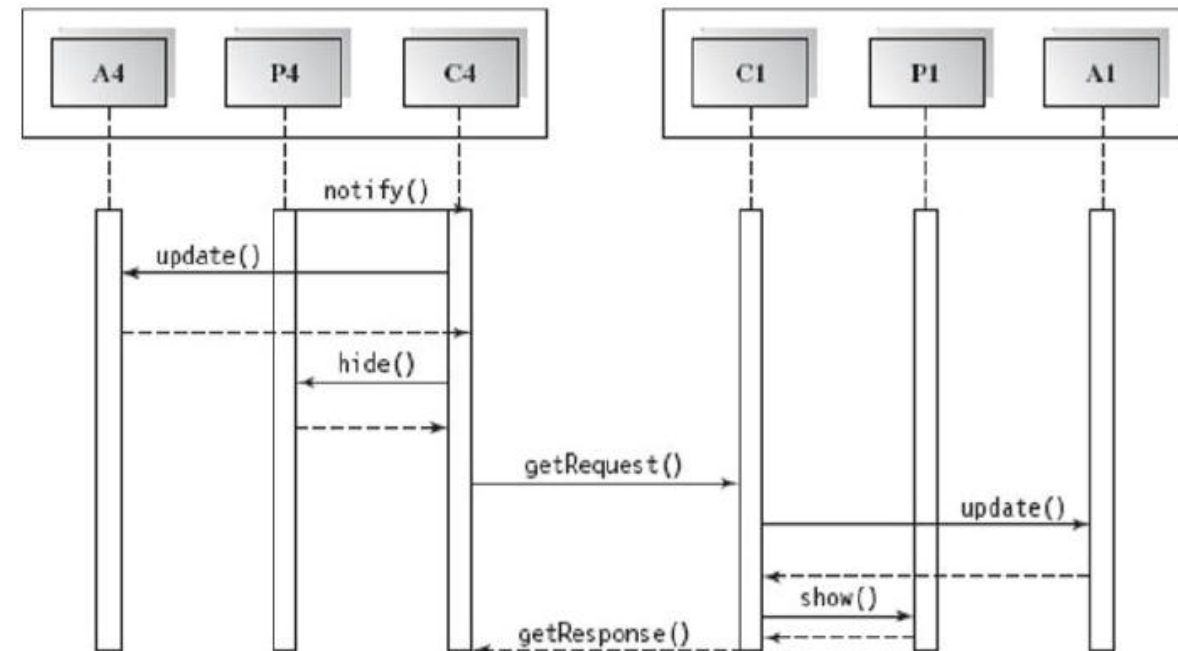
Example (cont'd)

□ Assumption

- The current page is the second to last page in the document.

□ When the **next** button is pressed to display the last page in the document

1. **P4** informs **C4** that the “next” button was pressed;
2. **C4** sends **update** to **A4**;
3. **C4** informs **P4** to update its presentation or shape; (hiding the “**next**” button)
4. **C4** contacts **C1** (a top level agent).
5. **C1** sends **update** to **A1** to move the pointer to next (last page)
6. **C1** instructs **P1** to display the last page.



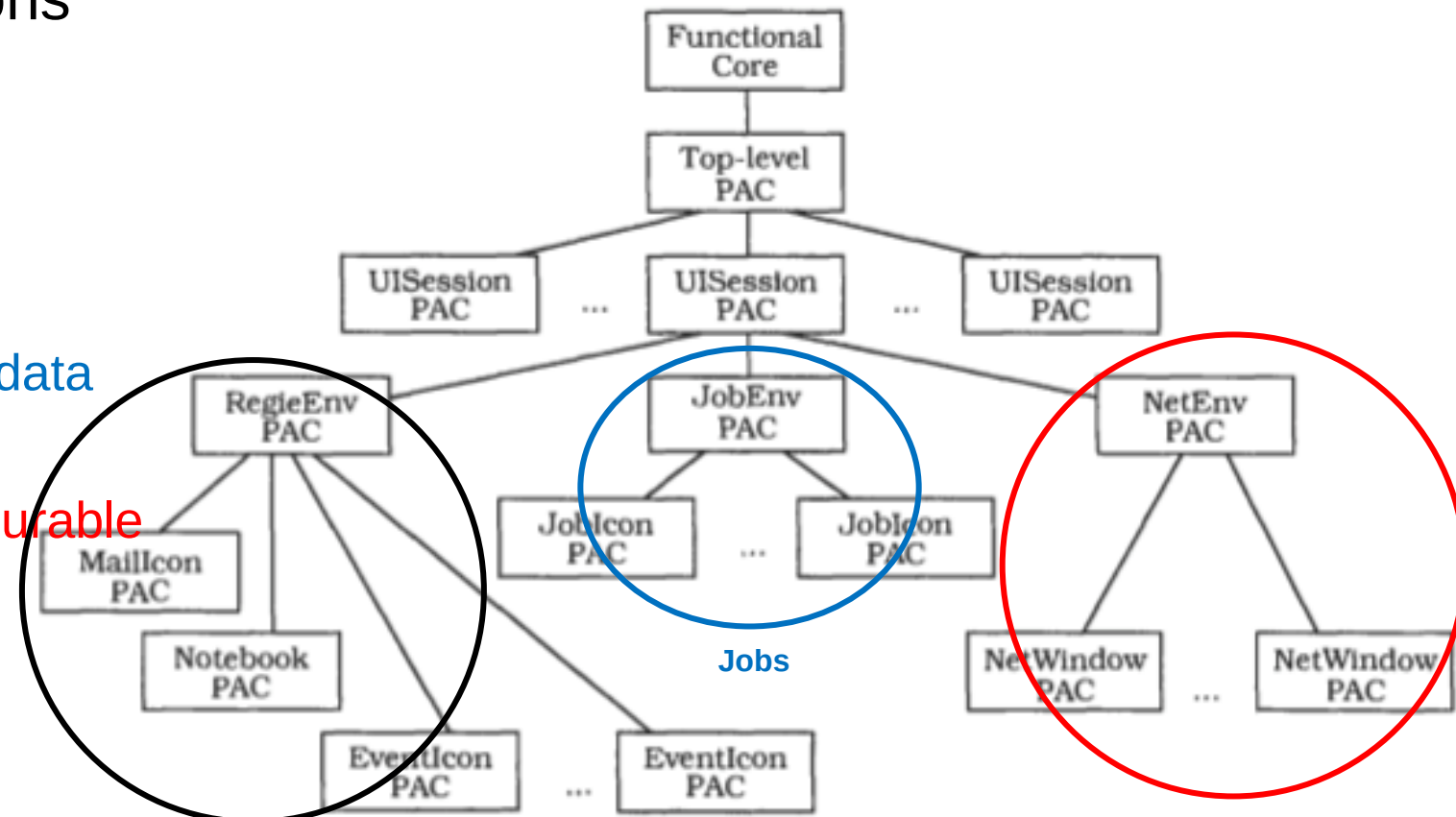
Applicable Domain

- ❑ **Suitable for an interactive system** where **the system can be divided into many cooperating agents in a hierarchical manner.**
 - Each agent has its own specific assigned job.
- ❑ **Suitable when the coupling among the agents is expected to be loose** so that changes on an agent does not affect others.

Example – Network Traffic Management

□ This system includes functions for:

- Gathering traffic data
- Threshold checking
- Logging
- Statistical evaluation of traffic data
- Visualizing of traffic flow
- Displaying various user-configurable views of the whole network
- Etc.

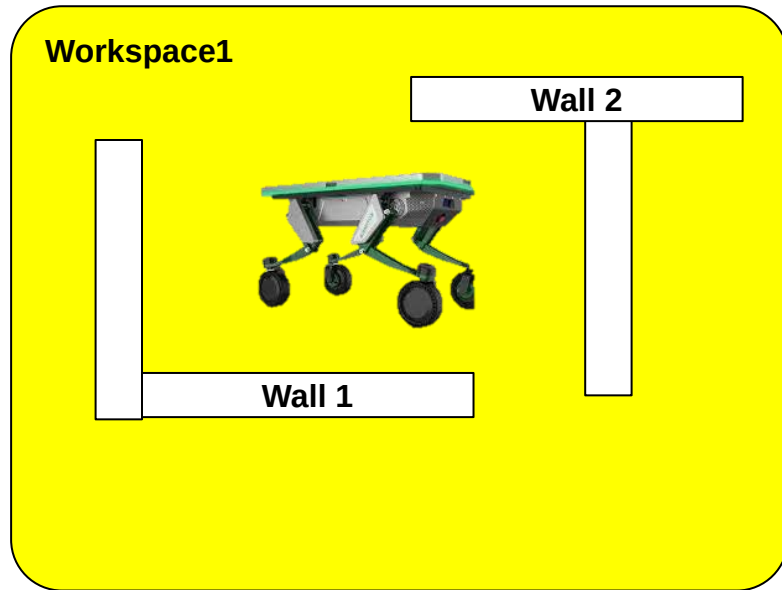


Additional services

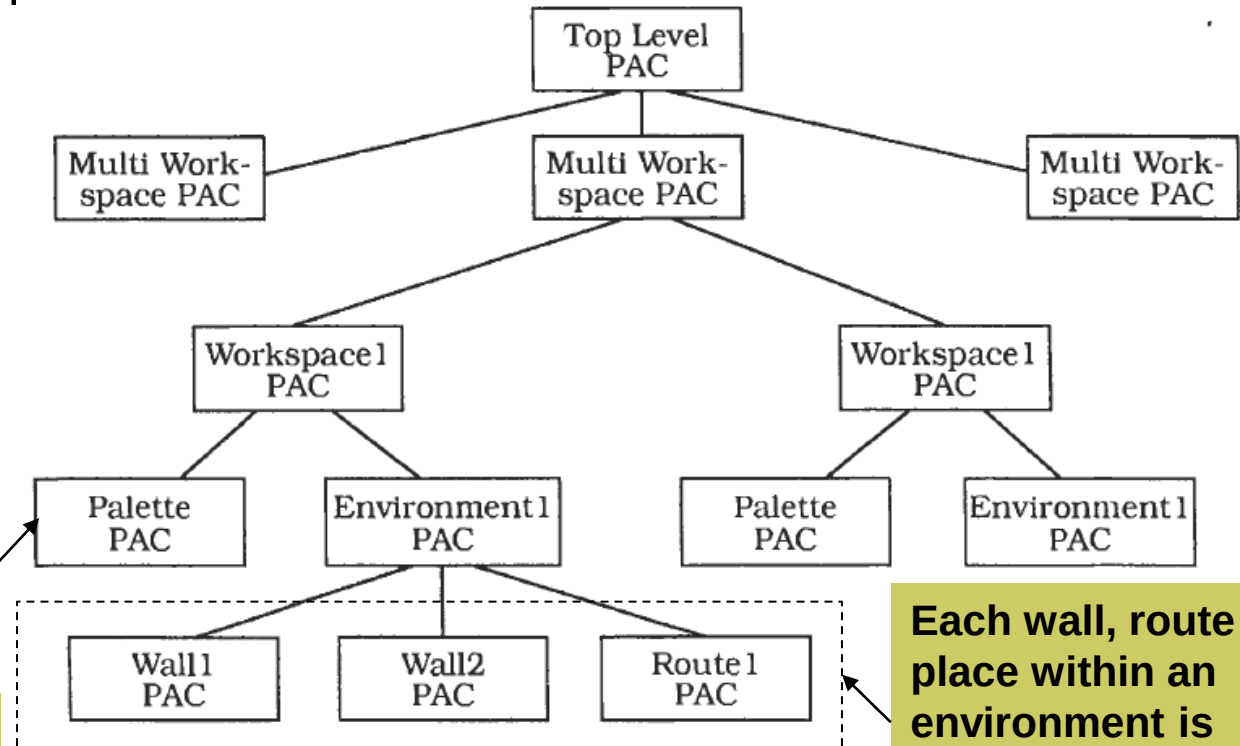
Views

Example – Mobile Robot

- The **Mobile Robot** interact with an operator navigating within a closed and hazardous environment



Palette PAC implements the control that operators can initiate on an environment



Each wall, route and place within an environment is represented by its own **bottom-level PAC agent**

Consequences

■ Benefits

■ Separation of concerns

- Different semantic concepts in the application domain are represented by separate agents.
- This allows the development of a dedicated data model and user interface for each semantic concept within the application, independently of other semantic concepts.

■ Support for change and extension

- Changes within the presentation or abstraction components of a PAC agent do not affect other agents in the system
- Easy to plug-in new agent or replace an existing one.

■ Support of multi-tasking.

- PAC agents can be distributed easily to different threads, processes, or machines.

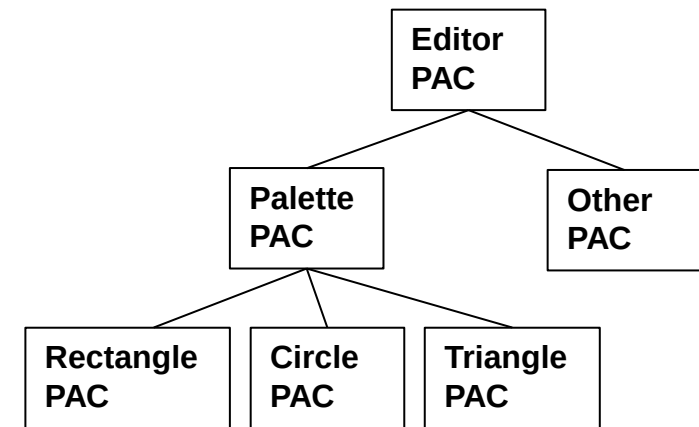
Consequences

■ Weaknesses

■ Increased system complexity

- The implementation of every semantic concept within an application as its own PAC agent may **result in a complex system structure**.
- E.g., Graphics Editor

■ **Overhead in the communication** between PAC agents may impact system **efficiency**

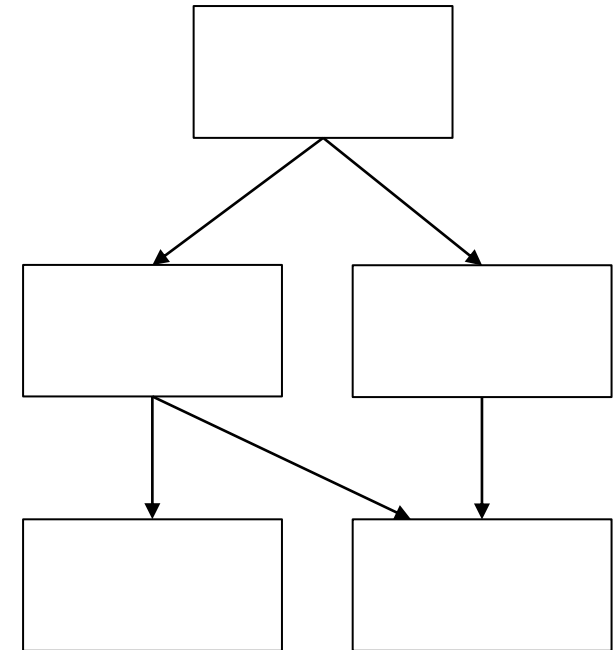


Architecture Style

HIERARCHICAL ARCHITECTURE

Hierarchical Architecture

- ❑ Hierarchical architecture **views the whole system as a hierarchy structure**
 - The software system is decomposed into **logical modules at different levels in the hierarchy**.
 - A lower-level module provides services to its **adjacent upper-level modules**, which invokes the methods or procedures in the lower level.
 - Modules at different levels are **connected by explicit or implicit method invocations** (call-and-return);



Styles of Hierarchical Architecture

- **Main-Subroutine**

- **Master-Slave**

- **Layered**

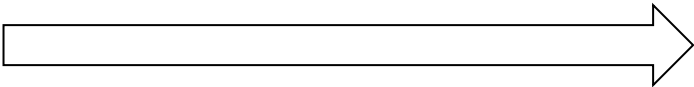
 - **Virtual Machine**

 - **Microkernel**

Main-Subroutine Style

- A software system is **decomposed into subroutines hierarchically** refined according to the desired functionality of the system

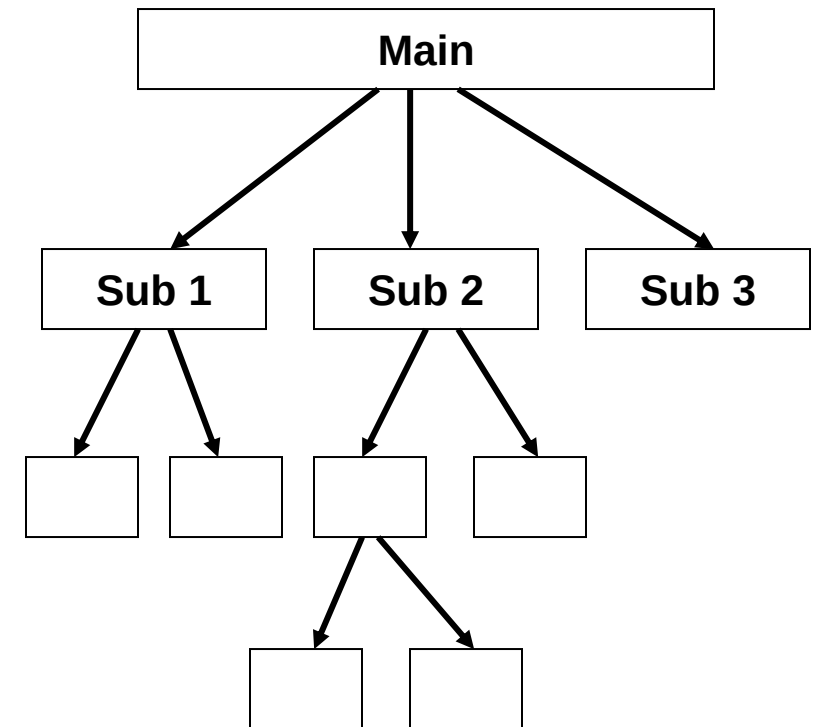
- **Elements**

- **Components:** Procedures (or functions) 
- **Connectors:** Procedure calls (or functional calls) 

- **Relations**

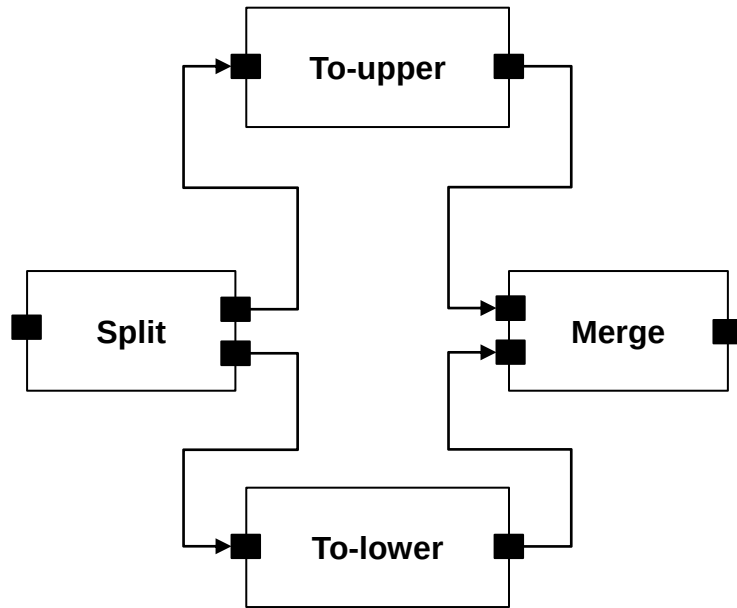
- **Constraints**

- Centralized
- A single thread of control
- Each component in the hierarchy gets this control from its parent and passes it along to its children

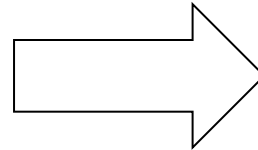


Example

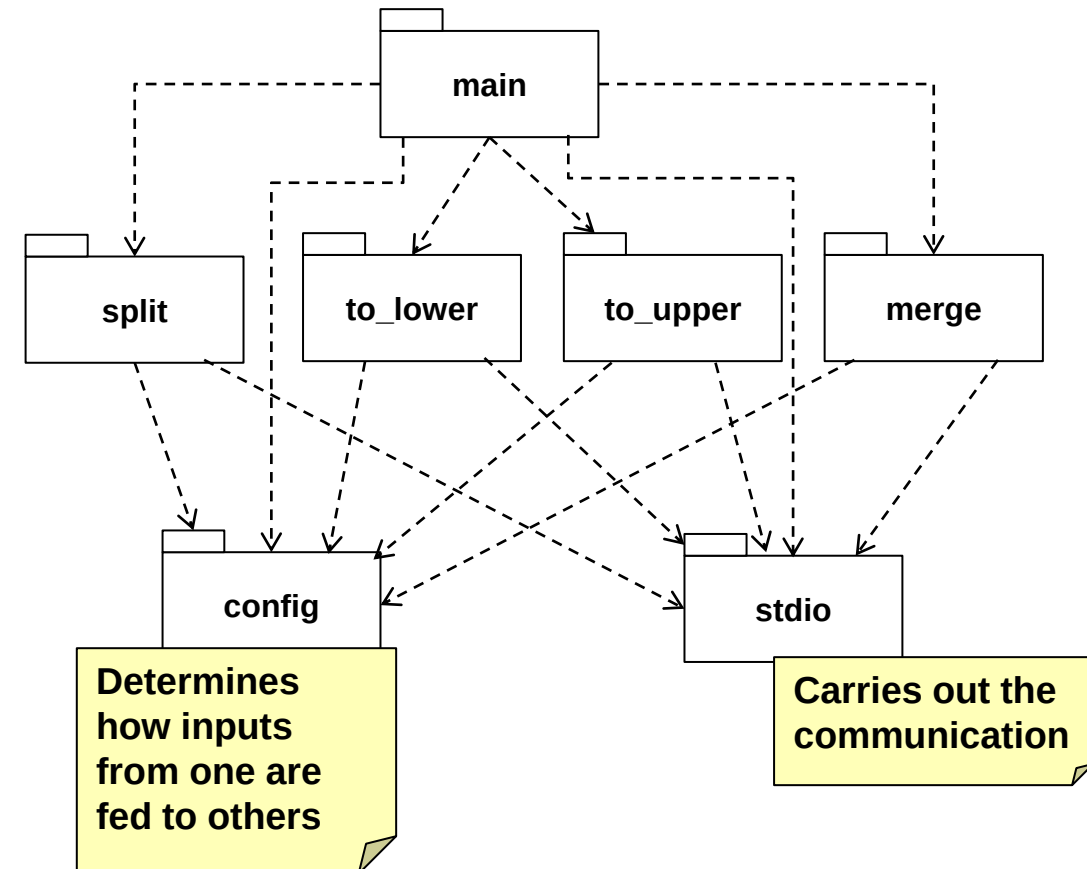
Pipe and Filter Style



→ Pipe
 ----> uses



Main-Subroutine style



Consequences

□ **Benefits**

■ **Simplicity**

- This style is relatively simple and easy to understand, especially for small to moderately sized programs. It follows a linear flow of control, making it straightforward to comprehend the program's logic

■ **Ease of testing**

- The modular structure makes unit testing easier. Each subroutine can be tested independently, facilitating overall testing of the program.

□ **Weaknesses**

- **Vulnerable as it contains globally shared data.**
- **Tight coupling may cause more ripple effects of changes.**

Master-Slave Style

- An architecture style that **supports fault tolerance and system reliability**.

- **Elements**

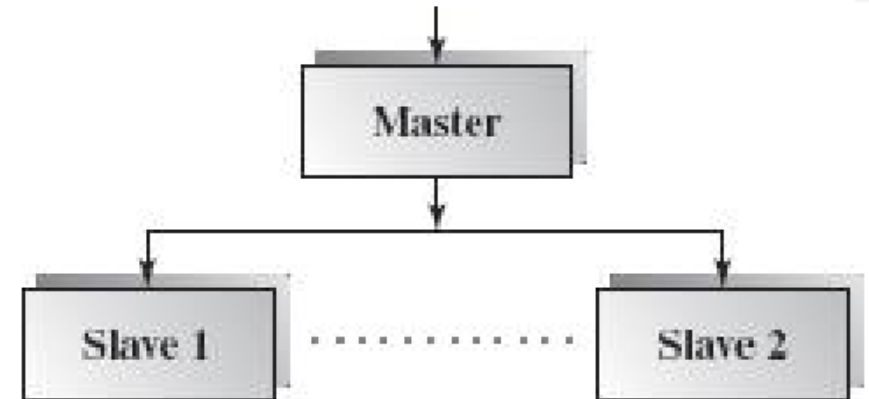
- **Slaves**

- provide to the master **replicated services**
 - the same functional task by different algorithms
 - a totally different functionality

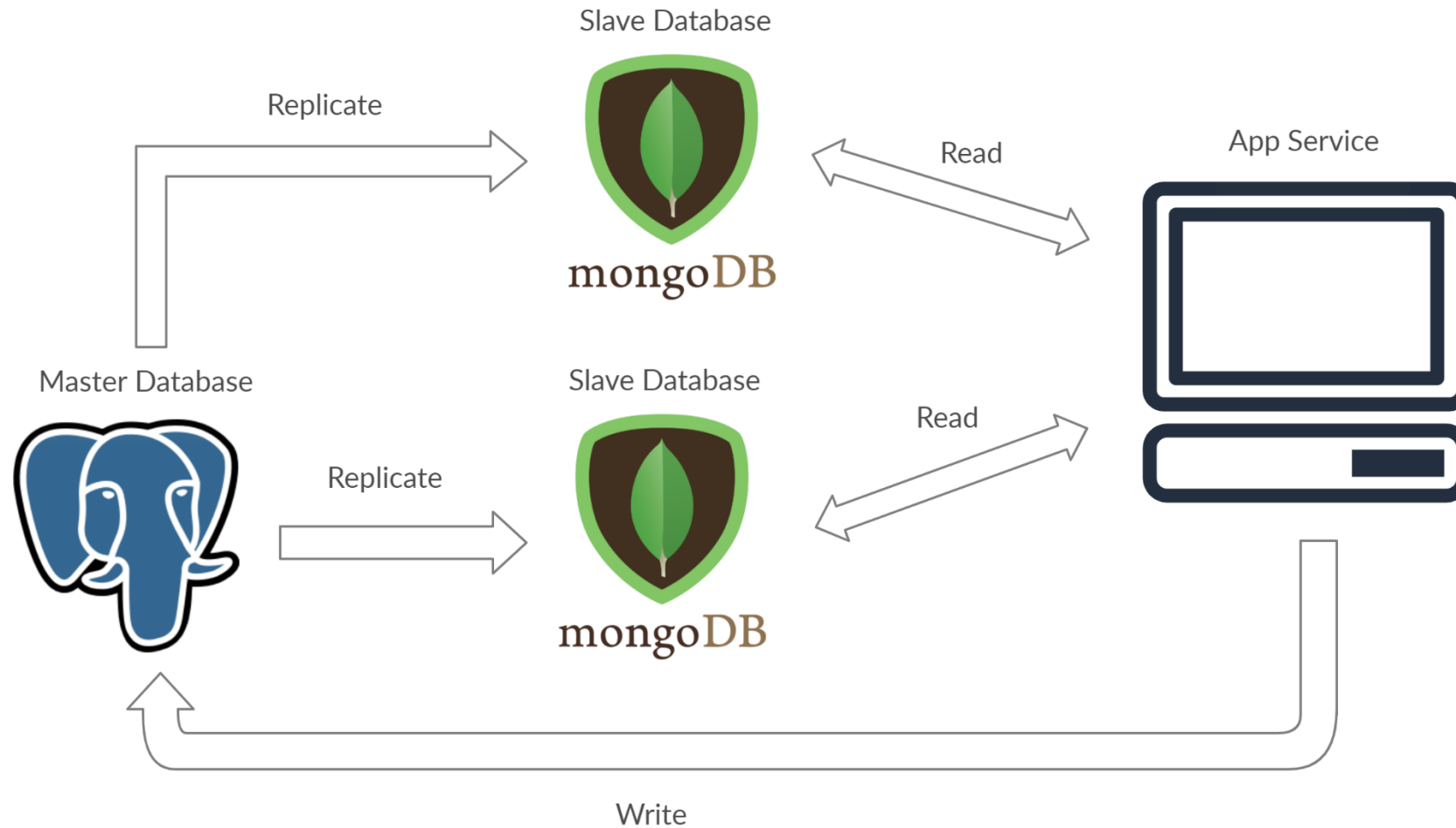
- **Master**

- configures the invocations of the replicated services and
 - receives the results back from all slaves and
 - selects a particular result among slaves by certain selection strategies

- **Relations**



Example



Consequences

□ Benefits

■ Fault tolerance.

- If a slave component fails, the master can reassign the task to other available slaves, ensuring that the system can continue functioning with minimal disruption.

■ Parallel computing

- All slaves can be [executed in parallel](#).

■ Scalability

- By adding more slave components to the system, the system can handle increased workload.

■ Accuracy of computation.

- Since the same task is delegated to several different implementations, [inaccurate results can be ruled out easily with a majority vote strategy](#) or other algorithms

Consequences

❑ Weaknesses

■ Single Point of Failure

- ❑ The master component can become a single point of failure. If the master fails, the entire system's operation may be affected, and tasks may not be assigned or managed correctly.

■ Communication Overhead

- ❑ The communication between the master and slaves can introduce communication overhead, especially in a distributed environment. This overhead can potentially reduce the overall performance gain achieved through parallel processing.

Layered Style

□ Elements

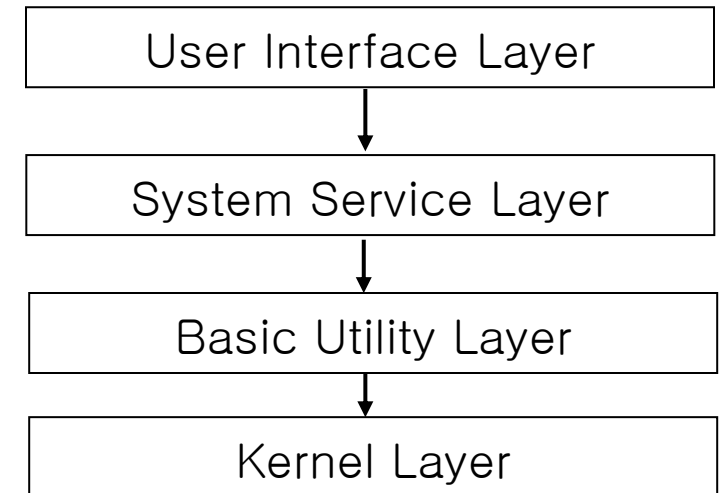
- **Layer** : a collection of components at the same

□ Relations

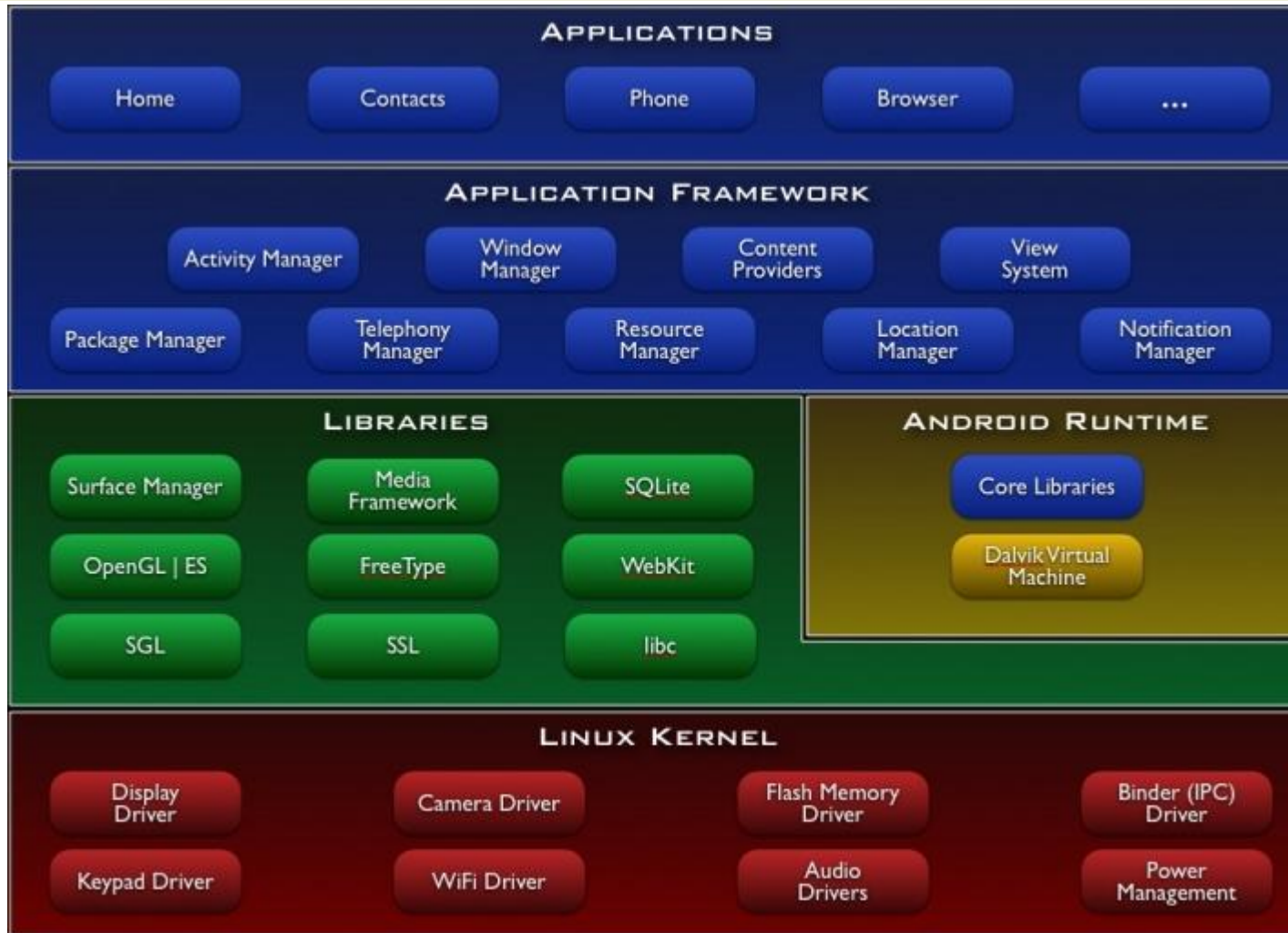
- **Allowed-to-use** (a specialization of a more generic depends-on relation)

□ Constraints

- The **allowed-to-use** relations should not be circular
- There are at least two layers



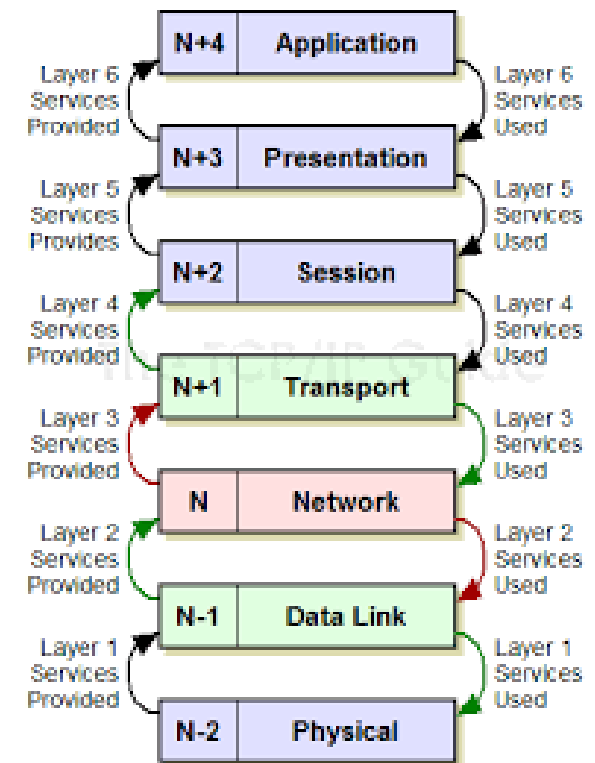
Examples – Android Architecture



Examples – OSI 7 Layer Architecture

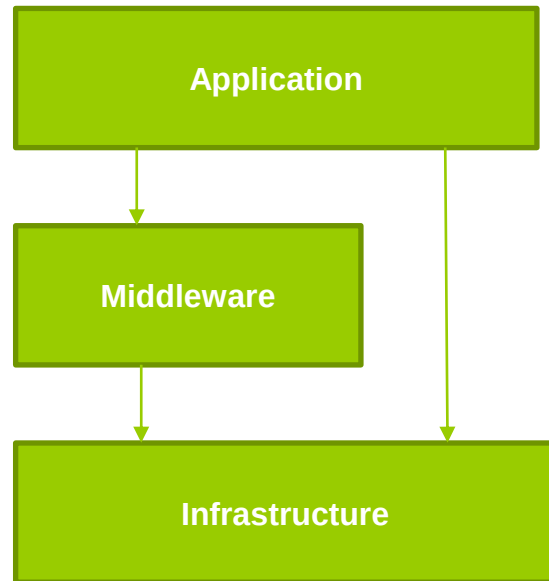
7	Application Layer	Human-computer interaction layer, where applications can access the network services
6	Presentation Layer	Ensures that data is in a usable format and is where data encryption occurs
5	Session Layer	Maintains connections and is responsible for controlling ports and sessions
4	Transport Layer	Transmits data using transmission protocols including TCP and UDP
3	Network Layer	Decides which physical path the data will take
2	Data Link Layer	Defines the format of data on the network
1	Physical Layer	Transmits raw bit stream over the physical medium

Each Layer deals with a specific aspect of communication and use the services of the next lower layer



Variants

❑ Relaxed Layered System



Performance vs Maintainability Tradeoff

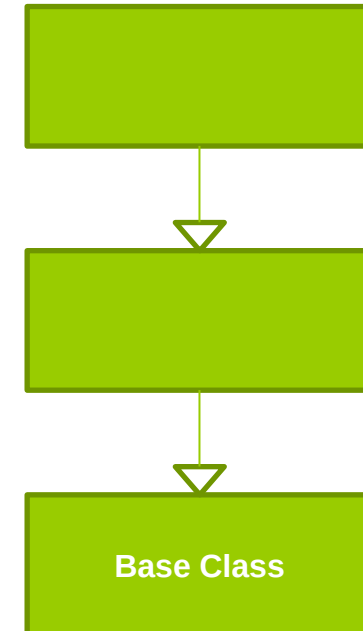
Examples : Unix Operating Systems or X Window Systems

The infrastructure systems are modified less often application systems and their performance is more important than their maintainability

Variants

□ Layering Through Inheritance

- A higher layer requesting services from a lower layer inherits from the lower layer's implementation
- Benefits
 - Higher layers can modify lower-layer services according to their needs
- Weaknesses
 - If the data layout of a base class changes, all subclasses must be recompiled



Consequences

□ Benefits

■ Independence

- Different components of the application can be independently deployed, maintained, and updated, on different time schedules
- Makes possible for team members to work in parallel on different parts of the application with minimal dependencies
- Testing the components independently of each other.

■ More secure

- Each layer may hide private information from other layers

■ Reusability

- Each layer, being cohesive and is coupled only to lower layers, makes it easier for reuse by others and easier to be replaced or interchanged

Consequence

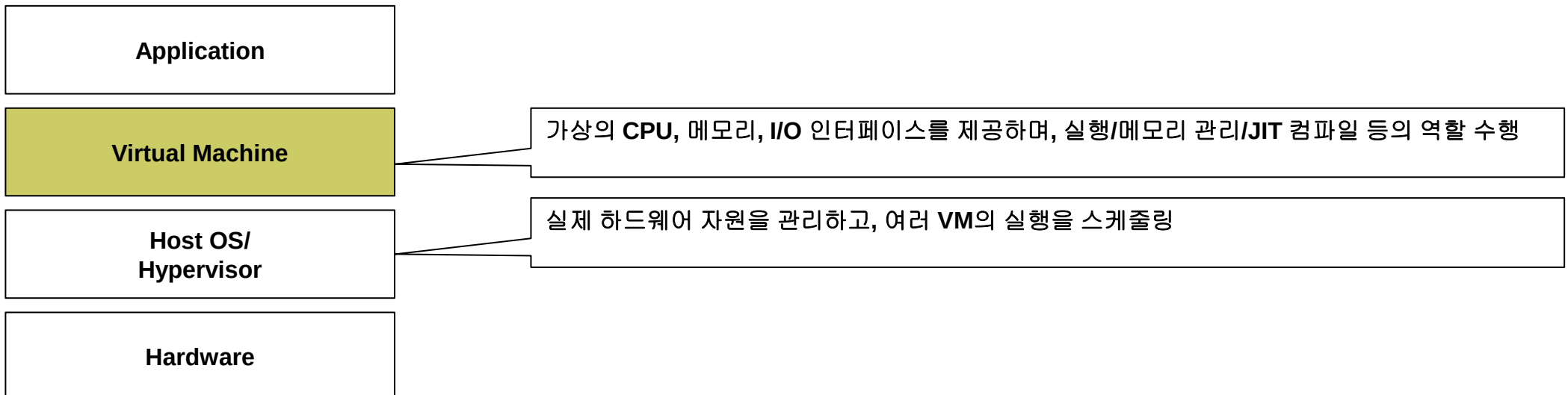
❑ Weaknesses

- Performance degrades if we have too many layers
- Difficulty of establishing the correct granularity of layers
 - ❑ Too few layers does not fully exploit the pattern's potential for reusability, changeability and portability
 - ❑ too many layers introduce unnecessary complexity and overhead
- Can't be used for simple applications because it adds complexity

Virtual Machine

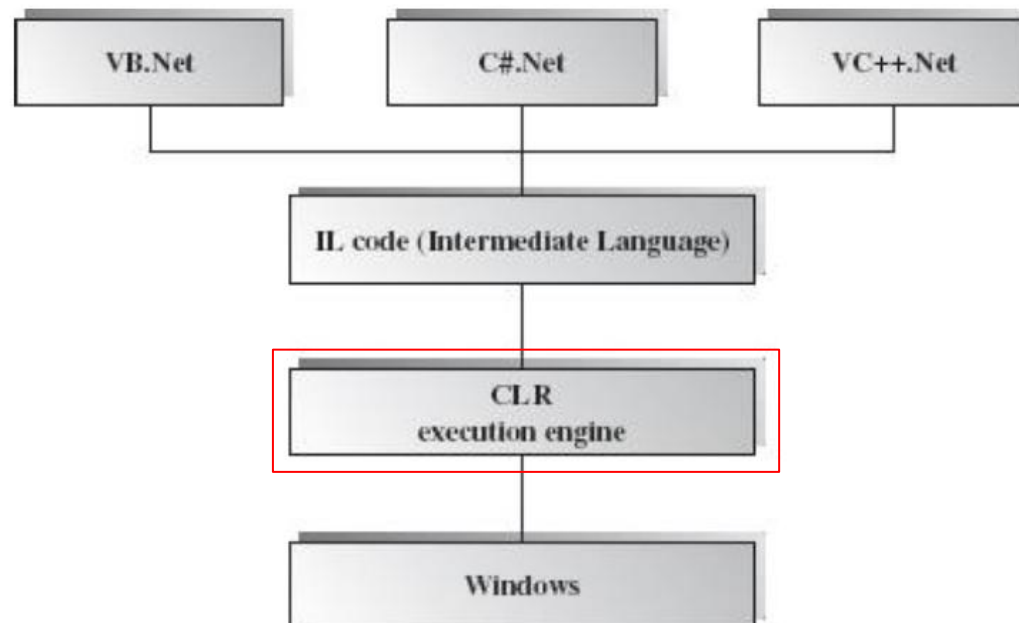
- 하드웨어나 운영체제 위에서 가상의 실행 환경(**Virtual Execution Environment**) 을 제공하는 소프트웨어 추상화 계층

- Elements



Example

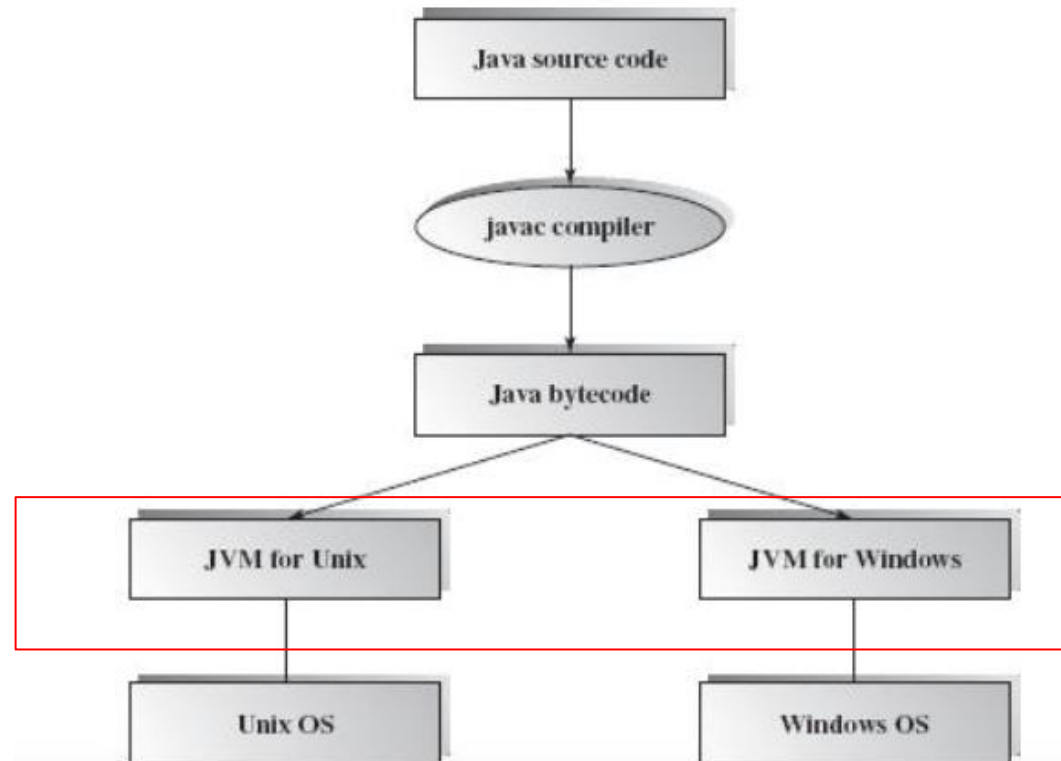
- ❑ **Common Language Runtime (CLR) of Microsoft .NET platform**
 - plays the role of a **virtual machine that uses a single intermediate language** to unify several modules VB.NET, VC.NET, and C#



Example

□ Java Virtual Machine (JVM)

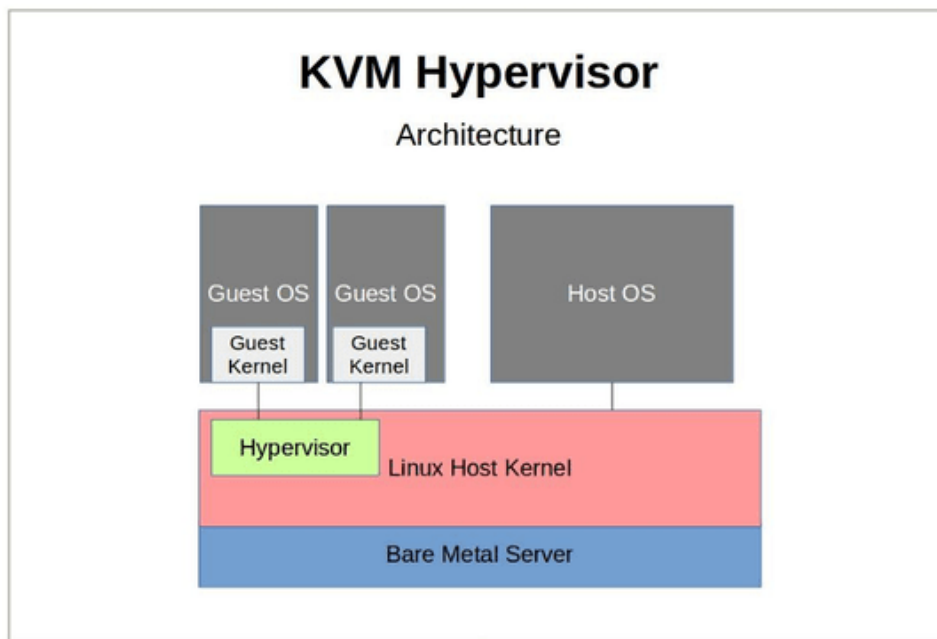
- The Java bytecode and other Java internal code generated by a compilation system (e.g., javac) can run on any operating system that supports JVM



Example

□ KVM

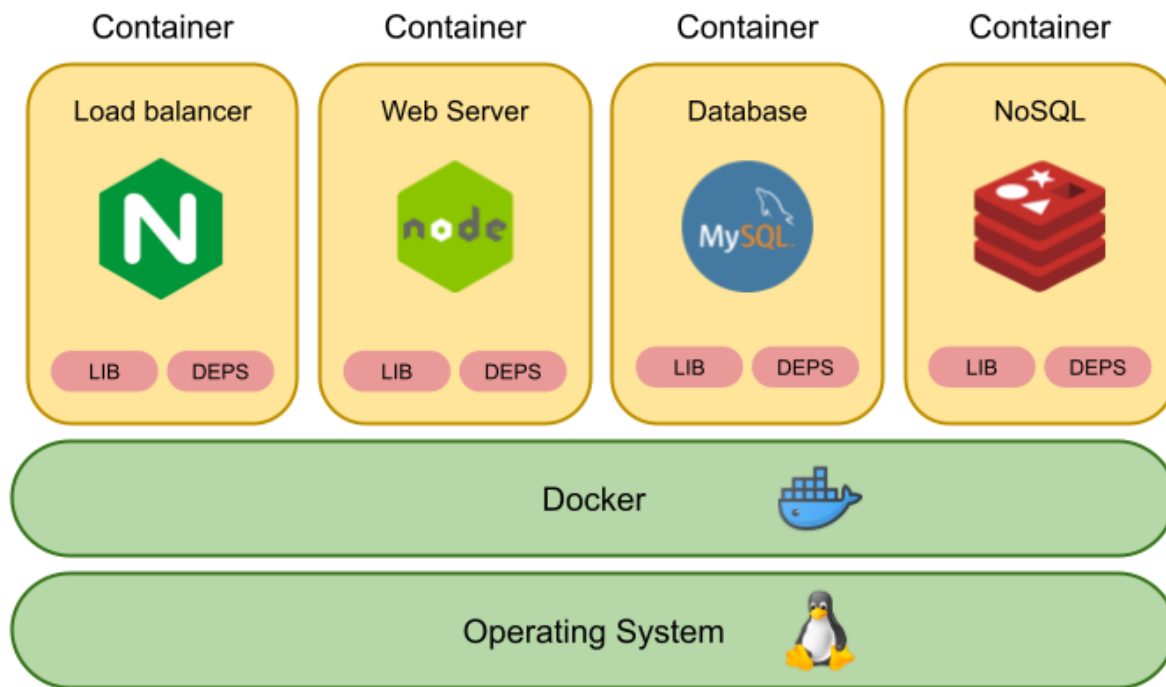
- 커널 기반 가상 머신(KVM)은 물리적 Linux 시스템에 설치하여 가상 머신을 생성할 수 있는 소프트웨어 기능
- OS 수준 가상화, 완전한 하드웨어 가상화 수행



Example

□ Docker based Microservices

- 서버 리스 및 컨테이너 기반의 가벼운 VM 실행 환경



Consequences

□ Benefits

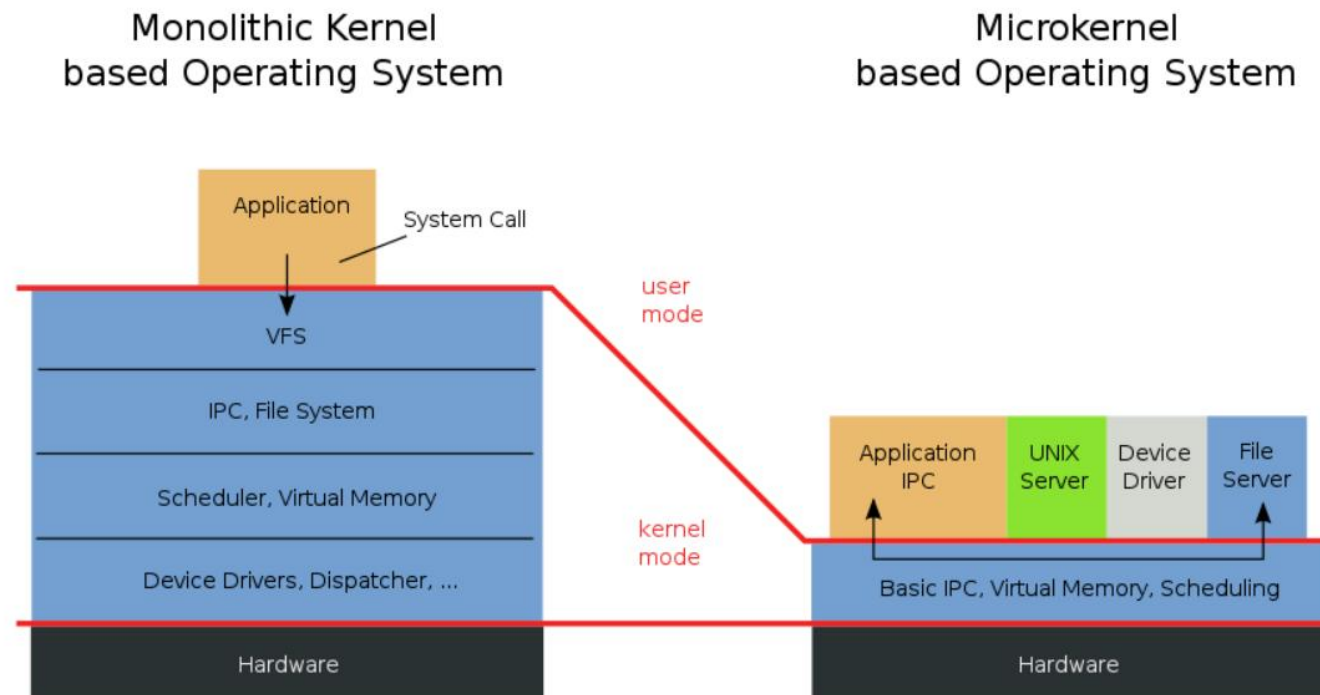
- **Portability** and machine platform independency.
- Simplicity of software development.
- **Simulation** for disaster working model.

□ Weaknesses

- **Slow execution** due to the interpreter nature.
- **Additional overhead** due to the new layer.

Microkernel

- Originally, a **microkernel** is the near-minimum amount of software that can provide the mechanisms needed to implement an operating system (OS)



<https://en.wikipedia.org/wiki/Microkernel>

Microkernel Style

□ Elements

■ Core System

- the minimal functionality required to run the system

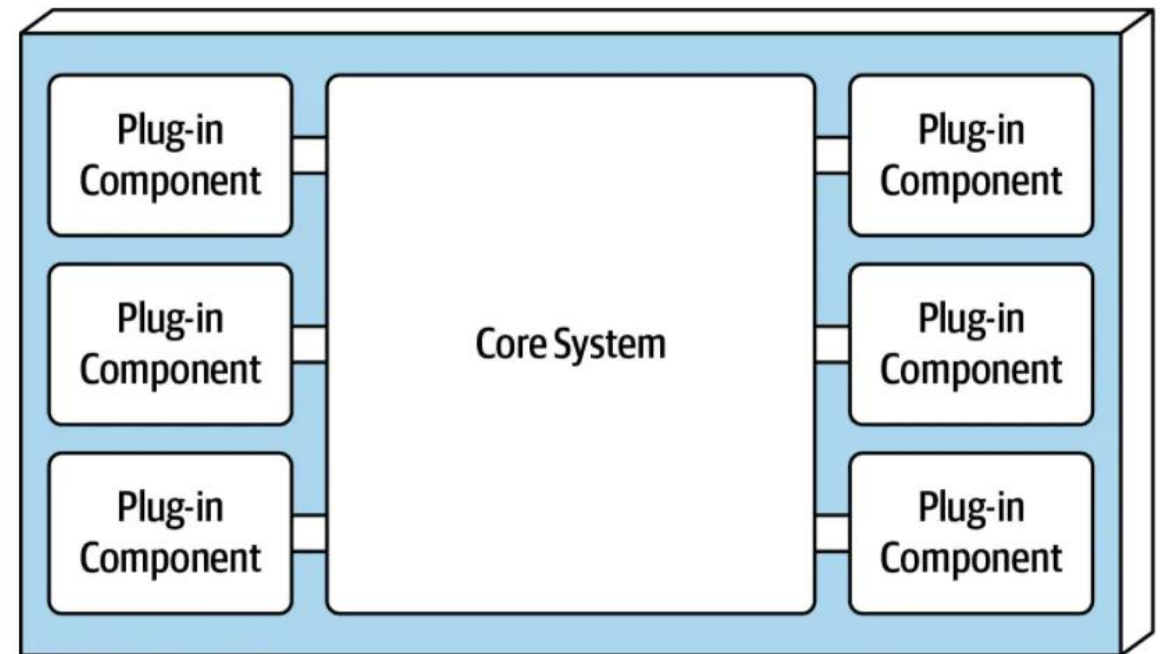
■ Plug-in Component

- standalone, independent components that contain specialized processing, additional features to enhance or extend the core system

□ Constraints

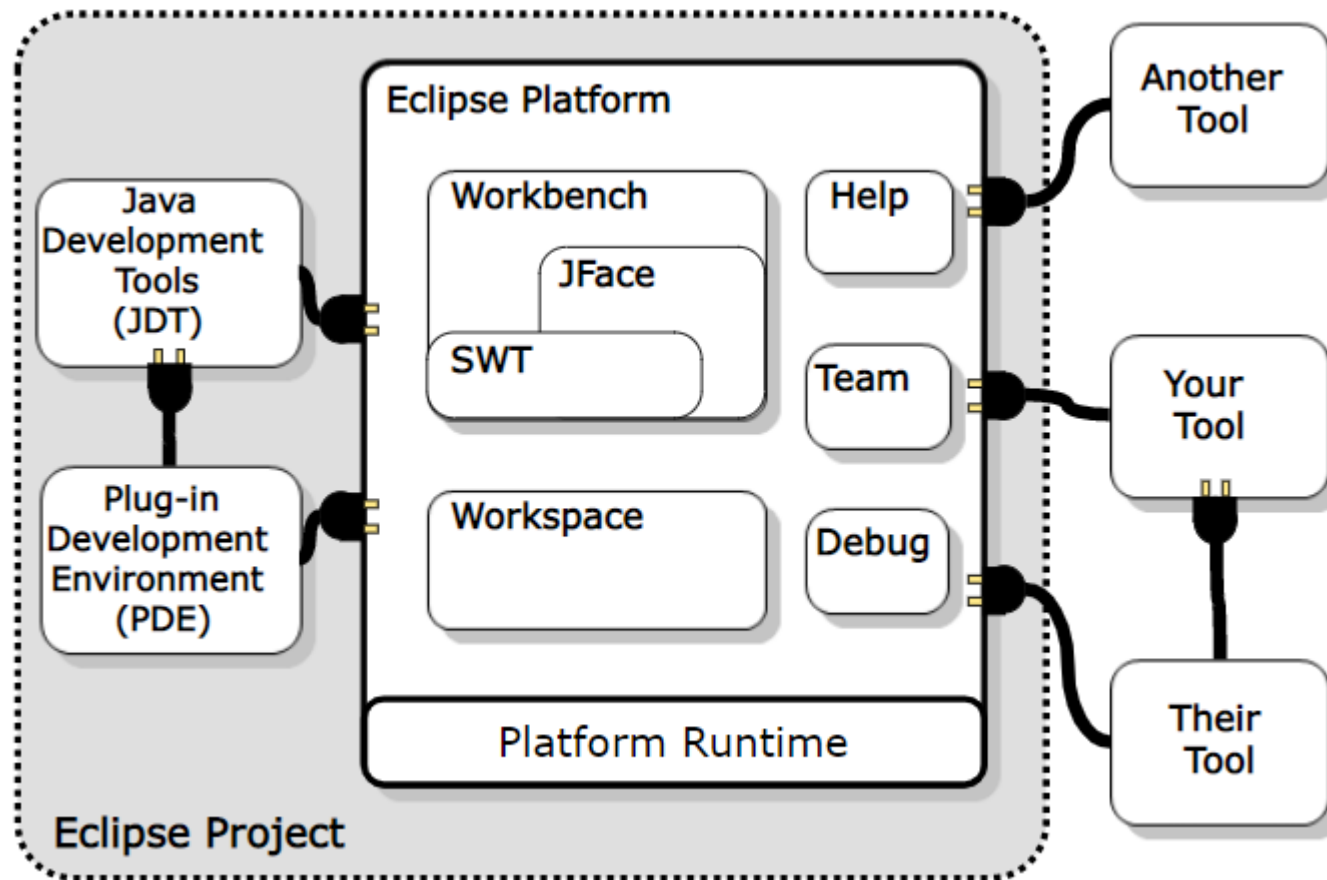
■ Minimality

- The microkernel itself should be kept minimal, containing only the essential functionalities



Example

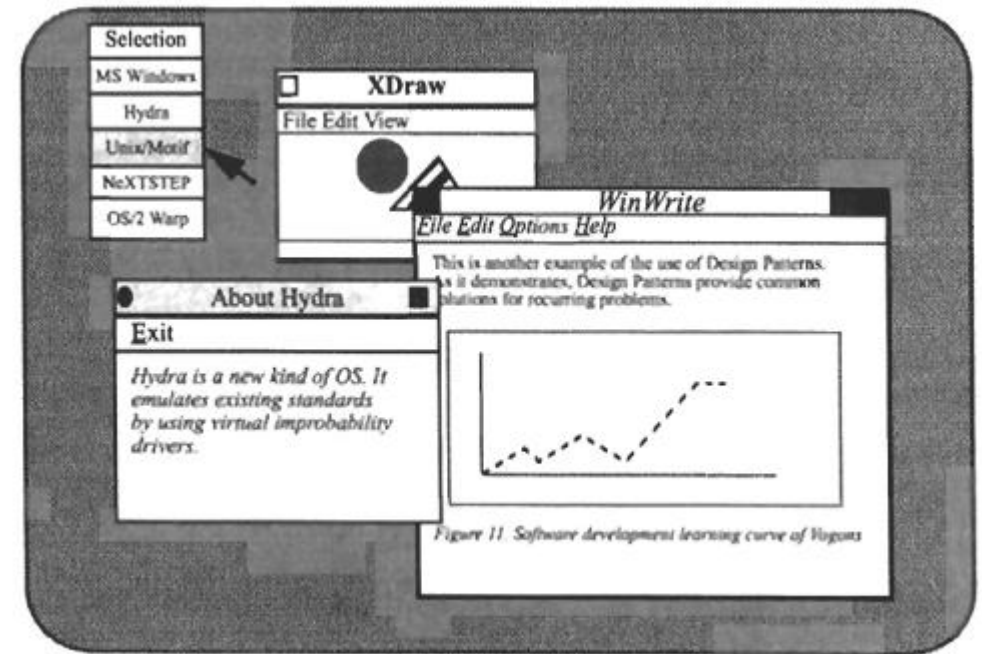
□ Eclipse Architecture



Microkernel Pattern [POSA1996]

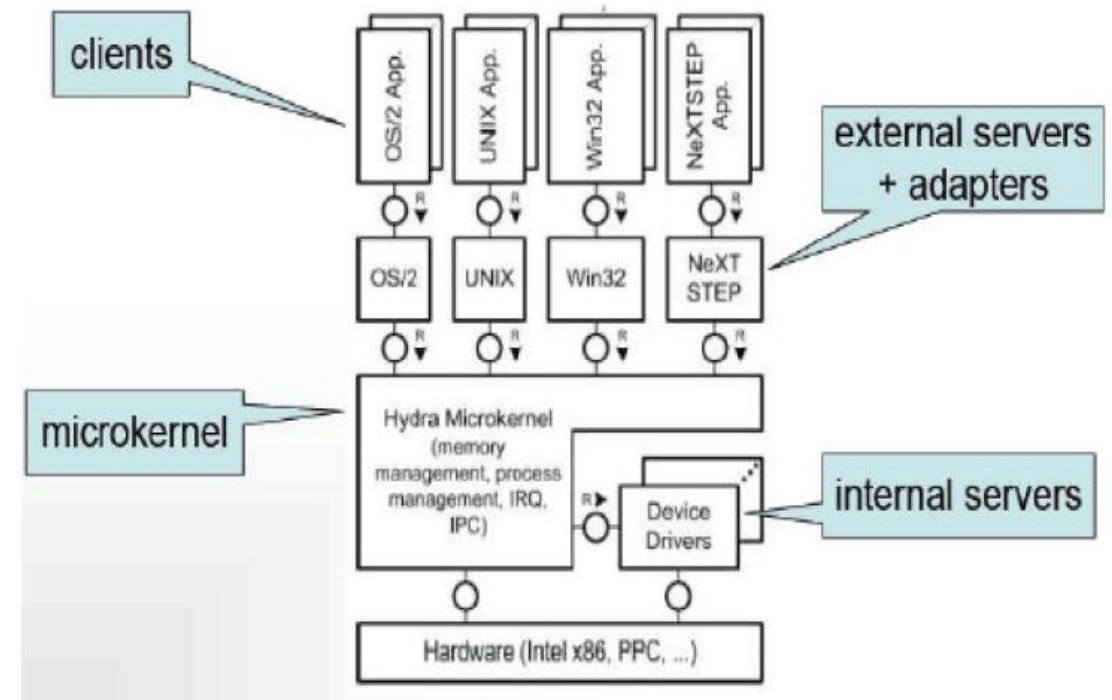
□ Problem

- Developing a new OS for desktop computers called Hydra
- Portable to the relevant hardware platforms
- Run applications written for other popular OS (e.g., Windows, UNIX, etc.)
- Hydra will display all the applications currently running within its main window



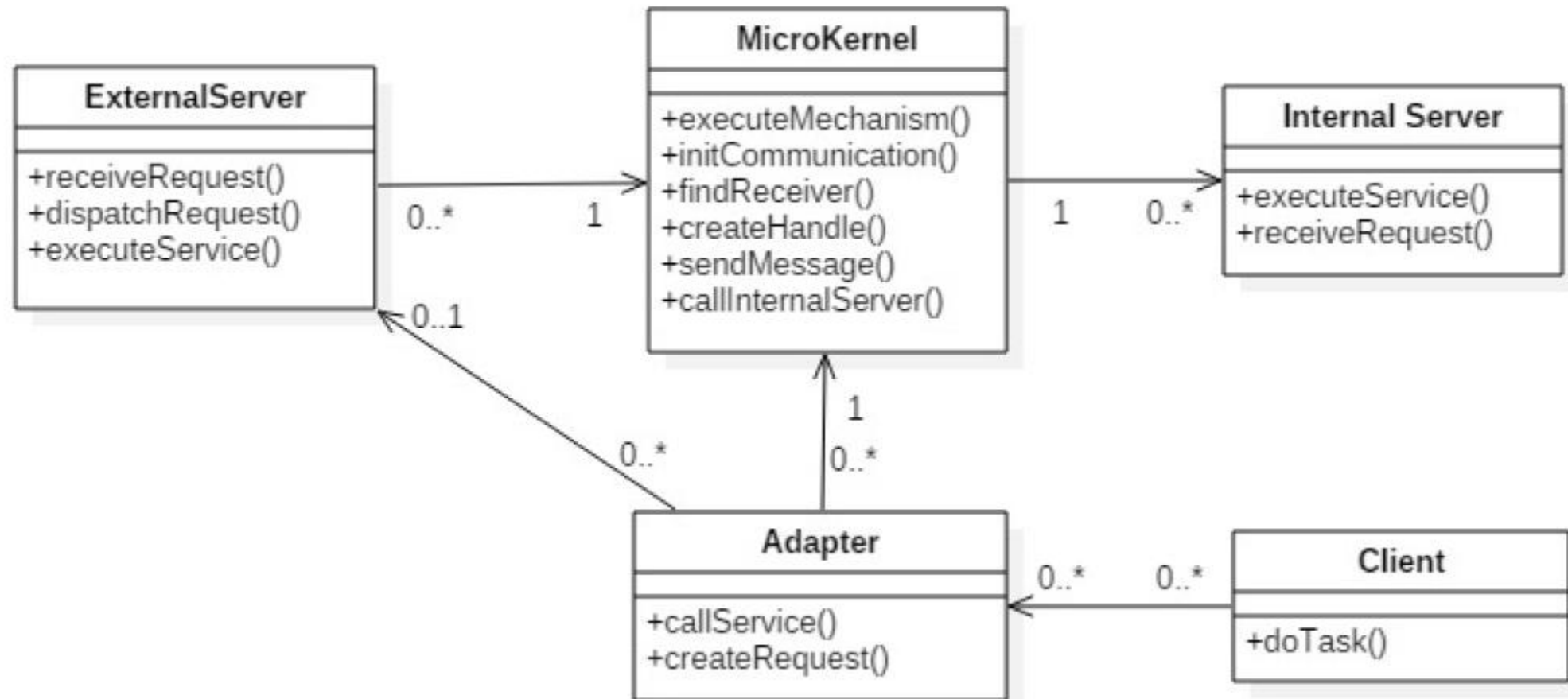
Microkernel Pattern

- ❑ The **microkernel architecture pattern** defines five kinds of participating components.
 - **Microkernel**
 - ❑ Minimal functional core
 - **Internal Servers**
 - ❑ Extended functionality
 - **External Servers**
 - ❑ Client-specific views of the microkernel
 - **Adapter**
 - ❑ Client-specific Interface
 - **Client**
 - ❑ an application that is associated with exactly one external server



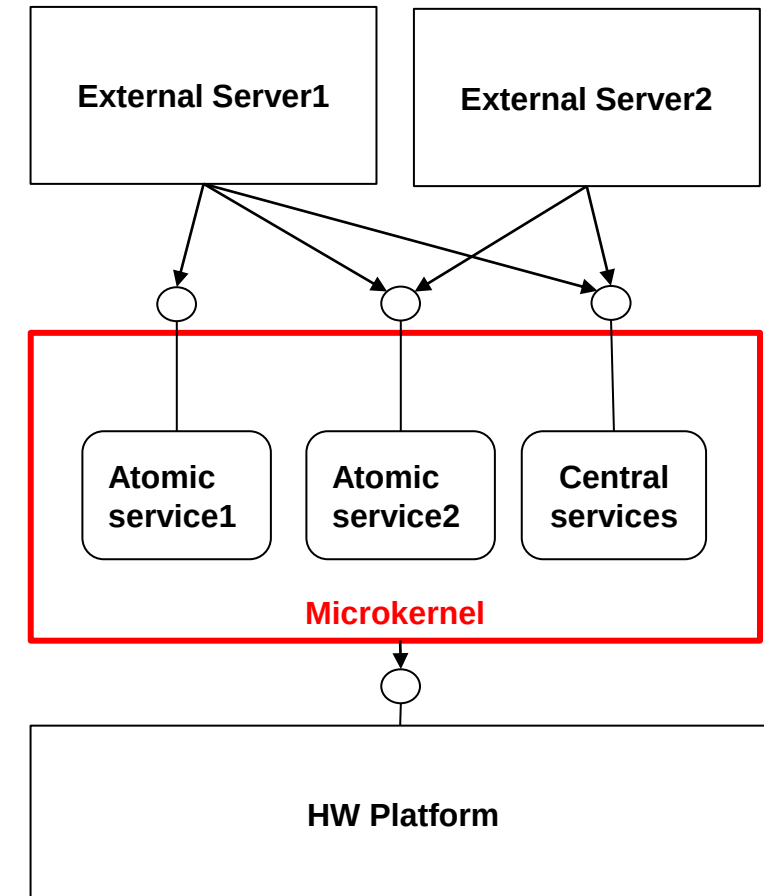
Hydra OS overview

Structure



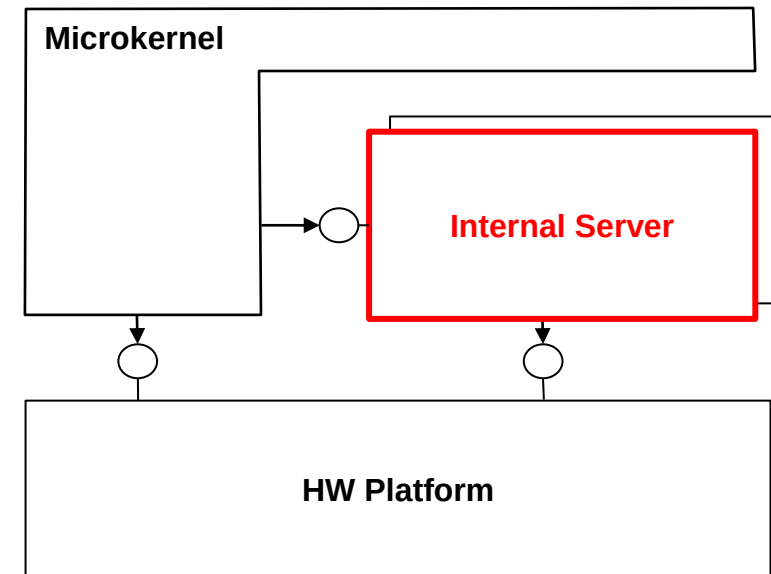
Microkernel Component

- ❑ implements **central services** (e.g., communication facilities or resource handing)
- ❑ implements **atomic services**, which serves as fundamental base on more complex functionality
 - Ex. (atomic services for OS process model)
 - ❑ A service for creating processes – Unix, OS/2 Warp
 - ❑ A service for copying existing address spaces - Unix
- ❑ **System specific dependencies** (e.g., hardware-dependent part) **are encapsulated** within the microkernel



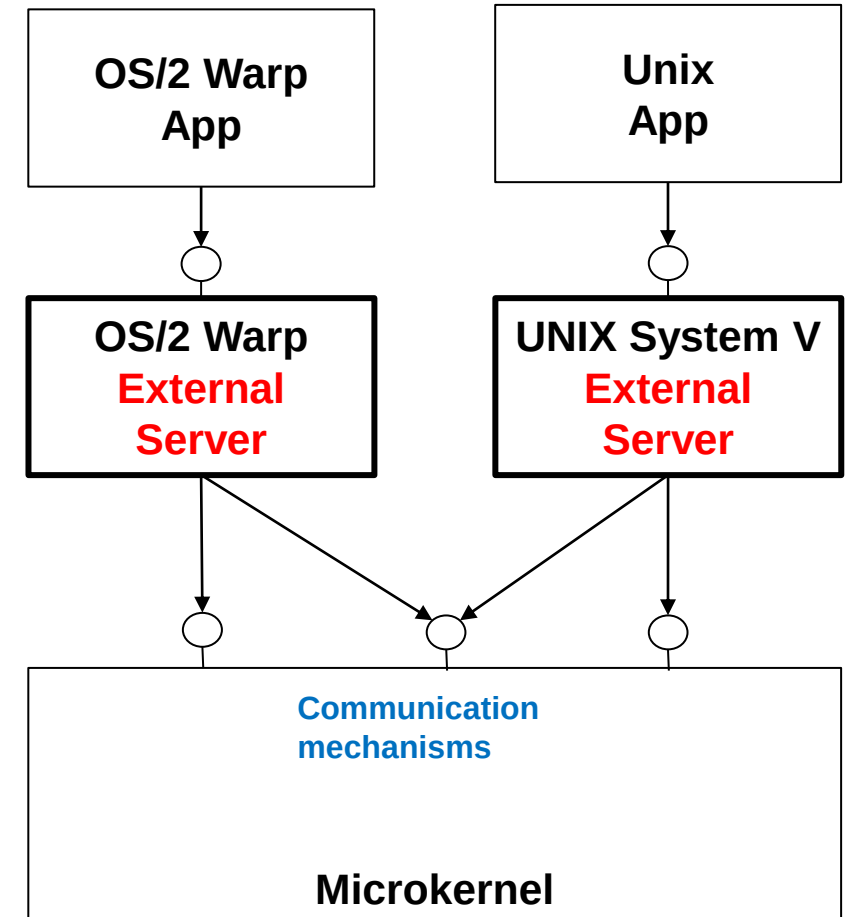
Internal Server Component

- ❑ **extends the functionality** provided by the microkernel.
- ❑ **encapsulates some dependencies on the underlying hardware or software system.**
- ❑ The microkernel **invokes the functionality of internal servers via service requests.**



External Server Component

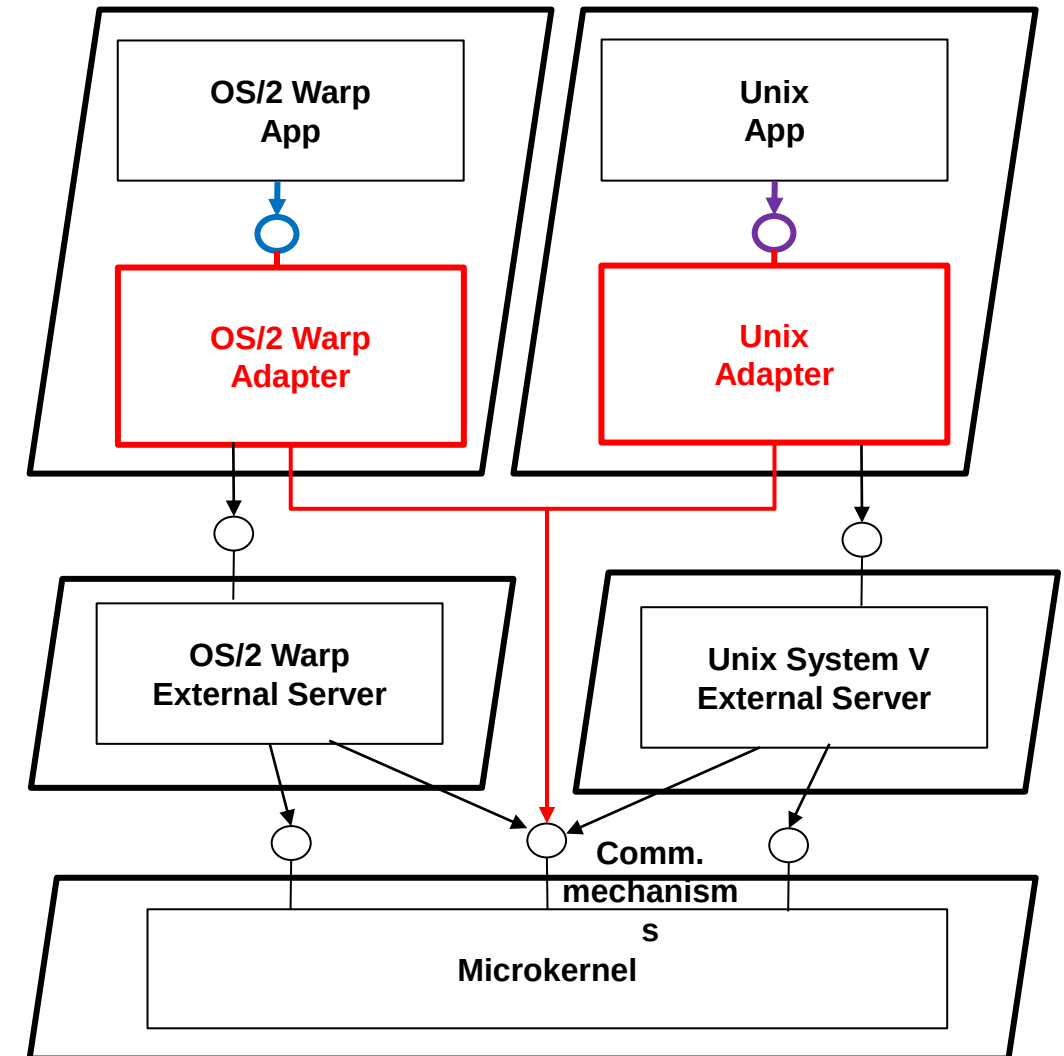
- ❑ An external server uses the microkernel for **implementing its own view of the underlying application domain**
- ❑ Each of these external servers runs in a separate process.
 - It receives service request from clients using communication facilities provided by the microkernel



Adapter Component

□ Adapters

- represent **interfaces** between clients and their external servers.
- Allow clients to access the services of their external server **in a portable way**



Consequences

□ Benefits

■ Flexibility and Extensibility

- If you need to implement an additional view, all you need to do is to add a new external server.
- Extending the system with additional capabilities only requires the addition or extension of internal servers

■ Availability

- A **distributed Microkernel architecture** allows you to run the same server on more than one machine, increasing availability

■ Portability

- Migrating the microkernel to a new hardware environment only requires modifications to the hardware-dependent parts

□ Weaknesses

- Slower than “monolithic systems”
- Complexity of design and implementation

Architecture Styles

DISTRIBUTED ARCHITECTURE

Distributed Architecture

- ❑ A system is divided by **a collection of computational and storage devices** connected through a communications network
- ❑ The subsystems or components within a distributed system **communicate with each other using a number of methods** including
 - message passing, remote procedure calls, and remote method invocation
- ❑ Types of classification
 - Multi-tier
 - ❑ Client-Server
 - Broker
 - ❑ Dispatcher
 - Service-oriented architecture
 - ❑ Microservice
 - Edge-based

Client-Server Style

- ❑ **Decomposes a system into two components: **client** and **server**.**

- ❑ **Elements**

- **The client component**

- ❑ issues a request to the server.

- **The server component**

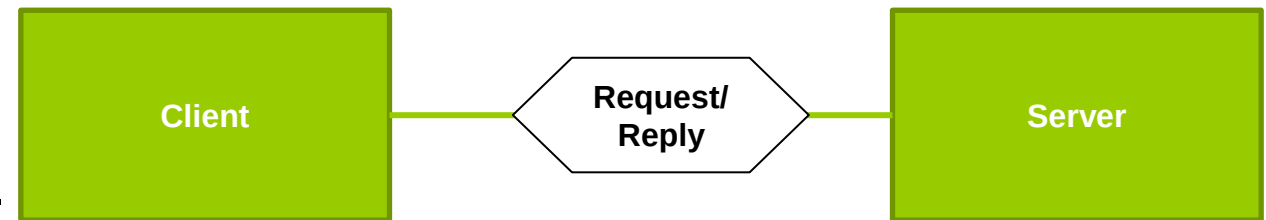
- ❑ receives the request (serving data from a database, printing a document), carries it out, and sends a reply to the client.

- **Request/reply connector**

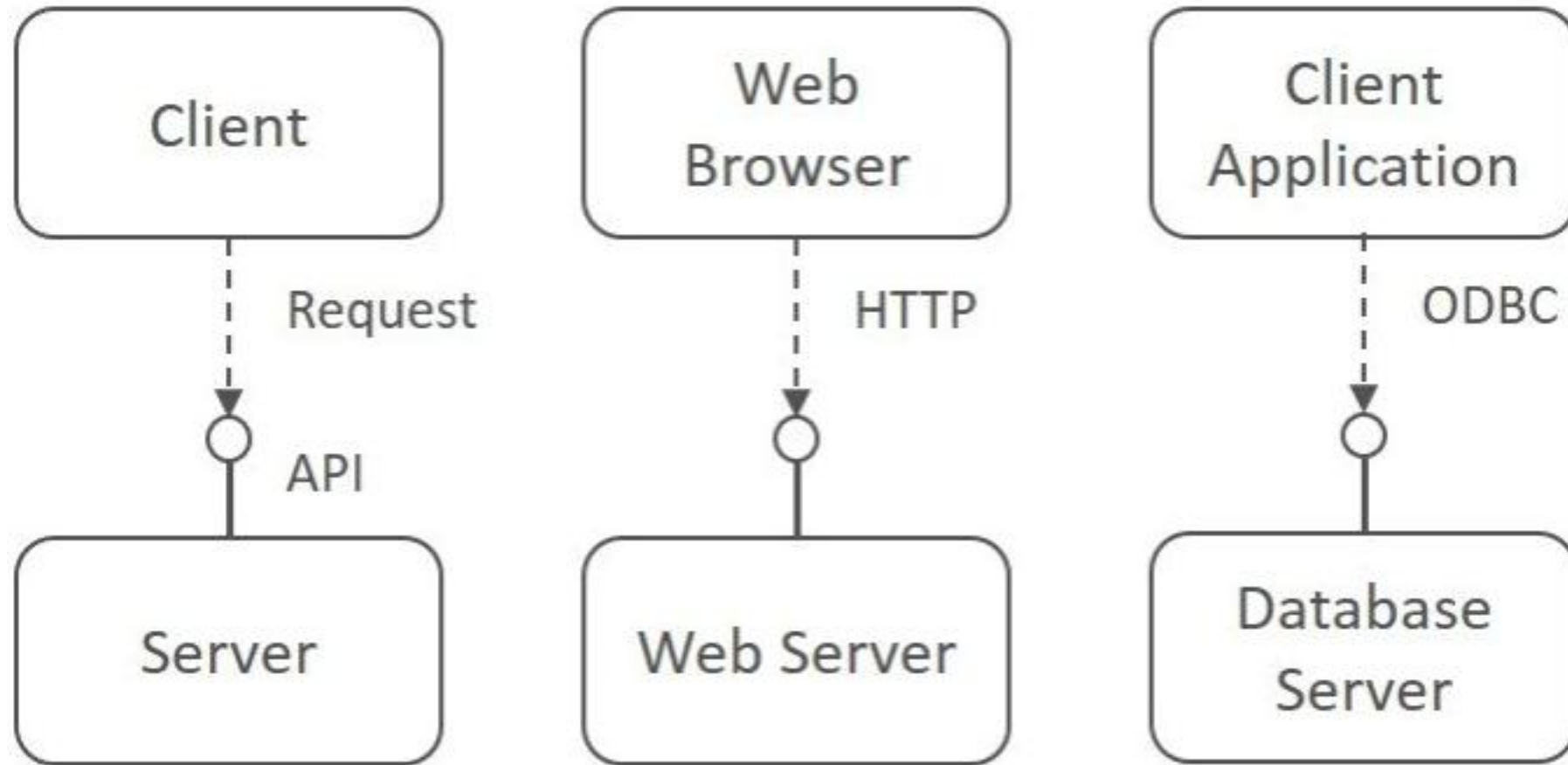
- ❑ a data connector employing a request/reply protocol, used by a client to invoke services on a server.

- ❑ **Constraints**

- Components may be arranged in tiers

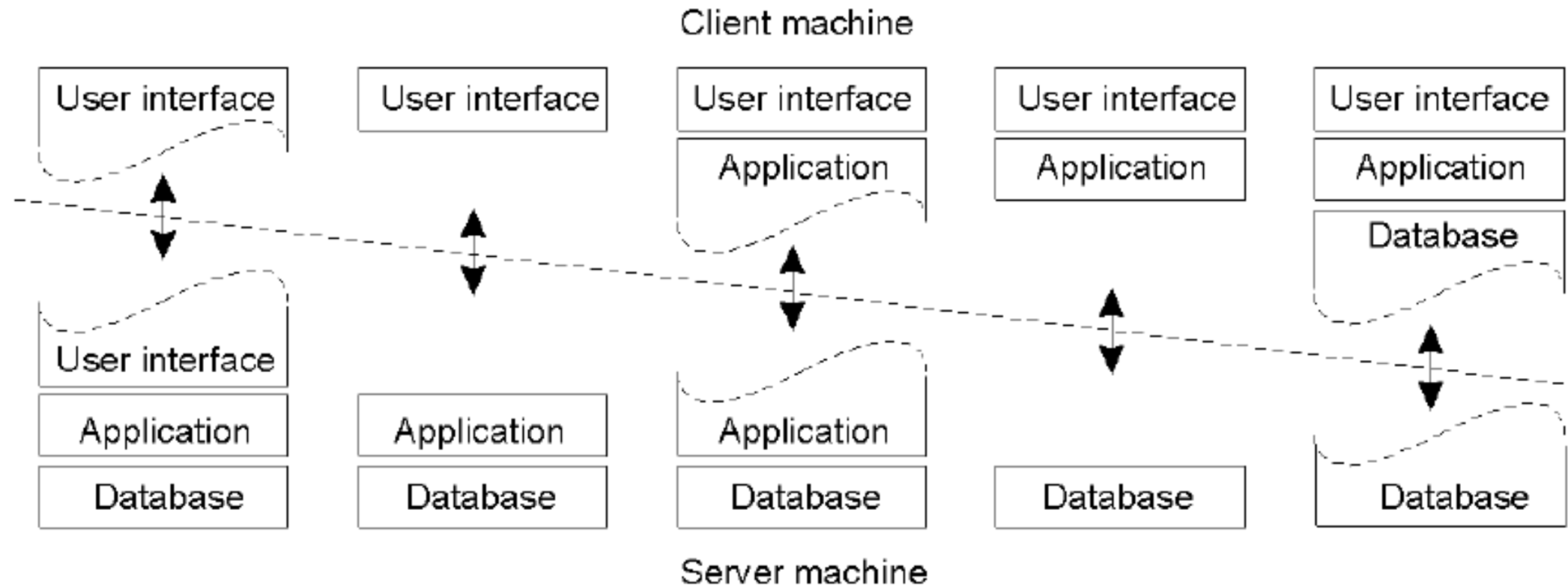


Examples



Variants

□ Thin vs. Thick Client



Consequences

□ Benefits

- **Separation of responsibilities** such as user interface presentation and business logic processing
- **Reusability of server component**
- **Potential for concurrency, scalability**
- **Effective use of resources** when a large number of clients are accessing a high-performance server.

□ Weaknesses

- Security complications
- Server can be a performance bottleneck
- Server can be a single point of failure
- Limited testability

Multi-tiers Architecture Style

- ❑ **Multi-tier architectures** decomposes the system into separate functional units, or **tiers**, with **each tier having specific responsibilities** and interacting with each other to fulfill the overall application's functionality
- ❑ **Layer vs. Tier**
 - A **layer** is a **logical structuring** mechanism for the elements that make up the software solution
 - A **tier** is a **physical structuring** mechanism for the system infrastructure
- ❑ **Variants**
 - 2-tier Architecture (known as **Client-Server** Architecture)
 - 3-tier Architecture
 - 4-tier Architecture

3-tier Architecture Style

□ Elements

■ Client

- Presentation Tier

■ Middleware

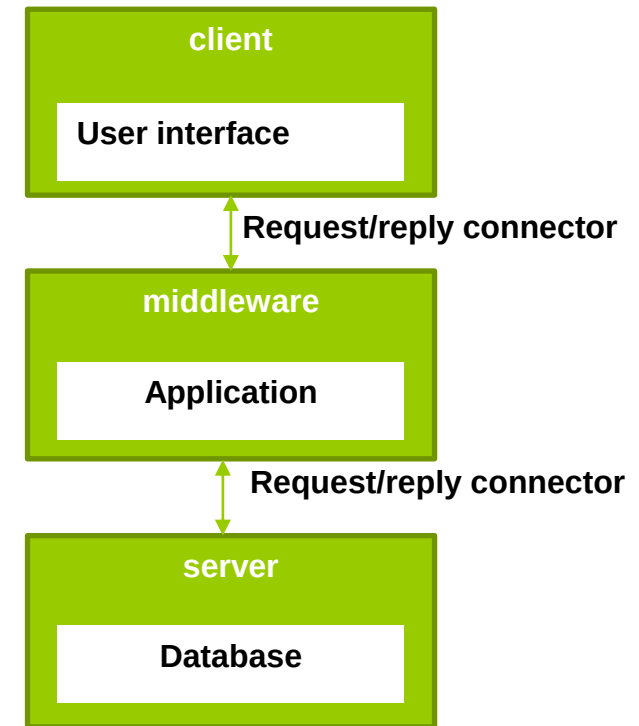
- Application Tier
 - Support the development of business logic and application-specific processing

■ Server

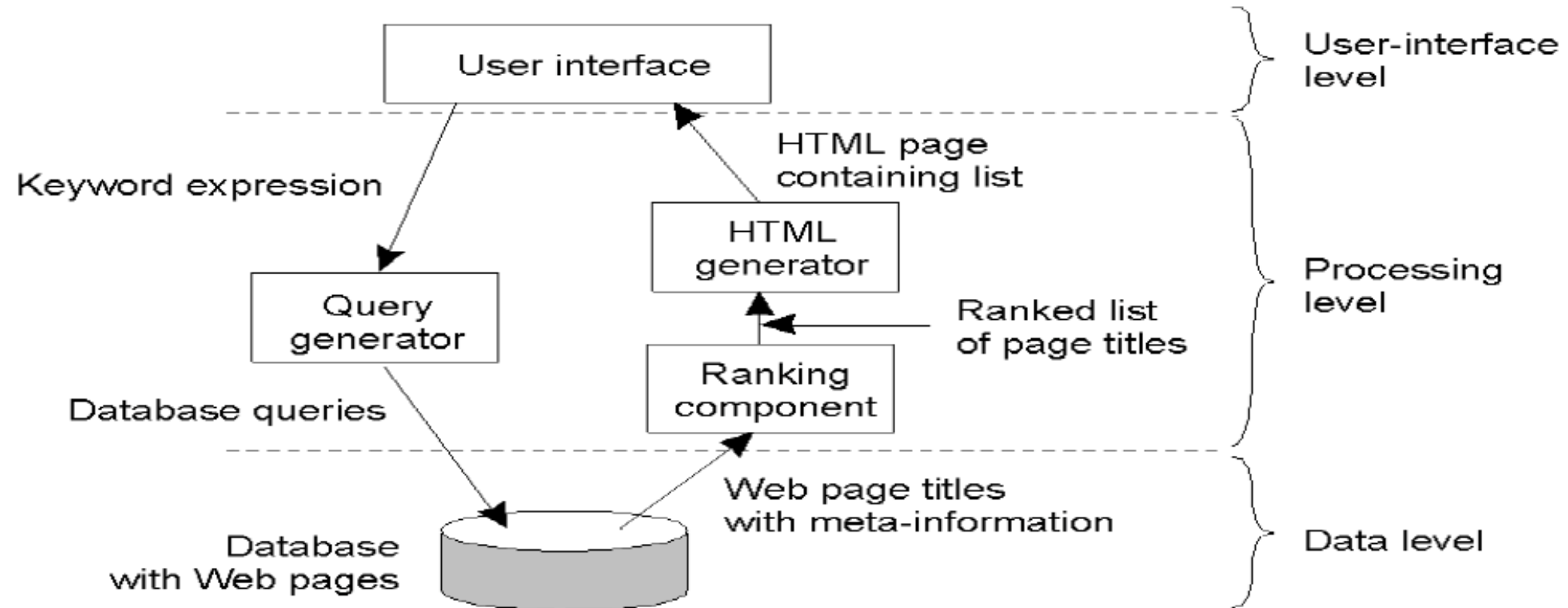
- Data management Tier

■ Request/reply connector

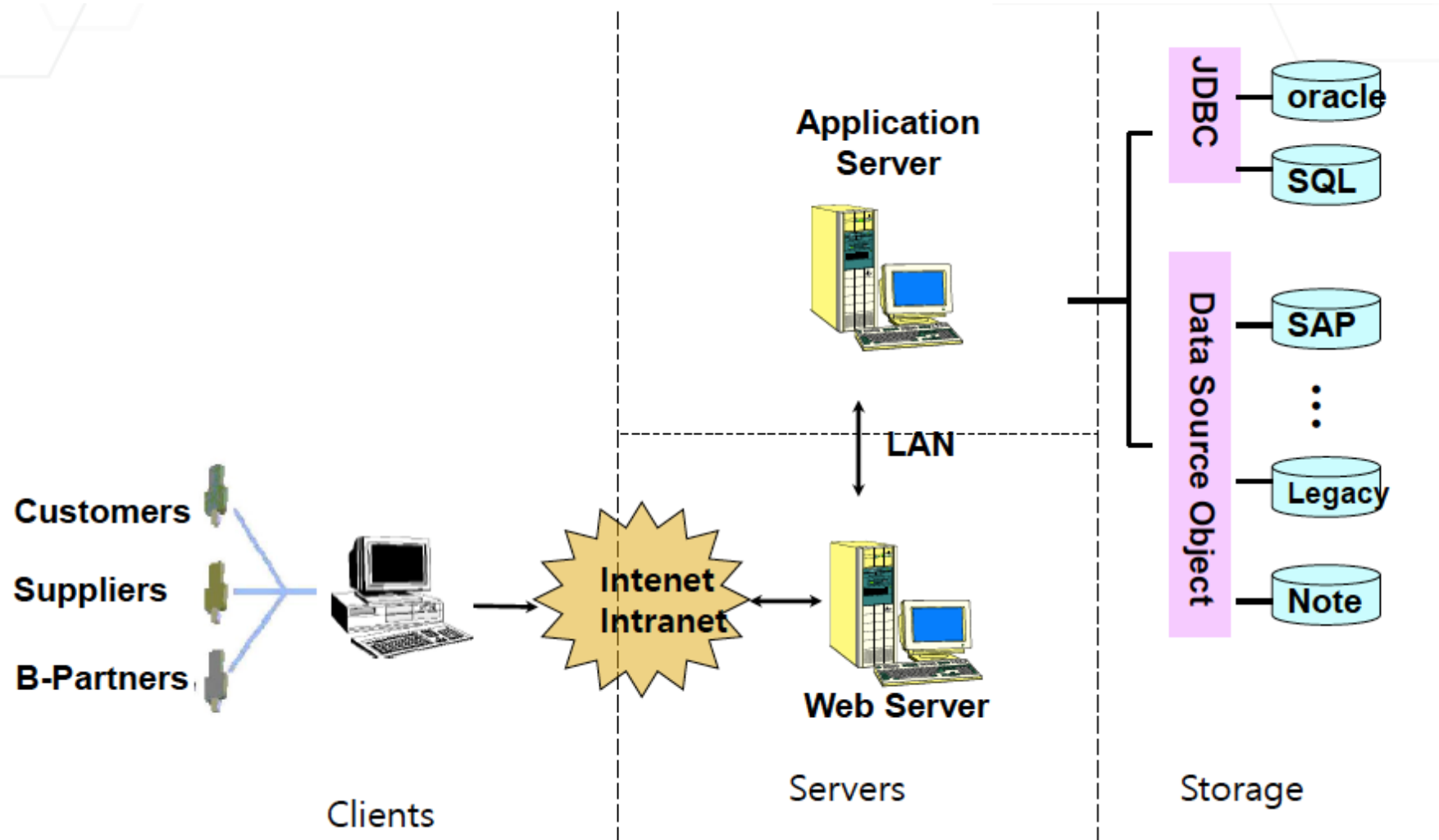
□ Relations



3-Tier Architecture Example



4-tier Architecture Example



Consequences

□ Benefits

■ Reusability and Modifiability.

- Any business related changes are made only to the middle tier.

■ The middle tier can provide multithreading supports for scalability and performance.

■ Enhancement of Security

- By separating the presentation, application, and data tiers, sensitive business logic and data can be protected from direct access by end-users.

□ Weaknesses

■ Complexity and overhead

■ testability due to the general lack of testing tools.

■ Adding multiple servers in the system makes the server reliability and availability even more critical.

Broker Architecture Style

- ❑ The broker architecture is a middleware architecture used in distributed computing **to coordinate and facilitate communication between registered servers and clients.**
 - Client and server components are **decoupled through the use of a Broker.**
 - A **Broker** component is **responsible for coordinating communication.**
 - ❑ Locating a proper server
 - ❑ Forwarding and dispatching requests
 - ❑ Sending responses back to clients

Broker Architecture Style

□ Elements

■ Servers

- Register with broker
- Present method interfaces to clients

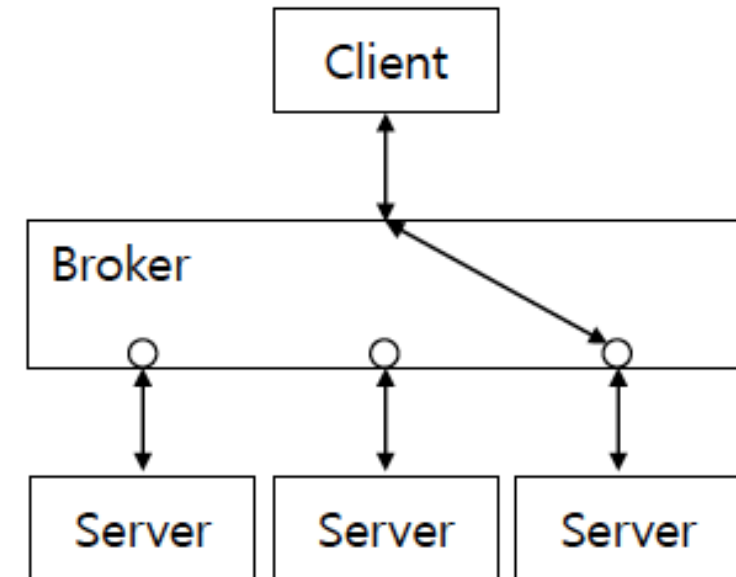
■ Clients

- Access server's methods via broker
- Uses same form to call server's methods

■ Broker's Tasks

- Locating appropriate server
- Forwarding requests to server
- Transmitting results and exceptions to client

□ Relations

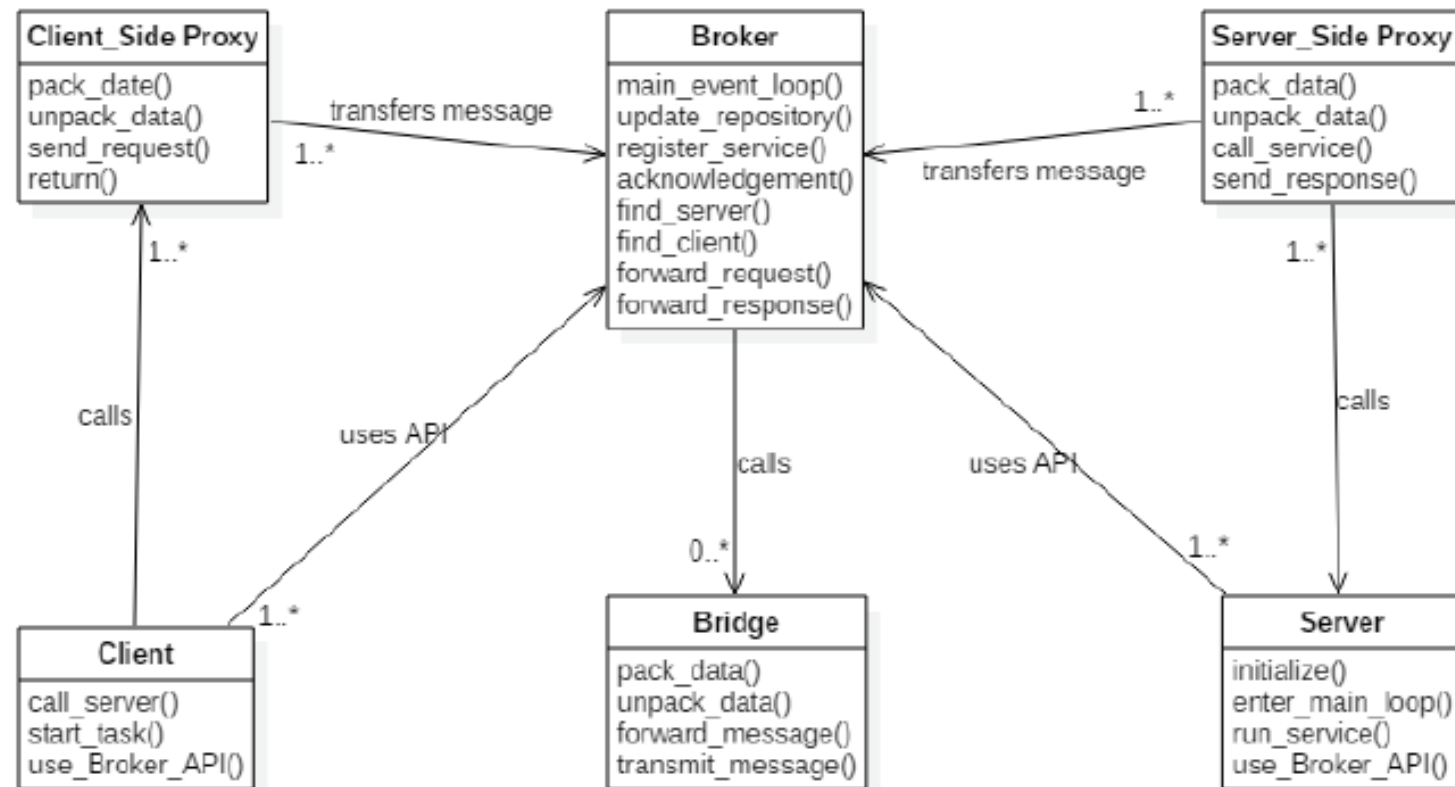


□ Constraints

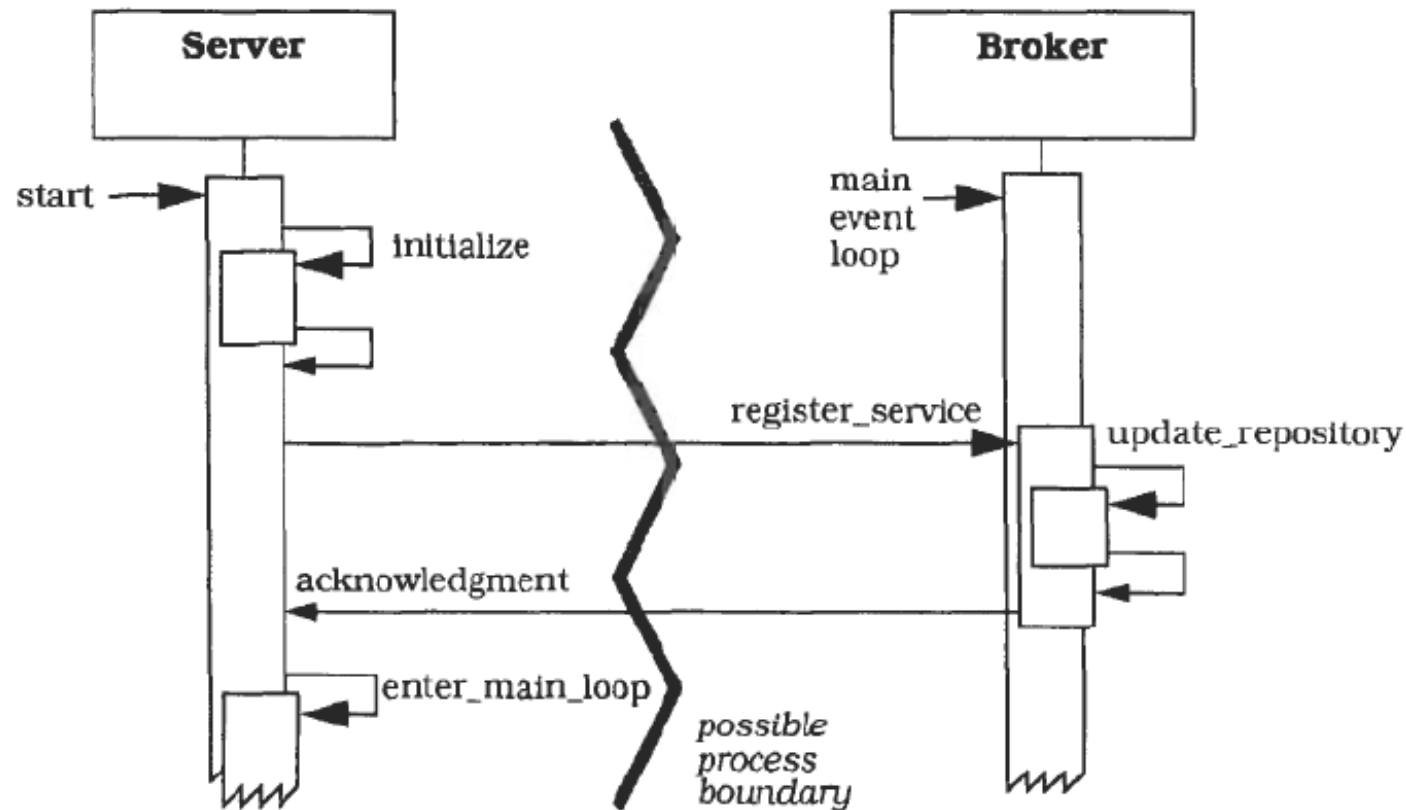
- No direct interaction between clients and servers

Examples

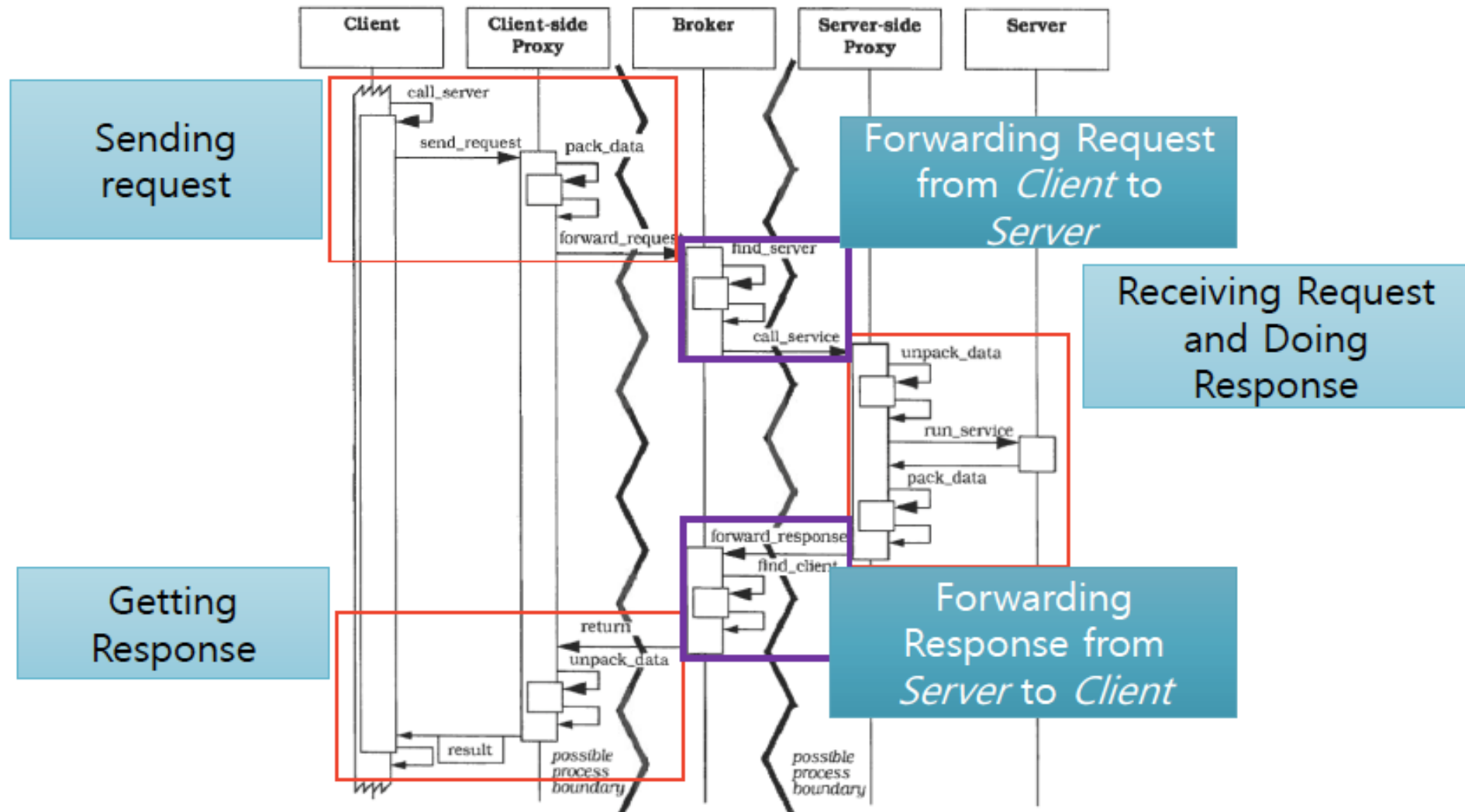
□ CORBA Communication Architecture



Dynamic Scenario of Server Registration

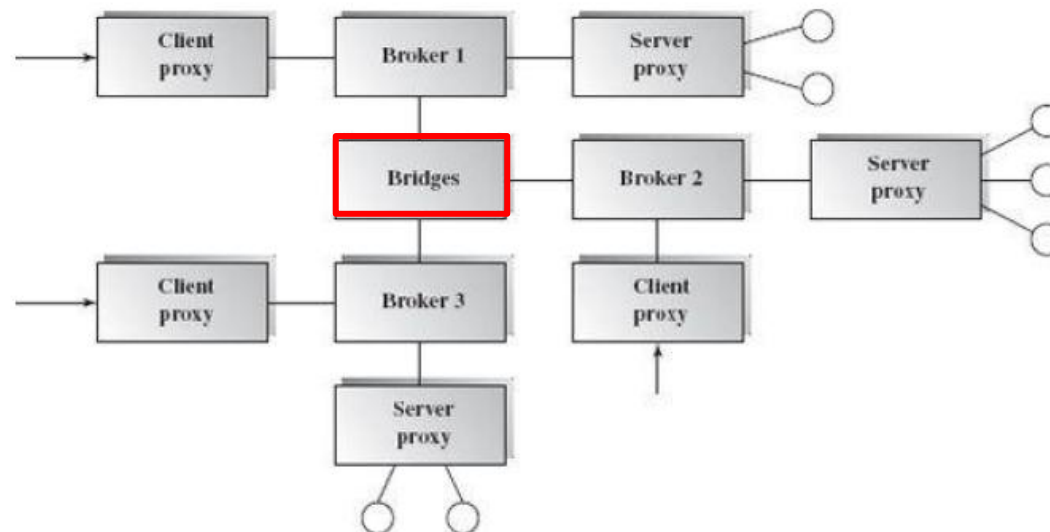


Dynamic Scenario of Service Process



Bridge

- ❑ A **Bridge** can connect two different networks based on different communication protocols
- ❑ It **encapsulates underlying network detail implementation** and mediate different brokers.
 - It can translate requests and parameters from one format to another



Consequences

□ Benefits

■ Location transparency

- As the broker is responsible for locating a server by using a unique identifier, **clients do not need to know where servers are located**

■ Changeability and extensibility of components

- **If servers change but their interfaces remain the same, it has no functional impact on clients**
- Modifying the internal implementation of the broker, but not the APIs it provides, has no effect on clients and servers other than performance changes

■ Portability of a Broker system

- **The Broker system hides operating system and network system details** from clients and servers by using indirection layers such as APIs, proxies and bridges.

Consequences

□ Benefits

■ Interoperability via broker bridges

- Different Broker systems may interoperate if they understand a common protocol for the exchange of messages
- This protocol is implemented and handled by bridges

■ Reusability of existing services

- When building new client applications, you can base the functionality of your application on existing services.

■ Runtime changes of server components (add or remove server components on the fly)

Consequences

❑ Weaknesses

■ Inefficiency

- ❑ due to the indirection through the proxies and the broker component

■ Low fault-tolerance

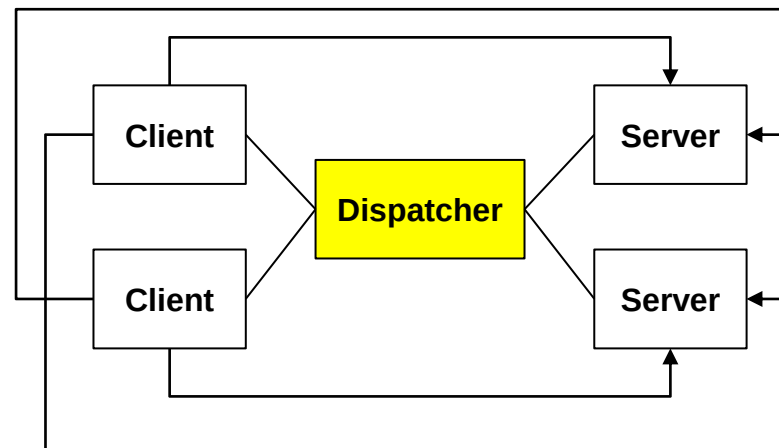
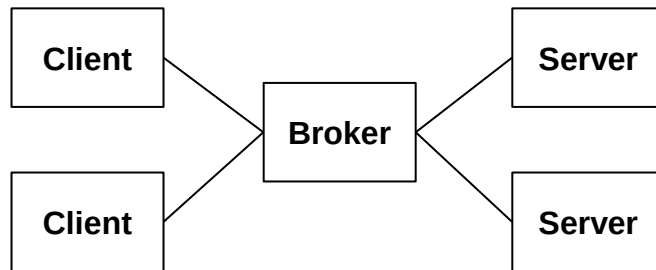
- ❑ Suppose that a server or a broker fails during program execution. All the applications that depend on the server or broker are unable to continue successfully.
- ❑ You can increase reliability through replication of components.

■ Difficulty in Testing and Debugging

- ❑ Debugging and testing a Broker system is a tedious job because of the many components involved.

Dispatcher Style

- ❑ The **Dispatcher** style introduces an intermediate layer between clients and servers, the **dispatcher** component.
 - A lightweight version of the **Direct Communication Broker** variant
- ❑ Clients rely on the dispatcher to locate a particular server and to establish a communication link with the server.



Dispatcher Style

□ Elements

■ Client

- Requests server connection from the dispatcher
- Invokes services of servers

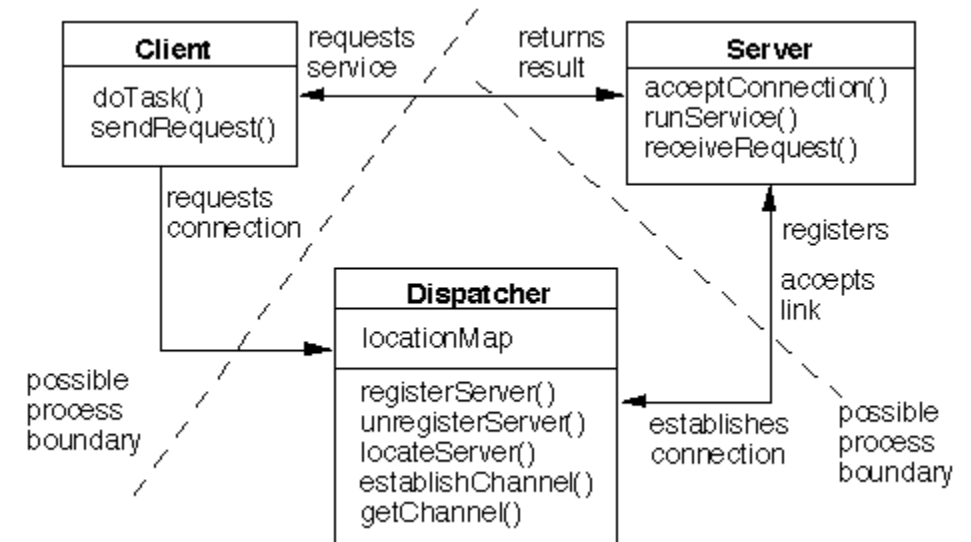
■ Server

- Provides services to clients
- Registers itself with the dispatcher

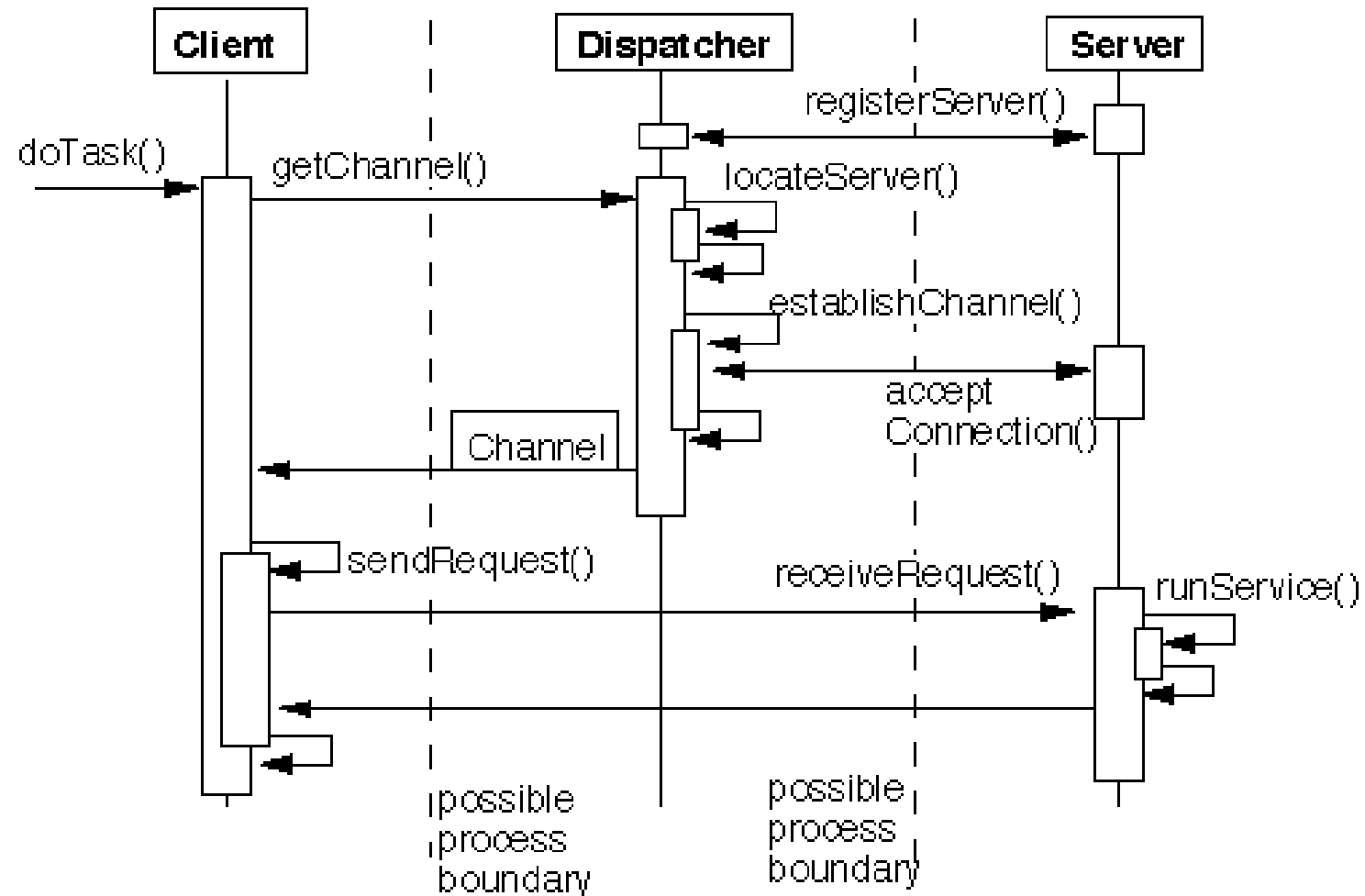
■ Dispatcher

- Establishes communication channels
- Locates servers
- Register servers
- Maintains a map of server locations

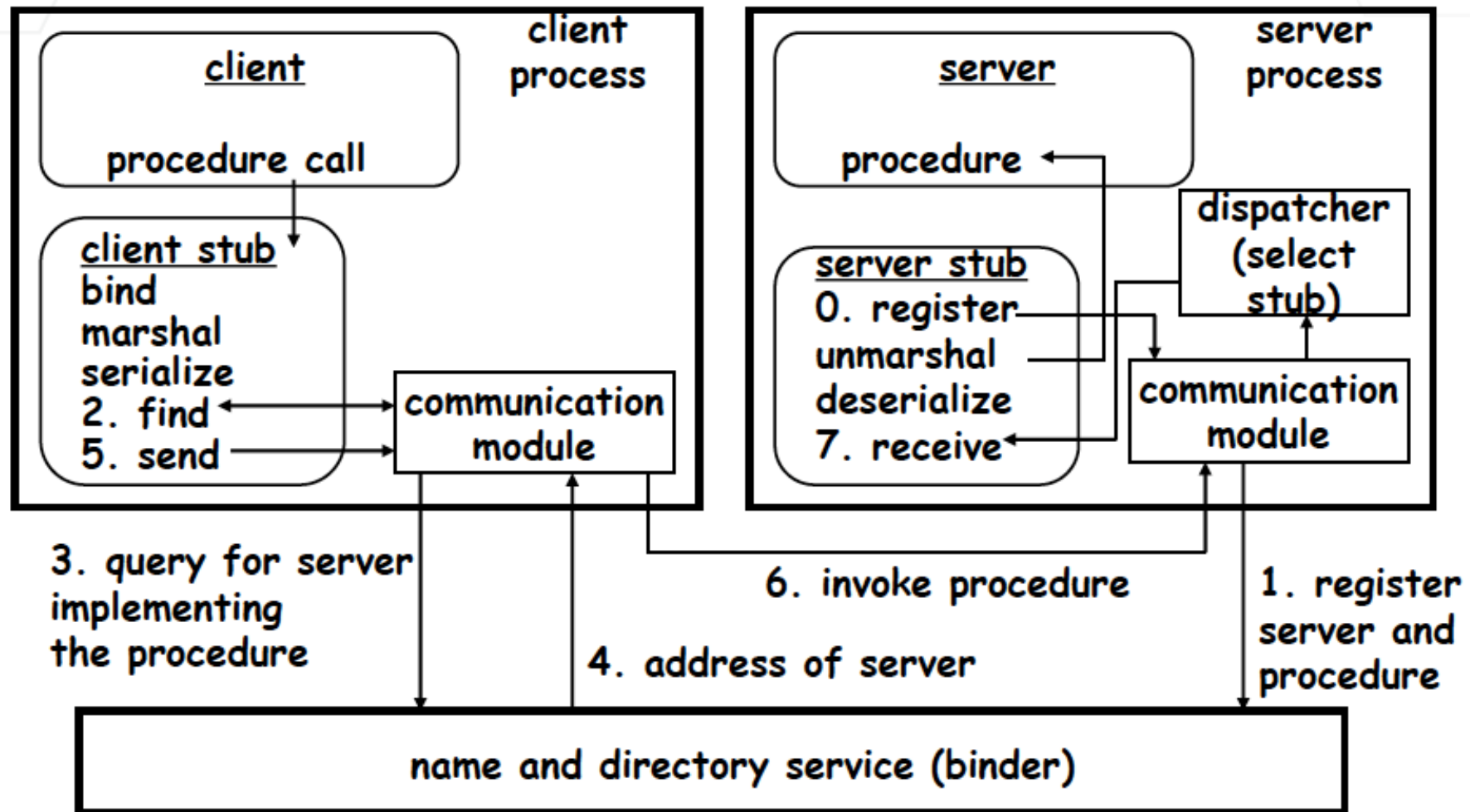
□ Relations



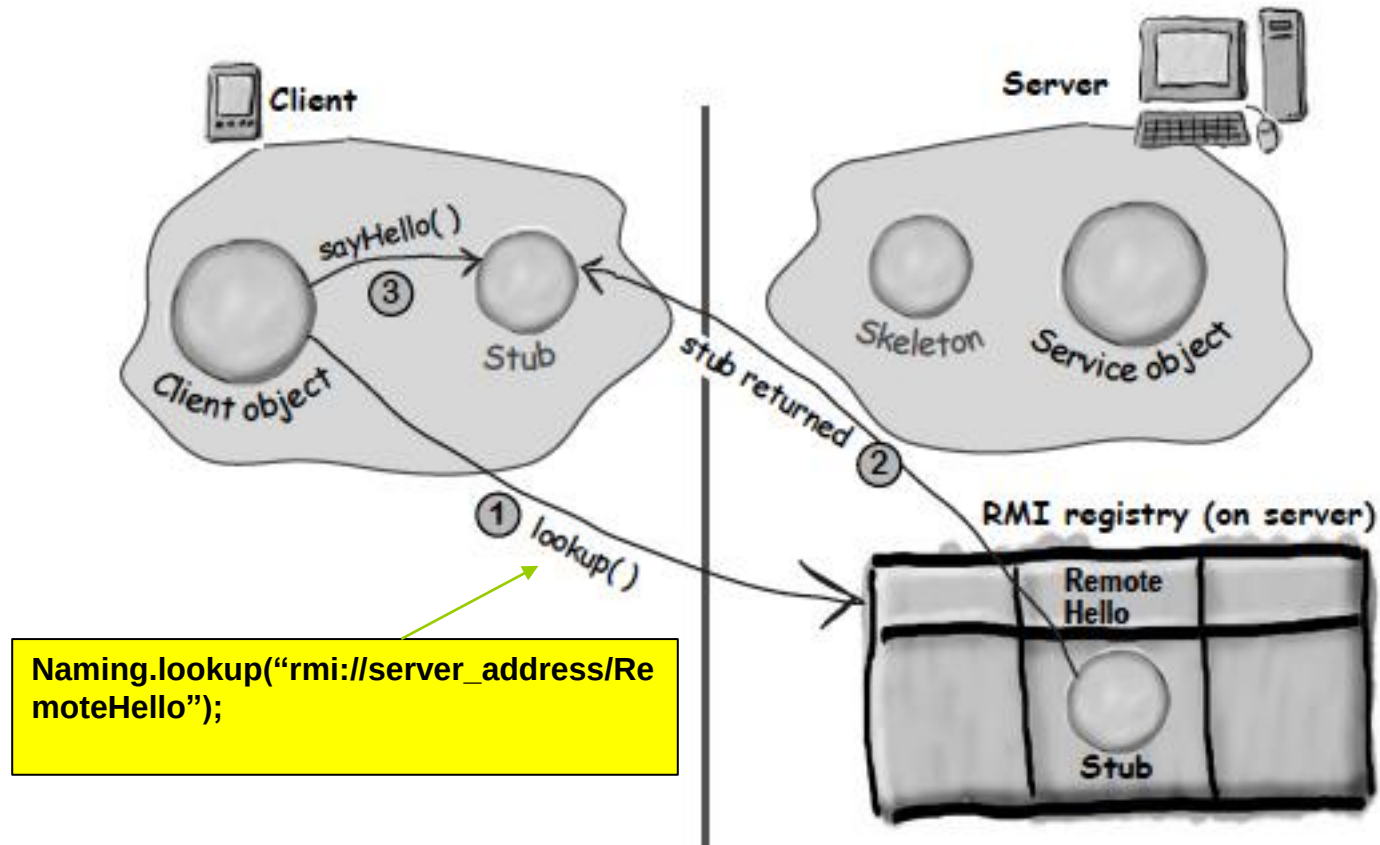
Dynamics



Example - RPC

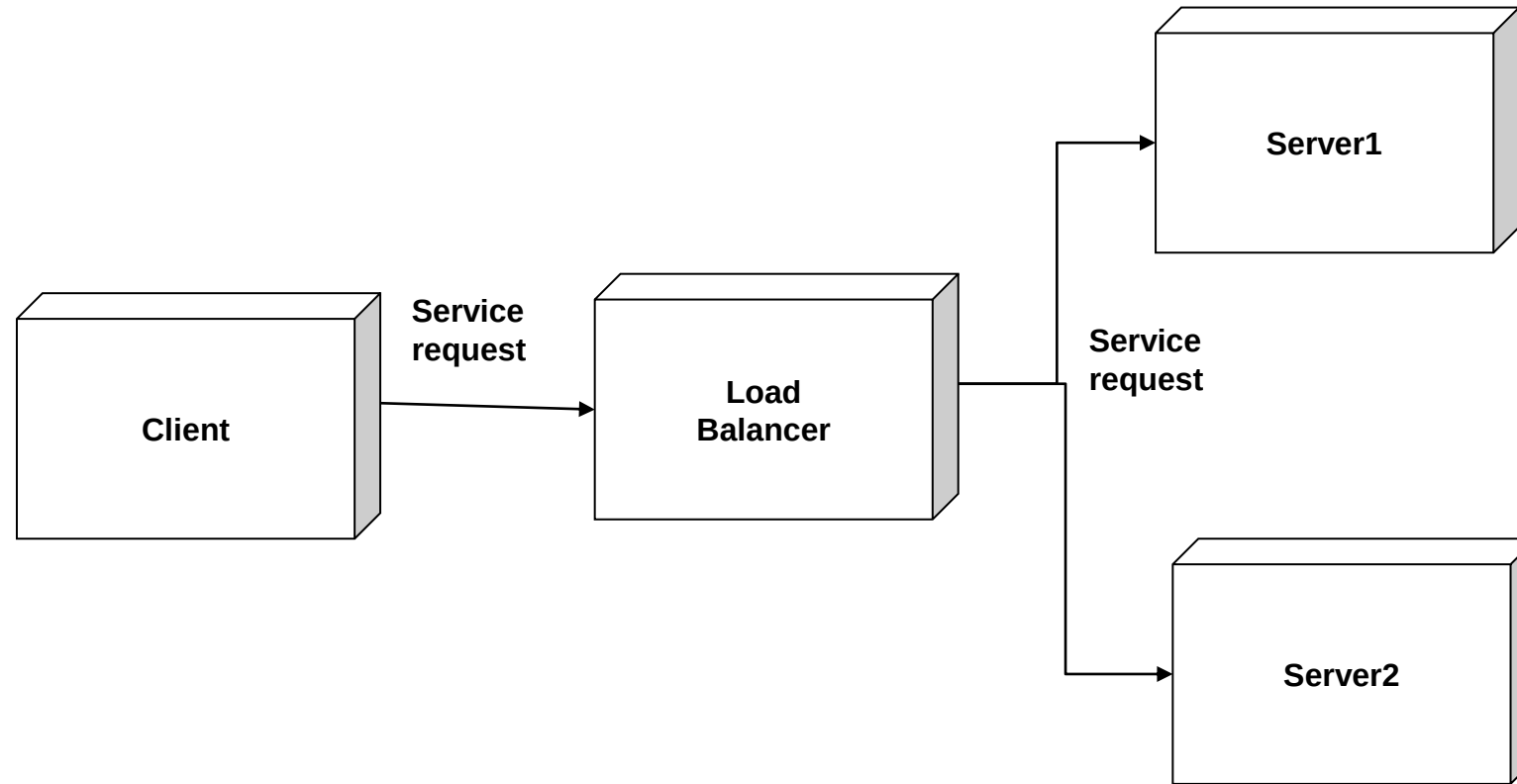


Example – Java RMI



```
Naming.lookup("rmi://server_address/RemoteHello");
```

Example – Load Balancer



Consequences

□ Benefits

■ Exchangeability of servers

- Servers can be added or modified without modifications to the dispatcher or clients

■ Location and migration transparency

- Servers can be dynamically migrated to other machines as long as a client is not connected to the server

■ Re-configuration

- Decisions about where and which servers are part of the system can be decided at start-up time or later

■ Fault Tolerance

- A new server can be activated on a different machine when there is a network or server failure

□ Weaknesses

■ Dispatcher overhead

■ Sensitivity to change in the interfaces of the dispatcher

Service Oriented Architecture

□ What is a service?

- A business functionality that is
 - well-defined
 - Self-contained
 - Independent from other services
 - Published and available to be used via standard programming interface

□ Service-oriented architecture

- a style of software design where services are provided to the other components by application components, through a communication protocol over a network.
- Its principles are independent of vendors and other technologies.

SOA Style

□ Element

■ Service provider

- Provides one or more services through published interfaces
- Registers the service information to the **intermediary component** (service registry)

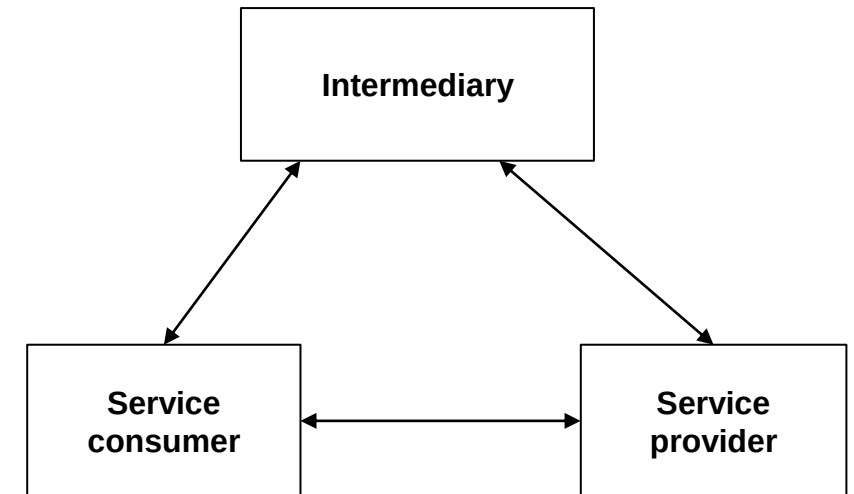
■ Intermediary component

- Makes the information regarding the services available to any potential **service consumer**.
- Can route and transform messages between **service providers** and **consumers**.
- Coordinates the interactions between **service consumers** and **providers**

■ Service consumer

- Invokes services directly or through an **intermediary**.

□ Relations

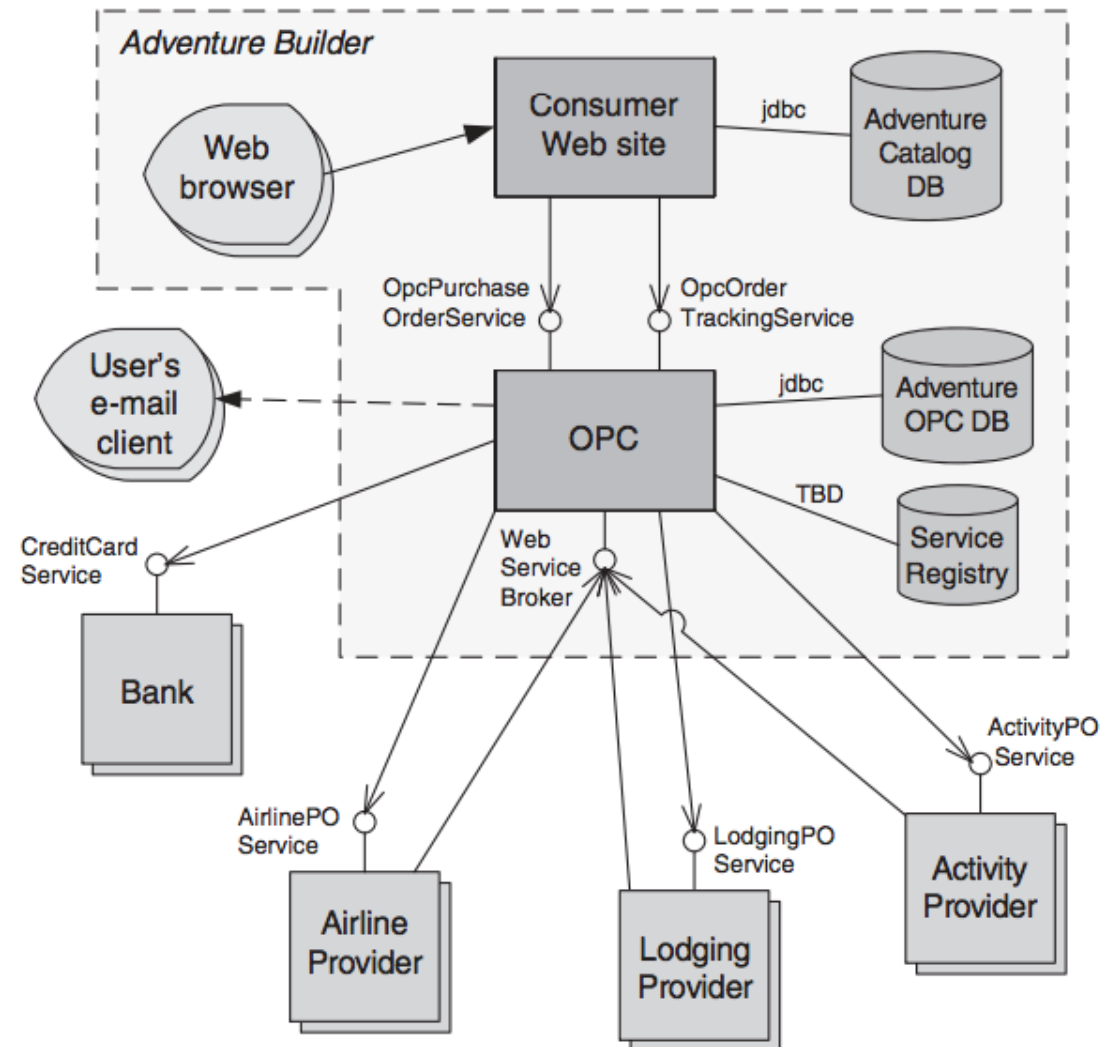
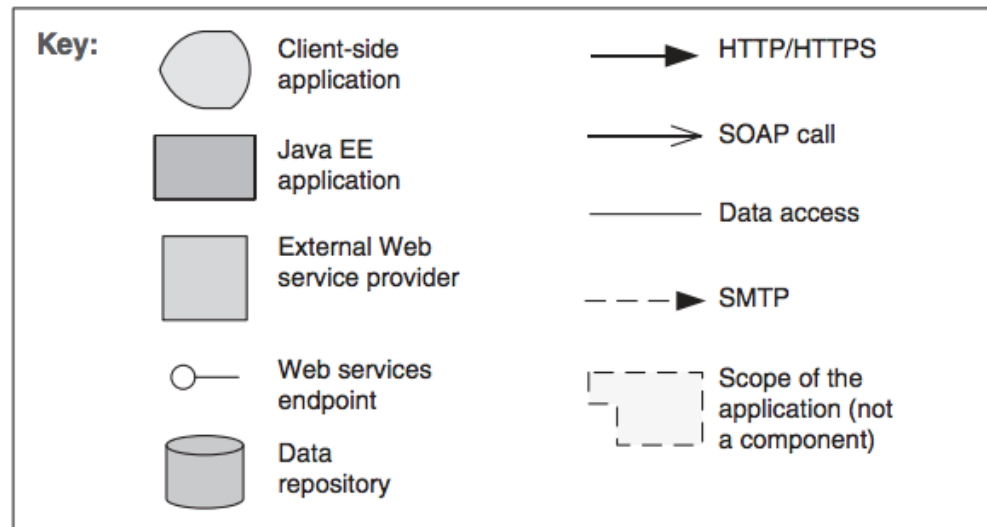


□ Constraints

- Platform independence
- Stateless services
- Standardized communication

Example

The Adventure Builder system interacts with external service providers to construct the vacation, and with bank services to process payment. The central OPC (Order Processing Center) component coordinates the interaction with internal and external service consumers and providers



Consequences

□ Benefits

■ Modifiability and Evolution

- Each service component is independent due to the stateless service feature.
- Any change in the implementation of a service will not affect its application as long as the exposed interface is not changed. This makes SOA software much easier to evolve and update.

■ Interoperability

- Technically, any client or service can access other services regardless of their platform, technology, vendors, or language implementations.

Consequences

□ Benefits

■ Reusability

- Any service can be reused by any other service. Because clients of a service need only to know its public interfaces, service composition and integration become much easier. This makes SOA-based business application development much more efficient in terms of time and cost.

■ Scalability and Workload Distribution

- Each service can run in a different process and even in a different computer
- If it is identified that the server utilization is reaching its max, it is possible to get another computer and at least split the charge by putting some services in one computer and the remaining services on the new computer.

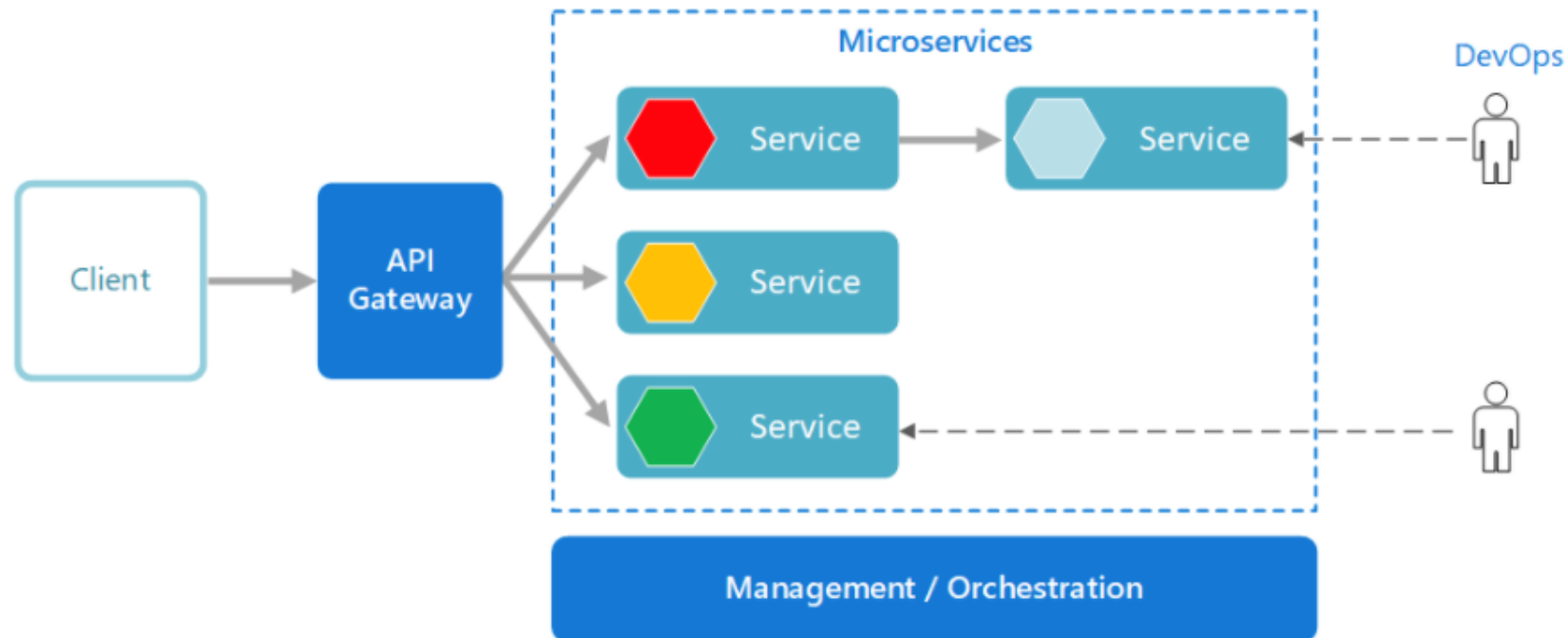
Consequences

□ Weaknesses

- SOA-based systems are typically complex to build.
- You don't control the evolution of independent services.
- There is a performance overhead associated with the middleware, and services may be performance bottlenecks, and typically do not provide performance guarantees.

Microservice Architecture

- A microservices architecture consists of **a collection of small, autonomous services**.
 - Each service is **self-contained** and implements **a single business capability**.



Elements of Microservice style

❑ **Microservice**

- Each service is a separate codebase, which can be managed by a small development team.
- Services can be deployed independently. A team can update an existing service without rebuilding and redeploying the entire application.
- Services are responsible for persisting their own data or external state. This differs from the traditional model, where a separate data layer handles data persistence.
- Services communicate with each other by using well-defined APIs. Internal implementation details of each service are hidden from other services.
- Services don't need to share the same technology stack, libraries, or frameworks.

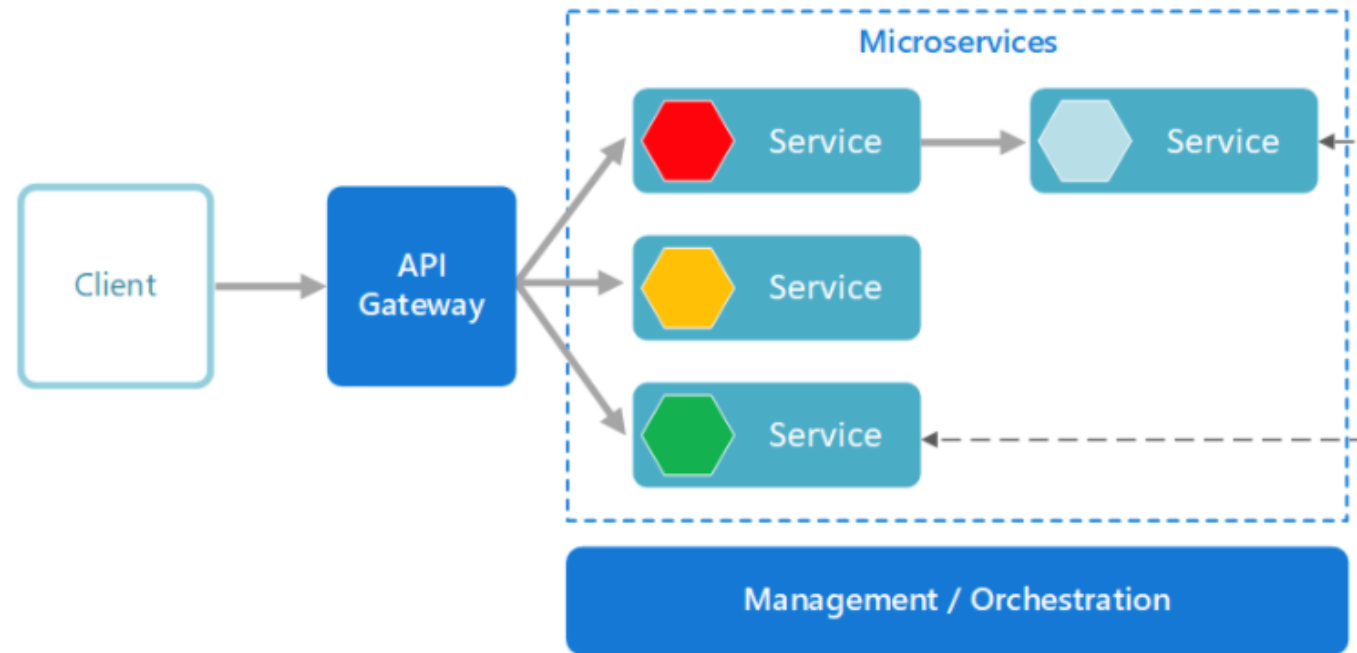
Elements of Microservice style

□ API Gateway

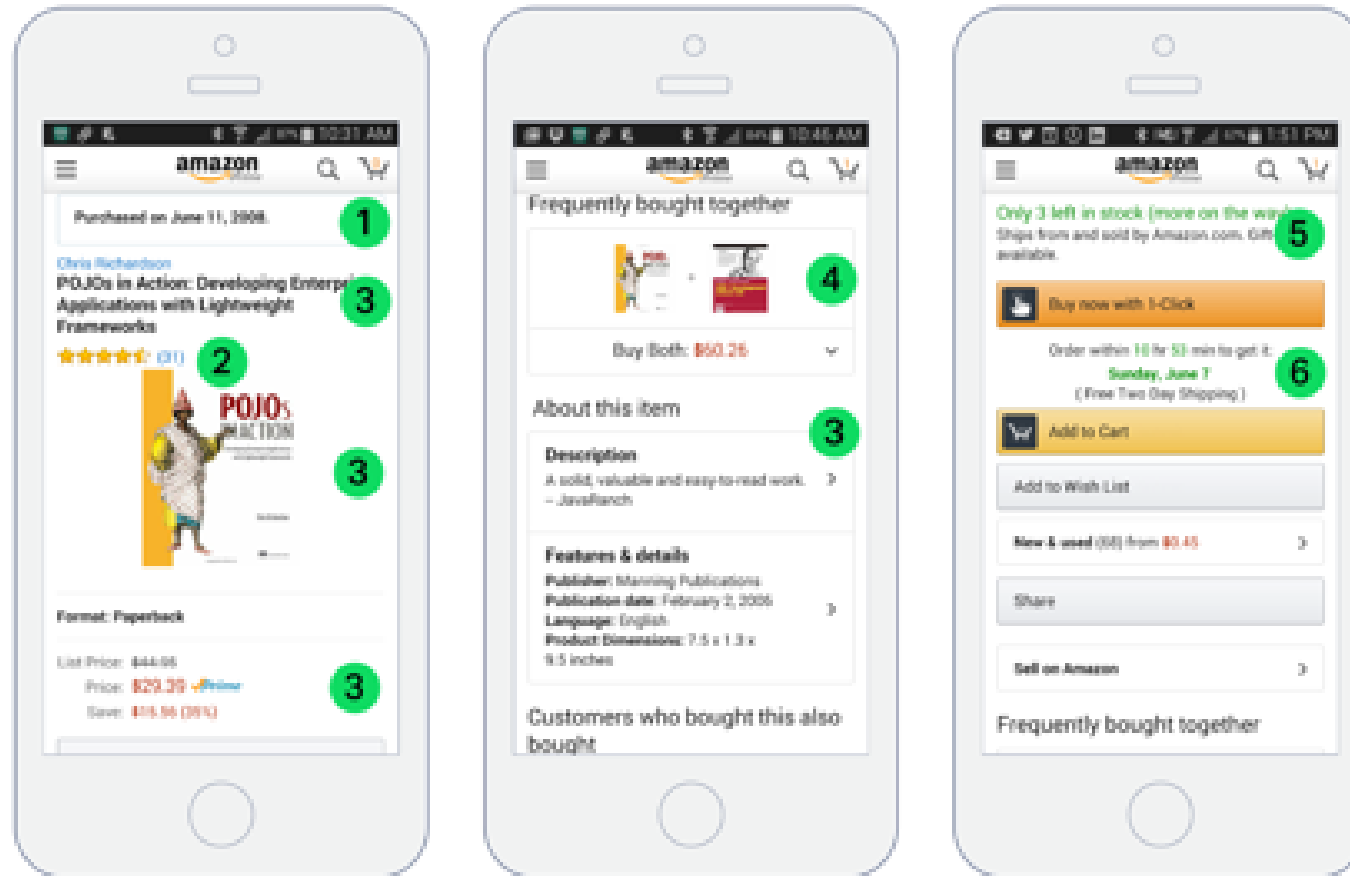
- the entry point for clients.
- Clients call the API gateway, which forwards the call to the appropriate services on the back end.

□ Management/orchestration

- This component is responsible for
 - placing services on nodes,
 - identifying failures,
 - rebalancing services across nodes, and so forth.

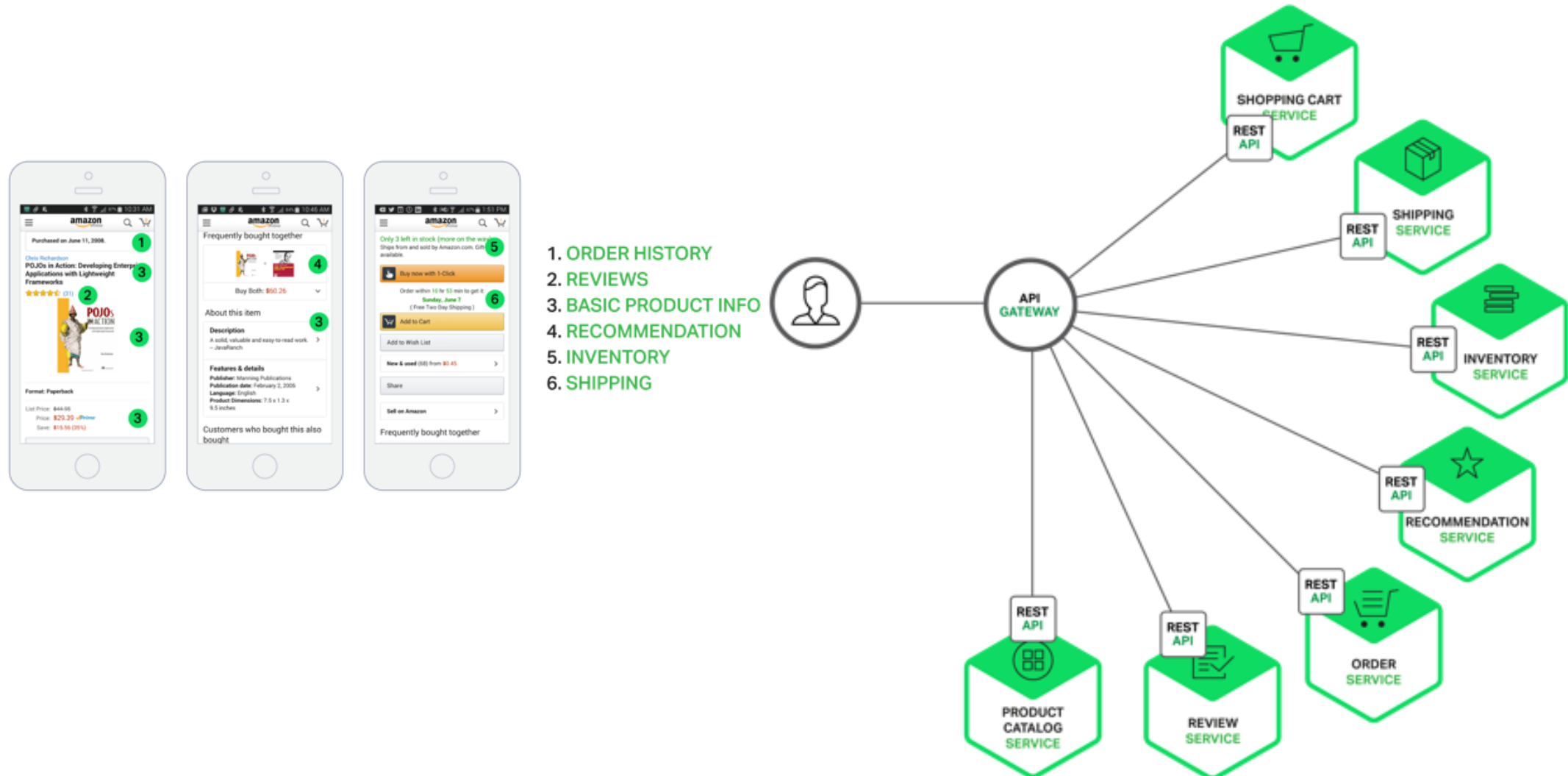


Example: RESTful Web Service



1. ORDER HISTORY
2. REVIEWS
3. BASIC PRODUCT INFO
4. RECOMMENDATION
5. INVENTORY
6. SHIPPING

Example: RESTful Web Service



Consequences

□ Benefits

■ Agility.

- Because microservices are deployed independently, it's **easier to manage bug fixes and feature releases**. You can update a service without redeploying the entire application and roll back an update if something goes wrong.

■ Easy to add new features.

- By not sharing code or data stores, **a microservices architecture minimizes dependencies**, and that **makes it easier to add new features**.

Consequences

□ Benefits

■ Fault isolation.

- If an individual microservice becomes unavailable, it won't disrupt the entire application, as long as any upstream microservices are designed to handle faults correctly.

■ Scalability.

- Services can be scaled independently, letting you scale out subsystems that require more resources, without scaling out the entire application.

■ Data isolation.

- It is much easier to perform schema updates, because only a single microservice is affected.

Consequences

❑ Weaknesses

■ Complexity.

- ❑ Each service is simpler, but the **entire system as a whole is more complex.**

■ Testing.

- ❑ If a service relies on other dependent services, it is challenging to test services based on service dependencies without tool support.

■ Lack of governance.

- ❑ It is advantageous that **services don't need to share the same technology stack, libraries, or frameworks**, but it can also lead to problems. **You may end up with so many different languages and frameworks that the application becomes hard to maintain.**

Consequences

❑ Weaknesses

■ Network congestion and latency.

- ❑ The use of many small, granular services can result in **more interservice communication**. If the chain of service dependencies gets too long, the latency can become a problem.

■ Data integrity.

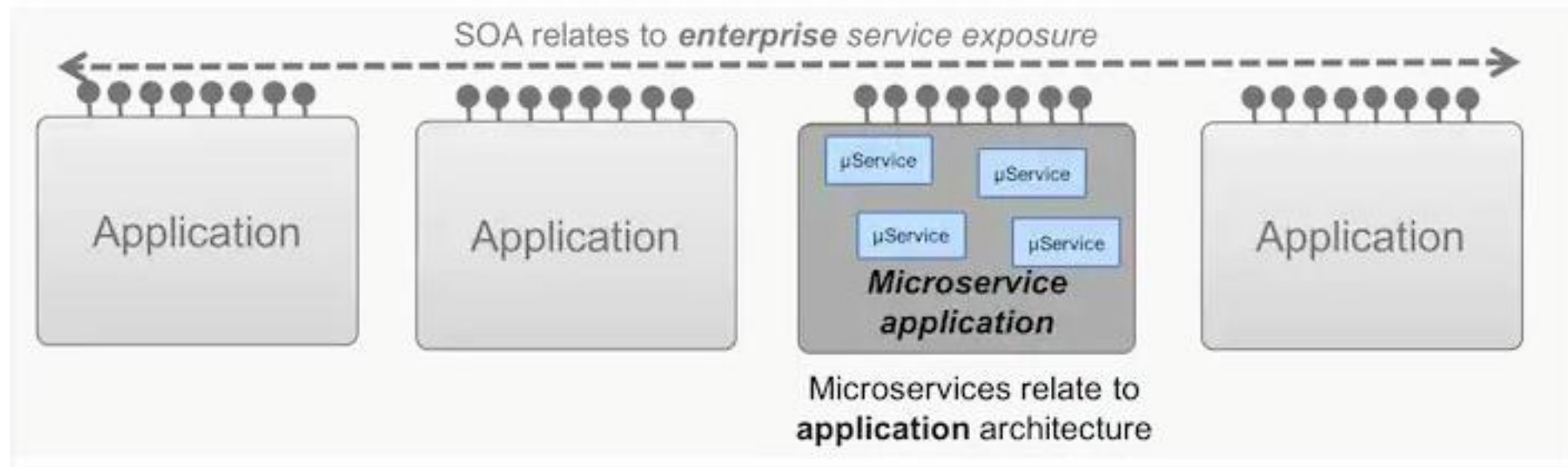
- ❑ With each microservice responsible for its own data persistence. As a result, **data consistency can be a challenge**.

■ Versioning.

- ❑ **Updates to a service must not break services that depend on it**. Multiple services could be updated at any given time, so without careful design, you might have problems with backward or forward compatibility.

SOA vs. Microservice Architecture

- The main distinction between the two approaches comes down to **scope**. To put it simply, **service-oriented architecture (SOA)** has an **enterprise scope**, while **the microservices architecture** has an **application scope**.



Architecture Styles

DEPLOYMENT ARCHITECTURE

Deployment Architecture

❑ Definition

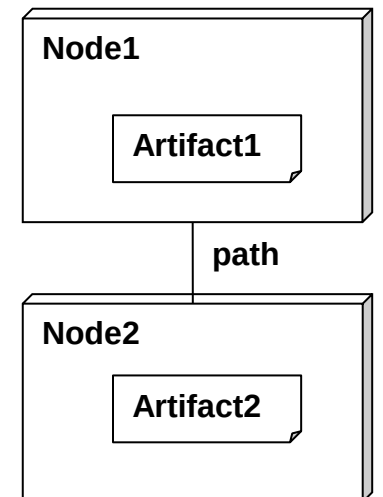
- **Deployment Architecture** specifies **how and where the system runs** —
 - ❑ defining the physical and virtual environment,
 - ❑ the distribution of software artifacts, and
 - ❑ the communication between them to achieve desired performance, scalability, and reliability

❑ Elements

- **Deployment Node** - The physical or logical environment where an artifact is deployed and executed
- **Deployable Artifact** - A deployable software package

❑ Relations

- **Communication Path** - Network connections among deployment nodes
- **Mapping** - Assignment of artifacts to nodes



Deployment Architecture

□ Types of classification

■ Container-Based Style

- Lightweight, portable runtime environments

■ Cloud-Based Style

- Fully managed distributed deployment

■ Edge-Based Style

- Compute close to data sources for low latency

Container-Based Style

❑ What is Container?

- Packages an application and its dependencies
- Runs in an isolated user space
- Provides consistent runtime behavior across machines
- Starts quickly and uses fewer resources than virtual machines

❑ **Container-Based Deployment Architecture Style** is

- a style that **packages applications into containers** and deploys, scales, and operates them through a container orchestration platform (e.g. Kubernetes, ECS).

Container-Based Style

□ Elements

■ Deployment Node (Container Host)

- the physical or virtual machine where containers run, equipped with a container runtime such as the Docker engine

■ Deployable Unit (Container Instance)

- the **operational runtime instance** that an orchestrator places and runs on a deployment node.

■ Deployable Artifact (Container Image)

- the **packaged specification or image** that defines everything needed to run a container.

■ Orchestrator

- the control system that schedules, deploys, scales, and manages runtime units across deployment nodes

■ Container Registry

- A centralized storage for saving, managing, and sharing container images (e.g., Docker Hub, GCR, ECR)

Container-Based Style

□ Relations

■ Deployment Relation

- Container Registry → Container Host/Orchestrator
 - The Container Image stored in the registry is requested by the orchestrator or downloaded locally by the host's runtime and executed as a Container.

■ Execution Relation

- Container Instance → Container Host:
 - The Container instance runs in an isolated environment while sharing the operating system kernel of the Container Host.

■ Management Relation:

- Orchestrator → Container Instance
 - The Orchestration Tool manages the lifecycle (creation, scaling, recovery, termination) of multiple Containers and handles networking and service discovery between them.

Container-Based Style

❑ **Constraint**

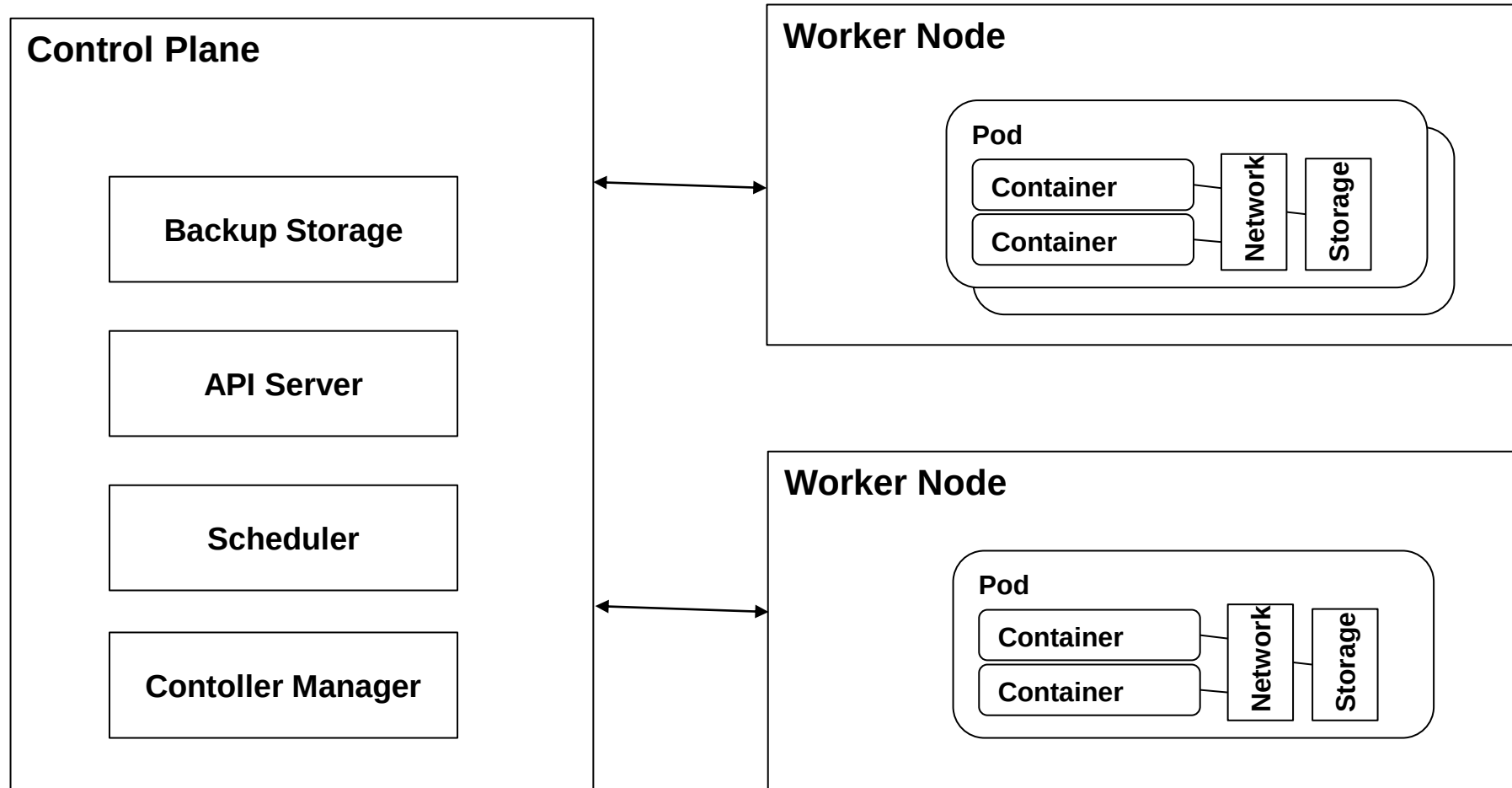
■ **Immutability**

- ❑ A Container Image that has been deployed is not modified. For updates, instead of changing the existing container, a new image must be built and the existing container must be replaced with a new one (rolling update)

■ **Statelessness Design (Preferred)**

- ❑ The Container itself should be designed not to store state. Persistent data must be stored externally in Volumes, databases (DB), or external storage services. This enhances resilience by allowing containers to be freely discarded and redeployed

Example : Kubernetes Architecture



Comparison

	Kubernetes	ECS	Docker
Deployment Node	Worker Node	EC2/Fargate Node	Docker Host
Deployable Unit	Pod	Task	Container
Deployable Artifact	Pod Spec + Image	Task Definition + Image	Image
Orchestrator	Kubernetes Control Plane	ECS Control Plane	X
Container Registry	Docker Hub, GCR/ACR/ECR	Amazon ECR (Elastic Container Registry)	Docker Hub

Consequences

□ **Benefits**

■ **Portability & Environmental Consistency**

- Solves the "It works on my machine..." problem
- They can run anywhere with a container runtime—on-premises, AWS, Azure, GCP, etc.—making infrastructure migration easier

■ **Efficiency & Lightweight**

- (Low Overhead) Unlike Virtual Machines (VMs), containers do not include an entire guest OS
- (Cost Reduction) Infrastructure costs are reduced because more applications can be run with fewer hardware resources

Consequences

□ Benefits

■ Rapid Deployment & Scalability

- **Accelerated CI/CD:** Container images standardize the unit of deployment.
- **Elastic Scaling:** With orchestration tools like Kubernetes, application instances can be quickly and elastically scaled

■ Isolation & Modularity

- **Prevents Dependency Conflicts:** Each container runs in an isolated environment, meaning libraries within one container do not affect others.
- **Microservices Support:** Containers are the ideal unit for encapsulating and independently deploying each service in a microservices architecture

Consequences

❑ Weakness

■ OS Kernel Dependency and Compatibility Limits

- ❑ Containers share the host OS kernel, which restricts cross-kernel compatibility (e.g., Windows containers cannot run on Linux hosts)

■ Increased Operational Complexity

- ❑ Large-scale container deployments require complex orchestration, networking, service discovery, monitoring, and security configurations

■ Security and Isolation Concerns

- ❑ Because containers share the same kernel, isolation is weaker than in full VMs, requiring careful configuration and additional hardening

■ Orchestrator Learning Curve

- ❑ Systems like Kubernetes introduce significant learning and operational complexity, especially for teams transitioning from monolithic or VM-based environments

Cloud-Based Style

□ **Cloud-Based Deployment Architecture Style**

- a Deployment Architectural Style that defines how to construct and deploy a software system by leveraging services within a cloud computing environment.
- focuses on **placing software artifacts onto cloud nodes** in a manner that maximizes cloud characteristics such as Elasticity, Scalability, and High Availability.

Cloud-Based Style

□ Elements

■ Software Artifacts

- All software components to be deployed, including application code, libraries, configuration files, and database schemas

■ Deployment Nodes

- **Cloud Resources:** Any computing, storage, and networking resources provided by a cloud vendor, such as IaaS (Virtual Machines/Containers), PaaS (Managed Services like App Services/Lambda/Functions), and FaaS (Serverless Functions).
- **Managed Services:** Cloud-managed services such as Databases (RDS, DynamoDB, Cosmos DB), Messaging Queues (SQS, Kafka), Load Balancers (ALB, ELB), and API Gateways.

■ Infrastructure-as-Code (IaC) Artifacts

- Scripts or configuration files (e.g., Terraform, CloudFormation, ARM Templates) that define and provision the infrastructure, ensuring automation and reproducibility of deployment

Cloud-Based Style

□ Relations

■ Deployment Relation

- **Software Artifacts** are deployed onto or consume **Deployment Nodes** (Cloud Resources and Managed Services).
- For example, a microservice artifact is deployed to nodes within a container orchestration platform (EKS/AKS/GKE)

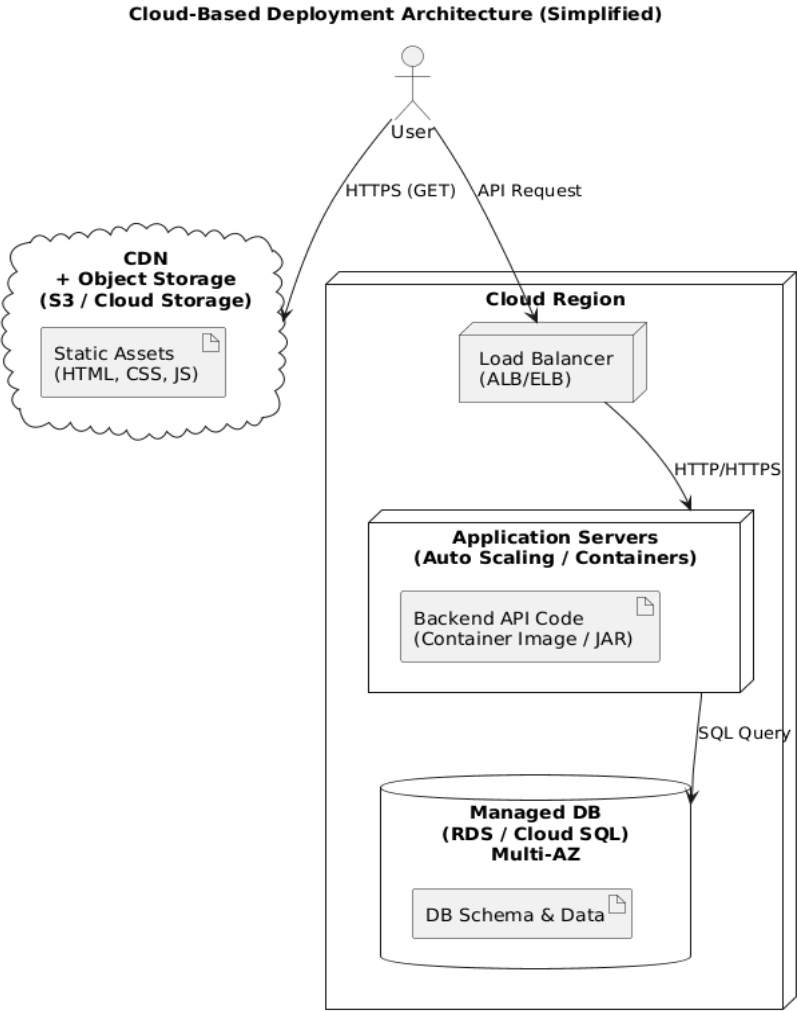
■ Provisioning Relation

- **IaC Artifacts** create (Provision) and configure **Deployment Nodes**.
- **IaC Artifacts** can allow multiple **Cloud Resources** nodes (e.g., web server instances) to scale in or out based on demand

Example: Cloud-Based Architecture

고가용성 3-Tier 웹 애플리케이션 (IaaS/PaaS Hybrid)

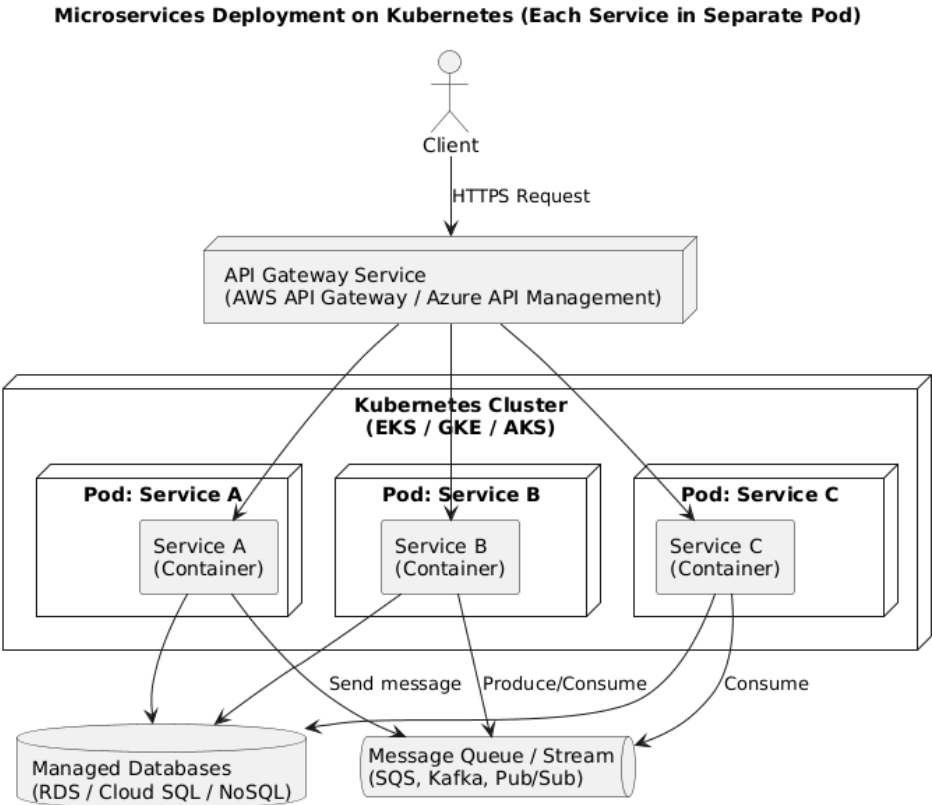
요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Presentation (프론트엔드)	웹 자산(HTML, CSS, JS)	CDN (Content Delivery Network) 및 S3/Cloud Storage에 배포.
Application (백엔드)	웹 서버, API 코드	Auto-Scaling Group 내의 VM (EC2/Compute Engine) 또는 Managed Container (EKS/GKE/ECS)에 배포. 여러 가용 영역(Availability Zones, AZs)에 분산.
Data (데이터베이스)	데이터베이스 스키마 및 데이터	Managed DB Service (RDS/Cloud SQL/Cosmos DB)를 Multi-AZ 구성으로 배포.
Networking	로드 밸런서, VPC	ELB/ALB를 통해 트래픽을 AZ 간 분산 및 WAF (Web Application Firewall) 사용.



Example: Cloud-Based Architecture

□ 마이크로서비스 배포
(Container/PaaS 중심)

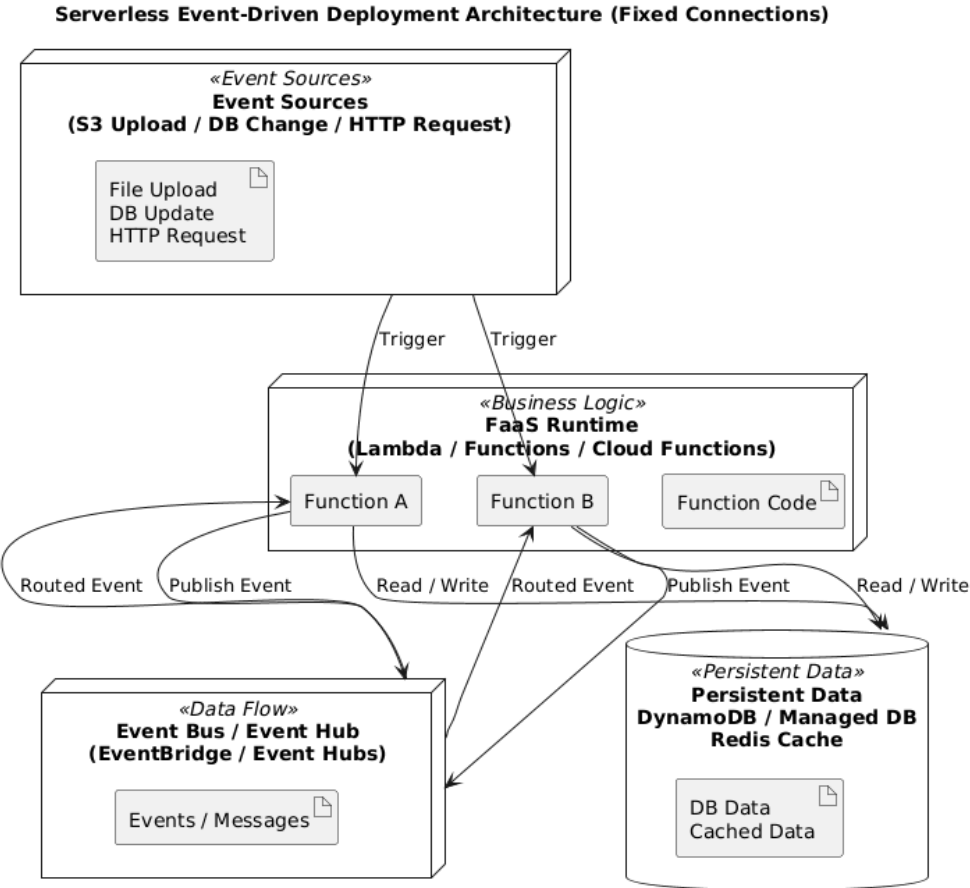
요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Services	수십~수백 개의 독립적인 마이크로서비스 (Containerized)	Kubernetes Cluster (EKS/GKE/AKS) 노드에 배포. 각 서비스는 독립적인 Pod 로 관리.
API Gateway	서비스 라우팅 정의	API Gateway Service (AWS API Gateway/Azure API Management) 를 사용하여 모든 트래픽의 단일 진입점 역할.
Communication	비동기 메시지	Managed Message Queue/Stream (SQS, Kafka, Pub/Sub) 노드에 배포.
Data Stores	서비스별 DB	서비스의 특성에 맞는 다양한 Managed DB (Relational, NoSQL) 를 사용.



Example: Cloud-Based Architecture

서버리스 기반 이벤트 중심 아키텍처 (FaaS 중심)

요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Business Logic	개별 함수 코드 (Function Code)	FaaS Runtime (AWS Lambda/Azure Functions/Cloud Functions)에 배포. 노드(서버) 관리는 클라우드에 위임.
Event Sources	파일 업로드, DB 변경, HTTP 요청	Storage (S3/Blob), Managed DB, API Gateway 등이 이벤트 트리거 노드 역할.
Data Flow	이벤트/메시지	Event Bus/Hub (EventBridge, Event Hubs)를 사용하여 함수 간 또는 외부 시스템으로 이벤트 라우팅.
Persistent Data	데이터베이스, 캐시	Managed NoSQL DB (DynamoDB) 또는 Caching Service (Redis) 사용.



Consequences

❑ **Benefits**

■ **High Availability & Fault Tolerance**

- ❑ Cloud platforms provide multi-AZ/region deployment, automated failure detection, and recovery to ensure high availability.

■ **Auto Scalability**

- ❑ Automatic scaling allows the system to expand during high traffic and shrink during low demand, ensuring cost-efficient operations

■ **Managed Service Utilization**

- ❑ Managed services for DB, messaging, security, and logging reduce operational overhead

■ **Improved Security & Compliance**

- ❑ Cloud providers offer robust security infrastructure, IAM, and key management to improve security

Consequences

❑ Weakness

■ Vendor Lock-in

- ❑ Strong dependency on specific cloud vendor services makes migration challenging.

■ Increased Complexity

- ❑ VPCs, IAM policies, and network/security configurations can increase architectural complexity

■ Unpredictable Cost

- ❑ Auto-scaling and usage-based billing can make cost forecasting difficult

■ Latency Sensitivity

- ❑ Hybrid connectivity or cross-region communication can introduce latency

■ High Data Transfer Costs

- ❑ Cross-region or cross-cloud data transfer can be costly.

Edge Based Architecture

□ Background

- The increase of IoT devices at the edge of the network is producing a massive amount of data to be computed at data centers, pushing network bandwidth requirements to the limit

□ Edge computing is a distributed computing paradigm that brings computation and data storage closer to the sources of data.

- This is expected to improve response times and save bandwidth
- Meaning **that most of the operations are done on a device (Edge)** instead of a server in the cloud,

□ Cloud computing operates on big data, while **edge computing operates on "instant data"** that is real-time data **generated by sensors or users.**

Edge Based Style

□ Elements

■ Edge Devices

- 온도 센서, 카메라, 차량 라이다, 스마트 미터 등
 - 주로 데이터 “생성” 역할

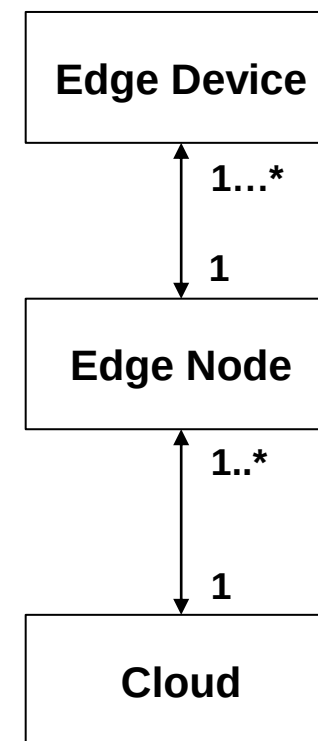
■ Edge Node

- 로컬 CPU/GPU/TPU 등 비교적 강한 연산 자원
 - 데이터 전처리, ML 모델 추론, 로컬 의사결정/제어, 등 수행

■ Cloud/Remote Data Centers

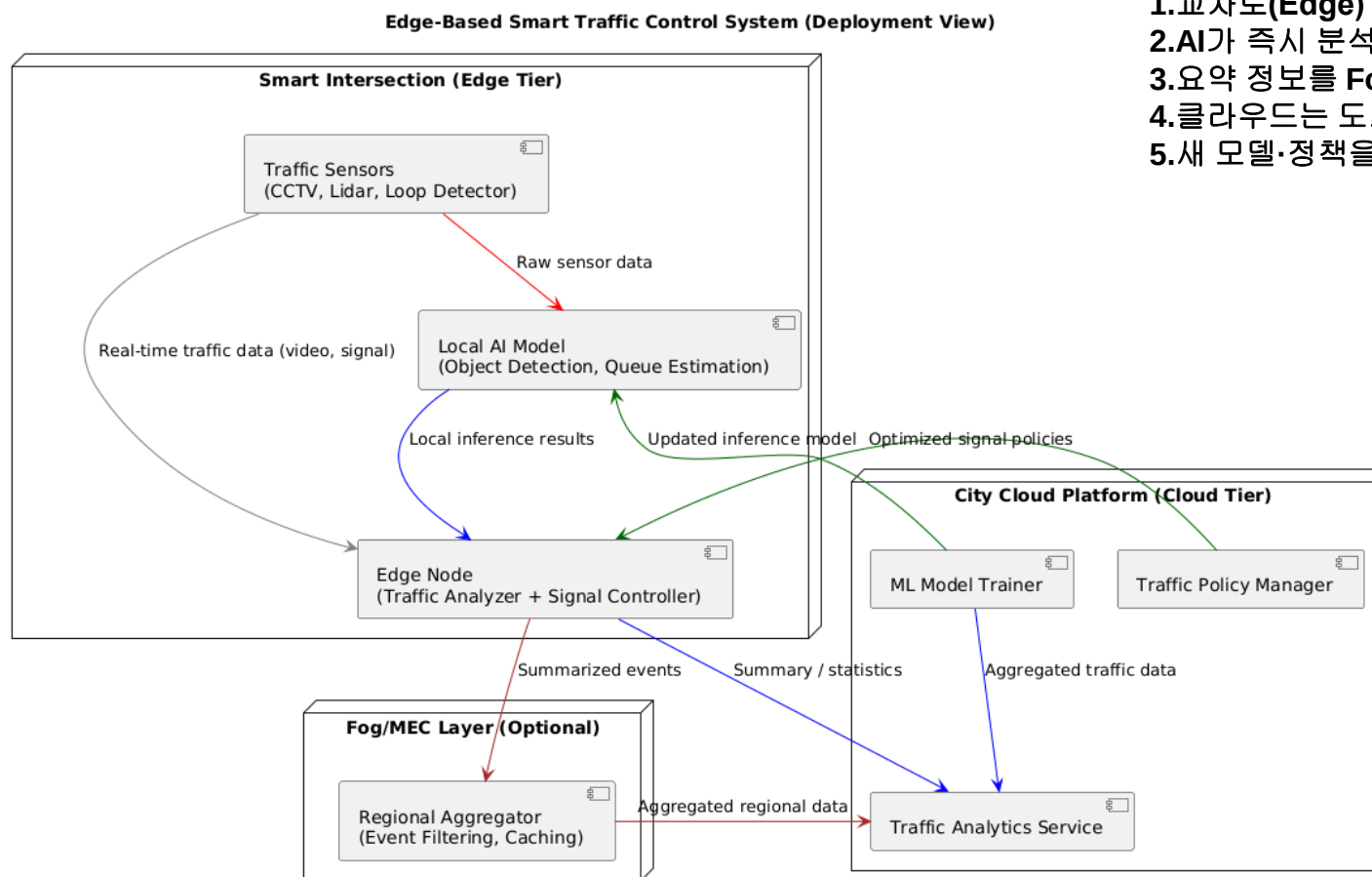
- Edge 단에서 처리하기에 적합하지 않은 즉, 컴퓨팅 파워가 필요한 서비스를 처리하는 역할을 수행

□ Relations



Example

□ 스마트시티 실시간 교통 제어 시스템

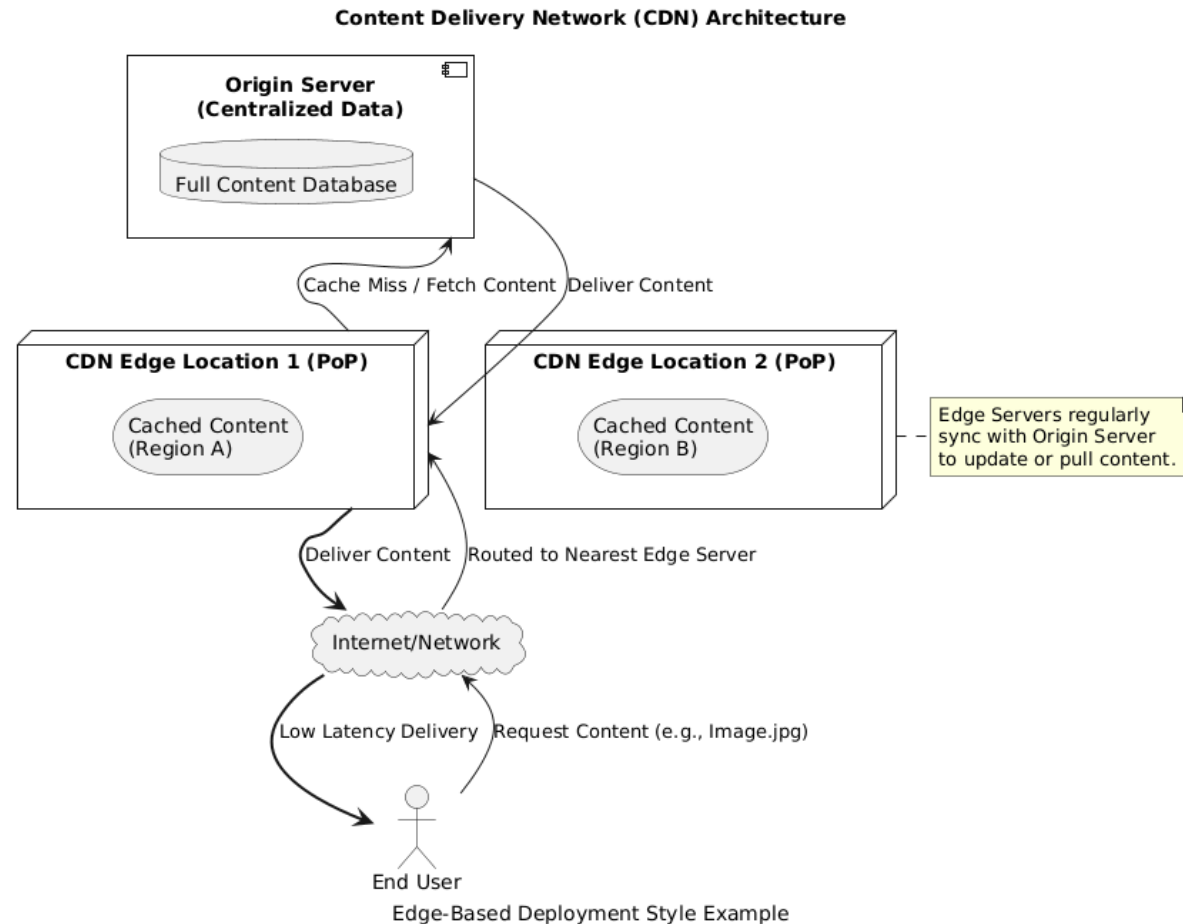


◇ 전체 흐름 요약

1. 교차로(Edge) → 센서 데이터 수집
2. AI가 즉시 분석하고 신호등 제어
3. 요약 정보를 Fog/MEC로 전달 (선택)
4. 클라우드는 도시 전체 데이터를 학습
5. 새 모델·정책을 Edge에 다시 보내 최적화

Example

□ Content Delivery Network (CDN)



Consequences

□ Benefits

■ Privacy

- By keeping data at the edge it is possible to shift ownership of collected data from service providers to end-users.

■ Speed

- Edge computing brings analytical computational resources close to the end users and therefore helps to speed up the communication speed.

■ Availability

- By performing computing functions on the local network, regional sites can continue to operate independently even if the central site ceases operations for any reason.

Consequences

❑ Weaknesses

■ Security

- ❑ As more data is being processed outside of the core, the risk for a leak, theft or a cybersecurity breach rises significantly.

■ Investment Cost

- ❑ Implementing an edge infrastructure can be expensive, because it requires more local hardware. This can overall lead to more efficiency but a significant investment is required.

Summary (Architectural Styles)

- ❑ An architectural style is **the abstraction** of similar software architectures
- ❑ An architectural style is determined by:
 - a set of element types
 - a set of relationships between the elements
 - a set of constraints on the topology and behavior
- ❑ A Catalog of Architectural Styles
 - 7 categories, 23 architectural styles
 - Many architectural styles overlap
 - ❑ Buffered Message Based, Broker, MVC, SOA

Summary

- ❑ **Architectural styles help software engineers reason about architectural qualities**
 - One architectural style has positive or negative effects on multiple quality attributes.
 - ❑ E.g., Layered style
 - Positive for modifiability
 - Negative for performance
 - For a single quality attribute, one architecture may have both positive and negative effects
 - ❑ E.g., Repository style
 - Positive for modifiability - Modifiable with respect to changing the functionality of any data client
 - Negative for modifiability - High dependency between data structure of data store and its clients
- ❑ Architectural styles/patterns help architects identifying and analyzing architectural solutions for desired quality attributes

Associate Architect 양성과정

아키텍처 설계

Architecture Views

Topics

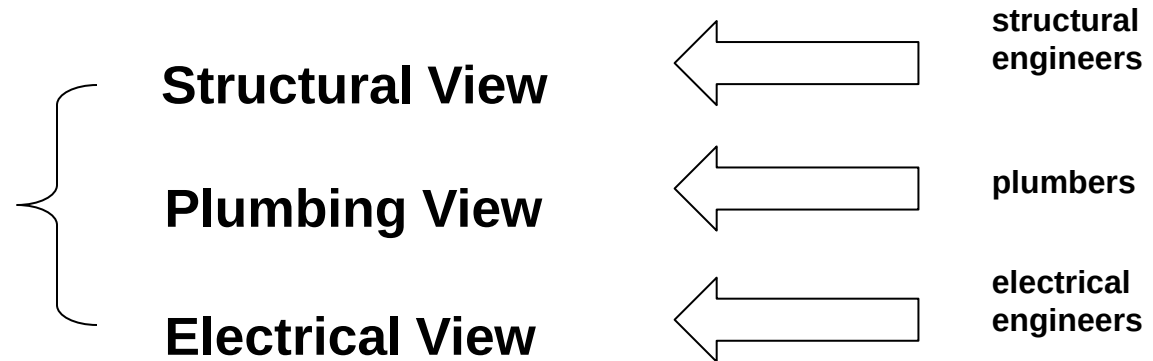
- ❑ **Introduction to Architecture views**
 - **4+1 View Model**
 - **Siemens View Model**
 - **SEI View Model**
- ❑ **Documenting Architecture**
 - **Documenting Architecture using SEI View Model**
 - **Documenting Architecture using Hierarchical Multi-View Model**
 - **Documenting Architecture using C4 Model**
- ❑ **Summary**

Architecture Views

INTRODUCTION

Architectural Views

- ❑ An architectural view refers to a representation of a system's architecture that focuses on specific concerns or aspects
- ❑ Modern systems are more than complex enough to make it difficult to grasp them all as a single structure.
- ❑ Different stakeholders always have different interest in the system



Structures and Views

- ❑ **A structure is the set of elements itself**, as they exist in software or hardware
 - **Software architecture Definition**
 - ❑ **The software architecture of a computing system is the set of structures** needed to reason about the system, which comprise software elements, relations among them and properties of both.
- ❑ **A representation of a structure** is usually called a ***view*** of the system.
- ❑ **Software architecture documentation is comprised of a collection of views**

Topics

- ❑ **Introduction to Architecture views**
 - **4+1 View Model**
 - **Siemens View Model**
 - **SEI View Model**
- ❑ **Documenting Architecture**
 - **Documenting Architecture using SEI View Model**
 - **Documenting Architecture using Hierarchical Multi-View Model**
 - **Documenting Architecture using C4 Model**
- ❑ **Summary**

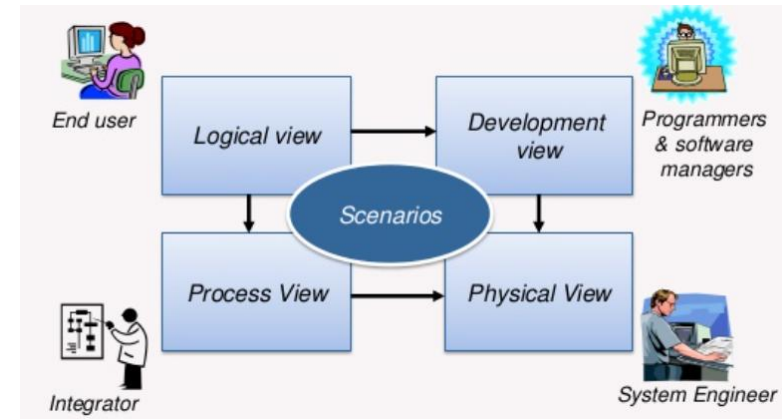
4+1 View Model

- ❑ **Proposed by Phillippe Kruchten**

- Describing the architecture of software-intensive systems, based on the use of multiple, concurrent views

- ❑ **The four views of the model**

- Logical View
 - Development View
 - Process View
 - Physical View



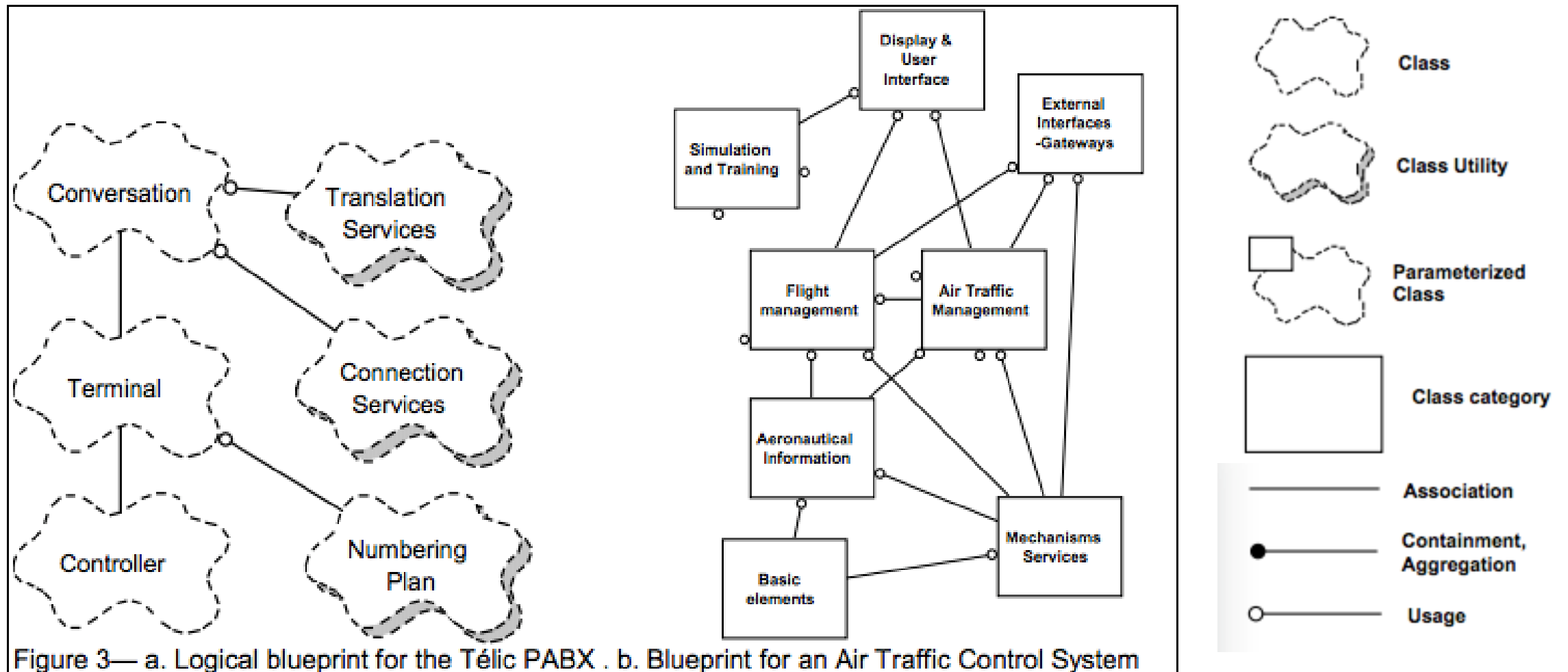
- ❑ In addition, selected use cases or scenarios are utilized to illustrate the architecture serving as the “+1” view.

4+1 View Model:

Logical View

- ❑ This view shows the components (objects) of the system as well as their interactions / relationships
- ❑ Viewer
 - End-user
- ❑ Considers
 - **Functional requirements**
 - Modeling elements at the logical level not the implementation level
- ❑ Relevant UML diagrams
 - class diagram, object diagram, sequence diagram, communication diagram

4+1 View Model: Example of Logical View



4+1 View Model:

Process View

- ❑ **The process view shows the processes of a system and how those processes communicate with each other.**

- ❑ **Viewer**
 - Integrators
- ❑ **Considers**
 - Non-functional requirements (concurrency, performance, scalability)
- ❑ **Relevant UML Diagrams**
 - activity diagram, component diagram

4+1 View Model: Example of Process View

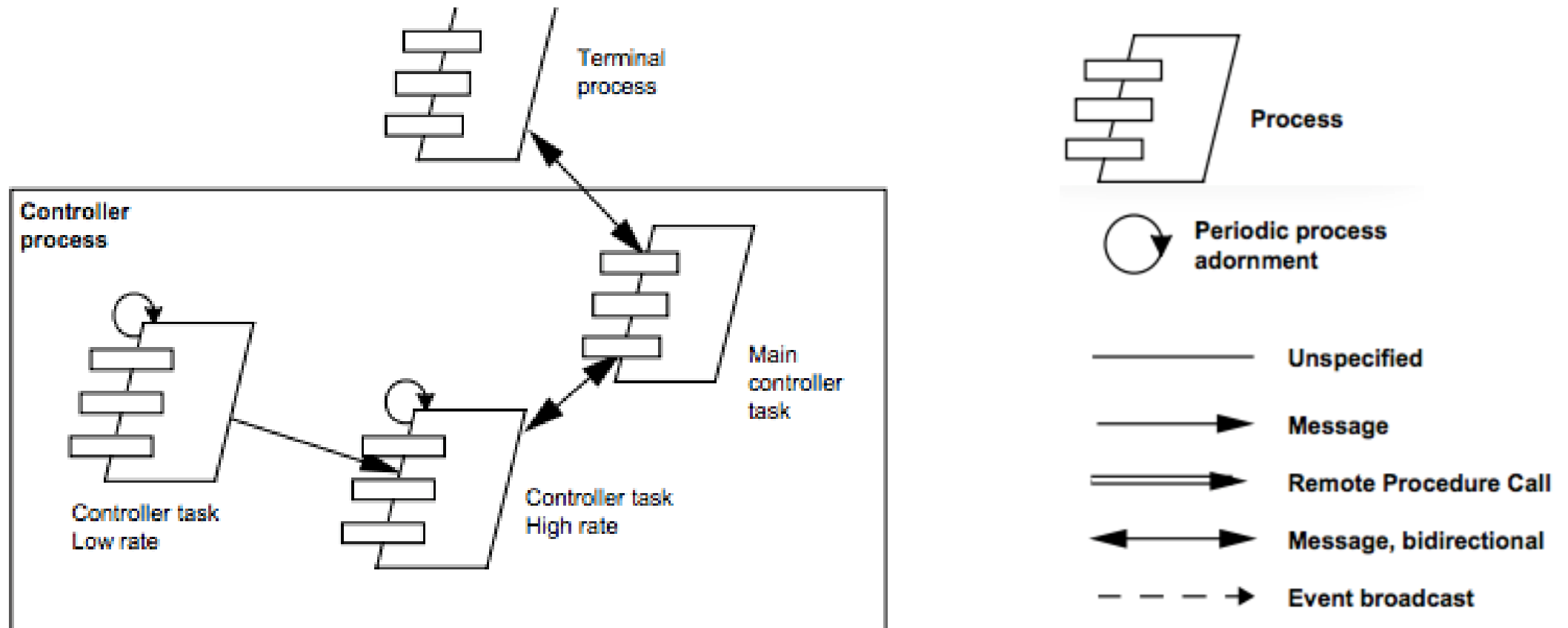


Figure 5 — Process blueprint for the Télec PABX (partial)

4+1 View Model: Development View

- ❑ **Development view gives a building block view of the system.**
 - E.g., classes, packages, and sub systems
- ❑ **Viewer**
 - Developers and project managers
- ❑ **Considers**
 - Non-functional requirements (adaptability, reuse, software management, constraints of tools)
- ❑ **Relevant UML Diagrams**
 - class diagram, package diagram, component diagram

4+1 View Model: Example of Development View

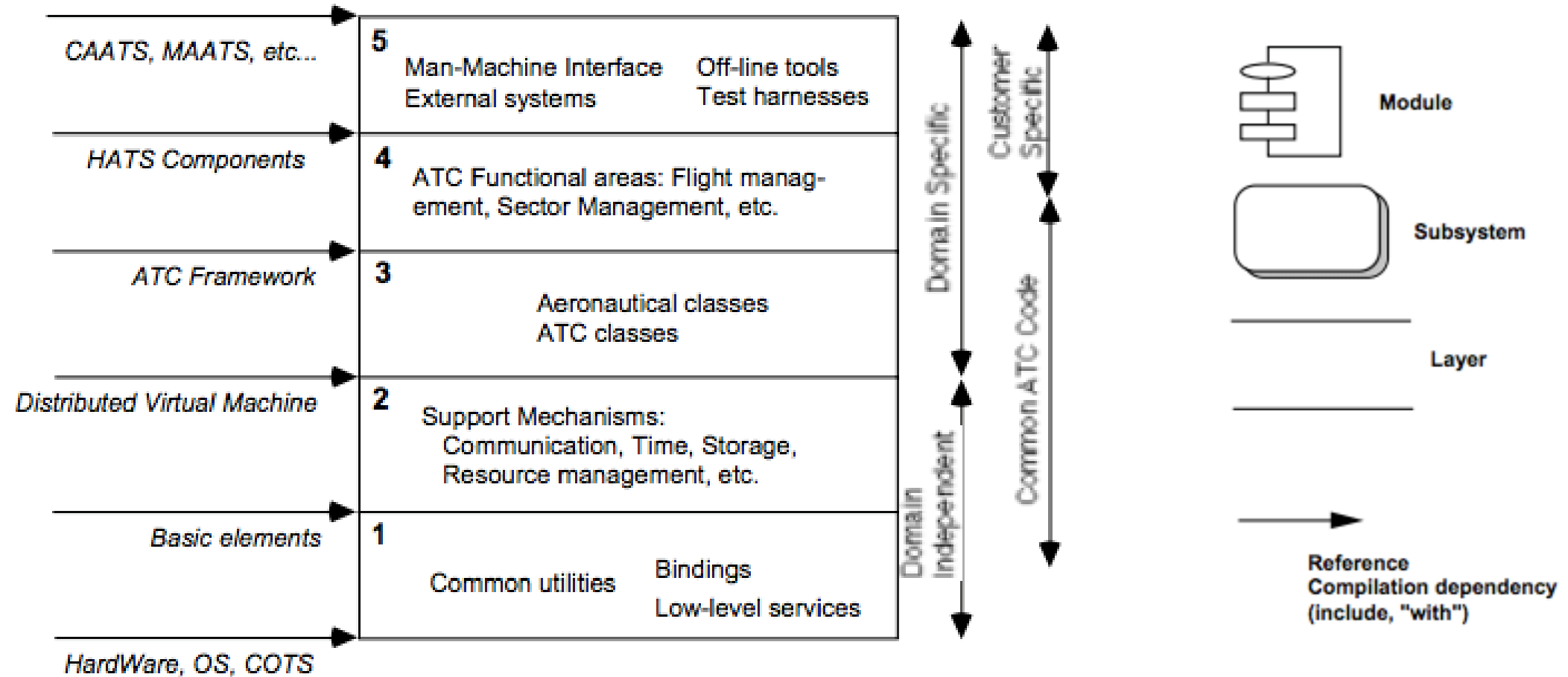


Figure 6 — The 5 layers of Hughes Air Traffic Systems (HATS)

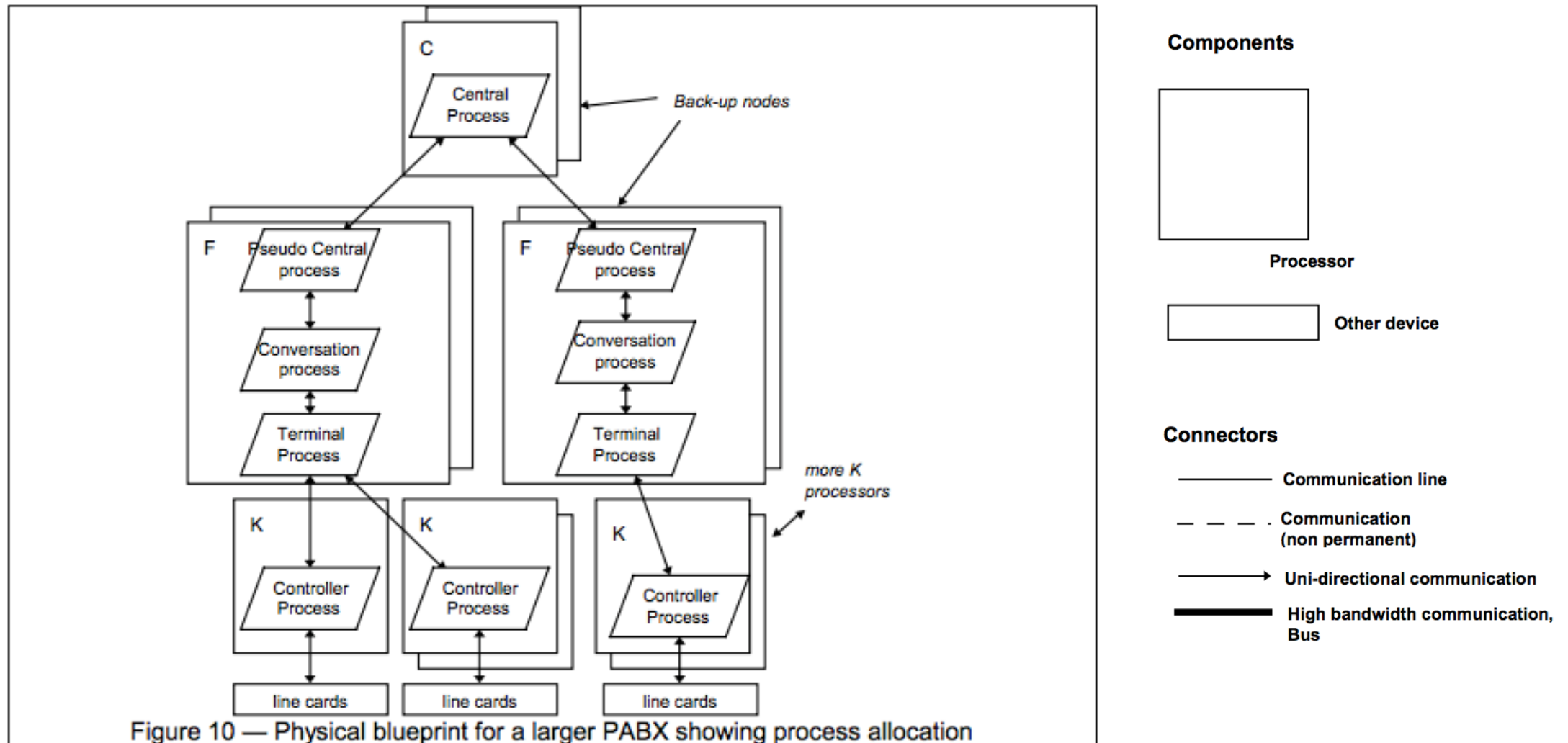
4+1 View Model:

Physical View

- ❑ **This view shows the systems execution environment**

- ❑ **Viewer**
 - **System engineers**
- ❑ **Considers**
 - **Non-functional requirements (concurrency, performance, scalability)**
 - **Constraints on underlying hardware (Topology, communication)**
- ❑ **Relevant UML Diagrams**
 - **Deployment diagram**

4+1 View Model: Example of Physical View

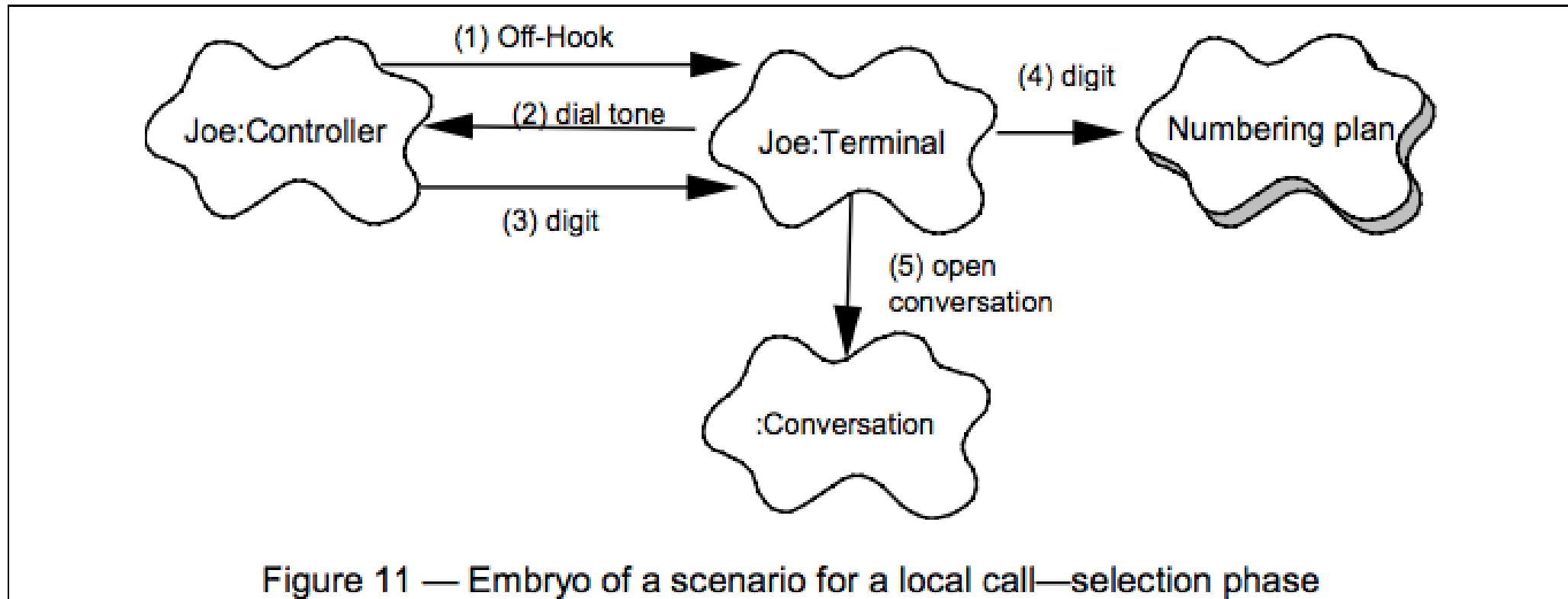


4+1 View Model:

Scenario / Use case View

- ❑ **This view is used to validate and illustrate the design**
 - It is redundant with other views
- ❑ **Viewer**
 - All users of other views and Evaluators
- ❑ **Considers**
 - System consistency, validity
- ❑ **Relevant UML Diagrams**
 - Use case diagram, Sequence diagram, Communication diagram

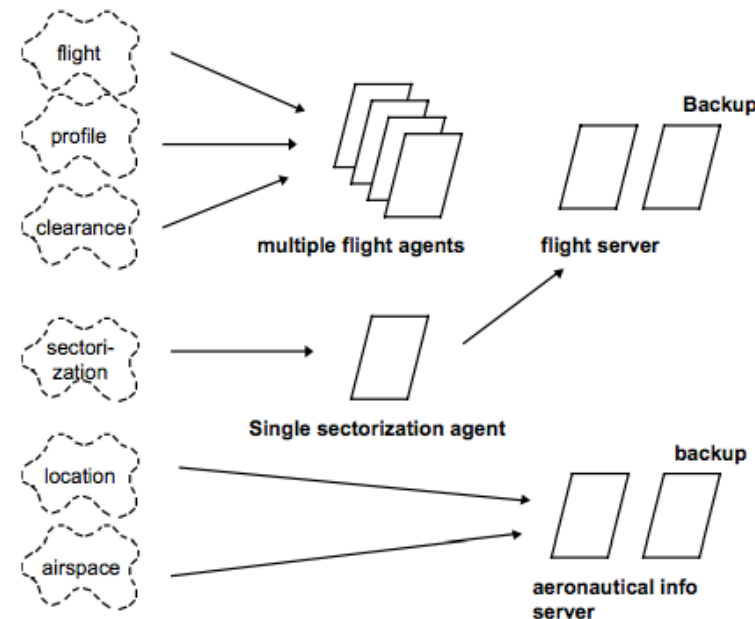
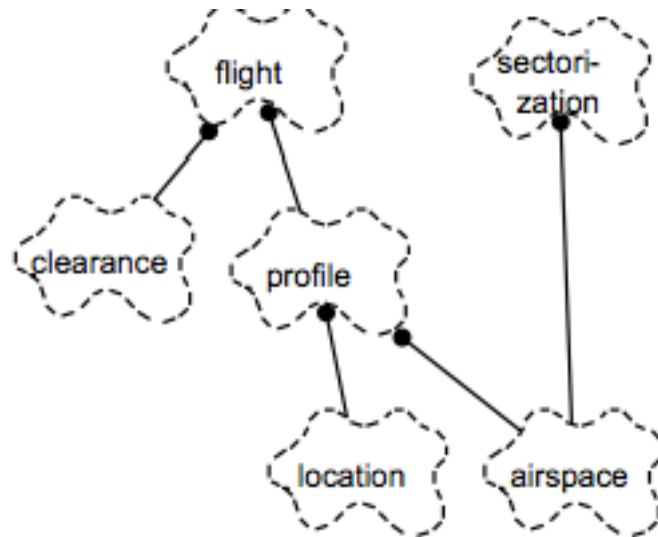
4+1 View Model: Example of Scenario



4+1 View Model: Correspondence between the views

□ From Logical to Process

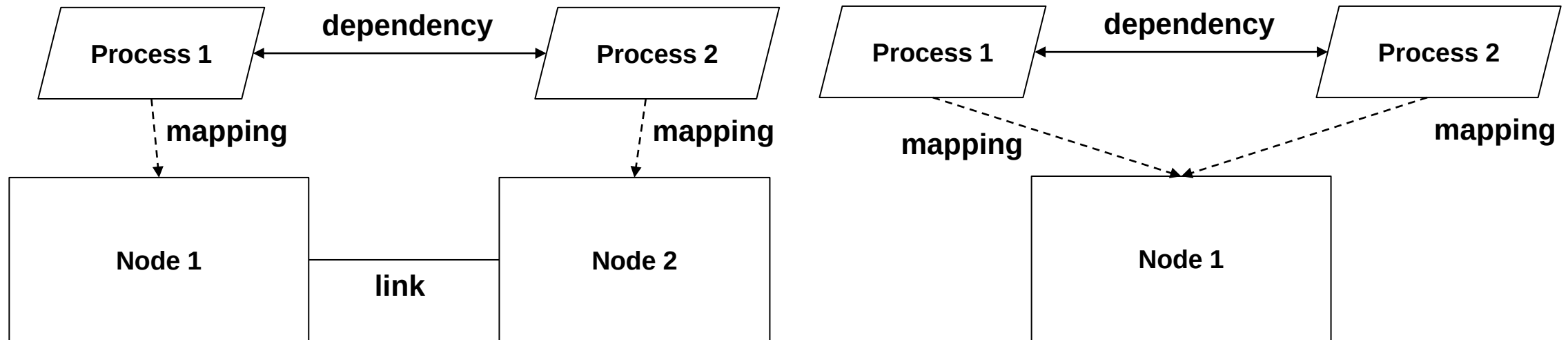
- Concurrency is not addressed in the logical view
- **Classes and their objects in the logical view** need to be mapped onto **processes in the process view** to address concurrency and synchronization



4+1 View Model: Correspondence between the views

□ From Process to Physical

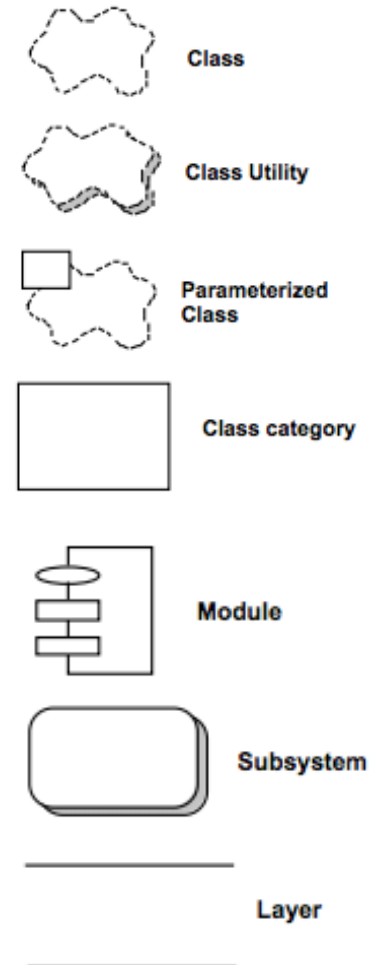
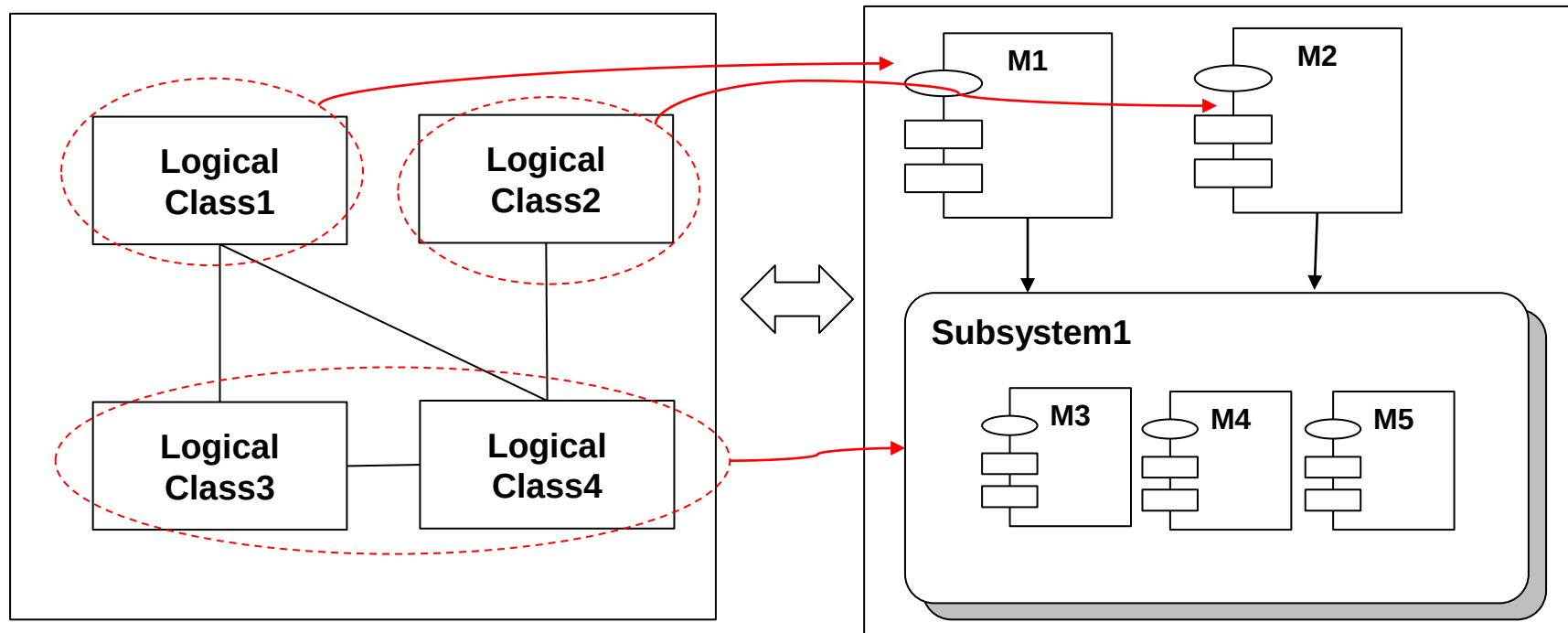
- The processes and process groups are mapped onto the processing nodes of a physical computer network to obtain the physical view
- For each dependency between components, there must be corresponding link between nodes



4+1 View Model: Correspondence between the views

□ From Logical to development

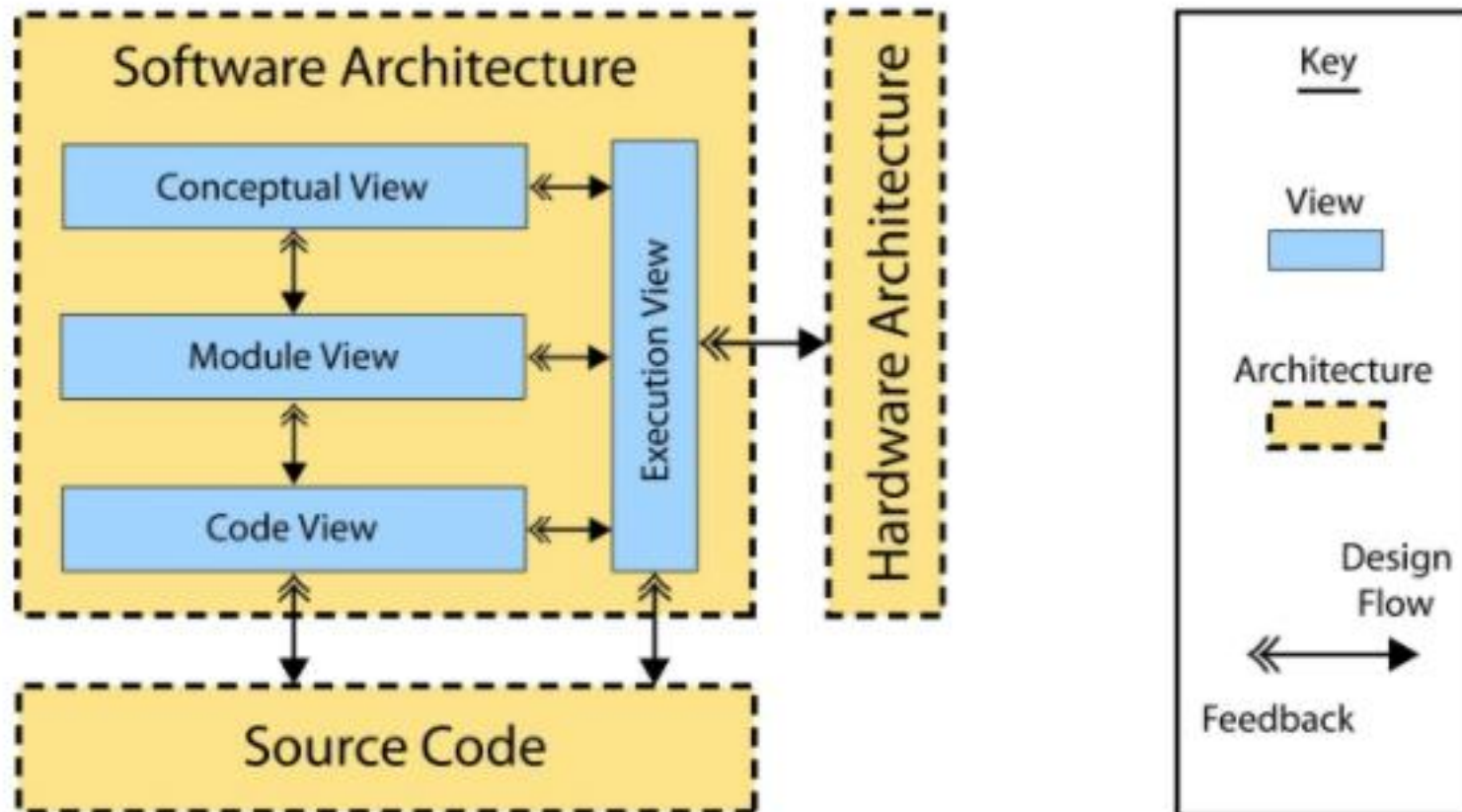
- The logical view is also the basis for the development view
 - Normally each class corresponds to a module
 - Collections of classes are grouped into packages or subsystems.



Topics

- ❑ **Introduction to Architecture views**
 - 4+1 View Model
 - **Siemens View Model**
 - SEI View Model
- ❑ **Documenting Architecture**
 - Documenting Architecture using SEI View Model
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ **Summary**

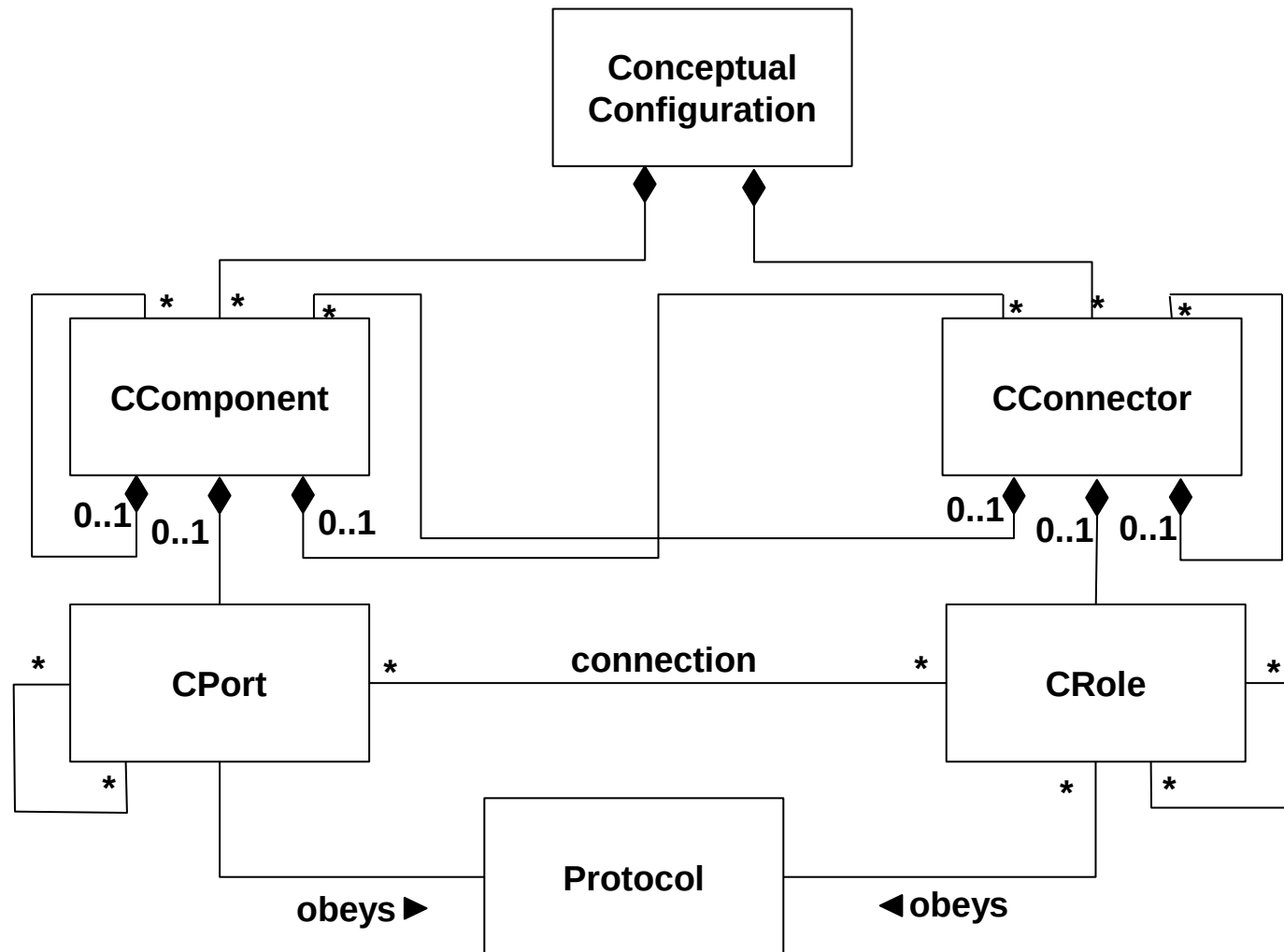
Siemens View Model



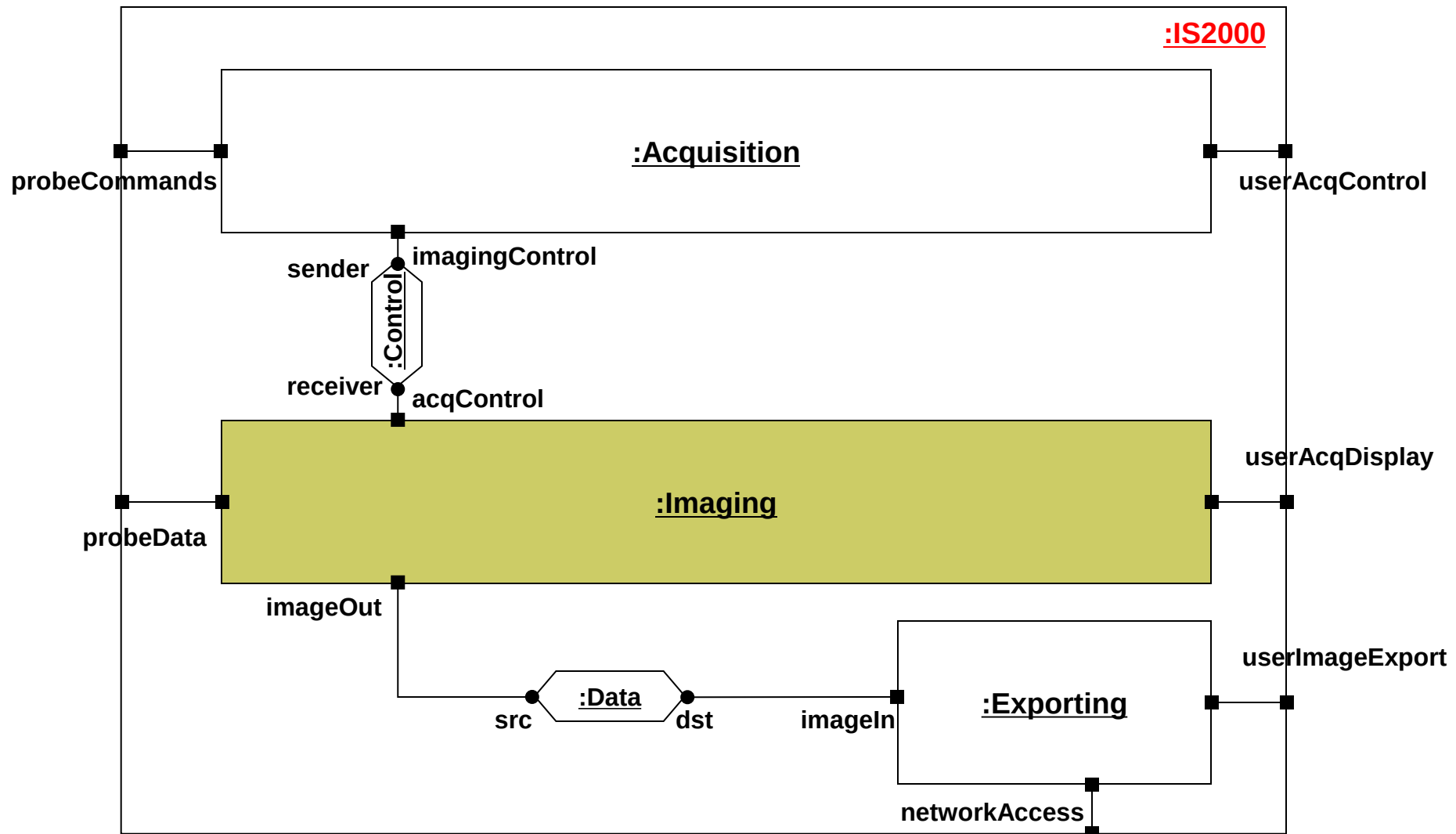
Siemens View Model: Conceptual View

- ❑ **In the conceptual view, functionality of the system is mapped to architectural elements:**
 - **Conceptual components for functional decomposition**
 - **Connectors for coordination and data exchanges**
- ❑ **This view is relatively independent of particular software and hardware techniques**
- ❑ **Engineering concerns**
 - **How to map the system's functionality to components and connectors**
 - **How to treat run time quality attributes such as performance**

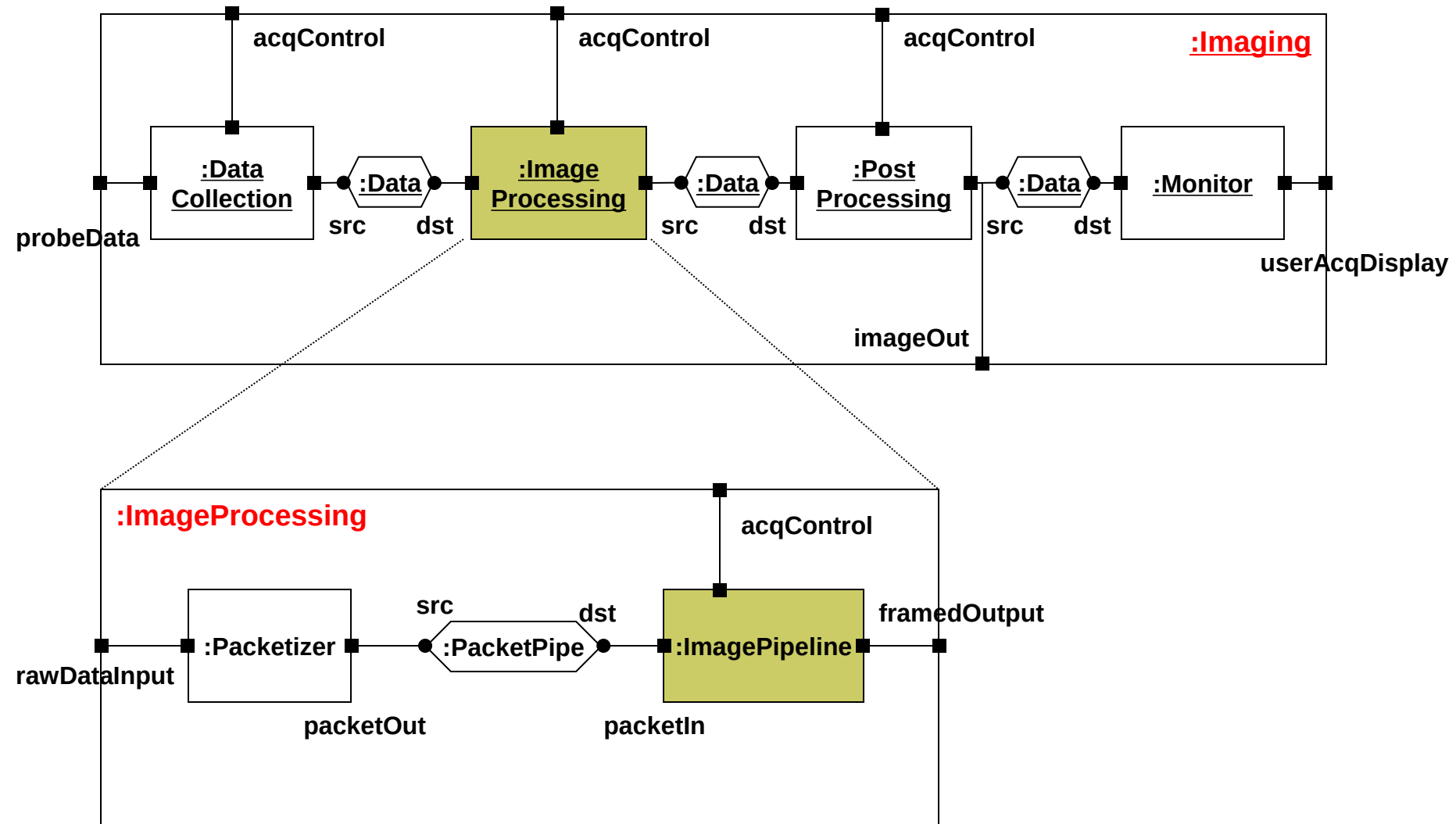
Siemens View Model: Meta-model of the conceptual architecture



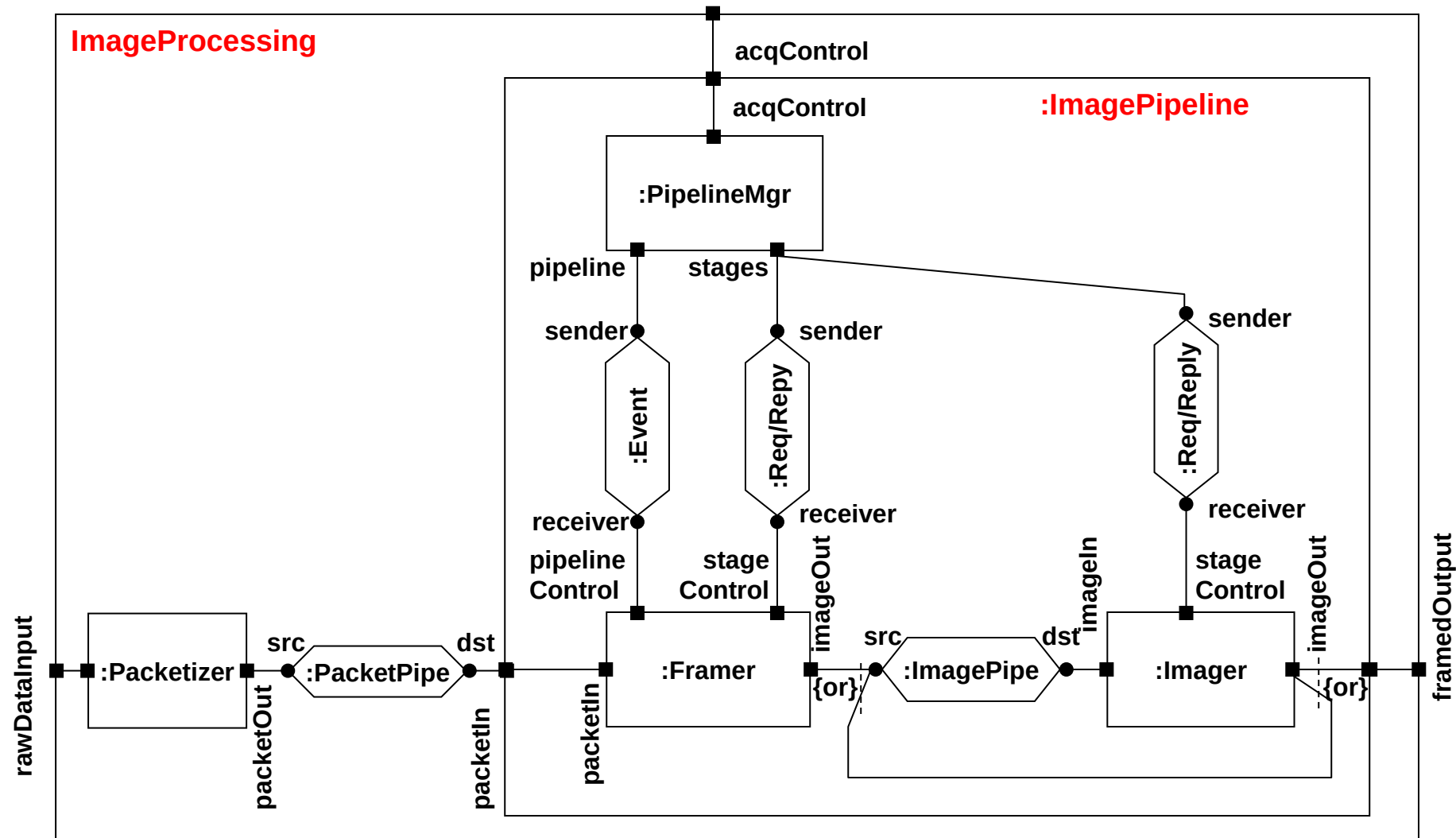
Siemens View Model: high-level of configuration (Example)



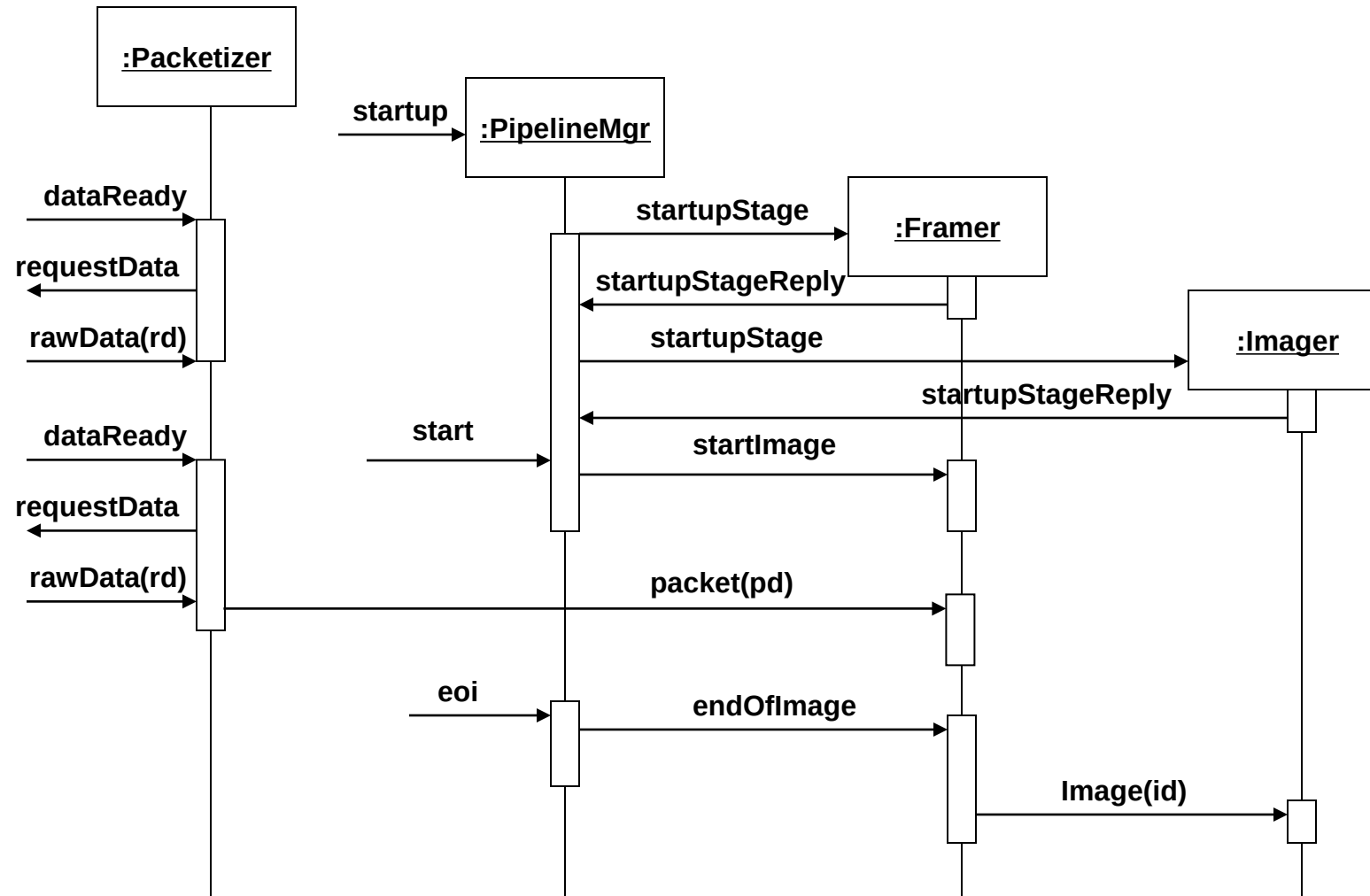
Siemens View Model: Refinements of Imaging (Example)



Siemens View Model: Refinement of ImageProcessing (Example)



Siemens View Model: Behavior Modeling of ImageProcessing (Example)



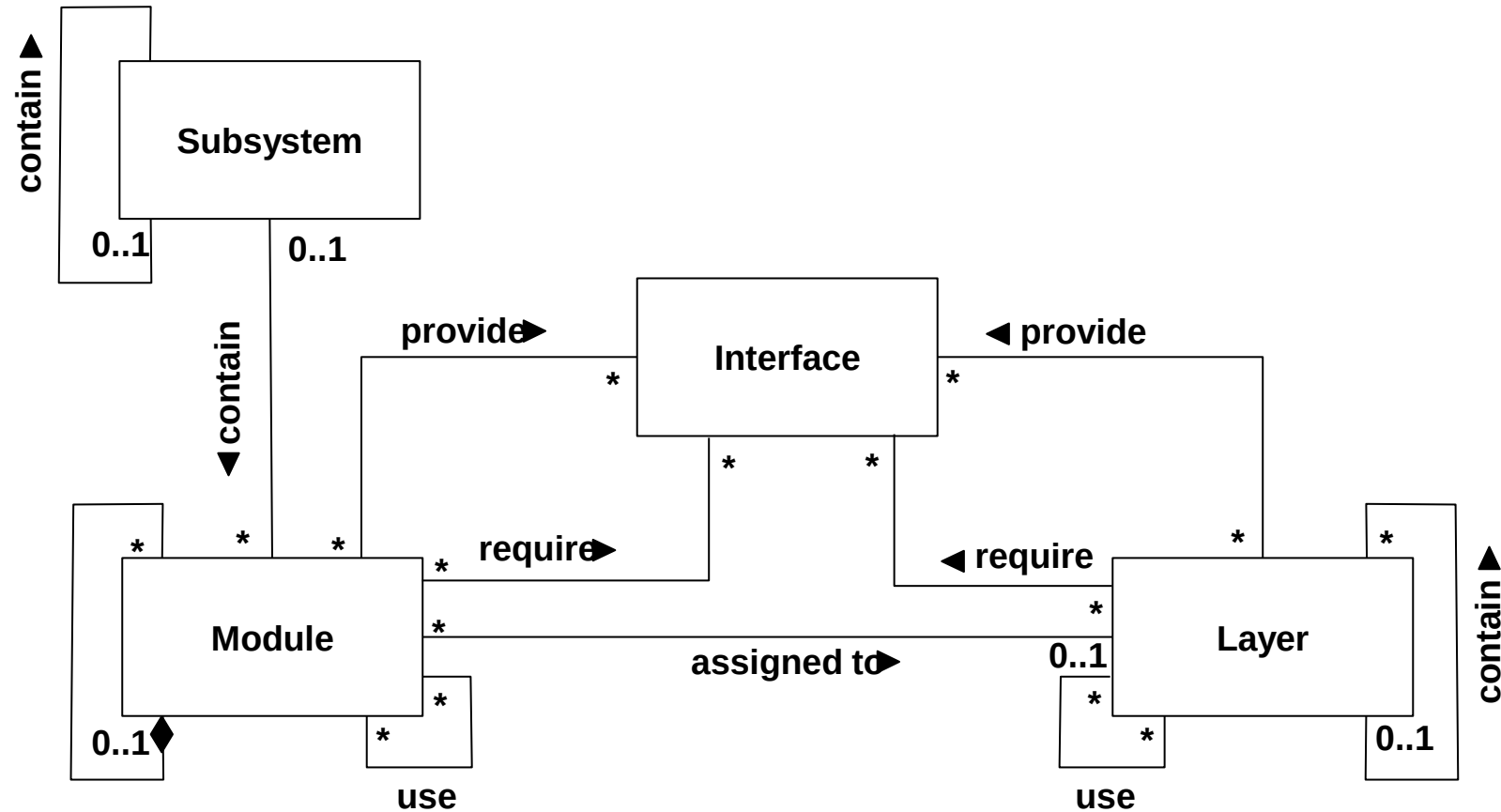
Siemens View Model:

Module View

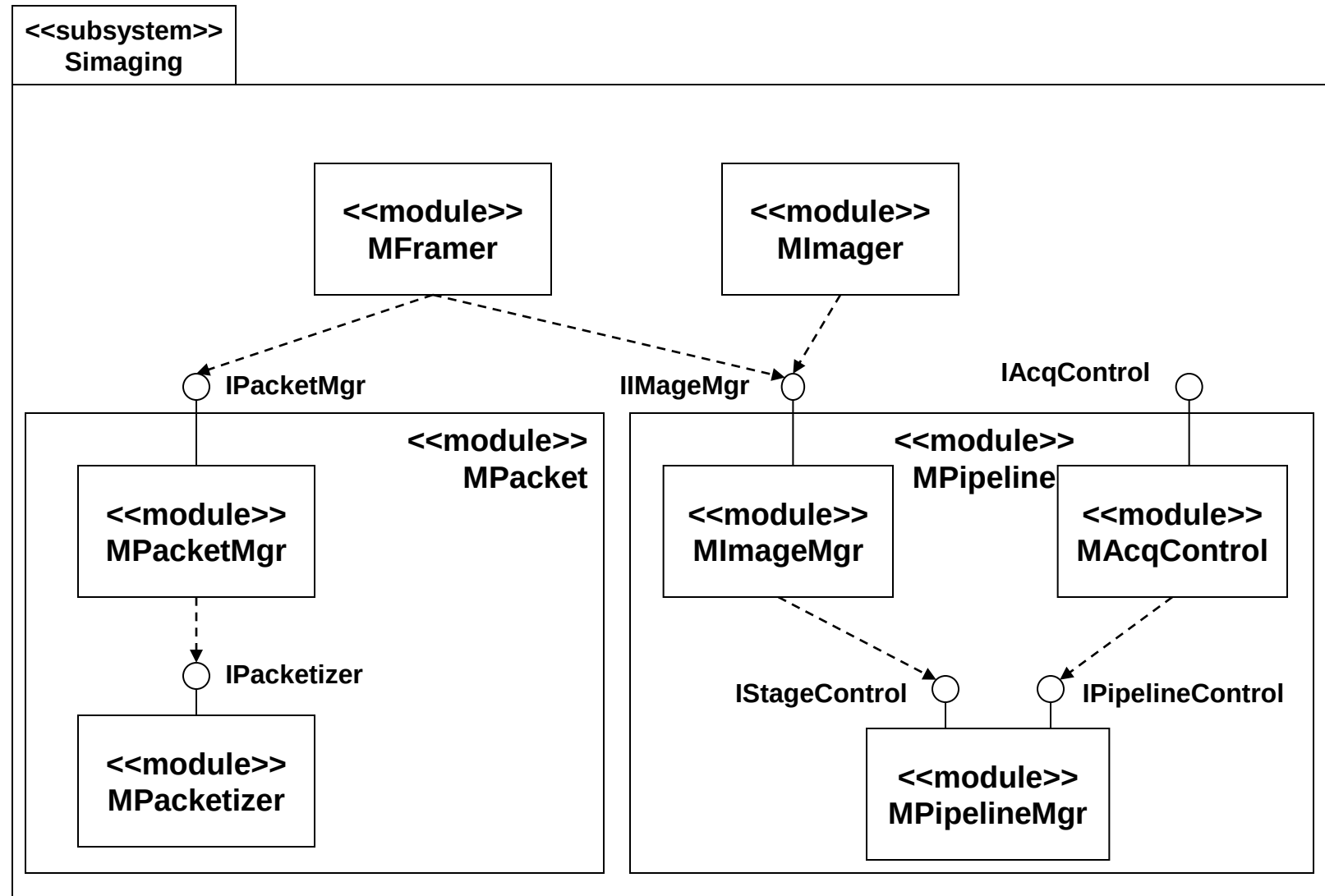
- ❑ In the module view, the components and connectors from the conceptual view are mapped to ***subsystems*** and ***modules***.
 - The architect addresses how the conceptual solution can be realized with today's software platforms and technologies.

- ❑ Engineering concerns
 - How can **dependencies** between modules **be minimized**?
 - How can **reuse** of modules and subsystems be maximized?
 - What techniques can be used **to insulate the product from changes** in COTS software, in the software platform, or changes to standards?

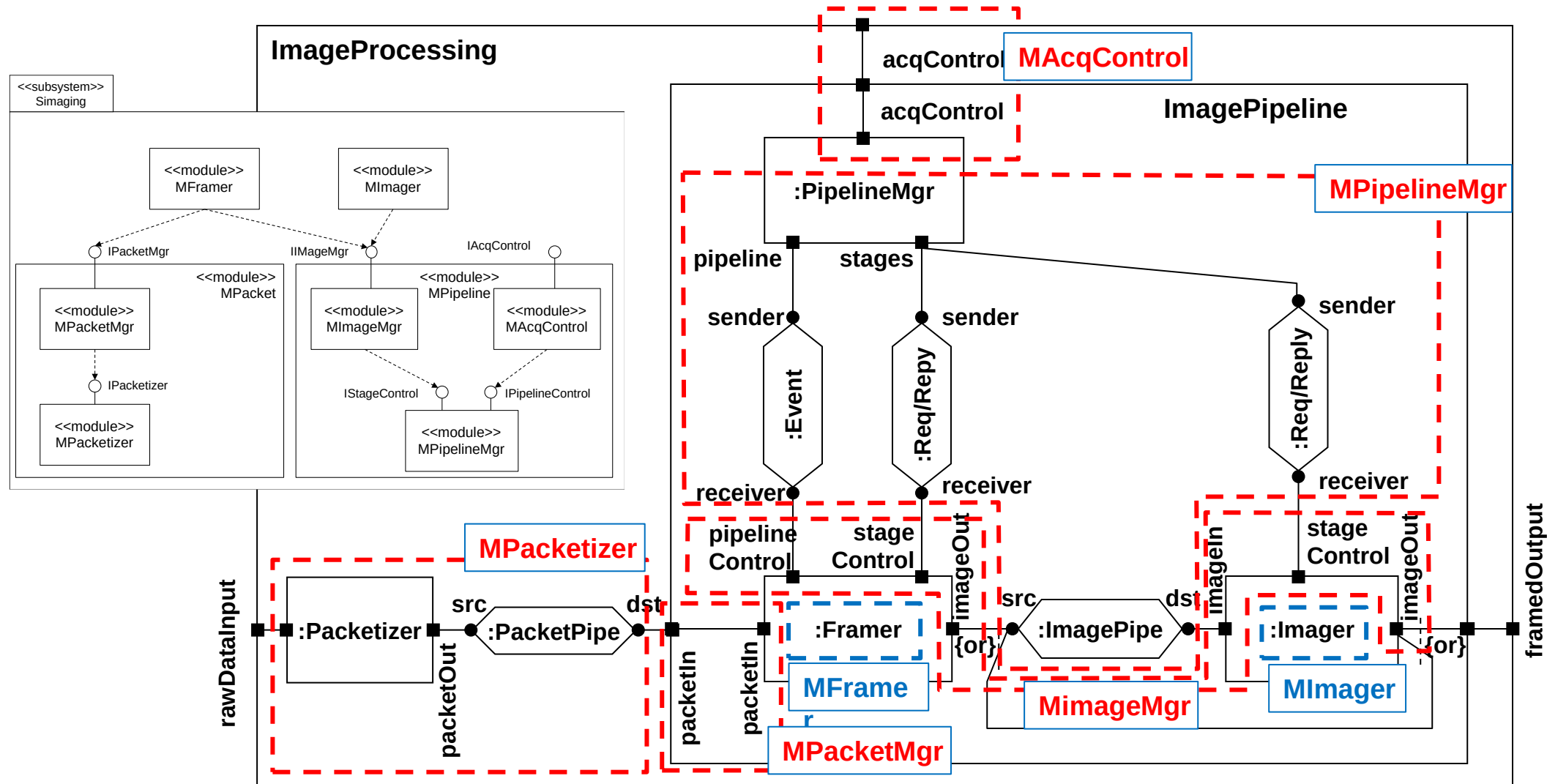
Siemens View Model: Meta-model of the Module architecture



Siemens View Model: Module View Example



Siemens View Model: Mapping Conceptual Elements to Modules



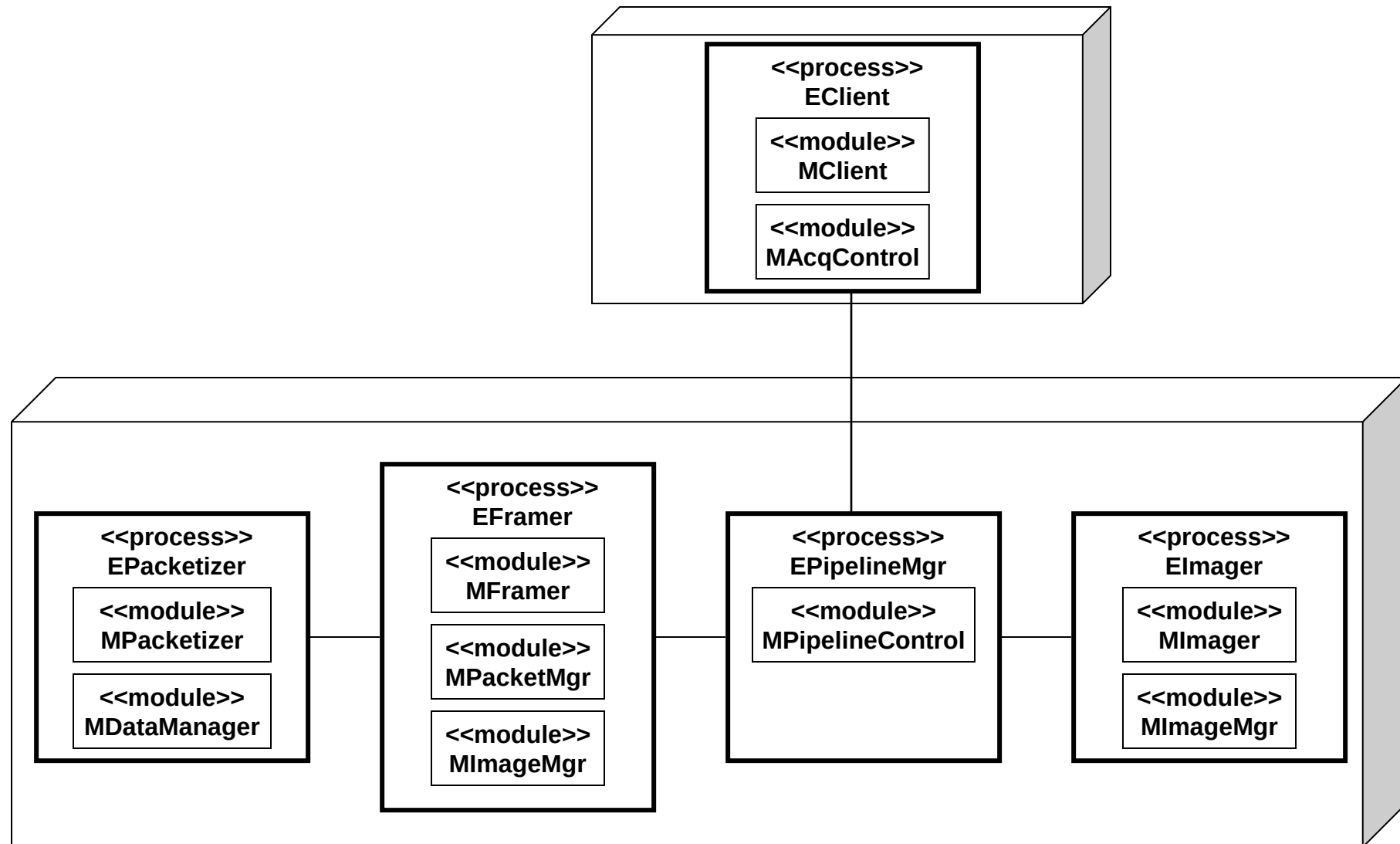
Siemens View Model: Mapping Conceptual Elements to Modules

Conceptual Elements		Module Element	
Name	Kind	Name	Kind
ImageProcessing	Component	SIImaging	Subsystem
Packetizer packetOut PacketPipe, src, dst	Component Port Connector and roles	Mpacketizer	Module
packetIn	Port	MPacketMgr	Module
acqControl	Port	MAcqControl	Module
PipelineMgr Pipeline,stages ImagePipe, src, dest Client/Server, sender, receiver Event, Sender, receiver	Component Ports Connector and roles Connector and roles Connector and roles	MPipelineMgr	Module
pipelineControl, stageControl, imageIn, imageOut	Ports	MImageMgr	Module
Framer	Component	MFramer	Module
Imager	Component	MImager	Module

Siemens View Model: Execution View

- ❑ **The execution view describes**
 - how modules are mapped to the elements provided by the runtime platform
 - how these are mapped to the hardware architecture
 - The system's runtime entities and their attributes, such as memory usage
- ❑ **Engineering concerns**
 - How does the system meet its performance, recovery, and reconfiguration requirements?
 - How can one balance resource usage?
 - How can one achieve the necessary concurrency, replication, and distribution?

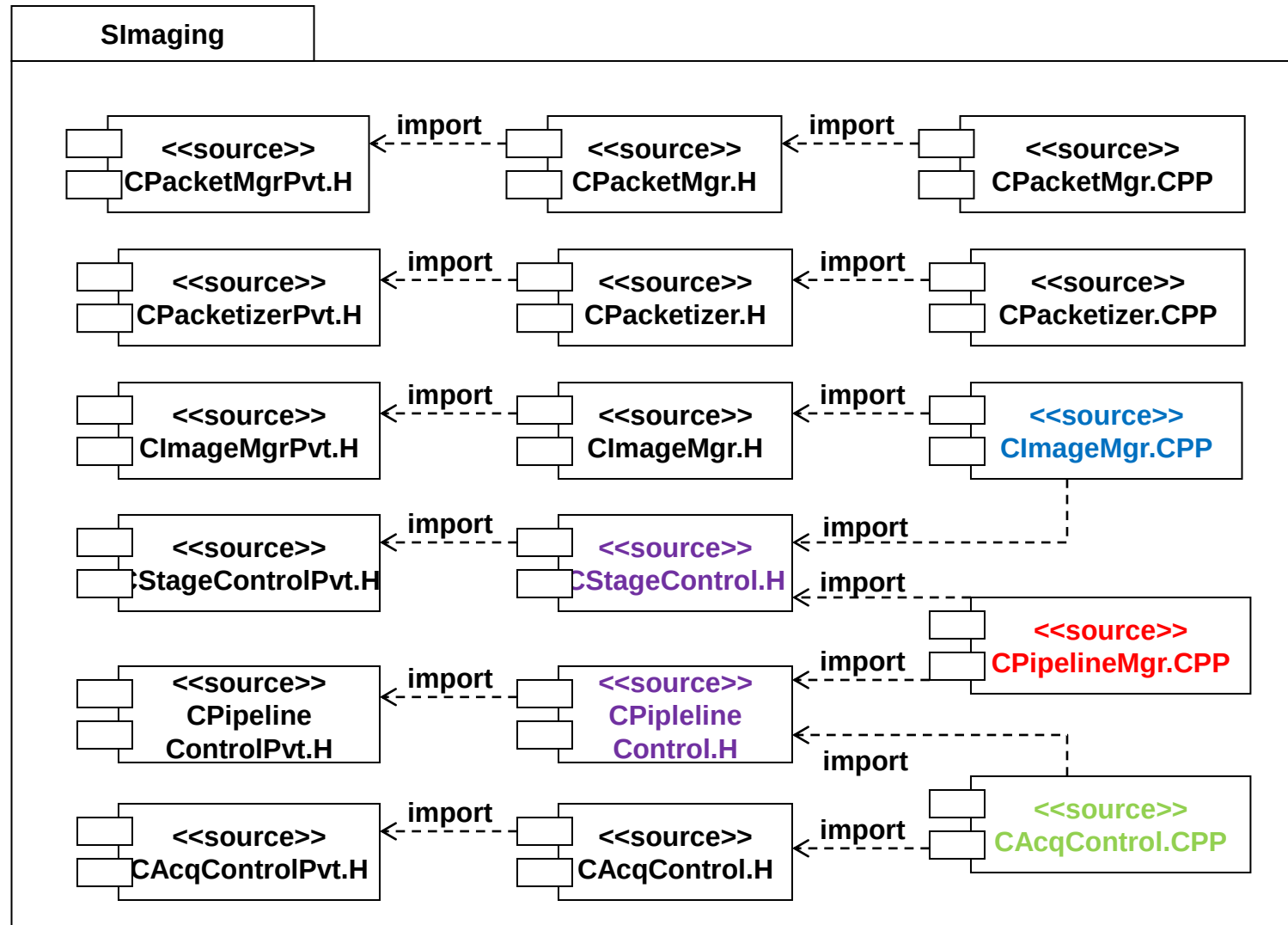
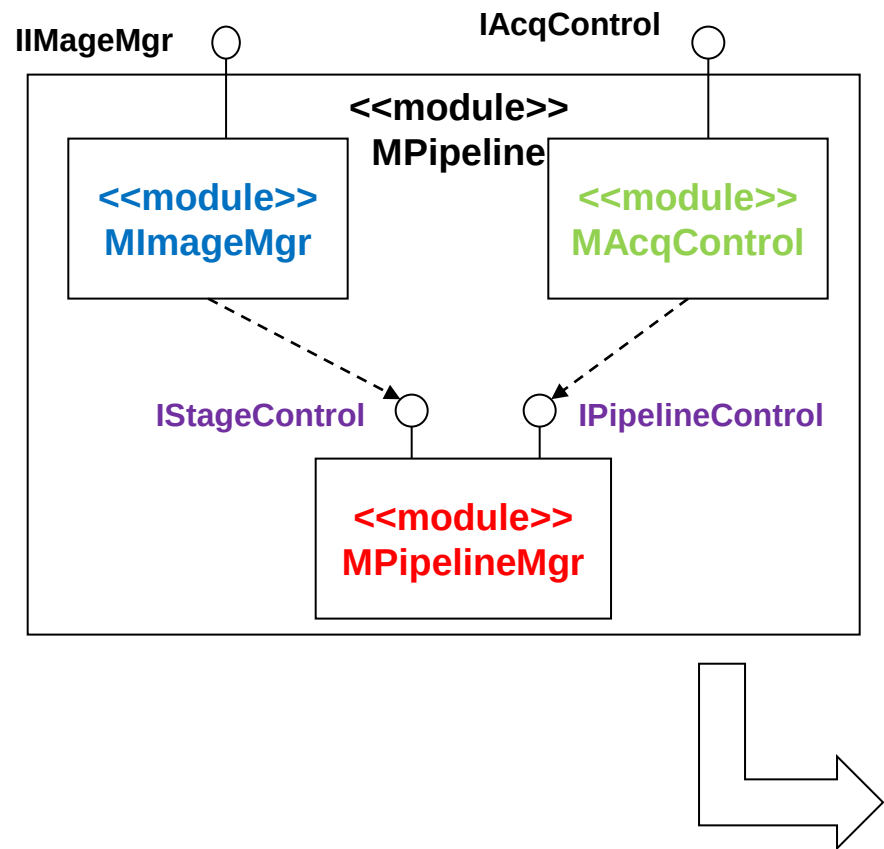
Siemens View Model: Execution View Example



Siemens View Model: Code View

- ❑ The code view describes how the software implementing the system is organized.
- ❑ For code view, the architect determines
 - how runtime entities from the execution view are **mapped to deployment components** (e.g., executable)?
 - how modules from the module view are **mapped to source components**?
 - how the **deployment components are produced from the source components**?
- ❑ Engineering concerns
 - How can the time and effort for product upgrades be reduced?
 - How should product versions and release be managed?
 - How can build time be reduced?

Siemens View Model: Code View Example



Topics

- ❑ **Introduction to Architecture views**
 - 4+1 View Model
 - Siemens View Model
 - **SEI View Model**
- ❑ **Documenting Architecture**
 - Documenting Architecture using SEI View Model
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ **Summary**

SEI View Model

- ❑ **Component-and-connector views (C&C views)**
 - structured as a set of elements that have **runtime behavior** (components) and **interactions** (connectors)
- ❑ **Module views**
 - structured as **a set of code units (modules)** and **their relationships**
- ❑ **Allocation views**
 - show **how the software system relates to non-software structures** in its environment.

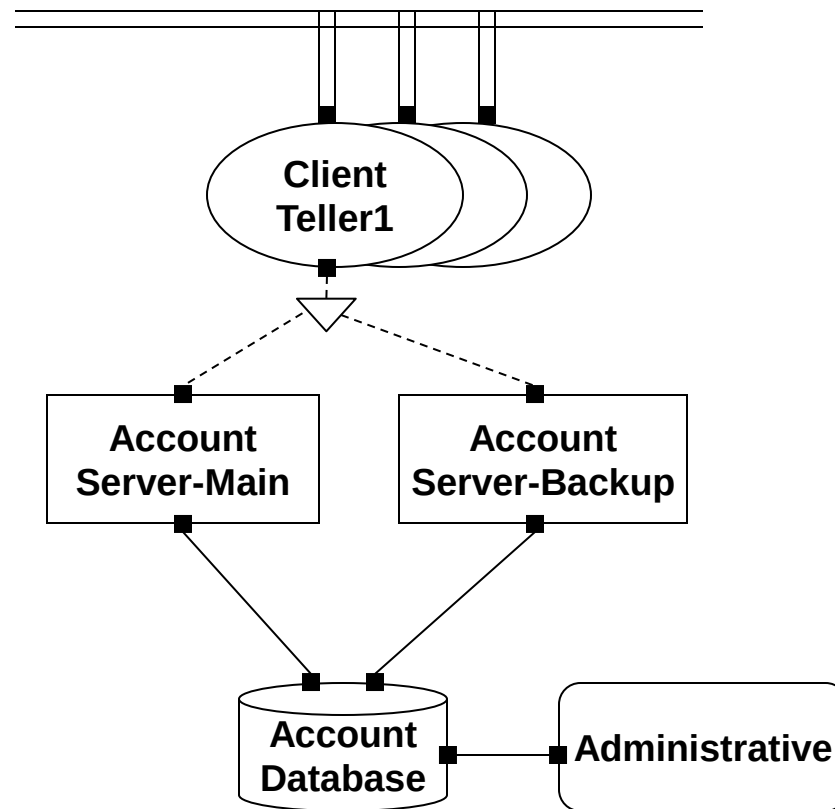
SEI View Model:

C&C view

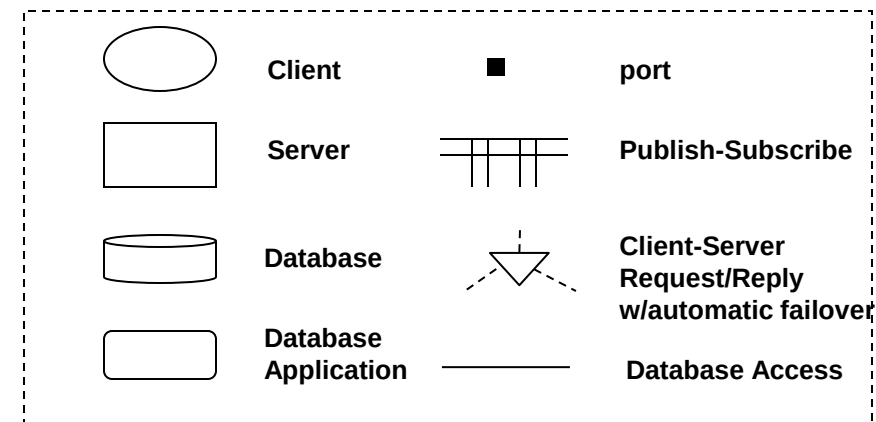
- A C&C view provides a picture of runtime entities and potential interactions
 - *Run time entities* represent **components**
 - Components represent principal processing units and data stores
 - Example
 - Processes, threads, objects, data stores, etc.
 - *Interactions* are captured as **connectors**
 - Complex forms of communication between components
 - Example
 - A protocol of communication between a client component and a server component
 - Event broadcast
 - N-way data synchronization
 - Components and connectors are associated with each other **to form a runtime system configuration**

SEI View Model: C&C View Example

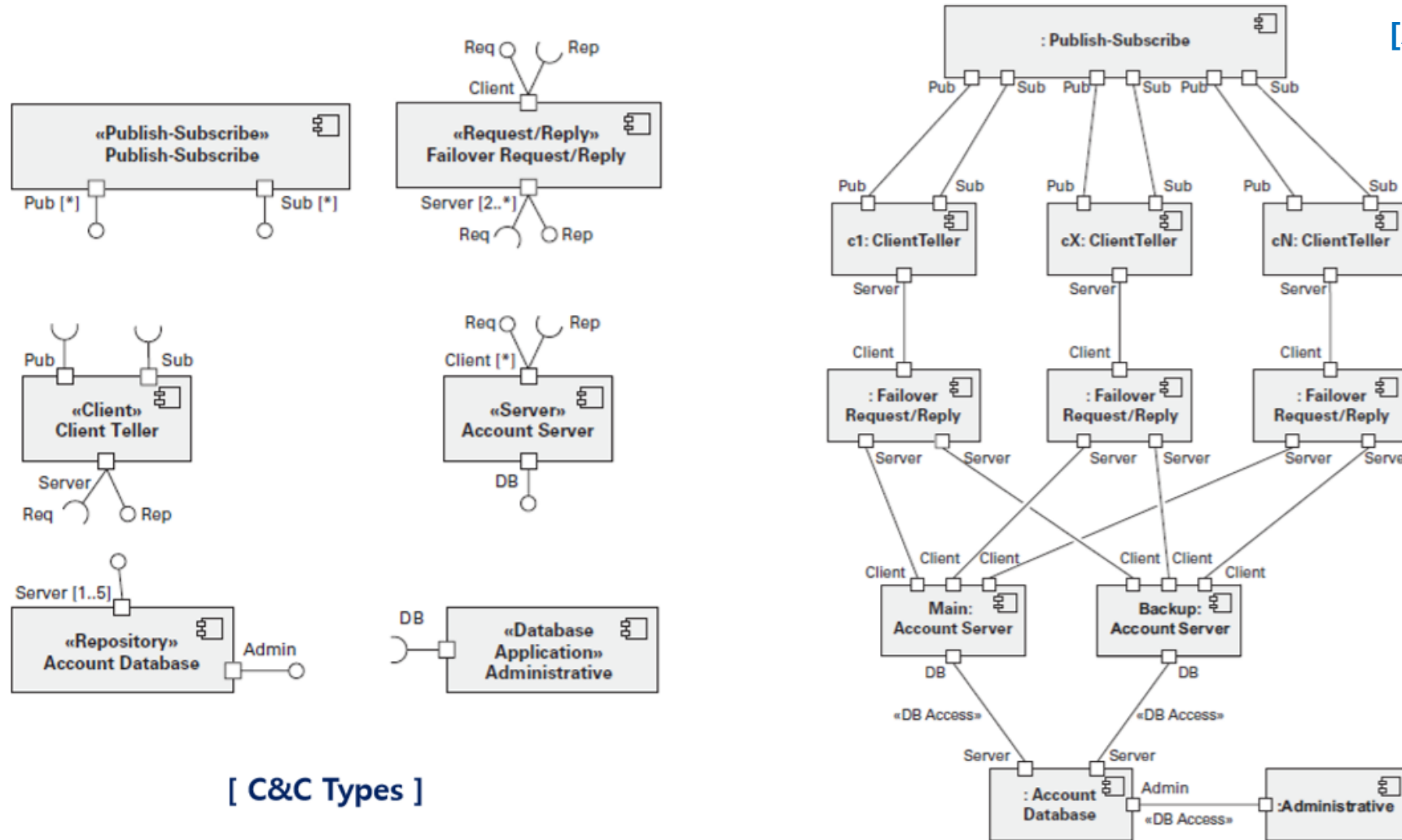
□ A bird's-eye-view



- A set of client tellers interact with the account repository servers
- If the main server goes down, the backup can take over
- The *Administrative* component allows an administrator to access and maintain the shared-data store

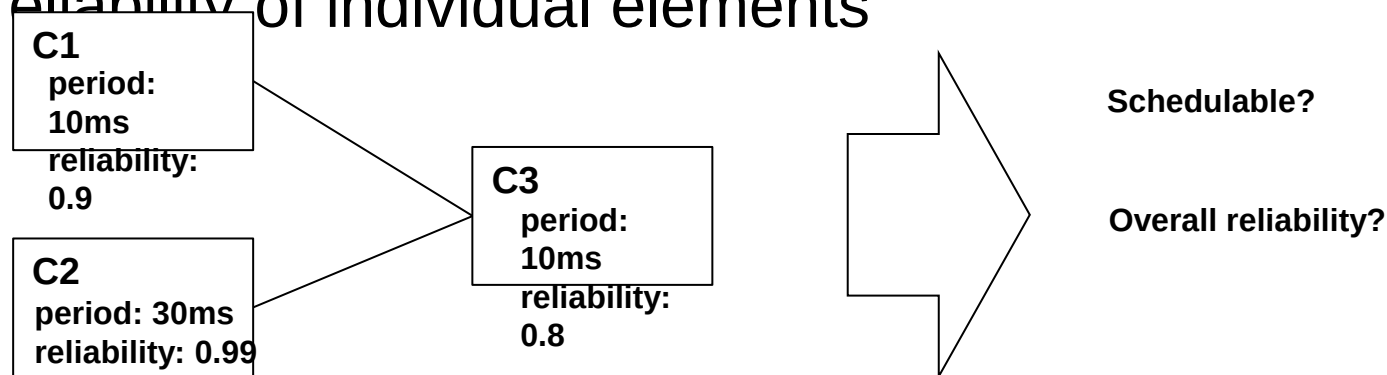


SEI View Model: C&C View Example



SEI View Model: What the C&C View is For

- ❑ Showing **how the system works**
- ❑ Helping architects **to reason about runtime system quality attributes**, such as **performance, reliability, and availability**
 - To determine whether the overall system can meet its real-time scheduling requirements, you will need to know the cycle time of each process
 - To estimate or calculating overall system reliability, you will need to know the reliability of individual elements

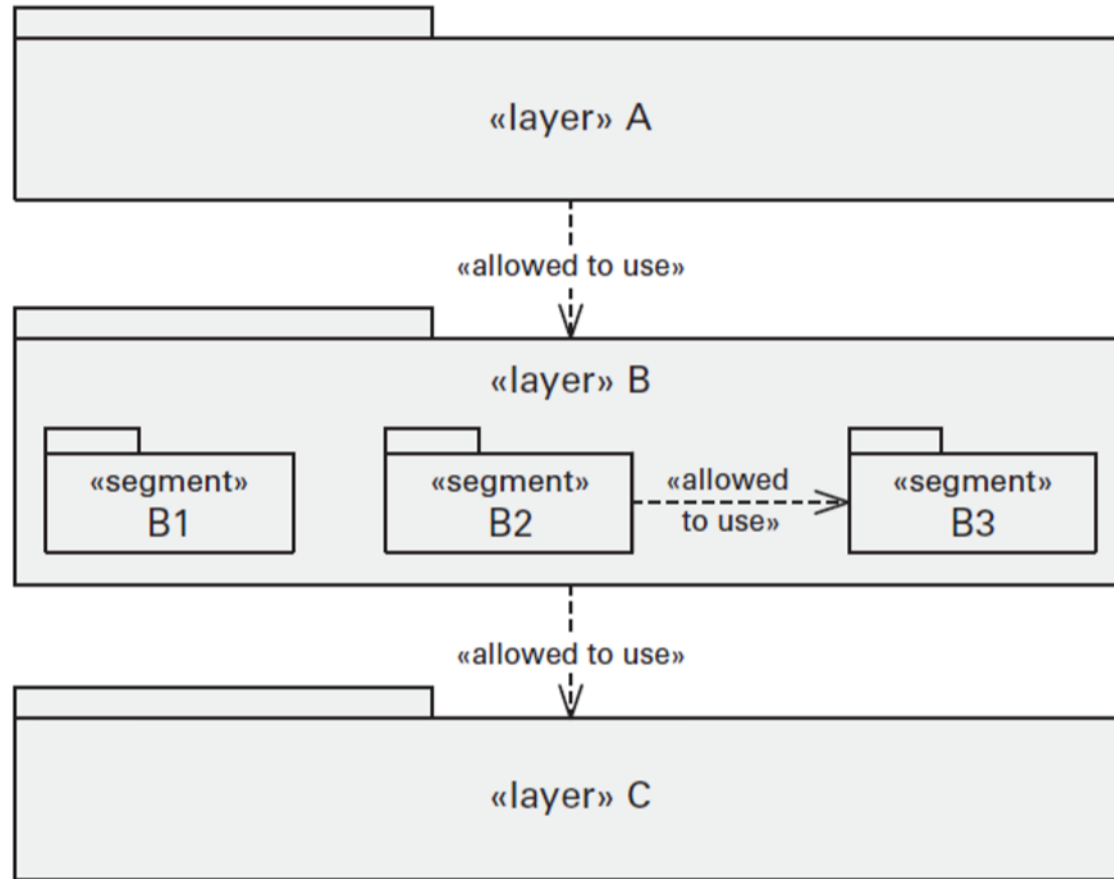


SEI View Model:

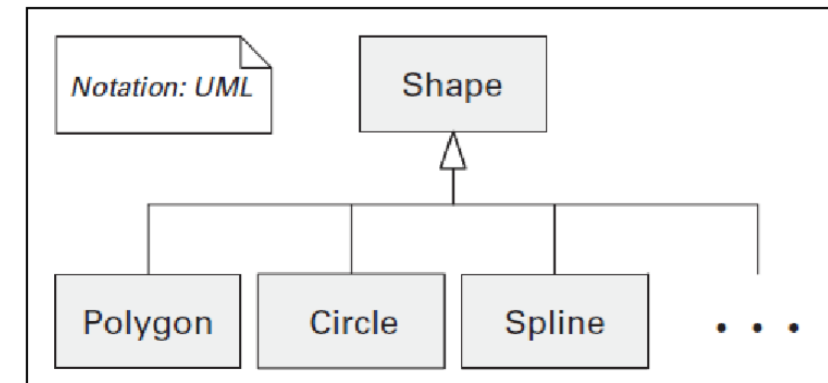
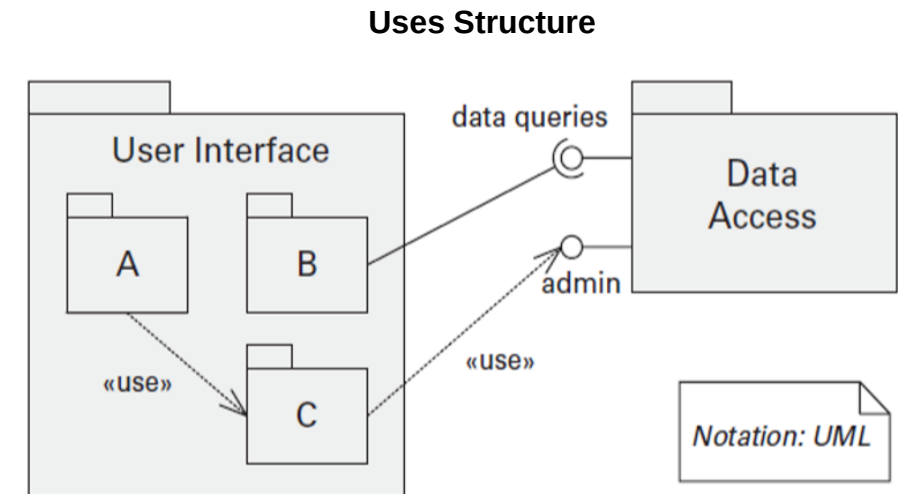
Module View

- ❑ A Module view describes **a set of modules** of a system and **the relations among them**.
- ❑ A module (the principal implementation unit)
 - An implementation unit of software that provides a coherent set of responsibilities
 - It can be *a class*, *a procedure*, *a layer* or any decomposition of the code unit
- ❑ Modules can be structured differently according to various module relations.

SEI View Model: Module View Example



Layered Structure



Generalization Structure

SEI View Model:

What the Module Views are for?

□ Module views are used for

■ Construction

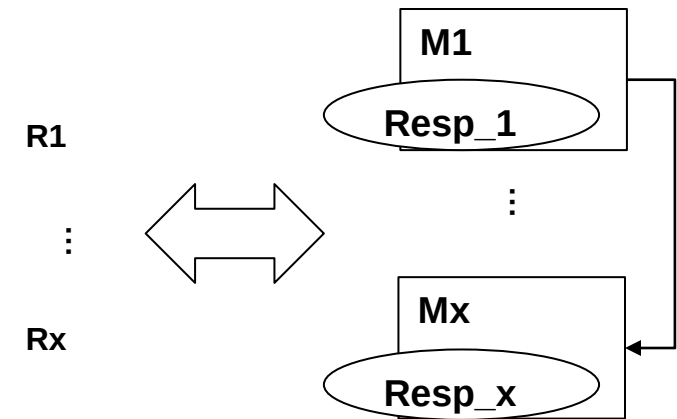
- A blueprint for the source code

■ Analysis

- Requirements traceability and impact analysis

■ Communication

- A top-down presentation of the system's responsibilities can help someone not familiar with the system understanding the system



□ Module views are not used for

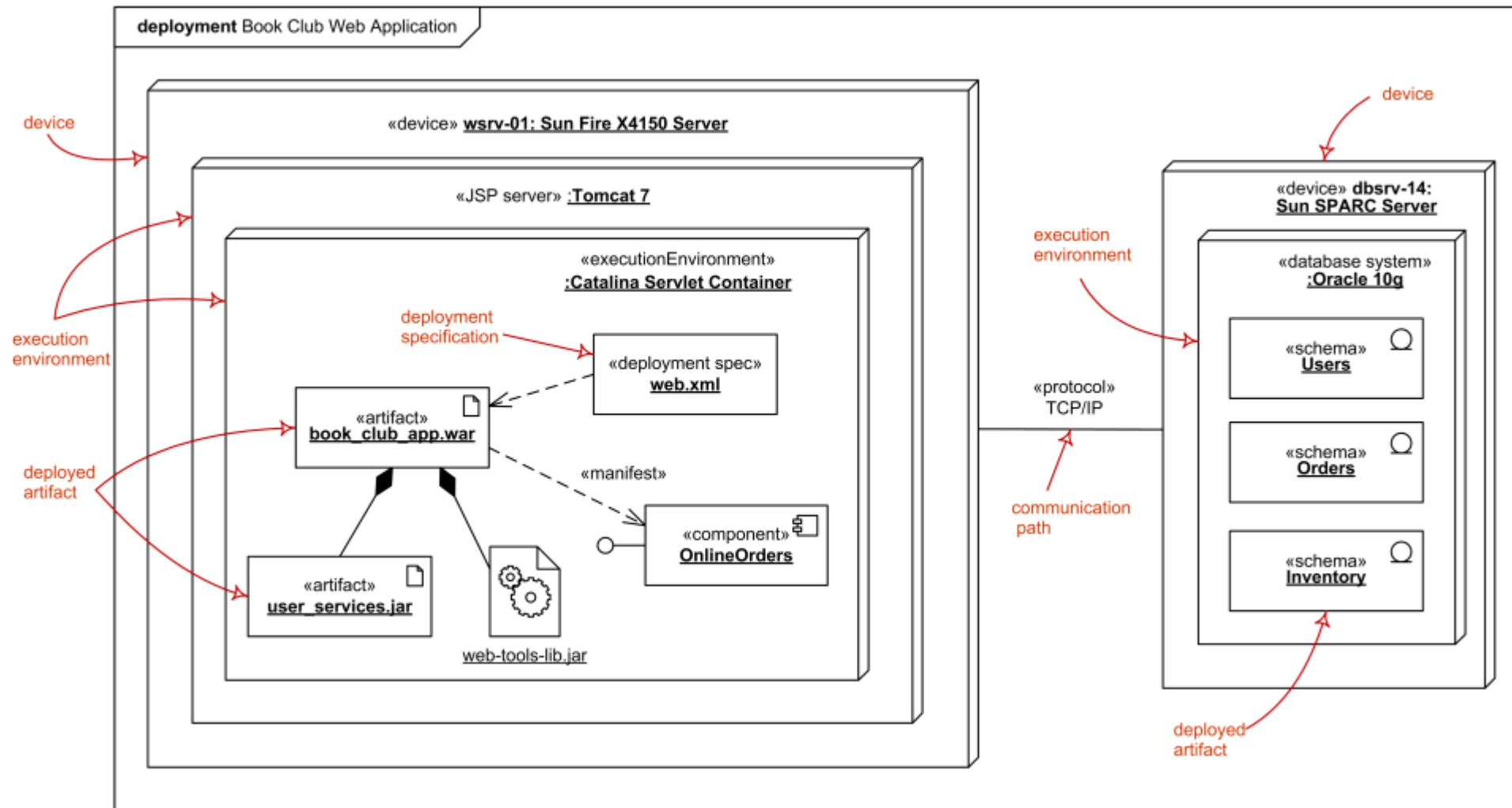
- Analysis of run time behavior and run time qualities (e.g., performance, reliability)

SEI View Model: Allocation View

- ❑ **An allocation view presents**
 - A mapping between **software elements** (from either a module view or a C&C view) and **environment elements**

- **Three allocation view styles**
 - **Deployment style**
 - ❑ The mapping of components and connectors onto the hardware on which the software executes
 - **Installation style**
 - ❑ The mapping between the software's components and structures in the file system of the production environment
 - **Work assignment style**
 - ❑ The mapping between the software's modules and the people, teams, or organizational work units

SEI View Model: Deployment Style Example



SEI View Model: What the Deployment Style is For

- ❑ A deployment style is used **for analyses of performance, reliability, and security**.
 - ❑ **Performance** is tuned by changing the allocation of SW to HW.
 - ❑ **Reliability** are directly affected by failed processing elements or communication channels.
 - ❑ **Security** are influenced by the configuration of the hardware and the allocation of software to it.
- ❑ A deployment view may be used **to support cost estimation** when evaluating purchasing options for hardware

Correspondence among the 3 view models

Views	4+1 View Model	SEI View Model		Siemens View Model
Logical/Conceptual View	Logical View	Module View		Conceptual View
Process/Concurrency View	Process View	C&C View		Execution View
Deployment View	Physical View	Allocation View	Deployment Style	
Installation View			Installation Style	
Work Assignment View			Work Assignment Style	
Development/Module View	Development View	Module View		Module View

Architecture Views

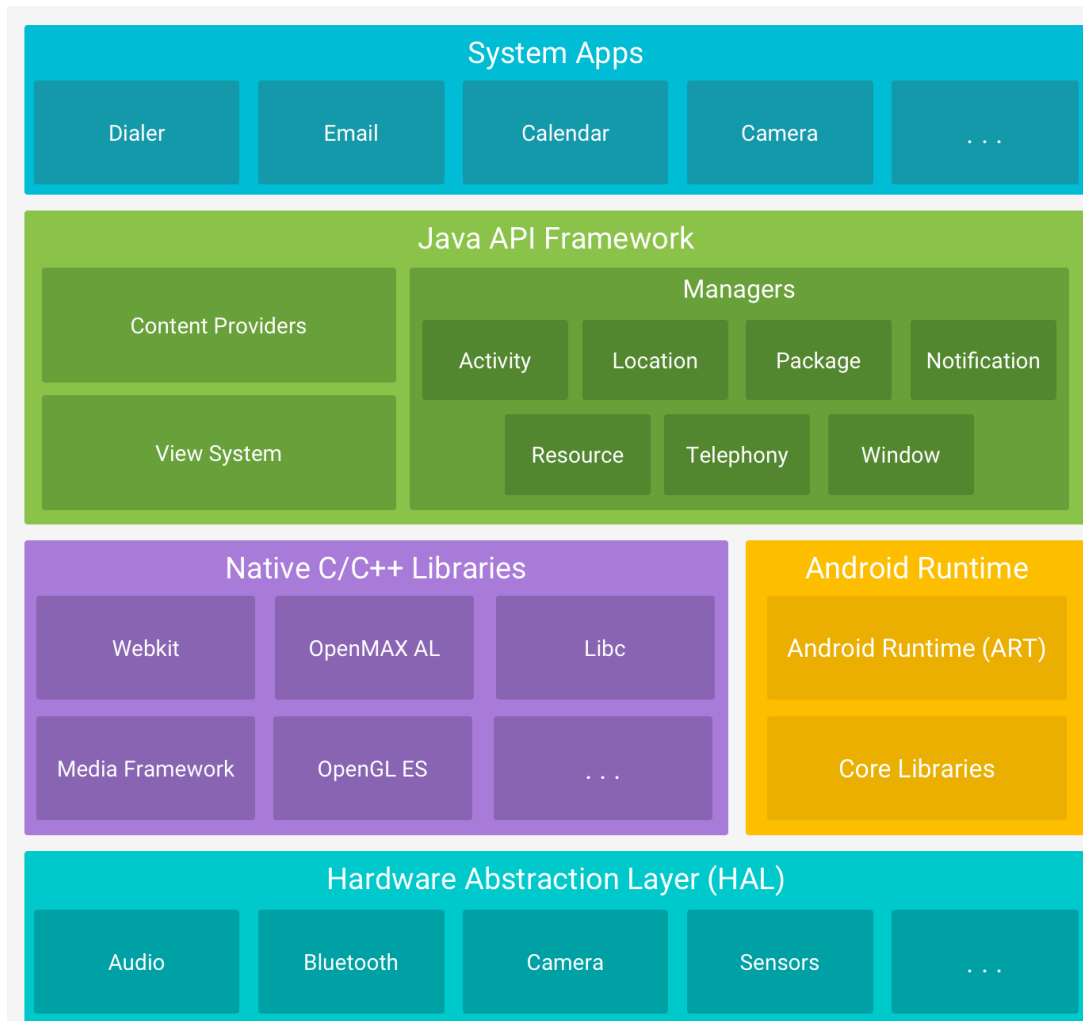
DOCUMENTING ARCHITECTURE

Purpose of Architecture Documentation

□ Architecture documentation serves

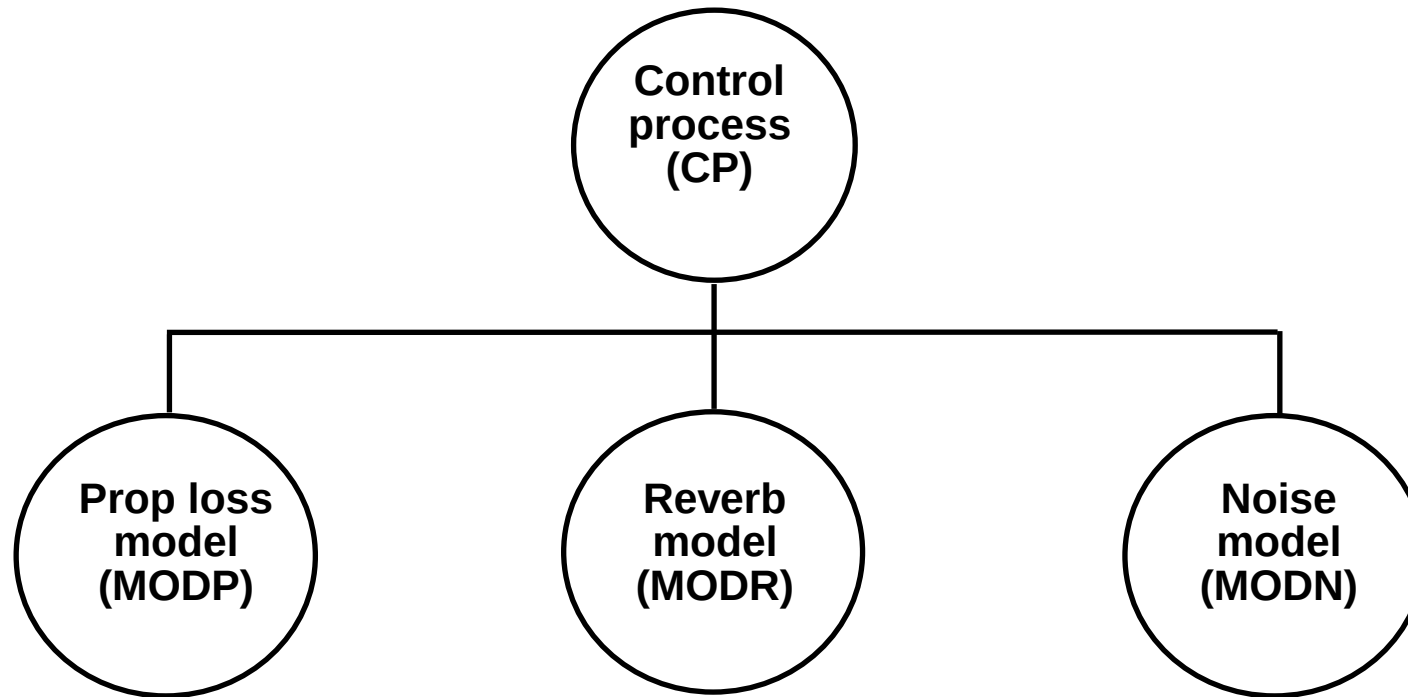
- as a means of education for new members of the team or the customer
- as a repository of important design decisions for the architect in the project's future
- as the basis for system construction. It tells implementers what to implement

Diagrams for Architecture Description



<https://developer.android.com/guide/platform>

Diagrams for Architecture Description



What's Wrong with the Diagram?

- ❑ **Ambiguous box-and-line diagrams**

- What do the circles or rectangles mean?
- What are interfaces or interactions among the elements?
- What is the significance of the layout?

- ❑ **No discussion of design alternatives with rationale**

- ❑ Box-and-line diagrams alone are not architectures; rather, they are a starting point

Basic Principles of Good Architectural Documentation

❑ **Graphics communicate semantics effectively**

- Elements have different shapes/colors and a legend to explain what these mean
- Connector types are clearly distinguished

❑ **Diagrams contain appropriate detail**

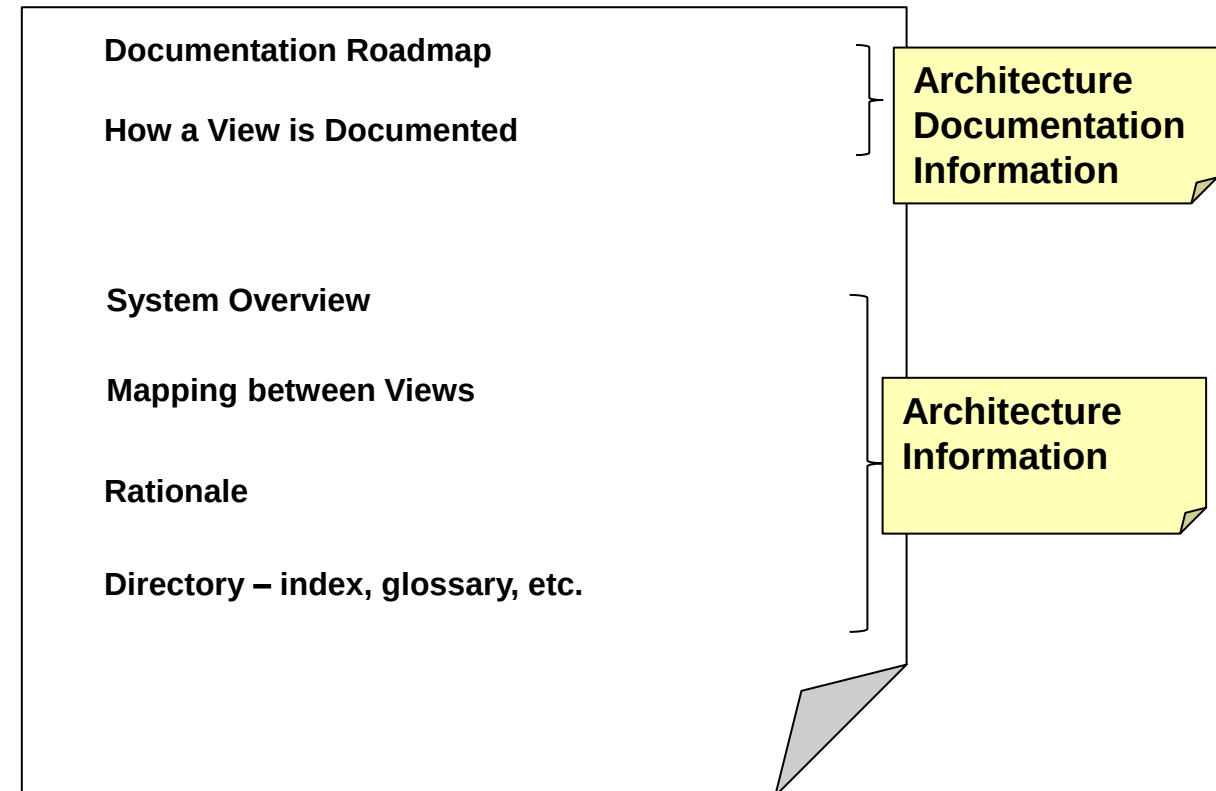
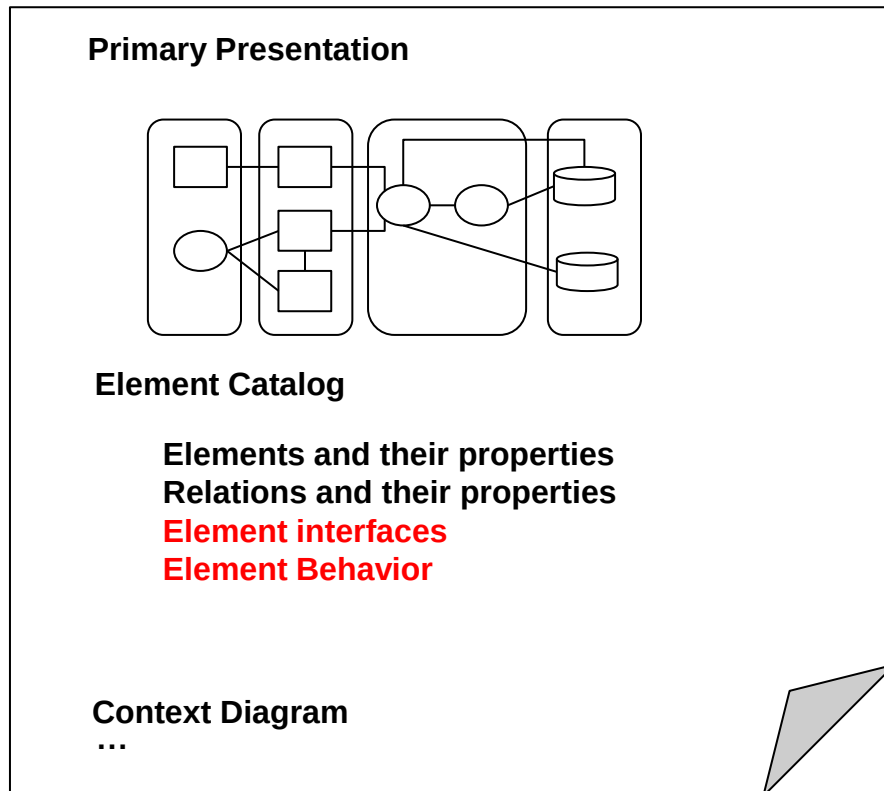
- Each view of architecture fits on a page
- Separation into views, where necessary
- Use of hierarchy

❑ **Clear distinction between views**

- Separation of concerns

Documenting Architecture

- Documenting an architecture is **a matter of documenting the relevant views and then adding documentation that applies to more than one view.**



Topics

- ❑ **Introduction to Architecture views**
 - 4+1 View Model
 - Siemens View Model
 - SEI View Model
- ❑ **Documenting Architecture**
 - **Documenting Architecture using SEI View Model**
 - ❑ **Documenting C&C View**
 - ❑ Documenting Module View
 - ❑ Documenting Allocation View
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ **Summary and Wrap-up**

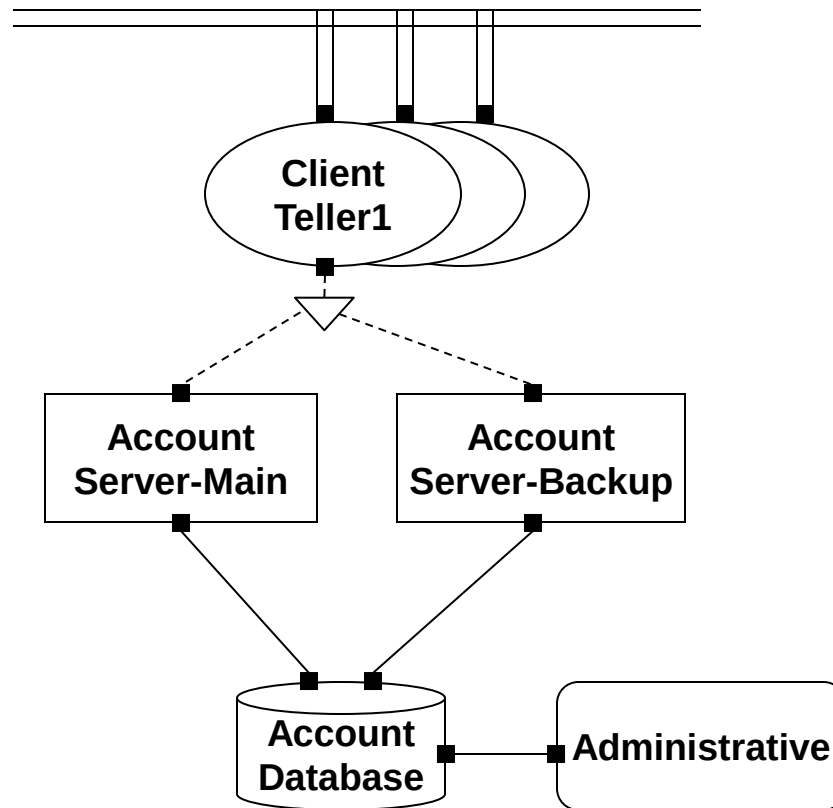
Documenting C&C Views

- ❑ A C&C view shows how components and connectors are associated with each other to form a runtime configuration

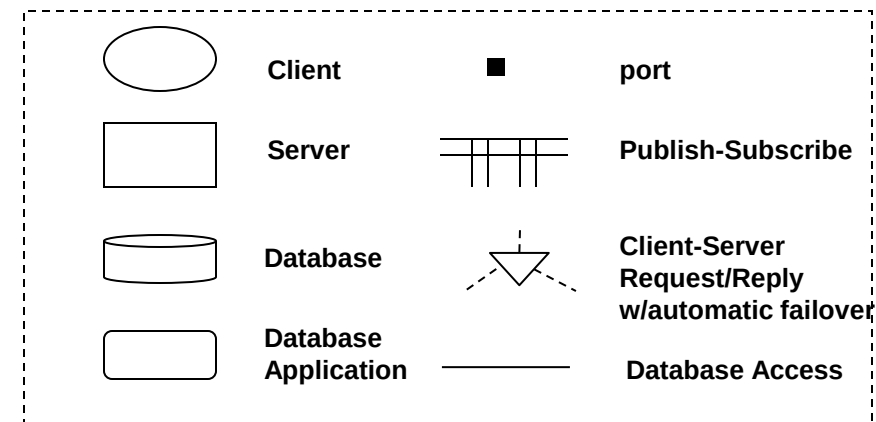
Elements	Component	• Principal processing units and data stores that are present at runtime • A component has a set of ports, as its interfaces
	Connector	• Interactions between components • A connector has a set of roles, as its interfaces
Relations	Attachment	• Component ports are associated with connector roles
	Interface Delegation	• Component ports may be associated the ports in an “internal” sub architecture

C&C View Example

□ A bird's-eye-view



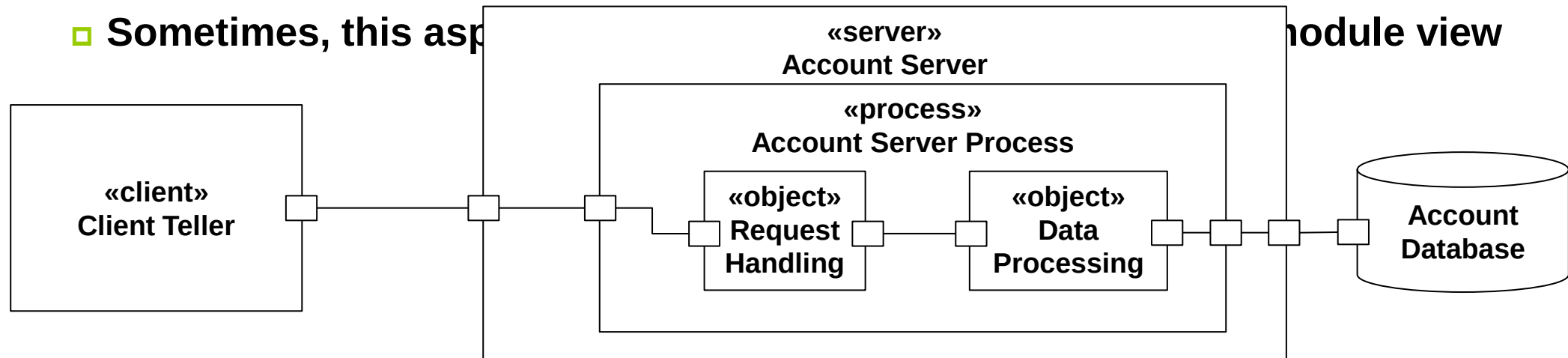
- A set of client tellers interact with the account repository servers
- If the main server goes down, the backup can take over
- The *Administrative* component allows an administrator to access and maintain the shared-data store



Documenting C&C View:

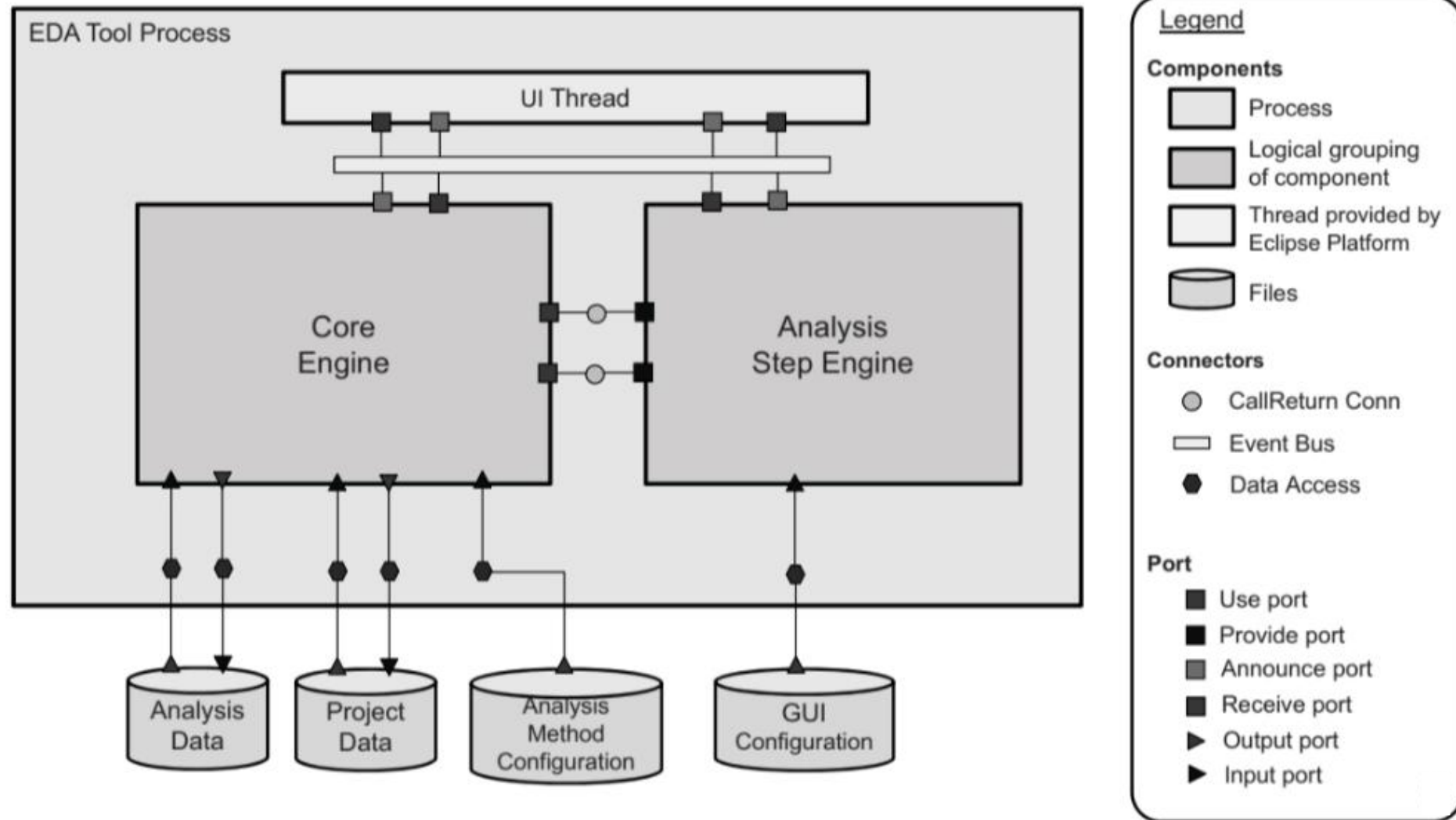
What are components in the C&C view?

- ❑ A C&C view shows **elements that have some runtime presence**.
 - **Computing and storage elements in a distributed system:** clients and servers, data store
 - **Concurrent units of execution :** processes or threads
 - **Principal processing units (logical or conceptual aspects):** objects
 - ❑ Sometimes, this aspect is called **module view**



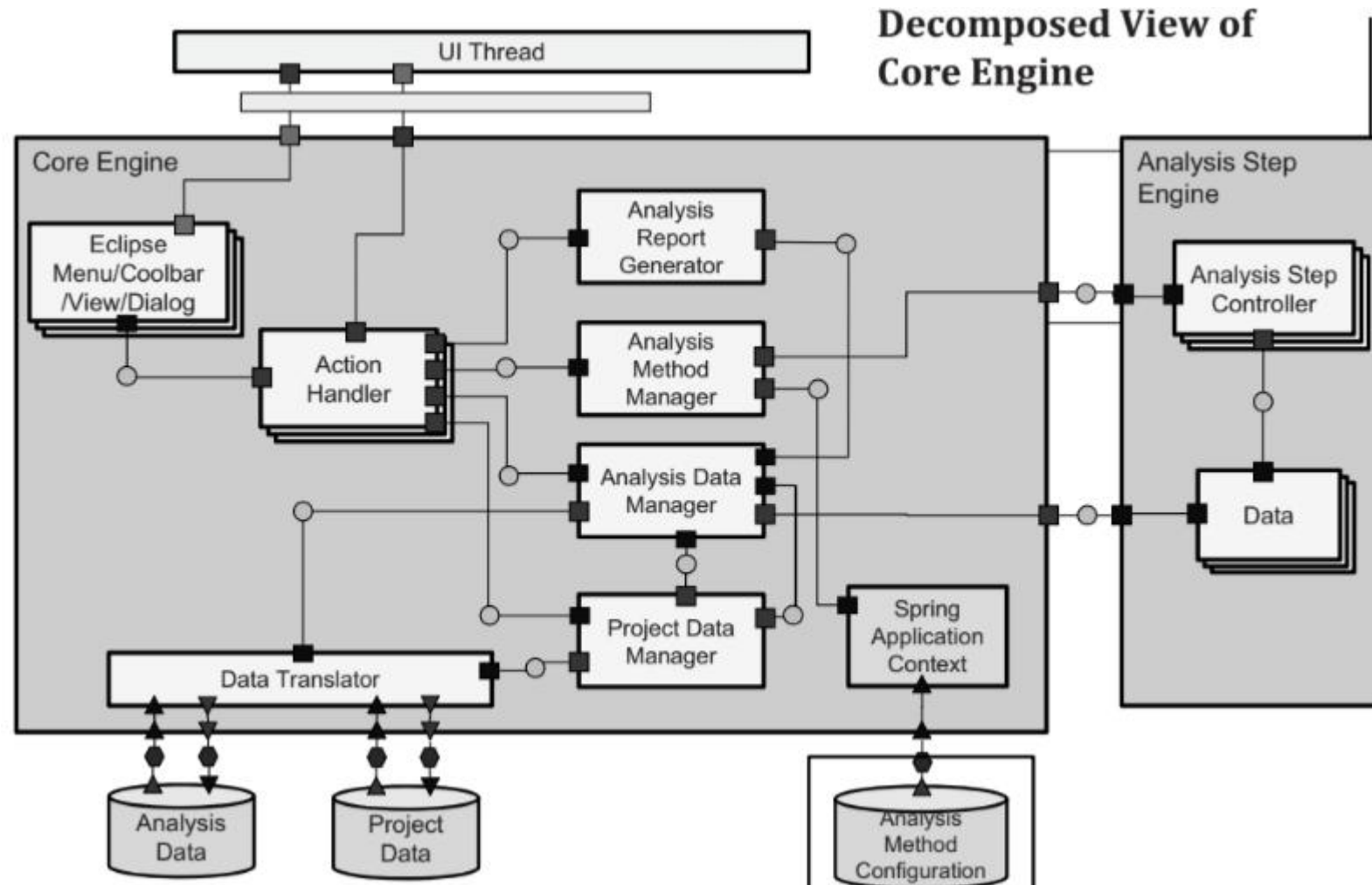
Documenting C&C View:

High level C&C View Example



Documenting C&C View:

Decomposed C&C View Example



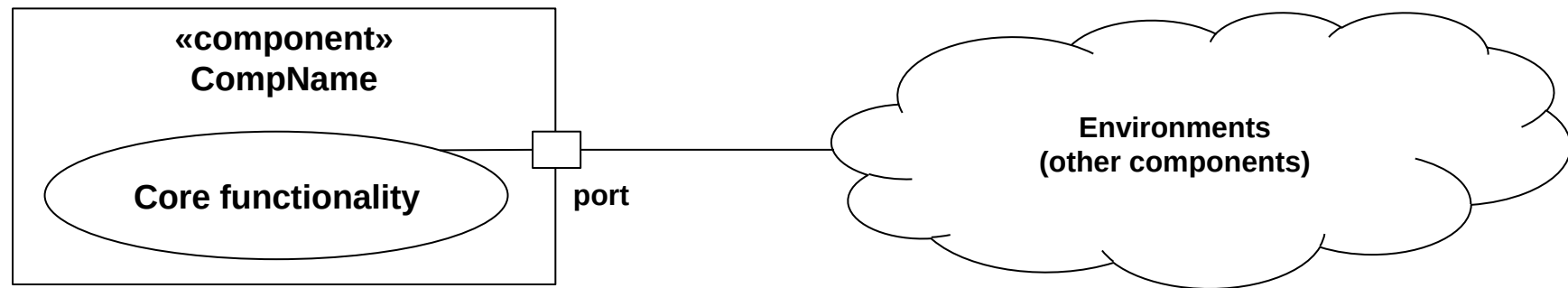
Documenting C&C View: How to define component

□ Component Name

- indicating the intended function of the component

□ A set of Ports (component interfaces)

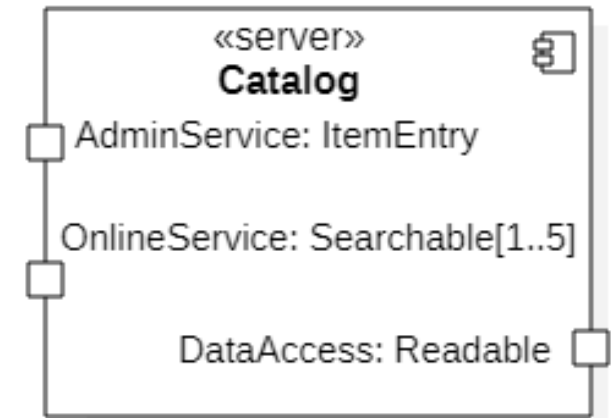
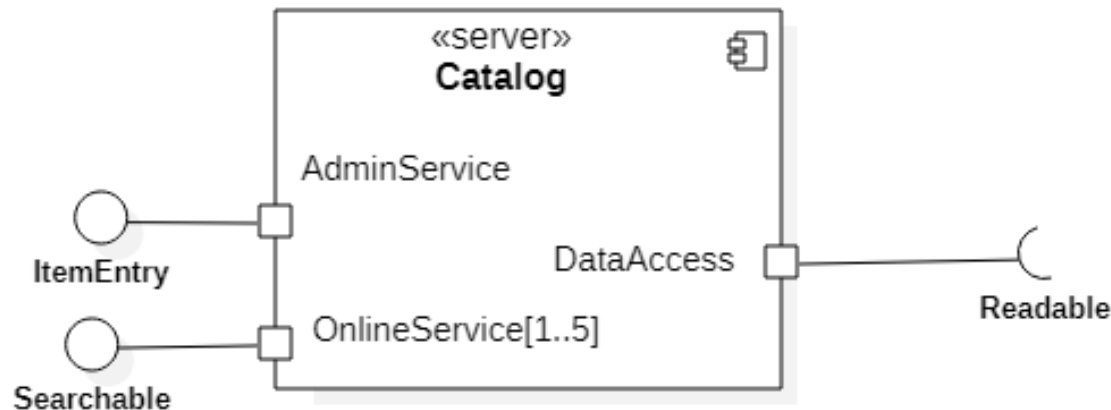
- A port defines **a specific point of potential interaction** of a component with its environment (e.g., other components).
 - Components can focus on their core functionalities, and **the interactions between components are managed through the ports**, ensuring a clear division of responsibilities.



Documenting C&C View: Component Type Notation

□ UML component

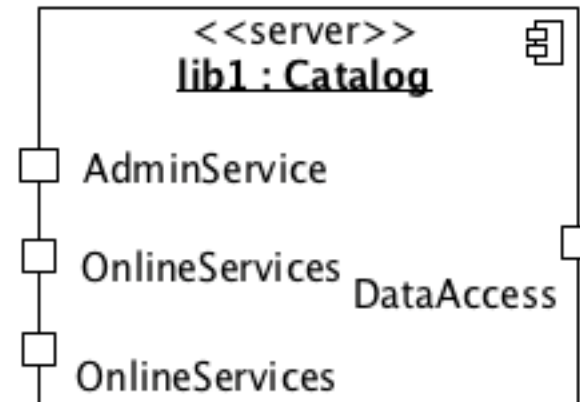
- Shows ports, possibly with name and port multiplicity
- May include required/provided services
- Uses stereotypes to related to component types



Documenting C&C View: Component Instance Notation

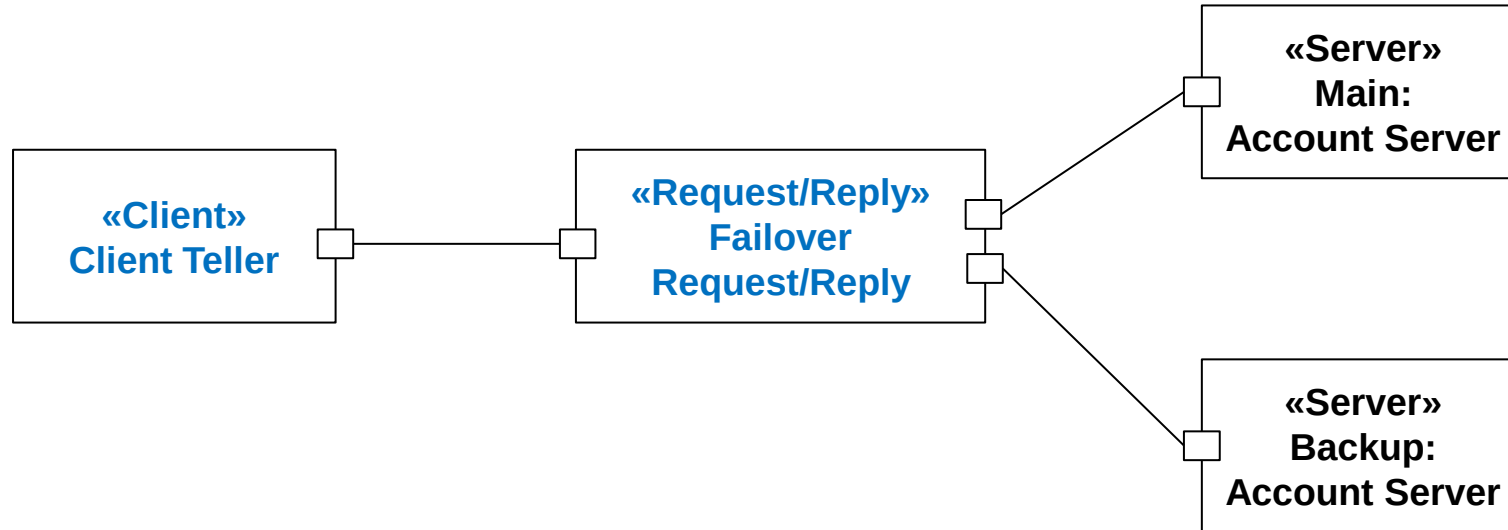
□ UML component instances

- Distinguished by use of ':'
 - Example of named component: "lib1:Catalog"
 - Example of anonymous component: ":Catalog"
- Show ports
 - Should be consistent with type
 - Multiple ports of the same type



Documenting C&C View: Component Modeling Guidelines

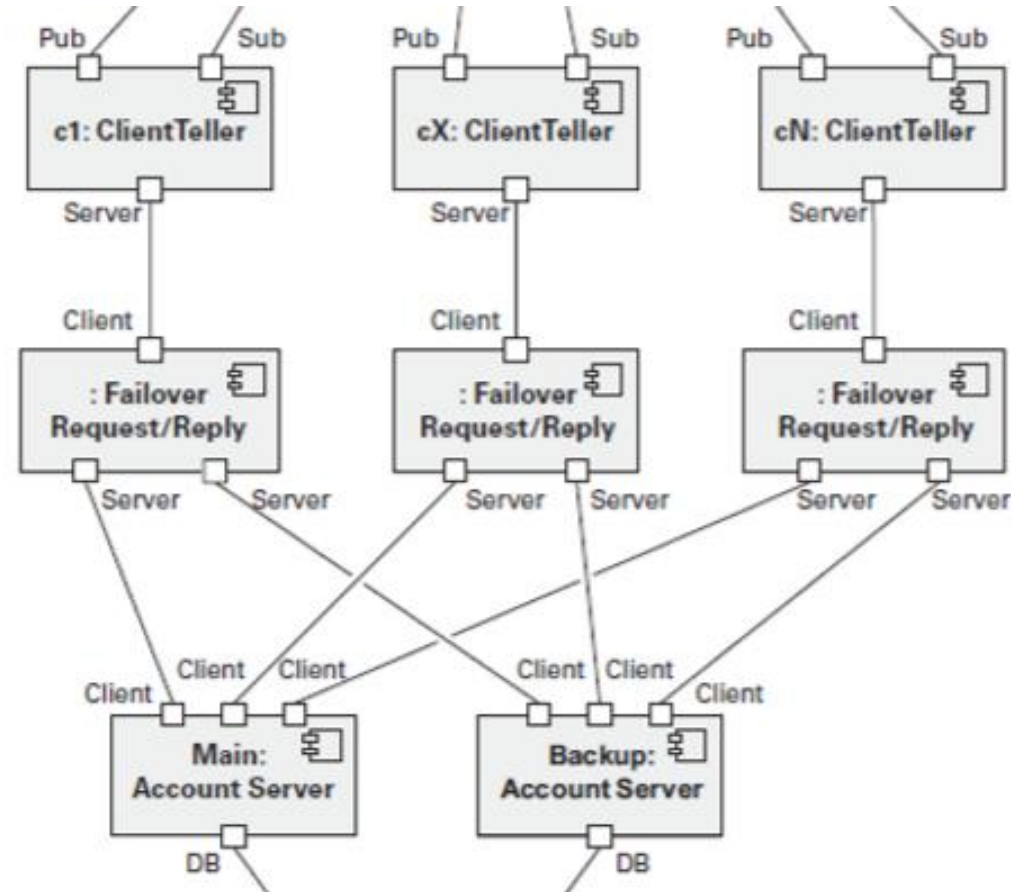
- ❑ **Don't mix component types and component instances in the same diagram**



Documenting C&C View: Component Modeling Guidelines

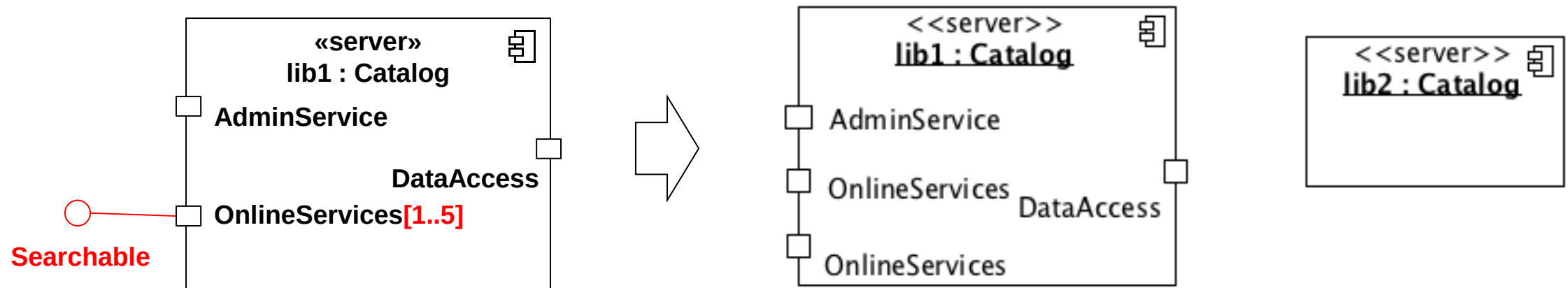
□ Use named ports on components

- unless it is obvious which ports are which
- Naming convention
 - The name of a port can be the role of the environmental element interacting with its containing component



Documenting C&C View: Component Modeling Guidelines

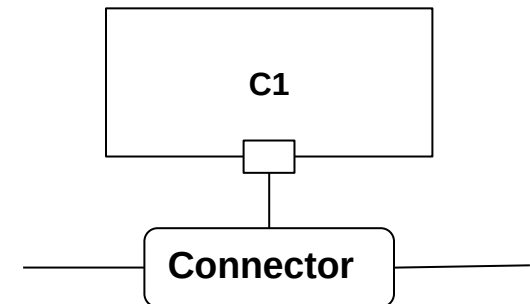
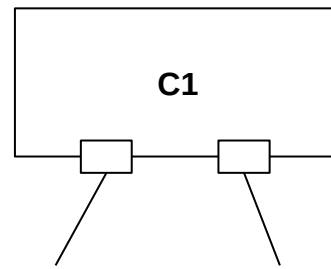
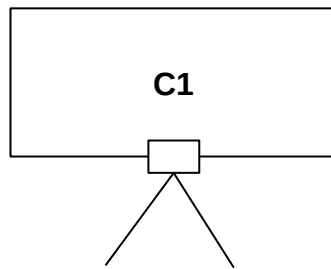
- ❑ Don't use interfaces (provides/requires lollypops) on component instances
 - Only component types
- ❑ Don't use multiplicity on ports of component instances



Documenting C&C View: Component Modeling Guidelines

❑ Three ways to handle multiple components interacting with a component

1. Component C1 has a single interface through which all its environments interact with it
2. Component C1 has a separate interface for each environment
3. Component C1 has a single interface attached to a connector that handles the mediation between the component and its environment



Documenting C&C Views

- ❑ A C&C view shows how components and connectors are associated with each other to form a runtime configuration

Elements	Component	• Principal processing units and data stores • A component has a set of ports, as its interfaces
	Connector	• Interactions between components • A connector has a set of roles, as its interfaces
Relations	Attachment	• Component ports are associated with connector roles
	Interface Delegation	• Component ports may be associated the ports in an “internal” sub architecture

Documenting C&C View: Connectors

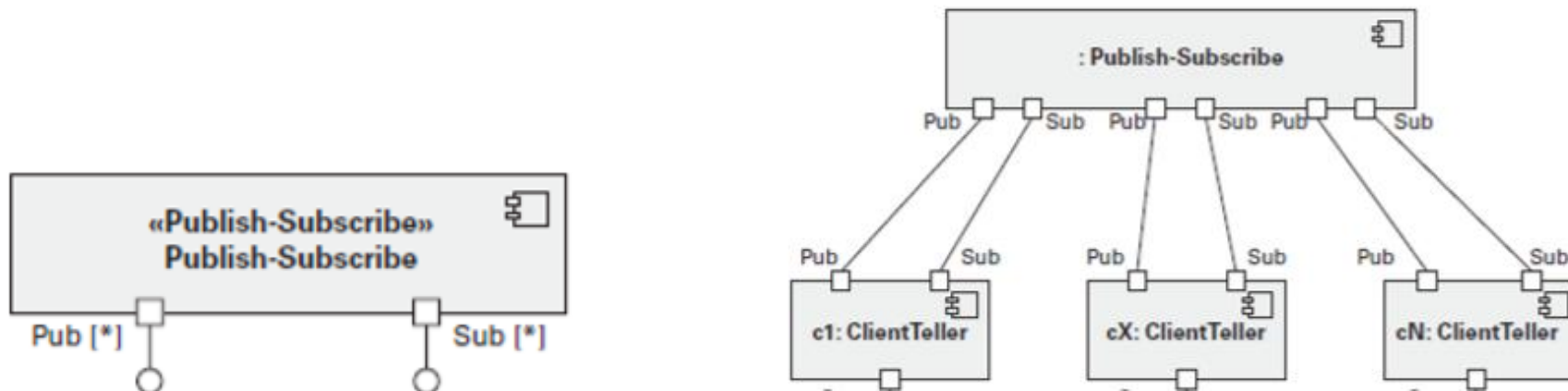
❑ Connector Name

■ indicating the nature of its interactions

- ❑ E.g., service invocation, event multicast, a transaction-oriented communication channel

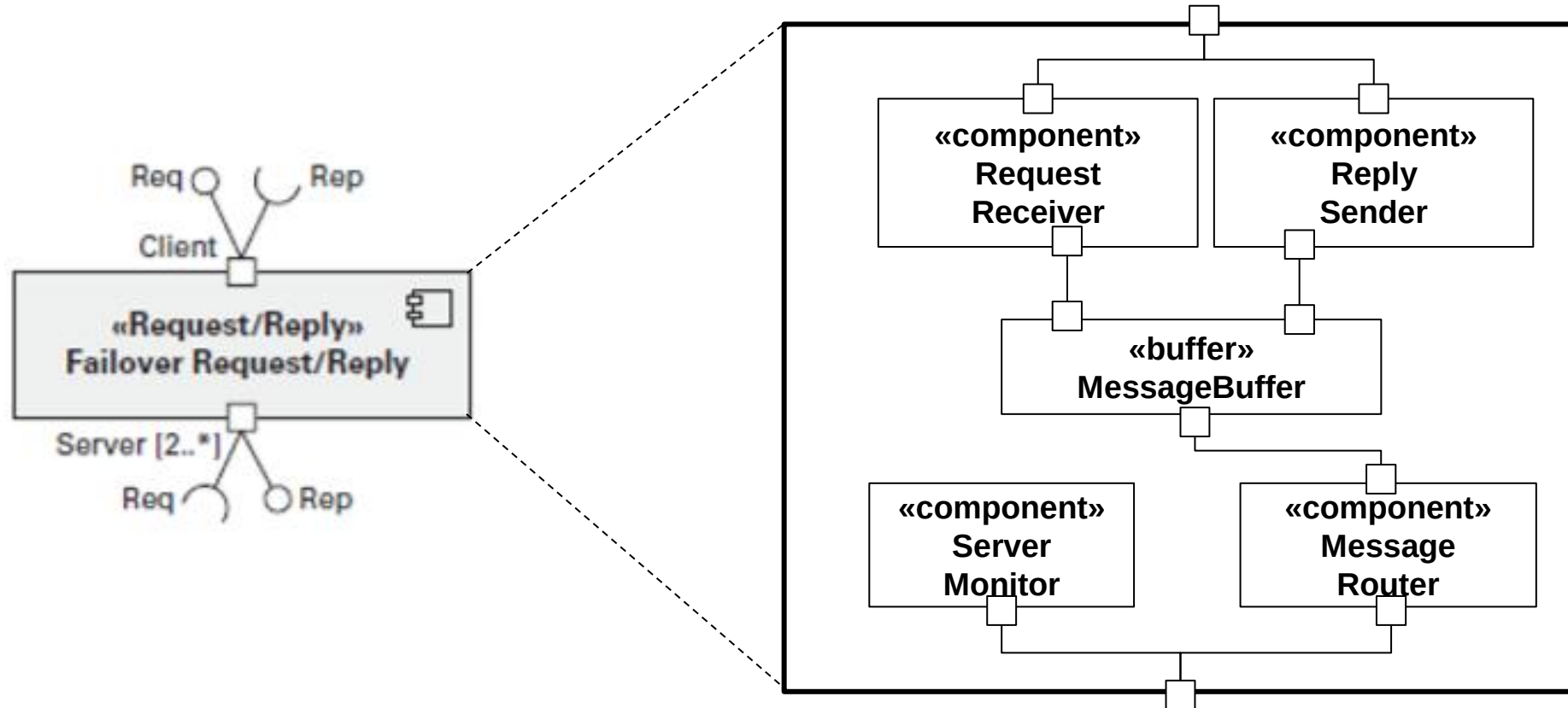
❑ A set of roles (Connector interfaces)

- A role defines how the connector is used by components to carry out interaction.
- A role may have a name and multiplicity.
 - ❑ Example (Publish-Subscribe connector)



Documenting C&C View: Connectors

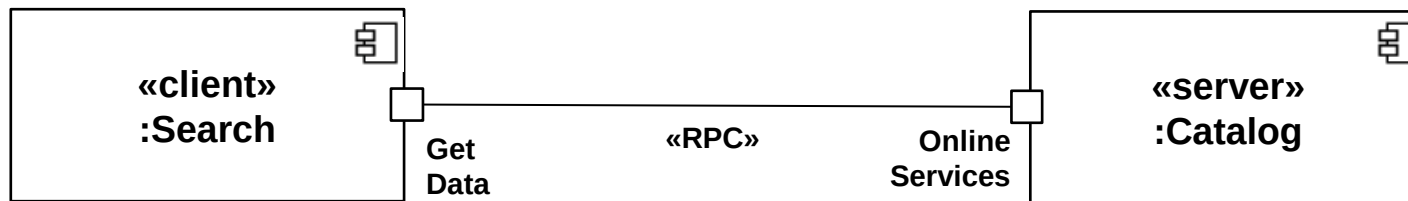
- ❑ Complex connectors may be decomposed into collections of components and connectors



Documenting C&C View: Connector Notations

□ UML connector

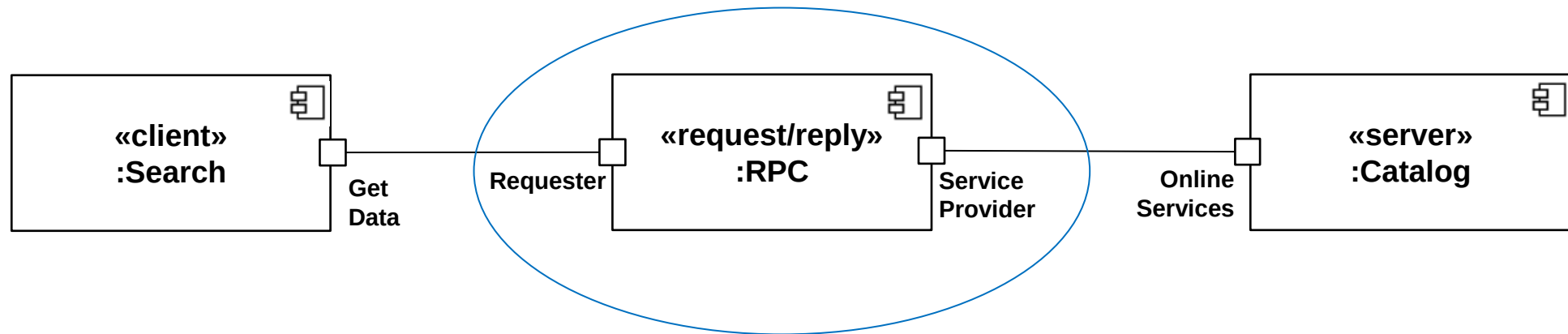
- Simple line connecting ports on the respective components
- Adding a stereotype to denote the connector's type
- UML connectors cannot have sub structure, properties, or behavioral descriptions.
- Connector roles can be indicated by labeling the connector ends



Documenting C&C View: Connector Notations

□ UML Component

- Useful if the connector is complex
 - E.g., publish/subscribe communication
- Can explicitly indicate connector roles



Documenting C&C View: Connector Notations

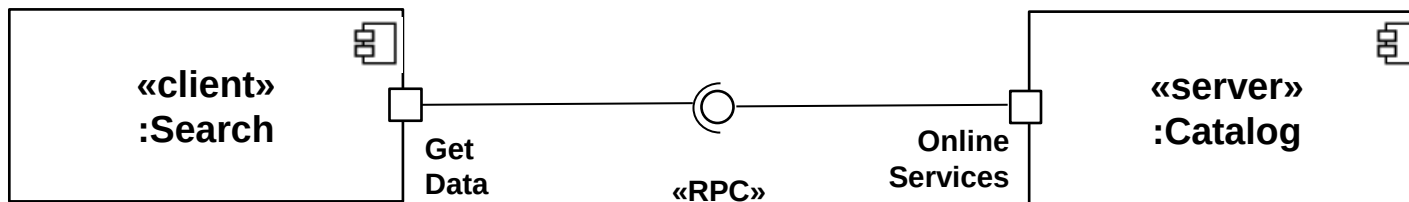
□ Assembly connectors

■ Use only when you have a call-return connector

- Two ports are connected, when the required interface of one port must be compatible with the provided interface of the other port.

■ This option is less useful when connectors do not match a provided/required intuition

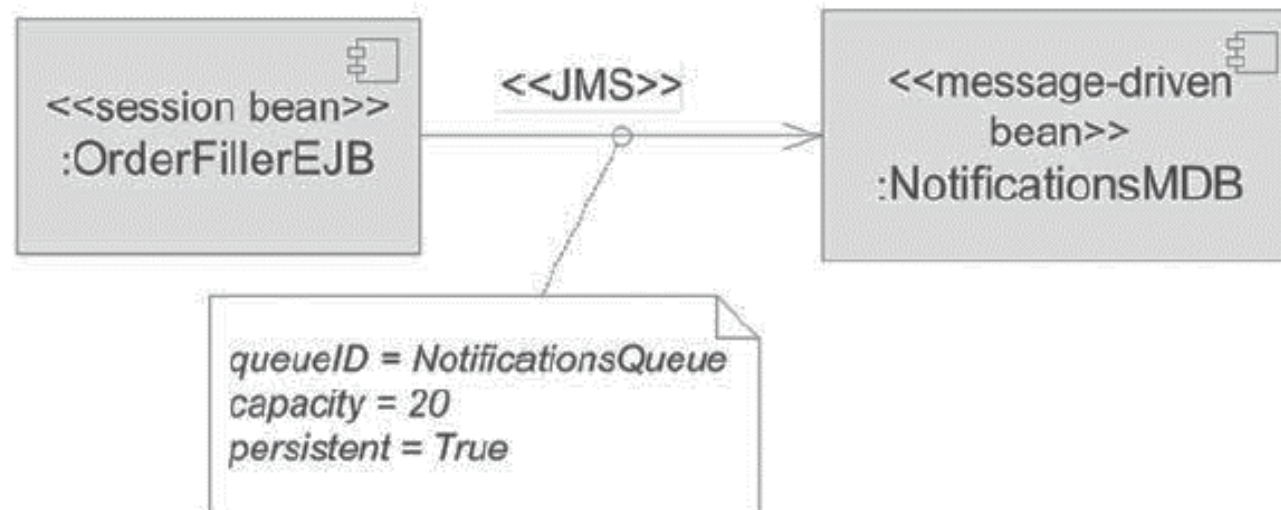
- For example, when an input port of a filter in a pipe-and-filter architecture does not clearly map to either a provided or required interface



Documenting C&C View: Connector Notations

□ Tagged values

- Allows for addition of more information
- Create a stereotype for the connector type and define attributes (tagged values) that become associated with the stereotype



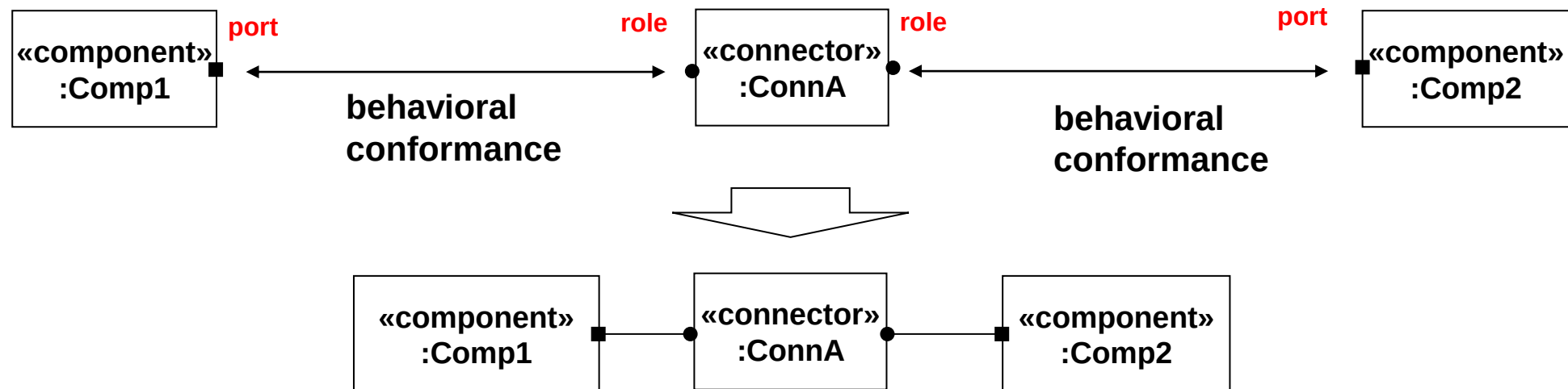
Documenting C&C Views

- A C&C view shows how components and connectors are associated with each other to form a runtime configuration

Elements	Component	• Principal processing units and data stores • A component has a set of ports, as its interfaces
	Connector	• Interactions between components • A connector has a set of roles, as its interfaces
Relations	Attachment	• Component ports are associated with connector roles
	Interface Delegation	• Component ports may be associated the ports in an “internal” sub architecture

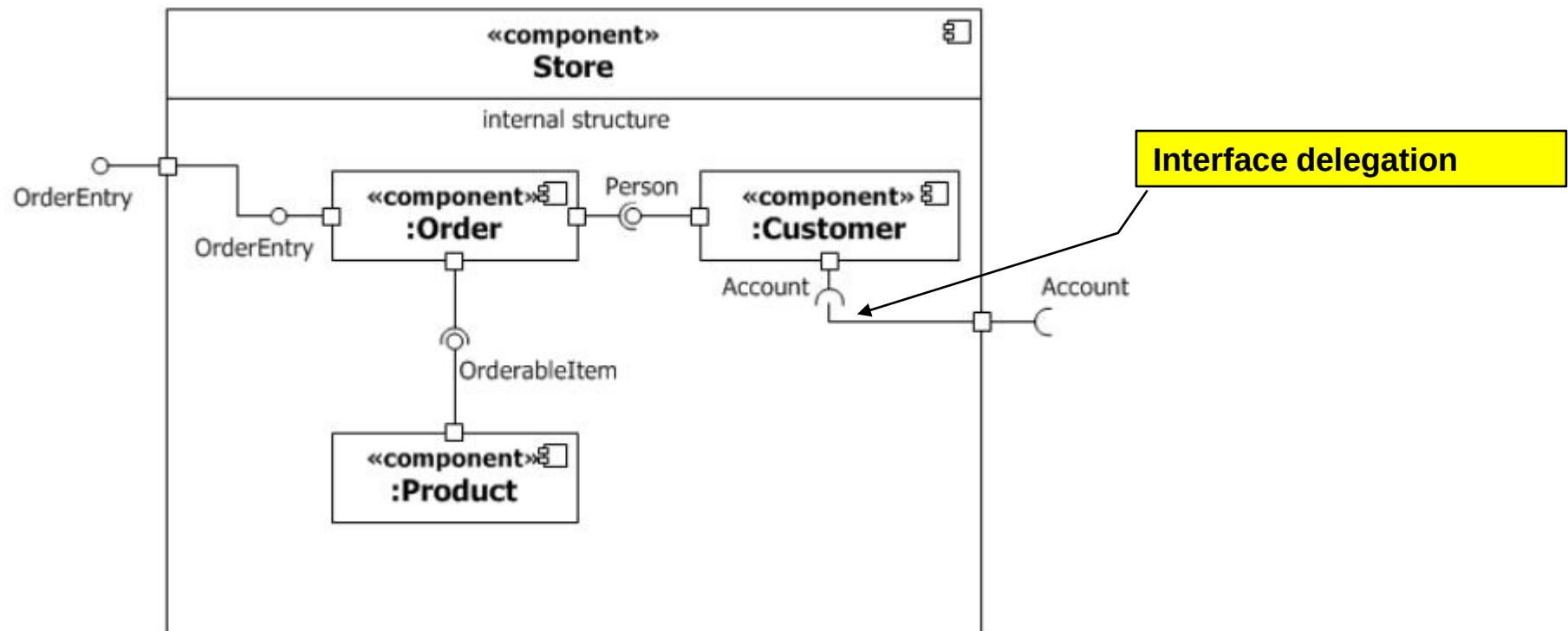
Documenting C&C View: Attachment Relations

- **Indicating which connectors are attached to which component**, thereby defining a system as a graph of components and connectors
 - a component **port** is attached to a connector **role**,
if a port's protocol is consistent with the behavior expected by the **role** to which it is attached.

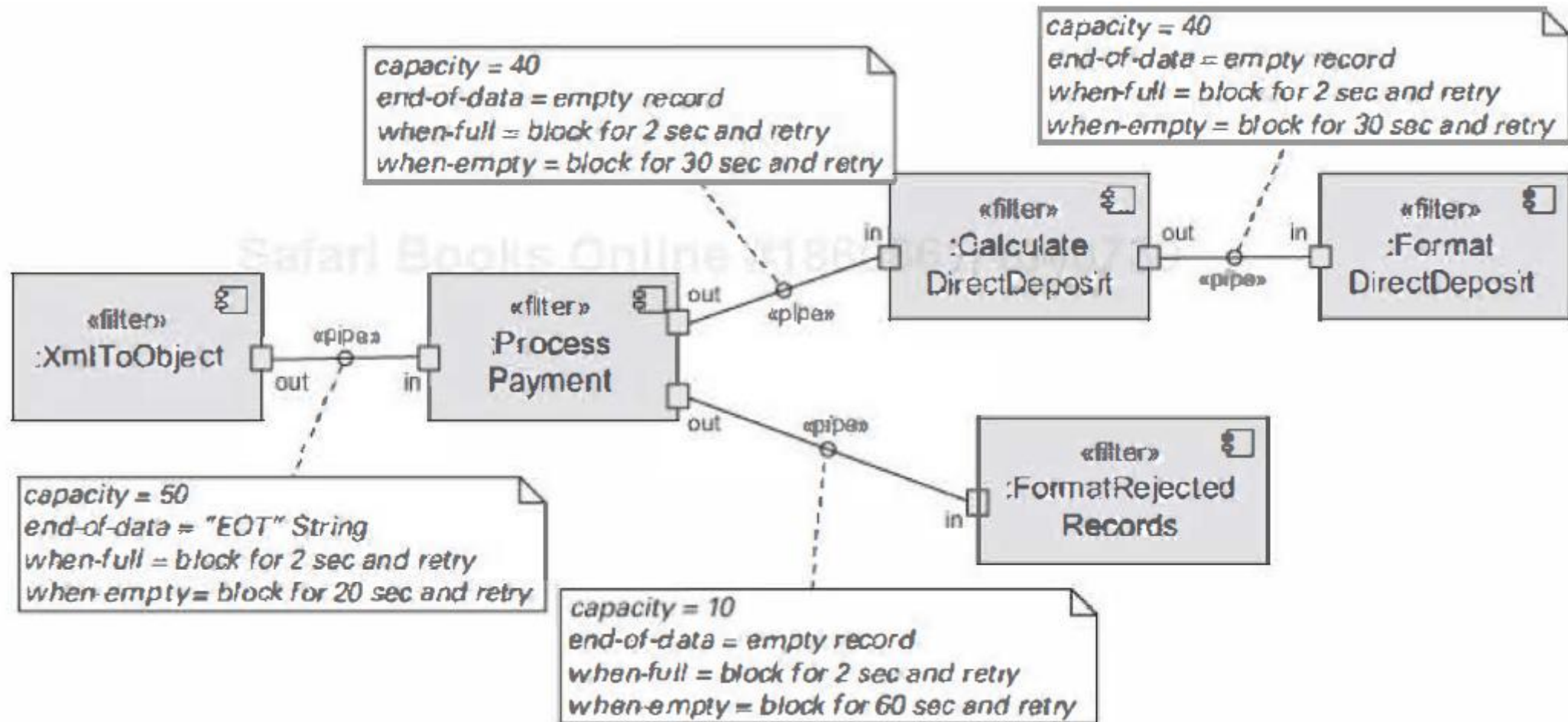


Documenting C&C View: Interface Delegation Relation

- Relating internal ports/roles to external ports/roles for components/connectors .

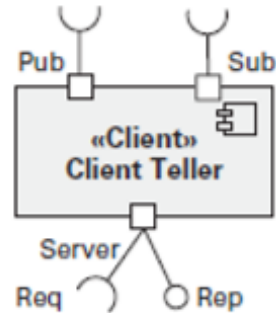


Documenting C&C View: Data Flow Style

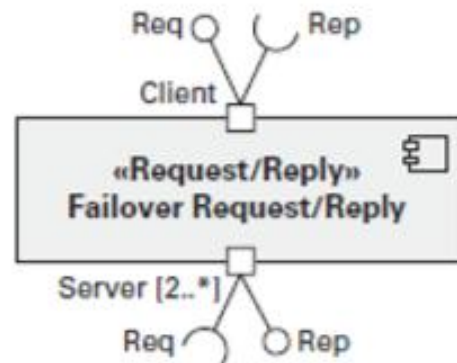


Documenting C&C View: Call-Return Style

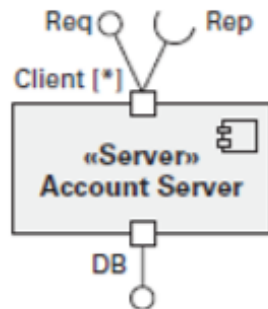
Component Type



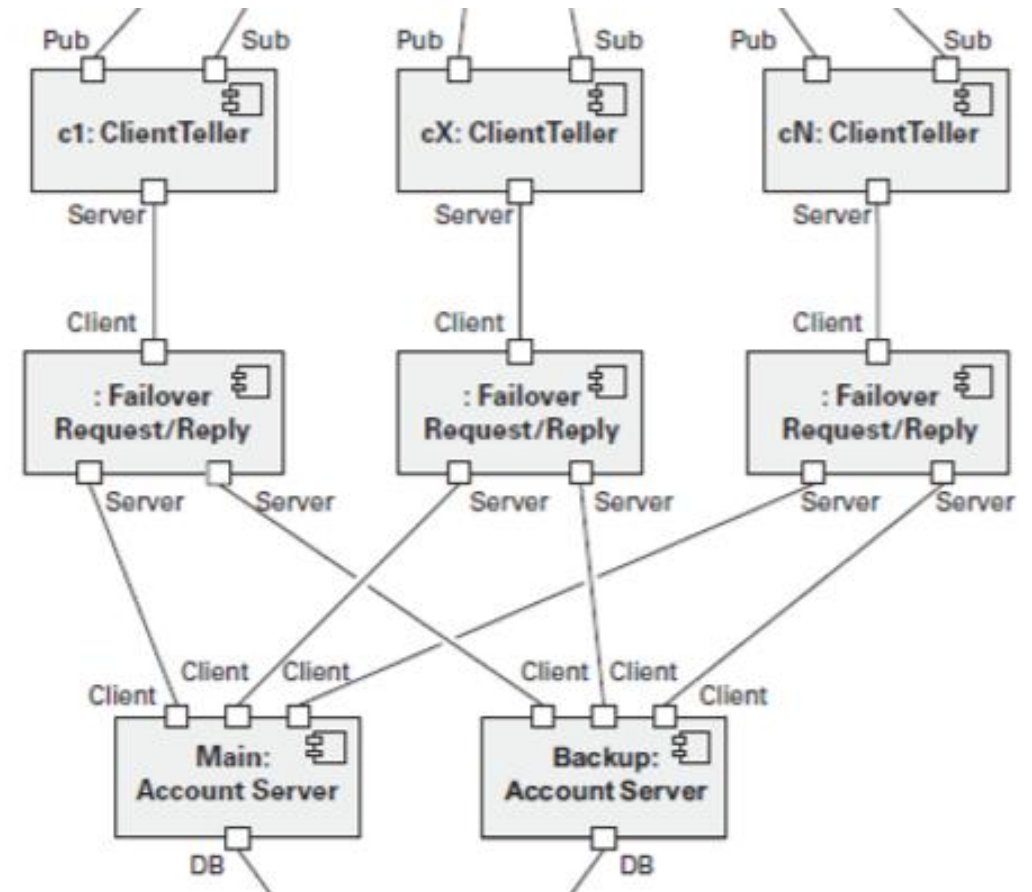
Connector Type



Component Type



Call-Return Style Architecture

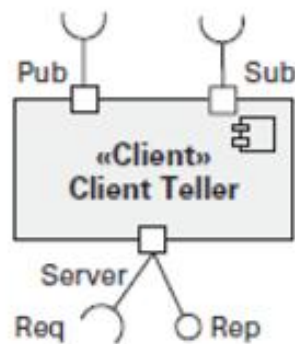


Documenting C&C View: Event Based Style

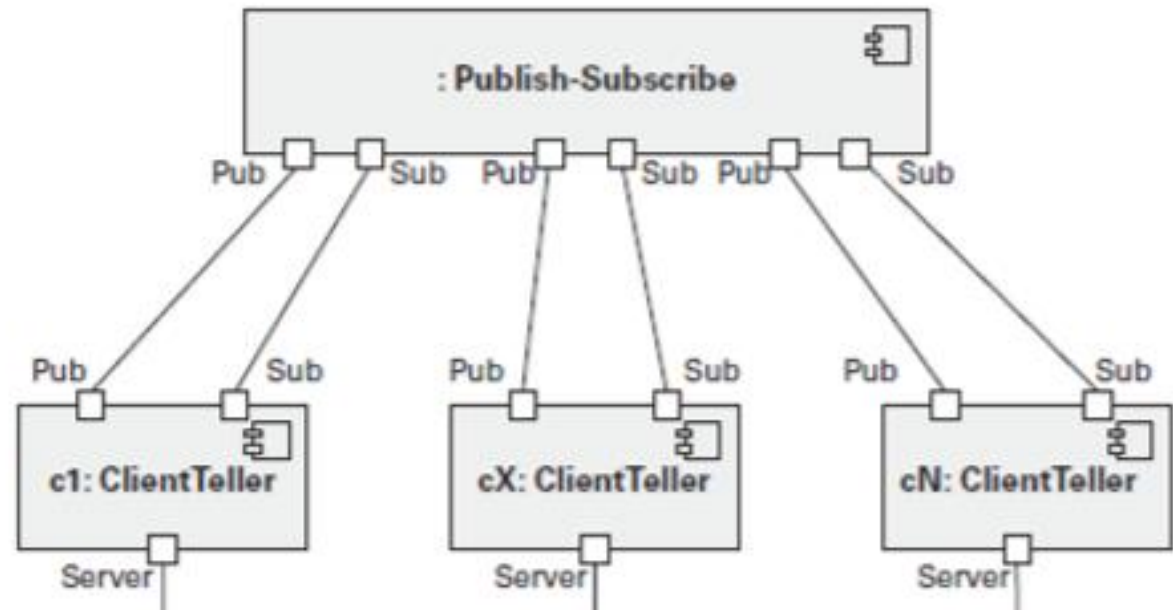
Connector Type



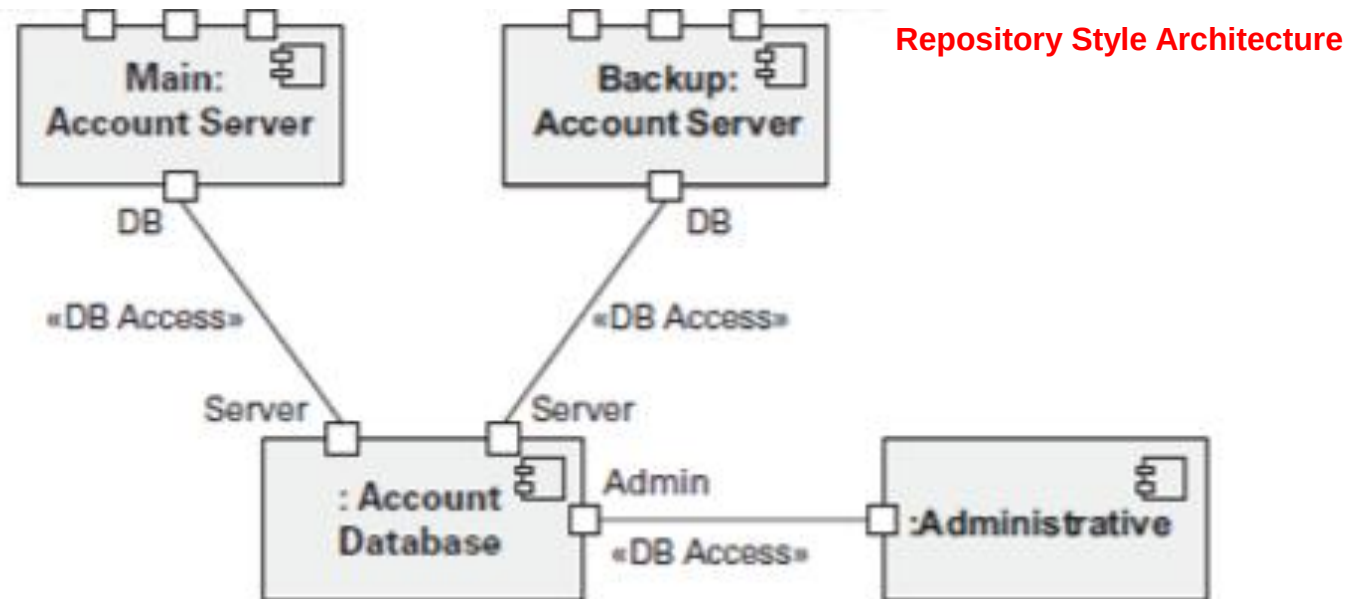
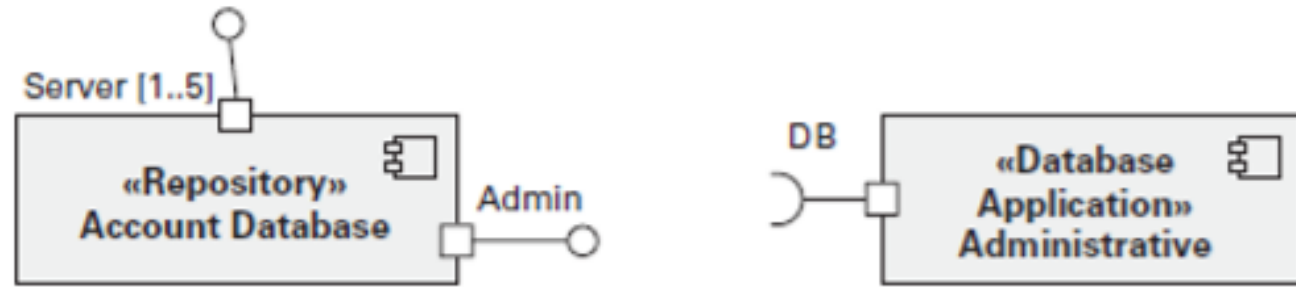
Component Type



Event Based Style Architecture

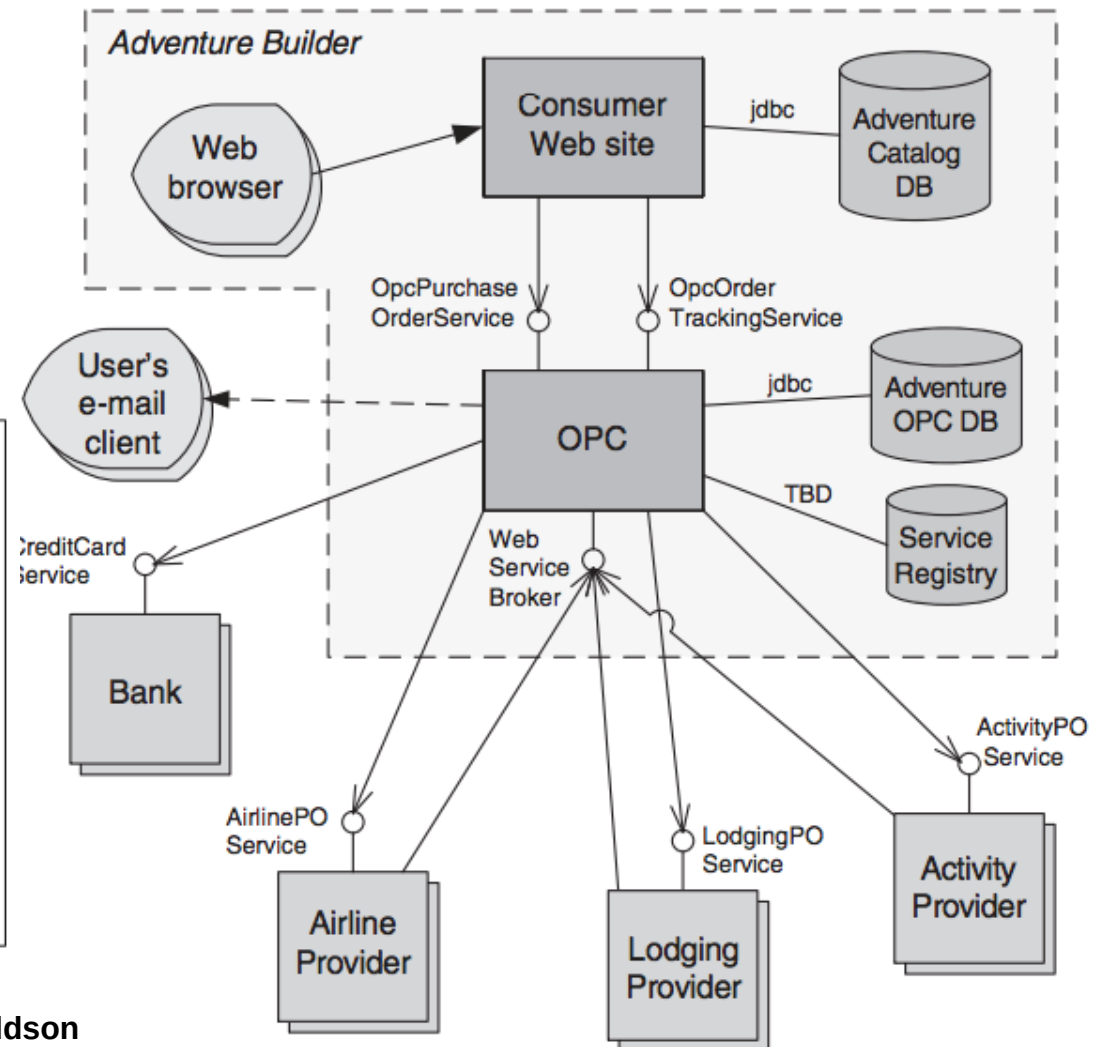
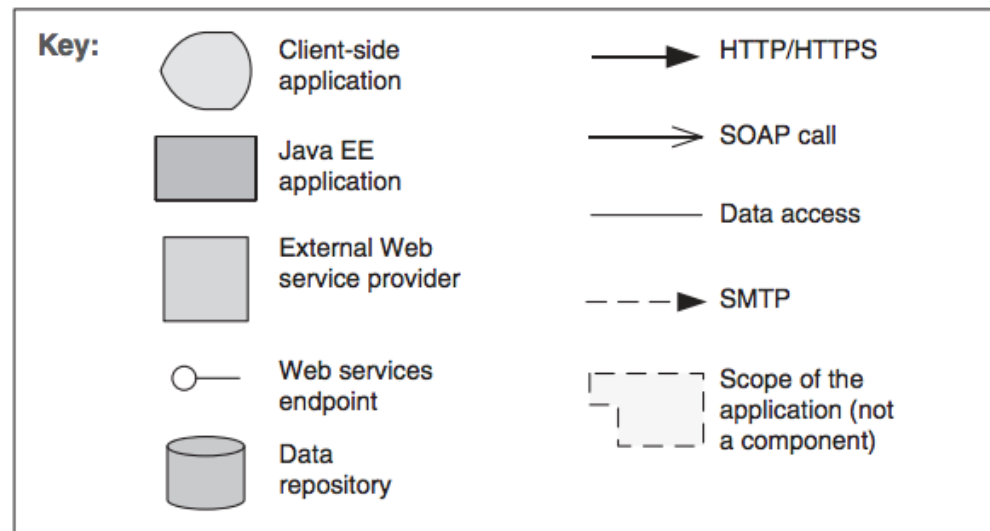


Documenting C&C View: Repository Style



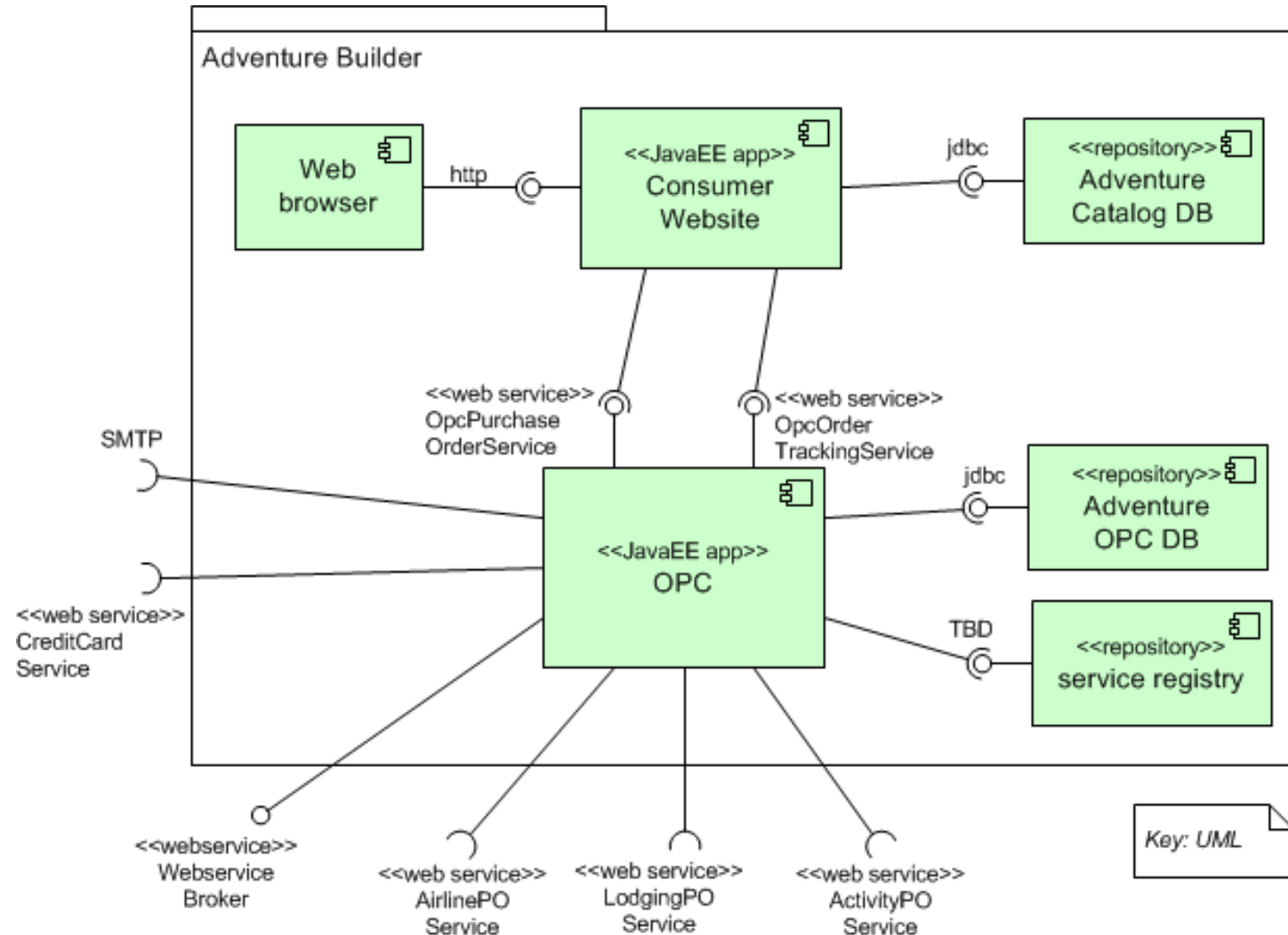
Documenting C&C View: Adventure Builder Example

The Adventure Builder system interacts with external service providers to construct the vacation, and with bank services to process payment. The central OPC (Order Processing Center) component coordinates the interaction with internal and external service consumers and providers



Documenting C&C View: Adventure Builder Example

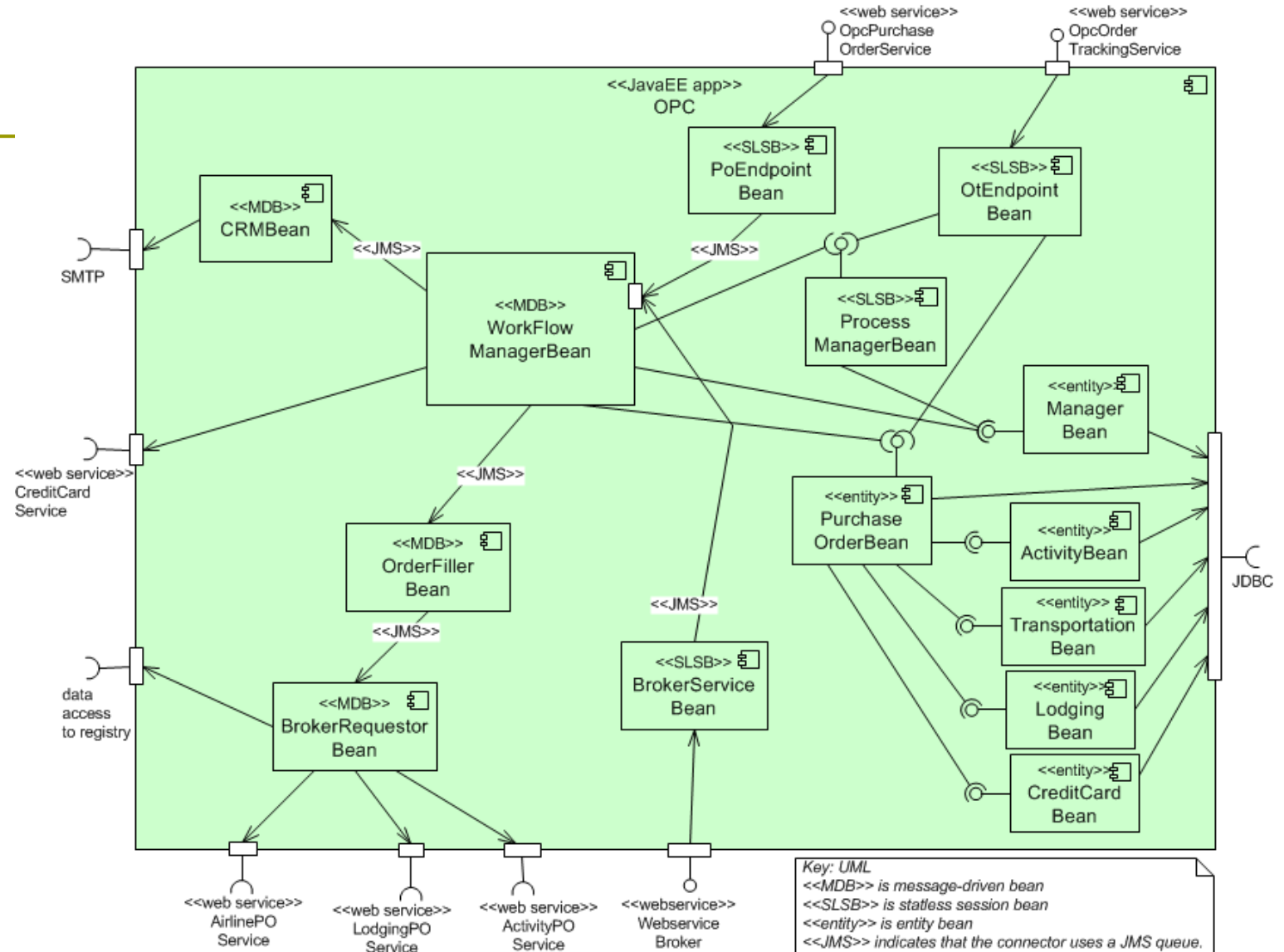
▣ Top Level View



<https://wiki.sei.cmu.edu/confluence/display/SAD/Top+Level+SOA+View>

Documenting C&C View: Adventure Builder Example

OPC C&C View



Summary for Documenting C&C Views

- ❑ **Elements in a C&C view have some runtime presence**
 - **Processing and storage elements in a distributed system:** clients, servers, data store
 - **Concurrent units of execution :** processes or threads
 - **Principal processing units (logical or conceptual aspects):** objects

- ❑ **C&C Styles**
 - **Data flow styles**
 - ❑ Pipe and filter style
 - **Call-return style**
 - ❑ Client-Server style, Multi-tier style, SOA style
 - **Event-based style**
 - ❑ Publish-subscribe style
 - **Repository style**

Summary for Documenting C&C Views with UML

- ❑ **Component types:** UML components
- ❑ **Components instances:** UML component instances
- ❑ **Ports:** UML ports
- ❑ **Connectors:** UML connectors
 - **Alternative**
 - ❑ UML components
 - ❑ UML assembly connectors
 - ❑ Tagged associations
- ❑ **Relations**
 - **Attachment:** connection b/w UML Components and UML connectors
 - **Interface delegation:** UML delegation connector

Topics

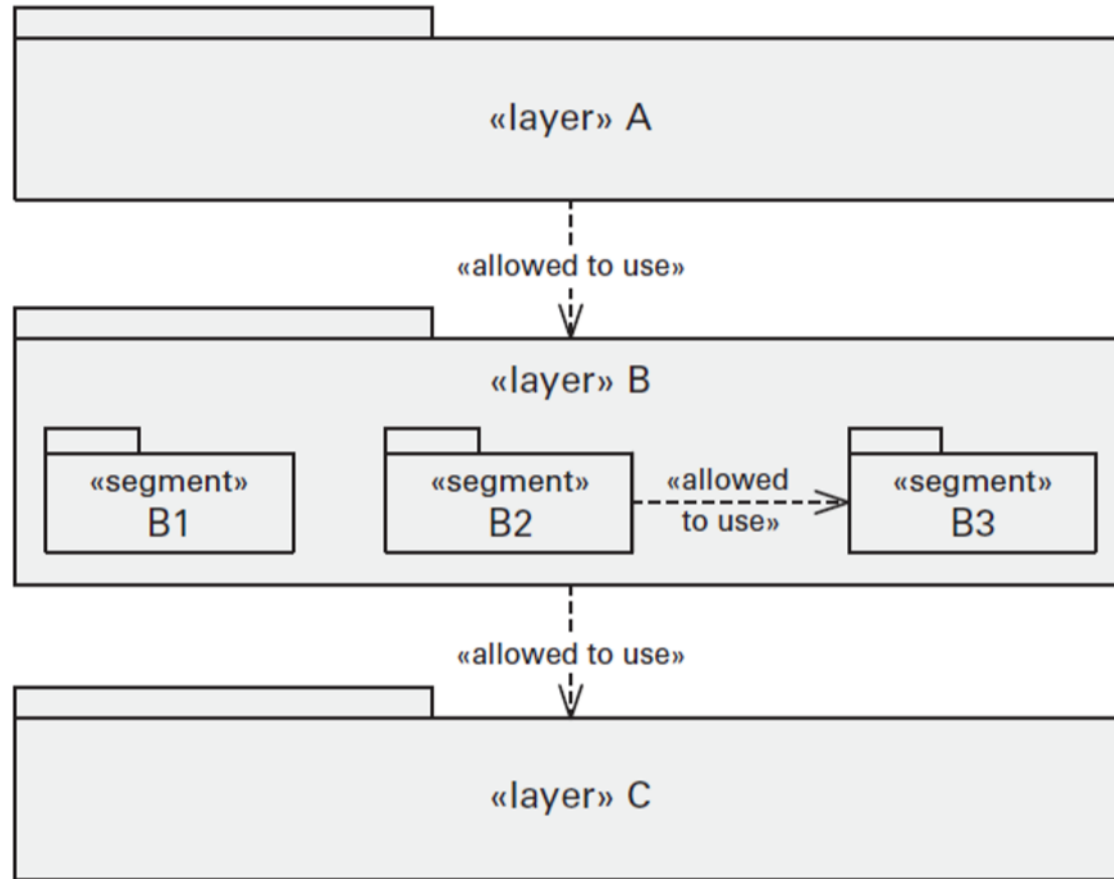
- ❑ **Introduction to Architecture views**
 - 4+1 View Model
 - Siemens View Model
 - SEI View Model
- ❑ **Documenting Architecture**
 - Documenting Architecture using SEI View Model
 - ❑ Documenting C&C View
 - ❑ **Documenting Module View**
 - ❑ Documenting Allocation View
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ **Summary and Wrap-up**

Documenting Module Views

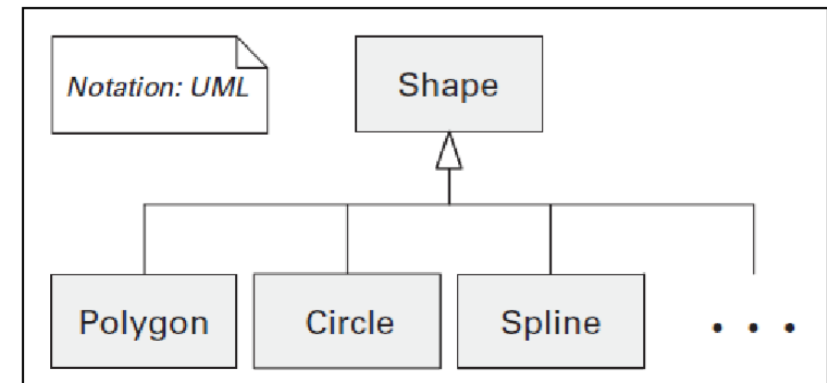
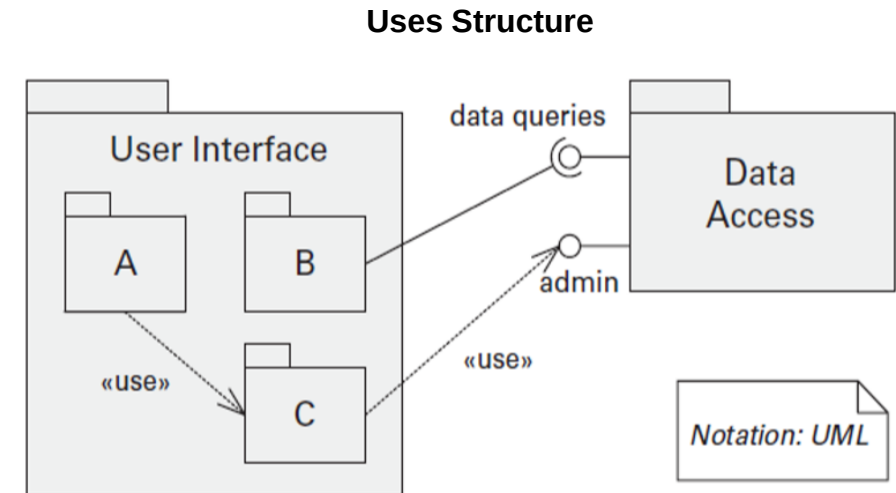
- ❑ A Module view describes **a set of modules** of a system and **the relations among them**.
- ❑ This view is used for construction, analysis, and communication

Elements	Module	• Implementation units of software that provide a coherent set of responsibilities
Relations	Is part of	• A part/whole relationship between the submodule (the part) and the aggregate module (the whole).
	Depends on	• Various forms of dependencies • E.g., uses, allowed to use, crosscuts, data entity relationships
	Is a	• A generalization/specialization relationship

Module View Example



Layered Structure



Generalization Structure

Documenting Module View:

What are modules?

- ❑ **Implementation units of software** that provide a coherent set of responsibilities
 - **Responsibility** – the kinds of features or functionality that a module handles
- ❑ **Various forms of modules**
 - **Procedures or functions** in procedural or functional programming
 - Classes in object-oriented programming
 - Aspects in aspect-oriented programming
 - Layer – a logical grouping of modules
 - Data entity - an object that holds information

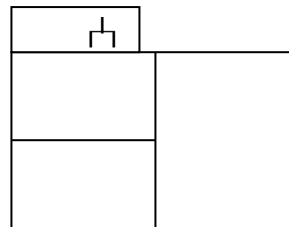
Document Module View:

Notations for the Module Views

- ❑ A module may be represented as a class, a package, or a subsystem using UML notations including stereotypes



Package



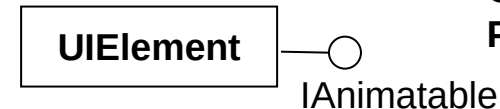
Subsystem



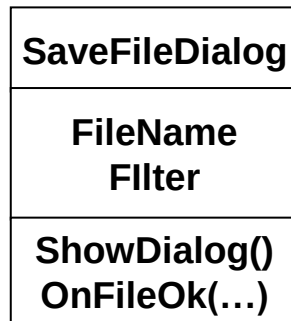
Abstract class



Class



Class with
Provided interface



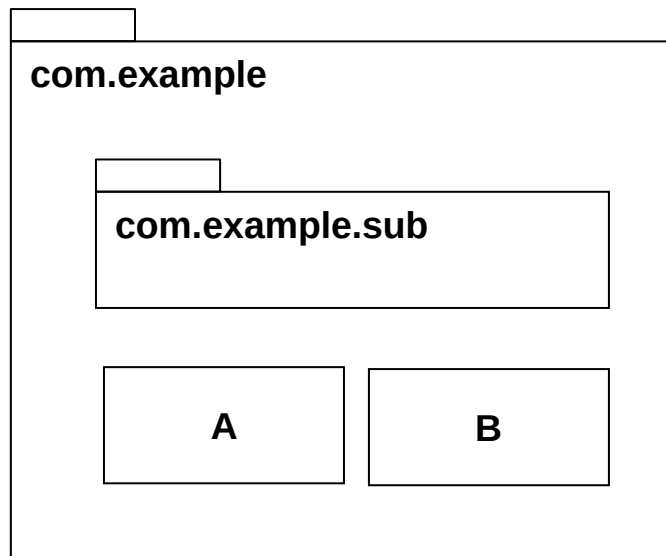
Class showing
Attribute and
Operation
compartments



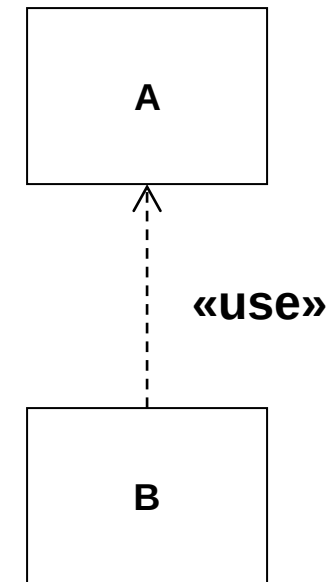
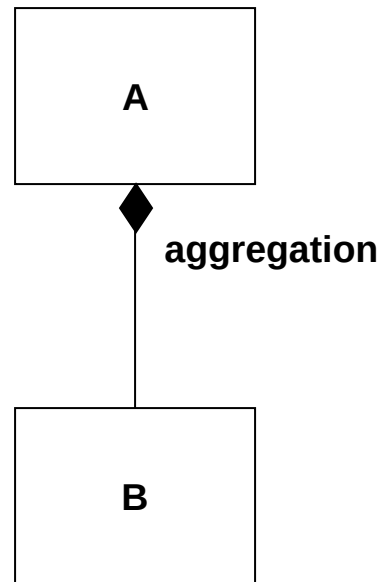
Interface not
shown as lollipop

Document Module View: Notations for the Module Views

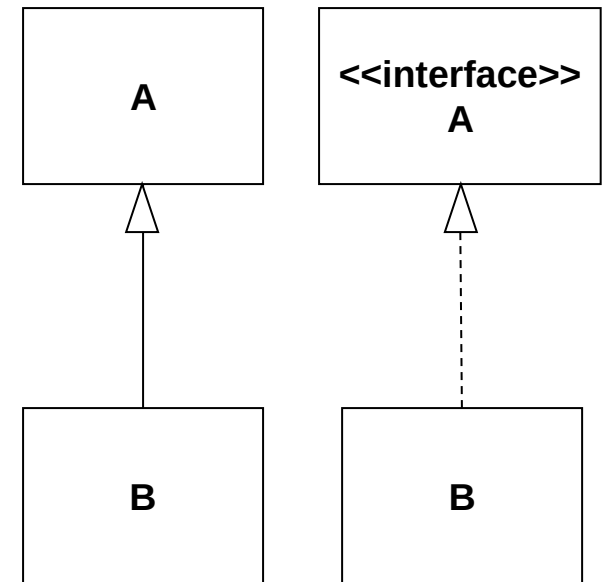
▣ Relations between modules



Is-Part-Of relation



Depends-On relation



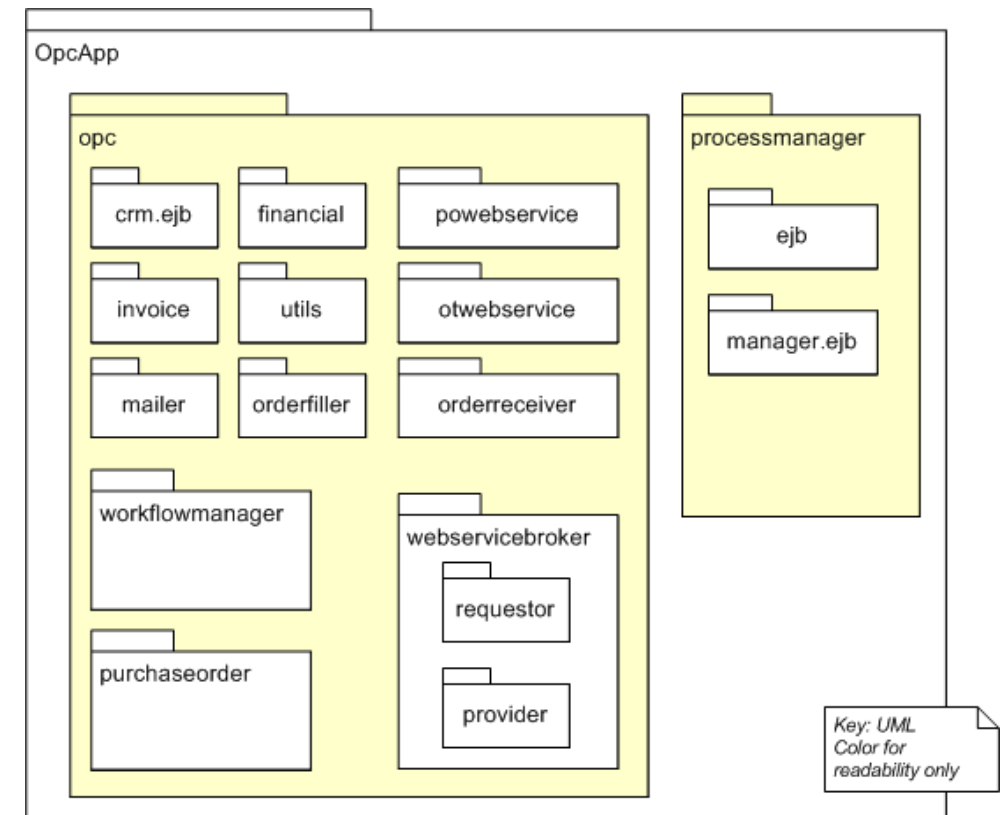
Is-A relation

Documenting Module View: Styles of the Module View

- ❑ **Decomposition Style**
 - ❑ **Uses Style**
 - ❑ **Generalization Style**
 - ❑ **Layered Style**
 - ❑ **Data Model Style**
-
- ❑ **Each of these styles constrains the basic module view, adding specialized versions of some of the elements and relation types.**

Document Module View: Decomposition Style

- ❑ Focusing on the *is-part-of* relation
- ❑ A decomposition style view **presents the functionality of a system in intellectually manageable pieces** that are recursively refined to convey more and more details
- ❑ This style is well **suited to support the learning process** about a system.
- ❑ This style most often **serves as the input for the work assignment view** of a system



Document Module View: Decomposition Style

- ❑ **The criteria used for decomposing a module**
 - **Achievement of certain quality attributes**
 - **Build-versus-buy decisions**
 - **Product line implementation**
 - **Team allocation**
 - ❑ **Separate modules that can be allocated to different teams**

Document Module View: Notations for the Decomposition Style

□ Informal Notations

- Named boxes that contain other named boxes
- Containments can be shown using indentation

Hardware-Hiding Module

Extended Computer Module

 Data Module

 Input/Output Module

 Computer State Module

 Parallelism Control Module

 ...

Device Interface Module

 Air Data Computer Module

 Angle or Attach Sensor Module

 Audible Signal Device Module

 Computer Fail Device Module

 ...

Behavior-Hiding Module

Function Driver Module

 Audible Signal Module

 Computer Fail Signal Module

 ...

Shared Services Module

 Mode Determination Module

 Panel I/O Support Module

 ...

Software Decision Module

Application Data Type Module

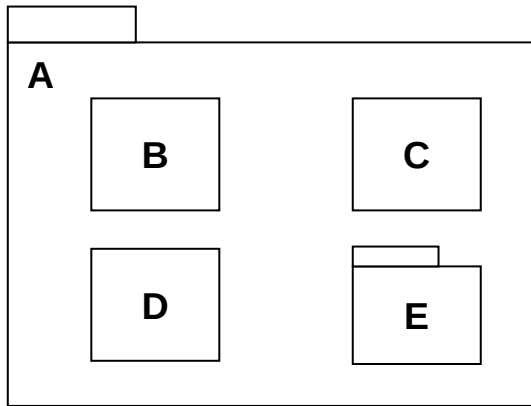
 Numeric Data Type Module

 State Transition Event Module

 ...

Document Module View: Notations for the Decomposition Style

- **UML Notations**
 - **Package construct**



Document Module View: UML Notations for the Decomposition Style

Hardware-Hiding Module

Extended Computer Module

Data Module

Input/Output Module

Computer State Module

Parallelism Control Module

...

Device Interface Module

Air Data Computer Module

Angle or Attach Sensor Module

Audible Signal Device Module

Computer Fail Device Module

...

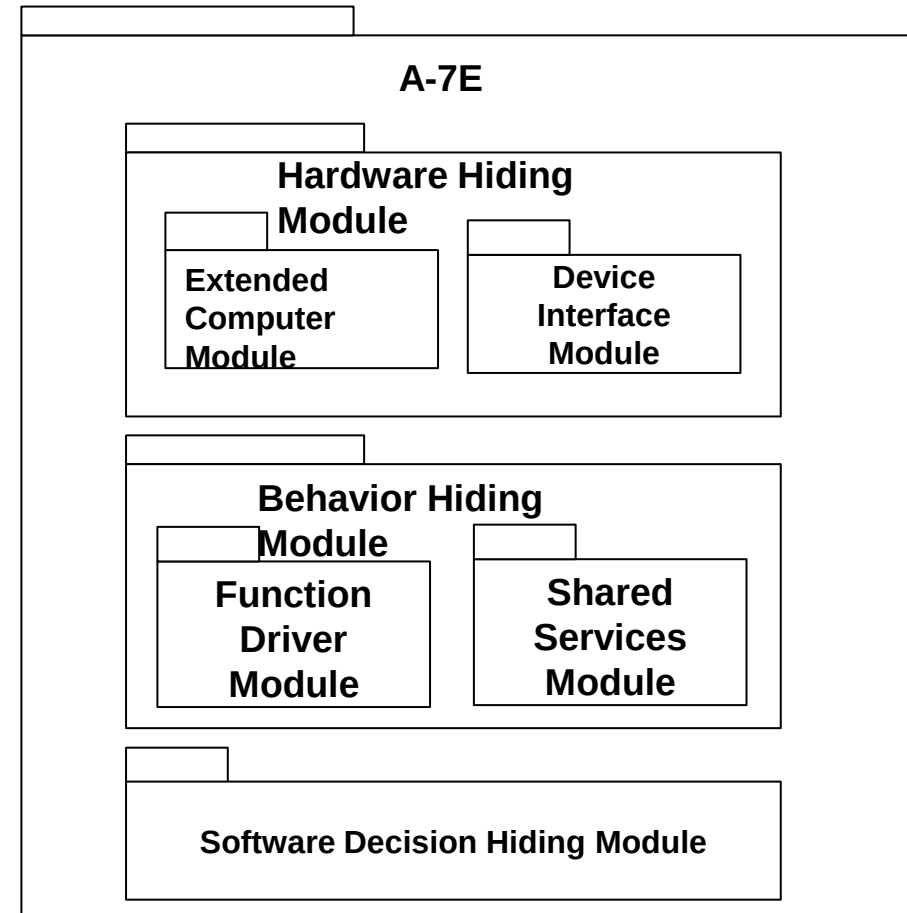
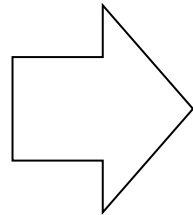
Behavior-Hiding Module

Function Driver Module

Audible Signal Module

Computer Fail Signal Module

...



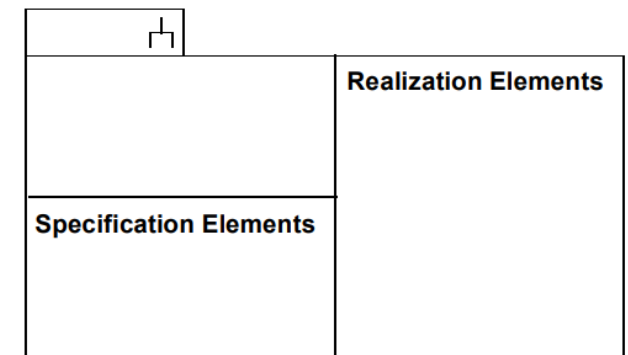
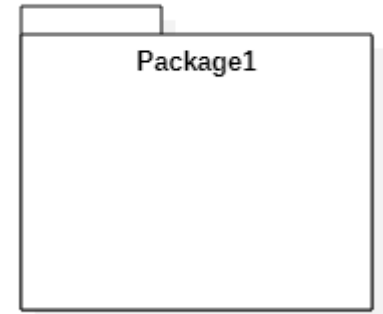
Document Module View: Subsystem vs. Package in UML

□ Package

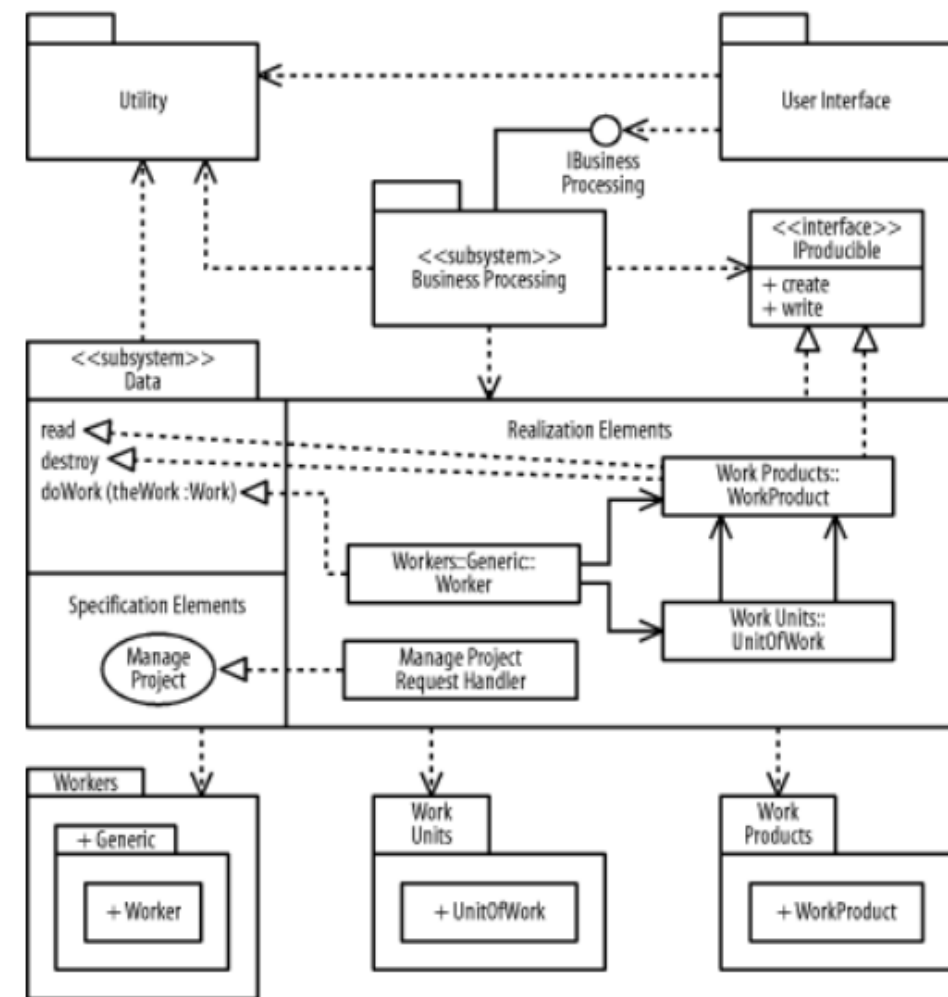
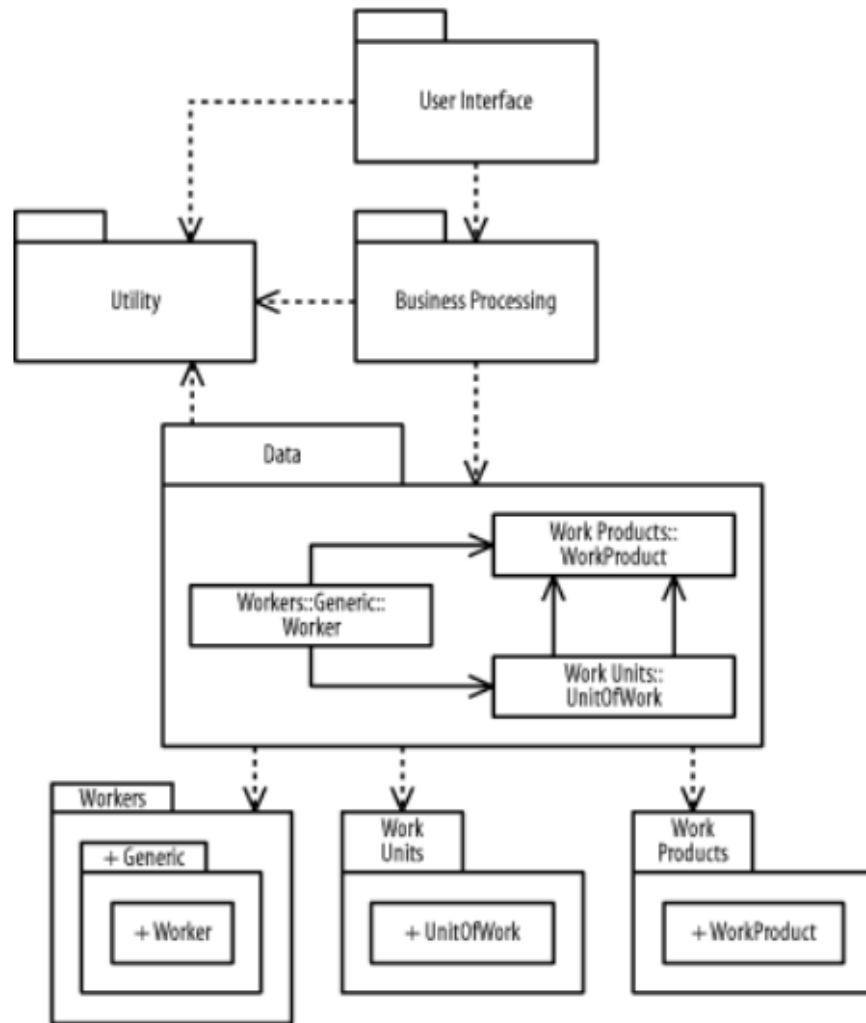
- A meaningful grouping of elements

□ Subsystem

- A part of a system that
 - Carries out a functionality cohesive subset of the overall system's mission
 - Can be executed independently
 - Can be developed and deployed incrementally
- A subsystem is **notated basically in the same way as a package**, with the addition of a fork symbol placed in the upper right corner of the large rectangle.

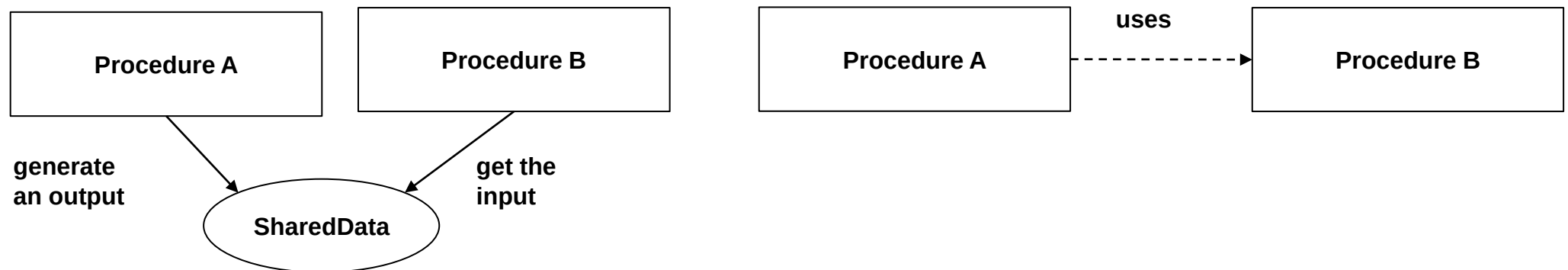


Packages vs. Subsystems



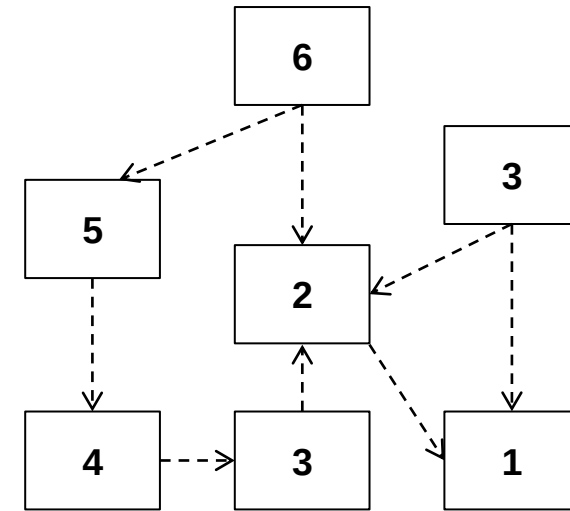
Document Module View: Uses Style

- ❑ The uses style comes about when the *depends-on* relation is specialized to uses.
- ❑ This style tells developers **what other modules must exist in order for their portion of the system to work correctly.**
- ❑ A uses relation does not always match a call relation between modules



Document Module View: Uses Style

- This style is useful for planning
 - incremental development
 - system extensions and subsets
 - debugging and testing
 - gauging the effects of specific changes



Document Module View: Notations for the Uses Style

□ Informal Notations

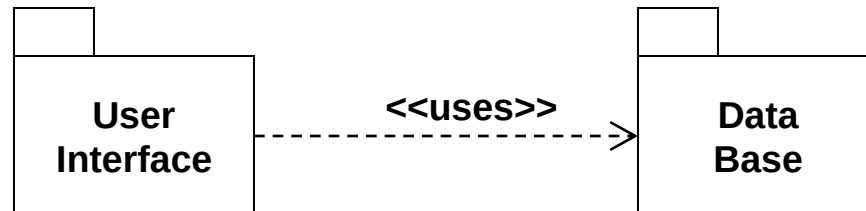
■ Matrix form

- a mark in element (x,y) indicates that module x uses module y.

■ Two-column table

- Using elements on the left and the element they use listed on the right

□ UML Notations



Document Module View: Example of the Uses Style

Matrix form

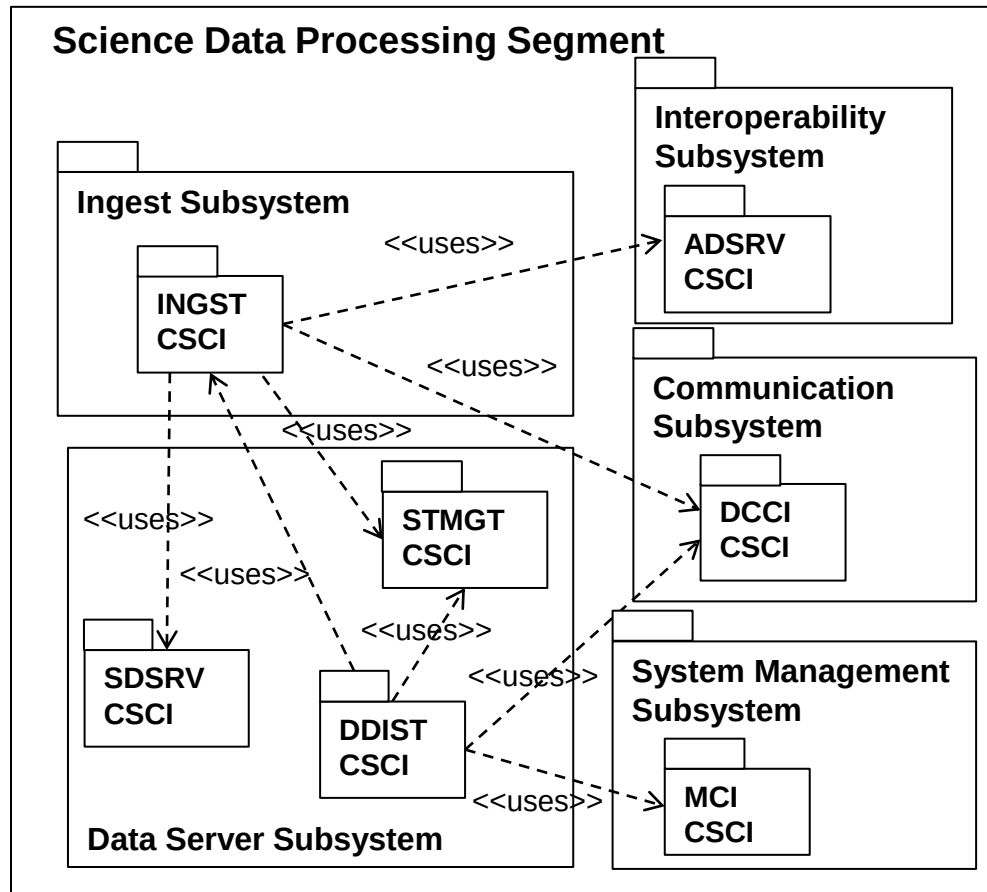
	INGST CSCI	STMGT CSCI	SDSRV CSCI	DDIST CSCI	DCCI CSCI	ADSRV CSCI	MCI CSCI
INGST CSC		1	1		1	1	
STMGT CSCI							
SDSRV CSCI							
DDIST CSCI	1	1			1		1
DCCI CSCI							
ADSRV CSCI							
MCI CSCI							

Two column table

SDPS Element Uses This Element			
Science Data Processing Segment			
[etc.]	Ingest Subsystem		
	[etc.]	INGST CSCI	ADSRV CSCI in the Interoperability Subsystem
			STMGT CSCI in the Data Server Subsystem
			SDSRV CSCI in the Data Server Subsystem
			DCCI CSCI in the Communications Subsystem
	[etc.]	Other CSCIs within the Ingest Subsystem	
	Data Server Subsystem		
	[etc.]	DDIST CSCI	MCI CSCI in the System Management Subsystem
			DCCI CSCI in the Communications Subsystem
			STMGT CSCI in the Data Server Subsystem
			INGST CSCI in the Ingest Subsystem
	[etc.]	Other CSCIs within the Data Server Subsystem	
	other subsystems within the Science Data Processing Segment		

Document Module View: Example of the Uses Style

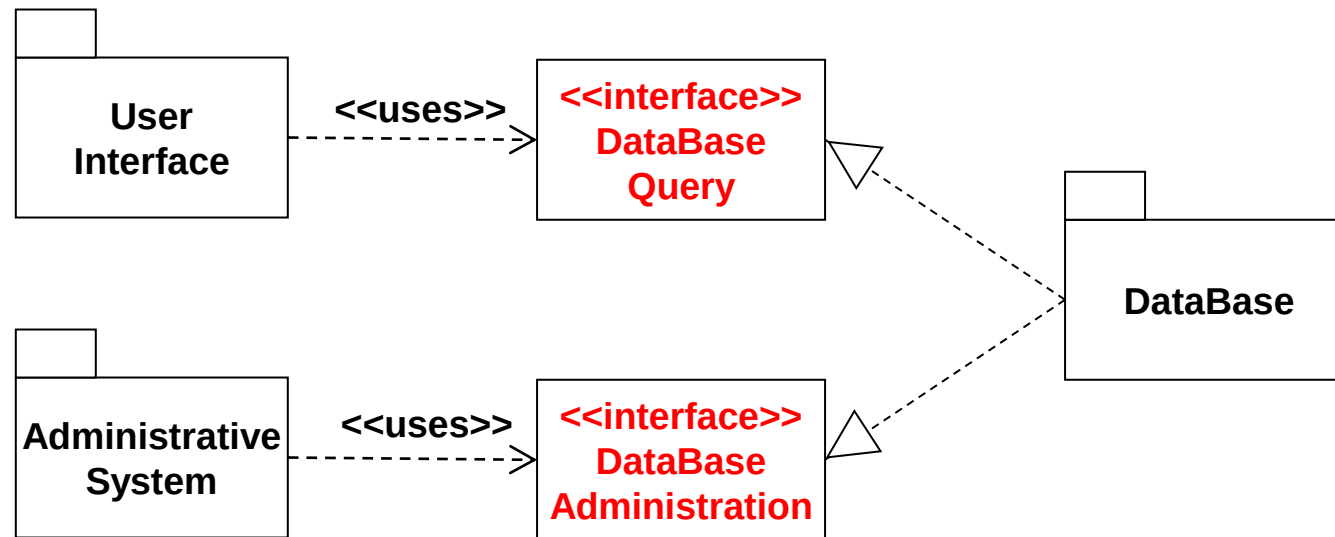
▣ UML Notation and Two column table

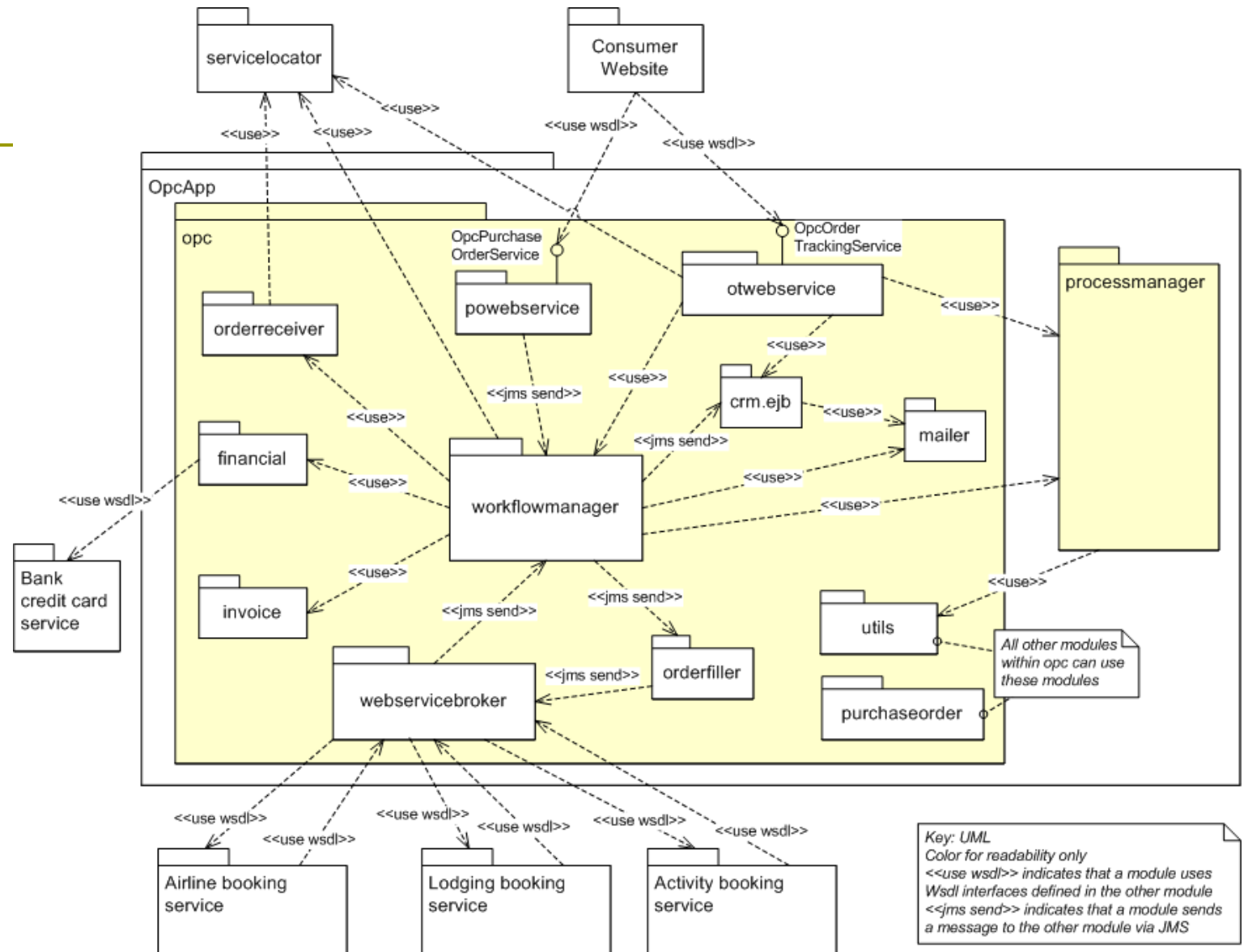


SDPS Element Uses This Element		
Science Data Processing Segment		
	Ingest Subsystem	
	INGST CSCI	ADSRV CSCI in the Interoperability Subsystem
		STMG T CSCI in the Data Server Subsystem
		SDSRV CSCI in the Data Server Subsystem
		DCCI CSCI in the Communications Subsystem
	[etc.]	Other CSCIs within the Ingest Subsystem
	Data Server Subsystem	
	DDIST CSCI	MCI CSCI in the System Management Subsystem
		DCCI CSCI in the Communications Subsystem
		STMG T CSCI in the Data Server Subsystem
		INGST CSCI in the Ingest Subsystem
	[etc.]	Other CSCIs within the Data Server Subsystem
[etc.]	other subsystems within the Science Data Processing Segment	

Document Module View: Example of the Uses Style

- ❑ In UML we can represent the *uses* relations and also show interfaces explicitly





Generalization Style

□ Element

■ Module

- A module can have the “**abstract**” property, which defines a module with interfaces but no implementation

□ Relations

■ Generalization

- The relation can be further specialized to **class inheritance**, **interface inheritance**, or **interface realization**

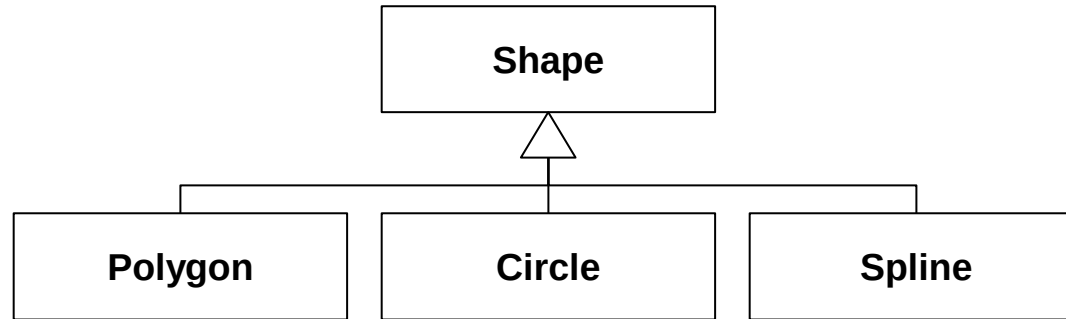
□ Constraints

- Cycles in the generalization relation are not allowed

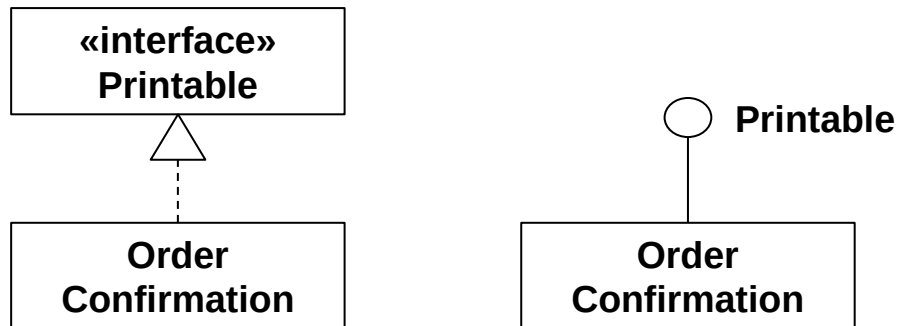
Document Module View: Notations for the Generalization Style

□ UML

■ Class or interface inheritance



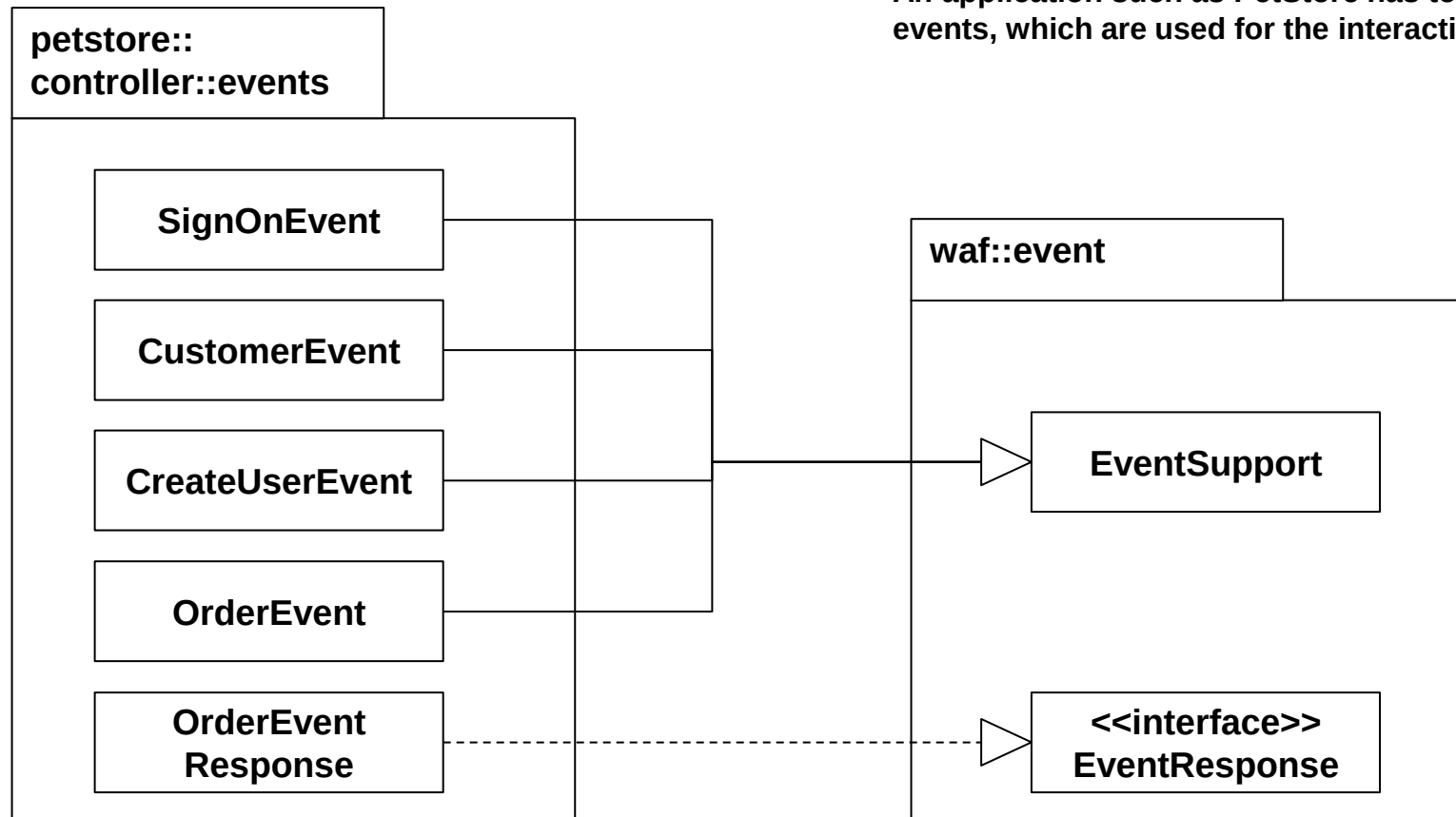
■ Interface realization



Document Module View: What the Generalization style is For

- This style is used for **extension and evolution** of architectures and individual elements

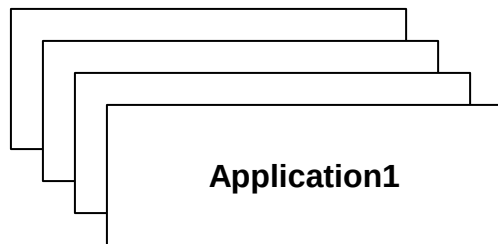
An application framework (waf) offers an event-handling service. An application such as PetStore has to define the application specific events, which are used for the interaction of other modules in the system.



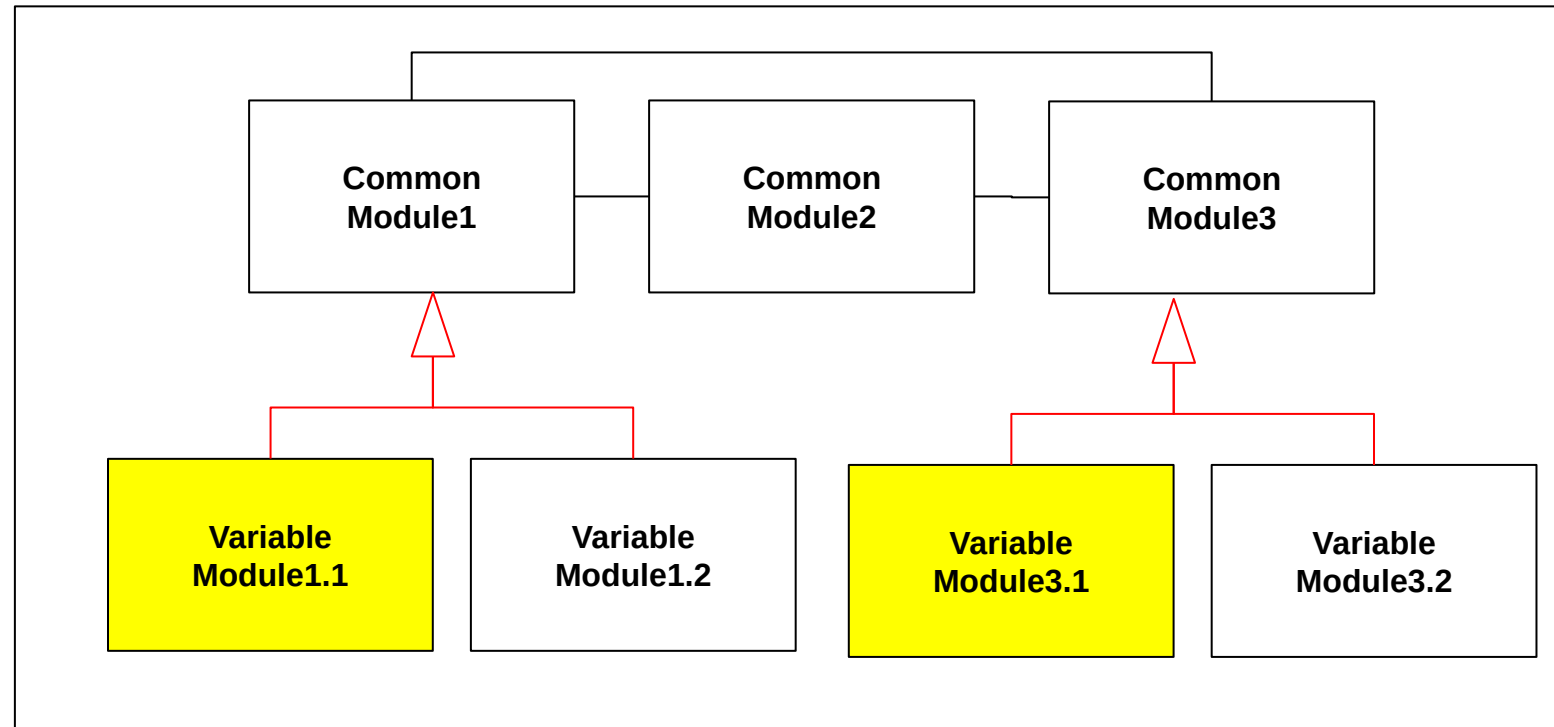
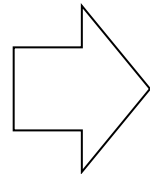
Document Module View: What the Generalization style is For

□ Commonalities and variations

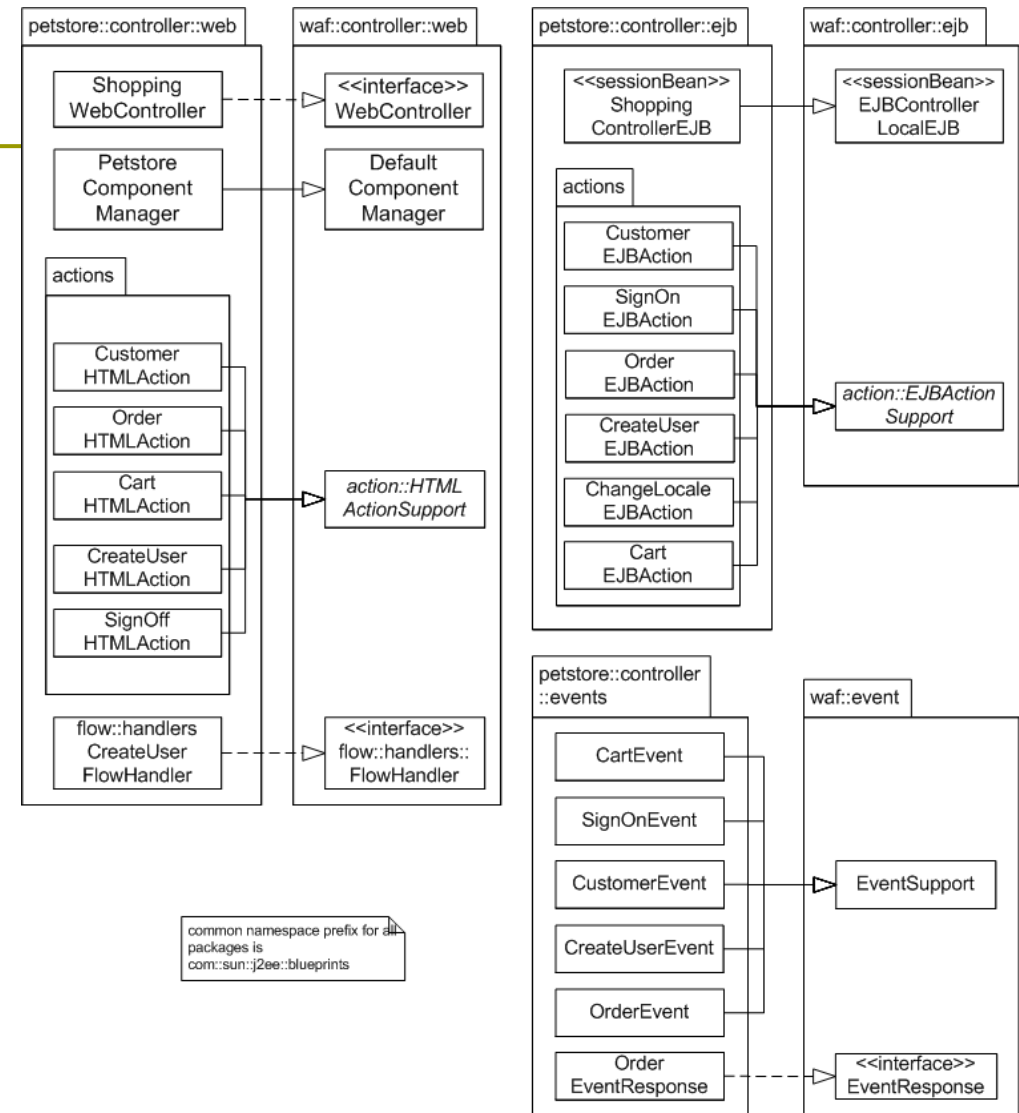
- Define commonalities on a higher level and variations as children of a module



A Product Line

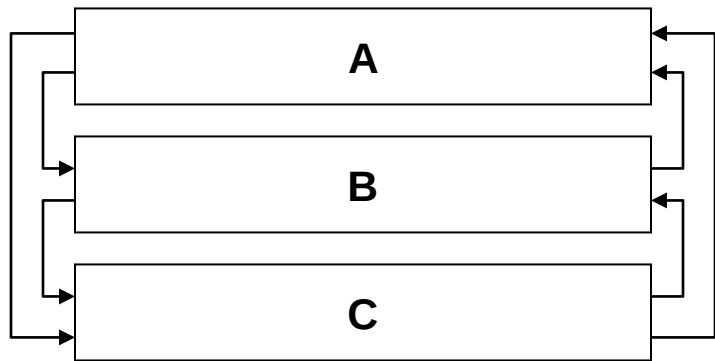


PL Architecture



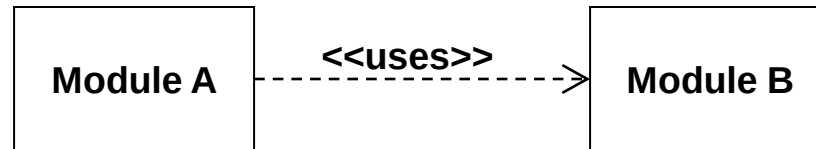
Document Module View: Layered Style

- ❑ **Layering** reflects a division of the software into layers, which represents a grouping of modules that offers a cohesive set of services
- ❑ This *resembles* a layering but *is not*



Document Module View: Layered Style

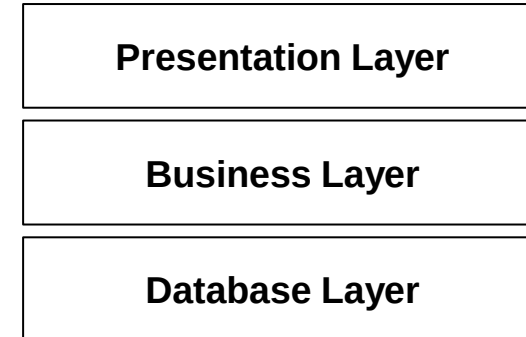
- ❑ **Layers cannot be derived by examining source code**



- **Should “Module B” be in a lower layer than “Module A”?**
 - **Should they be in the same layer?**
- ❑ **Layering is a conscious design decision that allocates modules to layers, based on coupling, cohesion, and the likelihood of changes**

Document Module View: What the Layered Style is For

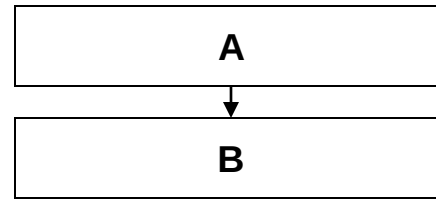
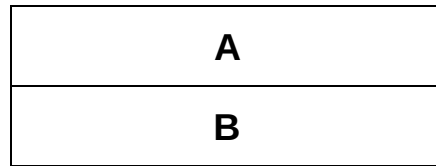
- ❑ **Promoting modifiability and portability to a software system**
- ❑ **Managing complexity and facilitating the communication of the code structure to developers**
- ❑ **Promoting reuse**
- ❑ **Achieving separation of concerns**



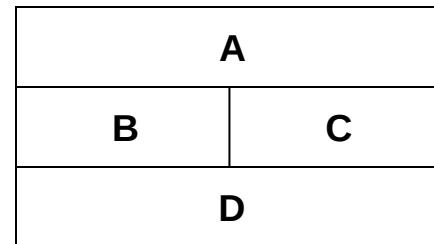
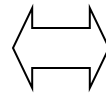
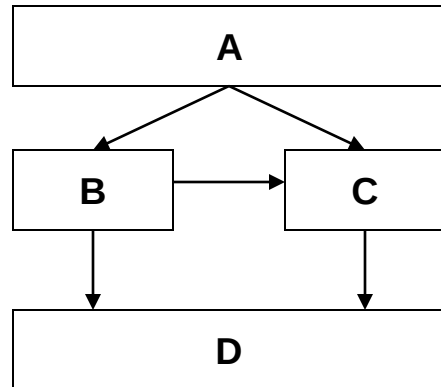
Document Module View: Notations for the Layered Style

□ Informal Notations

Stack



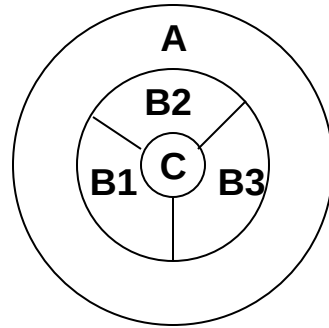
Segmented Layers



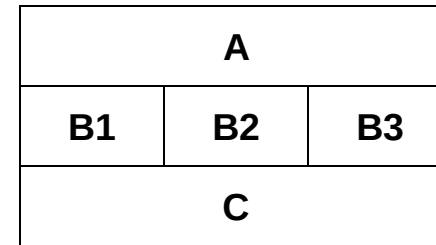
Document Module View: Notations for the Layered Style

□ Informal Notations

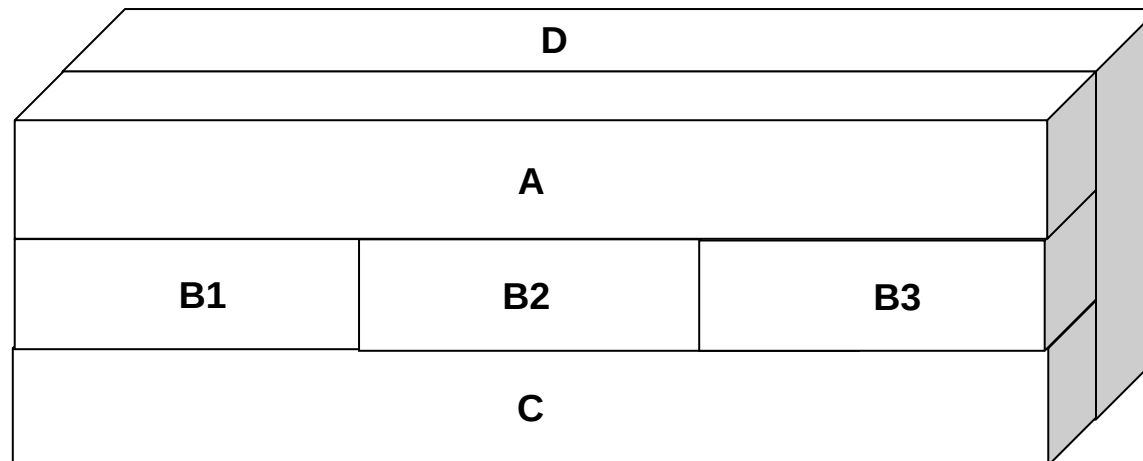
Ring



← Not equivalent →



3D layered diagram

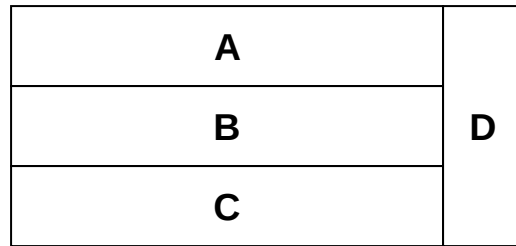


Layer D can be used by all
other layers.
Can D use all other layers?

Document Module View: Notations for the Layered Style

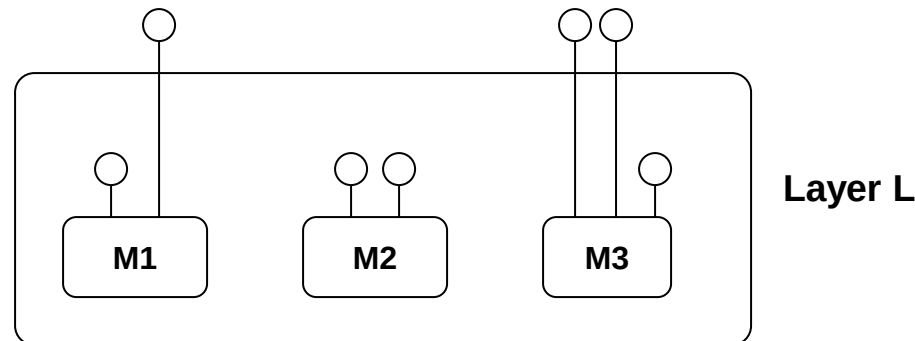
□ Informal Notations

Layers with a Sidecar



Is D allowed to use and be used by other modules?

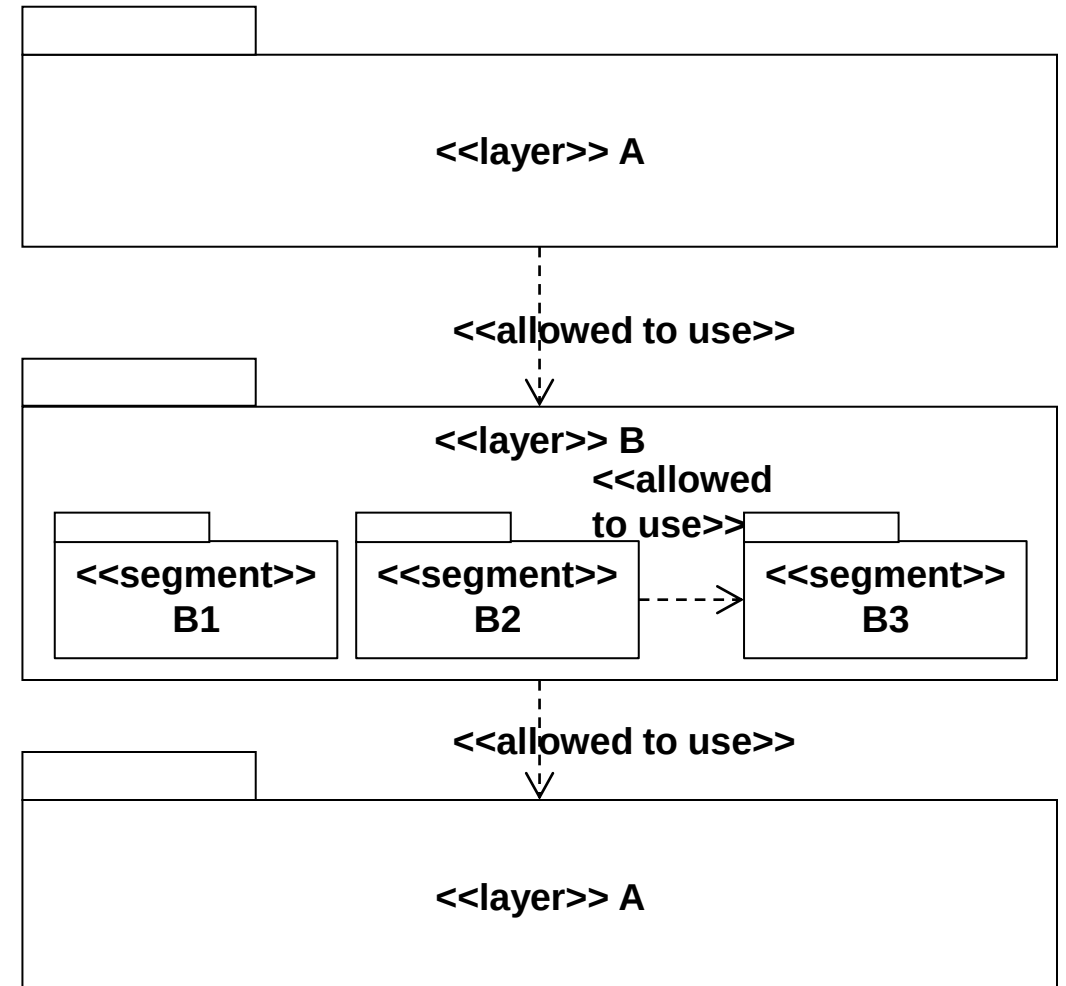
Contents and Interface



Document Module View: Notations for the Layered Style

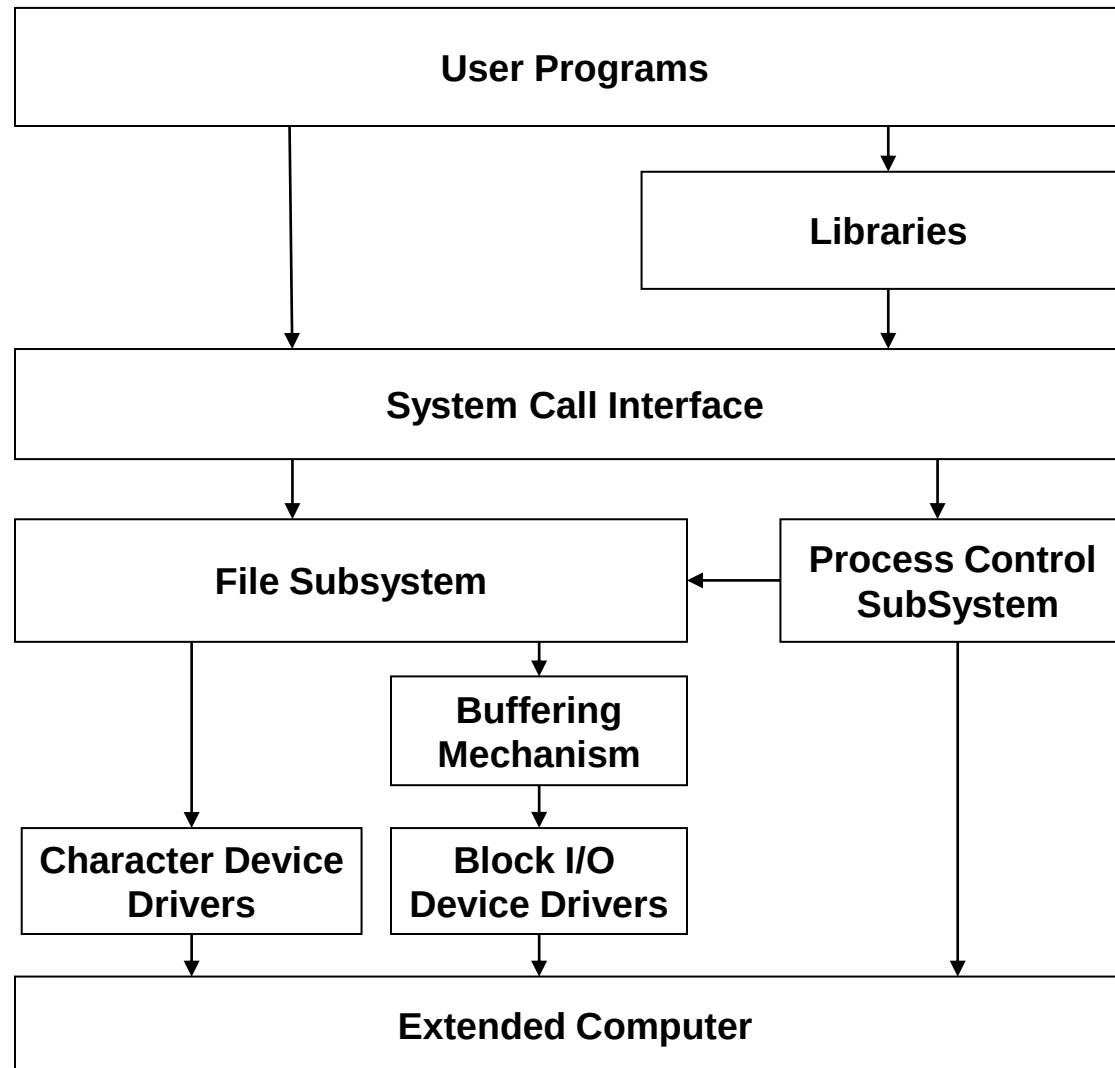
□ UML

- A **layer** can be shown as a **UML package**
- The **allowed-to-use** relation can be represented as a stereotype of **UML access dependency**
 - An element within a package is allowed to access other elements within the package
 - If a package access another package, all elements with public visibility in the accessed package are visible
 - Access dependencies are not transitive.



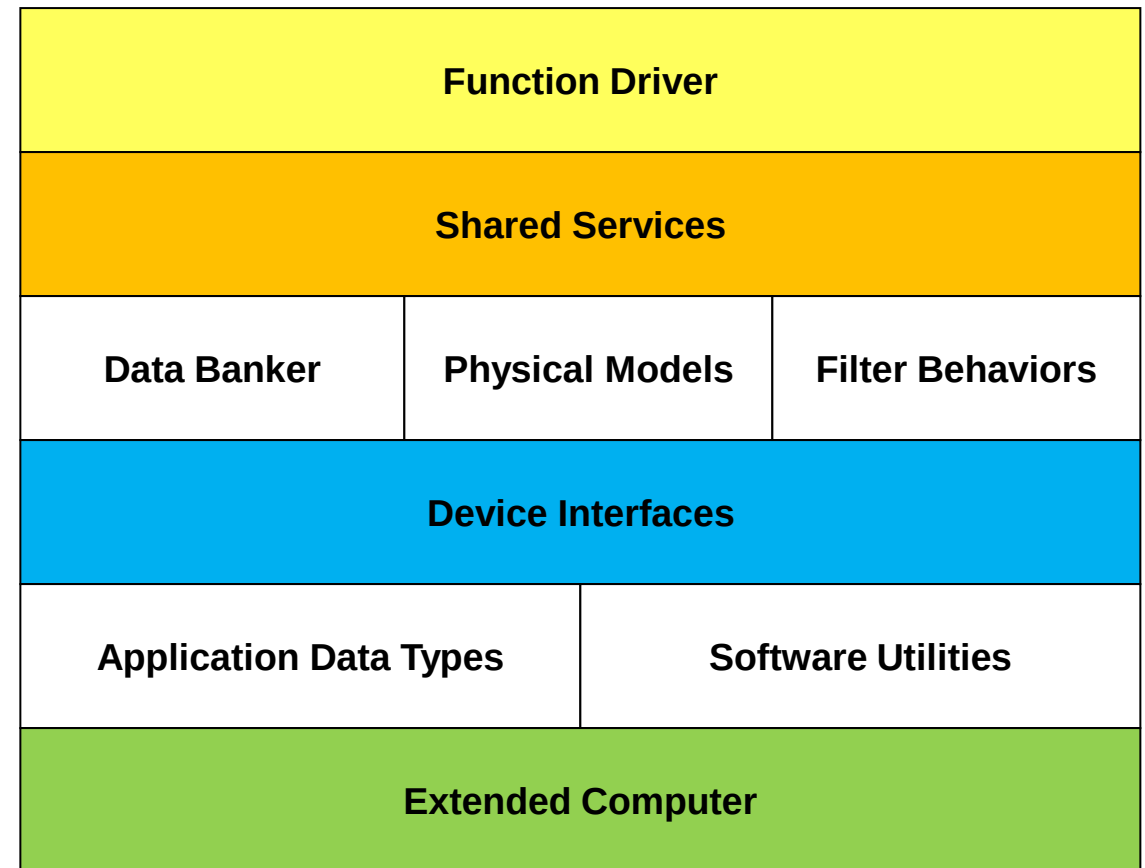
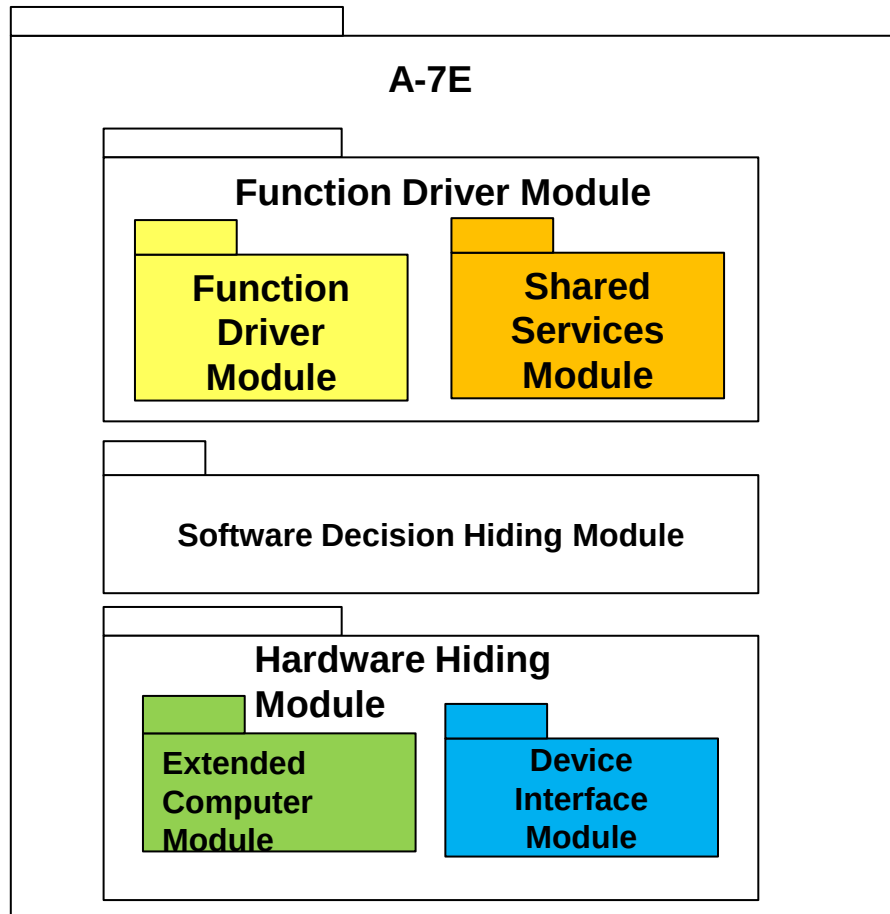
Document Module View: Examples of the Layered Style

Unix System V



Allowed to use

Document Module View: Decomposition Style vs. Layered Style



Data Model Style

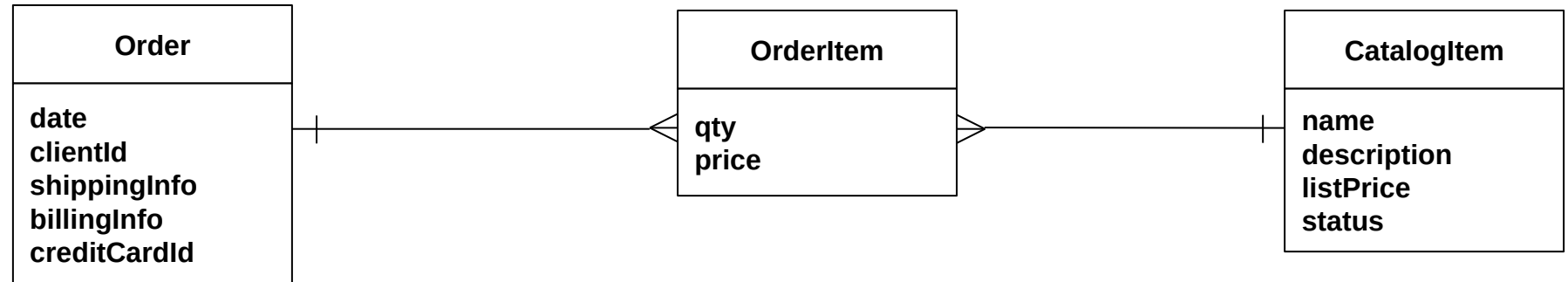
- ❑ **The static information structure in terms of data entities and their relationships**
- ❑ **Conceptual Data Model**
 - Focuses on the entities and their relationships as perceived in the problem domain
- ❑ **Logical Data Model**
 - An evolution of the conceptual data model toward a data management technology (e.g., relational databases)
- ❑ **Physical Data Model**
 - Incorporates optimizations that may include partitioning or merging entities, duplicating data, and creating identification keys and indexes.

Data Model Style

Conceptual Data Model



Logical Data Model



Physical Data Model

PurchaseOrder			OrderItem			CatalogItem		
PK	pold	INTEGER	PK,FK1	pold	INTEGER	PK	itemId	INTEGER
FK3,I1	clientId	INTEGER	PK,FK2	itemId	INTEGER		name	VARCHAR(80)
FK1	shippingInfoId	INTEGER		qty	NUMERIC(10,2)		description	TEXT(400)
FK2	billingInfoId	INTEGER		unit	CHAR(10)		listPrice	NUMERIC(10,2)
FK4	creditCardId	INTEGER		price	NUMERIC(10,2)		status	INTEGER
	totalPrice	NUMERIC(10,2)						

Data Model Style

□ Elements

■ Data entity

- An object that holds information that needs to be stored or somehow represented in the system

□ Relations

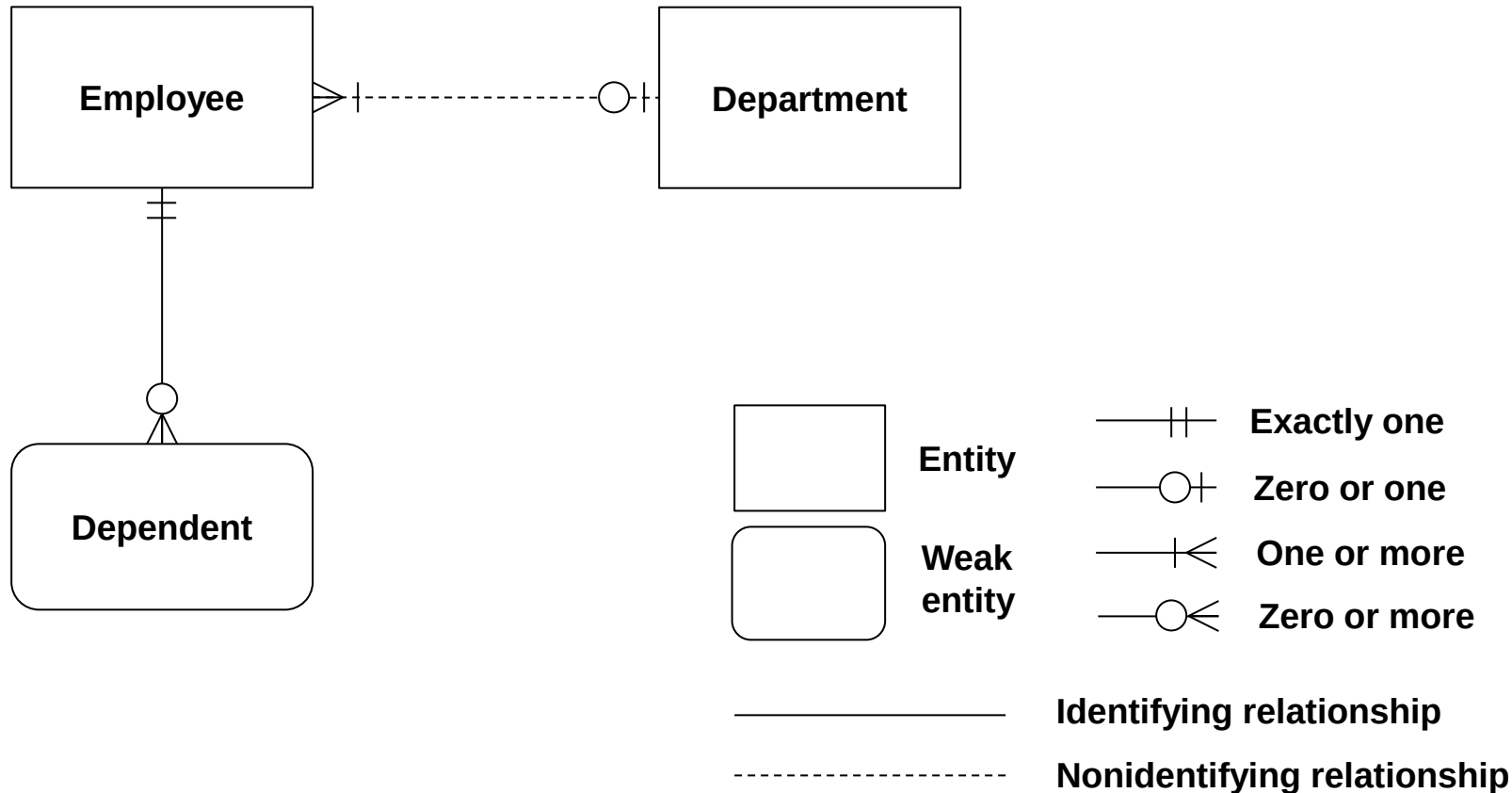
- Logical associations (one-to-one, one-to-many, many-to-many)
- Generalization/Specialization
- Aggregation

What the Data Model is For

- ❑ **Describing the structure of the data used in the system**
- ❑ **Performing impact analysis of changes to the data model**
- ❑ **Enforcing data quality by avoiding redundancy and inconsistency**
- ❑ **Guiding implementation of modules that access the data**

Notations for the Data Model Style

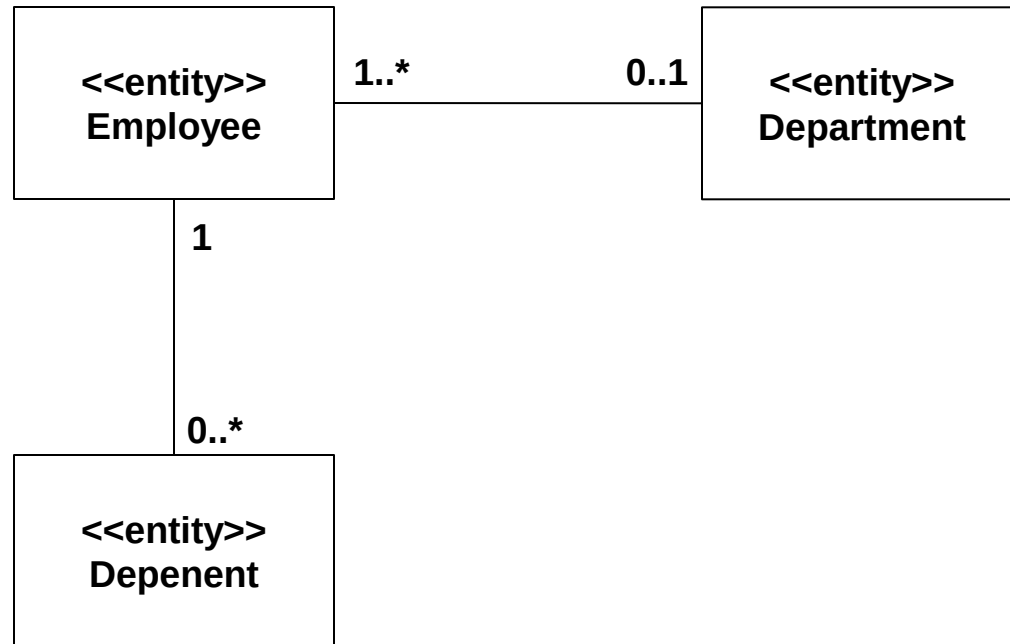
□ Crow's Foot ERD Notation



Notations for the Data Model Style

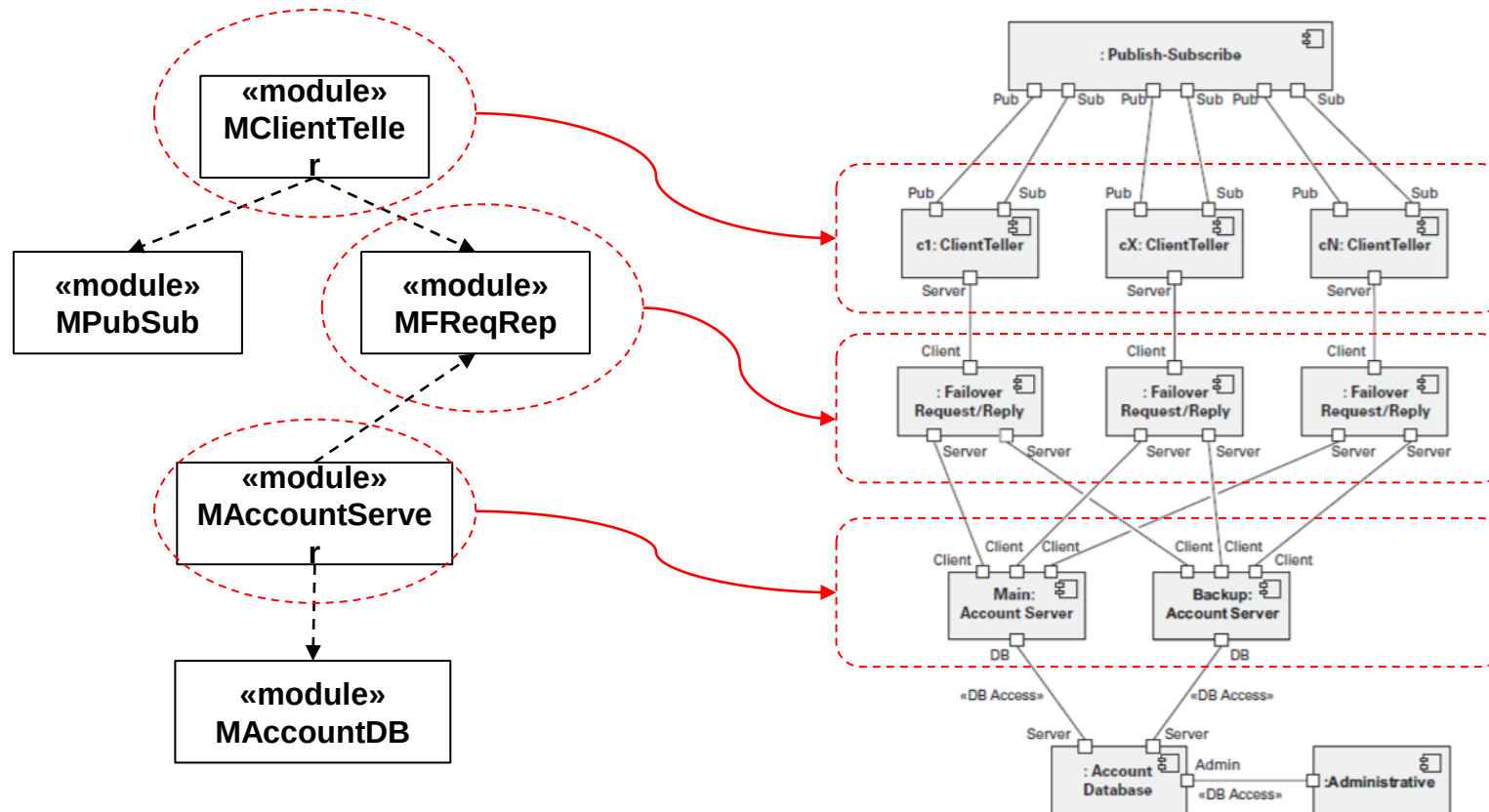
□ UML

- Represented as a UML class diagram



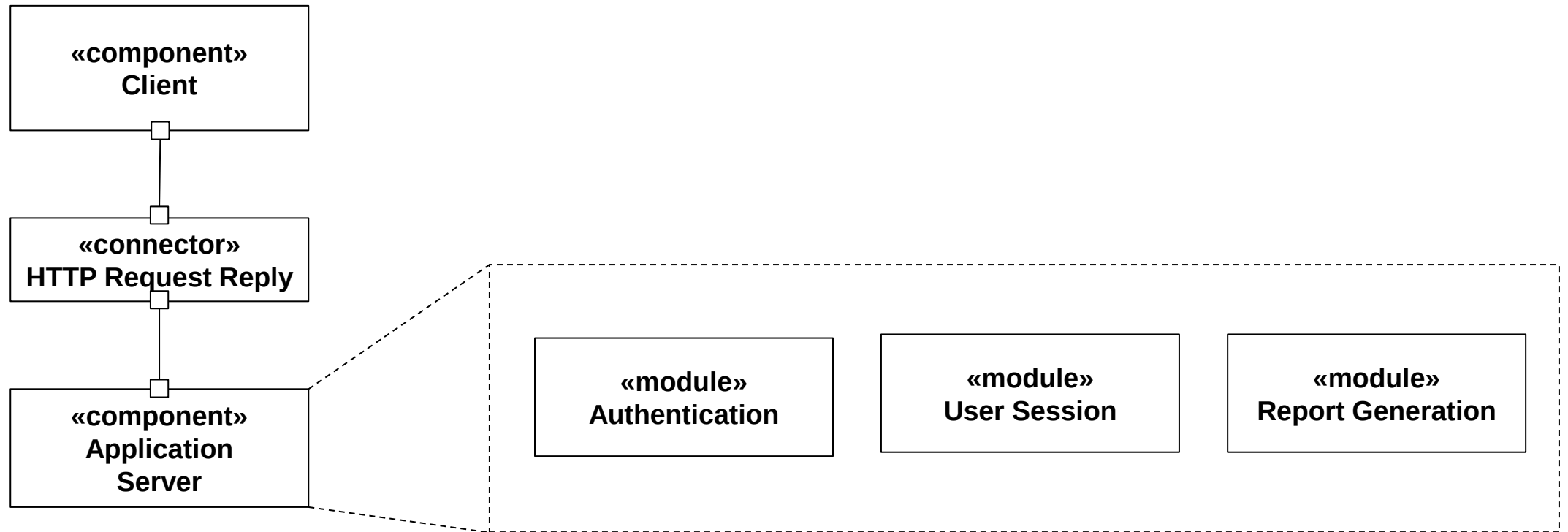
Relations b/w C&C and Module views

- ❑ The same code module might be executed by many of the elements of a C&C view



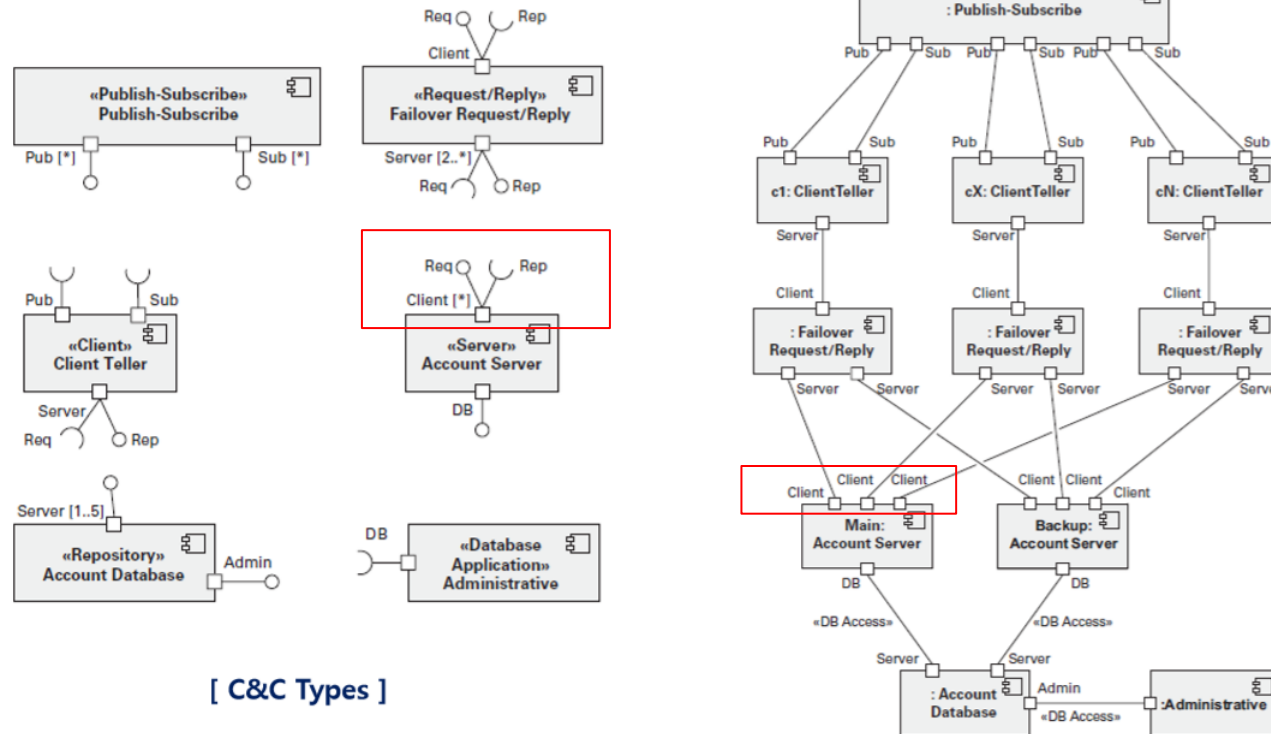
Relations b/w C&C and Module views

- ❑ A single component of a C&C view might execute code defined by many modules



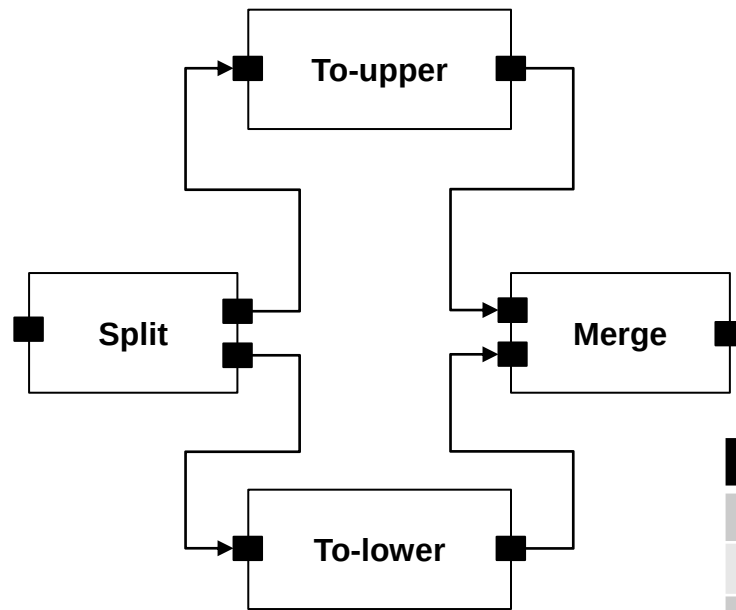
Relations b/w C&C and Module views

- A C&C component might have many points of interaction with its environment, each defined by the same module interface



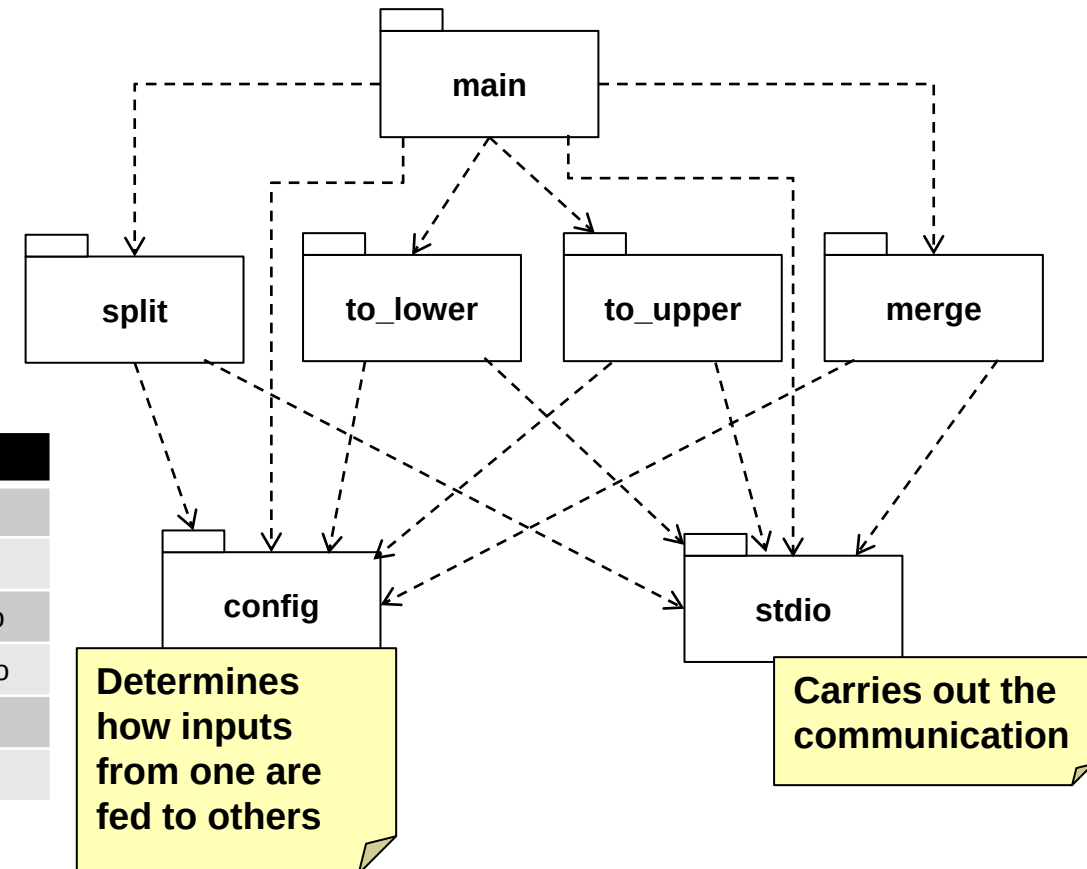
Relations b/w C&C view and Module view

C&C View



→ Pipe
---> uses

Module View



Mapping

C&C View	Module View
System as a whole	Main
Split	split, config, stdio
To-lower	to-lower, config, stdio
To-upper	to-upper, config, stdio
Merge	Merge, config, stio
Each pipe	stdio

Summary of Module View Documentation

- ❑ The **decomposition** style shows how responsibilities are allocated across modules
 - The decomposition style is an excellent learning and navigation tool for newcomers.
 - It also servers the input for the work assignment view of a system.
- ❑ The **uses** style shows how modules depend on one another
 - The uses style helps achieve incremental development and the building of useful subsets of the system
- ❑ The **generalization** style states modules by showing how one is generalization or specialization of the other
 - The generalization style is widely used in object-oriented systems. It shows inheritance, and is used to exploit commonality among modules

Summary of Module View Documentation

- ❑ The **layered** style divides a system into a set of virtual machines, related by the allowed-to-use relation
 - The layered style helps a system achieve portability and modifiability

- ❑ A **data** model style describe the structure of the data used in the system in terms of data entities and their relationships
 - It guides implementation and help to improve performance and modification in data-centric systems.

Topics

- ❑ **Introduction to Architecture views**
 - 4+1 View Model
 - Siemens View Model
 - SEI View Model
- ❑ **Documenting Architecture**
 - Documenting Architecture using SEI View Model
 - ❑ Documenting C&C View
 - ❑ Documenting Module View
 - ❑ Documenting Allocation View
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ **Summary**

Styles of the Allocation View

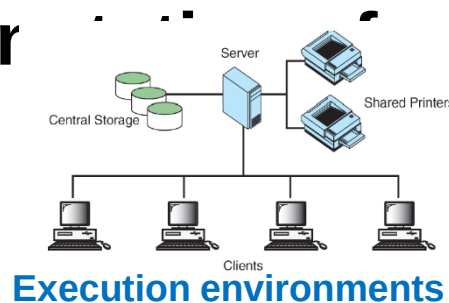
❑ **Allocation views** present a mapping between software elements and non-software elements in the software's environment

❑ The specific uses and r specific.

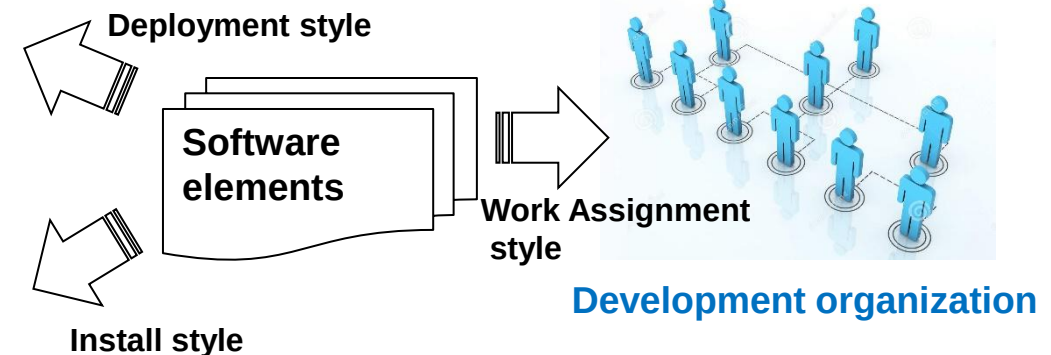
- Deployment Style

- Install Style

- Work Assignment Style



allocation views are style



Deployment Style

- ❑ **Mapping of components and connectors in the software architecture to the hardware of the computing platform**

- ❑ **Elements**
 - **Software element**
 - elements from a C&C view :
 - **Environmental element**
 - hardware of computing platform : processor, memory, disk, network, and so on.

Deployment Style

- ❑ **Elements of a C&C style are usually allocated to computing platforms**
- ❑ **Elements**
 - **Software element:** elements from a C&C view
 - ❑ **Properties (required)**
 - Processing requirements, resource requirements, and fault tolerance
 - **Environmental element:** computing hardware - processor, memory, disk, network, and so on.
 - ❑ **Properties (provided)**
 - CPU properties (processor clock speed, number of processors, etc.)
 - Memory properties (memory size and speed characteristics)
 - Disk or other storage unit capacity
 - Bandwidth, etc.

Deployment Style

□ Relations

■ Allocated-to (static)

- showing on which physical units the software elements reside

■ Migrates-to (dynamic)

- A software element can move from processor to processor

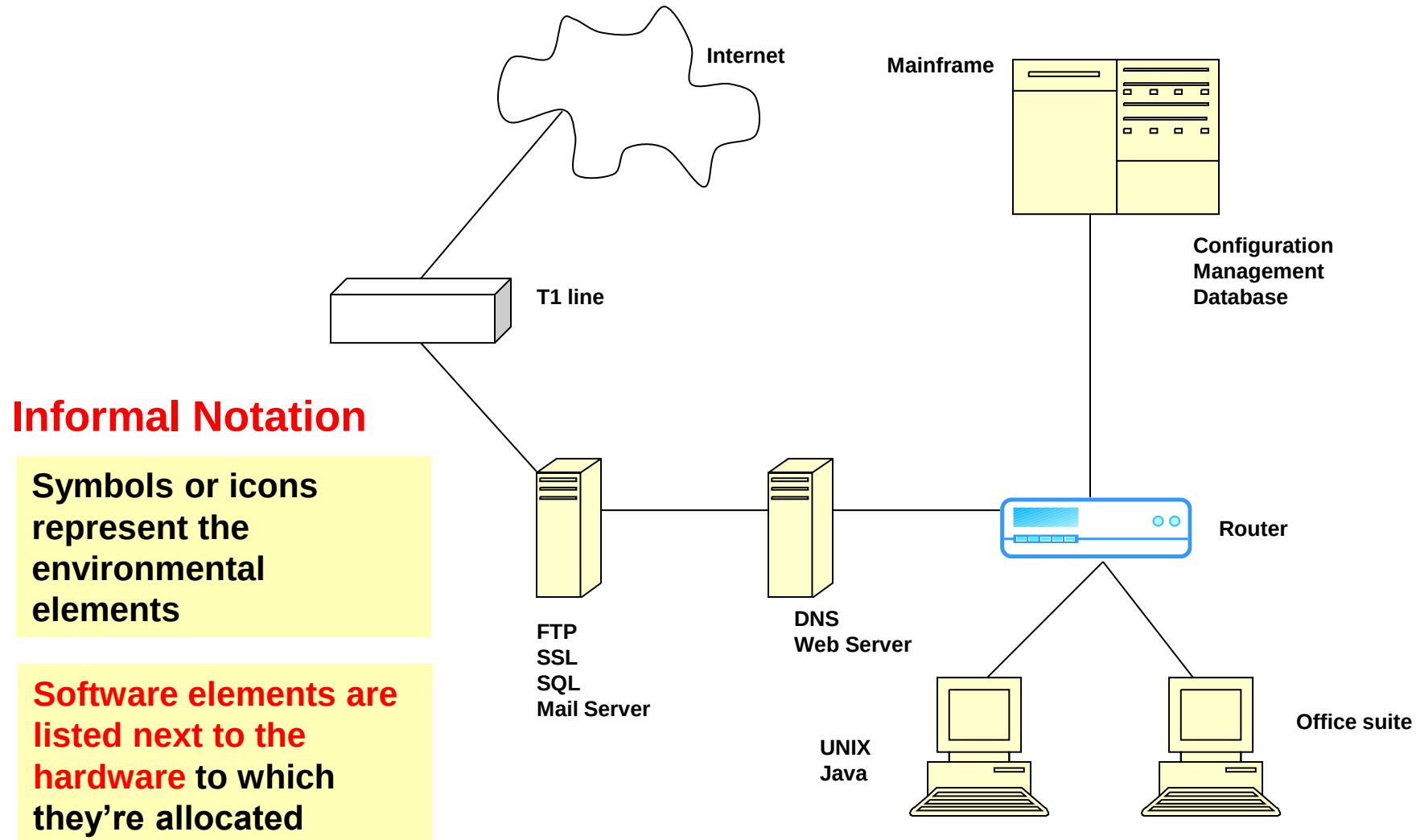
■ Copy-migrates-to (dynamic)

- A software element sends a copy of itself to the new processor

■ Execution-migrates-to (dynamic)

- A copy of a software element exists on more than one processor, but only one is active at any particular time.

Informal Notation for the Deployment Style

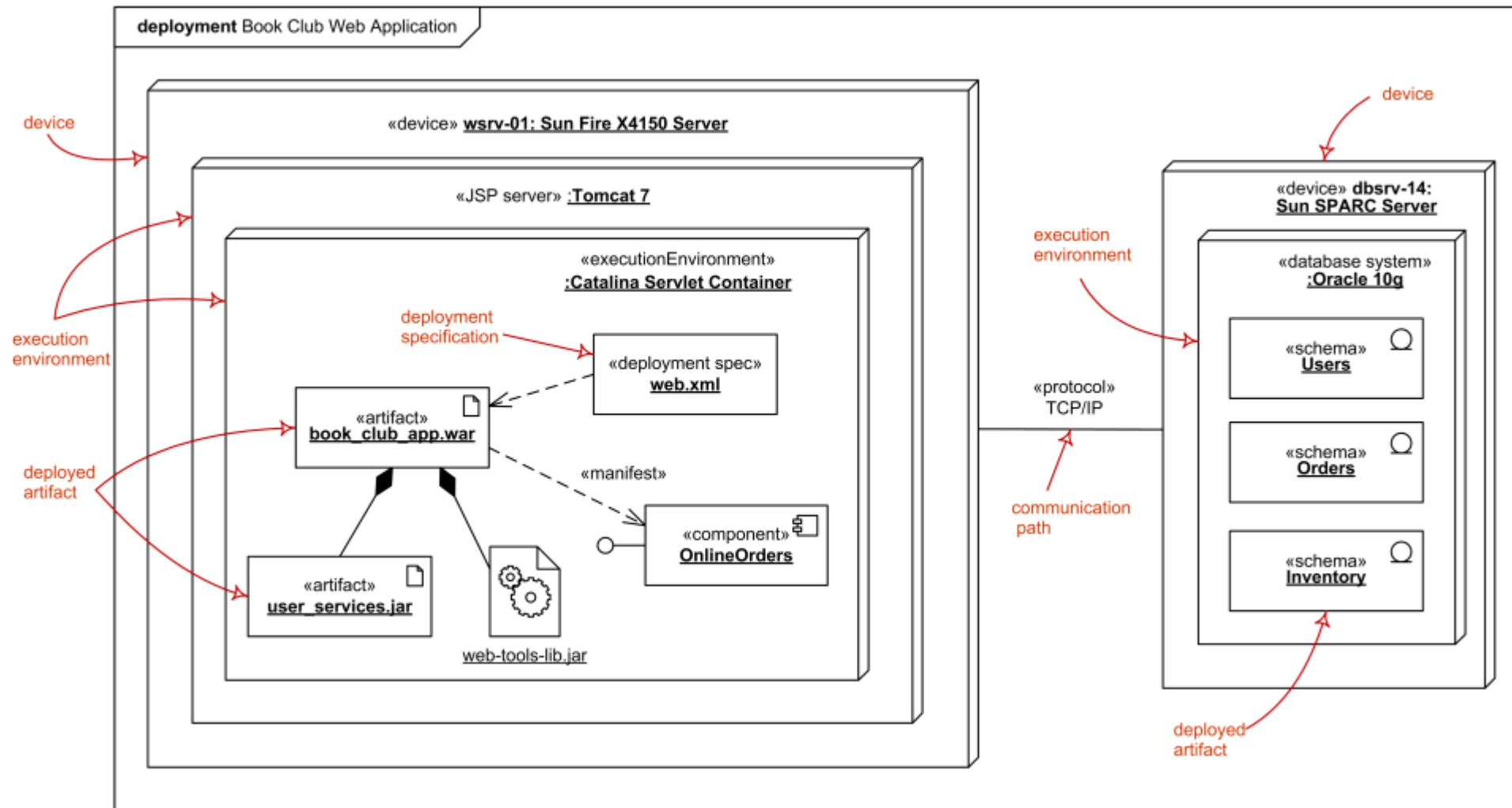


UML Deployment Diagram

□ Deployment diagram

- Represents the deployment of **software artifacts** to deployment targets (**nodes**)
 - **Nodes** is either **device** or some **software execution environment**
 - **Software artifacts** represent **concrete elements in the physical world** that are the result of a development process.
 - Examples of artifacts
 - executable files,
 - libraries,
 - archives,
 - database schemas,
 - configuration files, etc.

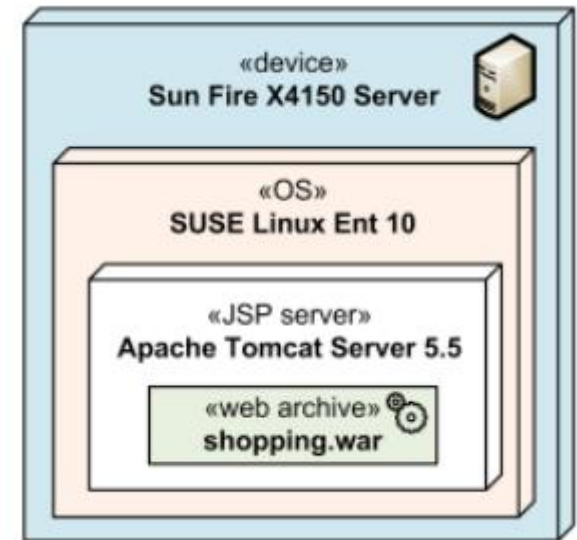
Deployment Diagram Example



Notations of the Deployment Diagram

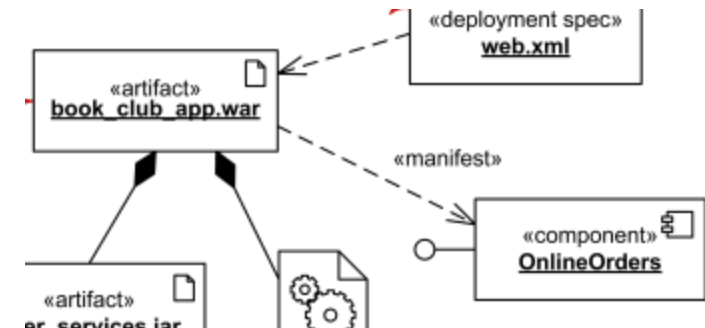
□ Node

- Shown as 3D box shape
- Several **software execution environments** can be nested into a server device



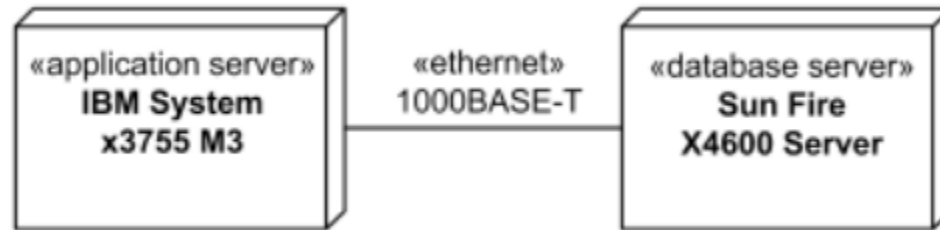
□ Artifact

- Denoted by a rectangle with Artifact name and document icon
- **<<manifest>>** relationship
 - Describe how the physical artifacts maps to logical components

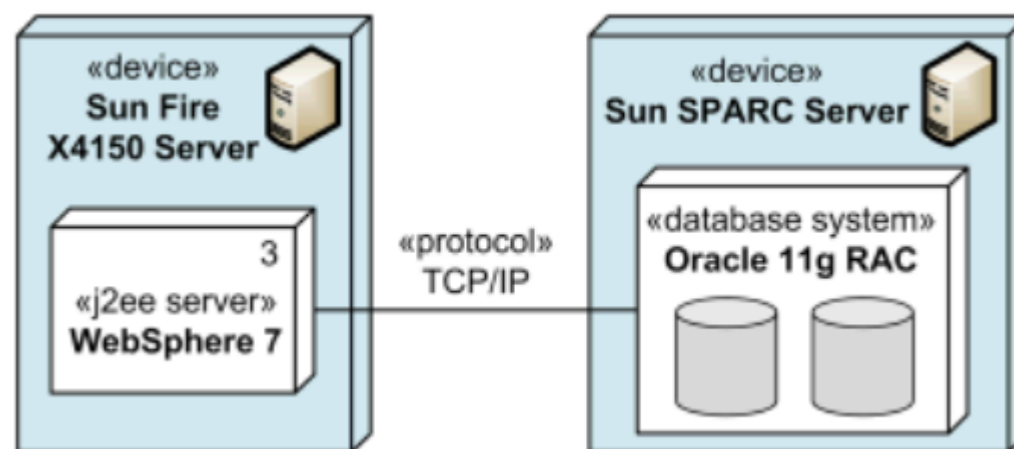


Notations of the Deployment Diagram

- ❑ A communication path represents
 - A physical link between physical device nodes

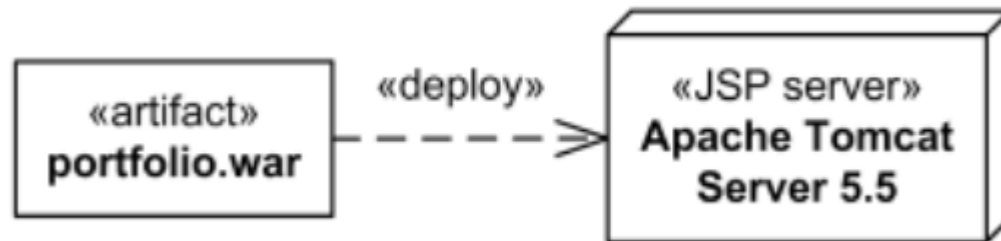


- Some protocol between software execution environment nodes

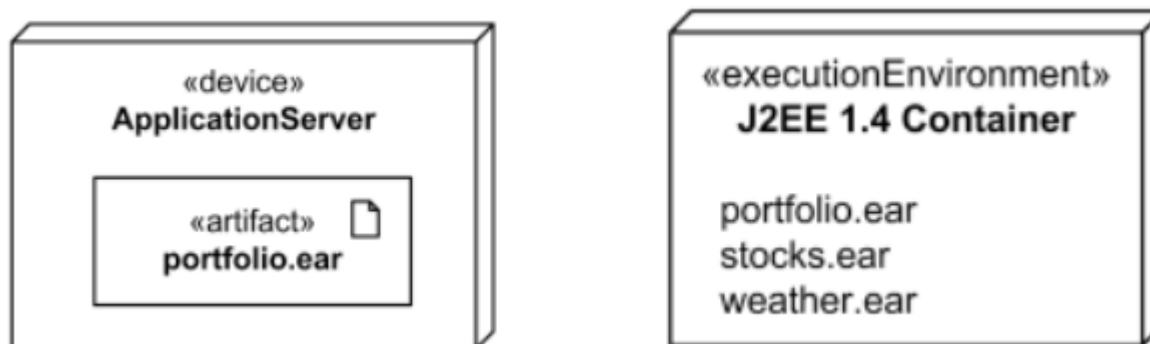


Notations of the Deployment Diagram

- ❑ A deployment is a dependency relationship which describes allocation of an artifact to a deployment target.
 - Dependency arrow from the artifact to deployment target

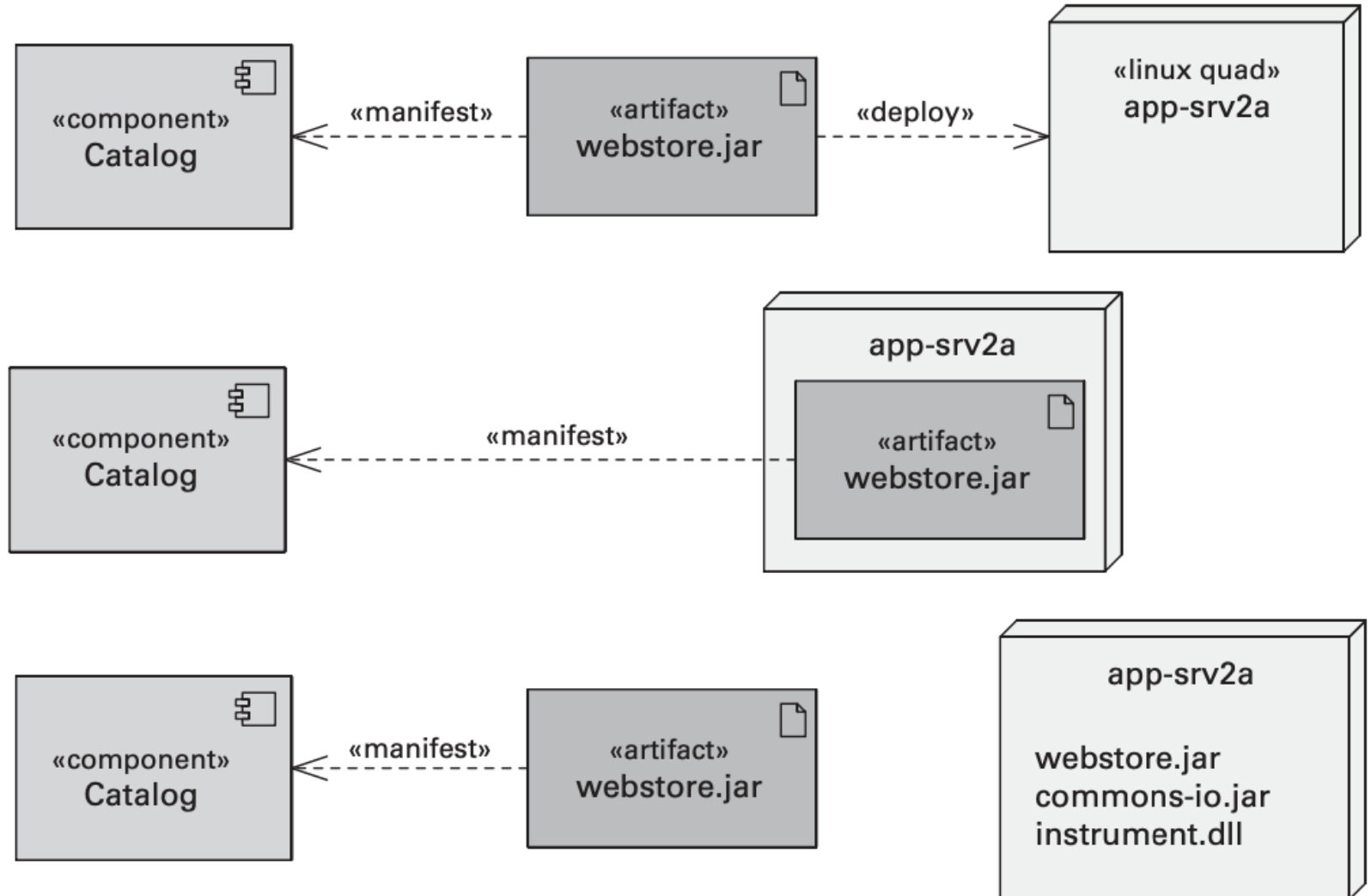


- Artifacts contained by a deployment target



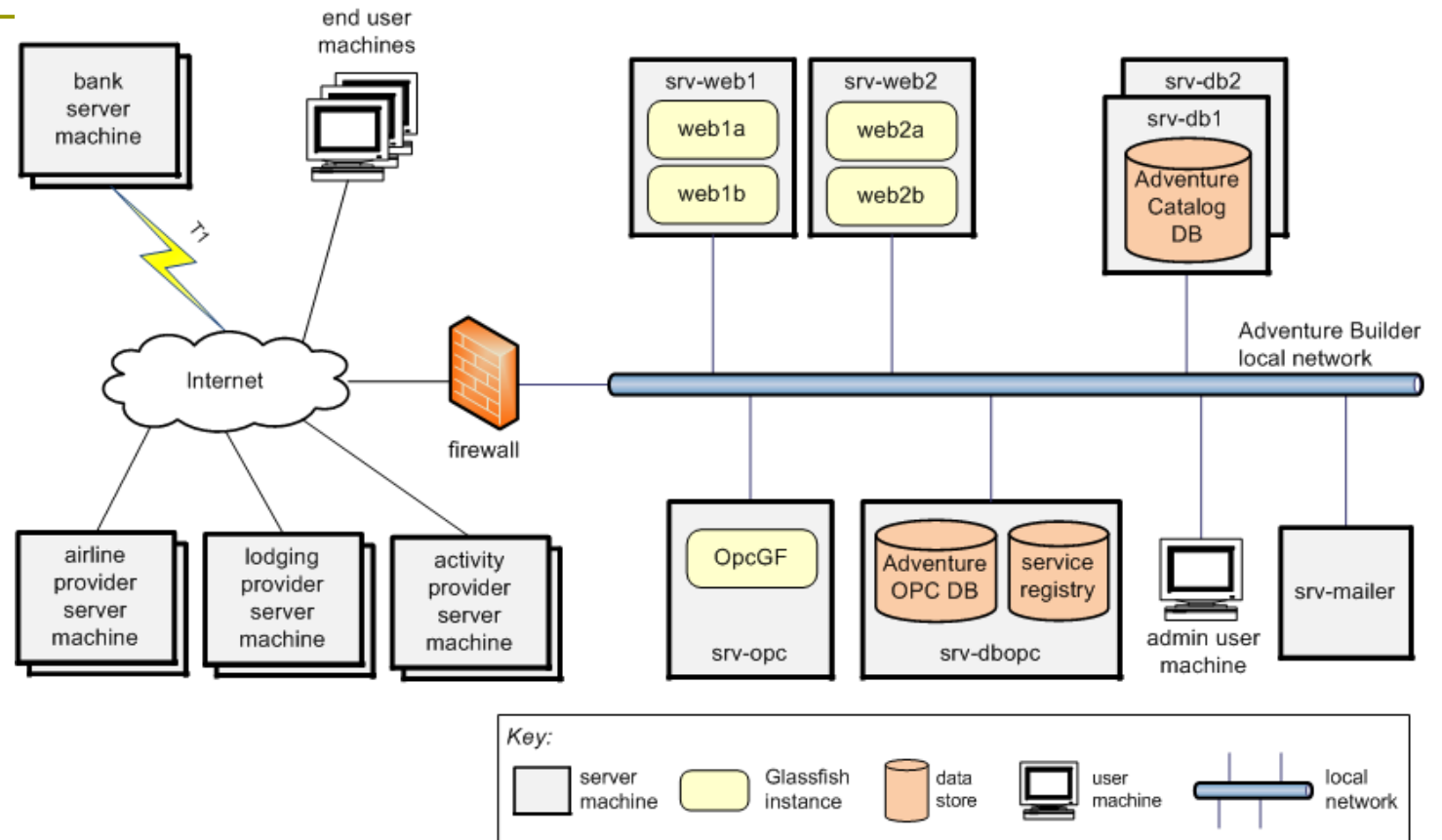
Notations of the Deployment Diagram

□ Three UML alternatives



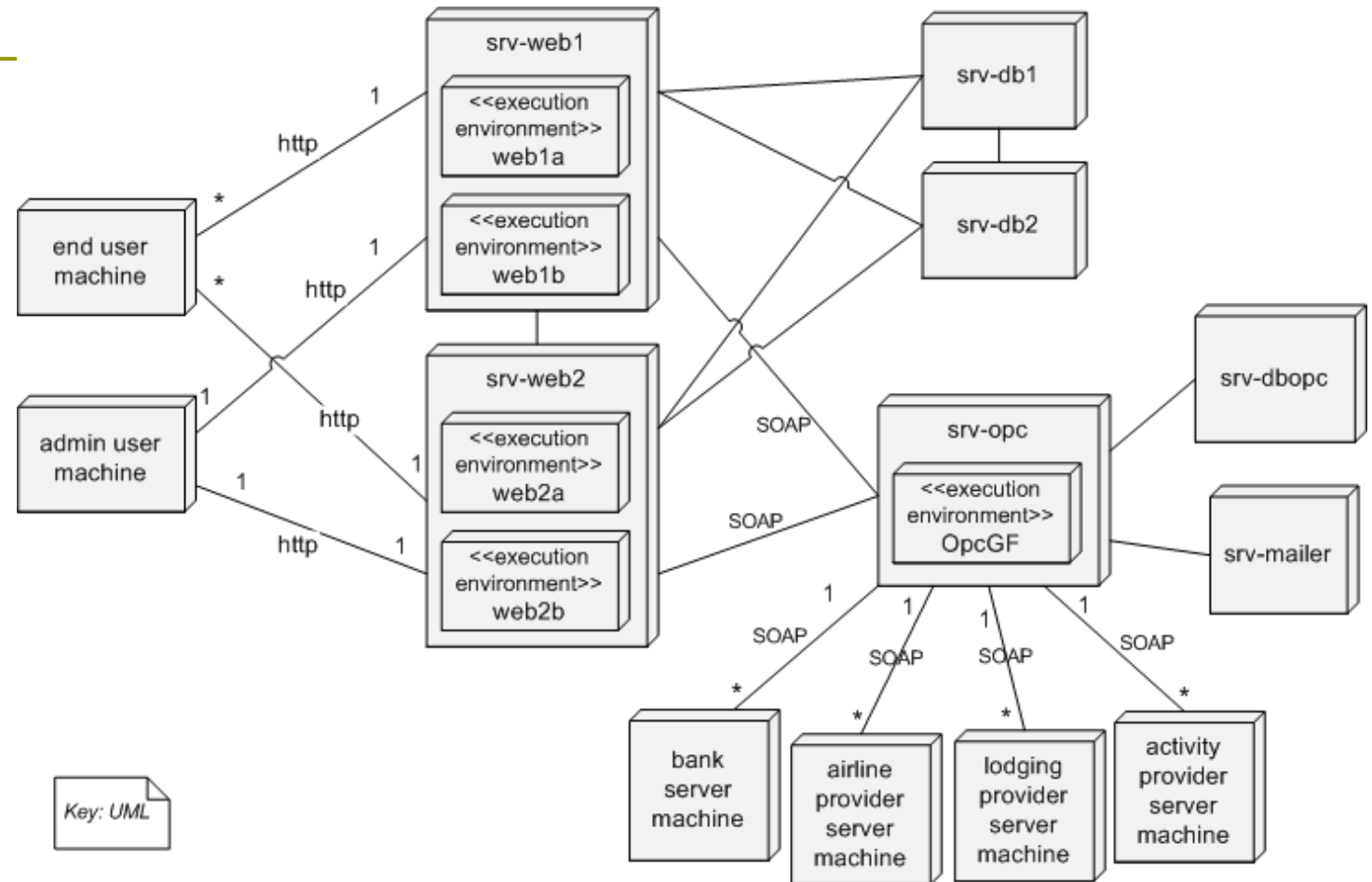
Deployment Diagram: Adventure Builder Example

- <https://wiki.sei.cmu.edu/confluence/display/SA/D/Deployment+View>



Deployment Diagram: Adventure Builder Example

- <https://wiki.sei.cmu.edu/confluence/display/SA/D/Deployment+View>



Install Style

- ❑ **The mapping of components in the software architecture to a file management system in the production environment**
- ❑ **Elements**
 - **Software element:** a C&C component
 - ❑ **Property (required)**
 - Requirements on production environments, such as a specific version of the Java runtime environment
 - **Environment element:** a configuration item, such as a file, or a folder
 - ❑ **Properties (provided)**
 - The characteristics provided by the production environments, such as how a configuration management system deals with histories and branches.

Install Style

□ Relations

■ Allocated-to

- A component is allocated to a configuration item

■ Containment

- A configuration item is contained in another

□ Constraints

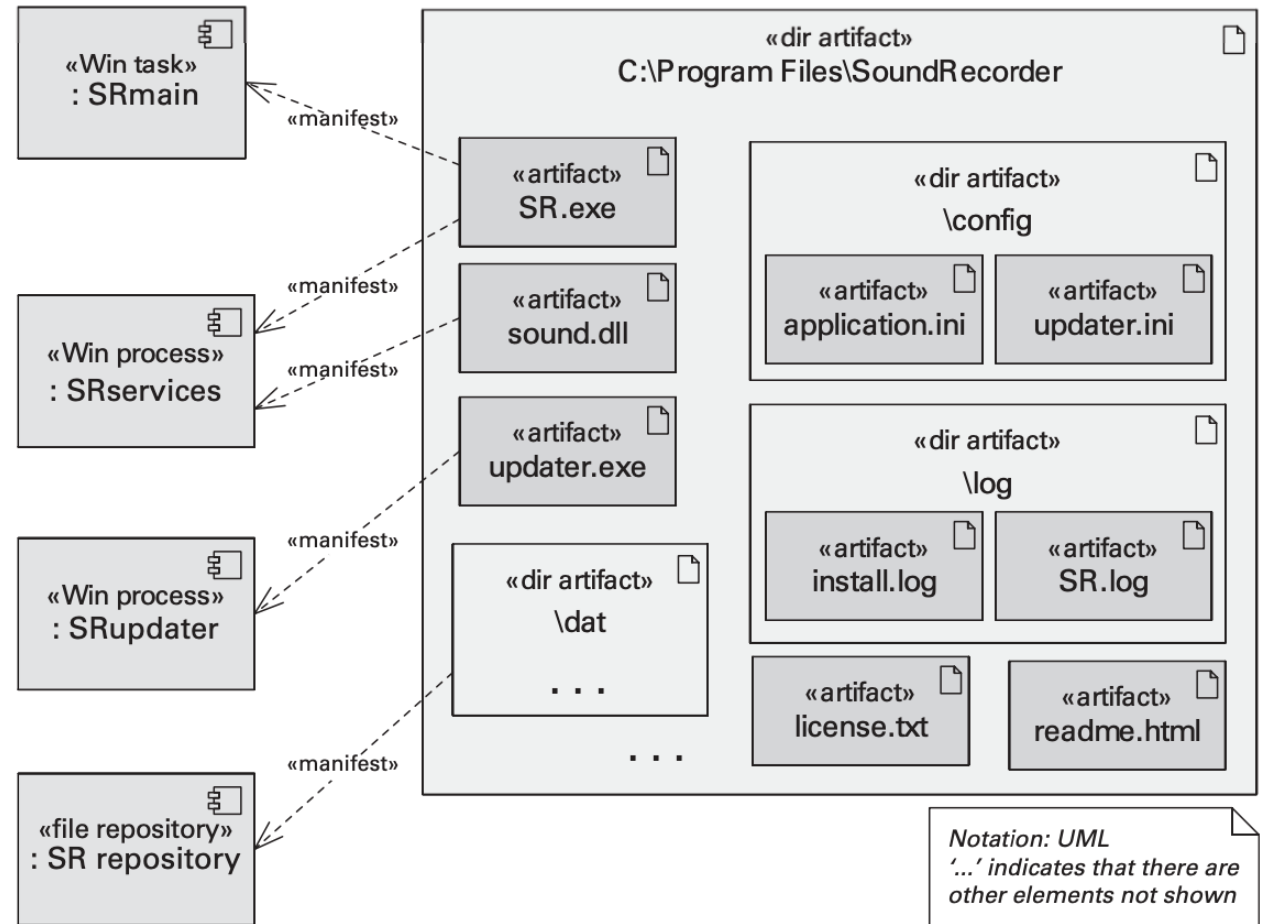
- Files and folders are organized in a tree structure, following an *is-contained-in* relation.

Notations for the Install Style

□ UML notations

■ Use stereo types

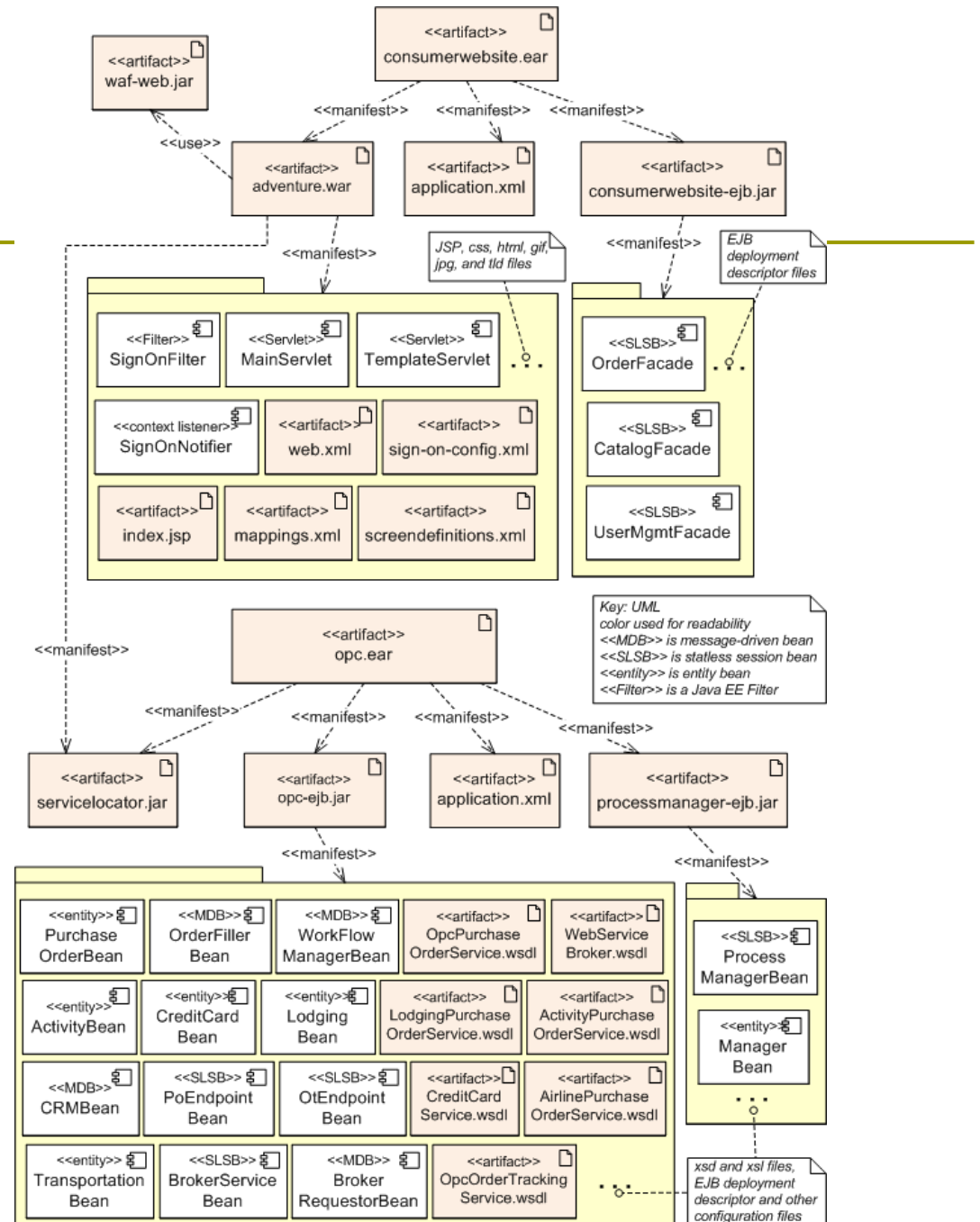
- <<artifact>> to denote a file (configuration item)
- <<manifest>> to indicate that the artifact contains the concrete physical representation of the software element



Install Style: Adventure Builder Example

□ <https://wiki.sei.cmu.edu/confluence/display/SAD/Install+View>

- **<<artifact>>** to denote a file (configuration item)
- **<<manifest>>** to indicate containments



Work Assignment Style

- ❑ **The mapping of the software architecture to the teams in the development organization.**
- ❑ **Elements**
 - **Software element:** a module
 - **Environment element:** an organizational unit, such as a person, a team, a department, and so on.
 - ❑ **Properties (provided)**
 - Skill set and capacity in terms of labor and calendar time available
- ❑ **Relations**
 - Allocate-to
- ❑ **Constraints**
 - Usually restricted that one module is allocated to one organizational unit

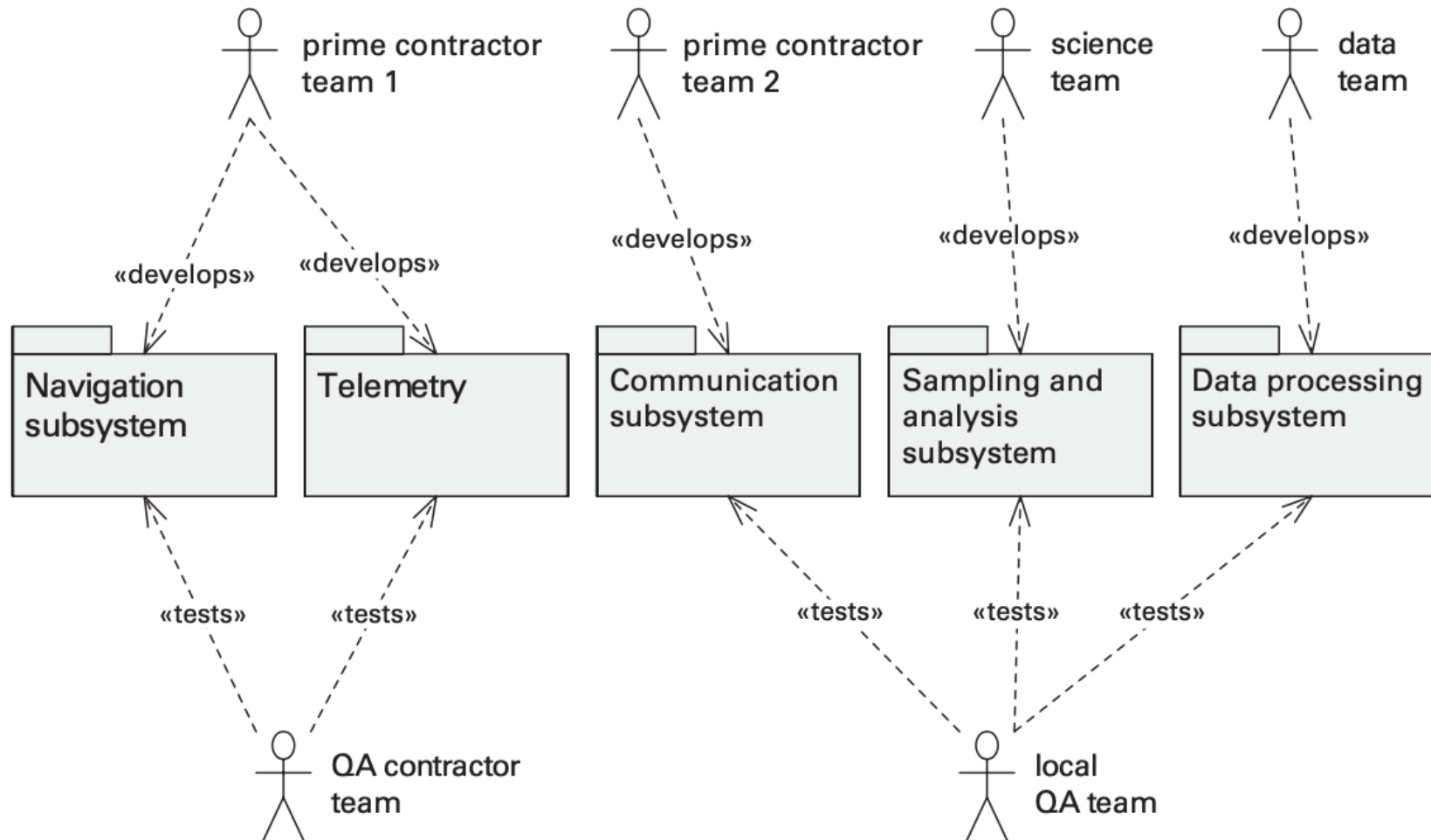
Notations for the Work Assignment Style

▣ Tabular notes

ECS Element (Module)		Organizational Unit
Segment	Subsystem	
Science Data Processing Segment	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment	Planning and Scheduling	Orbital vehicle team
	...	

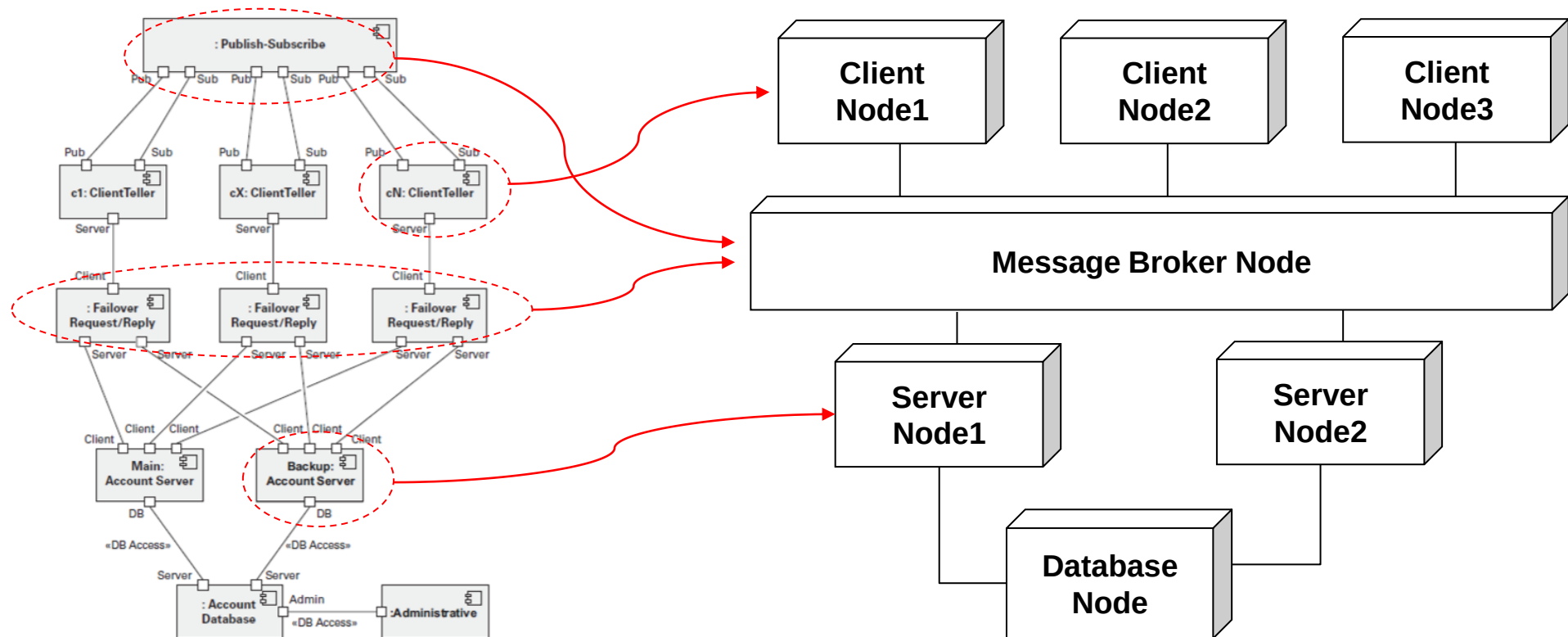
Notations for the Work Assignment Style

□ UML



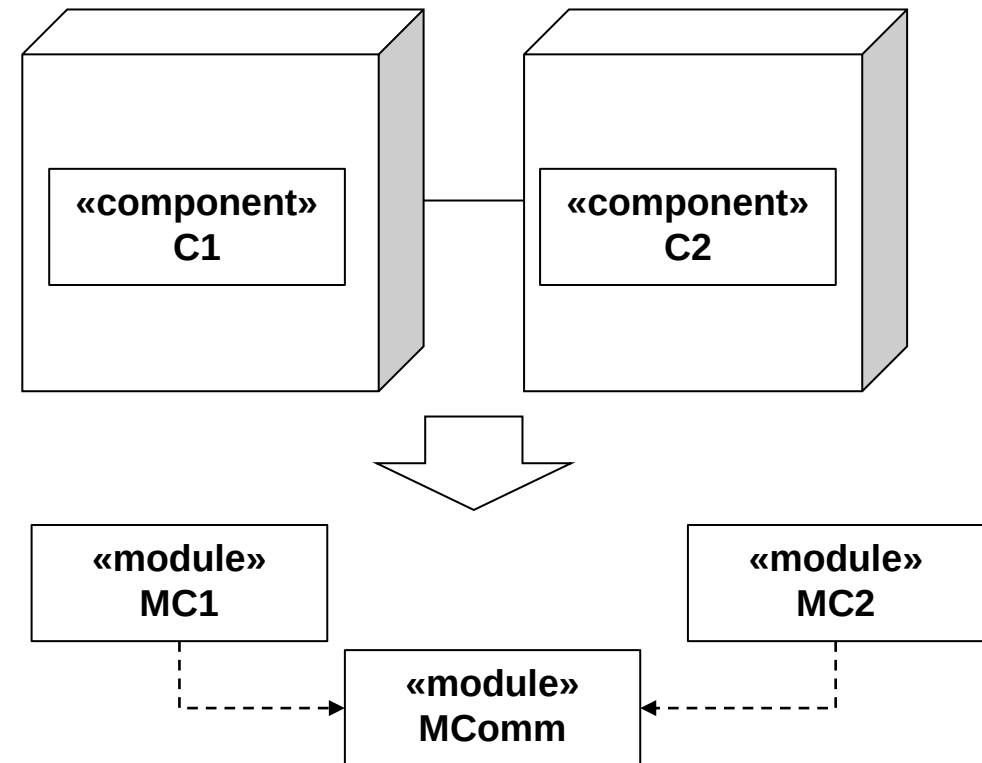
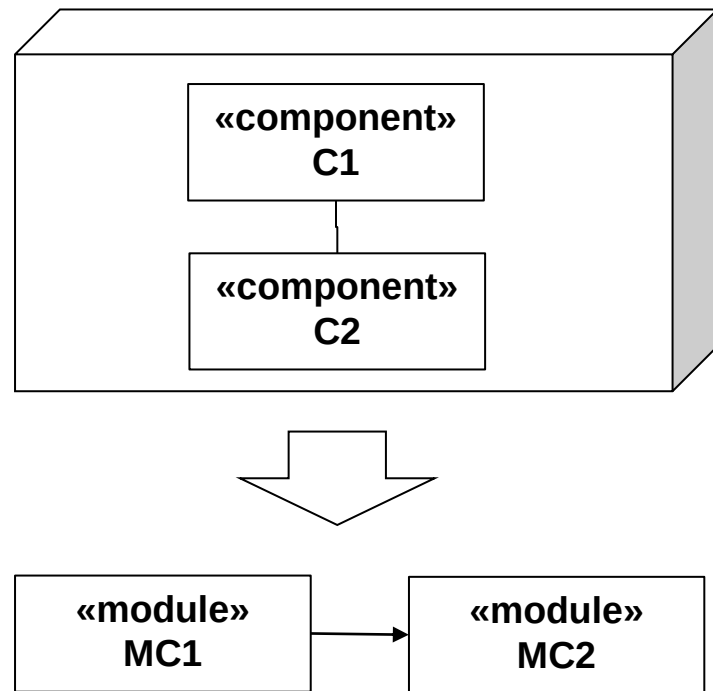
Relationship b/w C&C and Allocation views

- A C&C component is mapped onto the hardware of the computing platform in the allocation view



Relationship among C&C, Module, and Allocation views

- ❑ Variability in the deployment of C&C components to hardware nodes result in different module design



Summary of Allocation View

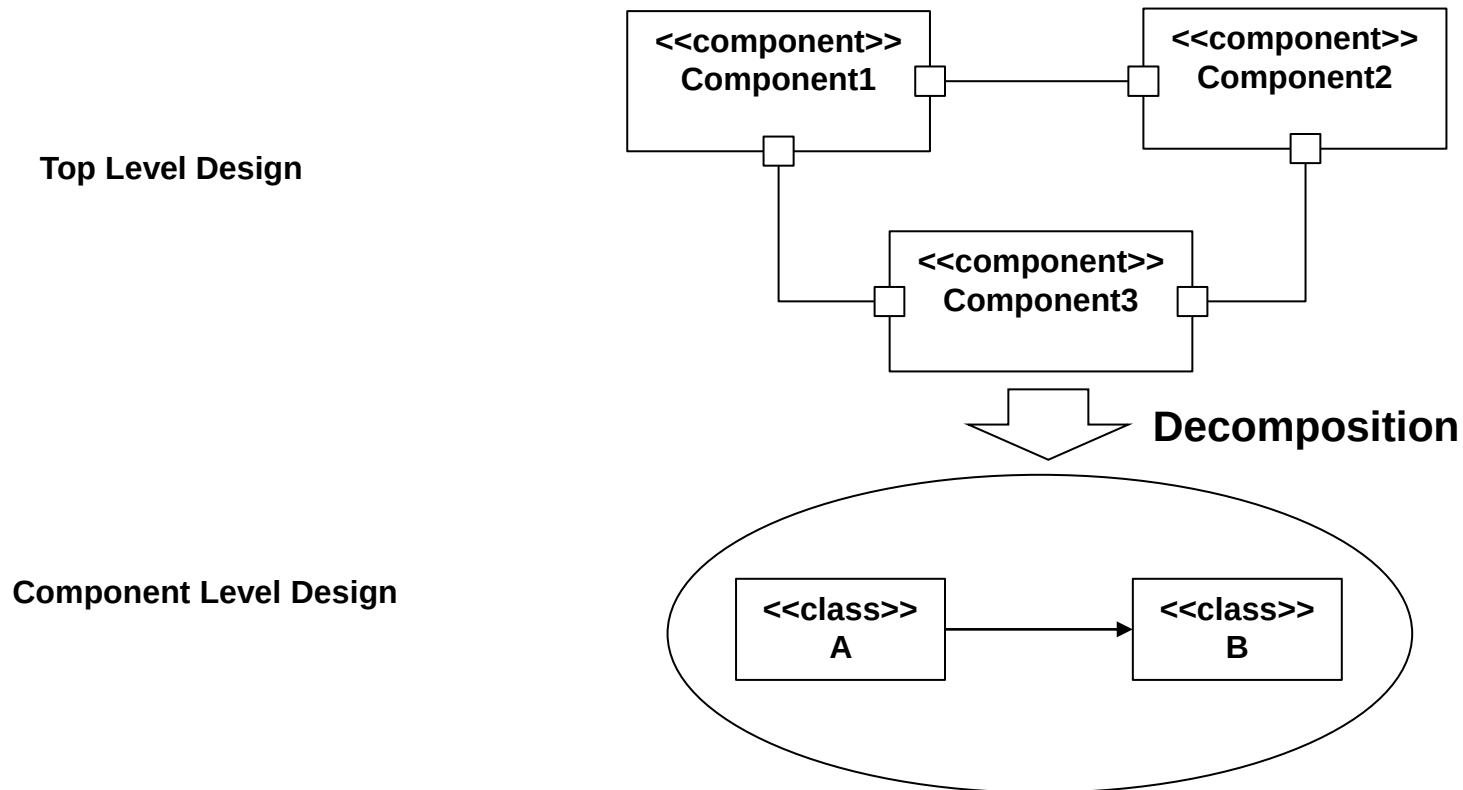
- ❑ **Styles in the allocation view map software structures to structures in the environment of the software**
 - ❑ The **deployment** style describes the mapping of runtime software elements to the hardware on which the software executes
 - This **install** style describes the mapping of components in the software architecture to a file system in the production environment
 - ❑ This **work assignment** style describes the mapping of the software architecture to the teams in the development organization.

Topics

- ❑ Introduction to Architecture views
 - 4+1 View Model
 - Siemens View Model
 - SEI View Model
- ❑ Documenting Architecture
 - Documenting Architecture using SEI View Model
 - Documenting Architecture using Hierarchical Multi-View Model
 - Documenting Architecture using C4 Model
- ❑ Summary

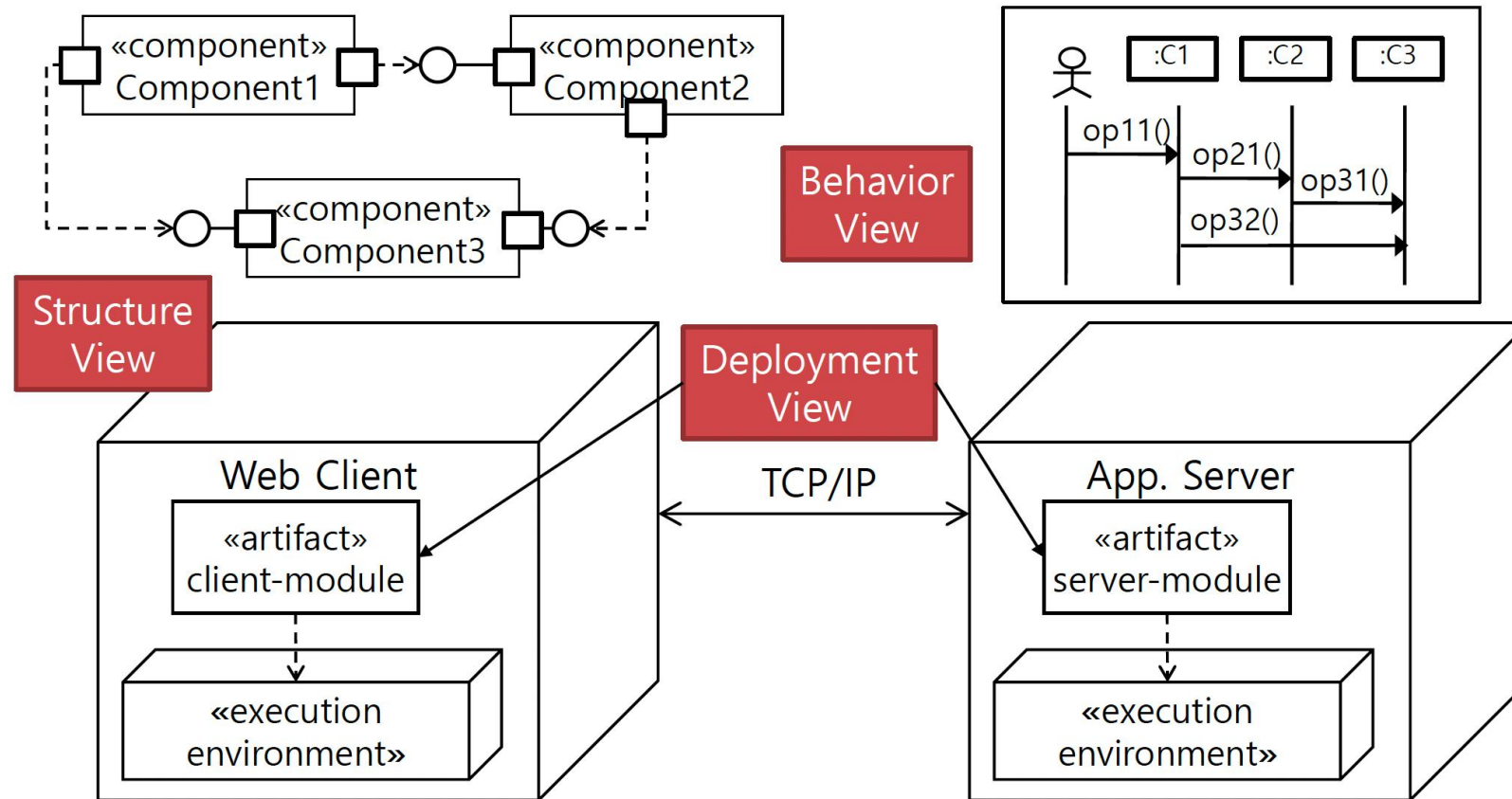
Architectural Design

- A system' structures are iteratively decomposed



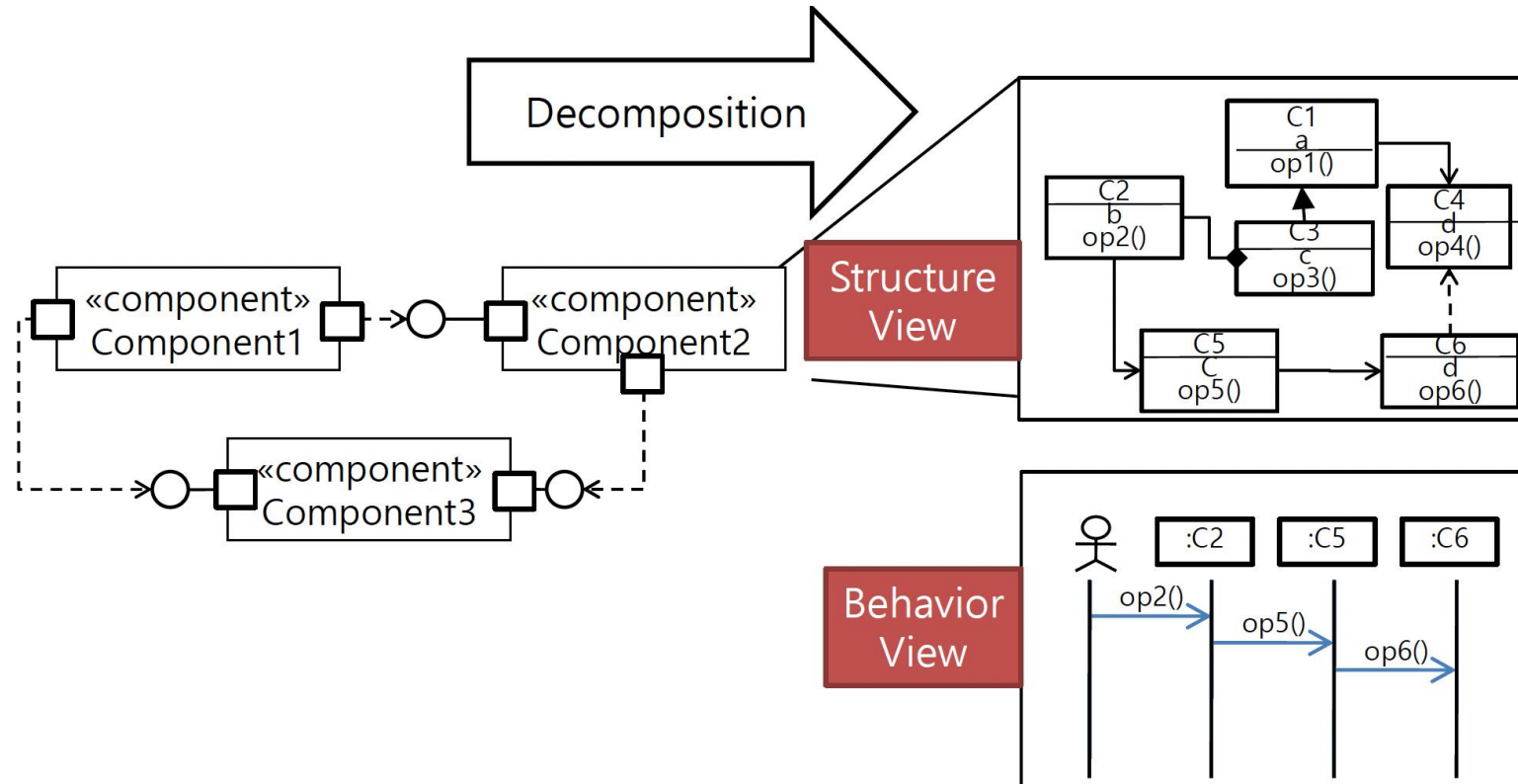
Top Level Design

□ The first level decomposition of the system



Component Level Design

- Decompose each component (in the top-level design) into fine-grained elements

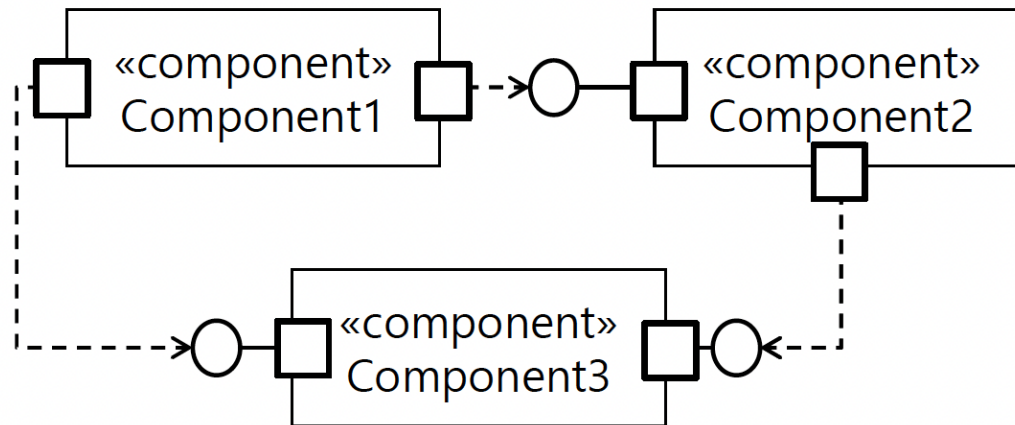


View for Architecture Documentation

	Top Level Design	Component Level Design
Structure View	Component Diagram Component Specification	Class Diagram
Behavior View	Interaction Diagram	Interaction Diagram
Deployment View	Deployment Diagram	

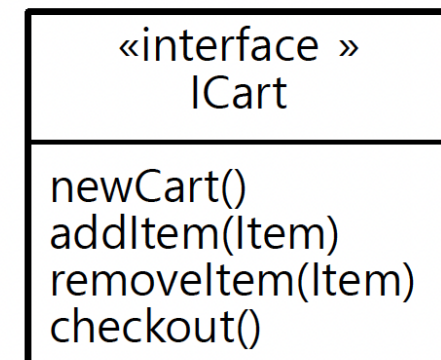
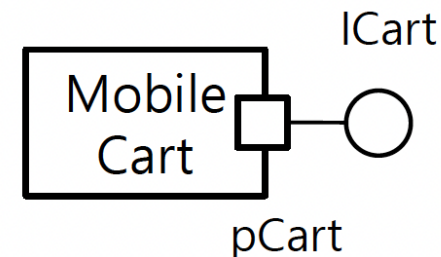
Structure View

UML Component Diagram (Static Structure)



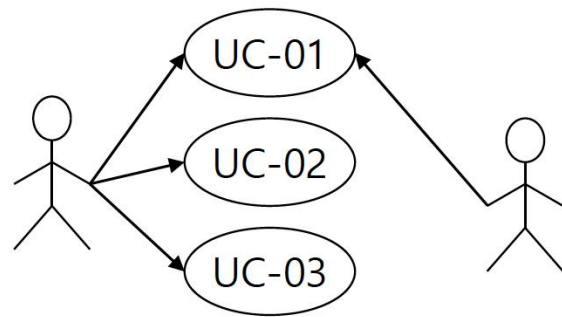
Component Specification

- Port
- Interface Specification

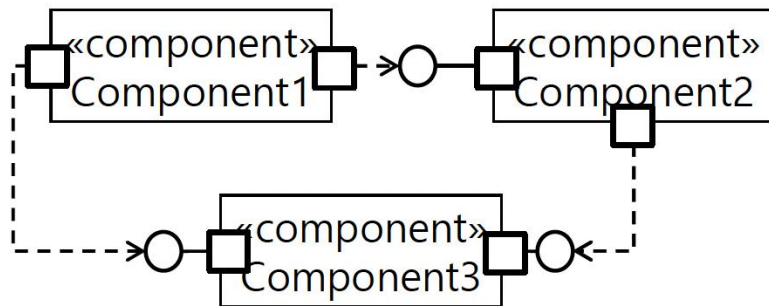


Behavior View

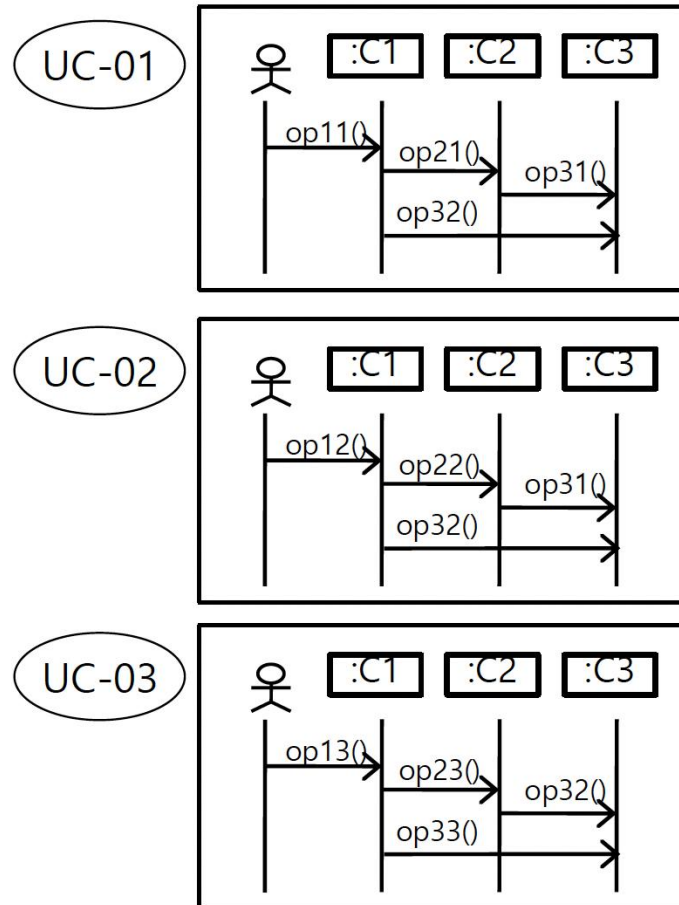
- For each use case, analyze interactions among components



Use Case Model

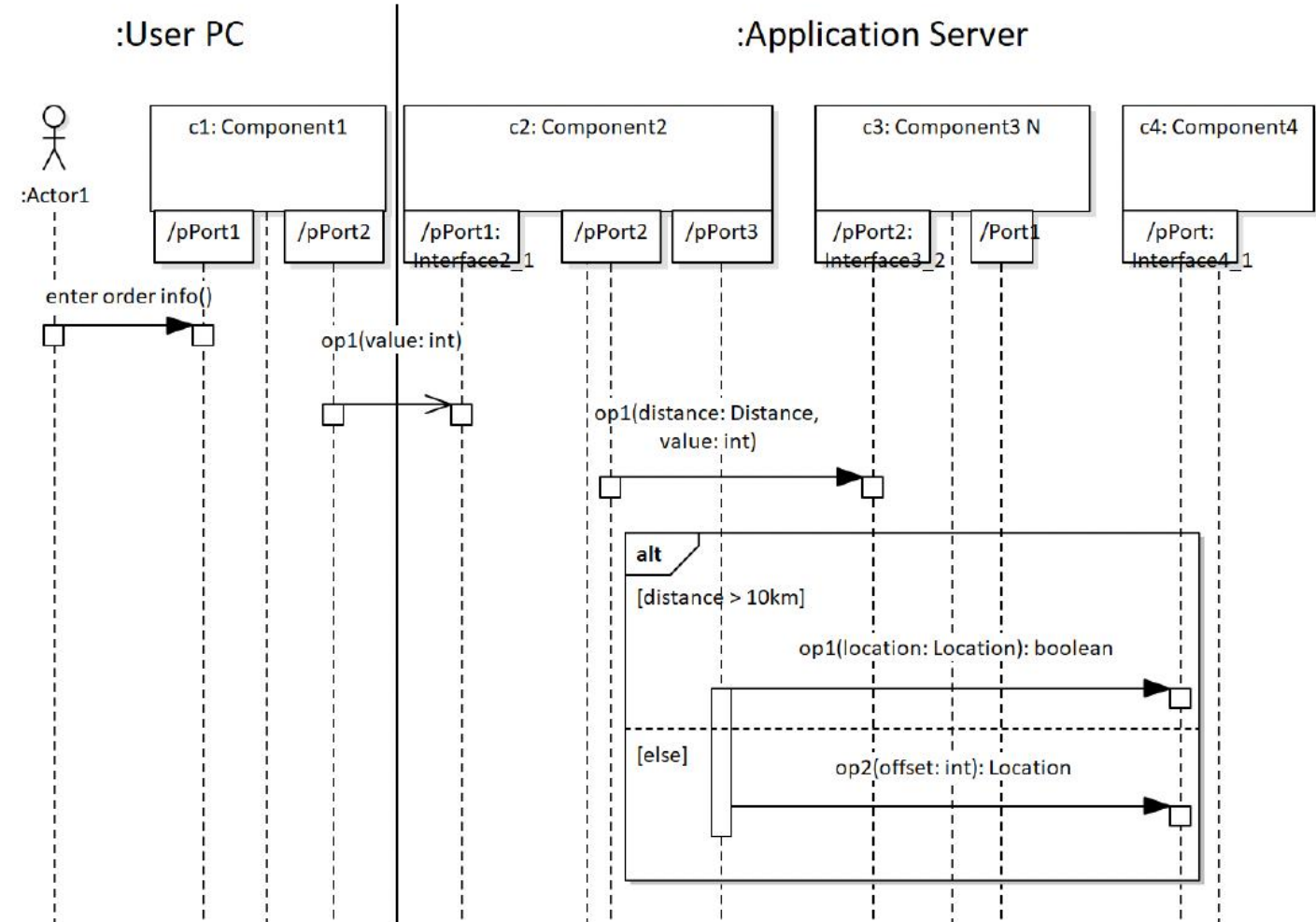
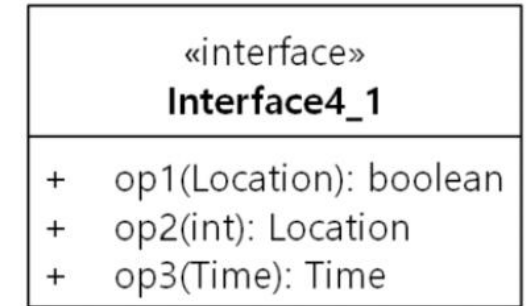
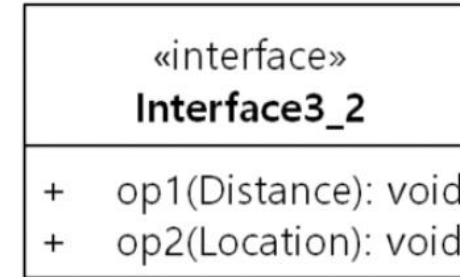
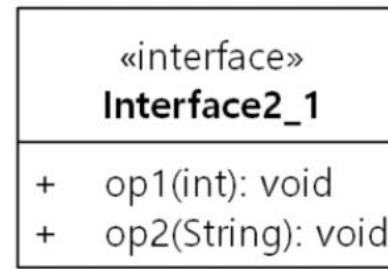


Static Structure Model



Behavior View

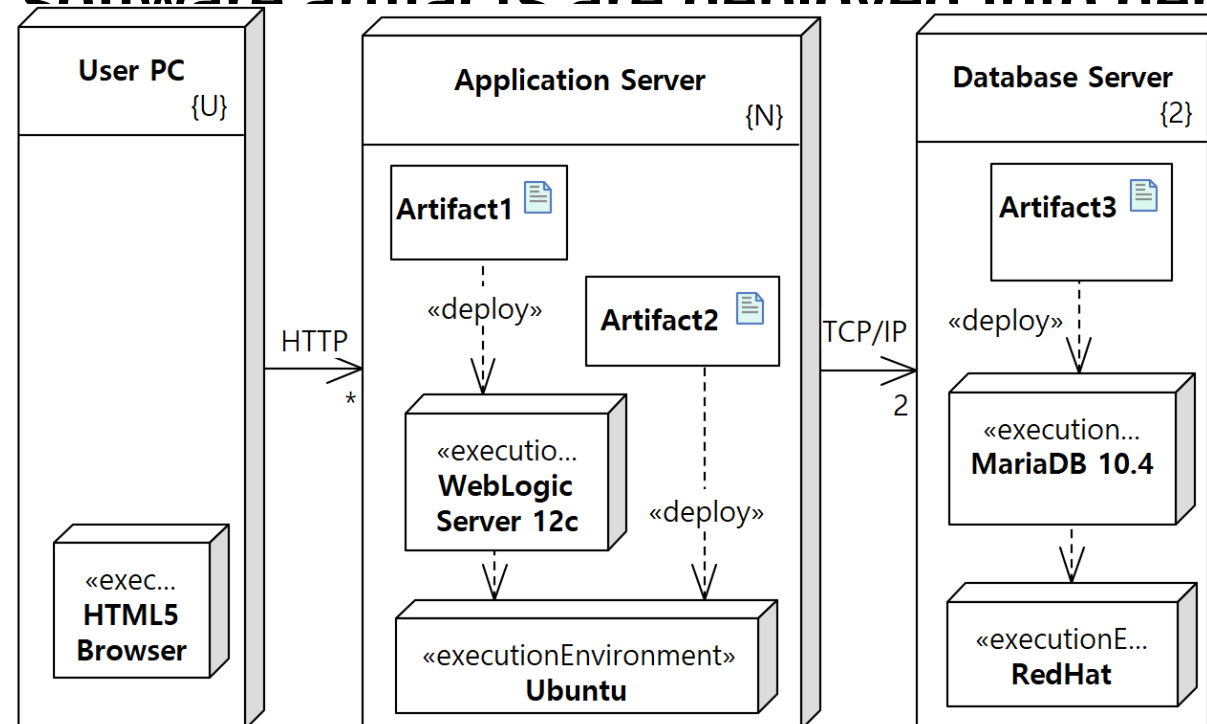
- Describe interactions among component instances through ports



Deployment View

□ Deployment Diagram

- Show key infrastructure elements (i.e., nodes) and communication paths
- Show how software artifacts are deployed into deployment targets (nodes)



SEI View Model vs. Hierarchical Multi-View Model

	Hierarchical Multi-View Model	SEI View Model
주요 초점	설계 단계별로 점진적 분해 과정을 강조	이해 관계자별 시스템 구조 표현 강조
View 구성	Top Level Design -Structure View (Component Diagram) -Behavior View	C&C View - run time structure (Component Diagram)
	Component Level Design – Structure View (Class Diagram) – Behavior View	Module View (Class Diagram) - development structure
	Deployment Diagram	Allocation View
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컴포넌트 → 클래스	직교적/독립적 (Orthogonal): 세 가지 View는 서로 다른 관심사를 다루며 동시에 존재함.

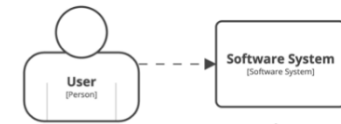
Topics

- ❑ **Introduction to Architecture views**
 - **4+1 View Model**
 - **Siemens View Model**
 - **SEI View Model**
- ❑ **Documenting Architecture**
 - **Documenting Architecture using SEI View Model**
 - **Documenting Architecture using Hierarchical Multi-View Model**
 - **Documenting Architecture using C4 Model**
- ❑ **Summary**

C4 Model

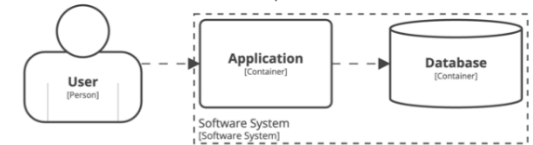
- 소프트웨어 아키텍처를 추상화 수준에 따라 4 단계의 계층 (Context, Container, Component, Code) 으로 문서화하는 경량화된 모델
 - 다양한 이해관계자에게 적절한 수준의 정보 전달
 - 줌인/줌아웃 방식
 - 단순하고 명확한 표기법

System Context



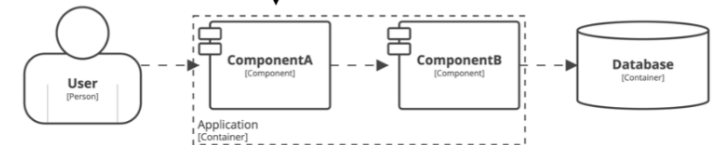
Zoom in to a software system

Containers



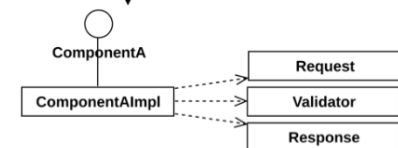
Zoom in to a container

Components



Zoom in to a component

Code



System Context Diagram

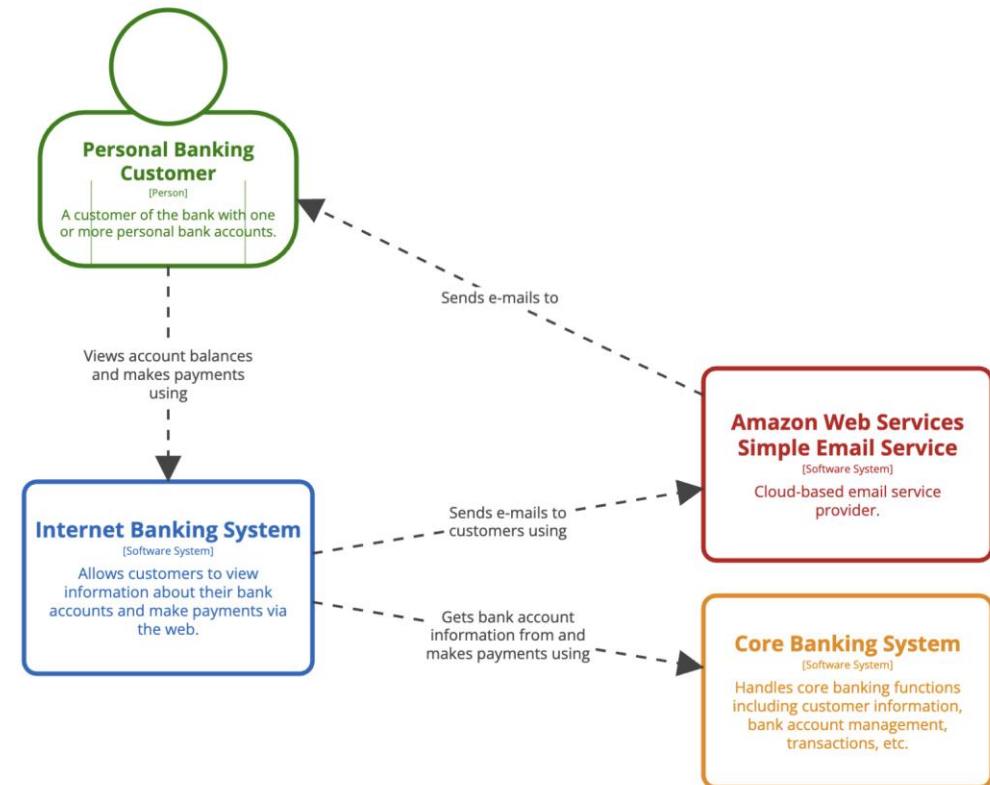
□ 소프트웨어 시스템이 외부 개체(사용자 및 다른 시스템)와 어떻게 상호작용하는 지 표현

■ 모델링 요소

- 시스템
- 사용자
- 외부시스템

■ 이해관계자

- 비즈니스 담당자
- 프로젝트/제품 관리자
- 기술팀 모두



System Context View: Internet Banking System
The system context diagram for a fictional Internet Banking System

Container Diagram

- 시스템 내부로 'zoom in'하여 시스템을 구성하는 주요 빌딩 블록들을 표현

- 모델링 요소

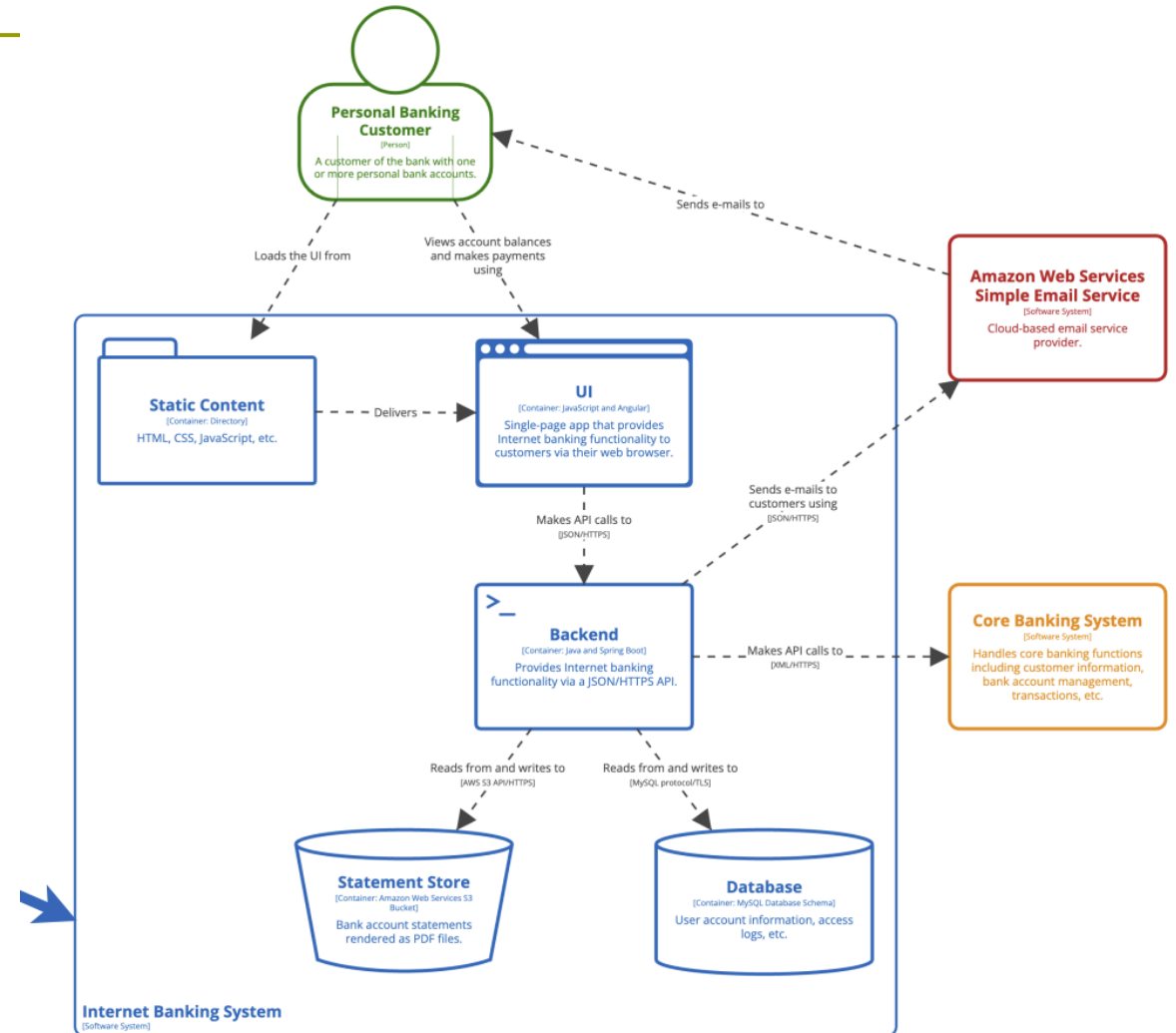
- 컨테이너 (Container) : 독립적으로 배포/실행 가능한 단위 (예, 애플리케이션, 데이터베이스)

- 사용자

- 외부시스템

- 이해관계자

- 개발팀
- 아키텍트
- 운영팀



Container View: Internet Banking System
The container diagram for the Internet Banking System

Component Diagram

- 특정 컨테이너 내부로 '쫂민'하여 컨테이너를 구성하는 컴포넌트들의 관계를 표현

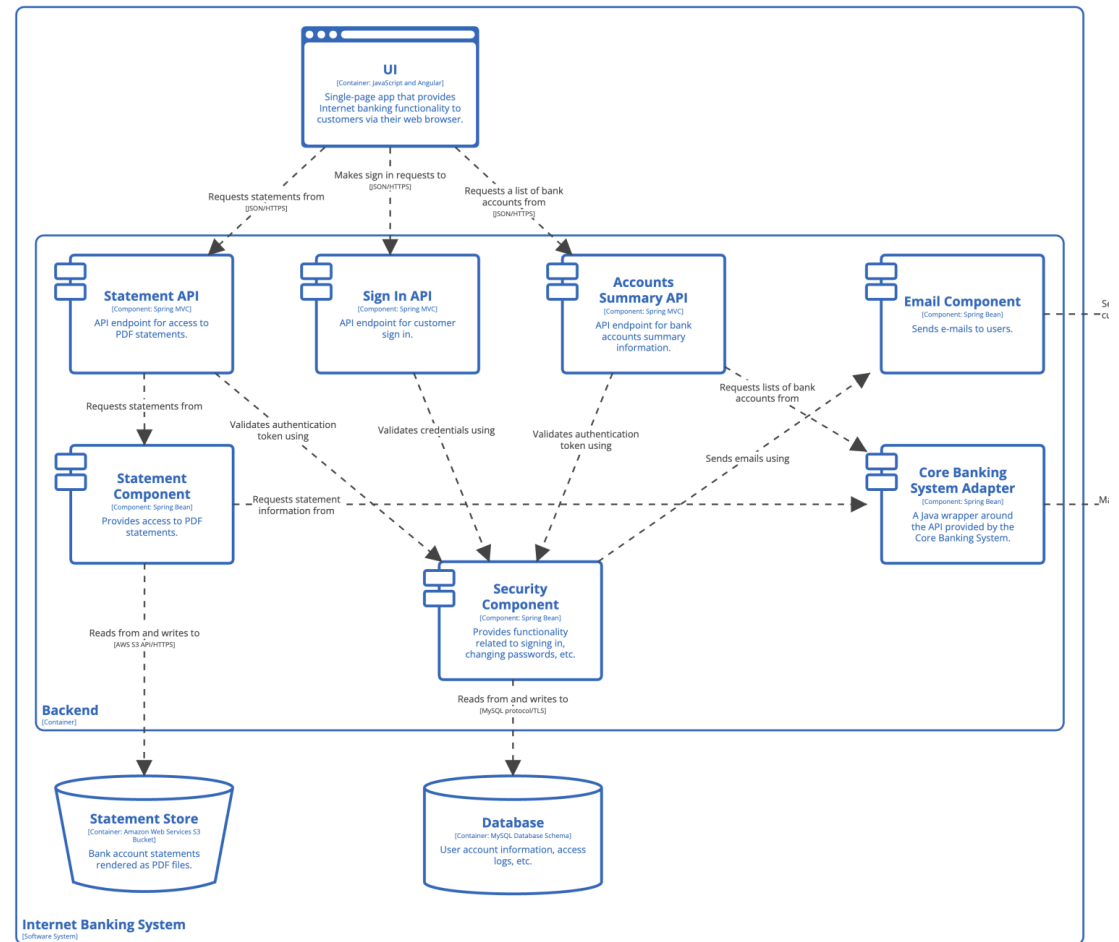
- 모델링 요소

- 컴포넌트: 기능을 제공하는 코드의 논리적 묶음, 독립적으로 배포될 수 없음

- 컨테이너

- 이해 관계자

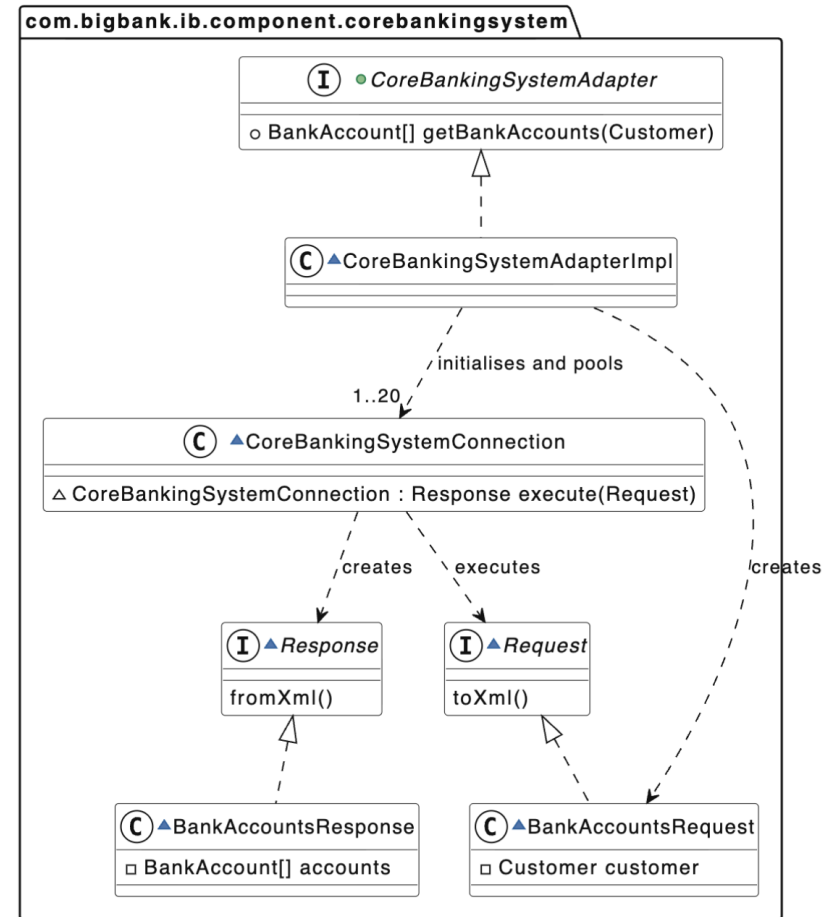
- 핵심 개발팀
 - 아키텍트



Component View: Internet Banking System - Backend
The component diagram for the Internet Banking System Backend

Code Diagram

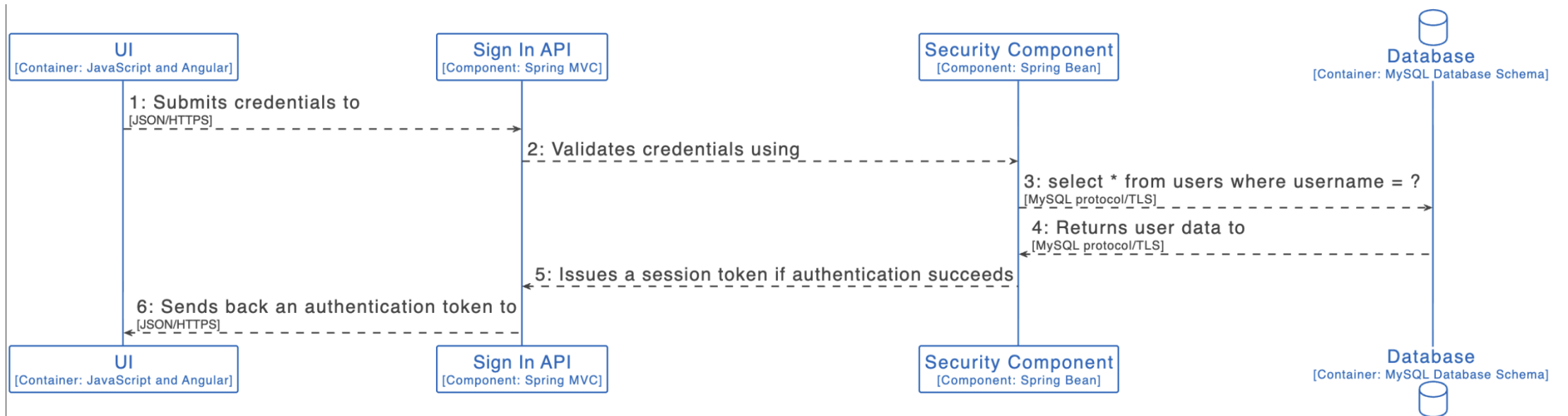
- 특정 컴포넌트 내부로 '쭈민'하여 코드를 구성하는 가장 상세한 구조를 표현
 - 모델링 요소
 - 클래스, 인터페이스, 함수
 - 이해관계자
 - 개발자
- 이 단계는 보통 **UML** 클래스 다이어그램 형태로 표현되며, 이상적으로는 코드에서 자동 생성



Code View: Internet Banking System - Backend - Core Banking System Adapter
A summary of the implementation details for the Core Banking System Adapter component

Supporting Diagram

□ Use dynamic diagrams to describe complex interactions

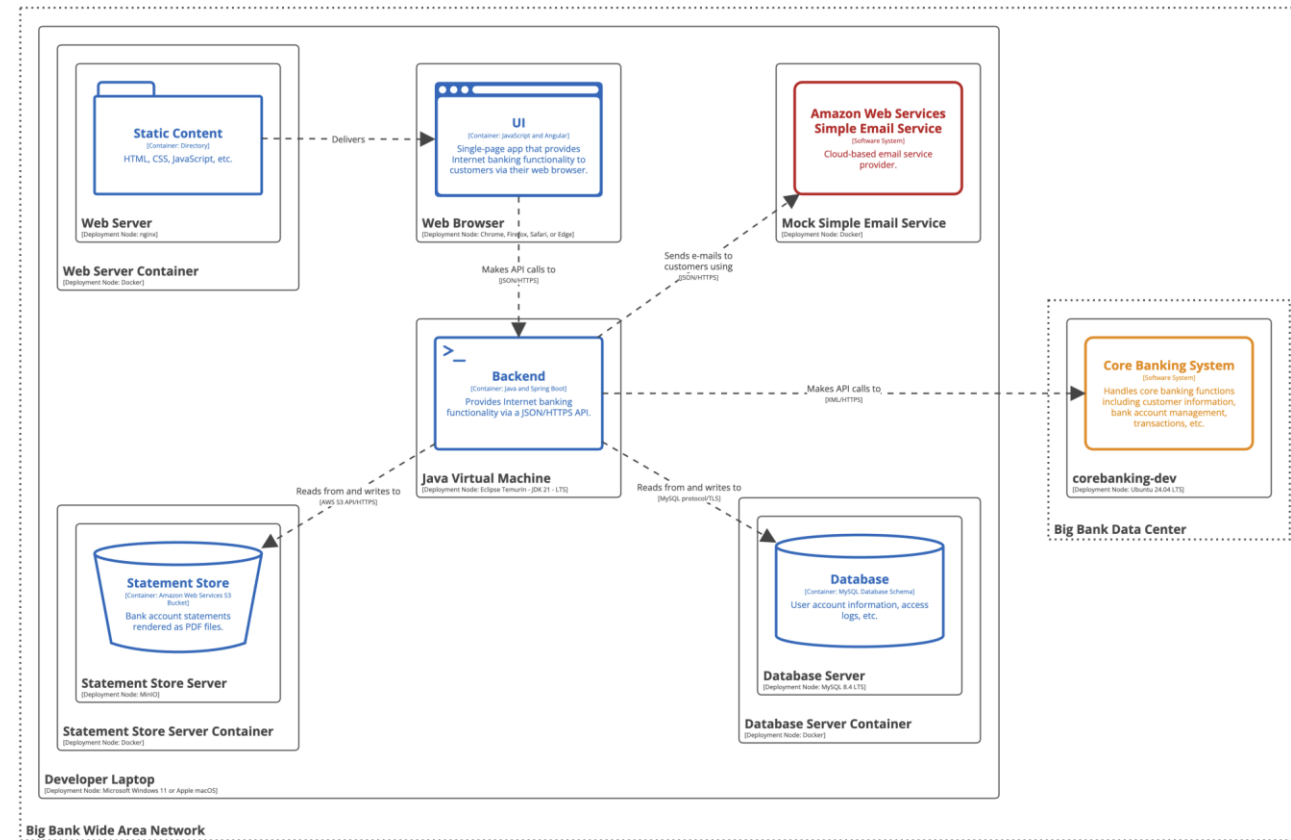


Dynamic View: Internet Banking System - Backend

Summarises how the sign in feature works in the single-page application

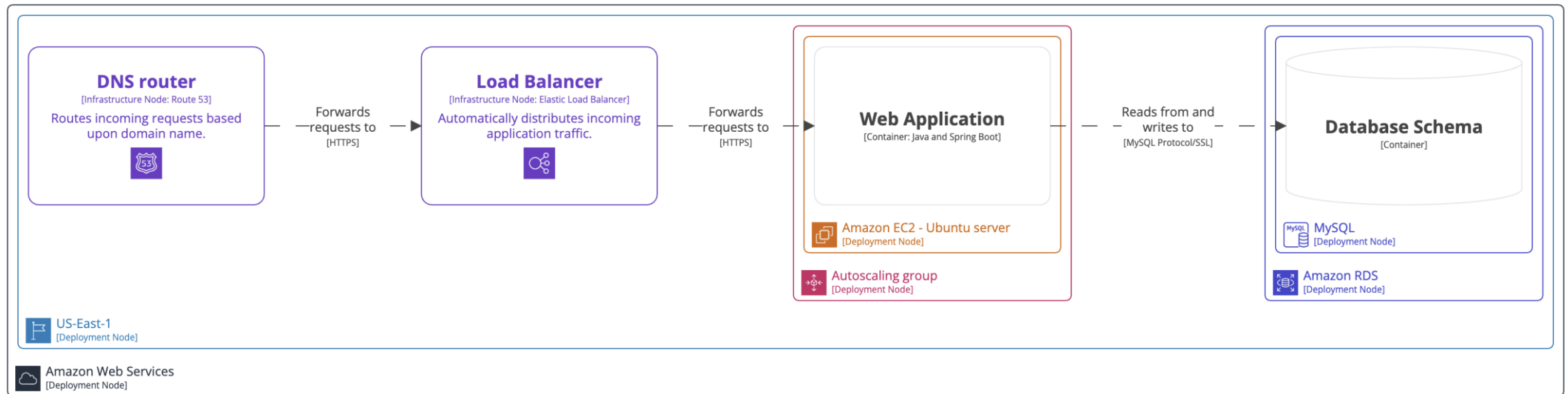
Supporting Diagram

- Use deployment diagrams to describe the mapping of containers to infrastructure



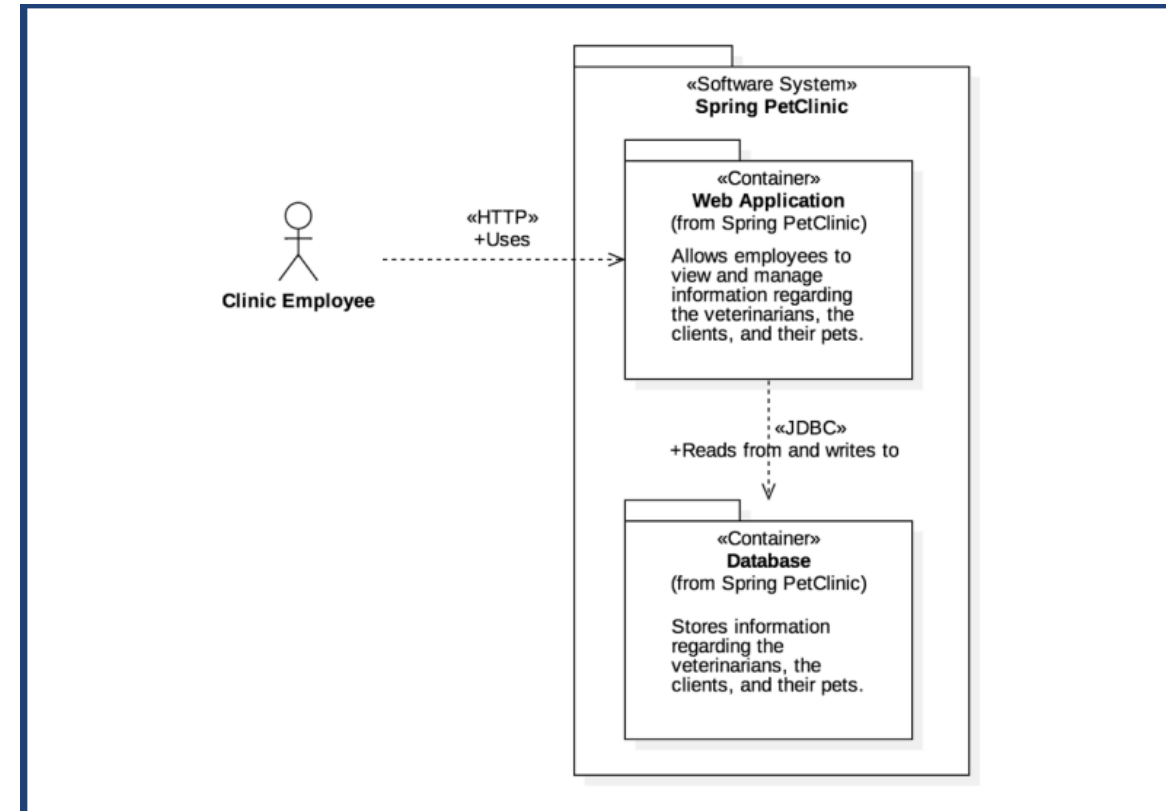
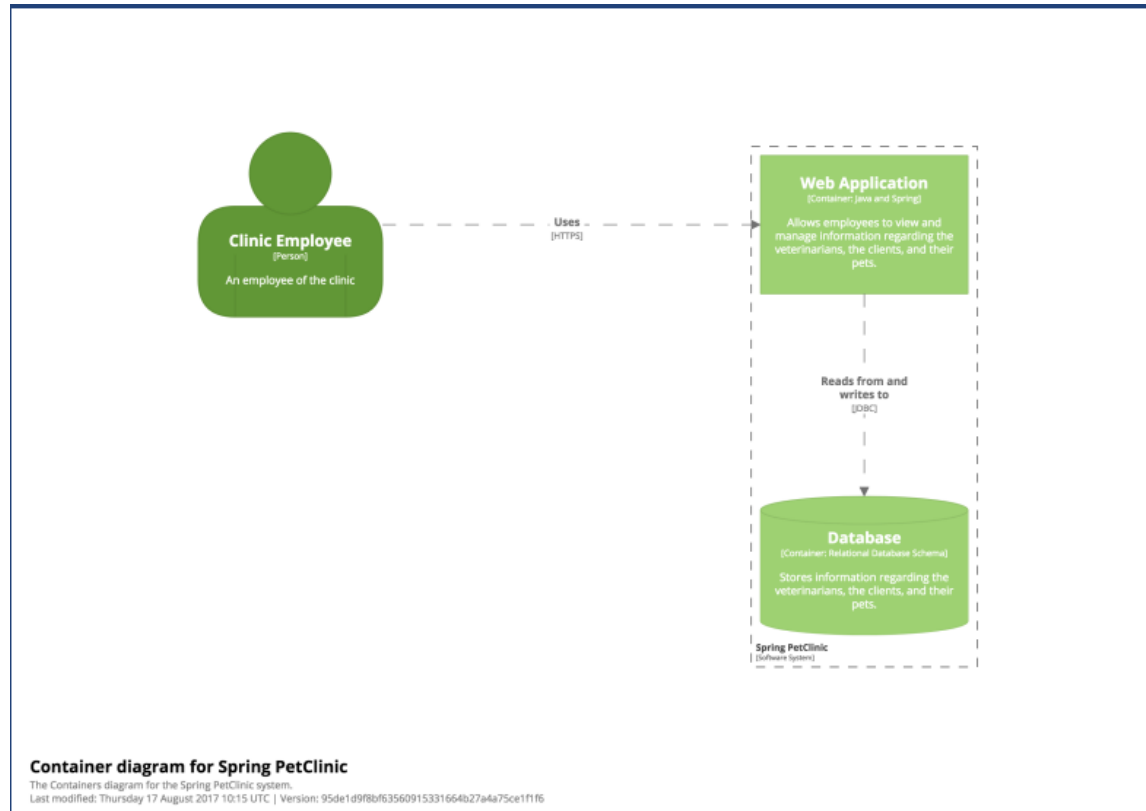
Deployment View: Internet Banking System - Development
An example development deployment scenario for the Internet Banking System

Deployment Diagram Example



C4 Model Notation

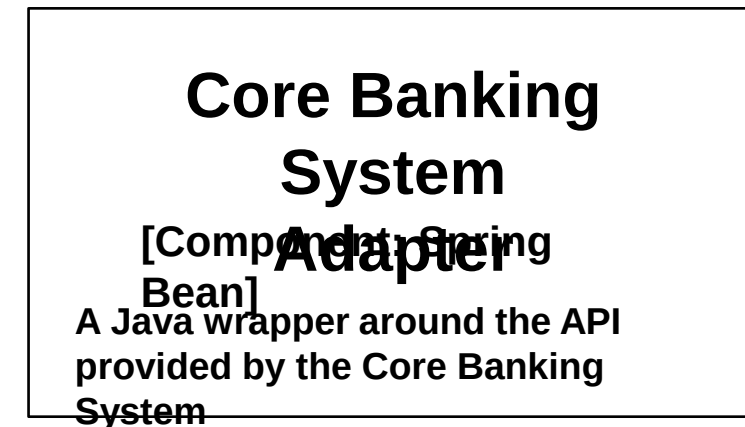
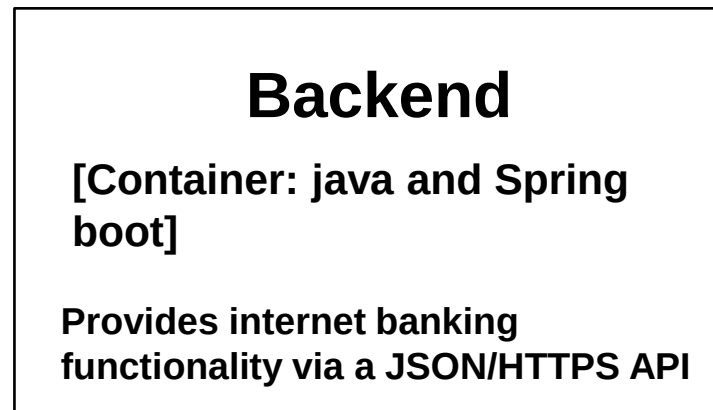
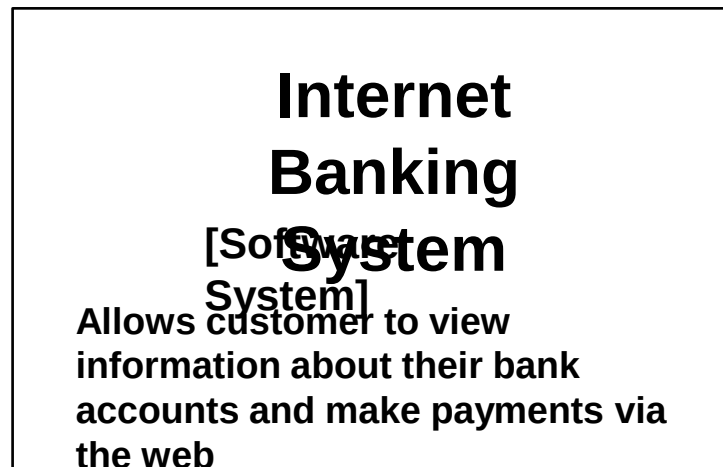
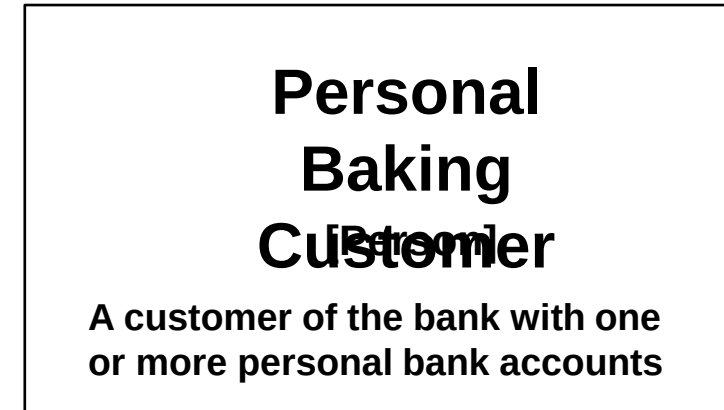
□ C4 Model은 특정한 표기법을 강제하지 않음



C4 Model Notation

□ Boxes

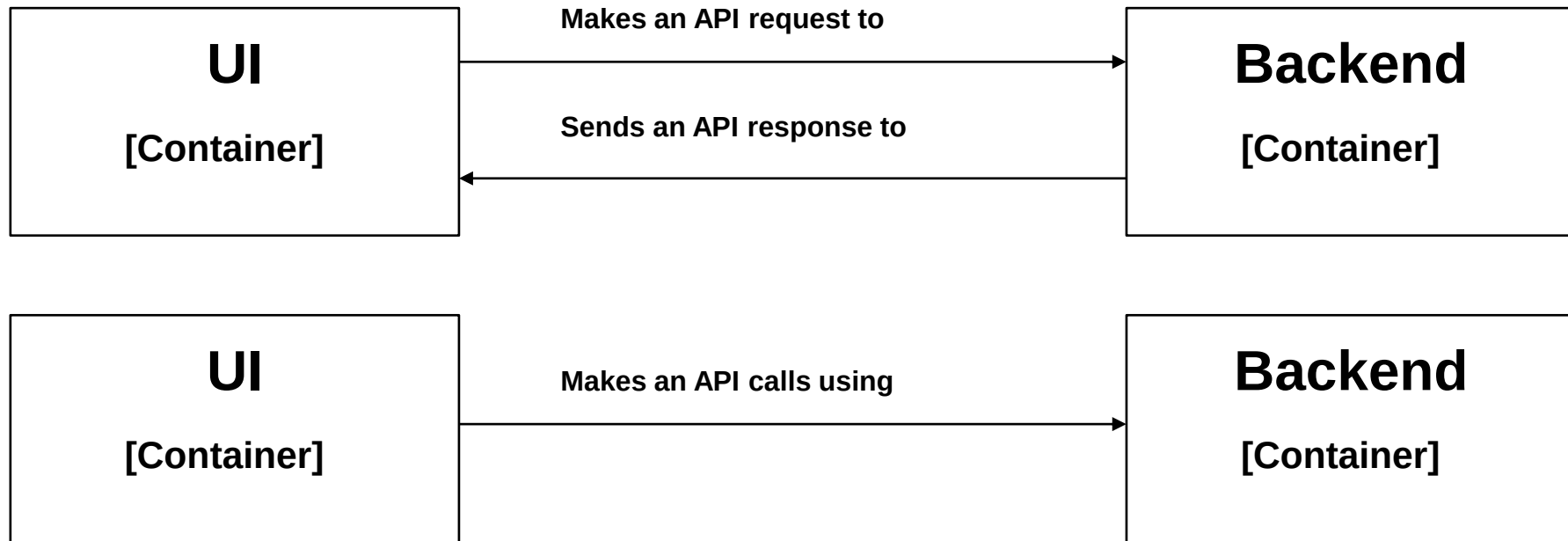
- 요소 이름, 유형, 기술(해당되는 경우) 및 설명/책임을 표현



C4 Model Notation

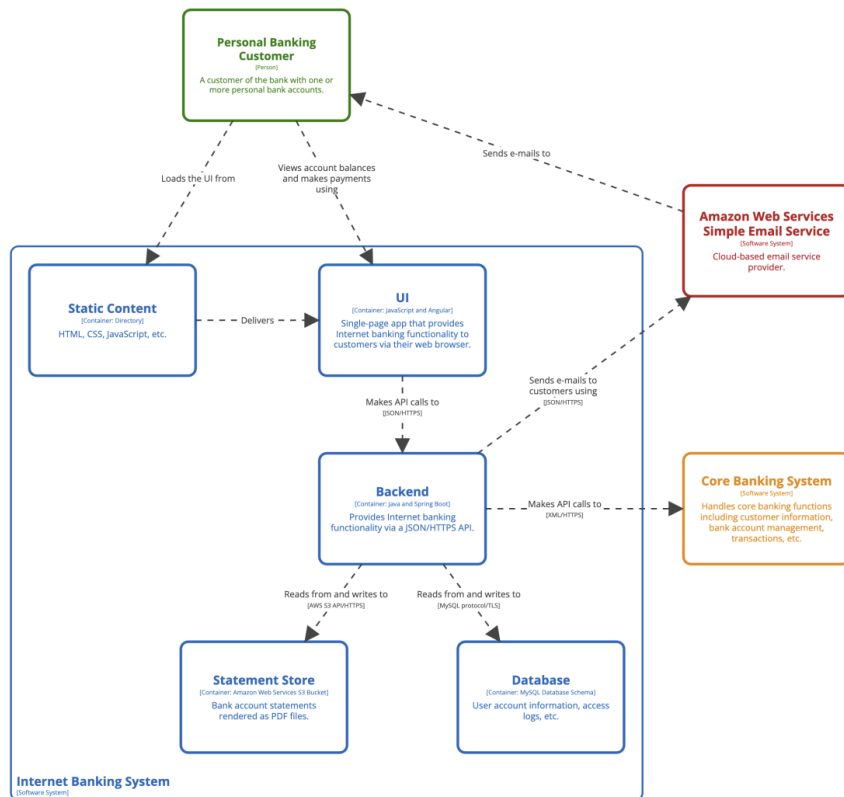
□ Lines

- 가장 중요한 종속성이나 데이터 흐름을 보여주는 단방향 선을 선호하며, 선의 목적과 방향을 명시하는 주석을 추가

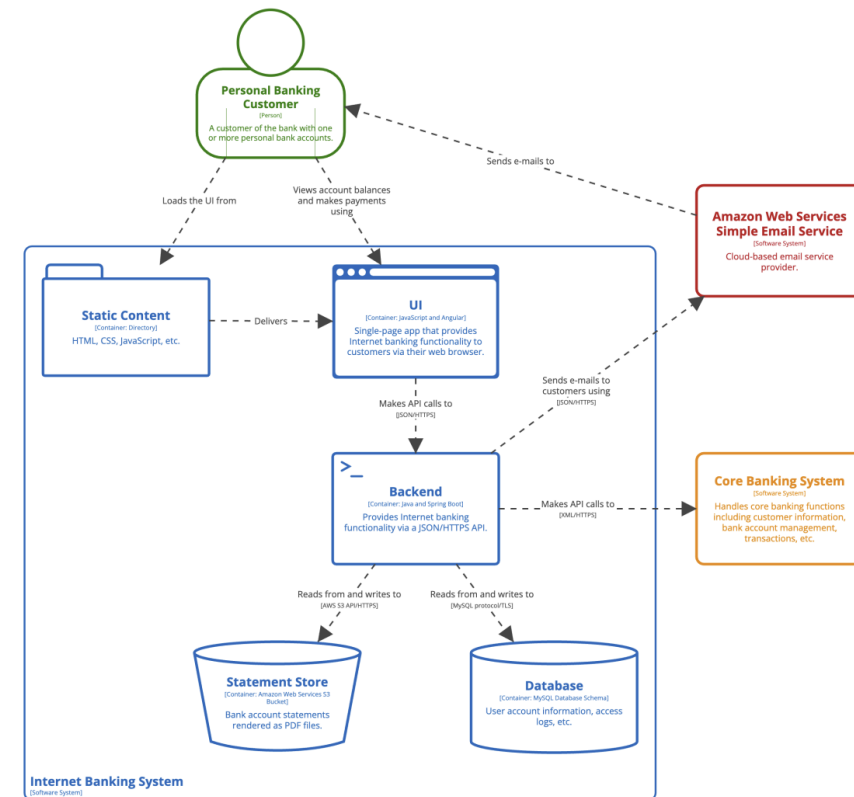


C4 Model Notation

- 모양, 색상, 크기를 사용하여 다이어그램을 보완할 수 있음



Container View: Internet Banking System
The container diagram for the Internet Banking System



Container View: Internet Banking System
The container diagram for the Internet Banking System

C4 Model vs. SEI View Model

	C4 Model	SEI View Model
주요 초점	시스템의 정적 구조(Static Structure)를 계층적으로 분해하는 것에 집중.	이해 관계자별 시스템 구조 표현 강조
View 구성	Context Diagram	C&C View, Allocation View
	Container Diagram	C&C View
	Component Diagram	Module View, C&C View
	Code Diagram	Module View
	Deployment Diagram (Optional)	Allocation View
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컨테이너 → 컴포넌트 → 코드	직교적/독립적 (Orthogonal): 세 가지 View는 서로 다른 관심사를 다루며 동시에 존재함.

C4 Model vs. Hierarchical Multi-View Model

	C4 Model	Hierarchical Multi-View Model
주요 초점	시스템의 정적 구조(Static Structure)를 계층적으로 분해하는 것에 집중.	설계 단계별로 점진적 분해 과정을 강조
View 구성	Context Diagram	
	Container Diagram	Top-Level Design – Structure View (Component Diagram)
	Component Diagram	Component-Level Design – Structure View (Class Diagram)
	Code Diagram	
	Deployment Diagram (Optional)	Deployment View (Deployment Diagram)
	Dynamic Diagram (Optional)	Behavior View (Interaction Diagram)
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컨테이너 → 컴포넌트 → 코드	관심사 기반의 기능적 분해: 기능, 동시성, 구현, 배포, 사용 사례(시나리오)

Summary (Architectural Views)

- ❑ We have examined a number of techniques for documenting software architectures
 - Documenting primary representation (i.e., C&C views, module views, allocation views)
 - Documenting interfaces and behaviors of the elements
 - Documenting architectural decisions with rationale
- ❑ The key points
 - documentation is not just boxes and lines
 - different views require different notations
 - level of detail will depend on context of use
 - producing good documentation is not a trivial enterprise

Associate Architect 양성과정

아키텍처 설계



Architectural Tactics

Topics

- ❑ **Introduction to Architectural Tactics**
- ❑ **Architectural Tactics**
 - **Availability Tactics**
 - **Deployability Tactics**
 - **Energy Efficiency Tactics**
 - **integrability Tactics**
 - **Modifiability Tactics**
 - **Performance Tactics**
 - **Safety Tactics**
 - **Security Tactics**
 - **Testability Tactics**
 - **Usability Tactics**
- ❑ **Tactics and Architectural Patterns**

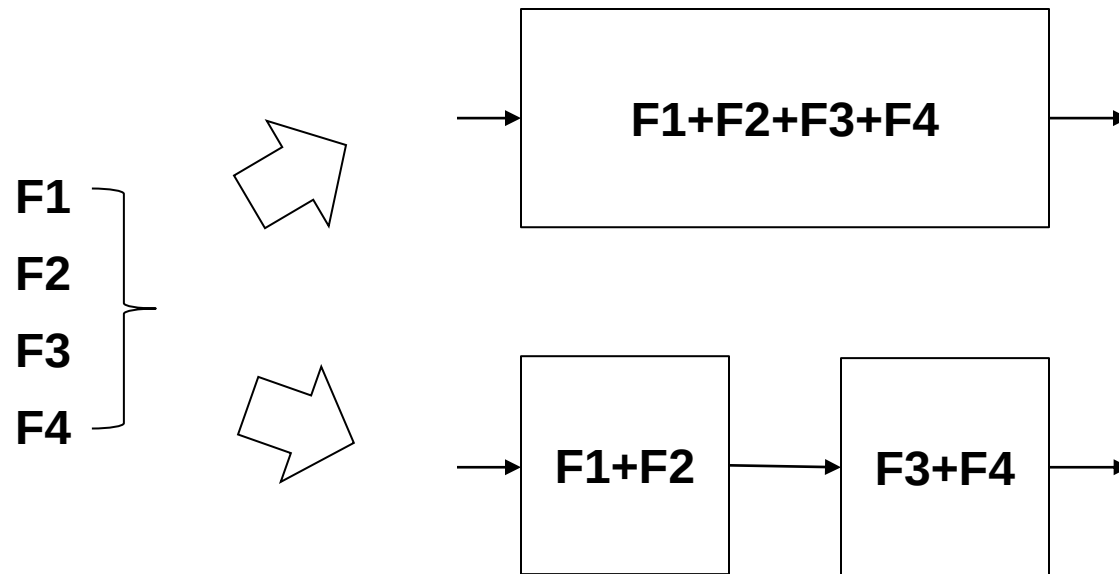
Architecture tactics

INTRODUCTION

Functionality and Architecture

□ **Functionality is largely independent of structure**

- There may be a number of possible structures that satisfy the given functionality



Quality Attributes and Architecture

□ Quality attributes influence design decisions that assign functionality to components

■ Maintainability, Understandability

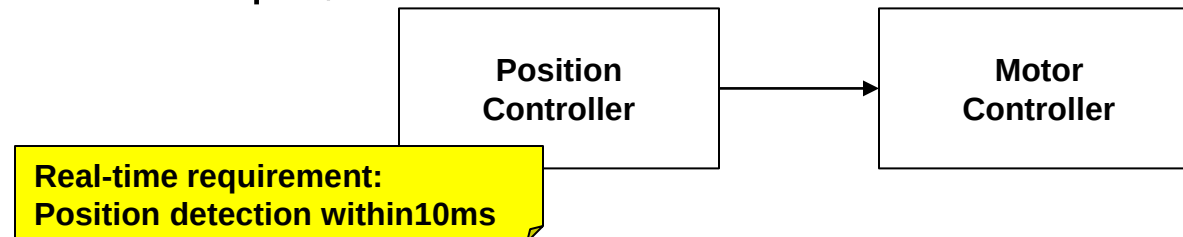
→ Allocating functionality based on sematic coherence

□ Example: Layered Architecture

■ Performance

→ Separating a time critical function

□ Example,



Quality Attributes and Architecture

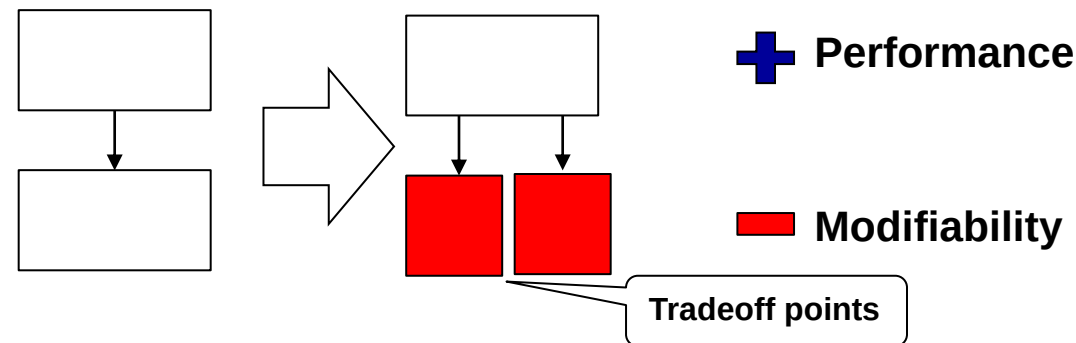
- ❑ **Quality attributes must be taken into consideration from the early phase of architectural design.**
 - The achievement of various quality attributes is dependent upon early architecture design decisions.
 - In general, you can't implement the functionality, and then go back and add the quality attributes.

- ❑ **No quality attribute is entirely dependent on architectural design, nor is it entirely dependent on implementation**
 - A performance requirement can be achieved through
 - ❑ the allocation of functionality (architectural)
 - ❑ the choice of algorithms (non-architectural)

Quality Attributes and Architecture

□ Architectural choices promote some quality attributes and inhibit others.

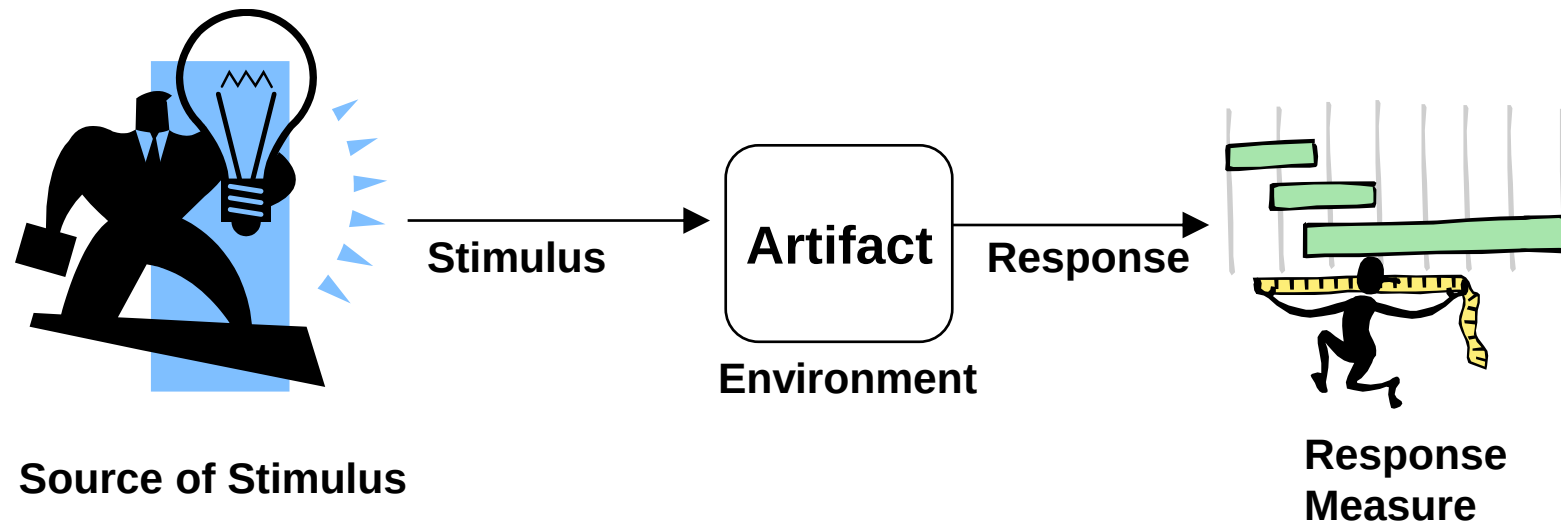
- A change in structure that improves one quality attribute may promote or inhibit the achievement of others.



- Architecture is critical to balance *quality attribute tradeoffs* before detailed design and implementation

How to Specify Quality Requirements?

- ❑ It is meaningless to say that “a system should be a modifiable”
- ❑ Quality attribute scenario is an effective way of identifying and specifying quality-attribute-specific requirements.



Quality Attribute Scenario

Source of Stimulus	The entity that generated the stimulus
Stimulus	Stimulus varies depending on the QA - Performance: service request - Modifiability: change request - Availability: faulty situation - Security: attach
Artifact	The system or part of the system at which the stimulus arrives
Environment	the set of circumstances in which the scenario takes place
Response	the activity that occurs as the result of the arrival of the stimulus
Response Measure	When the response occurs, it should be measurable in some fashion so that the scenario can be tested

General and Concrete Scenarios for Quality Requirements

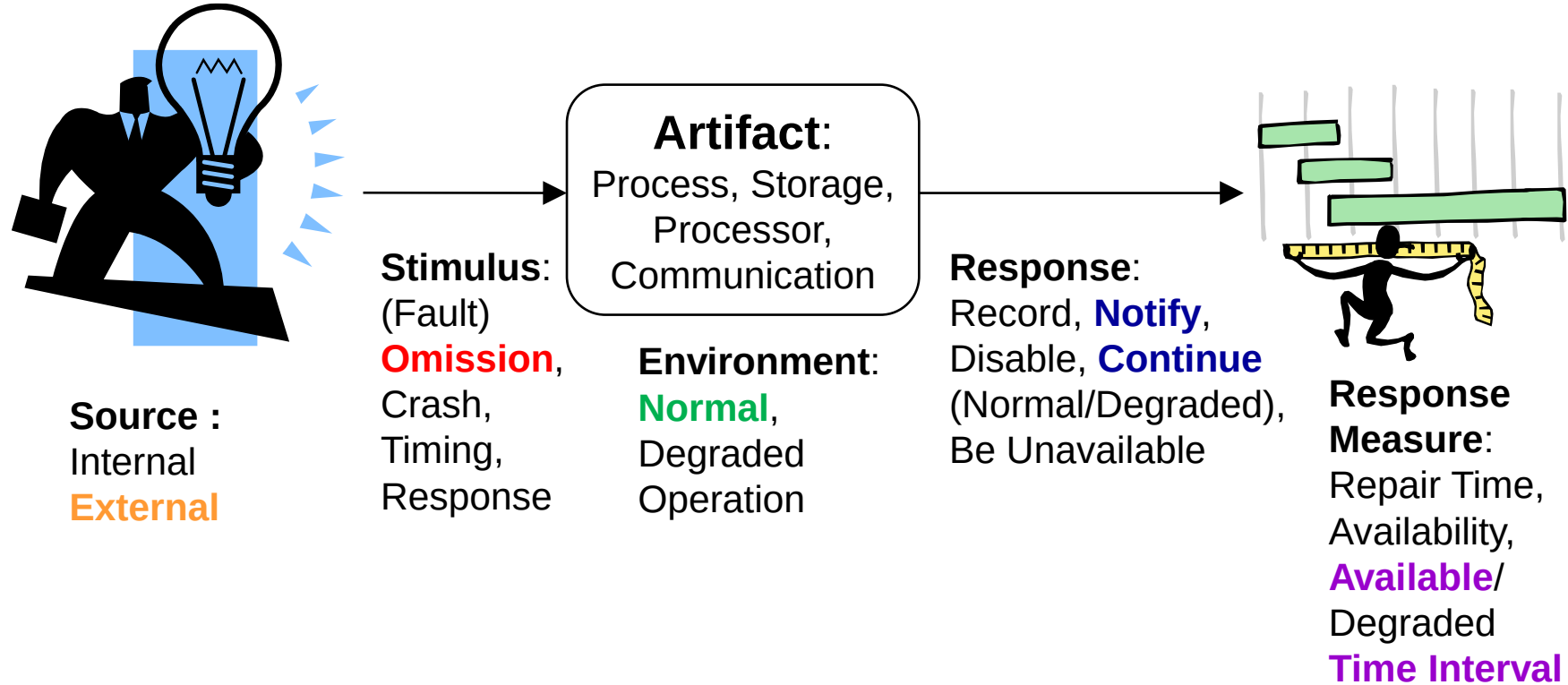
□ General quality attribute scenarios

- System independent.
- Can potentially pertain to any system.

□ Concrete quality attribute scenarios

- Specific to the particular system under considerations.
- Instantiated from the attribute characterizations of general scenarios.

General Scenarios vs. Concrete Scenarios



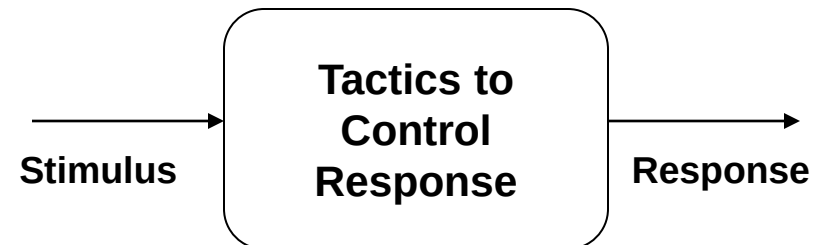
“An **unanticipated external message** is received by a process during **normal operation**. The process **informs the operator of the receipt of the message** and **continues to operated with no downtime**”

Tactics

□ ***(Architectural) Tactic***

- **A design decision that influences the achievement of a quality attribute response.**

- The focus of a tactic is on a single quality attribute response without consideration of tradeoffs

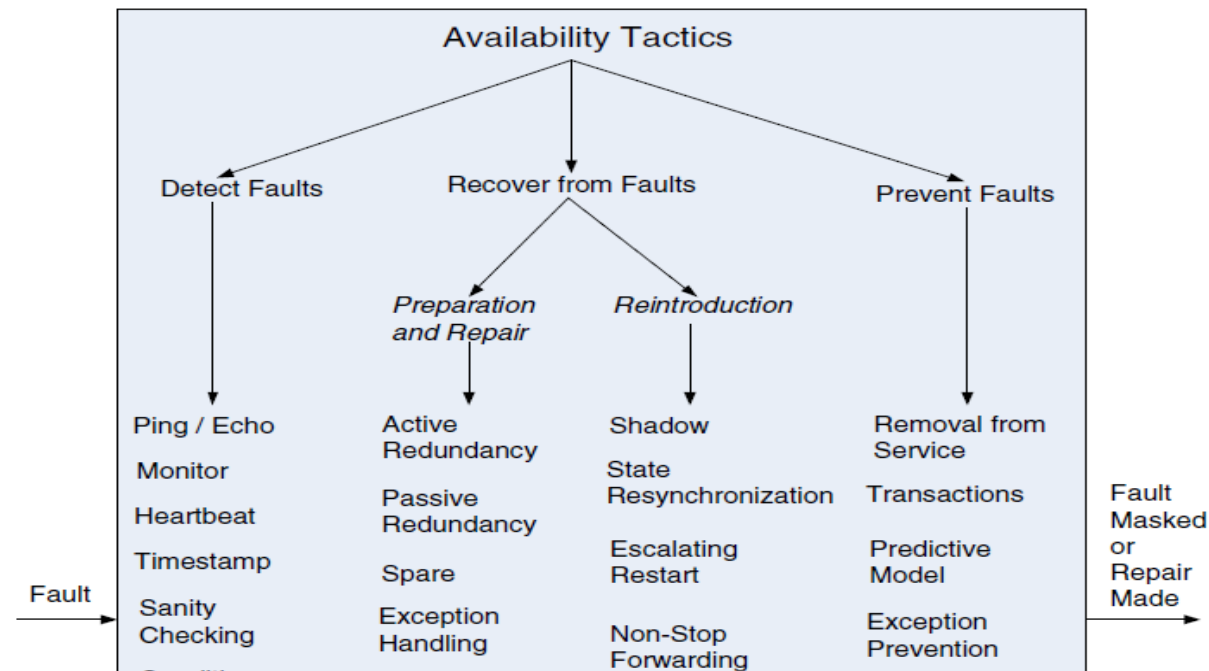


- **Each tactic is a decision option for the architect.**

- Tactics are those that architects have been doing in practice for years.

Tactics

- Tactics can refine other tactics.
 - For each quality attribute, tactics can be organized as a hierarchy according to their specialization or decomposition relationships.



Quality Design Decisions

□ Categories

- Allocation of responsibilities
- Coordination model
- Data model
- Management of resources
- Mapping among architectural elements
- Binding time decisions
- Choice of technology

□ **These categories help architects identifying every important decision during architectural design.**

Architectural tactics

AVAILABILITY TACTICS

Availability

- **Availability** refers to a property of software that it is ready to carry out its task when you need it to be.
 - Availability = reliability + the notion of recovery
- **Availability** refers to **the ability of a system to mask or repair faults** such that the cumulative service outage period does not exceed a required value over a specified time interval.
- Fundamentally, availability is about **minimizing service outage time by mitigating faults**

Availability vs. Reliability

- **Availability** is a measure of the percentage of time the equipment is in an operable state

Uptime / Total time (Uptime + Downtime)

- **reliability** is a measure of how long the item performs its intended function.

Mean Time Between Failure (MTBF)

= total time in service / number of failures

- Generally, a reliable machine has high availability, but an available machine may or may not be very reliable

ISO/IEC 25010

□ Reliability

- Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time.
- Maturity
 - Degree to which a system, product or component meets needs for reliability under normal operation.
- Availability
 - Degree to which a system, product or component is operational and accessible when required for use.
- Fault tolerance
 - Degree to which a system, product or component operates as intended despite the presence of hardware or software faults.
- Recoverability
 - Degree to which, in the event of an interruption or a failure, a product or system can recover the data directly affected and re-establish the desired state of the system.

Availability General Scenario

❑ **Source of Stimulus**

- **Internal or External:** people, hardware, software, physical infrastructure, physical environment

❑ **Stimulus:** a fault one of the following classes

- **Omission:** a component fails to response to an input
- **Crash:** the component repeatedly suffers omission faults
- **Timing:** a component responds but the response is early or late
- **Incorrect Response:** a component responds with an incorrect value

Availability General Scenario

❑ Artifact

- Processor, communication channel, process, or storage

❑ Environment

- The state of the system when the fault occurs
 - ❑ Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation

❑ Response

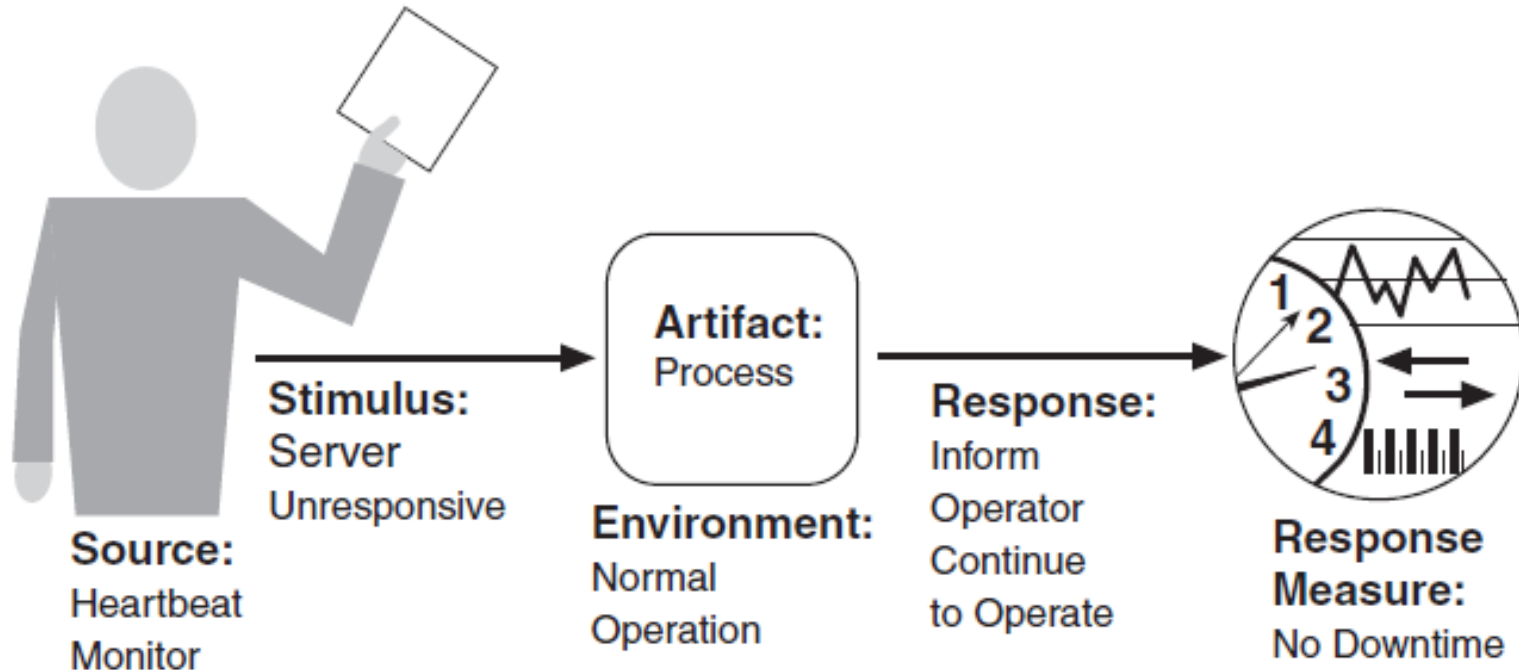
- Logging the failure,
- notifying selected users or other systems,
- switching to a degraded mode,
- shutting down external systems,
- becoming unavailable during repair.

Availability General Scenario

□ Response Measure

- An availability percentage
- A time to detect the fault
- A time to repair the fault
- Time interval in which the system must be available
- Time interval in which system can be in degraded mode

Sample Concrete Scenario



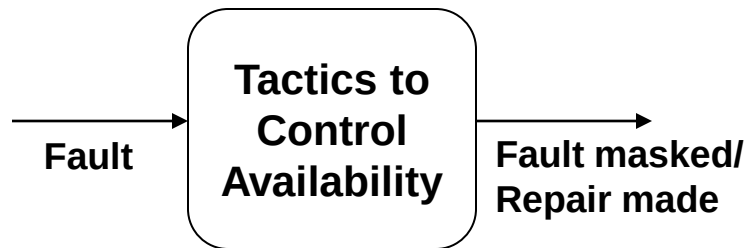
The **heartbeat monitor** determines that **the server** is **nonresponsive** during **normal operations**.

The system **informs the operator** and **continues to operate** with **no downtime**.

Availability Tactics

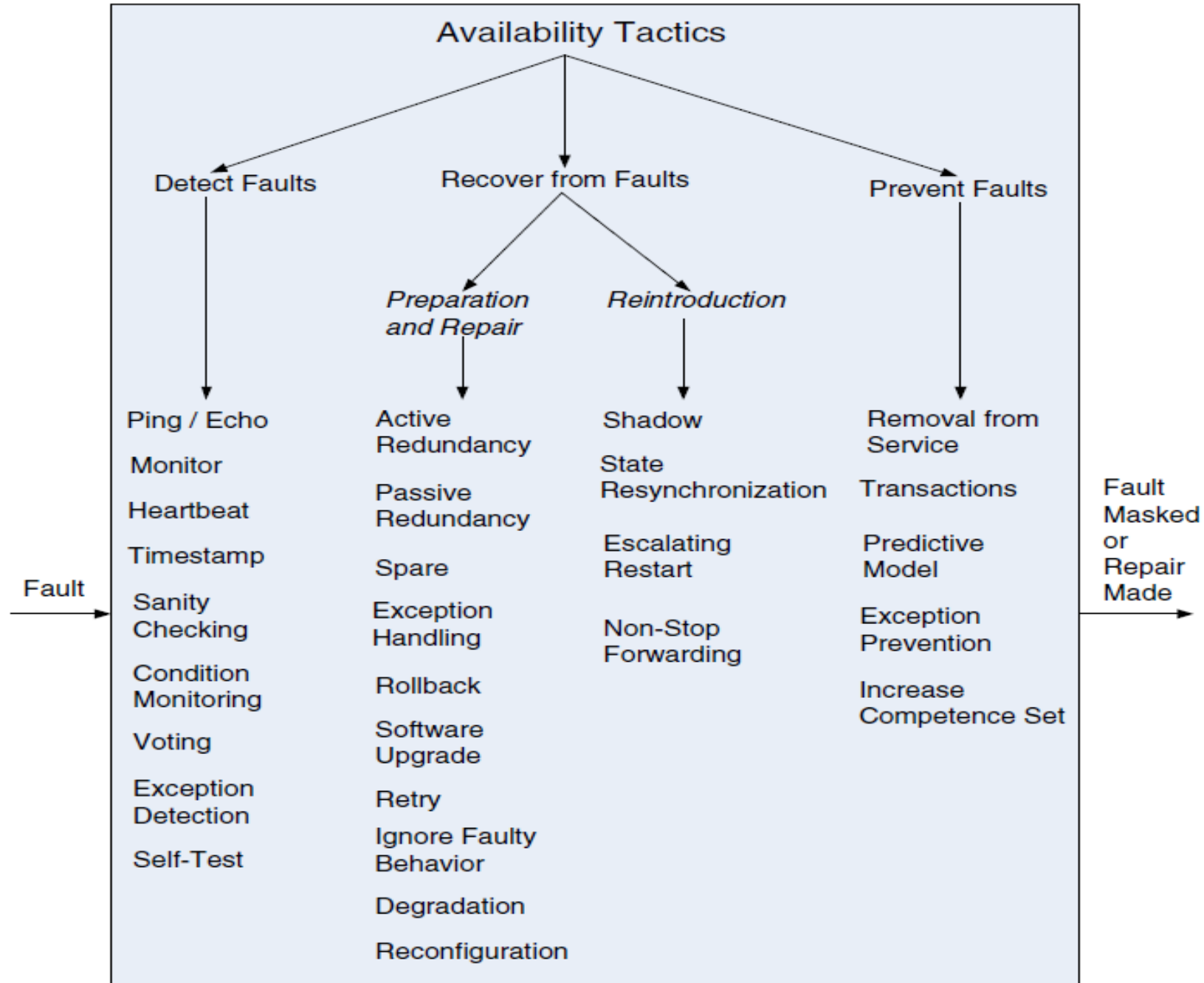
❑ Goals of Availability tactics

- keep faults from becoming failures or
- at least bound the effects of the fault and
- Make repair possible



❑ Approaches to maintaining availability

- Some type of redundancy
- Some type of monitoring to detect a failure
- Some type of recovery when a failure is detected



Fault Detection

□ Ping/echo

- One component **issues a ping and expects to receive back an echo**, within a predefined time from the component under surveillance.
- The ping is often sent by a ***system monitor***.

□ Heartbeat

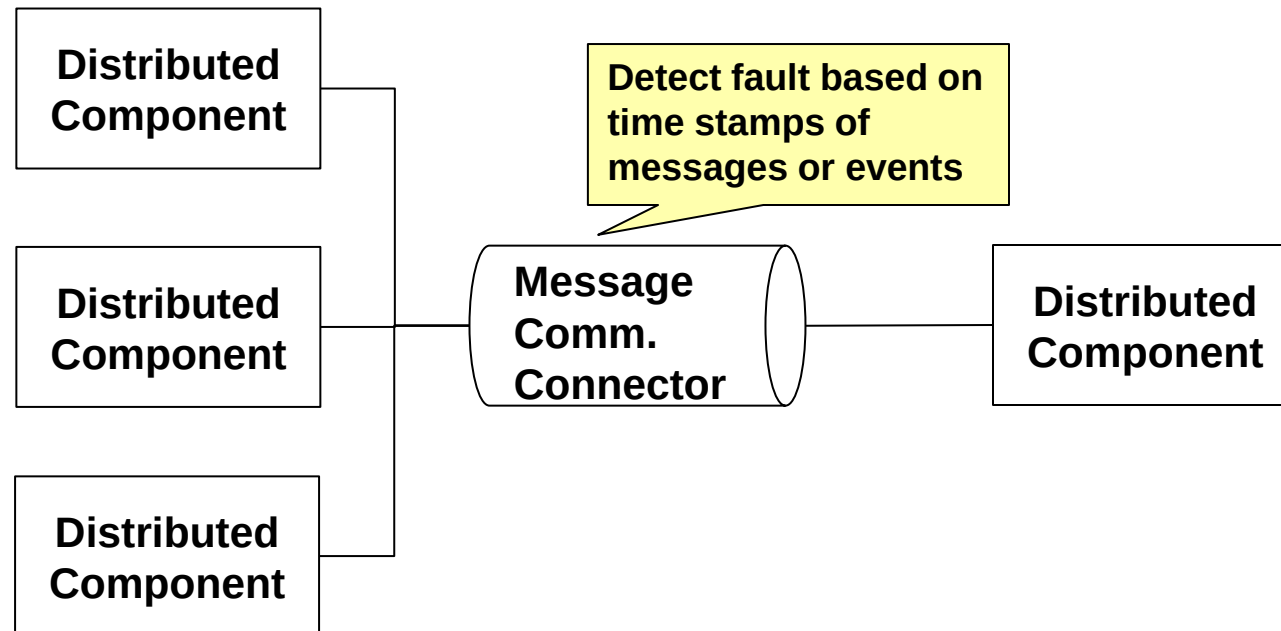
- One component **emits a heartbeat message periodically** and a ***system monitor*** listens for it.
- Heartbeat messages can be delivered by overlaying with other control messages to reduce message overhead.



Fault Detection

□ Time stamp

- A time stamp of an event can be established to detect incorrect sequences of events, primarily in distributed message-passing systems.



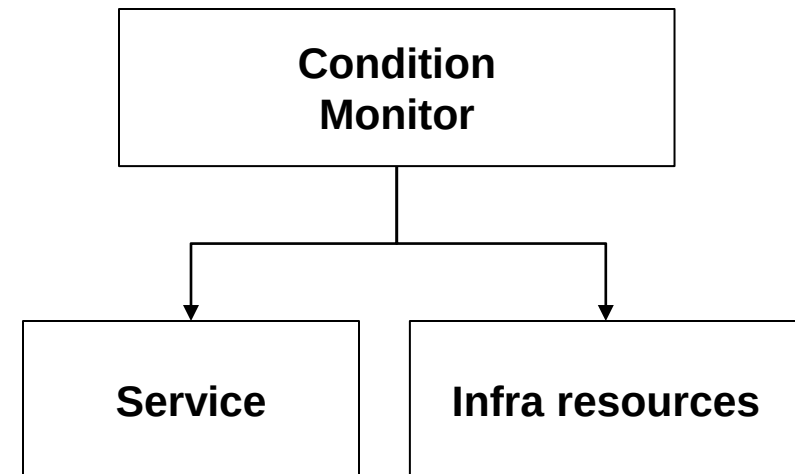
Fault Detection

□ Condition monitoring

- A method to minimize downtime by observing the status of services or infrastructure (runtime/OS, network, storage) and detecting problems before failure occurs.

- Techniques

- Real-time monitoring
- Periodic monitoring
- Remote monitoring
- Predictive maintenance



Fault Detection

□ Exception detection

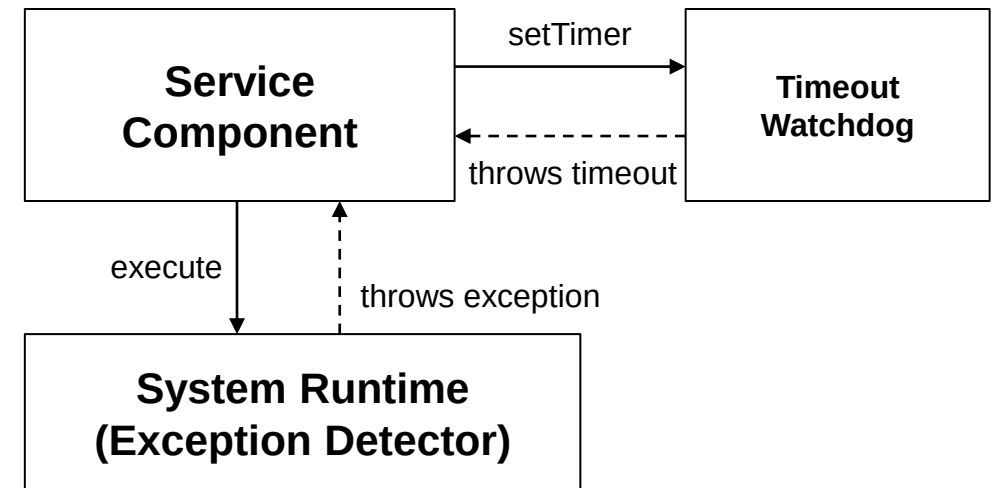
- The detection of a system condition that alters the normal flow of execution.

■ System exceptions

- E.g., divide by zero, bus and address faults, etc.

■ Timeout exceptions

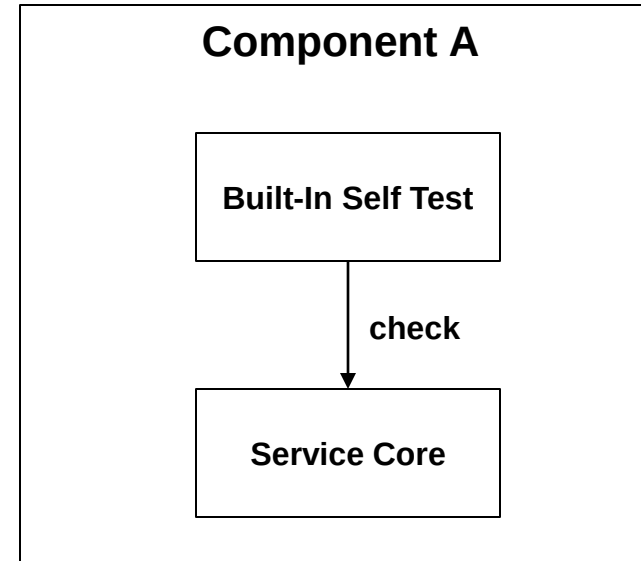
- e.g. a component waiting for a response from another component may raise an exception if the waiting time exceeds a threshold.



Fault Detection

❑ Self-test

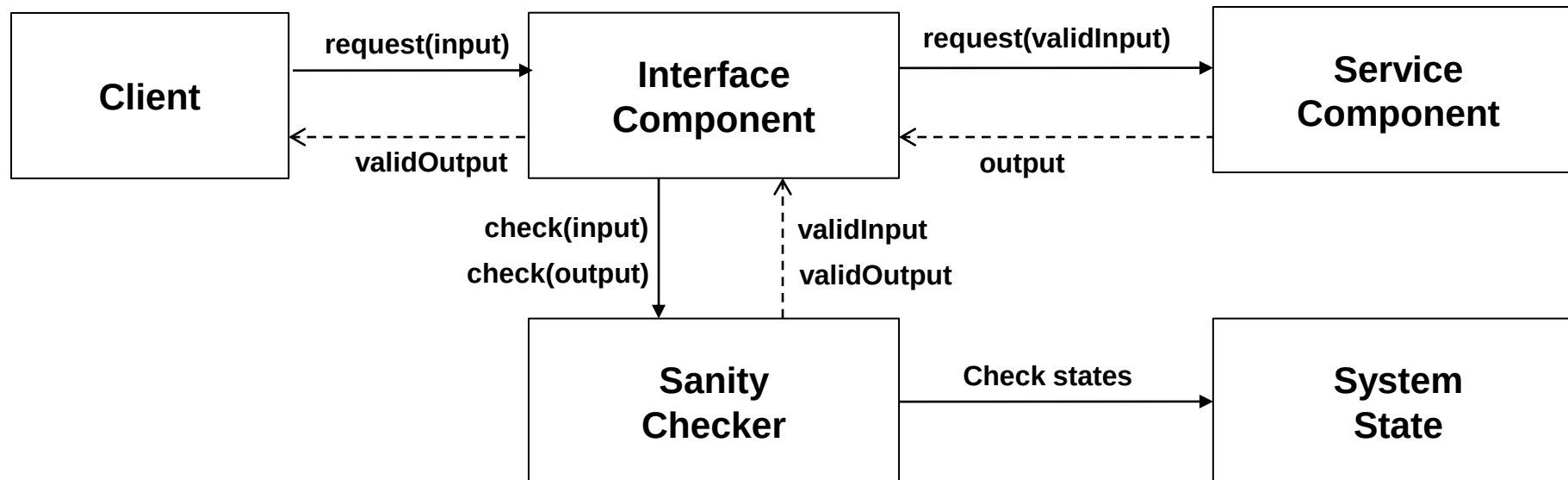
- Components (or, more likely, whole subsystems) can **run procedures to test themselves** for correct operation
 - ❑ Startup self test
 - ❑ Periodic self test
 - ❑ Continuous self test



Fault Detection

□ Sanity checking

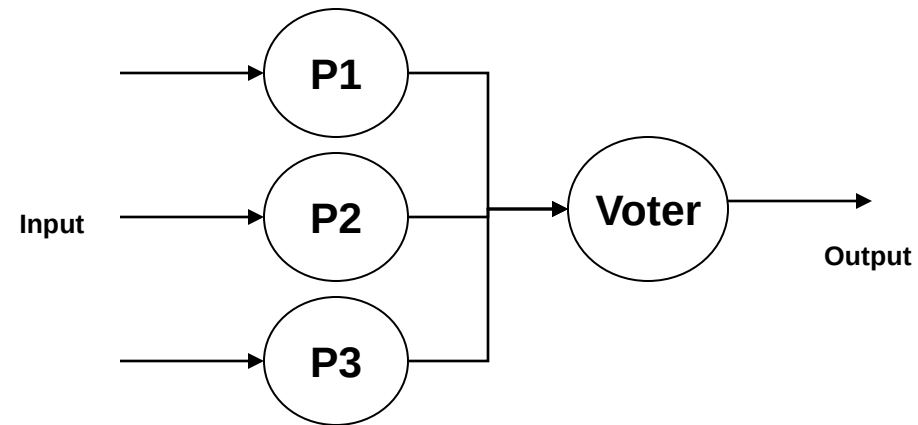
- A method to check whether a specific task or result of a component is valid, and to detect failures early by checking at runtime based on internal design knowledge, current system status, and data properties



Fault Detection

□ Voting

- **Processes running on redundant processors** each take equivalent input and compute a simple output value that is sent to a voter.
- If **voter detects different behavior from a single processor**, it discards it.
- Voting is used when the inputs to the voter are easy to classify as equivalent or deviant.



■ Diversity of Redundancy

- Same software on different processors
- Different software on different processors

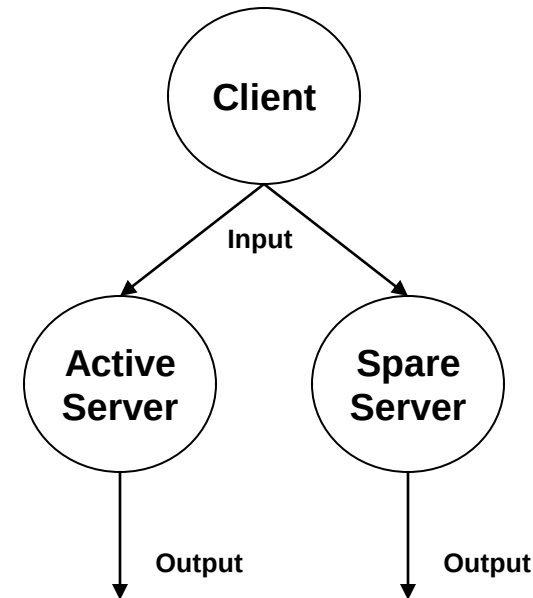
Summary (Fault Detection)

- ❑ **To detect aliveness**
 - Ping/echo
 - Heartbeat
- ❑ **To detect cases that may lead to errors**
 - Time stamp
 - Condition monitoring
 - Exception detection
 - Self-test
- ❑ **To check validity of the system**
 - Sanity checking
 - Voting

Fault Recovery: Preparation and Repair

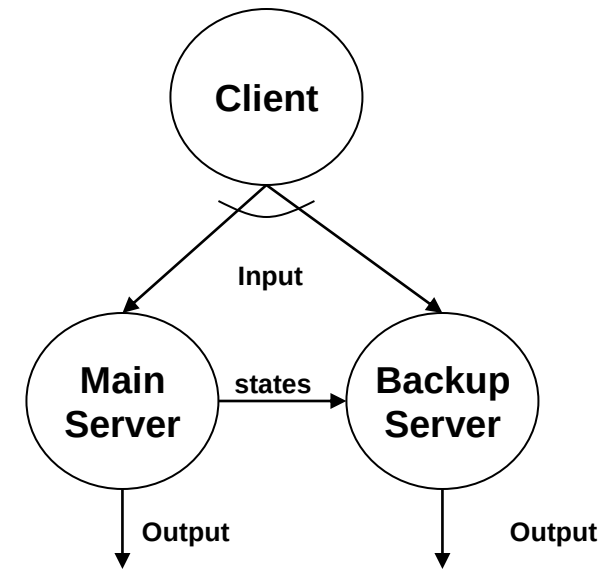
❑ Active Redundancy (hot spare)

- All redundant components (active or redundant spare) respond to inputs in parallel.
 - ❑ The output from only one component is used
- The redundant spare can take over from a failed component in a matter of milliseconds.



Fault Recovery: Preparation and Repair

- ❑ **Passive Redundancy (warm spare)**
 - **One active component responds to events** and informs the redundant spare(s) of **state updates** periodically
 - **A balanced solution** between **the hot spare tactic** (highly available and expensive) and **the cold spare tactic** (less available and significantly cheaper)



Fault Recovery: Preparation and Repair

❑ Spare (cold spare)

- Cold sparing refers to a configuration where the **redundant spares of a protection group remain out of service until a fail-over occurs.**
- **It must be rebooted** to the appropriate software configuration and **have its state initialized** when a failure occurs
- Due to its poor recovery performance, **cold sparing is better suited for systems having only high-reliability (MTBF) requirements**
 - ❑ MTBF = mean time to between failure

Fault Recovery: Preparation and Repair

□ (Preparing) **Exception handling**

- Ranging from **simple function return** to **the use of exception classes** that contain information **to mask the fault**, usually by correcting the cause of the exception and retrying the operation.

□ **Rollback**

- This tactic permits the system to **revert to a previous known good state (a checkpoint)** upon the detection of a failure.
- Checkpoints can be stored in a fixed location and updated at regular intervals, or at convenient or significant times

Fault Recovery: Preparation and Repair

❑ Software upgrade

- This tactic achieves **in-service upgrades to executable code images** in a non-service-affecting manner
- **For bug fixes**
 - ❑ **A functional patch** is used for the procedural programming environment to update the new version of the software function dynamically
 - ❑ **A class patch** is applicable for targets executing OO code to enable the runtime addition of the class members (data and functions)
- **For new features**
 - ❑ **A hitless in-service software upgrade (ISSU)** uses the active redundancy or passive redundancy tactics to achieve non-service-affecting upgrades

Fault Recovery: Preparation and Repair

❑ Retry

- **Retrying the operation to mask transient faults**
- There should be a limit on the number of retries
- This tactic is used in networks and in server farms where failures are expected and common.

❑ Ignore faulty behavior

- **Ignoring messages sent from a particular source when those messages are determined to be spurious.**
 - ❑ For example, we would like to **ignore the messages** of an external component launching a denial-of-service attack **by establishing Access Control List filters**.

Fault Recovery: Preparation and Repair

❑ Degradation

- Maintaining **the most critical system functions in the presence of component failures**, dropping less critical functions

❑ Reconfiguration

- Recovering from component failures by **reassigning responsibilities to the** (potentially restricted) **resources left functioning**, while maintaining as much functionality as possible

Summary

(Recovery: Preparation and Repair)

- ❑ **Preparation-and-repair tactics** are based on **introducing redundancy, retrying a computation, or preparing for recovery**
- ❑ **Introducing redundancy**
 - Active redundancy, passive redundancy, spare
- ❑ **Retrying a computation**
 - Rollback, Retry, Reconfiguration
- ❑ **Preparing for recovery**
 - Exception handling
 - Software upgrade
- ❑ **Limited functioning**
 - Ignore faulty behavior
 - degradation

Fault Recovery-Reintroduction

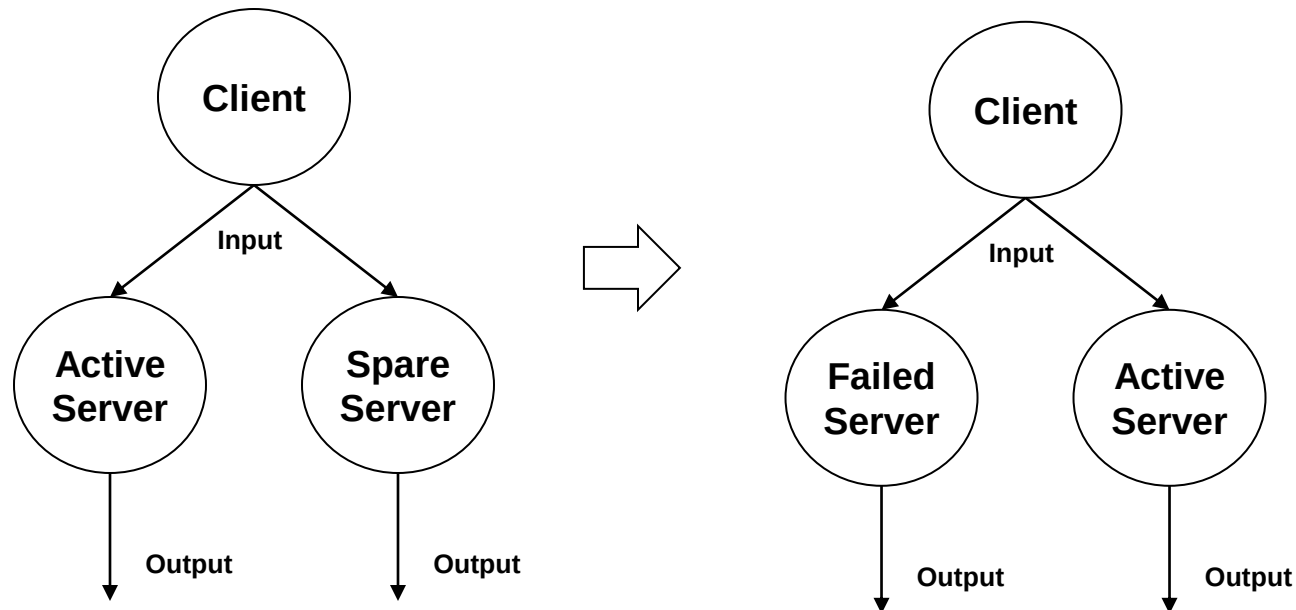
- ❑ Reintroduction is where **a failed component is reintroduced** after it has been corrected

- ❑ Shadow operation
 - **A previously failed component or in-service upgraded component** may be **run in “shadow mode”** for a short time prior to reverting the component back to an active role.
 - During this duration its behavior can be monitored for correctness.

Fault Recovery-Reintroduction

□ State Resynchronization

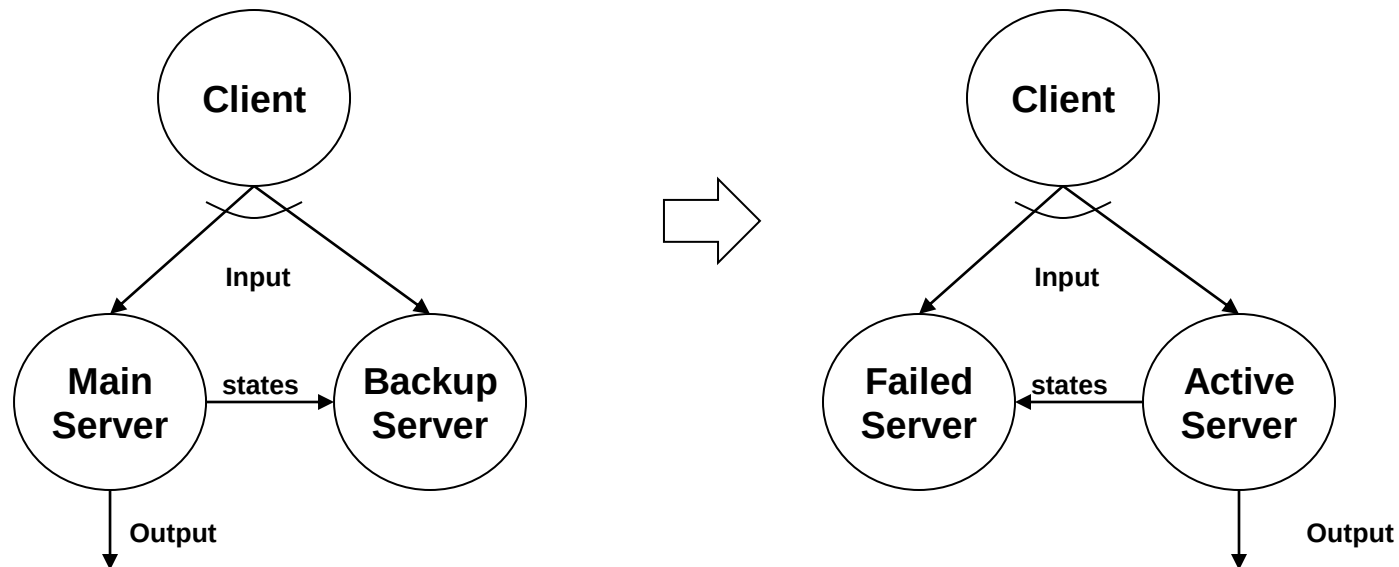
- This tactic is **a reintroduction partner** to **the Active Redundancy** and **Passive Redundancy** tactics.
- When used alongside the **Active Redundancy** tactic,
 - the **State Resynchronization** occurs organically



Fault Recovery-Reintroduction

□ State Resynchronization

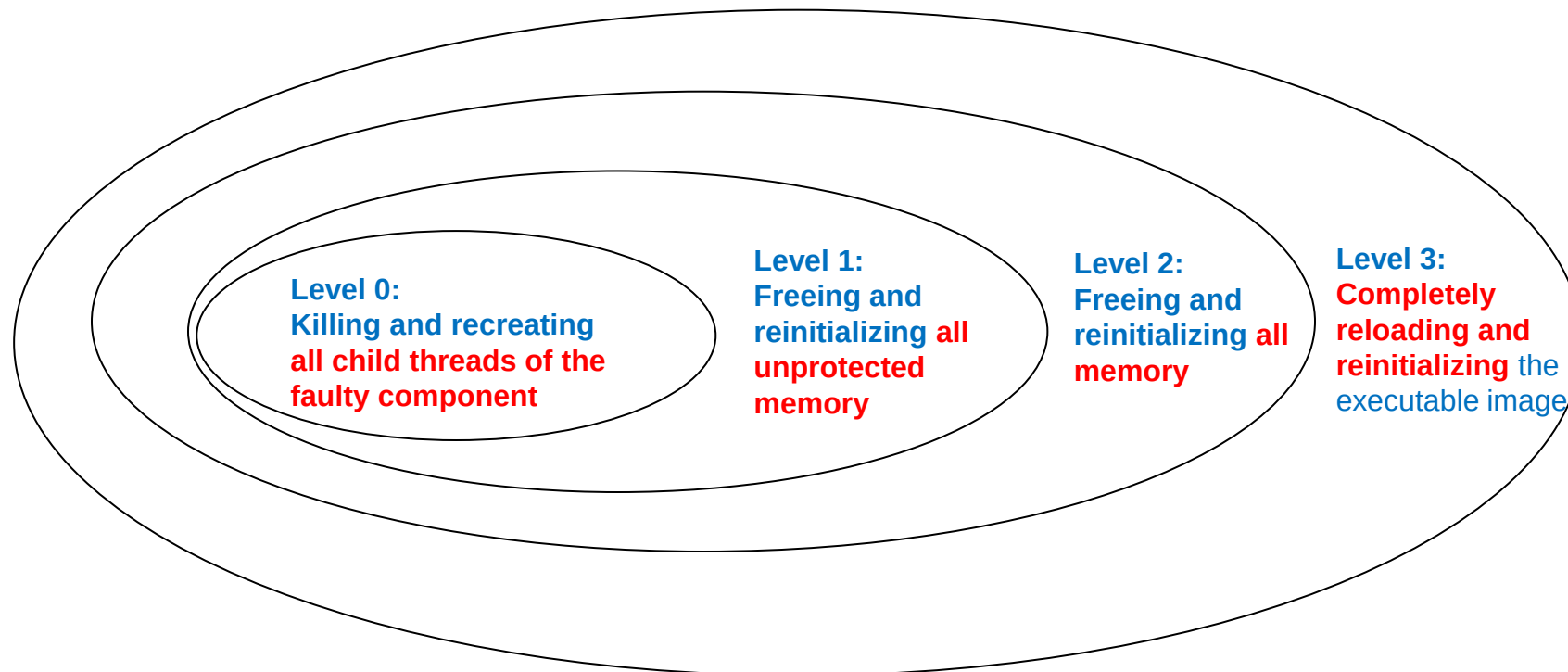
- When used alongside the **Passive Redundancy** (warm sparing) tactic,
 - **State Resynchronization** is based solely on periodic state information transmitted from the active component(s) to the standby component(s).



Fault Recovery-Reintroduction

❑ Escalating Restart

- allows the system to recover from faults by **varying the granularity of the component(s) restarted** and **minimizing the level of service affected**



Fault Prevention

❑ Removal from service

- temporarily places a system component in an out-of-service state for the purpose of mitigating potential system failures.
- E.g., taking a component out of service and resetting it to eliminate memory leaks

❑ Predictive model

- monitors the state of health of a system process and takes corrective action when conditions that could predict future faults are detected

Fault Prevention

❑ Exception prevention

- preventing system exceptions from occurring.
- E.g., Smart Pointers/Wrappers
 - ❑ an abstract data type that simulates a pointer while providing added features, such as automatic memory management or bounds checking.

Fault Prevention

□ Increase competence set

- designing a component to handle more exceptional cases as part of its normal operation.
- E.g., when a component is blocked to access to a shared resource
 - Case 1: A component might throw an exception
 - Case 2: A component might return immediately with an indication that it will complete its operation on its own the next time it does has access
 - The component in Case 2 has a larger competence set than the the component in Case 1.

Summary

(Fault Prevention)

- ❑ Fault prevention tactics dealing with runtime monitoring
 - Removal from service
 - Predictive model
- ❑ Fault prevention tactics using mechanisms eliminating faults
 - Exception Prevention: Smart Pointer
- ❑ **Increase competence set** can be classified as the **Fault Recovery (preparation and repair)** tactic

Architectural tactics

DEPLOYABILITY TACTICS

Deployment

□ 배포(Deployment)란?

- 개발된 소프트웨어를 사용 가능한 상태로 만드는 일련의 활동들을 말합니다
 - 즉, 소프트웨어를 개발 환경에서 실제 운영 환경으로 이동하고 설치하고 구성하며 활성화하는 과정

□ 배포 파이프라인

- 배포 과정을 여러 단계로 세분화하고, 가능한 한 자동화하여 흐름을 구성한 것



배포용이성 (Deployability)

- 배포용이성은 소프트웨어가 예측 가능하고 허용 가능한 시간과 비용 내에서 운영환경에 배포될 수 있는 능력을 의미
 - 운영 중단의 최소화, 배포로 인한 장애 및 롤백을 신속히 처리
- 배포용이성과 관련한 아키텍처 고려사항
 - 배포 단위
 - 전체 시스템, 일부 구성요소, 독립 배포 가능한 단위 (마이크로서비스 구조)
 - 모니터링 및 롤백
 - 아키텍처는 배포된 항목들의 운영을 모니터링하고, 성공적이지 못한 배포를 롤백 할 수 있는 기능을 제공
 - 효율성
 - 아키텍처는 합리적인 수준의 시간과 노력으로 배포 지원

배포용이성 일반 시나리오

부분	설명	가능한 값
자극원	배포 요청의 주체	최종 사용자, 개발자, 시스템 관리자, 운영 인력 컴포넌트 마켓 플레이스, 제품 소유자
자극	배포 요청	• (최종사용자) 사용자는 신규 기능 활성화를 요청 • (개발자) 코드 변경 결과를 배포 요청 • (시스템 관리자) 인프라 설정(예, 보안 패치) 변경으로 인한 운영환경 서비스 재배포 요청. • (운영 인력) 운영 인력이 일부 사용자에게만 새로운 버전 배포 요청 • (컴포넌트 마켓 플레이스) 외부 구성요소(플러그인, 라이브러리 등)의 변경으로 인한 배포 요청 • (제품 소유자) 비즈니스 요구나 릴리즈 계획에 따른 배포 결정
대상	배포 대상	• 서비스 컴포넌트(혹은 모듈) • 시스템의 플랫폼 컴포넌트 • UI 컴포넌트 • 시스템과 상호 운영되는 다른 시스템 컴포넌트

배포용이성 일반 시나리오

부분	설명	가능한 값
환경	배포 환경	스테이징 환경, 실제 운영 환경
응답	배포 요청에 따른 시스템의 반응	• 새로운 컴포넌트가 배포된다. • 새로이 배포된 컴포넌트를 모니터링 한다. • 배포된 컴포넌트가 이전 배포로 롤백 된다.
응답측정	배포에 대한 비용, 시간, 효과성 측정	• 시간 (배포시간, 서비스 중단 시간, 롤백 시간) • 노력 비용 • 배포 성공/실패율 • 다른 기능 및 품질에 영향을 미치는 정도

구체적인 배포용이성 시나리오

“인증 및 권한 부여 서비스의 **신규 릴리스가 컴포넌트 마켓플레이스에서 사용 가능해졌다**. 새로운 서비스에 대한 테스트와 실제 운영 환경으로의 배포는 40시간 이내로 걸리고 노력이 120 맨아워 이내로 들어간다. 배포로 인해 어떤 결함도 발생하지 않고, SLA도 준수해야 한다”



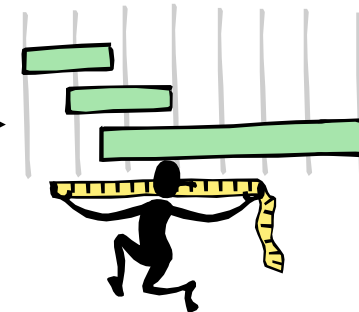
자극원 :
컴포넌트
마켓플레이스

자극:
인증 및
권한 부여
서비스의
신규
릴리스

대상:
인증/권한
부여 서비스

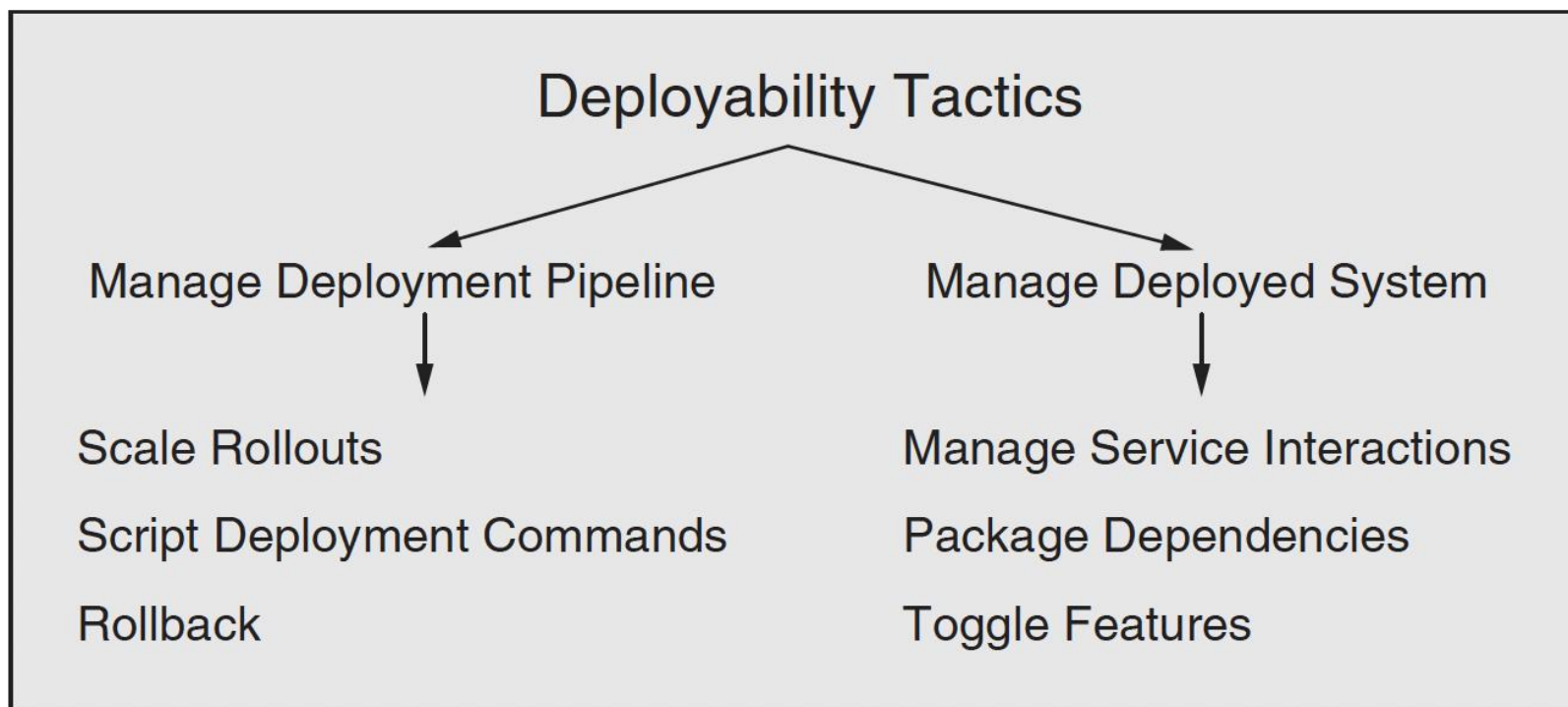
환경:
실제 운영 환경

응답:
새로운 서비스를
자체적으로
테스트하고 실제
운영 서버로
배포한다.



응답측정:
시간은 40시간 이내,
노력은 120 맨아워 이내,
결함 없이 SLA 준수

배포용이성택틱



Scale Rollouts (단계적 롤아웃)

□ 기본 개념

■ 초기 배포 단계

- 새 버전(V2) 을 소수의 인스턴스나 사용자 그룹에 먼저 배포

■ 모니터링 및 검증 단계

- 성능 저하, 오류율, 사용자 피드백 등을 관찰

■ 점진적 확대 단계

- 문제 없음이 확인되면 점진적 확장 → 배포 비율을 점차 늘림

■ 완료 및 안정화

- 전체 시스템으로 확장 완료 후 상태 점검 및 보고

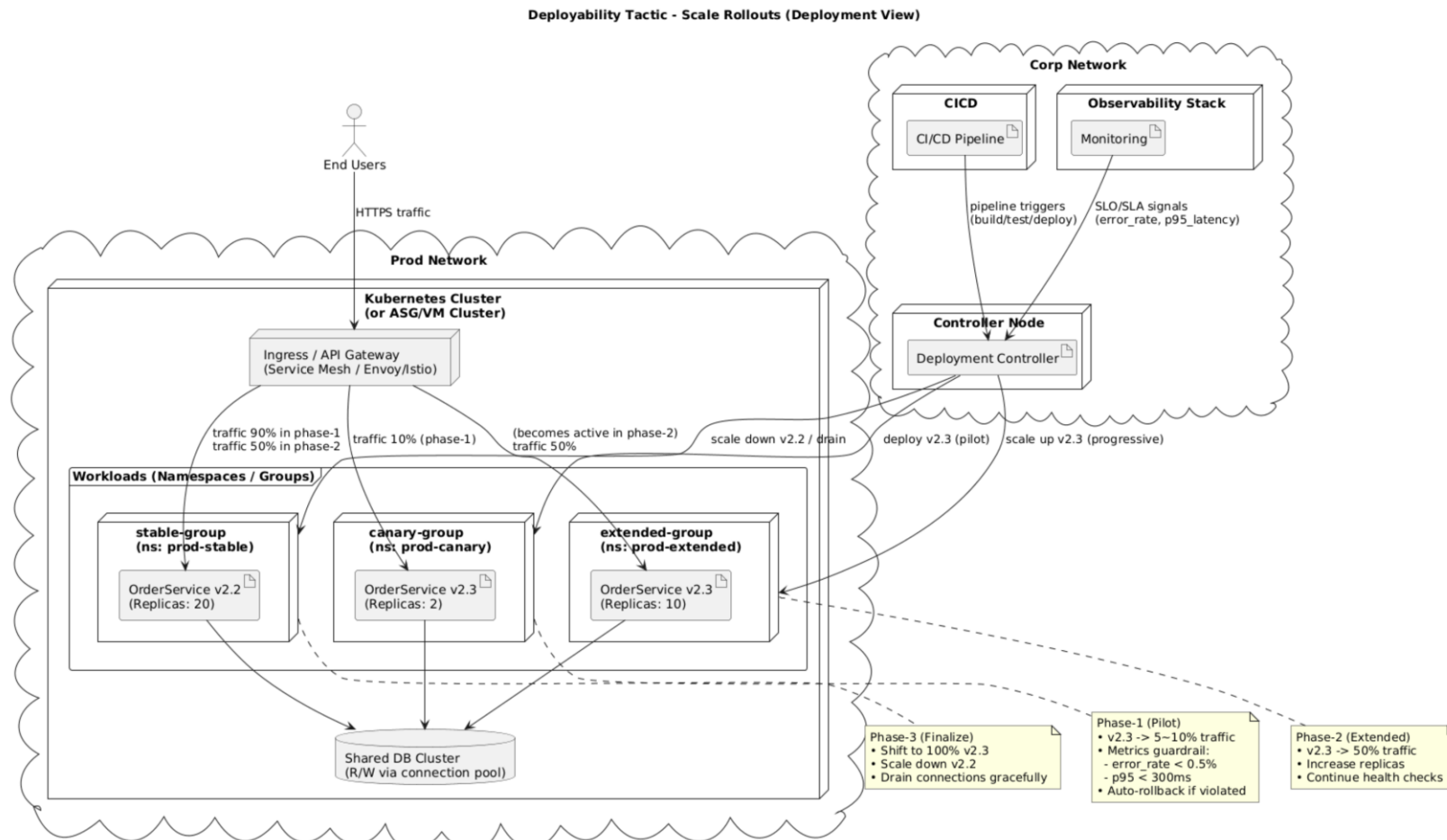
□ 관련 패턴

■ 카니리아 테스트

- **제한된 사용자(1~10%)**를 대상으로 새 버전을 신규 릴리즈를 테스트하여, 이상 유무를 조기 탐지하는 구체적 기법

Scale Rollouts (단계적 롤아웃)

□ 카나리아 적용 예시



Rollback (롤백)

□ 기본 개념

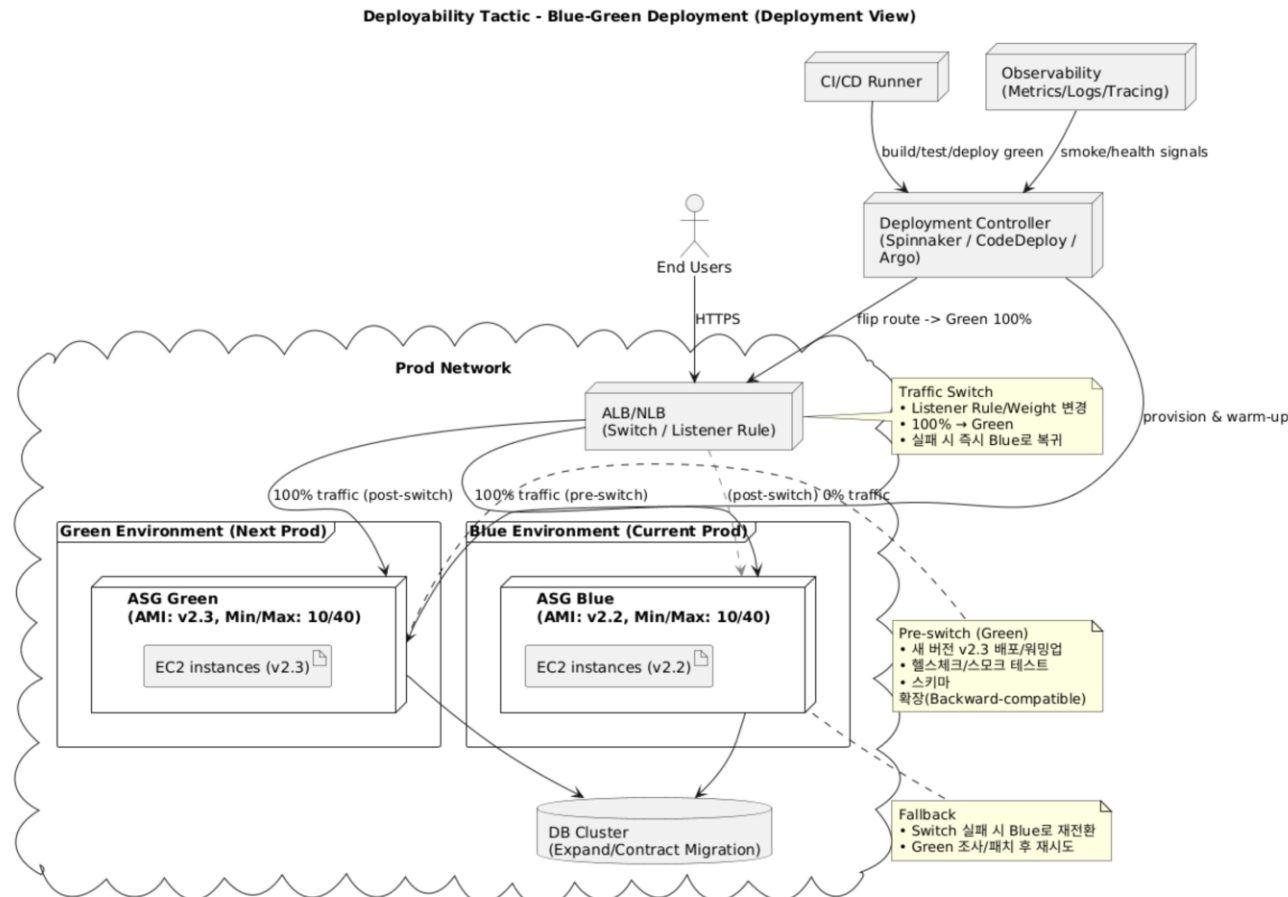
- 새로운 버전의 배포 중 또는 배포 이후 문제가 발생하였을 때, 시스템을 이전의 안정된 버전으로 신속하게 복원하는 방법
- 롤백 시, 배포에 의해 이루어진 모든 업데이트 결과를 전부 추적하여 되돌리 수 있어야 함.

□ 구현 유형

유형	설명	적용예시
Versioned Rollback	이전 애플리케이션 버전(컨테이너 이미지, 바이너리 등)을 유지하고 즉시 복원	[Kubernetes] kubectl rollout undo [AWS CodeDeploy] rollback
Blue-Green Rollback	이전 버전 환경(Blue)을 유지하고 트래픽만 전환	Blue-Green 배포 전략의 핵심
Snapshot Rollback	DB나 상태 저장소의 스냅샷을 복구	RDS Snapshot Restore EBS Snapshot
Transaction-Level Rollback	배포 중 트랜잭션 단위로 취소/보상 수행	Saga Pattern

Rollback (롤백)

□ Blue-Green Rollback 적용 예시



Rollback (롤백)

□ Saga Rollback 적용

■ Saga 패턴

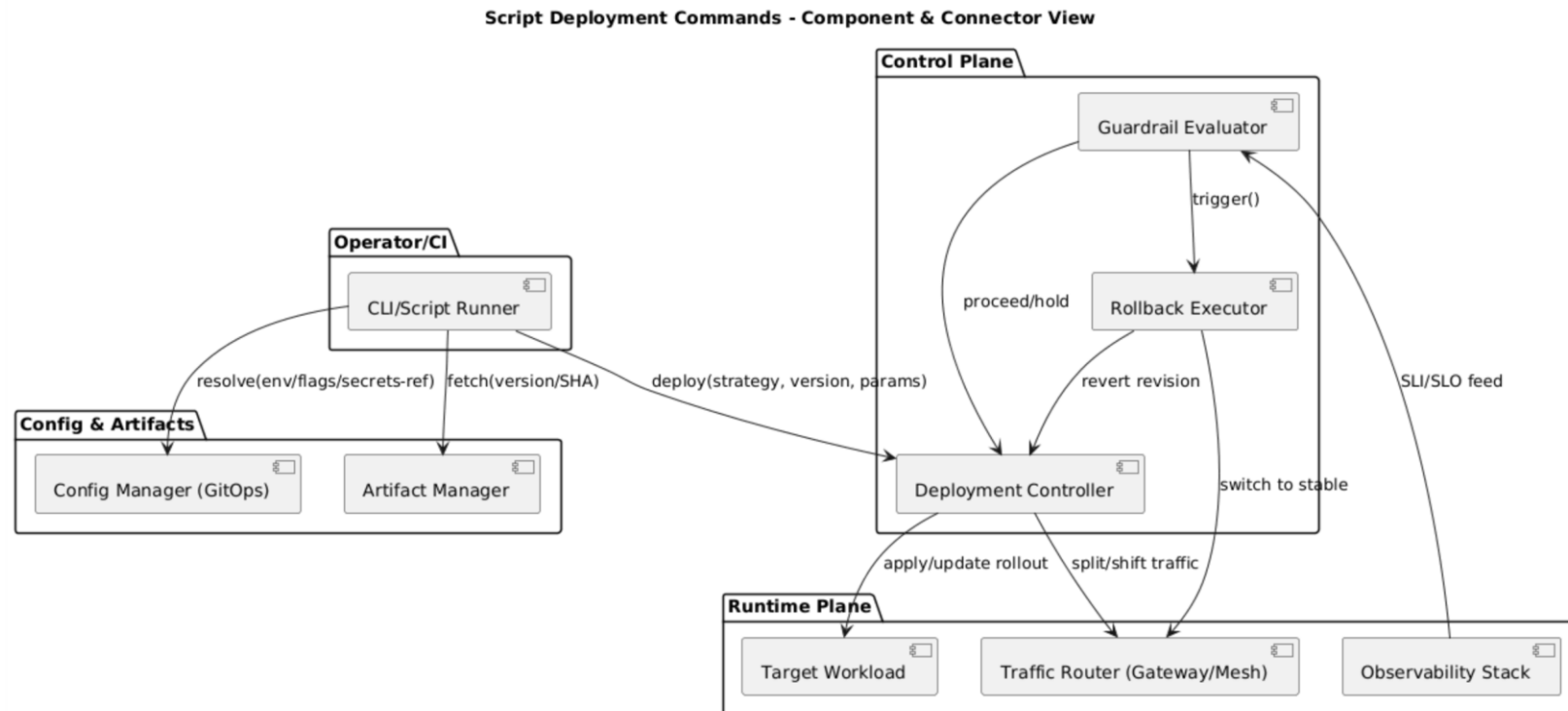
- 로컬 트랜잭션: 각 서비스의 자체 **DB**에서 원자적으로 완료.
- 보상 트랜잭션(**Compensation**): 실패 시 이미 끝난 앞 단계들을 역순으로 되돌리는 취소 작업.
- 최종적 일관성: 순간적으로 불일치가 있어도, 재시도/보상 후 일관된 상태로 수렴.

- Saga 패턴은 배포 중/직후에 이미 발생한 도메인 부작용(결제, 재고, 포인트 적립 등)을 보상 트랜잭션으로 되돌리는 비즈니스 레벨 롤백

Script Deployment Commands

□ 기본 개념

- 빌드/패키징/배포/검증/롤백까지를 **스크립트명령 세트**로 일관되게 수행하도록 만든 기술.



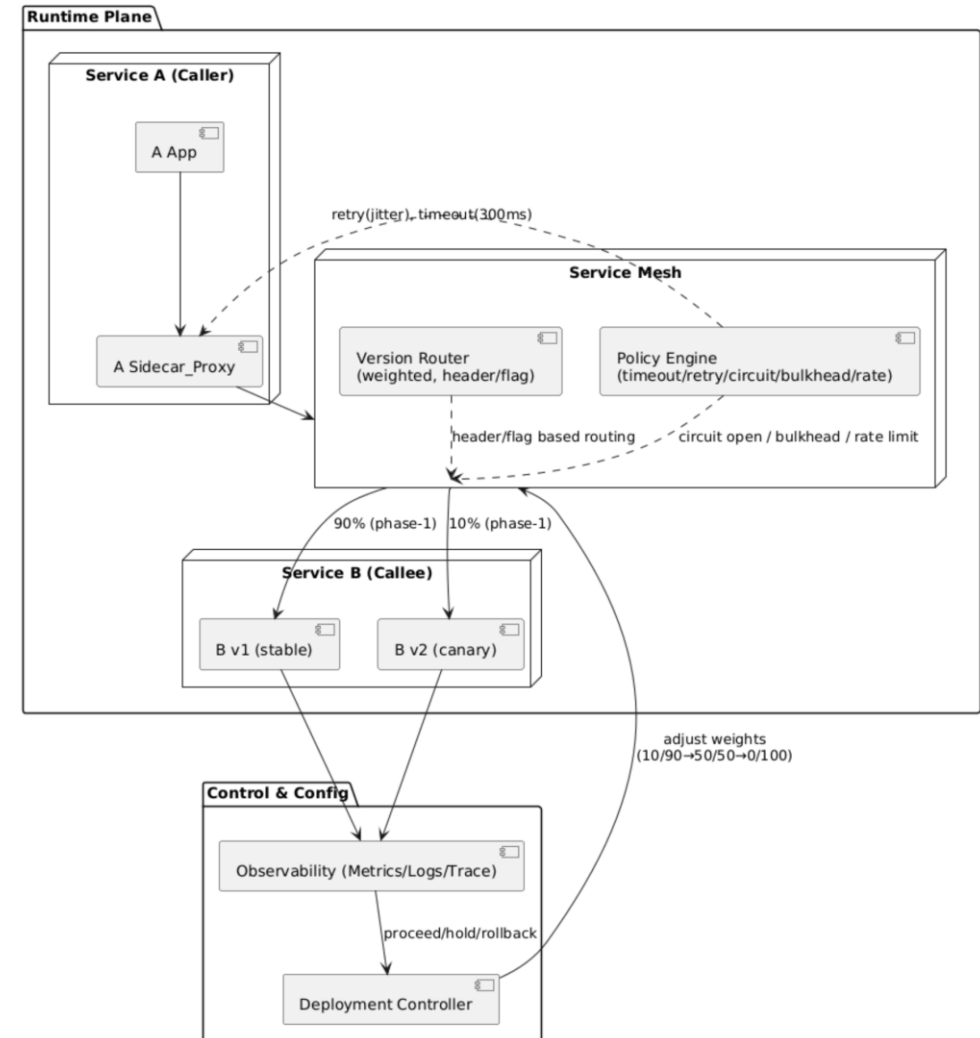
Manage Service Interactions

□ 기본 개념

- 동일한 서비스의 여러 버전이 동시에 운영 중인 상황에서 버전 호환성 문제를 선제적으로 피하기 위해서 서비스 간 상호작용을 중재하는 기술

□ 다이어그램 설명

- **Sidecar_Proxy+Service Mesh**가 버전 라우팅과 호출 안정화 정책을 집행.
- 관측 지표 기반으로 컨트롤러가 배포 비율을 자동 조정(진행/보류/롤백)
- 결과적으로 무중단·저위험 배포와 연쇄장애 방지, 구·신버전 공존이 가능해짐



Package Dependencies

□ 기본 개념

- 배포 시점에 애플리케이션이 필요로 하는 모든 종속 요소들을 함께 패키징”하여, 개발 환경에서 실제 운영 환경으로 이동할 때, 종속 요소들의 버전이 일관되게 유지될 수 있도록 하는 기술
- 종속 요소들
 - 소스 코드 라이브러리외에 빌드·런타임·플랫폼·운영까지 배포 성공에 영향을 주는 모든 의존 대상을 포함
- 종속 요소를 패키지화하는 방법
 - 가상머신
 - 컨테이너

Package Dependencies

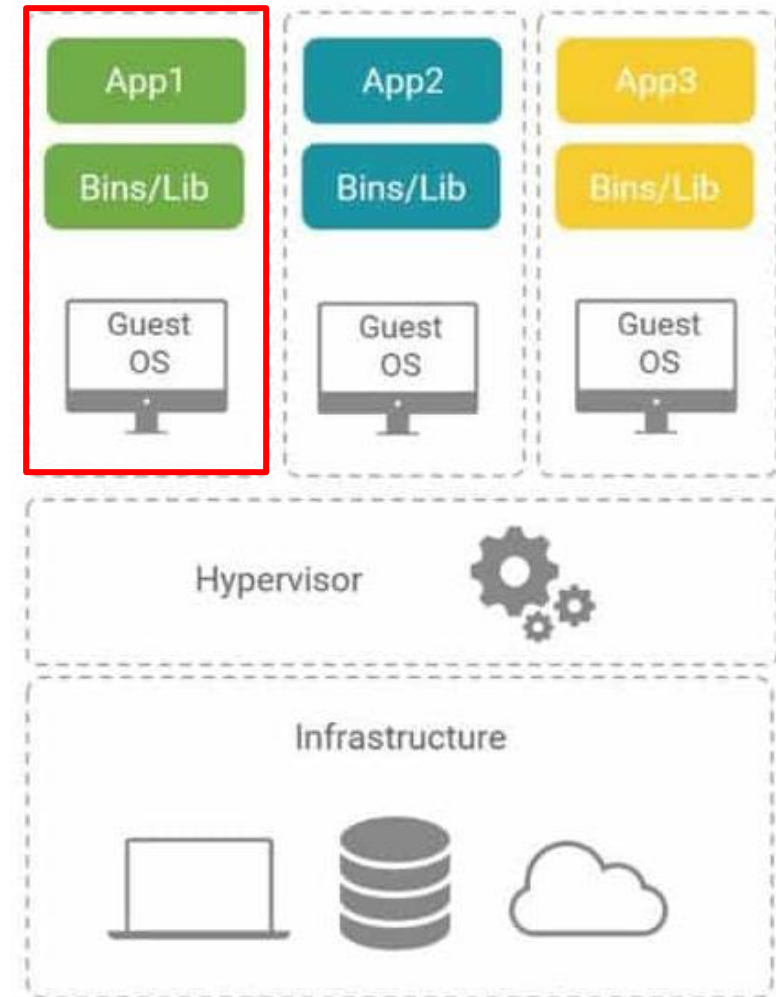
□ 가상머신 기반 패키징

■ 개념

- Guest OS(커널 포함) + 시스템 패키지 + 앱을 통째로 이미지로 패키징

■ 격리 경계

- 하드웨어 수준(하이퍼바이저)
- 의존성 충돌 위험 최소화(호스트와 완전 분리)



Package Dependencies

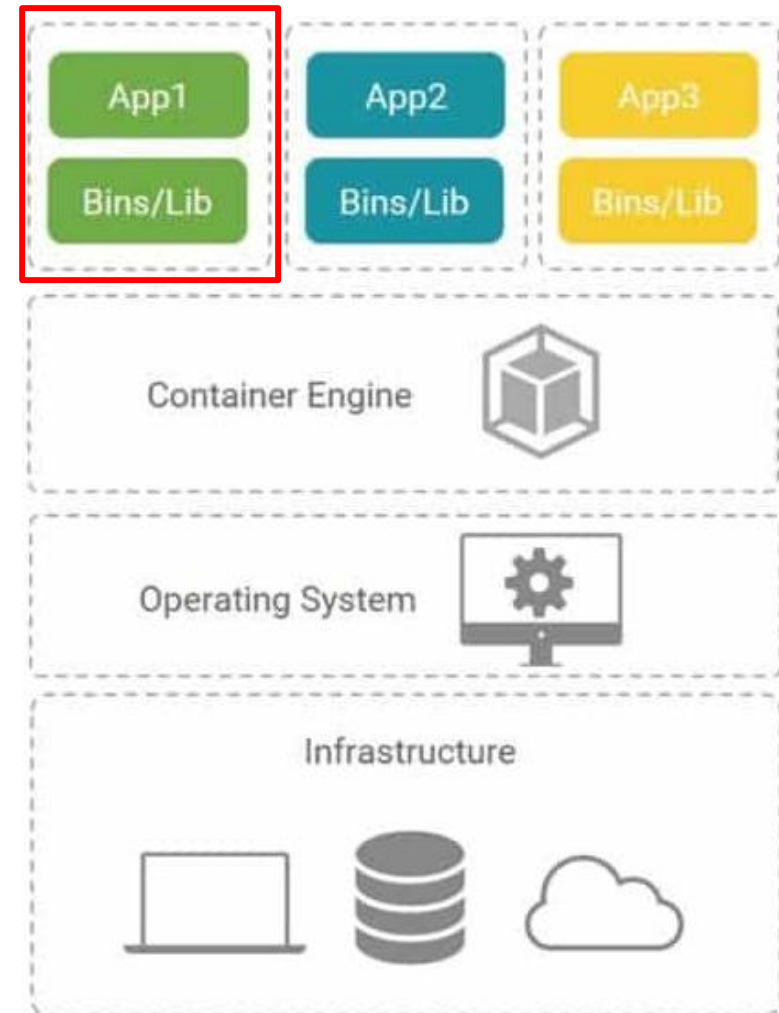
□ 컨테이너 기반 패키징

■ 개념

- 애플리케이션 실행에 필요한 언어 런타임 + 시스템 라이브러리 + 앱을 이미지로 패키징
- OS 커널은 호스트와 공유

■ 격리 경계

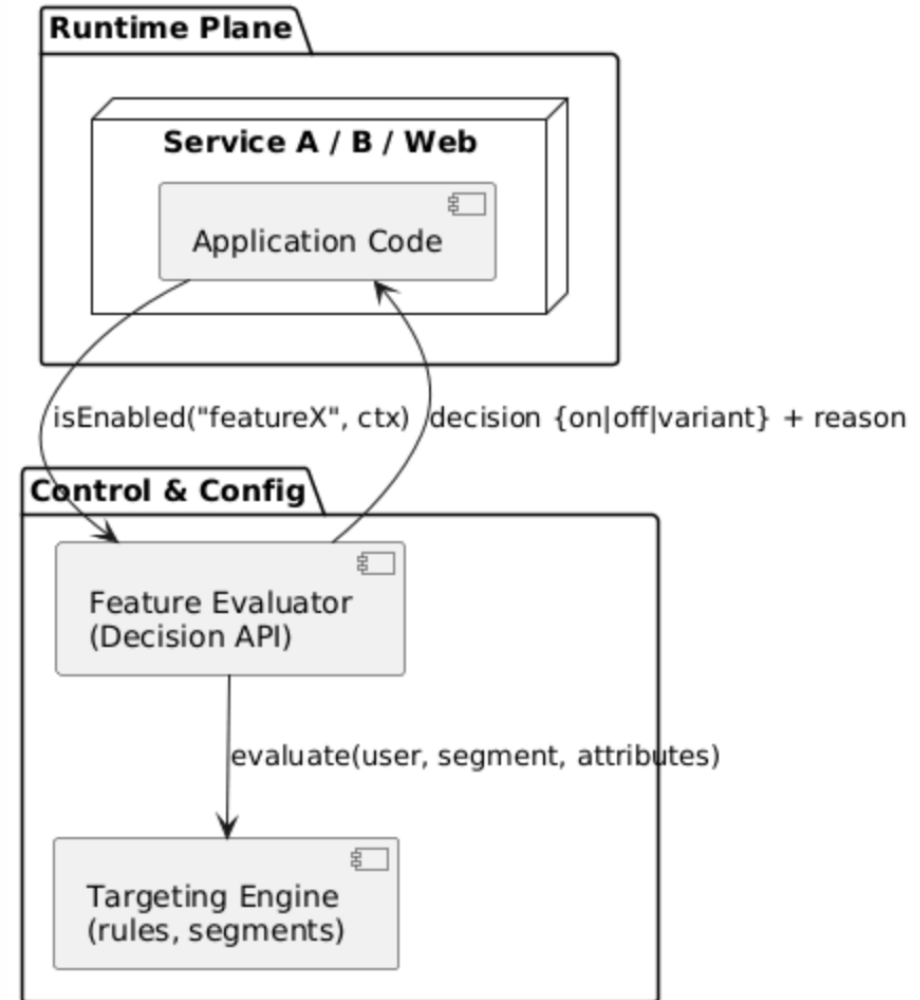
- 프로세스/파일시스템/네트워크 네임스페이스



Toggle Features (기능 활성화/비활성화)

□ 기본 개념

- 신규 기능을 배포한 이후에, '비상 정지 스위치'를 통해서 실행 시간에 기능을 비활성화 시킴으로써, 재배포의 비용과 위험을 제어하는 기술



Architectural tactics

ENERGY EFFICIENCY TACTICS

에너지 효율성 (Energy Efficiency)

□ 에너지 효율성 (정의)

- 소프트웨어 시스템이 주어진 기능을 수행하는 동안 소비되는 에너지의 양을 최소화할 수 있는 능력

□ 에너지 효율성과 관련한 아키텍처 고려사항

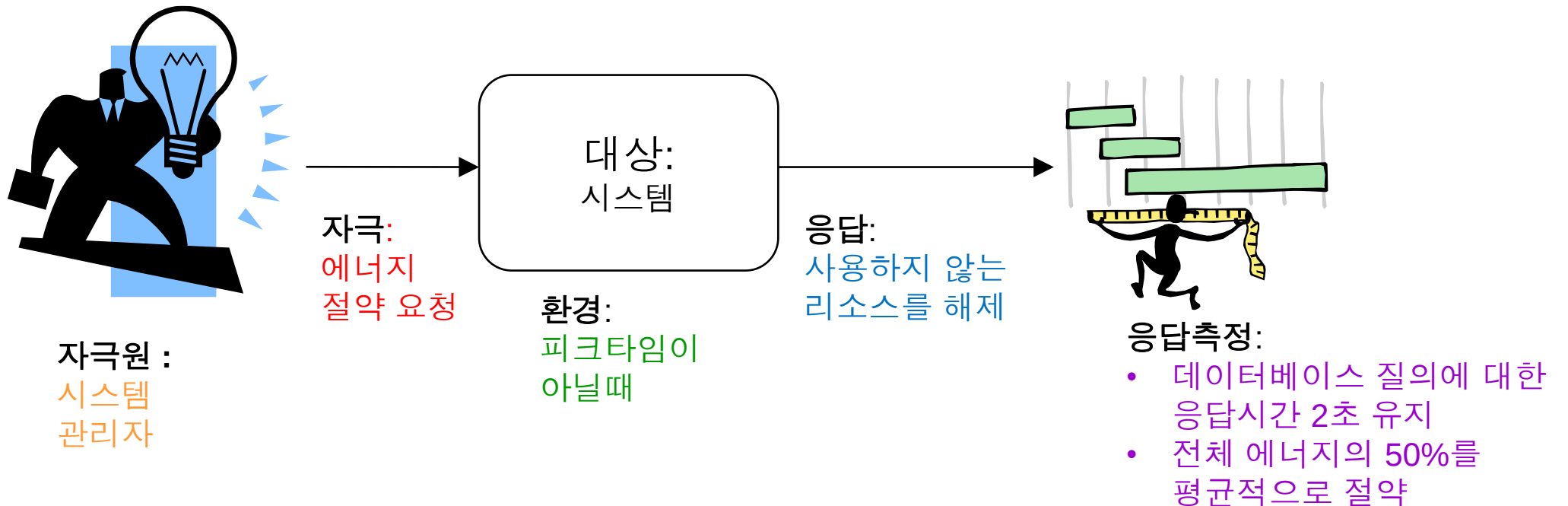
- 소프트웨어가 실행될 때 **CPU**, 메모리, 네트워크, 스토리지 등의 사용량을 최소화하도록 설계하는 것을 의미
 - 필요한 만큼의 컴퓨팅 자원을 적시에 사용할 수 있는 방법은?
 - 데이터 처리를 위한 양과 시간을 최소화할 수 있는 방안은?
 - 네트워크 통신을 위한 빈도와 통신 양을 최소화 할 수 있는 방안은?

에너지 효율성 일반 시나리오

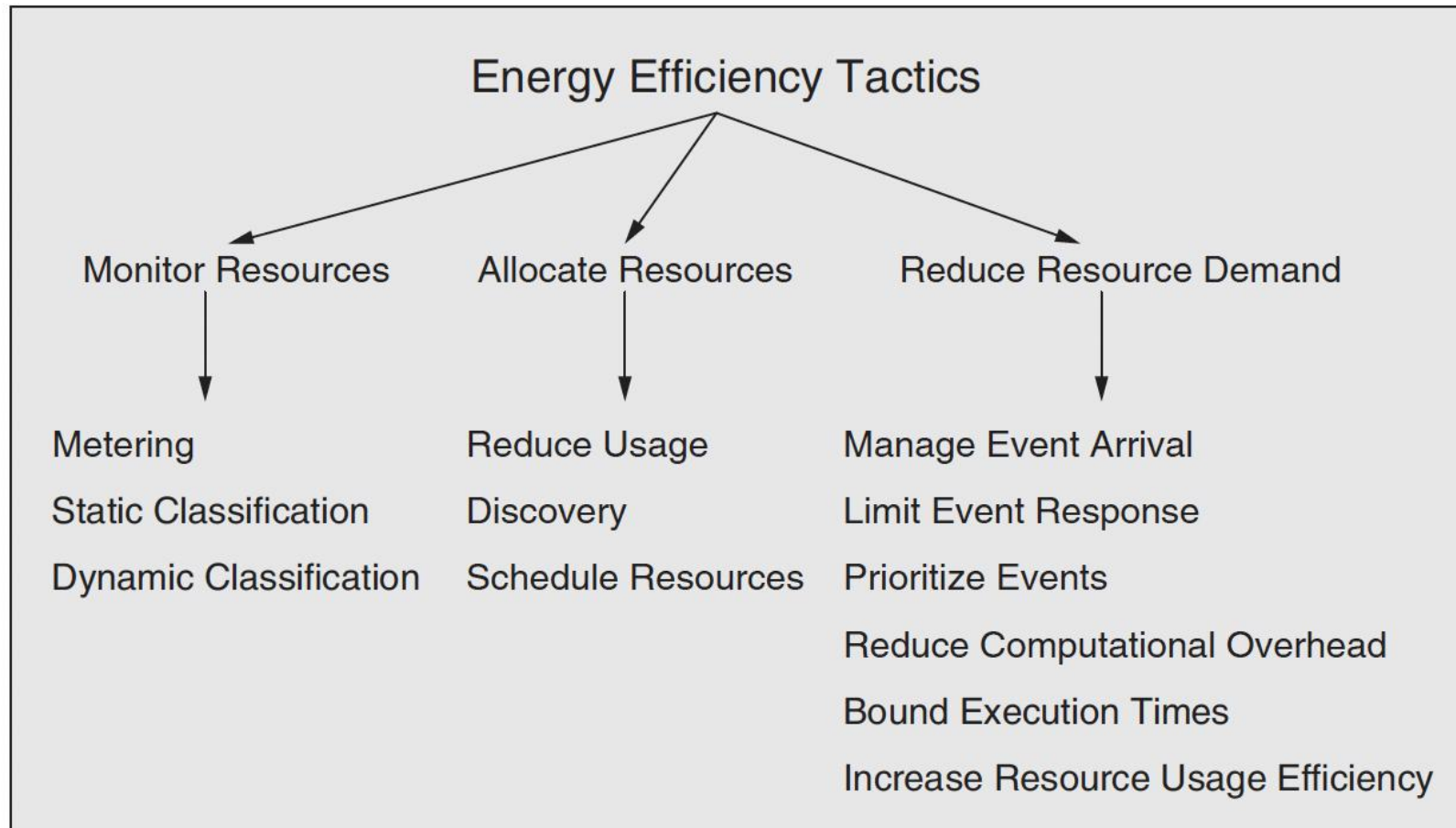
부분	설명	가능한 값
자극원	에너지 절약 요청의 주체	최종 사용자, 시스템 관리자, 에너지 관리 시스템
자극	에너지 절약 요청	불필요하게 높은 전력 소비 (전체 사용량, 순간 최대 사용량, 평균 사용량)를 절약하길 원함
대상	관리할 대상	특정 장치, 서버, VM, 클러스터
환경	어떤 운영 상태에서 요청 발생하는가?	정상 운전 상태, 혹은 부하가 낮거나 유휴(Idle) 상태의 실행 환경
응답	에너지 절약을 위한 행동	- 서비스 비활성화 - 런타임 서비스 할당 해제 - 서비스의 서버 할당 변경 - 서비스 수준 변경
응답 측정	얼마나 에너지를 절감했는 가를 어떻게 측정하는가?	- 절약된 에너지의 소비량 (평균/전체량) - 배터리 소모 감소율(%) - CPU/GPU 사용 감소율(%)

구체적인 에너지 절감 시나리오

“시스템 관리자는 피크타임이 아닐때 사용하지 않는 리소스를 해제 함으로써 에너지를 절약하기를 원한다. 시스템은 데이터베이스 질의에 대한 응답 시간 요건인 2초를 유지하면서 리소스를 해제해 필요한 전체 에너지의 50%를 평균적으로 절약한다.”



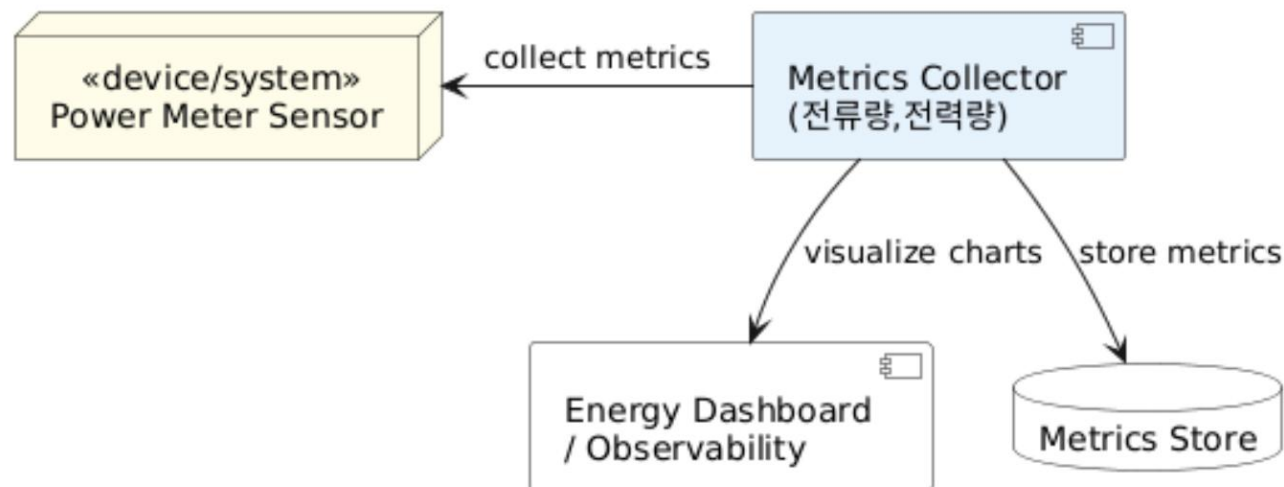
에너지 효율성 تاک



Monitor Resources::Metering

□ 기본 개념

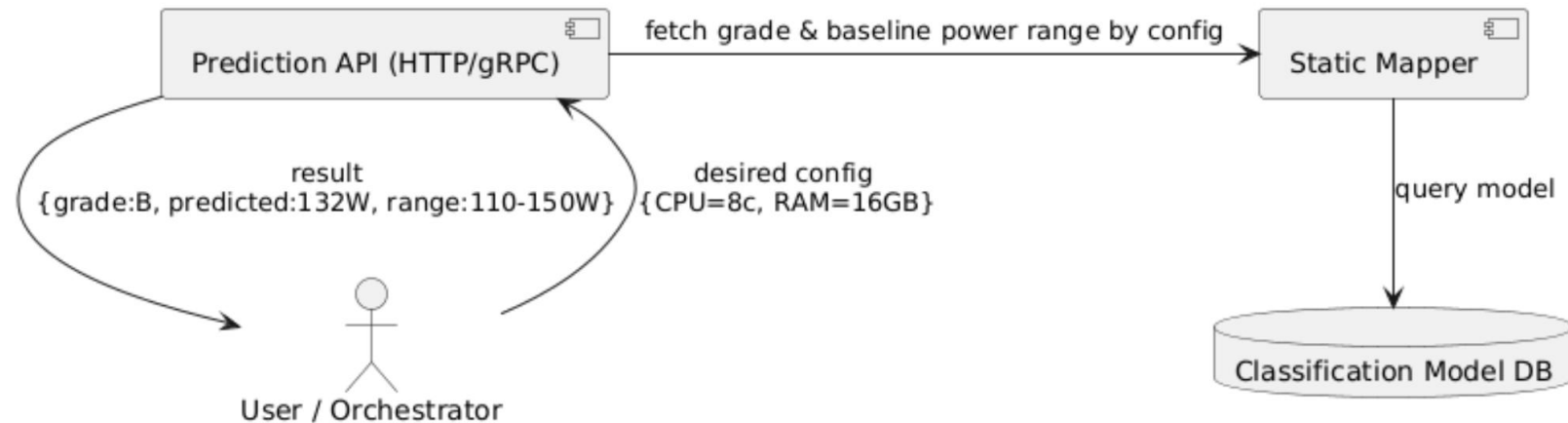
- **Metering**은 시스템의 에너지 소비를 센서 인프라를 통해서 실시간 혹은 주기적으로 측정하고 모니터링
 - 데이터센터의 전력량계 : 데이터 센터의 에너지 소비 측정
 - 전류계 나 전력계: 개별 서버나 하드 드라이브의 에너지 소비 측정
 - 배터리 관리시스템: 배터리로 운영되는 시스템의 에너지 소비 측정



Monitor Resources::Static Classification

□ 기본 개념

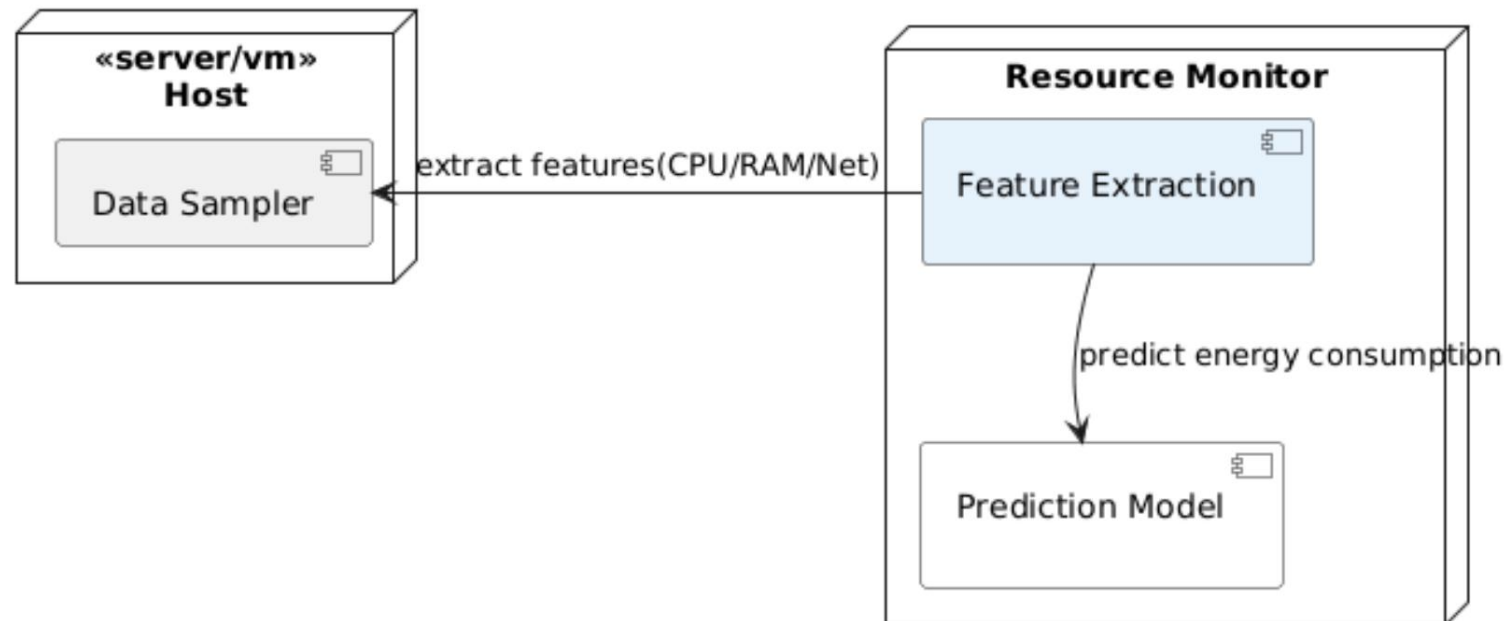
- 실시간 에너지 데이터 수집이 불가능한 경우 (원격 클라우드 사용), 자원(**Resources**)의 사전 정의되거나 고정된 특성을 기반으로 에너지 소비량을 예측



Monitor Resources::Dynamic Classification

□ 기본 개념

- 컴퓨팅 리소스의 정적 모델이 불충분한 경우, 다음과 같은 동적인 요인을 분석하여 실시간으로 에너지 소비를 예측한다.
 - 작업부하(**Workload**): 현재 **CPU** 사용률, 메모리 사용량, 네트워크 트래픽 등



Allocate Resources::Reduce Usage

□ 기본 개념

■ 에너지 사용 최소화 방법

- 화면 재생 빈도 줄이기
- 백그라운드 어둡게 하기
- 하드 드라이브 회전 속도 줄이기
- **CPU**를 느린 클럭 속도로 실행
- 가상머신들을 최소 개수의 물리적 서버에 배치
- 모바일 애플리케이션의 경우 컴퓨팅의 일부를 클라우드에 배치

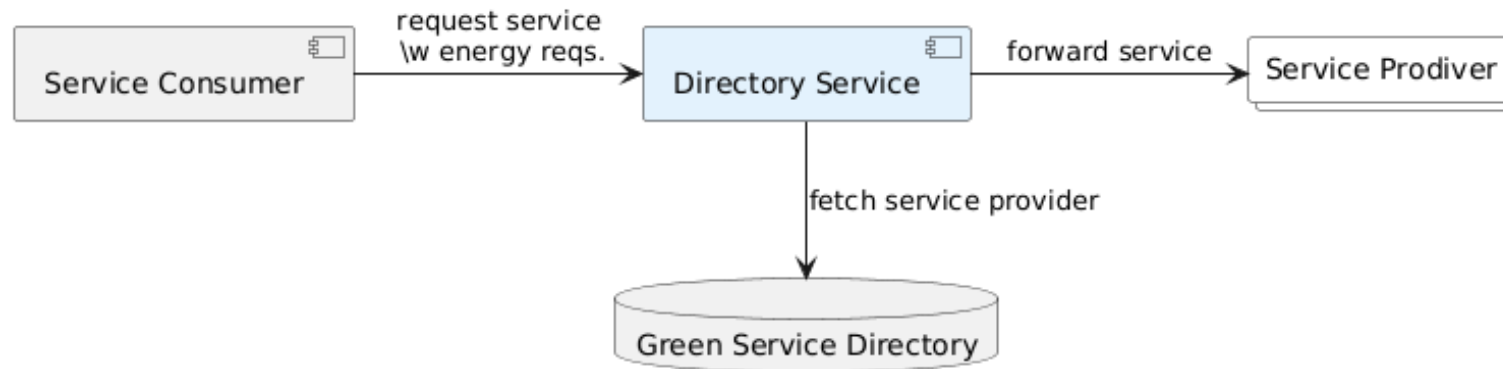
■ 불필요한 에너지 소비 제거 방법

- **CPU** 또는 서버 끄기
- 사용 중이지 않는 프로세서의 블록에 공급되는 전류를 끄기

Allocate Resources::Discovery

□ 기본 개념

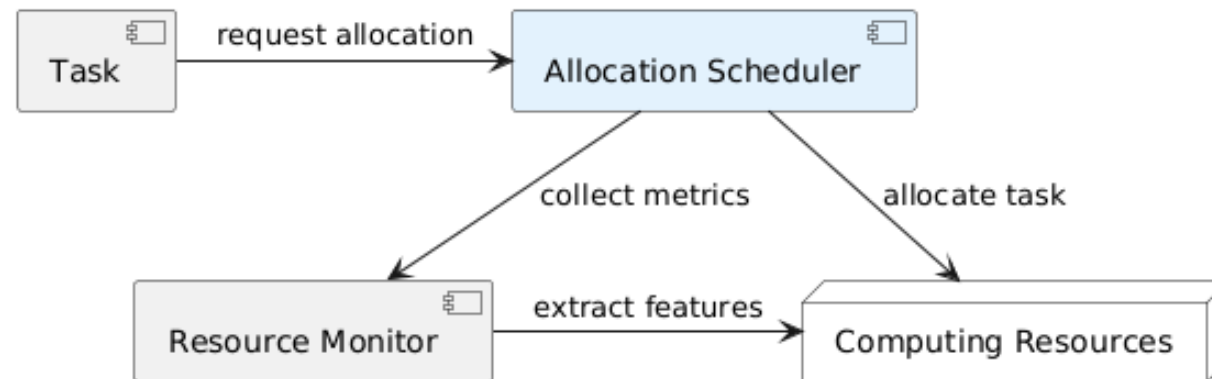
- **디스커버리 서비스**는 클라이언트로부터의 서비스 요청을 서비스 공급자와 매칭하는 역할
 - **그린 서비스 디렉토리**는 각 서비스 공급자(예: 가상 머신, 서버, 마이크로 서비스)에 대한 정량화된 에너지 특징(예, 트랜잭션당 에너지 소비율)을 저장
 - **디스커버리 서비스**는 서비스 요청이 에너지 정보(예, 최대 허용 에너지 소비량)를 포함하는 경우, **그린 서비스 디렉토리**를 통해서 에너지 특징을 기반으로 서비스 공급자(예: 가상 머신, 서버, 마이크로 서비스)를 선택



Allocate Resources::Schedule Resources

□ 기본 개념

- 에너지 인식 스케줄링은 작업을 에너지 효율성 관점에서 컴퓨팅 리소스에 할당하는 것
 - 하나 이상의 리소스 모니터링 택틱을 사용해 데이터를 수집
 - 에너지 효율성 관점에서 작업을 컴퓨팅 리소스에 할당
 - 컴퓨팅 작업을 에너지 효율성이 높은 혹은 에너지 비용이 낮은 컴퓨팅 리소스에 할당
 - 사용되는 컴퓨팅 리소스의 수를 최소화할 수 있도록 작업을 할당



Architectural tactics

INTEGRABILITY TACTICS

통합 용이성(Integrability)

□ 통합 문제

- 시스템 S 가 단위 소프트웨어들의 집합 $\{C_i\}$ 을 통합해야 하는 상황
 - S 는 통제할 수 있는 요소이지만, $\{C_i\}$ 는 통제 범위 밖의 대상(예, 외부 벤더에서 제공되는 대상)일 수 있음
 - 실제 통합이 이루어지는 시점은 미래 $\rightarrow \{C_i\}$ 에 대한 정보는 불완전
- 시스템 S 가 단위 소프트웨어들의 집합 $\{C_i\}$ 을 쉽게 통합할 수 있는 방안?
 - 시스템 S 의 아키텍처가 $\{C_i\}$ 과의 통합에 대한 예상을 바탕으로 수용 가능하도록 설계되었다면 $\rightarrow$ 통합 용이

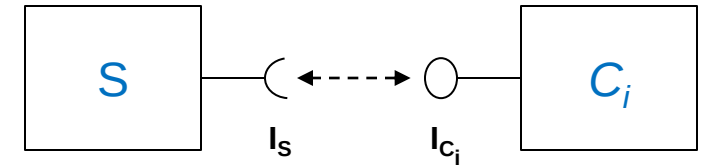
통합용이성(Integrability)

□ 통합 난이도

■ S 와 $\{C_i\}$ 인터페이스 간의 크기와 거리의 함수

□ 크기: S 와 $\{C_i\}$ 인터페이스 간의 의존관계 개수

□ 거리: 각 의존관계에 있는 인터페이스 간의 차이를 해결하는 난이도



분류	예시
구문적 거리	인터페이스에 정의된 데이터요소의 타입 및 개수 차이
데이터 의미론적 거리	I_s 측 고도 데이터(미터 단위) vs. $I_{\{C_i\}}$ 측 고도 데이터(피트 단위)
동작적 의미론적 거리	I_s 는 항상 사용가능 vs. $I_{\{C_i\}}$ 는 모드에 따라 사용가능
시간적 거리	I_s 는 이벤트 B 발생 이후에 즉시 이벤트 A 발생 vs. $I_{\{C_i\}}$ 는 이벤트 B 발생후 50ms 이내에 이벤트 A 발생
리소스 거리	I_s 는 리소스 R1에 대한 독점적인 권한 요구 vs. $I_{\{C_i\}}$ 는 리소스 R1에 대한 공유 접근 권한 기대

통합용이성 일반 시나리오

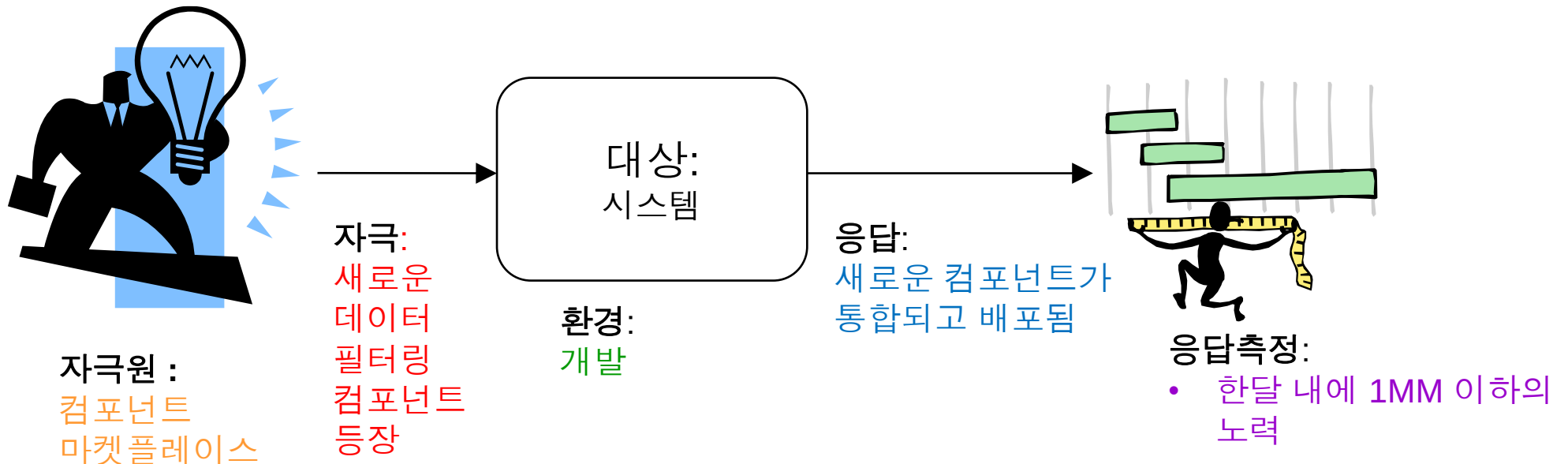
부분	설명	가능한 값
자극원	통합요청의 주체	다음 중 하나 이상이 가능하다. • 시스템 이해 관계자 • 컴포넌트 마켓 플레이스 • 컴포넌트 벤더
자극	통합요청의 종류	• 새로운 컴포넌트 추가 • 기존 컴포넌트의 신규버전 통합 • 기존 컴포넌트들을 새로운 방법을 통합
대상	시스템의 어떤 부분이 통합에 영향을 받는가?	• 전체 시스템 • 특정 컴포넌트 집합 • 컴포넌트 메타데이터 • 컴포넌트 환경설정

통합용이성 일반 시나리오

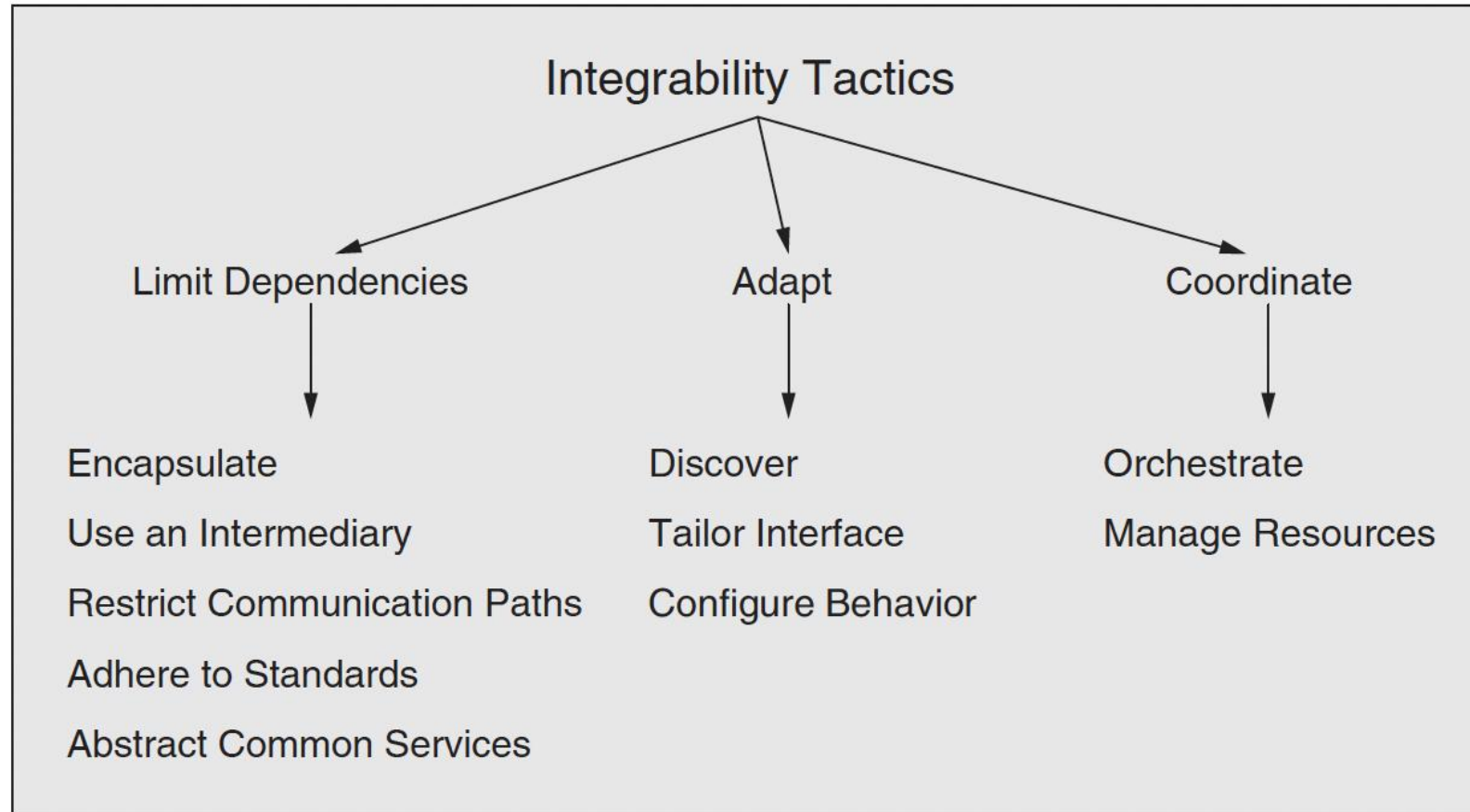
부분	설명	가능한 값
환경	어떤 상태에서 통합요청이 발생하는가?	• 개발 • 통합 • 배포 • 런타임
응답	통합 요청에 대한 반응	다음 중 하나 이상이 가능하다. • 변경이 (완료, 통합, 테스트, 배포)됐다. • 컴포넌트들이 새로운 환경 설정에서 • 올바르게 정보를 교환하고 있다. • 어떤 리소스 제한도 위반하지 않는다.
응답 측정	응답을 어떻게 측정하는가?	다음 중 하나 이상의 관점에서 비용을 측정 • 변경된 컴포넌트 수 • 변경된 코드의 퍼센트 • 변경된 코드의 줄 수 • 노력 • 돈 • 실제 걸린 시간

구체적인 통합용이성 시나리오

“새로운 데이터 필터링 컴포넌트를 컴포넌트 마켓플레이스에서 구매할 수 있게 됐으며, 해당 컴포넌트를 한 달 안에 시스템에 통합하고 배포해야 한다. 이때 1MM 이상의 노력이 들어가서는 안된다”



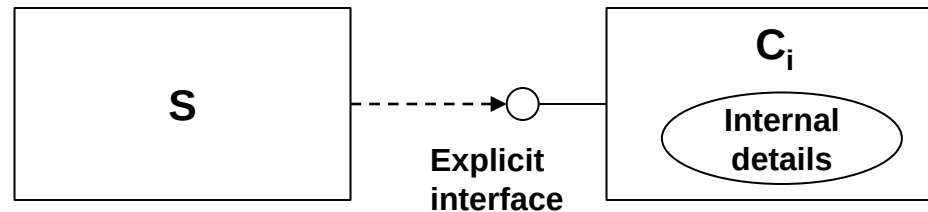
통합용이성 택틱



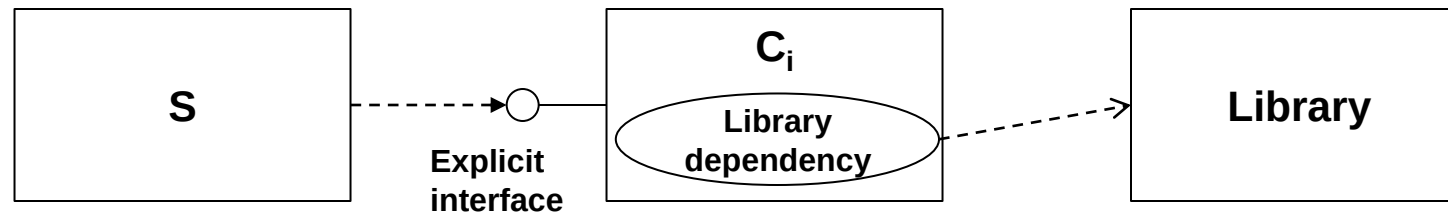
Limit Dependencies::Encapsulate

□ 기본 개념

- 캡슐화는 어떠한 요소에 대한 명시적인 인터페이스를 정의하고, 해당 요소에 대한 모든 접근이 해당 인터페이스를 통해서 이루어지도록 함
- 의존 관계의 수 & 거리를 줄임
 - 요소 내부 사항에 대한 의존성 제거



- 특정 통합작업과 무관한 인터페이스는 숨길 수 있음



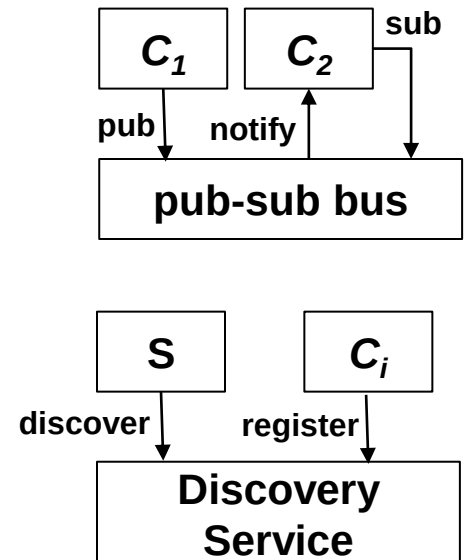
Limit Dependencies::Use an Intermediary

□ 기본 개념

- 중재자 (Intermediary)는 컴포넌트 C_i 간의 혹은 컴포넌트 C_i 와 시스템 S 간의 의존관계를 깨기 위한 용도로 사용됨

□ 사례

- 데이터 의존관계를 위한 중재자
 - 발행-구독 버스 (publish-subscribe bus)
 - 데이터 소비자의 데이터 생산자에 대한 의존성 제거
- 서비스 의존관계를 위한 중재자
 - 동적 서비스 디스커버리(dynamic service discovery)
 - 서비스들은 동적인 조회를 통해 서로의 존재를 인지함



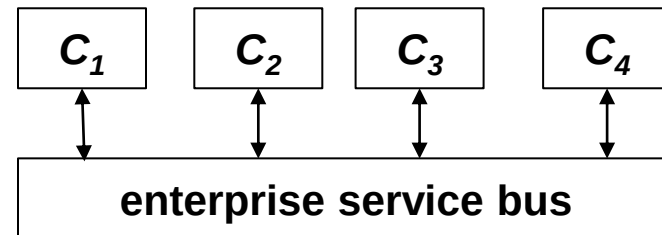
Limit Dependencies::Restrict Comm. Paths

□ 기본 개념

- 어떤 요소가 통신할 수 있는 대상(요소들의 집합)을 제한하는 전술
- 구체적인 기법
 - 가시성 제한 (인터페이스가 노출된 대상만 통신 가능)
 - 권한 부여 (권한이 있는 요소들만 접근 가능)

□ 사례

- 엔터프라이즈 서비스 버스 (**enterprise service bus**)
 - SOA에서 모든 요청에 대한 라우팅, 중재, 프로토콜 변환 등을 수행



Limit Dependencies::Adhere to Standards

□ 기본 개념

- 표준은 다양한 플랫폼과 벤더에 걸쳐 효과적인 통합을 가능케 하는 핵심 기술임
- 표준이 무엇을 규정하는 지에 따라, 표준 준수 기술은 구문적 거리, 데이터 의미론적 거리, 동작적 거리, 시간적 거리와 같은 다양한 차원의 거리를 해결할 수 있음
- 해당 표준의 효과성은 다음 요소에 의해 결정된다.
 - 해당 표준이 어떤 측면의 차이점을 다루는지
 - 향후 컴포넌트 공급사들이 해당 표준을 준수할지

Limit Dependencies::Abstract Common Services

□ 기본 개념

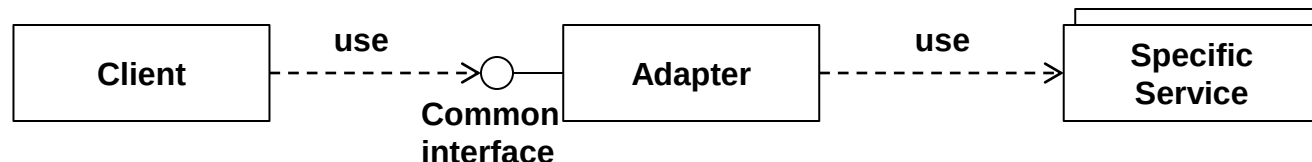
- 유사 서비스를 제공하는 두 요소를 공통의 추상화(인터페이스) 뒤에 숨겨, 범용적인 서비스로 통합하는 전술

□ 구체적인 방법

- 공통 인터페이스의 구현



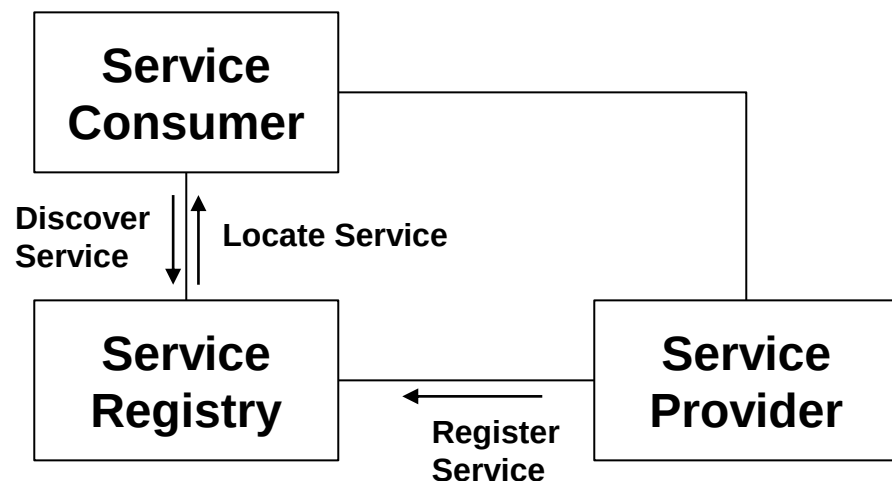
- 공통 서비스 요청을 구체적인 서비스 요청으로 변환하는 중재자(예, 어댑터)
 - 특정 요소들 간의 구문적, 의미론적 차이점을 정규화할 수 있음



Adapt::Discover

□ 기본 개념

- 런타임(Runtime)에 필요한 서비스 제공자(Service Provider) 또는 데이터 소스를 찾아내고 연결하는 기술



작동 원리

1. 등록 (Registration): 서비스 제공자가 시작될 때마다 자신의 주소(IP, Port), 제공하는 서비스 이름, 현재 상태 등을 서비스 레지스트리에 등록
2. 조회 (Lookup): 서비스 요청자(클라이언트)는 서비스를 사용하기 전에 레지스트리에 서비스 이름을 질의
3. 응답 (Resolution): 레지스트리는 요청된 서비스를 제공하는 사용 가능한 인스턴스의 네트워크 주소를 응답
4. 호출 (Invocation): 요청자는 응답받은 주소를 사용하여 해당 서비스 인스턴스를 직접 호출

Adapt::Discover

□ 통합용이성 측면의 강점

■ 위치 투명성 (Location Transparency)

- 서비스 제공자의 물리적 위치(서버)가 바뀌거나 스케일 인/아웃(Scale In/Out)으로 인스턴스가 추가/삭제되어도, 요청자는 동적으로 주소를 받아 사용하기 때문에 요청자 코드를 수정할 필요가 없습니다.

■ 복원력 (Resilience)

- 서비스 레지스트리가 장애가 발생한 인스턴스를 목록에서 제외하면, 요청자는 자동으로 정상 인스턴스만 찾아 호출할 수 있어 시스템의 복원력이 향상됩니다.

■ 유연한 통합

- 새로운 서비스 인스턴스를 추가하거나 기존 인스턴스를 교체하는 것이 매우 쉽고 빠르기 때문에 시스템 통합 및 확장이 용이합니다

Adapt::Tailor Interface

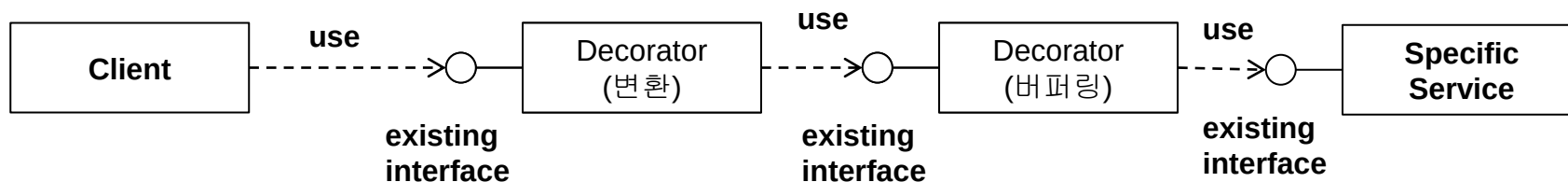
□ 기본 개념

- 통합하고자 하는 시스템 간의 차이(**Discrepancy**)를 줄이기 위해서, **API**나 구현을 변경하지 않고도 기존 인터페이스에 기능을 추가하거나 기존 인터페이스의 기능을 감추는 기술

□ 구체적인 사례

■ 데코레이터 패턴

- 기존 컴포넌트의 인터페이스 변경없이, 데이터 포맷 변환(구문적 차이)과 버퍼링(시간적 차이) 기능을 데코레이터로 구현하여 원활한 통합을 지원

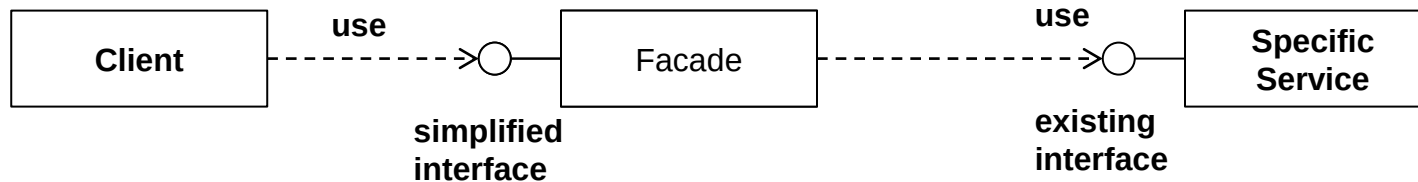


Adapt::Tailor Interface

구체적인 사례

■ Façade 패턴

- 제공되는 서비스가 너무 많은 기능을 포함하고 있어, 클라이언트가 실제로 필요한 기능만 노출하도록 인터페이스를 간소화



■ API 게이트웨이

- 불필요한 필드를 제거하고 필요한 데이터만 묶어서 전달



Adapt:: Tailor Interface

□ 통합용이성 측면의 강점

■ 통합 노력 감소 (Reduced Integration Effort)

- 클라이언트가 서비스 제공자의 복잡하거나 이질적인 인터페이스에 억지로 맞춰 코드를 수정할 필요가 없습니다. 대신, 클라이언트에게 가장 익숙하고 필요한 형태로 인터페이스가 제공되므로, 통합 개발 시간이 단축됩니다.

■ 사용 용이성 증대 (Increased Usability)

- 인터페이스가 단순해지면 클라이언트 개발자가 서비스를 오용하거나 잘못 통합할 위험이 줄어듭니다.

■ 기존 시스템 보호:

- 인터페이스 조정 계층이 추가됨으로써, 서비스 제공자는 자신의 핵심 비즈니스 로직(Core Logic)이나 레거시 시스템의 인터페이스를 외부 요구에 맞춰 자주 변경할 필요가 없어집니다.

Adapt:: Configure Behavior

□ 기본 개념

- 시스템 **S**의 부분으로 환경 설정 가능한 동작을 포함시킴으로써, 시스템 **S**가 더 넓은 범위의 잠재적인 **C**들을 지원할 수 있도록 하는 기술 시스

□ 작동 방식

■ 외부화 (Externalization)

- 통신 프로토콜, 타임아웃 값, 자격 증명(Credentials), 서비스 엔드포인트 URL 등 통합에 필수적인 세부 정보를 XML, JSON, YAML 파일, 데이터베이스, 환경 변수 등 외부 설정 파일에 저장합니다.

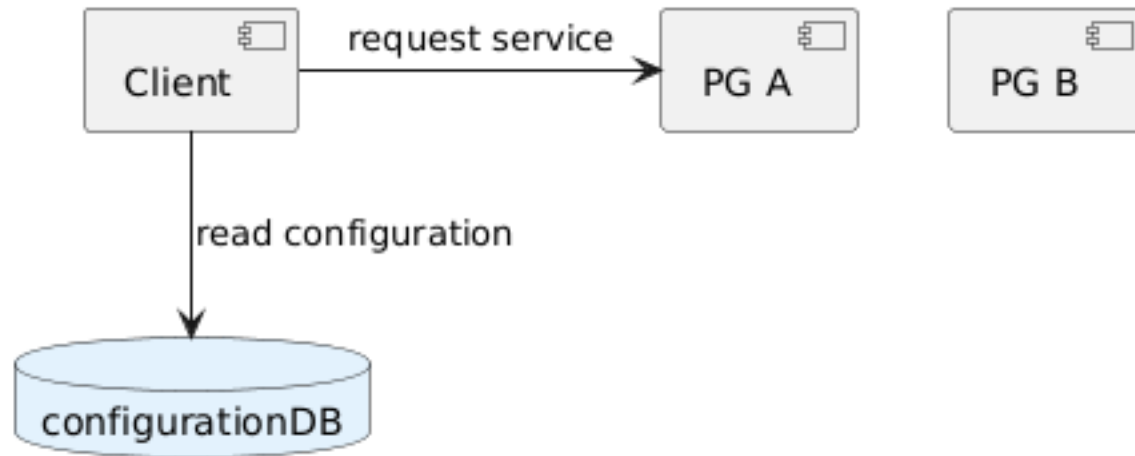
■ 런타임 로딩 (Runtime Loading)

- 시스템은 시작 또는 런타임 중에 이 외부 설정 파일을 읽어와 자신의 행동 방식을 결정합니다

Adapt:: Configure Behavior

□ 사례

- 클라이언트 시스템이 두 개의 서로 다른 결제 서비스 (**PG A**, **PG B**) 중 하나와 통합되어야 하는 상황을 가정합니다.



Coordinate:: Orchestrate

□ 기본 개념

- 시스템 **S**와 컴포넌트 $\{C_i\}$ 간의 의존 관계의 수를 줄이고, 컴포넌트 $\{C_i\}$ 간의 명시적인 의존관계를 제거하고, 중앙화함으로써 동작하는 전술



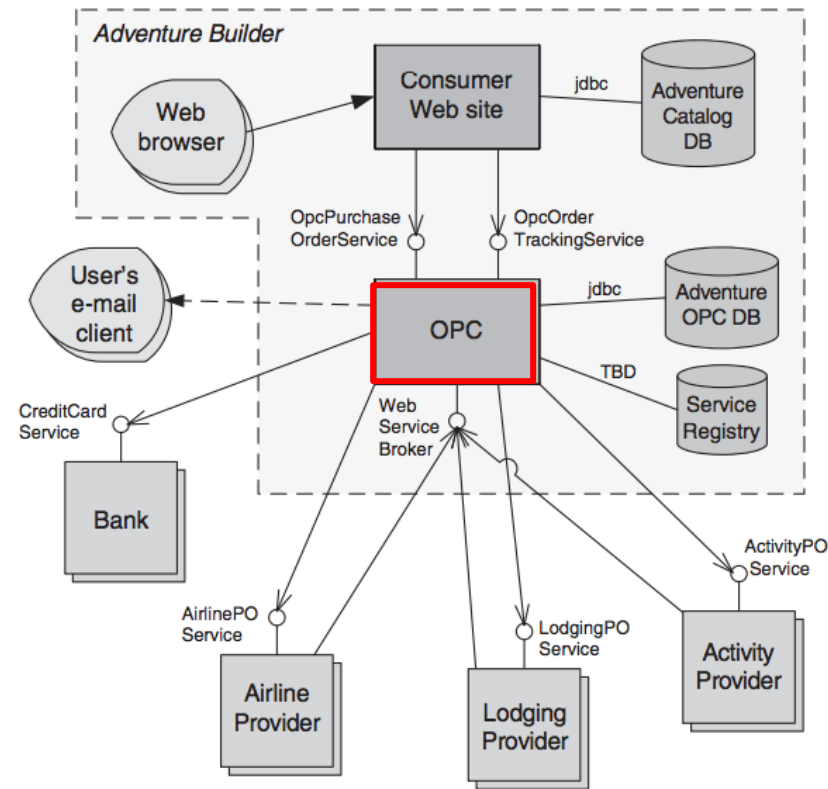
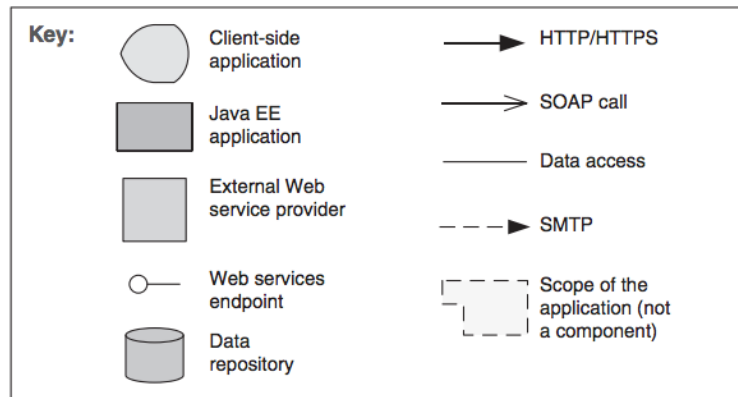
작동 원리

1. 단일 요청 수신: 시스템 **S** 오케스트레이터에게 복합적인 작업(예: "주문 처리")을 요청합니다.
2. 흐름 제어 (**Flow Control**): 오케스트레이터는 미리 정의된 **비즈니스 프로세스(Workflow)**에 따라 서비스 호출 순서를 결정합니다.
3. 순차적 호출 및 데이터 변환: 오케스트레이터는 첫 번째 서비스 **C1** 를 호출하고, **C1**의 결과를 받아 필요한 경우 데이터를 변환한 후, 이를 두 번째 서비스 **C2** 의 입력으로 전달합니다.
4. 최종 응답: 모든 서비스 호출이 완료되면, 오케스트레이터는 결과를 종합하여 시스템 **S**에게 최종 응답을 제공합니다.

Coordinate:: Orchestrate

□ 사례

The Adventure Builder system interacts with external service providers to construct the vacation, and with bank services to process payment. The central OPC (Order Processing Center) component coordinates the interaction with internal and external service consumers and providers



* L. Bass et al., Software Architecture in Practice (3rd Edition), Addison Wesley

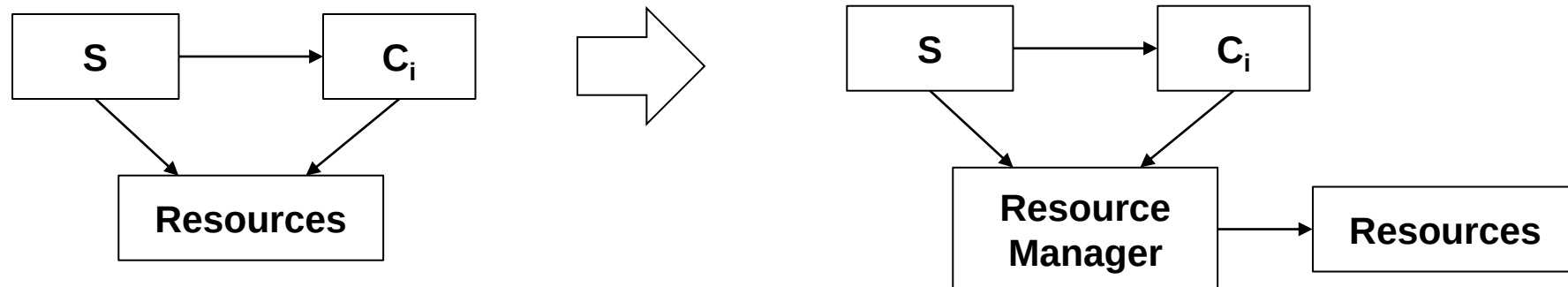
Coordinate:: Manage Resources

□ 기본 개념

- 시스템 **S**와 컴포넌트 **C** 사이의 리소스 거리를 줄이고, 리소스 요구사항을 명확하게 노출하고 해당 리소스에 대한 사용을 관리하는 전술

□ 리소스 관리자

- 컴퓨팅 리소스에 대한 접근을 관장하는 특화된 형태의 중개자
- 시스템 **S**와 다른 컴포넌트 **{C_i}**들이 통합될 때 필요한 자원의 획득, 사용, 해제 과정을 체계적으로 관리



Coordinate:: Manage Resources

□ 사례

풀링 대상	설명	효과
커넥션 풀 (Connection Pool)	DB 또는 외부 서비스와의 연결을 미리 맺어놓고 재활용.	요청마다 발생하는 연결 설정 및 해제 오버헤드를 제거. 통합 속도 향상.
객체 풀 (Object Pool)	생성 비용이 높은 객체들을 미리 만들어 저장해 두고 필요할 때 빌려 씀	메모리 할당 및 가비지 컬렉션 부담 감소
스레드 풀 (Thread Pool)	작업 처리에 사용할 스레드의 개수를 제한하고 관리.	스레드 생성 오버헤드를 줄이고, 시스템이 처리할 수 있는 동시성(Concurrency)을 통제하여 오버로드 방지.

Architectural tactics

MODIFIABILITY TACTICS

Modifiability

- **Modifiability** is about change
- Our concern centers on the cost and risk of making changes
 - What can change?
 - What is likelihood of the change?
 - When is the change made and who makes it?
 - What is the cost of the change?
 - The cost of **introducing the mechanism(s)** to make the system more modifiable
 - The cost of **making the modification using the mechanism(s)**

General Modifiability (Adaptability)

Scenarios

- ❑ **Source of Stimulus: who makes the changes**
 - Developer, end user, system administrator
- ❑ **Stimulus: the changes to be made**
 - Addition/deletion/modification of a function, quality attribute, capacity, technology
- ❑ **Artifact: what is to be changed**
 - Components or modules
 - The system's platform
 - Its user interface
 - its environment
 - Another systems with which it interoperates

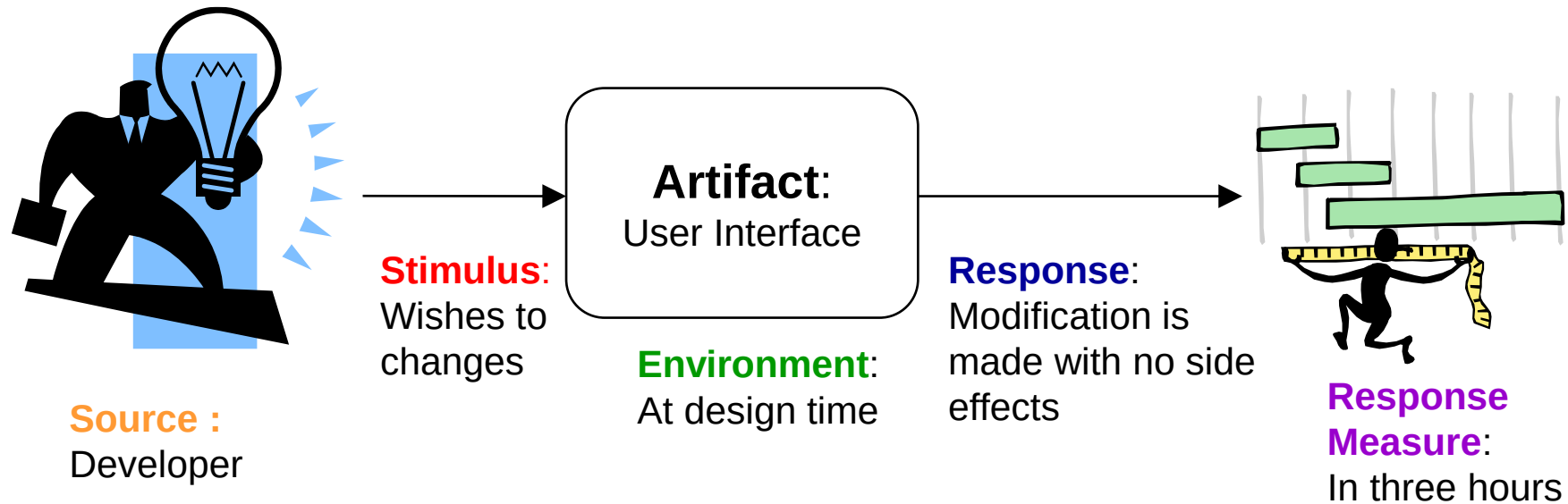
General Modifiability (Adaptability)

Scenarios

- ❑ **Environment: when the change can be made**
 - Design time, compile time, build time, initiation time, or run time
- ❑ **Response**
 - Makes modification
 - tests modification
 - deploys modification
- ❑ **Response Measure**
 - Number, size, complexity of affected elements
 - Effort
 - Calendar time
 - Money

Concrete Modifiability (Adaptability) Scenario

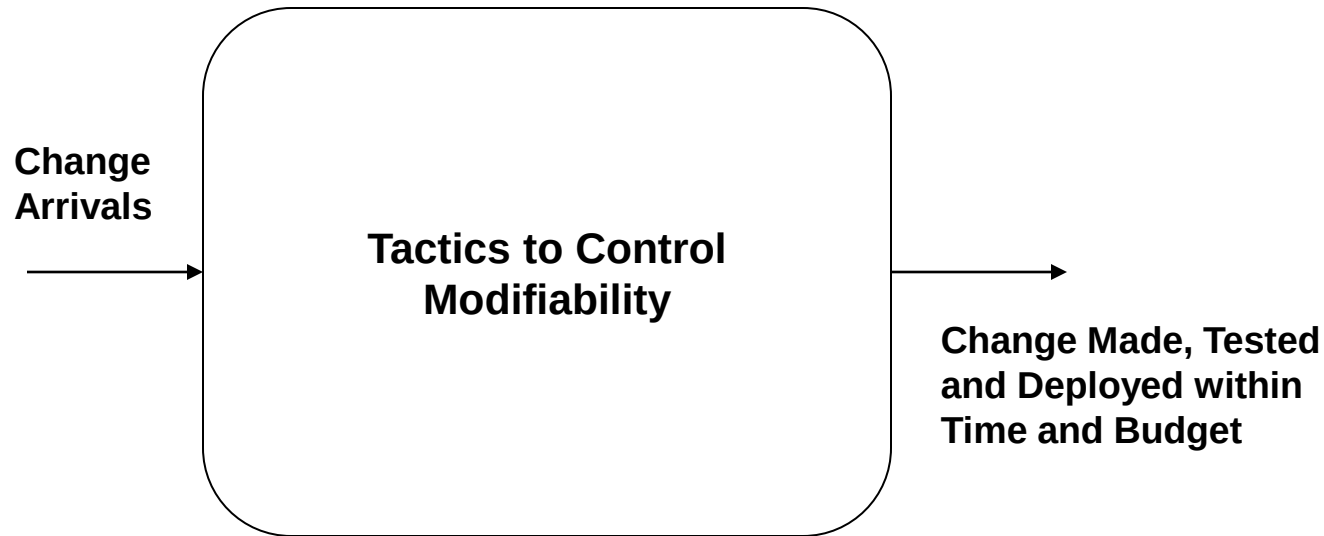
“A **developer** wishes to **change** the user interface **to make a screen’s background color blue**. **This change will be made to the code at design time**. **It will take less than three hours to make and test the change** and **no side effect changes will occur in the behavior**”



Modifiability Tactics

□ Goals for modifiability tactics

- To control the complexity of making changes as well as the time and code to make changes.



Coupling and Cohesion

- **Modules have responsibilities**

- by making changes to the modules, these responsibilities are changed in some way.

- ***Coupling*** measures the probability that a modification to one module will propagate to the other

- High coupling is an enemy of modifiability

- ***Cohesion*** measures how strongly the responsibilities of a module are related.

- The higher the cohesion, the lower the probability that a given change will affect multiple responsibilities in the other modules

Approaches to achieving modifiability

❑ Reduce size of a module

- If the system is made up from several smaller modules, it will be easier to make the change than if it was just one big module.

❑ Increase cohesion

- If a module has low cohesion, this can be improved by removing responsibilities until the cohesion is high enough

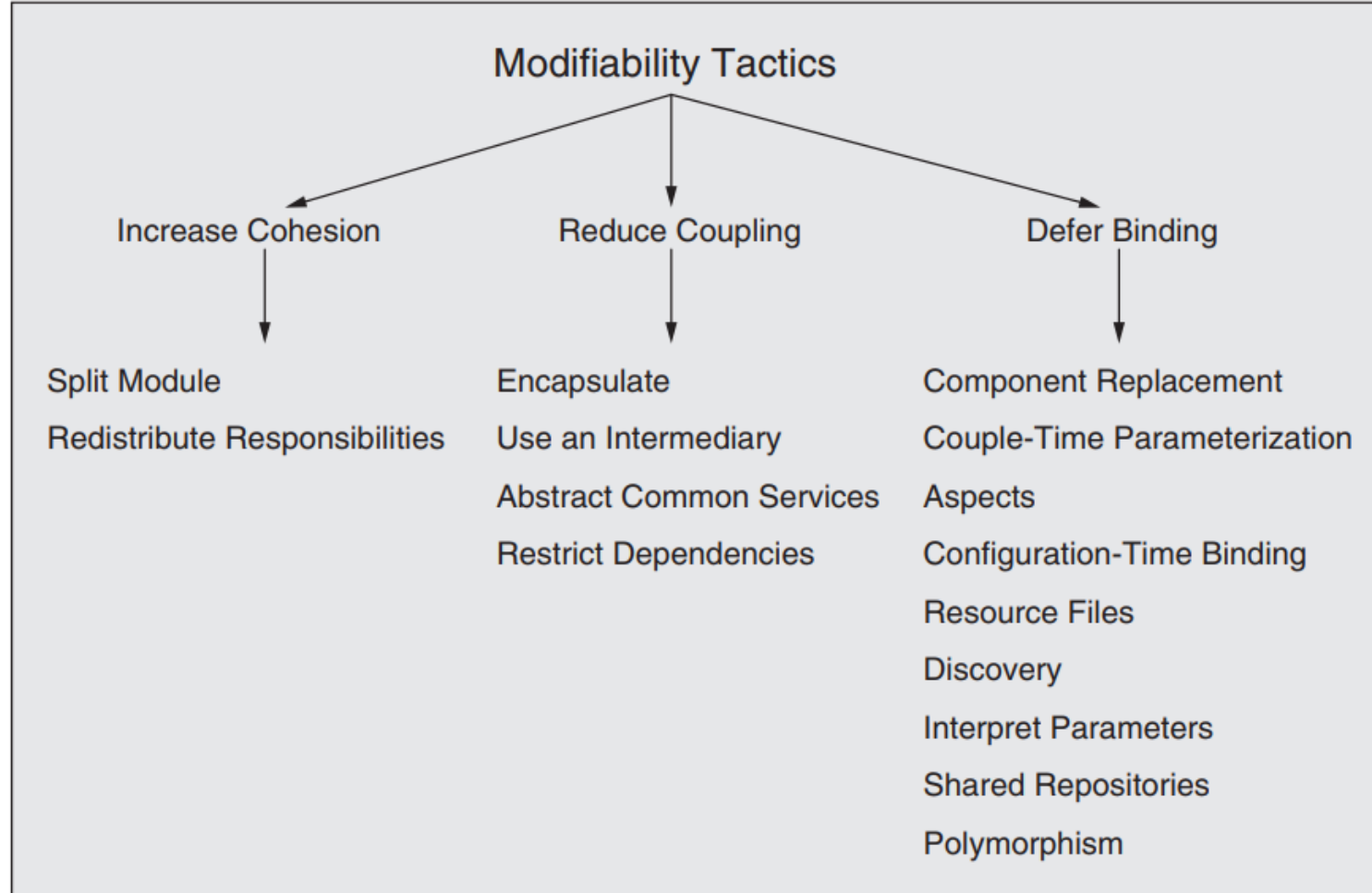
❑ Reduce coupling

- Reducing the coupling between different modules will help us to assure that a change to one module does not propagate to another module

❑ Binding time of modification

- If you want to make your system highly modifiable, you want to bind things as late as possible.

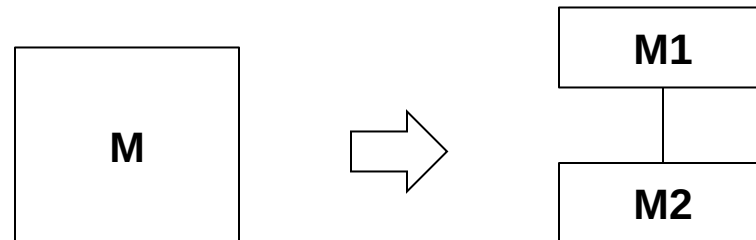
Hierarchy of Modifiability Tactics



Increase Cohesion

□ Split module

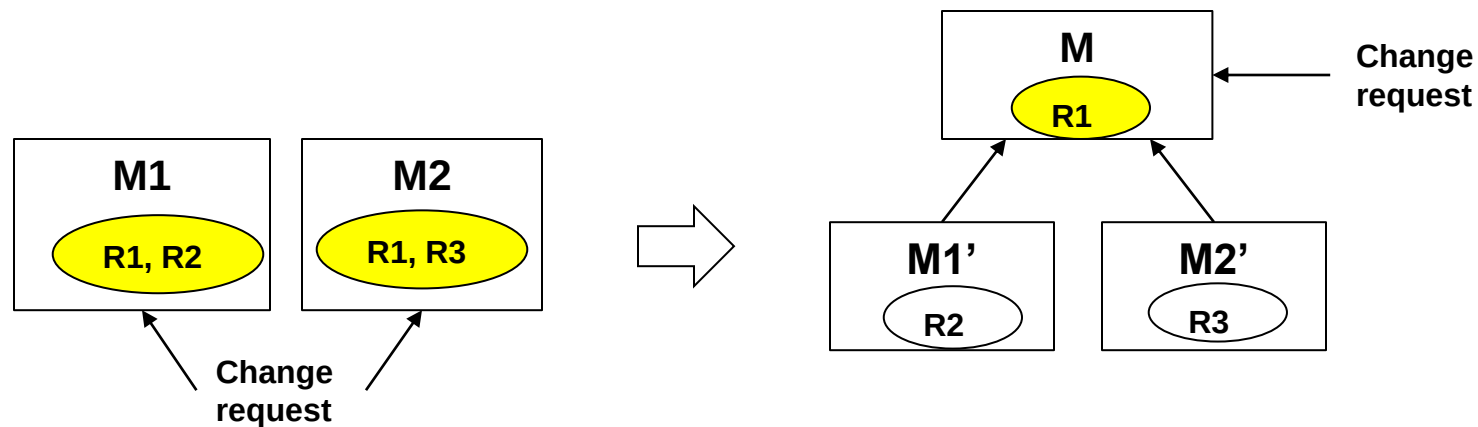
- If the module being modified includes a great deal of capability, the modification costs will likely be high.
- **single-responsibility principle (SRP)**
 - Every module, class or function in a computer program should have responsibility over a single part of that program's functionality
- Refining the module into several smaller modules should reduce the average cost of future changes.



Increase Cohesion

□ Redistribute responsibility

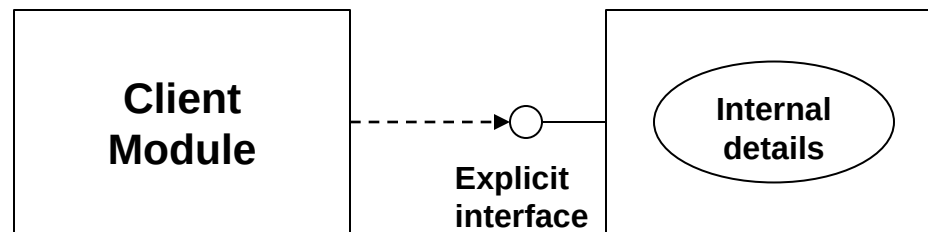
- If the responsibilities A, A', and A'' (all similar responsibilities) are sprinkled across several distinct modules, they should be placed together.
- If two modules are affected by the same change because they are (at least partial) duplicates of each other, **common responsibilities** (and the code that implements them) are “**factored out**” of the modules.



Reduce Coupling

□ Encapsulate

- Encapsulation introduces an explicit interface to a module.
 - This interface should hide the internal details of a module
- This tactic **reduces the probability that a change to one module propagates to other modules**

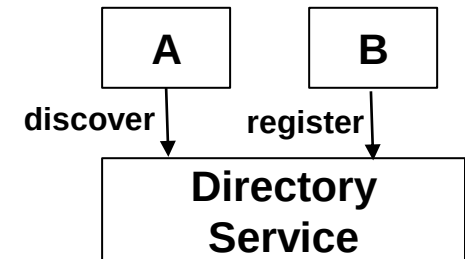
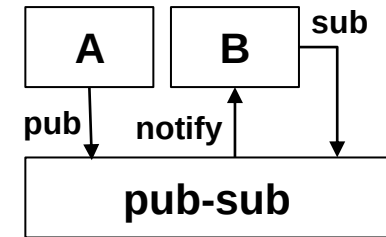


Reduce Coupling

□ Use an intermediary

- Given a dependency between **responsibility A** and **responsibility B**, the dependency can be broken by using an intermediary.
- **Intermediary for data dependency**
 - a publish-subscribe intermediary
 - Removes the data producer's knowledge of its consumers
- **Intermediary for service dependency**
 - **Directory Service component in SOA**
 - Services discover each other by dynamic lookup

A
⋮
B



Reduce Coupling

❑ Restrict dependencies

- Restricting a module's visibility or access to only authorized modules

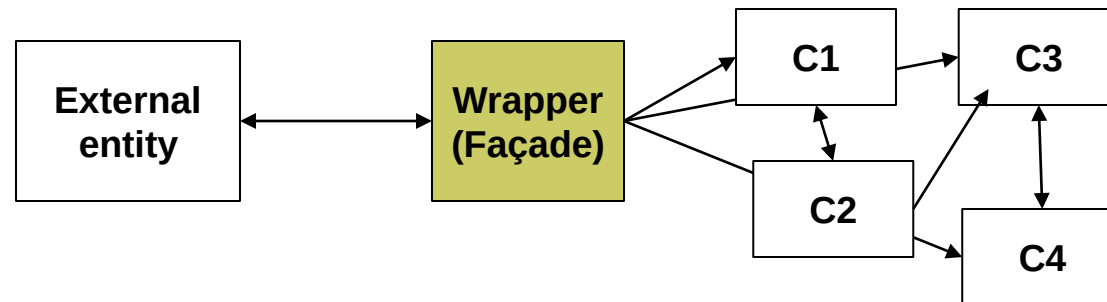
■ Example

❑ Layered architectures

- a layer is only allowed to use lower layers

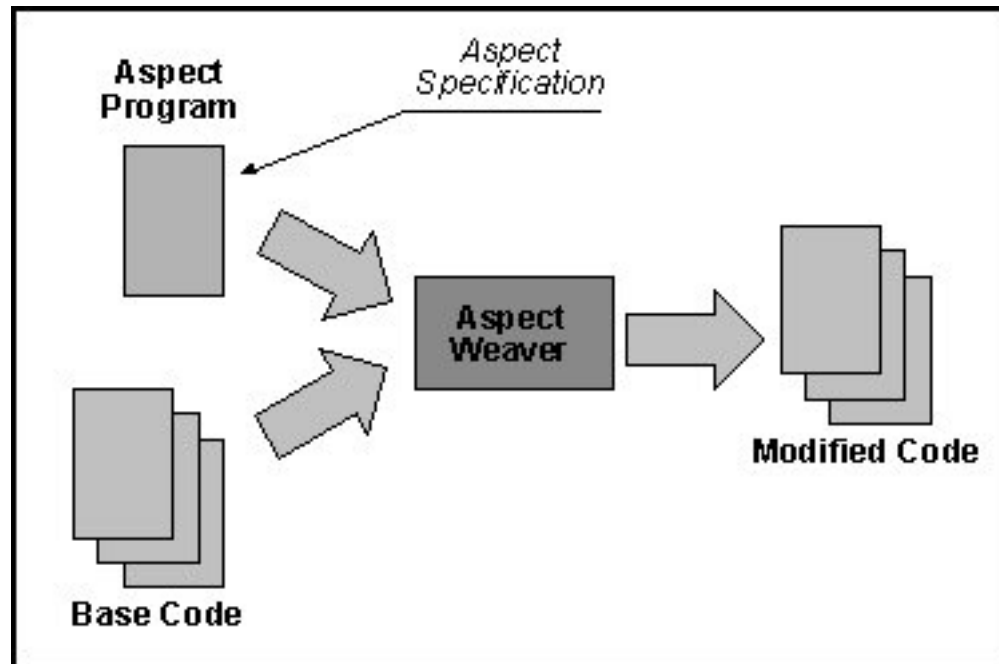
❑ Wrappers (or Façade)

- external entities can only see (and hence depend on) the wrapper and not the internal functionality that it wraps



Defer Binding Time

- ❑ **Compile or build time binding tactics**
 - **Component replacement (for example, in a build script or makefile)**
 - **Compile-time parameterization (for example, preprocessing)**
 - **Aspects**



Defer Binding Time

- ❑ **Startup or initialization time binding tactics**
 - **Resource files**
 - ❑ Set parameters at startup
- ❑ **Runtime binding tactics**
 - **Service registration and discovery**
 - **Publish-subscribe**
 - **Shared repositories**
 - **Dynamic Plug-ins**
 - **Polymorphism**
 - **AOP**

Summary

- ❑ **Modifiability** deals with **change and the cost** in time or money **of making a change**
- ❑ **Tactics to reduce the cost of making a change** include
 - **making modules smaller**
 - **increasing cohesion**
 - **reducing coupling**
 - **deferring binding**
- ❑ **Reducing coupling** is a standard category of tactics that includes
 - **encapsulating, using an intermediary, restricting dependencies, refactoring, and abstracting common services**

Architectural tactics

PERFORMANCE TACTICS

Performance

- **Performance is about time**

- When events occur, the system must respond to them in time

- **Essential characterization of performance**

- **Arrival pattern for events**

- **Periodic**: events arrive at regular time intervals
 - **Stochastic**: events arrive according to some probabilistic distribution.
 - **Sporadic**: periodic nor stochastic

- **Time-based response to those events**

- **Latency**: the time between the arrival of the event and the system's response
 - **Jitter**: the allowable variation in the latency
 - **Throughput**: the number of events that can be processed within a particular time interval
 - **Miss rate**

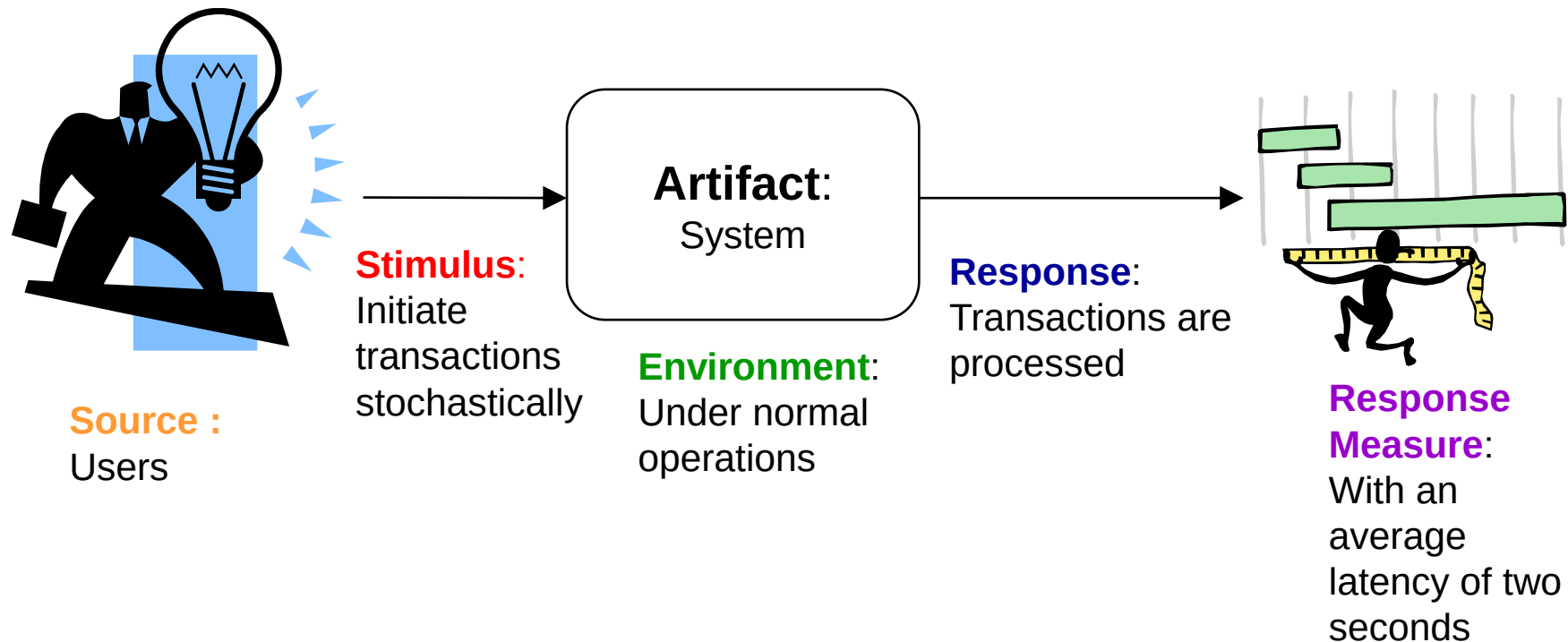
$$\frac{\text{\textit{\#of events not processed}}}{\text{\textit{total \# of events}}}$$

General Performance Scenarios

- ❑ **Source**
 - Internal or external to the system
- ❑ **Stimulus**
 - Arrival of a periodic, sporadic, or stochastic events
- ❑ **Artifact**
 - System
 - One or more components in the system
- ❑ **Environment**
 - Operation mode: Normal, emergency, peak load, overload
- ❑ **Response**
 - Processes events
 - Changes level of service
- ❑ **Response Measure**
 - Latency, Deadline, Jitter, Throughput, Miss rate

Concrete Performance Scenario

“**Users** initiate 1,000 transactions per minute stochastically under normal operations, and the system should process the transactions with an average latency of two seconds”



Performance Tactics

❑ Two basic contributors to the response time

■ Processing time

❑ Processing consumes resources, which take time

- HW resources: CPU, data stores, network communication bandwidth, memory
- SW resources: entities defined by the system under design

■ Blocked time

❑ A computation can be blocked because of

- contention for some needed resource.
- Unavailability of the resource
- Unavailability of the result of other components that the computation depends on

Approaches to Performance Tactics

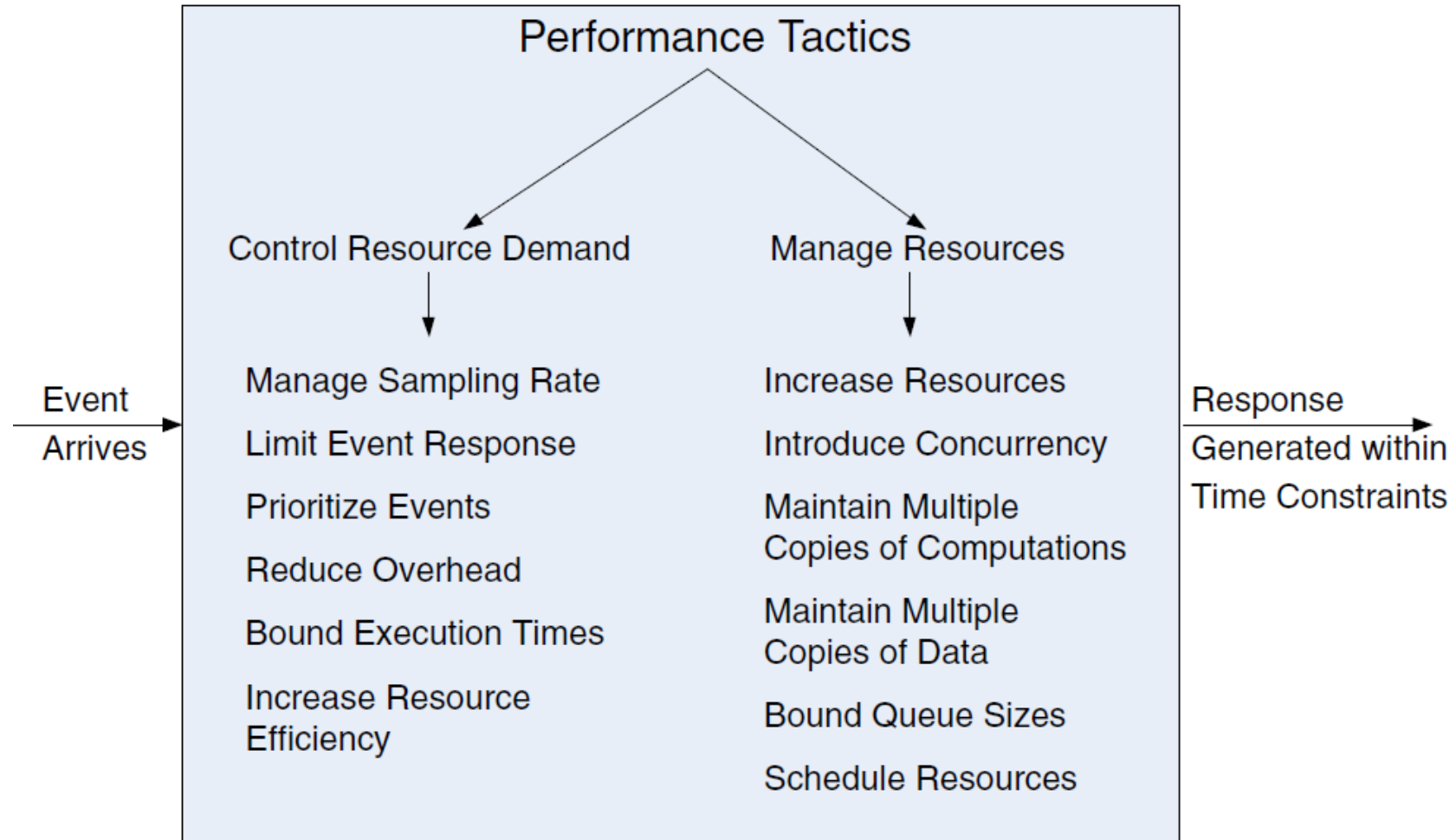
□ Control resource demand

- Produces smaller demand on the resources that will have to service the events. (demand side)

□ Manage resources

- Manages resources more effectively in handling demands. (response side)

Performance Tactics



Control Resource Demand

□ Manage sampling rate

- If it is possible to **reduce the sampling frequency** at which a stream of environmental data is captured, then **demand can be reduced**.
- The sampling frequency may be adjusted based on factors such as workload or resource availability
- maintaining **predictable levels of latency**
- **a balance between accuracy and efficiency** particularly in cases where continuous, high-frequency processing is not necessary.

Control Resource Demand

□ Limit event response

- If the arrived events reach the maximum queue size or processor utilization measure exceeds some warning measure, you may **limit processing events** only up to a set maximum rate, thereby **ensuring more predictable processing**.
- Possible policies for handing the situation in which some events are dropped
 - Logging the dropped events
 - Simply Ignoring them
 - Notifying to users or administrators

Control Resource Demand

❑ Prioritize events

- If not all events are equally important, you can impose a priority scheme that ranks events according to how important it is to service them.

❑ Reduce overhead

- Removing intermediaries or a chain of components
- Hosting cooperating components on the same processor to avoid the time delay of network communication
- a periodic cleanup of resources that have become inefficient (e.g., re-initialization of hash tables)

Control Resource Demand

□ **Bound execution times**

- Place a limit on how much execution time is used to respond to an event
 - For iterative, data-dependent algorithms, limiting the number of iterations is a method for bounding execution times.
- The cost is a less accurate computation

□ **Increase resource efficiency**

- Improving the algorithms used in critical areas will decrease latency

Manage Resources

□ Increase resources

- Faster processors, additional processors, additional memory, and faster networks all have the potential for reducing latency.
- Cost is usually a consideration when choosing resources

□ Introduce concurrency

- Processing different streams of events on different threads
- Creating additional threads to process different sets of activities
- Appropriate scheduling policies should be used

Manage Resources

□ Schedule resources

- Whenever there is contention for a resource, the resource must be **scheduled**. Processors are scheduled, buffers are scheduled, and networks are scheduled.
- Some common scheduling
 - First-in/First-out
 - Fixed-priority scheduling
 - Rate monotonic: assigns higher priority to processes with shorter periods
 - Dynamic priority scheduling
 - Earliest deadline first
 - Static scheduling
 - A cyclic executive schedule

Manage Resources

❑ Maintain multiple copies of computations

- Allocating replicas of computation to multiple servers can reduce the contention that would occur if all computations took place on a single server.
- A load balancer assigns new work to one of the available duplicate servers
 - ❑ criteria for assignment
 - round-robin
 - assigning the next request to the least busy server

Manage Resources

❑ Maintain multiple copies of data

■ Caching

- ❑ Keeping copies of data (possibly one a subset of the other) on storage with different access speeds.
 - E.g., Micro professor's cache, web browser' cache
- ❑ The choice of the data to be cached
 - Recently requested
 - Predicting based on the patterns of user behavior

■ Data replication

- ❑ keeping separate copies of the data to reduce the contention from multiple simultaneous accesses.
- ❑ The copies must be kept consistent and synchronized

Summary

- ❑ **Performance** can be improved by **reducing demand** or by **managing resources** more appropriately.
- ❑ **Reducing demand** will have the side effect of reducing fidelity or refusing to service some requests.
- ❑ **Managing resources** more appropriately can be done through scheduling, replication, or just increasing the resources available.

Architectural tactics

SAFETY TACTICS

안전성(Safety)

□ 기본 개념

- “안전성(Safety)”은 시스템의 잘못된 동작이나 장애로 인해 인명, 재산, 환경 등에 피해가 발생하지 않도록 하는 능력
- 잘못된 동작이나 장애를 일으키는 요인들
 - 누락 : 이벤트 발생의 실패
 - 오발 : 의도하지 않거나 원하지 않은 이벤트의 잘못된 발생
 - 타이밍 오류: 필요한 시점보다 이르게 혹은 느리게 동작 및 이벤트 발생
 - 시스템 값의 오류: 검출 가능한 잘못된 값, 검출하기 어려운 미묘한 잘못된 값
 - 순서오류: 연속적인 이벤트 도착의 순서가 잘못된 경우

안전성 일반 시나리오

부분	설명	가능한 값
자극원	시스템의 잘못된 동작이나 장애를 일으키는 요인의 주체	다음 중 하나 이상이 가능하다. • 센서 • 소프트웨어 컴포넌트 • 통신 채널 • 장치 (예, 시계)
자극	시스템의 잘못된 동작이나 장애를 일으키는 요인 (예, 생략, 실수, 타이밍 오류)	누락의 구체적인 사례 • 값이 도착하지 않음 • 기능이 수행되지 않음 오발의 구체적인 사례 • 기능이 잘못 수행됨 • 장치가 허위 이벤트를 생성함 타이밍 오류의 구체적인 사례 • 데이터가 너무 늦게나 너무 일찍 도착 • 이벤트가 잘못된 순서로 일어난다. 값의 오류에 대한 구체적인 사례 • 소프트웨어 컴포넌트가 잘못된 결과를 생성

안전성 일반 시나리오

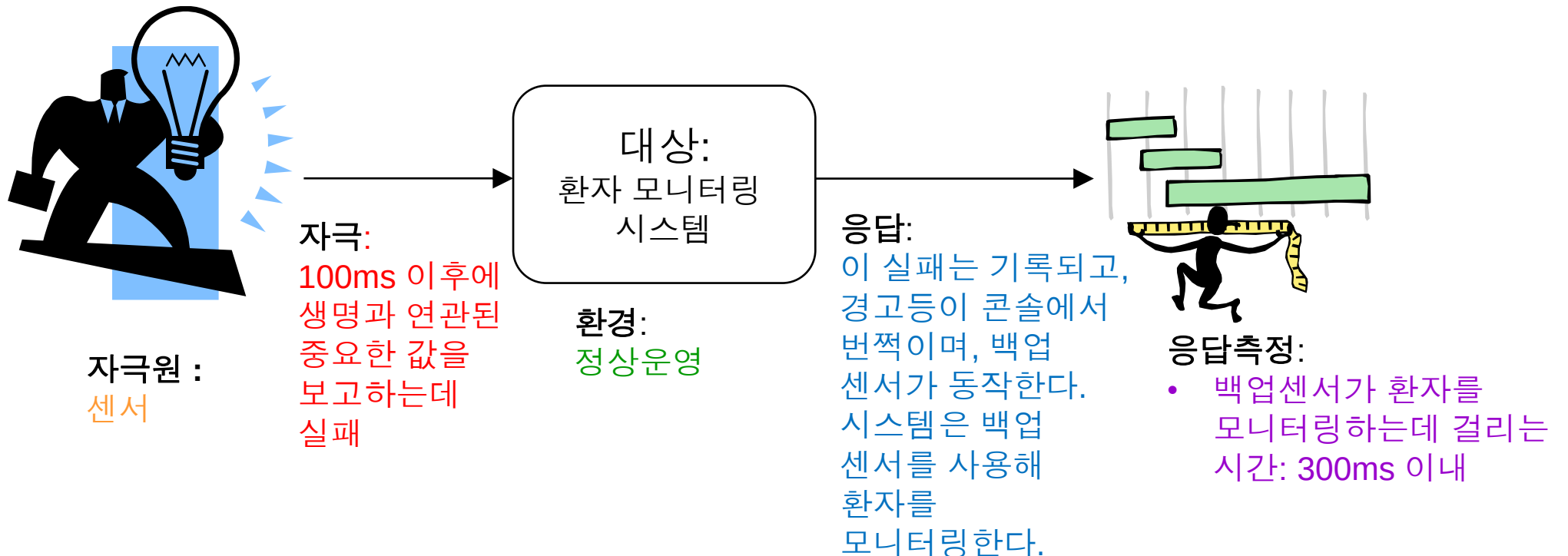
부분	설명	가능한 값
대상	시스템의 어떤 부분이 자극에 영향을 받는가?	• 전체 시스템 • 특정 컴포넌트 집합
환경	어떤 상태에서 자극이 발생하는가?	• 정상 운영 • 성능 저하 운영 • 수동 운영 • 복구 모드
응답	자극에 대한 반응	다음 중 하나 이상이 가능하다. • 안전하지 않은 상태를 회피한다. • 복구한다. • 저하 혹은 안전 모드로 진행한다. • 셧다운한다 • 수동 운전/백업시스템으로 전환한다. • 적절한 대상(사람 혹은 시스템)에 보고한다. • 안전하지 않은 상태를 기록한다.

안전성 일반 시나리오

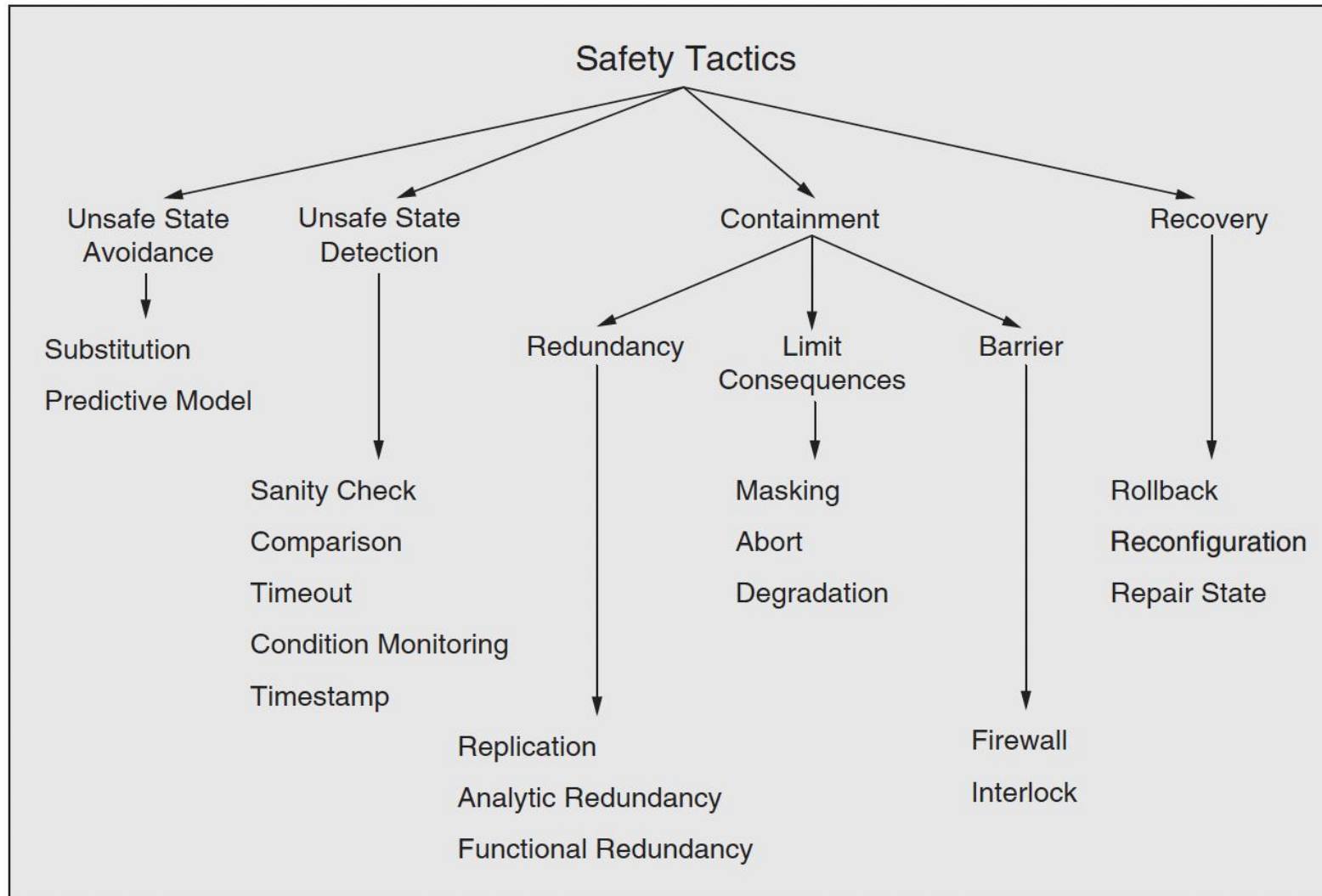
부분	설명	가능한 값
응답	자극에 대한 반응	다음 중 하나 이상이 가능하다. • 안전하지 않은 상태로의 진입을 회피한 횟수나 비율 • 안전하지 않은 상태 중에 시스템이 자동으로 회복 가능한 상태의 비율 • 시스템이 복구 가능한 시간 • 시스템이 저하 모드나 안전 모드로 동작하는 시간 • 수동 운영/안전모드/저하모드로 진입 후 복구될 때까지 걸린 시간

구체적인 안전성 시나리오

“환자 모니터링 시스템의 **센서**가 **100ms** 이후에 **생명과 연관된 중요한 값을 보고하는 데 실패한다**. 이 실패는 기록되고 경고등이 콘솔에서 번쩍이며 백업 센서가 동작한다. 시스템은 **300ms** 이내에 백업 센서를 사용해 환자를 모니터링 한다.”



안전성 택틱



Unsafe State Avoidance::Substitution

□ 기본 개념

- **대체(Substitution)** 택틱은 잠재적으로 위험한 소프트웨어 설계 기능을 하드웨어 기반 메커니즘으로 대체하여 잠재적 위험요소를 제거함.

□ 사례

- **Watchdog Timer, Monitor, Interlock** 등의 하드웨어 보호 장치는 소프트웨어 기반 보호장치에 비해서 자신의 리소스를 기반으로 동작하므로, 리소스를 확보 받지 못해서 발생하는 문제를 회피할 수 있음
→ 대개, 대체하는 기능이 상대적으로 단순한 경우에 효과가 있음

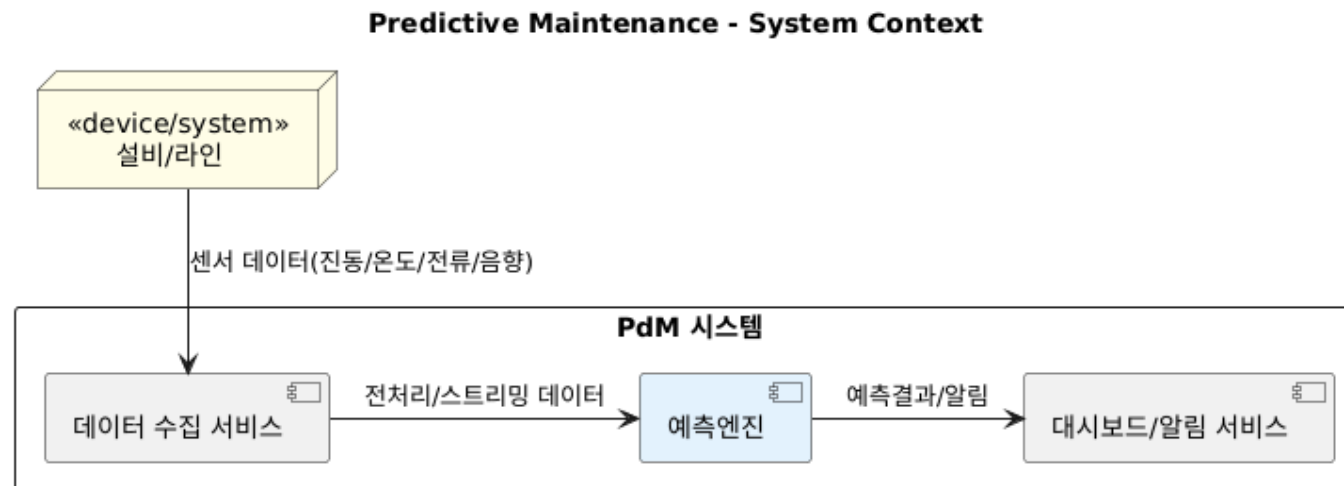
Unsafe State Avoidance::Predictive Model

□ 기본 개념

- 예측모델 (Predictive Model)은 다양한 동적 데이터(센서 값, 로그, 성능 메트릭 등)를 분석하여, 특정 조건이 임박한 미래의 위험 또는 고장 확률을 예측하는 데 사용
 - 주로, 상태 모니터링 (Condition Monitoring)과 결합

□ 사례

- 예측 정비



Unsafe State Detection 서버택틱

- **Sanity Checking** – 가용성 택틱과 동일
 - 특정 작업 결과나 컴포넌트의 입력 또는 출력의 유효성이나 합리성 확인
- **Comparison** – **Voting** 택틱과 관련
 - 동기화되거나 복제된 여러 요소가 생성한 출력 값을 비교하여 안전하지 못한 상태를 감지
- **Timeout** – **Heartbeat, Ping/Echo** 같은 가용성 택틱과 관련
 - 타이밍 제약사항이 만족되지 않음을 감지
- **Condition Monitoring** – 가용성 택틱과 동일
 - 위험한 행동으로 이어질 수 있는 시스템 상태 식별
- **Timestamp** – 가용성 택틱과 동일
 - 분산 메시지 전달 시스템에서 잘못된 이벤트 순서를 감지

Containment 택틱

- **Containment (억제)** 택틱은 이미 진입한 안전하지 못한 상태와 관련된 피해를 제한하는 것
- 서브 택틱
 - **Redundancy (다중화)**
 - 다중화된 컴포넌트를 통해 시스템의 동작을 지속 시킴
 - **Limit Consequences (영향 제한)**
 - 안전하지 못한 상태에 들어간 시스템이 초래할 수도 있는 나쁜 영향을 제한
 - **Barrier (장벽)**
 - 문제가 퍼지지 못하도록 예방 및 문제상황 완화

Containment::Redundancy 서브 택틱

	Replication	Functional Redundancy	Analytic Redundancy
기본 개념	동일 컴포넌트를 복수개 배치	동일한 기능을 수행하지만 서로 다른 설계, 기술, 혹은 알고리즘을 사용하여 구현	물리적인 하드웨어의 추가 없이 수학적 모델링을 사용하여 중복성을 확보
사례	3개의 동일 센서 중 하나가 이상 값을 출력하면, Voting 방식을 사용해 고장을 감지하고 오류를 무시	서로 다른 프로그래밍 언어로 작성된 2개의 소프트웨어 모듈이 동일한 계산을 수행하여 결과를 비교	시스템의 수학적 모델을 기반으로 센서 측정값을 예측하고, 이 예측 값과 실제 측정값 사이에 큰 차이가 발생하면 센서 고장으로 판단
장점	구현이 단순	설계 오류 방지에 효과적	물리적 비용 절감
단점	설계 오류에 취약	개발 비용이 높음	모델의 정확도에 의존

Containment::Limit Consequences 서브택틱

□ Abort (취소)

- 안전하지 못하다고 판단된 작업은 피해를 최소화 시키기 위해서 취소
- 사례
 - 로켓 발사 중 심각한 엔진 고장 발생 시 **발사 중단(Launch Abort)**하고 승무원 탈출 시스템을 작동. 공장 생산 라인에서 센서 오작동 감지 시 전체 라인 비상 정지

□ Degradation (성능저하)

- 일부 기능을 버리거나 대체함으로써 가장 중요한 시스템 기능들을 유지
- 사례
 - 자율 주행 차량에서 하나의 센서가 고장 났을 때, 자율 주행 레벨을 낮추고 운전자에게 제어를 요청

□ Masking (마스킹)

- 고장이 발생했음에도, 시스템이 고장을 자동으로 보정하고 무시하여 고장이 전혀 발생하지 않은 것처럼 정상적인 기능과 성능을 유지
- 사례
 - 시스템에서 세 개의 동일한 센서 중 하나의 오류가 발생할 때, 투표 메커니즘을 통해 오류를 마스킹

Containment::Barrier 서브 택틱

□ Firewall (방화벽)

- 시스템 내의 한 영역에서 발생한 오류(Fault)나 고장(Failure)이 다른 안전한 영역으로 확산되는 것을 방지하여 피해를 격리하는 방법
- 사례
 - 하나의 서브시스템(A)이 오류를 출력하더라도, 이 서브시스템의 출력이 다른 서브시스템(B)에 전달되기 전에 데이터 유효성을 검사하여(예: 범위 검증, 타임스탬프 검증) 잘못된 데이터를 차단하는 메커니즘

□ Interlock (인터락)

- 특정 조건이 충족되지 않으면 위험한 동작이나 명령의 실행을 물리적 또는 논리적으로 금지하는 안전 메커니즘
- 사례
 - 모터에 정방향 명령이 활성화된 상태에서는 역방향 명령이 들어와도 실행되지 않도록 제어 시스템에서 차단

Recovery 서브택틱

□ Rollback (롤백)

- 시스템을 최근에 알려진 안전한 상태로 되돌리는 방법
 1. 오류가 감지되면, 안전모드로 설정
 2. 최근에 알려진 안전한 상태 스냅샷을 기반으로 복원
 3. 롤백 후 제어상태/설정/모델/캐시 간의 일관성 검증
 4. 재가동

Recovery 서브택틱

□ Repair State (상태 수리)

- 시스템에서 오류 또는 고장이 발생한 후, 해당 시스템을 정상 작동 상태로 되돌리기 위한 일련의 활동

활동	설명
물리적 수리/교체	고장 난 디스크, 전원 장치, 모듈 등 하드웨어 구성 요소를 교체
데이터 재구성	RAID 시스템이나 분산 데이터베이스에서 고장 난 디스크로 인해 손실된 데이터를 다른 복제본을 기반으로 재구성(Reconstruction)
소프트웨어 패치	소프트웨어 버그가 고장의 원인이었을 경우, 해당 버그를 해결하는 패치를 적용
시스템 재동기화	고장/복구 과정에서 불일치해진 주 시스템과 예비 시스템(Active/Standby)의 상태(State)를 다시 일관성 있게 맞춤

Recovery 서브 택틱

□ Reconfiguration (재설정)

- 시스템의 일부가 고장나면, 해당 부분을 즉시 분리하고, 남은 리소스에 재연결하여 기능을 유지하게 하는 방식
- 장애 전환(Failover)
 - 이중화 구조에서 장애 컴포넌트를 대기중인 백업 컴포넌트로 전환
- 점진적 성능저하 (Graceful degradation)
 - 치명적이지 않는 기능을 비활성화하여 남은 자원을 핵심적인 기능에 집중

Architectural tactics

SECURITY TACTICS

Security

□ Security

- a measure of the system's ability to protect data and information from unauthorized access while still providing access to people and systems that are authorized

□ Three characteristics:

■ Confidentiality:

- The property that data or services are protected from unauthorized access

■ Integrity

- The property that data or services are not subject to unauthorized manipulation

■ Availability

- The property that the system will be available for legitimate use.

Security General Scenarios

□ Source

■ Individual or system

- that is correctly identified, identified incorrectly, of unknown identity
- who is internal/external, authorized/not authorized
- with access to limited resources, vast resources

□ Stimulus

■ Tries to

- display data, change/delete data, access system services, reduce availability to system services

Security General Scenarios

❑ Artifact

- system services
- data within system

❑ Environment

- Online or offline
- Connected or disconnected
- Firewalled or open to network
- Fully, partially, or not operational

Security General Scenarios

□ Response

■ Transactions are carried out in a fashion such that

- data or services are protected from unauthorized access
- data or services are not being manipulated without authorization
- parties to a transaction are identified with assurance
- the parties to the transaction cannot deny their involvements
- the data, resources, and system services will be available for legitimate use

■ The system tracks activities within it by

- Recording access or modification
- Recording attempts to access data, resources, or services
- Notifying appropriate entities (people or systems) when an apparent attack is occurring

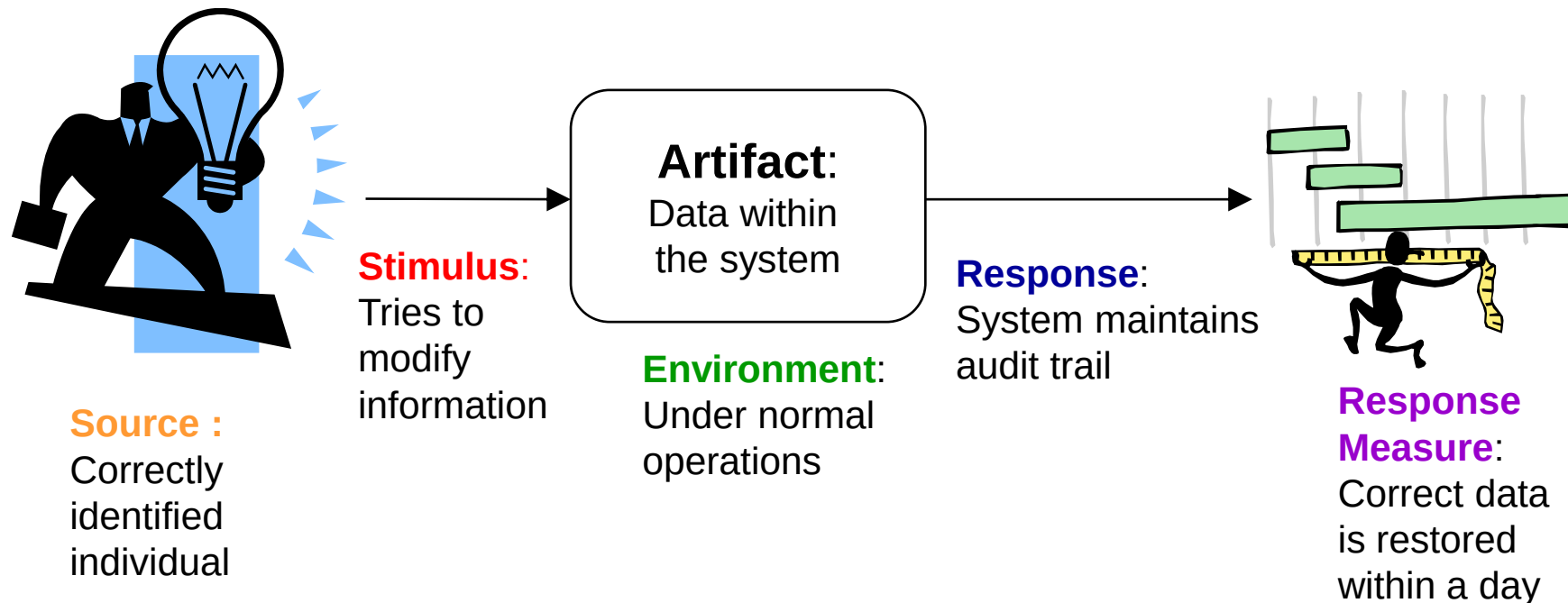
Security General Scenarios

□ **Response Measure**

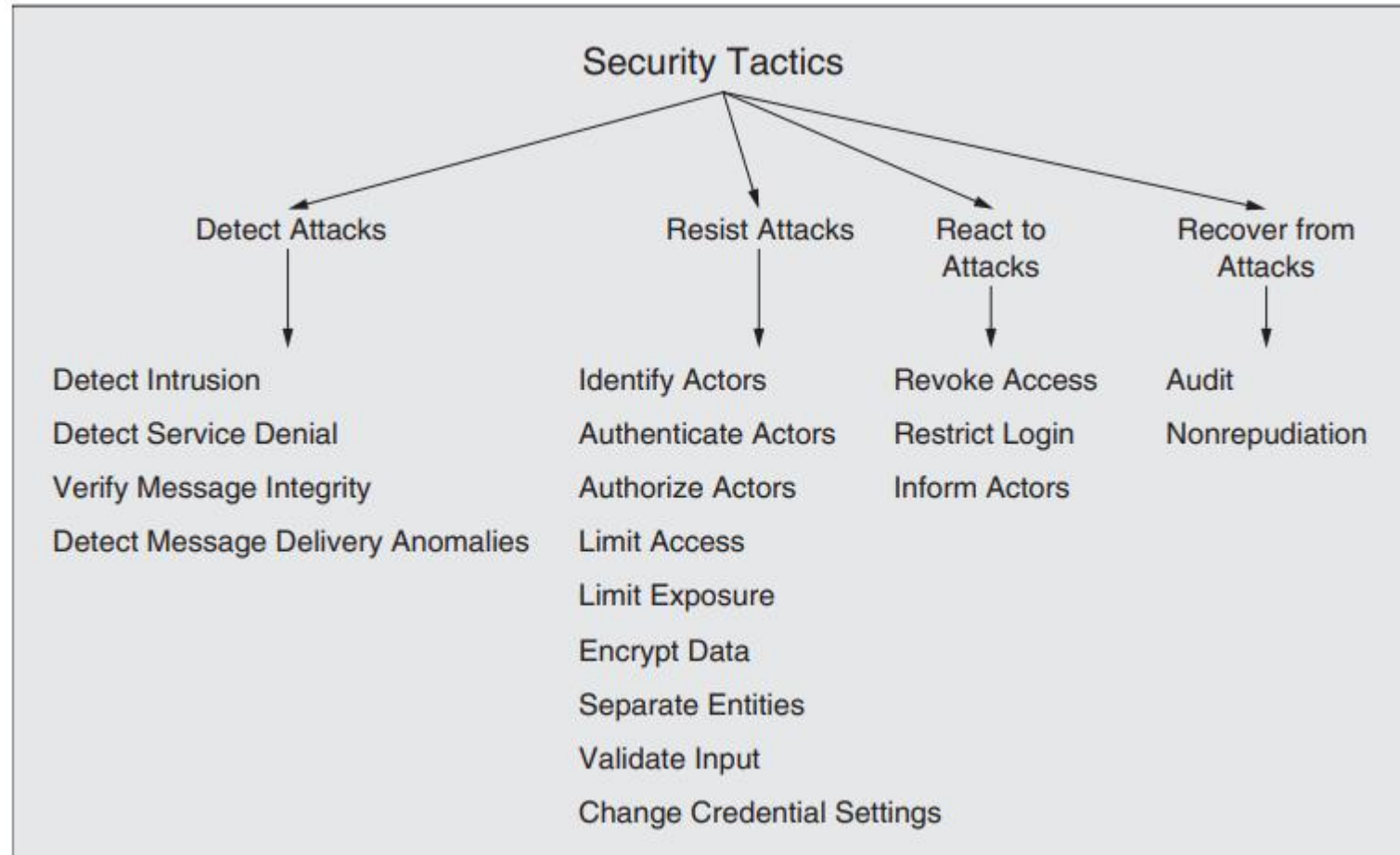
- how much of a system is damaged when a particular component or data value is corrupted
- how much time passed before an attack was detected
- How many attacks were resisted
- how long it took to recover from a successful attack
- how much data was vulnerable to a particular attack.

Concrete Security Scenario

“A correctly identified individual tries to modify system data from an external site; system maintains an audit trail and the correct data is restored within one day”



Security Tactics



Detect Attacks

❑ Detect intrusion

- The comparison of network traffic or service request patterns within a system to a set of signatures or known patterns of malicious behavior stored in a database.

❑ Detect service denial

- The comparison of the pattern or signature of network traffic coming into a system to historic profiles of known **denial-of-service attacks**.

❑ Verify message integrity

- This tactic employs techniques such as **checksums or hash values** to verify the integrity of messages or resource files

❑ Detect message delay

- This tactic **detects potential man-in-the-middle attacks**, by checking the time that it takes to deliver a message

Resist Attacks

- ❑ **A number of well-known means of resisting an attack**
 - **Identify actors**
 - **Authenticate actors**
 - **Authorize actors**
 - **Limit access**
 - **Limit exposure**
 - **Encrypt data**
 - **Separate entities**
 - **Validate Input**
 - **Change Credential settings**

Resist Attacks

□ Identify actors

- identifying the source of any external input to the system.
- Users are typically identified through user IDs. Other systems may be “identified” through access codes, IP addresses, protocols, ports, and so on.

□ Authenticate actors

- ensuring that an actor (a user or a remote computer) is actually who or what it purports to be.
- Passwords, one-time passwords, digital certificates, and biometric identification provide a means for authentication.

Resist Attacks

□ Authorize actors

- ensuring that **an authenticated actor has the rights to access** and modify either data or services.
- This mechanism is usually enabled **by providing some access control mechanisms** within a system.



Resist Attacks

□ Limit access

■ controlling access to resources

- E.g., DMZ (DeMilitarized Zone)

□ Limit exposure

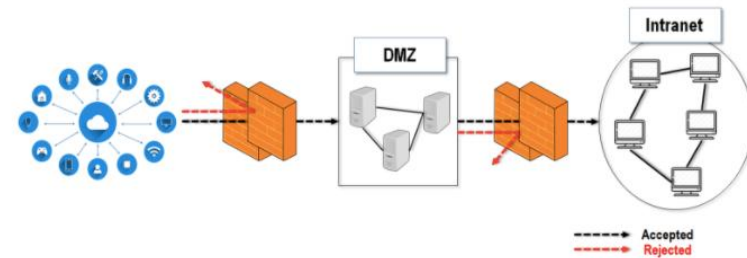
■ Reducing the probability of a successful attack, or restricting the amount of potential damage

- E.g., a decision to distribute servers amongst several geographically dispersed data centers

□ Separate entities

■ Separating different entities within the system can be done through physical separation on different servers.

- E.g., separating sensitive data from non-sensitive data can reduce the attack possibilities



Resist Attacks

❑ Encrypt data

- Encryption provides **extra protection** to persistently maintained data beyond that available from authorization.
- Communication links, on the other hand, may not have authorization controls. In such cases, **encryption is the only protection for passing data over publicly accessible communication links.**

❑ Validate input

- Cleaning and checking input is an important early line of defense in resisting attacks
 - ❑ Example, SQL Injection

❑ Change credential settings

- **Forcing the user to change default settings** will prevent attackers from gaining access to the system through settings that are, generally, publicly available

React to Attacks

❑ Revoke access

- If an attack is underway, then access can be severely limited to sensitive resources, even for normally legitimate users and uses

❑ Lock computer

- Many systems limit access from a particular computer if there are repeated failed attempts to access an account from that computer

❑ Inform actors

- The set of relevant actors must be notified when the system has detected an attack

Recover from attacks

- ❑ The set of availability tactics can be brought

- ❑ **Maintain Audit Trail**
 - An **audit trail** (also called **audit log**)
 - ❑ a **security-relevant chronological record** that provide documentary evidence of the sequence of activities that have affected at any time a specific operation, procedure, or event.
 - We may analyze audit trail to create better defenses in the future.

Summary

- ❑ **The goal of security tactics**
 - to **detect attacks**, **resist attacks**, **react to attacks**, or **prevent from attacks**.
- ❑ **The detect attacks category** consists of
 - Detect intrusion, Detect service denial, Verify message integrity, and Detect message delay
- ❑ **The resist attacks category** consists of
 - Identify actors, authenticate actors, authorize actors
 - Limit access, Limit exposure, Separate entities
 - encrypt data, change default settings
- ❑ **The react to attacks category** consists of
 - Revoke access, Lock computer, Inform actors
- ❑ Recovering from an attack involves many of the same tactics as availability.

Architectural tactics

TESTABILITY TACTICS

Testability

- **Testability** refers to

- The ease with which software can be made to demonstrate its faults through testing
- **the probability**, assuming that the software has at least one fault, that **it will fail on its next test execution**

- **The response measures** for testability deals with

- how effective the tests are in discovering faults
- how long it takes to perform the tests to some desired level of coverage

General Testability Scenarios

□ Source

- **Human** (unit testers, integration testers, system testers, or acceptance testers and end users)
- **Automated tester**

□ Stimulus

- A set of tests is executed due to
 - **the completion of a coding increment** such as a class layer or service,
 - **the completed integration of a subsystem**
 - **the complete implementation of the whole system**, or
 - **the delivery of the system** to the customer.

General Testability Scenarios

❑ Artifact

- The portion of the system being tested

❑ Environment

- Design time, development time, compile time, integration time, deployment time, run time

❑ Response

- execute test suite and capture results,
- capture activity that resulted in the fault,
- control and monitor the state of the system

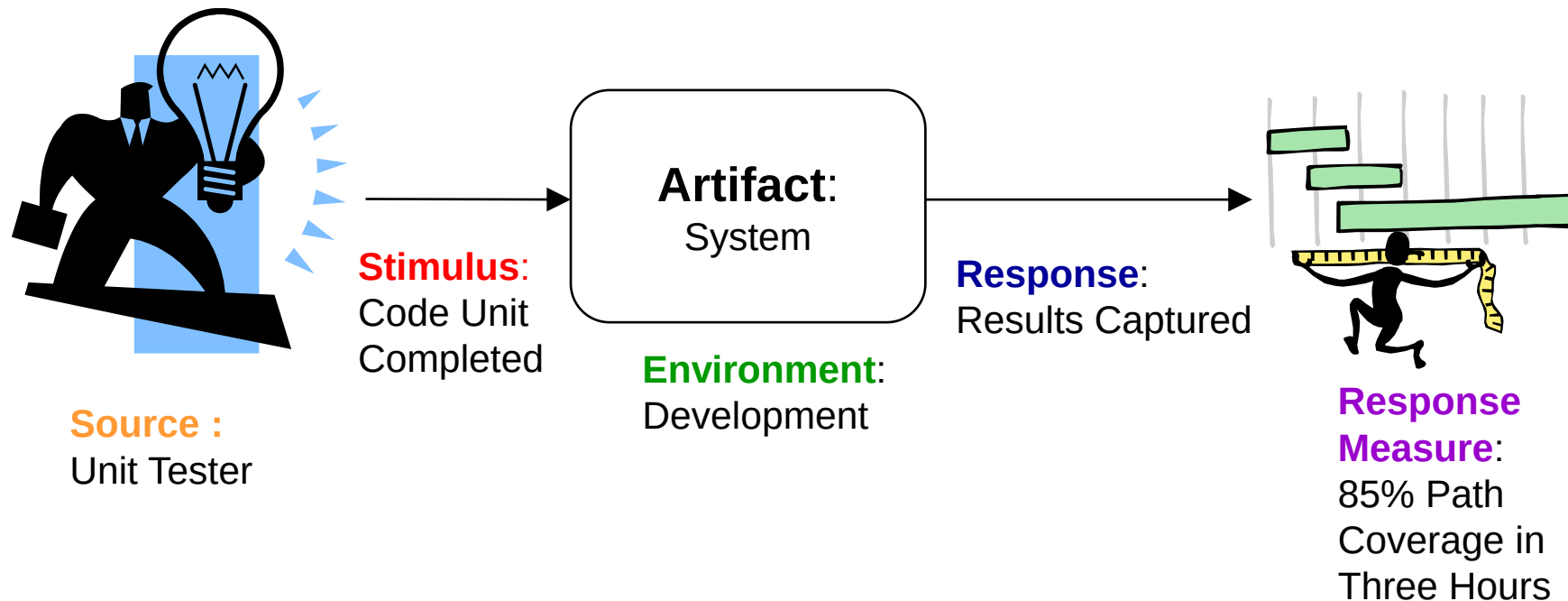
General Testability Scenarios

□ Response Measure

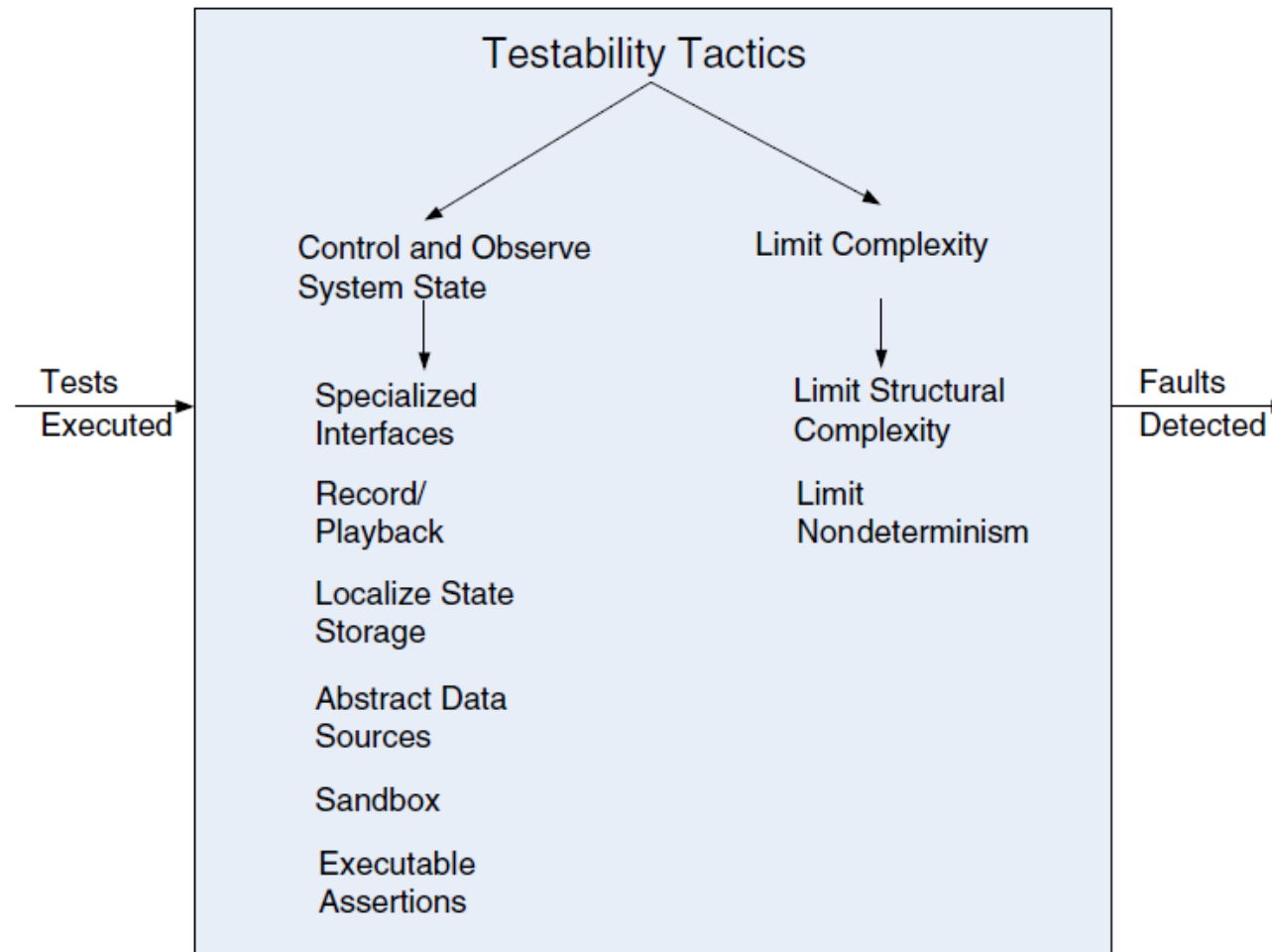
- effort to find a fault or class of faults
- effort to achieve a given percentage of state space coverage
- probability of fault being revealed by the next test
- time to perform tests
- effort to detect faults, length of longest dependency chain in test, length of time to prepare test environment
- reduction in risk exposure ($\text{size}(\text{loss}) \times \text{prob}(\text{loss})$)

Concrete Testability Scenario

“The unit tester completes a code unit during development, performs a test sequence whose results are captured and that gives 85% path coverage within three hours of testing”



Testability Tactics



Control and Observe System State

- ❑ **Control and observation are central to testability**
- ❑ **These tactics**
 - cause a component to maintain some sort of state information,
 - allow testers to assign a value to that state information, and/or
 - make that information accessible to testers on demand.
- ❑ **These tactics are the following:**
 - Specialized interfaces
 - Record/playback
 - Localize state storage
 - Abstract data sources
 - sandbox
 - Executable assertions

Control and Observe System State

❑ *Specialized interfaces*

- Specialized testing interfaces to control or capture variable values for a component
- Examples
 - ❑ A **set** and **get** method
 - ❑ A **report** method that returns the full state of the object
 - ❑ A **reset** method to set the internal state to a specified internal state
 - ❑ A method to turn on detailed output, various levels of event logging, or resource monitoring
- Specialized testing interfaces should be clearly identified or kept separate from the interfaces for required functionality, so that they can be removed if needed

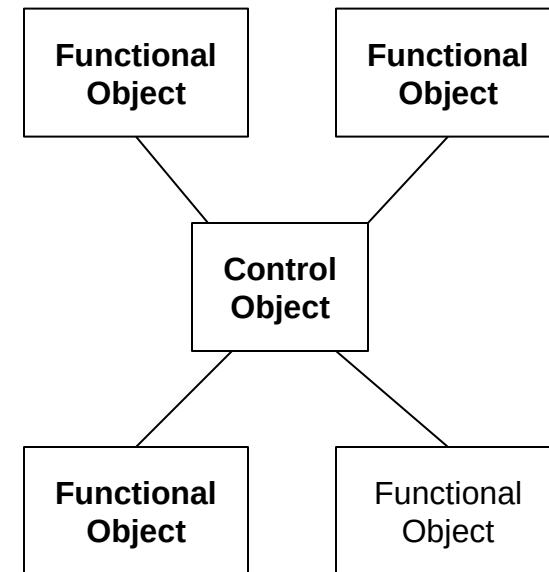
Control and Observe System State

□ *Record/playback*

- Recording the state when it crosses an interface allows that state to be **used to “play the system back” and to re-create the fault.**

□ *Localize state storage*

- To start a system, subsystem, or module in an arbitrary state for a test, it is most convenient if that state is stored in a single place.
- A convenient way to “externalize” state storage is to **use a state machine** (or state machine object) as the mechanism to track and report current state

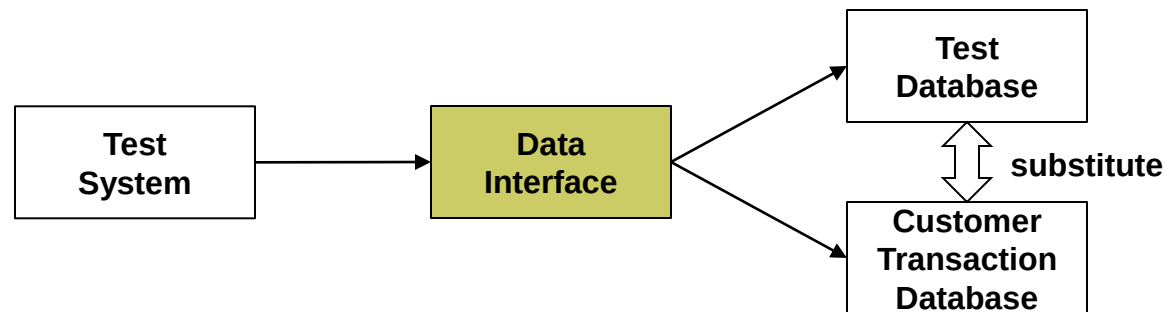


Control and Observe System State

□ *Abstract data sources*

■ **Abstracting interfaces** lets you substitute test data more easily.

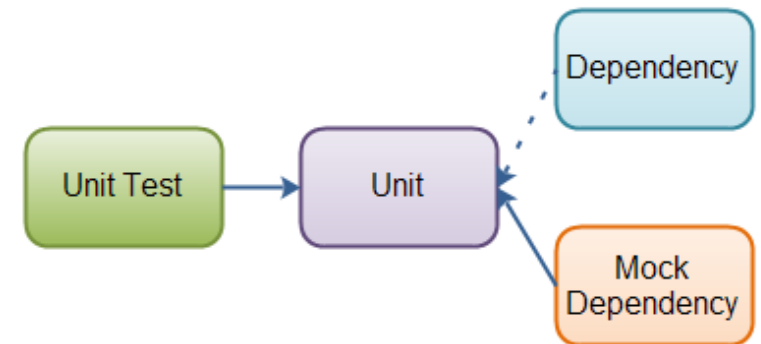
- For example, if you have a database of customer transactions, you could design your architecture so that it is easy to point your test system at other test databases, or possibly even to files of test data instead, without having to change your functional code



Control and Observe System State

□ **Sandbox**

- A **sandbox** is **an isolated testing environment** that enables users to run programs or execute files **without affecting the application**, system or platform on which they run
- A common form of sandboxing is **to virtualize resources**.
 - Example
 - Virtualizing system clock
 - Similar virtualizations could be done for other resources, such as memory, battery, network, and so on
 - Stubs, mocks, and dependency injection are simple but effective forms of virtualization



Control and Observe System State

□ Executable assertions

- Assertions are (usually) hand-coded and placed at desired locations **to check that data values satisfy specified constraints.**
- Assertions can be expressed as **pre- and post-conditions for each method** and also as **class-level invariants**
- E.g., JUnit

```
public class Calcurator {  
    public double sum(double a,  
double b){  
        return a + b;  
    }  
}
```

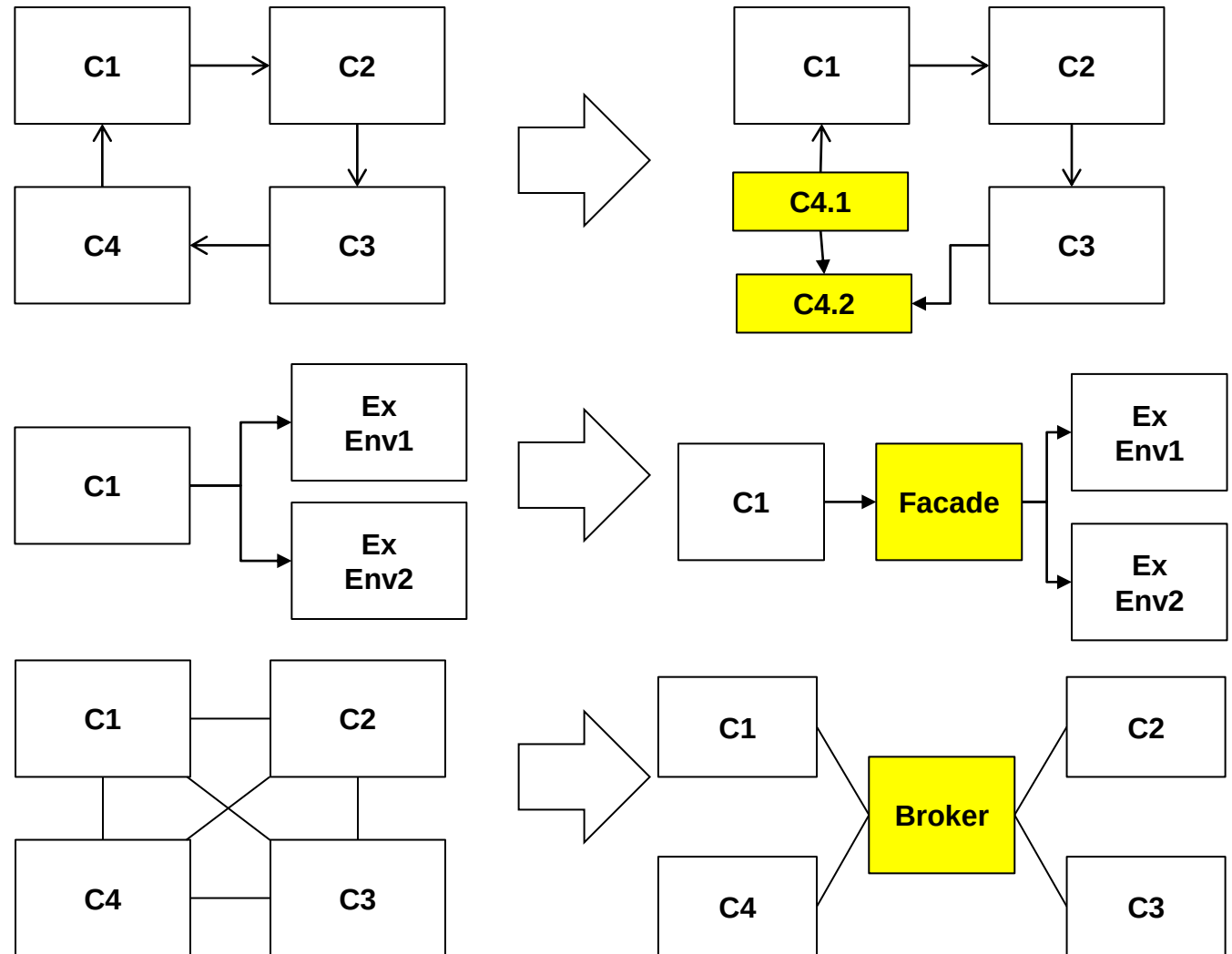
```
public class CaluratorTest {  
    @Test  
    public void testSum(){  
        Calcurator c = new Calcurator();  
        double result = c.sum(10, 50);  
        assertEquals(60, result, 0);  
    }  
}
```

Limit Complexity

- ❑ Complex software is harder to test
- ❑ Testing is about finding the fault that caused the failure so that it can be removed
 - we are concerned with **making behavior repeatable.**
- ❑ This category has two tactics:
 - Limit structural complexity
 - Limit nondeterminism

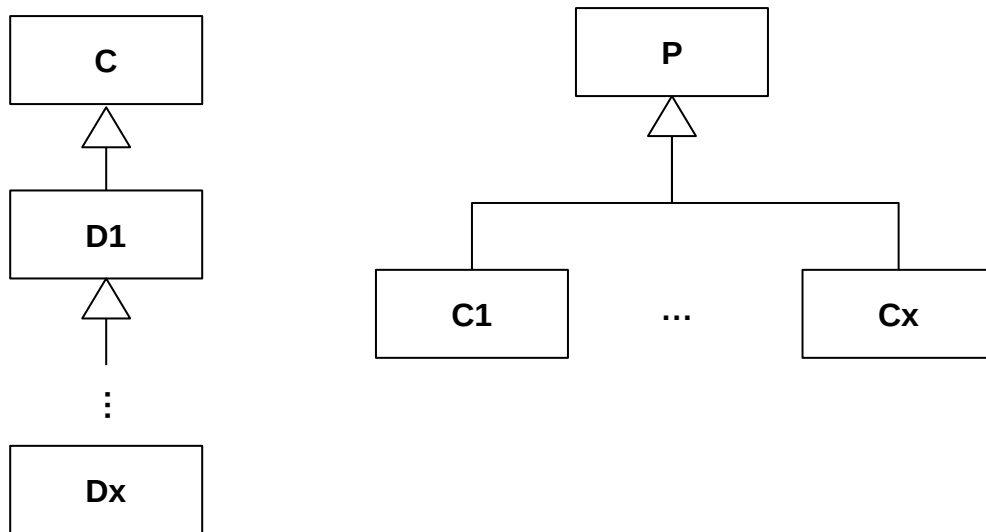
Limit Structural Complexity

- ❑ avoiding or resolving cyclic dependencies between components
- ❑ isolating and encapsulating dependencies on the external environment
- ❑ reducing dependencies between components in general

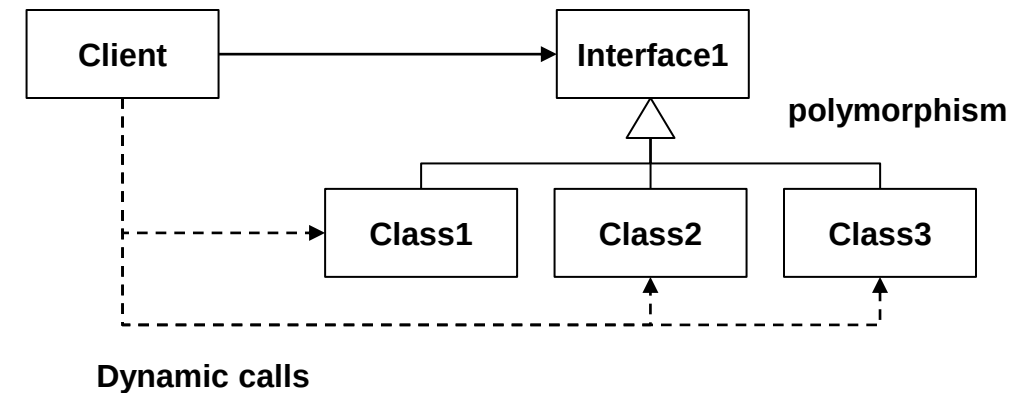


Limit Structural Complexity

- Limit the depth of the inheritance tree, and the number of children of a class



- Limit polymorphism and dynamic calls



- All modifiability tactics can also help with testability
 - having high cohesion, loose coupling, and separation of concerns

Limit Complexity

□ Limit nondeterminism

- This tactic involves **finding all the sources of nondeterminism**, such as unconstrained parallelism, and **removing them as much as possible**.
 - E.g.,
 - Randomness control with a seed value
 - Minimize the parts of concurrent execution
- Some sources of nondeterminism are unavoidable
 - E.g., in multithreaded systems that respond to unpredictable events
 - for such systems, other tactics (such as record/playback) are available.

Summary

- ❑ **Controlling and observing the system state is a major class of testability tactics.**
 - Specialized interfaces
 - Record/Playback
 - Localize State Storage
 - Abstract Data Sources
 - Sandbox
 - Executable Assertions
- ❑ **Complex systems are difficult to test. Consequently, keeping the system simple is another class of tactics that supports testability**

Architectural tactics

USABILITY TACTICS

Usability

□ Usability is concerned with

- **how easy it is** for the user to accomplish a desired task
- **the kind of user support** the system provides

□ Relevant issues

- **Learning system features**, e.g., help feature
- **Using a system efficiently**, e.g., suspend/resume feature
- **Minimizing the impact of errors**, e.g., cancel feature
- **Adapting the system to user needs**, e.g., automatic input
- **Increasing confidence and satisfaction**, e.g., progressive bar

General Usability Scenarios

- ❑ **Source**
 - **End user**
- ❑ **Stimulus**
 - **want to**
 - ❑ **Learn system features**
 - ❑ **use system efficiently**
 - ❑ **minimize impact of errors**
 - ❑ **adapt system**
 - ❑ **feel comfortable**
- ❑ **Artifact**
 - **System or the specific portion of the system**
- ❑ **Environment**
 - **At runtime or configure time**

General Usability Scenarios

□ Response

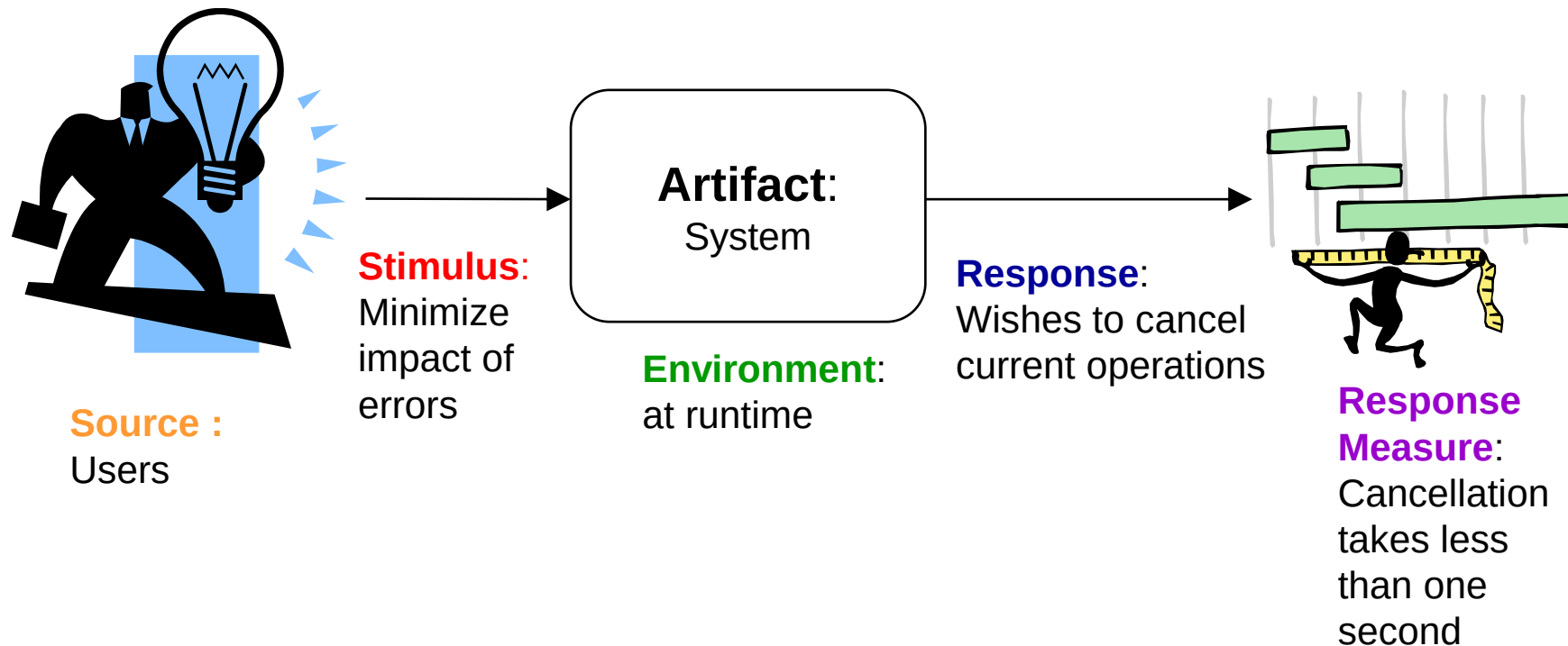
- The system should either provide the user with the features needed or anticipate the user's needs.

□ Response Measure

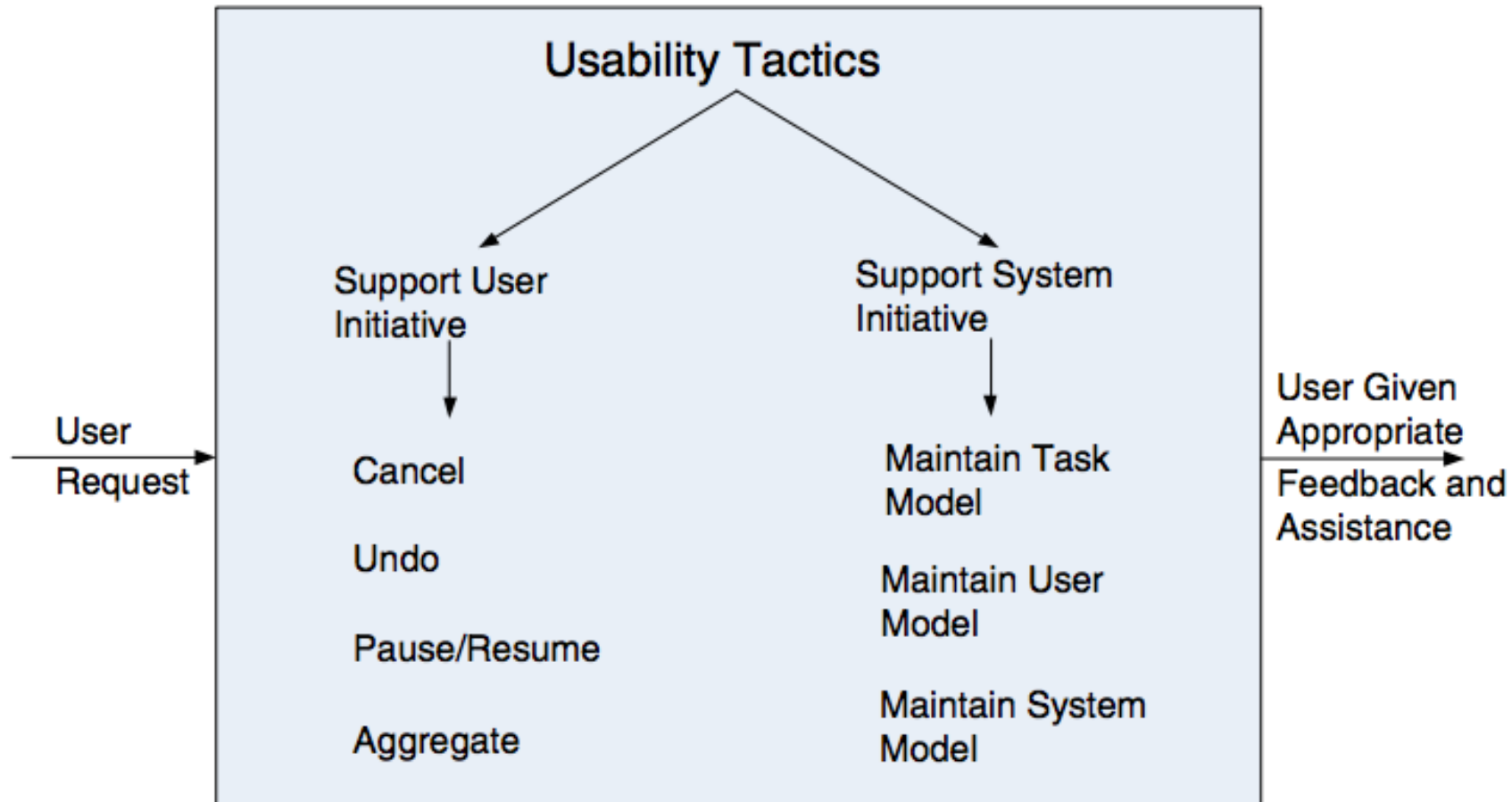
- Task time
- number of errors
- number of tasks accomplished
- user satisfaction
- gain of user knowledge
- ratio of successful operations to total operations
- amount of time/data lost when an error occurs

Concrete Usability Scenario

“A user wanting to minimize the impact of an error, wishes to cancel a system operation at runtime; cancellation takes place in less than one second”



Usability Tactics



Support User Initiative

- ❑ **Usability can be enhanced**
 - by allowing the user to make appropriate responses

- ❑ **Relevant tactics**
 - **to support the user in either correcting errors**
 - ❑ **Cancel**
 - ❑ **Undo**
 - **to support the user in being more efficient**
 - ❑ **Pause/resume**
 - ❑ **Aggregate**

Support User Initiative

❑ To support the user in either correcting errors

■ Cancel

- ❑ When the user issues a cancel command, the system must be listening for it;
- ❑ the command being canceled must be terminated;
- ❑ any resources being used by the canceled command must be freed
- ❑ Components that are collaborating with the canceled command must be informed

■ Undo

- ❑ To support the ability to undo, the system must maintain a sufficient amount of information about system state so that an earlier state may be restored

Support User Initiative

□ To support the user in being more efficient

■ Pause/resume

- When a user has initiated a long-running operation (e.g., downloading a large file from a server)
 - it is often **useful to provide the ability to pause and resume the operation**

■ Aggregate

- When a user is performing repetitive operations, or operations that affect a large number of objects in the same way,
 - it is **useful to provide the ability to aggregate the lower-level objects into a single group**

Support System Initiative

- ❑ **User support the system provides** for usability rely on
 - a model of the user,
 - the task being undertaken by the user
 - the system state itself
- ❑ The *support system initiative* tactics **maintain and use models to predict** either the system's behavior or the user's intention.
 - Maintain task model
 - Maintain user model
 - Maintain system model

Support System Initiative

□ Maintain task model

- The **task model** is used to determine the context of a user (what the user is attempting) and provide assistance.
 - E.g, correcting a lowercase letter in the position that sentences start.

□ Maintain user model

- The **user model** explicitly represents the user's knowledge of the system and the user's behavior.
 - E.g., a user model can control the speed of mouse selection so that not all of the document is selected

Support System Initiative

□ Maintain system model

- The **system model** is used to determine expected system behavior so that appropriate feedback can be given to the user
 - E.g., A progress bar indicating the time needed to complete the current activity

Connection with Modifiability

- ❑ To improve the usability of the system, **the user interface needs to be quickly and efficiently changed.**
- ❑ Tactics for modifiability support this goal
 - Increase semantic coherence, encapsulate, and co-locate related responsibilities, which localize user interface responsibilities to a single place
 - Restrict dependencies, which minimizes the ripple effect to other software when the user interface changes
 - Defer binding, which lets you make critical user interface choices without having to recode

Summary

- ❑ **Usability can be achieved by two types of tactics: support user initiative and support system initiative.**
- ❑ **Support User Initiative tactics**
 - Cancel, Undo, Pause/Resume, Aggregate.
 - ❑ These tactics allow the user to use a system efficiently and with minimal impact on errors.
- ❑ **Support System Initiative tactics**
 - **Maintain task model, Maintain user model, Maintain system model**
 - ❑ These tactics keep an explicit model of the user, the system, and the task to predict user or system responses.

Architectural tactics

TACTICS AND ARCHITECTURAL PATTERNS

Tactics and Architectural Patterns

- ❑ Tactics and architectural patterns are **ways of capturing proven good design solutions**, so that they can be reused
- ❑ Architecture patterns
 - a class of architectures (an architecture style)
 - a package of design decisions that is found repeatedly in practice
- ❑ Tactics
 - **foundational building blocks of design**, from which architectural patterns or styles are created.

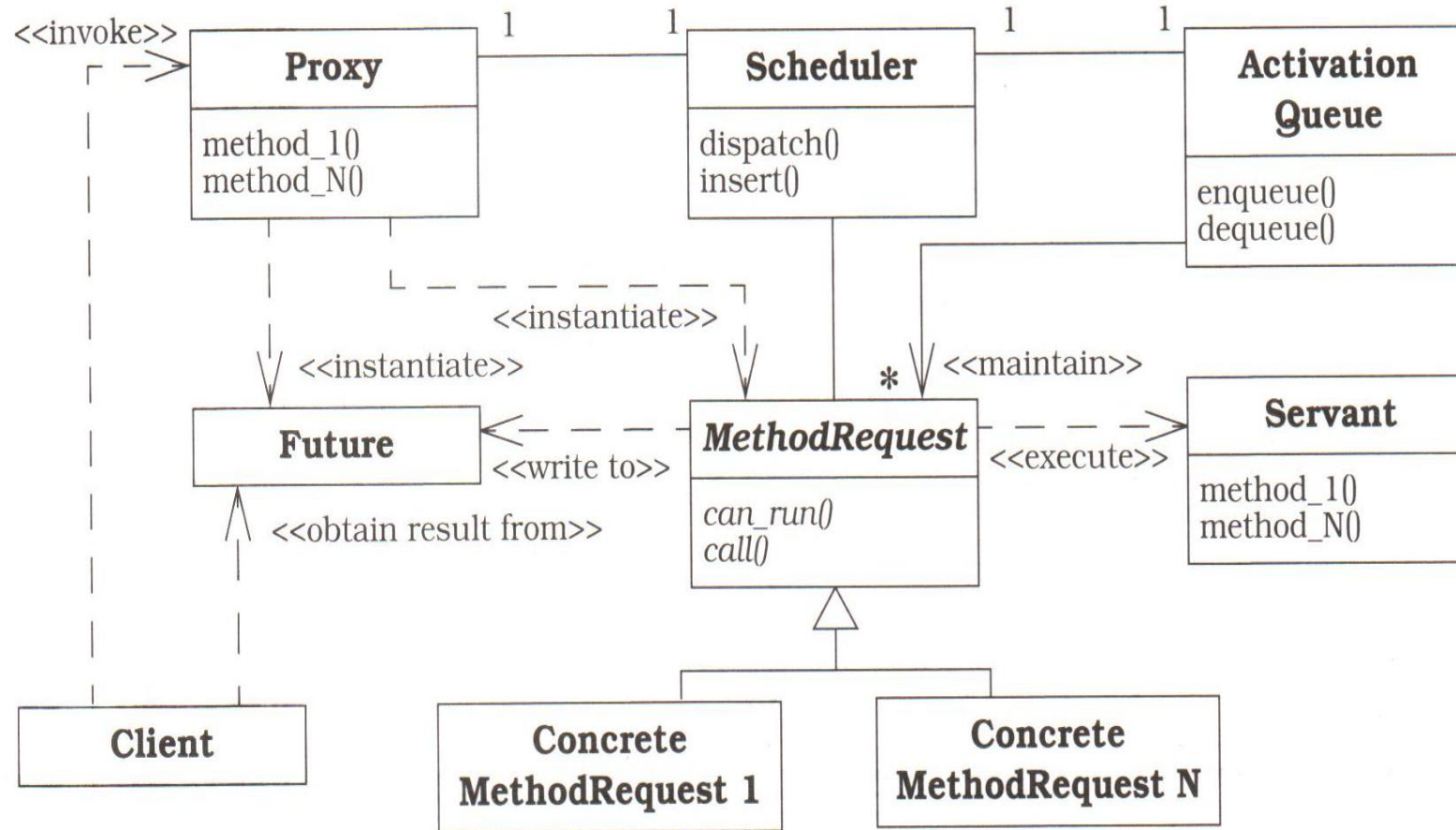
Tactics in the Layered Style

- ❑ **The layered style can be seen as the amalgam of several tactics:**
 - **Increase semantic coherence**
 - ❑ A layer's responsibilities all should work together without excessive reliance on other layers.
 - ❑ This goal can be achieved by choosing responsibilities that have semantic coherence
 - **Restrict dependencies**
 - ❑ Layers define an ordering and only allow a layer to use the services of its adjacent lower layer.
 - ❑ This limitation has a great influence on the dependencies between the layers and makes it much easier to limit the side effects of replacing a layer

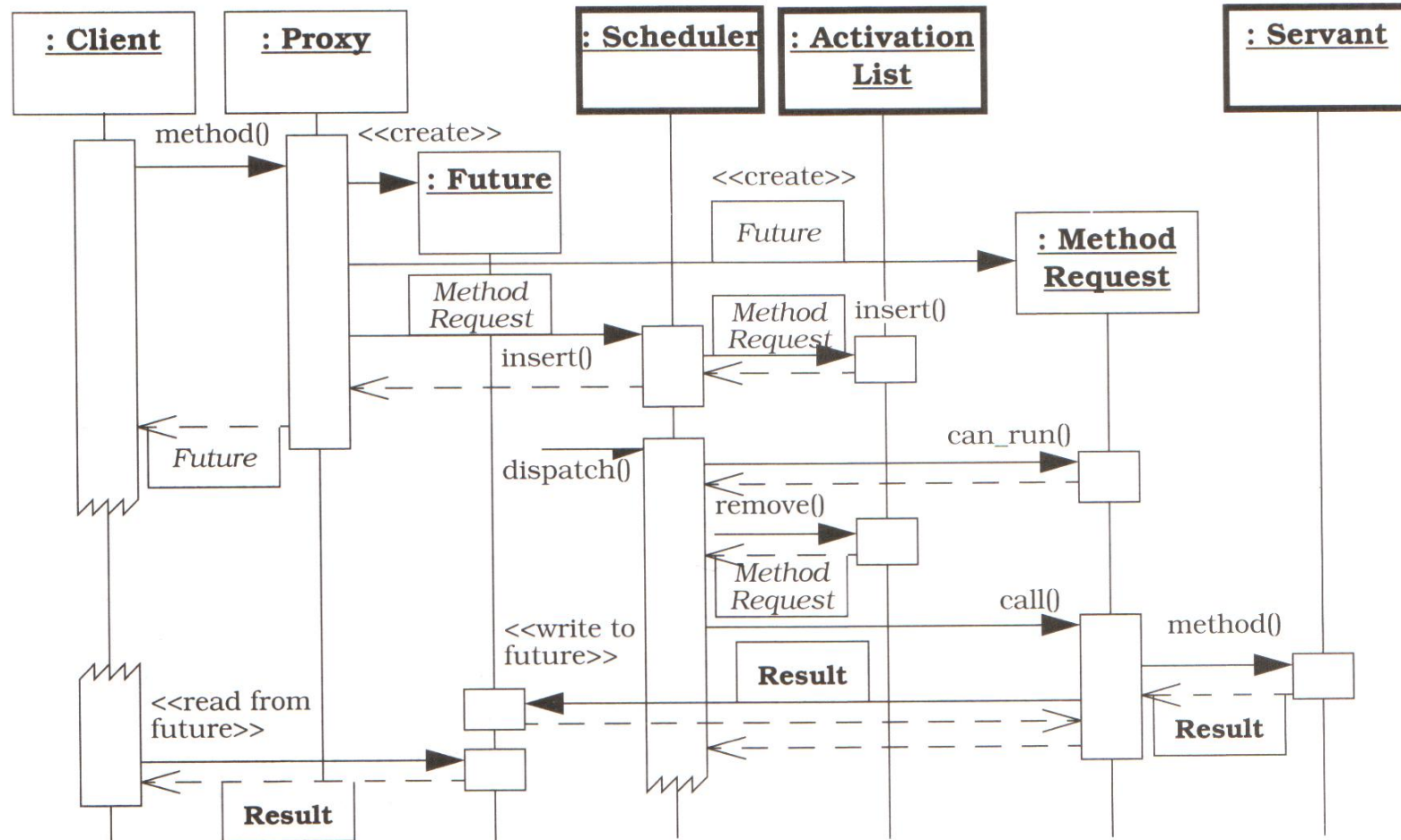
Tactics in the Active Object Pattern

- ❑ The *Active Object* [SSR+00] design pattern **decouples method execution from method invocation** to enhance concurrency and simplify synchronized access to objects that reside in their own threads of control.
 - This pattern consists of six elements:
 - A **proxy** provides an interface that allows clients to invoke publicly accessible methods on an active object.
 - A **method request** defines an interface for executing the methods of an active object.
 - An **activation queue** maintains a buffer of pending method requests.
 - A **scheduler** decides which method request to execute next.
 - A **servant** defines the behavior state modeled as an active object.
 - A **future** allows the client to obtain the result of the method invocation

Active Object Pattern



Active Object Pattern



Tactics in the Active Object Pattern

- ❑ The main purpose of *Active Object* [SSR+00] design pattern is to implement the “**introduce concurrency**” tactic to enhance performance. This pattern, however, involves other tactics as well.
 - **Encapsulate** (modifiability) – Each element hides the achievement details of its responsibilities behind an interface.
 - **Use an intermediary** (modifiability) – The proxy acts as an intermediary that will buffer changes to the method invocation.
 - **Defer binding time** (modifiability) – The binding of a client to the proxy is left open in terms of binding time.
 - **Schedule resources** (performance) – The scheduler implements some scheduling policy.

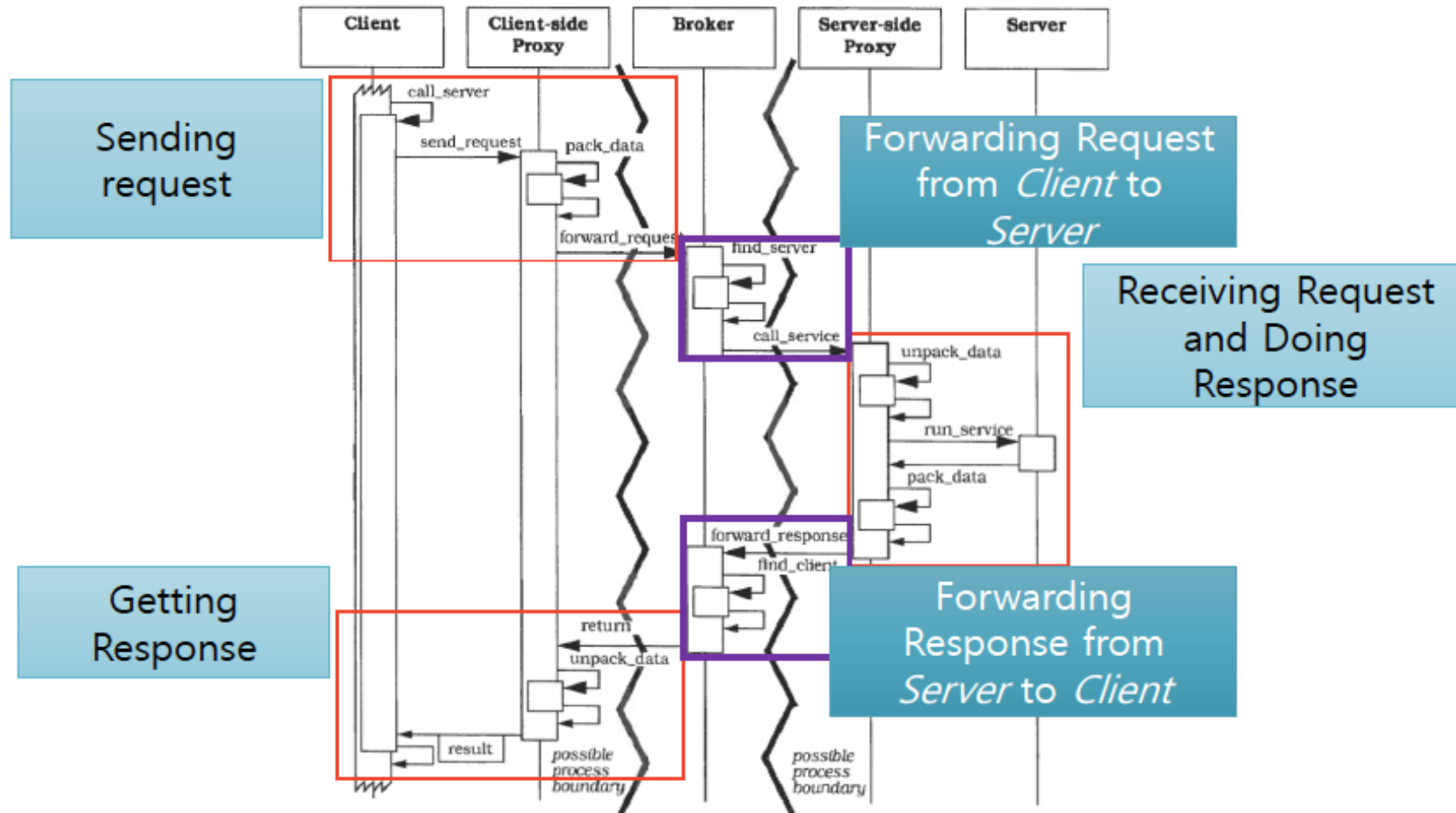
Architectural Patterns and Corresponding Tactics

Pattern	Modifiability									
	Increase Cohesion		Reduce Coupling				Defer Binding Time			
	Increase Semantic Coherence	Abstract Common Services	Encapsulate	Use a Wrapper	Restrict Comm. paths	Use an Intermediary	Raise the Abstraction Level	Use Runtime Registration	Use Startup-Time Binding	Use Runtime Binding
Layered	X	X	X		X	X	X			
Pipes and Filters	X		X		X	X			X	
Blackboard	X	X			X	X	X	X		X
Broker	X	X	X		X	X	X	X		
Model View Controller	X		X			X				X
Presentation Abstraction Control	X		X			X	X			
Microkernel	X	X	X		X	X				
Reflection	X		X							

Using Tactics to Augment Patterns

- ❑ **Architectural styles/patterns** promote some quality attributes and inhibit others
- ❑ Once a pattern (or a style) has (have) been selected, many decisions still remain
- ❑ **Broker Pattern**
 - a client requesting some information from a server does not need to know the location or APIs of the server.
 - The client simply contacts the broker (typically through a client-side proxy);

Dynamics of the Broker Pattern



Weakness of the Broker Pattern

❑ Availability

- **The broker is a single point of failure.** The liveness of servers, the broker, and perhaps even the clients need to be monitored, and repair mechanisms must be provided.

❑ Performance

- The levels of indirection between the client (requesting the information or service) and the server (providing the information or service) add overhead, and hence add latency.
- Also, the broker is a potential performance bottleneck if direct communication between the client and server is not desired (for example, for security reasons).

Improving the Broker pattern with Tactics

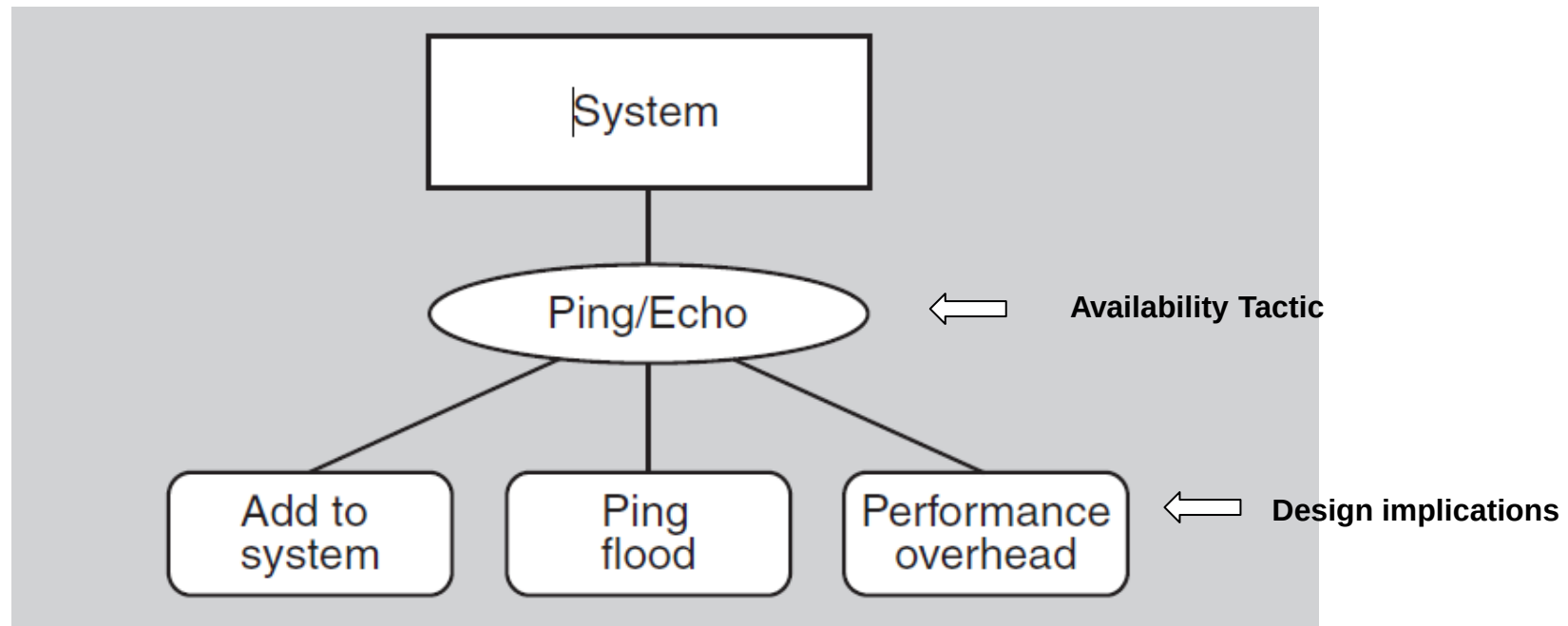
- ❑ The **maintain multiple copies** tactic would lead to multiple brokers, to help with performance and availability
- ❑ Load balancing (an application of the **scheduling resources** tactic) would ensure that one broker is not overloaded while another one sits idle.
- ❑ **Heartbeat, exception detection, or ping/echo** would give the replicated brokers a way of notifying clients and notifying each other when one of them is out of service, as a means of detecting faults.

Applying Tactics in Architectural Design

- ❑ **Tactics are design primitives aimed at managing a single quality attribute response**
- ❑ **Every tactic has its side effects and tradeoffs.**
- ❑ **Before applying a tactic to architectural design, architects should take into account the design implications (side effects and tradeoffs) related to the tactic**

Example of Applying Tactics

- Consider a system that needs to detect faults in its components.



Modifiability: How to add ping/echo to the existing architecture?

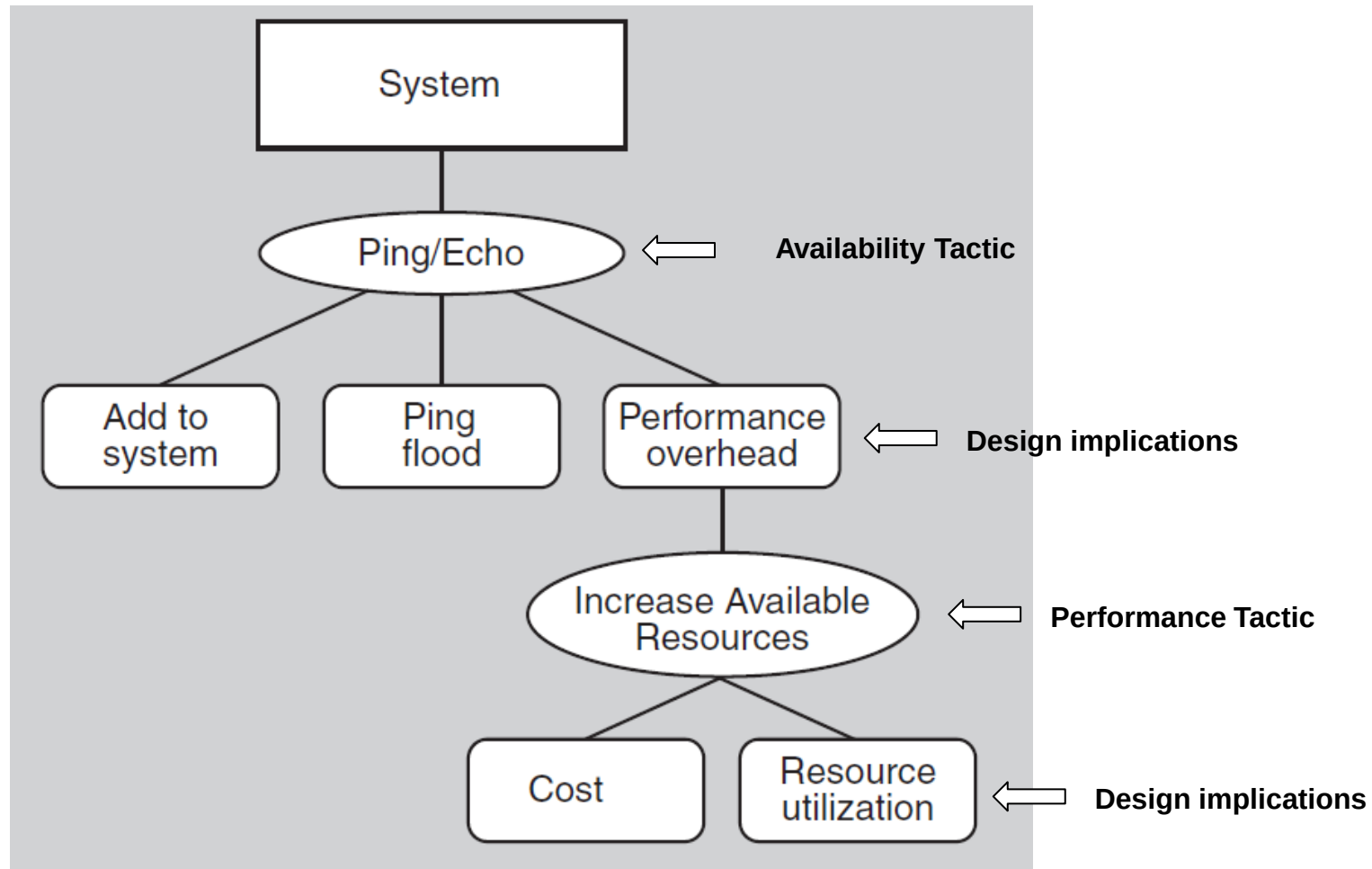
Security: How to prevent a ping flood attack?

Performance: How to ensure that the performance overhead of ping/echo is small?

Example of Applying Tactics

- ❑ Suppose the architect determines that the **performance tradeoff** (the overhead of adding ping/echo to the system) **is the most severe**.
- ❑ A tactic to address the performance side effect is *increase available resources*.
- ❑ Design implications associated with the tactic are these:
 - **Cost**. Increased resources cost more.
 - **Performance**. How to utilize the increased resources efficiently?

Example of Applying Tactics

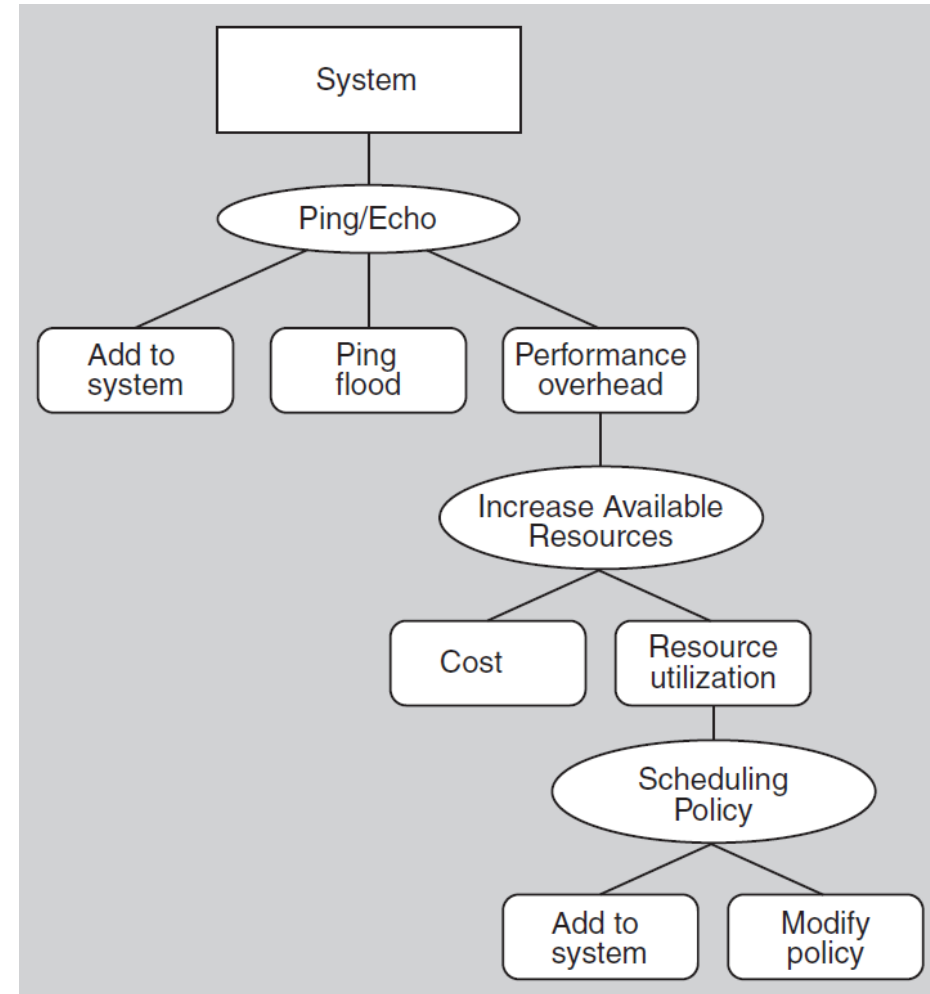


Example of Applying Tactics

- ❑ Now the architect decided to deal with the **resource utilization** issue further to address the performance side effect.
- ❑ A tactic to address the efficient use of resources is *scheduling policy*.
- ❑ Design implications associated with this tactic are these:
 - **Modifiability**. How to add the scheduling policy to the existing architecture?
 - **Modifiability**. How to change the scheduling policy in the future?

Example of Applying Tactics

- Applying successive tactics is like moving through a game space, and is a little like chess
 - the consequences of the next move are important, and looking several moves ahead is helpful



Summary

- ❑ Architects realize particular quality attributes using tactics
- ❑ Architectural tactics are design options architects to choose to satisfy quality attributes.
- ❑ Architectural patterns are packages of tactics
- ❑ We can make a pattern more specific to our problem by augmenting it with tactics.

Architecture Design



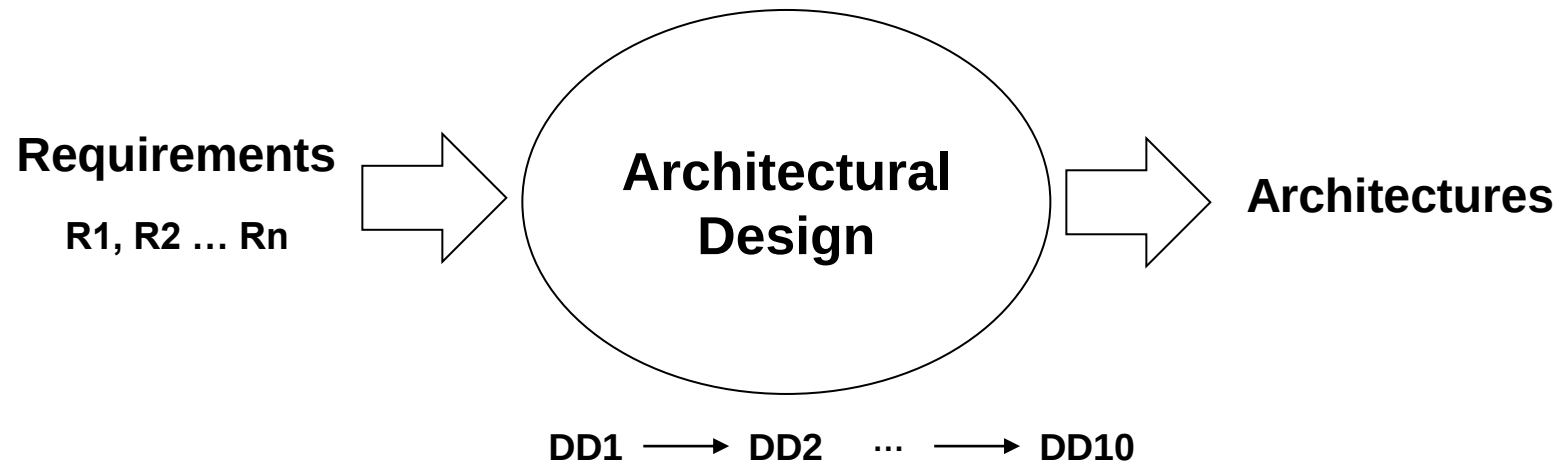
Architectural Design with Tactics and Patterns

Topics

- ❑ **What are architectural design**
- ❑ **Architectural Drivers**
- ❑ **Design Concepts**
- ❑ **Attribute-Driven Design (ADD)**
- ❑ **Case Study: FCAPS system**

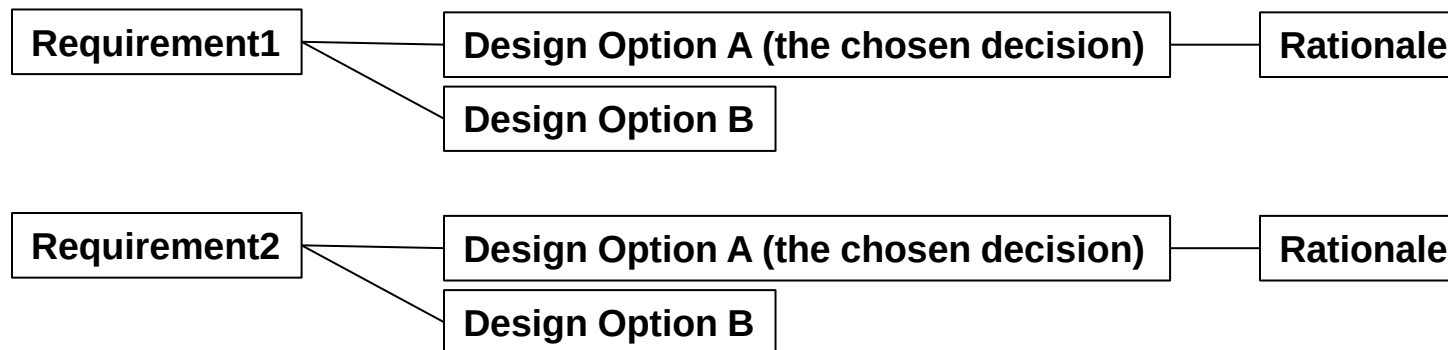
What are architectural design?

- **Architectural design** is the activity of creating an architecture by making series of design decisions that satisfy the given requirements.



What are architectural design?

- ❑ **Architectural design should be performed in a systematic way**
 - Design decisions are made with respect to the requirements
 - Possible design options should be explored and evaluated before making decisions.
 - Design decisions are recorded and justified



Architectural Drivers

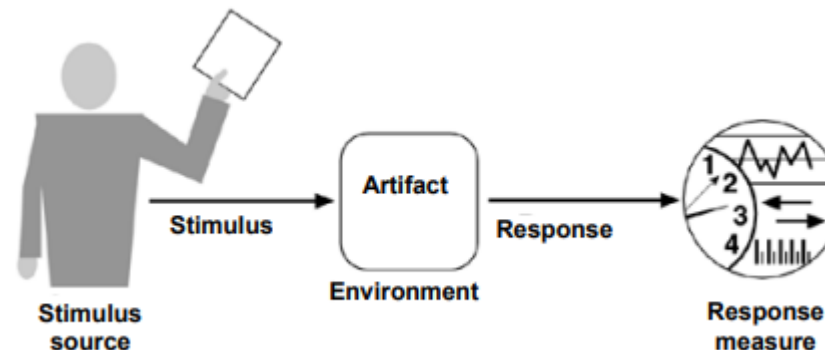
- ❑ **Architectural drivers** are requirements that will shape the software architecture.
 - **Primary Functionality**
 - ❑ Functionality that is critical to achieve the business goals
 - **Constraints**
 - ❑ Design decisions already made for the designers
 - **Quality attributes**
 - ❑ Characteristics that the system must possess in addition to functionality
- ❑ **Other drivers include**
 - **Design Purpose**
 - ❑ The purpose of the design that you want to achieve
 - **Architectural Concerns**
 - ❑ additional aspects that need to be considered

Primary Functionality

- ❑ **Criteria for primary functionality**
 - directly supports the business goals
 - Implies a high level of technical difficulty
 - requires the interaction of many architectural elements
- ❑ As a rule of thumb, approximately 10 percent of your use cases or user stories are likely to be primary.
- ❑ How the rest of functionality should be allocated to module is decided based on the decisions regarding the allocation of primary functionality.

Quality Attributes

- ❑ Among the drivers, quality attributes are the ones that shape the architecture the most significantly.
- ❑ Quality attributes are measurable characteristics of interest to users and developers
 - Performance, Availability, Modifiability, etc.
 - Can be specified using the scenario technique



Constraints

- ❑ **Constraints are limitations or restrictions**
 - They may be technical or nontechnical
 - ❑ E.g., technical - the use of open source technologies
 - ❑ E.g., nontechnical – it must be delivered by December 15.
 - They may originate from the customer but also from the development organization
- ❑ To design the best system that you can, despite the constraints you face. **Sometimes you might be able to argue for loosening a constraint**, but in most cases you have no choice but to design around the constraints.

Design Purpose

- ❑ Design purpose will change what and how you design, e.g.
 - You may be doing architecture design **as part of a project proposal**.
 - ❑ Such an architecture would not be very detailed
 - You may be doing architecture design **as part of the process of creating an exploratory prototype**.
 - ❑ the purpose of such an architecture is not so much to create a releasable or reusable system, but rather to explore the domain, to explore new technology
 - You may be **designing your architecture during development**.
 - ❑ In this case, the purpose is to do enough design work to satisfy requirements, guide system construction and work assignments, and prepare for an eventual release

Architectural Concerns

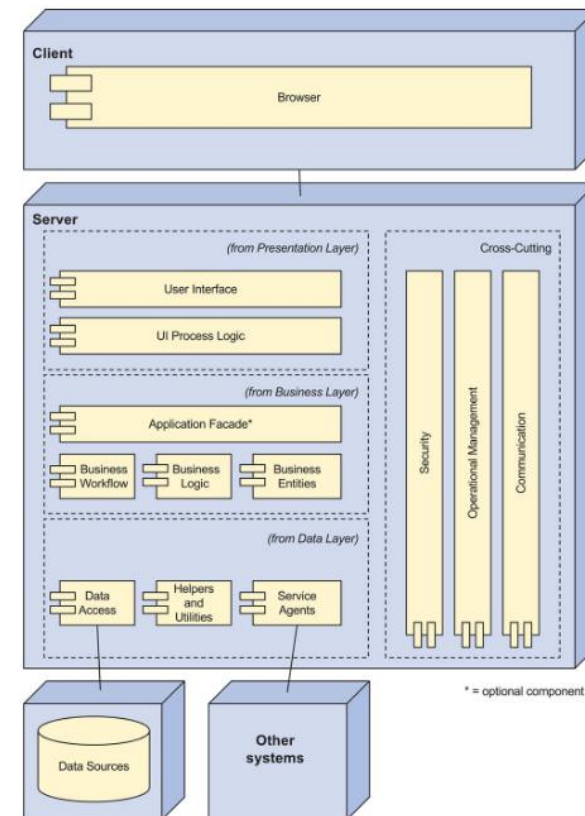
- ❑ **Architectural concerns** encompass additional aspects that need to be considered as part of architectural design but that are not expressed as traditional requirements.
 - Creating an overall logical and physical structure
 - Input validation
 - Exception management and logging
 - Communications
 - Deployment and updating
 - Data migration and backup
 - Organization of the codebase

Design Concepts

- ❑ Most sub-problems that are addressed during an iteration can be solved using existing solutions, i.e. **design concepts**
 - It is better (and faster) to use a proven solution to a problem for which we may not be experts
 - Creativity in design involves identifying, adapting and combining them
- ❑ There are several **categories of design concepts**, some are abstract and some more concrete. These are:
 - Reference Architectures
 - Deployment Patterns
 - Architectural / Design Patterns
 - Tactics
 - Externally developed components (e.g. Frameworks)

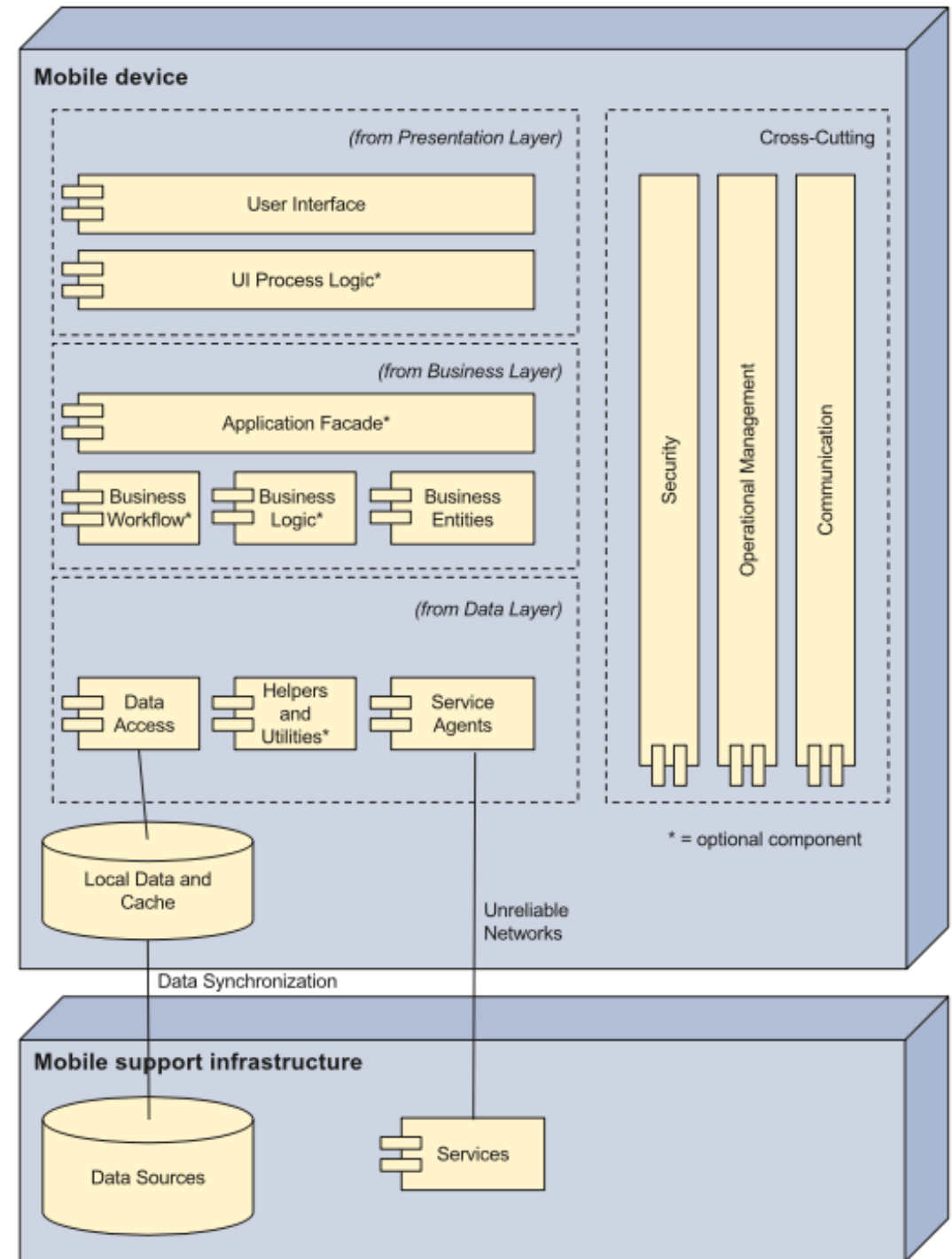
Reference Architecture

- They provide a blueprint for structuring an application. Examples for the enterprise application domain include
 - Mobile applications
 - Rich client applications
 - Rich internet applications
 - Service Applications
 - Web applications

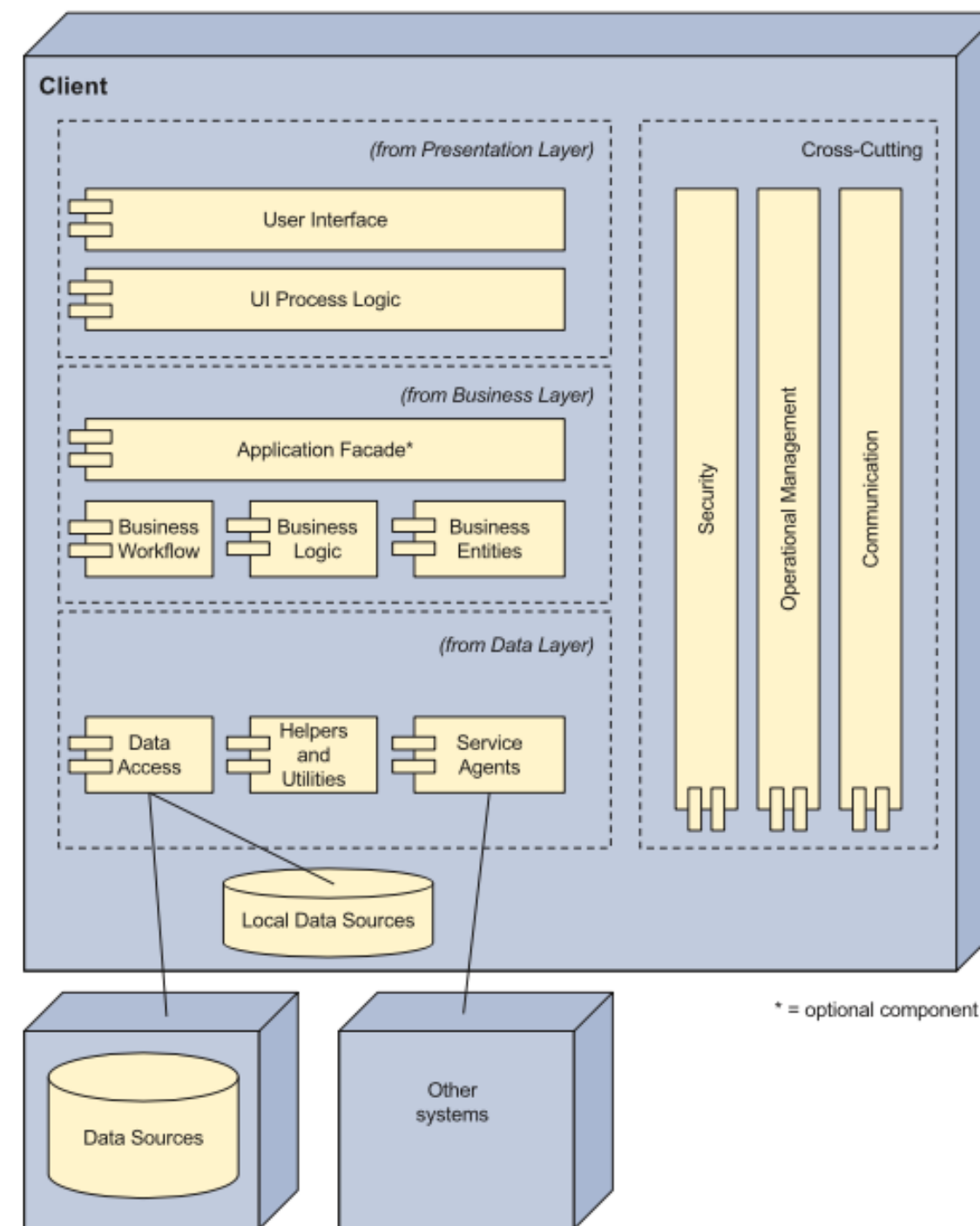


Example reference architecture for the development of web applications from the *Microsoft Application Architecture Guide*

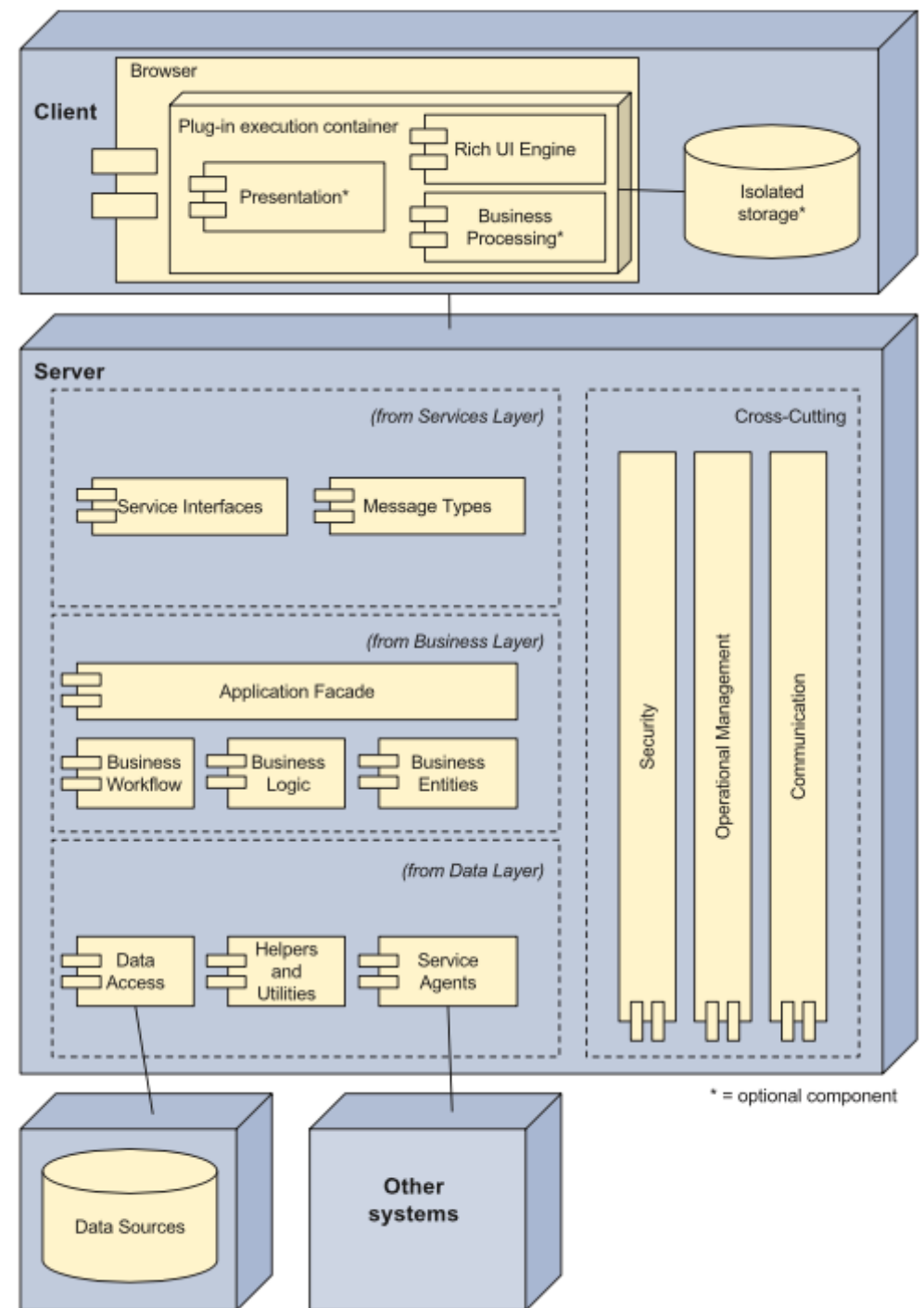
Mobile applications



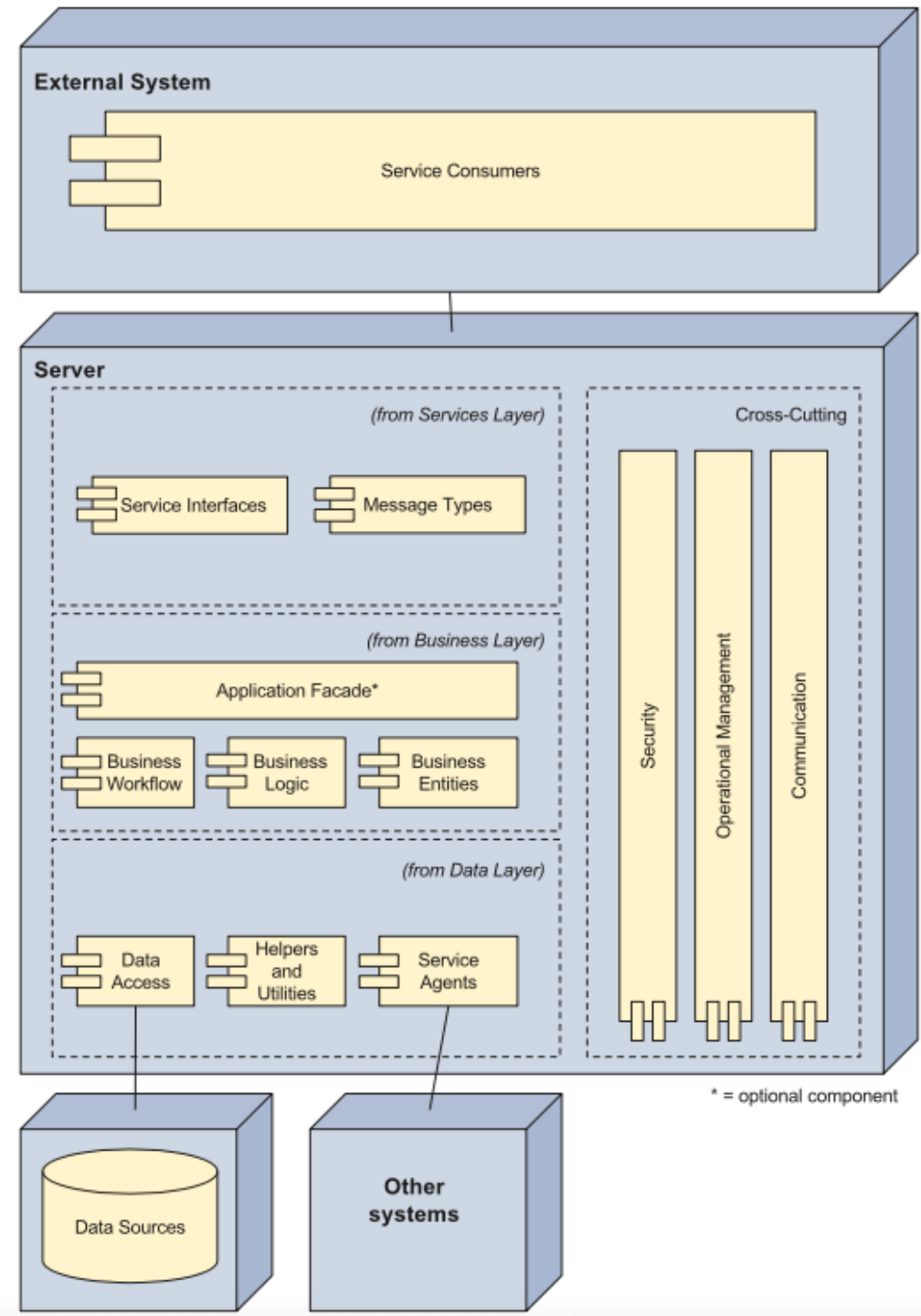
Rich client applications



Rich internet applications

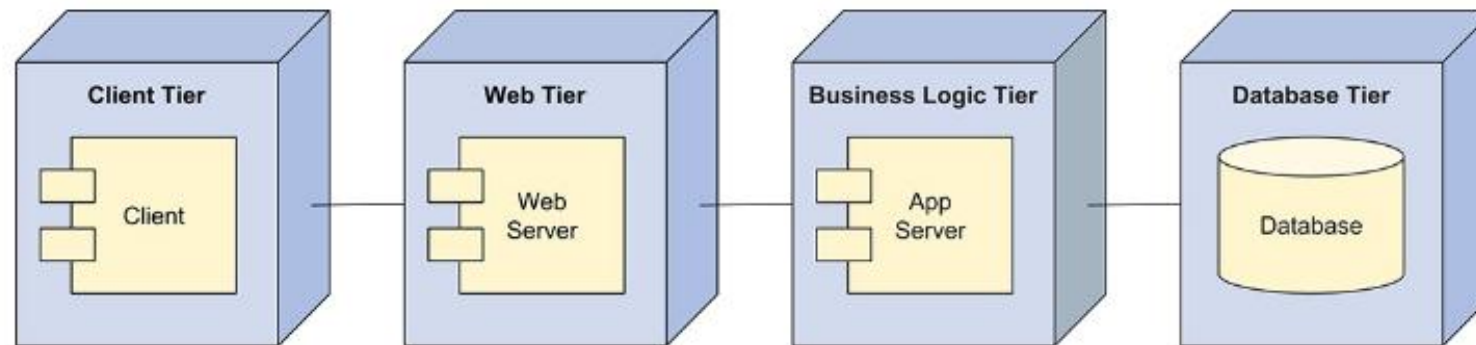


Service applications



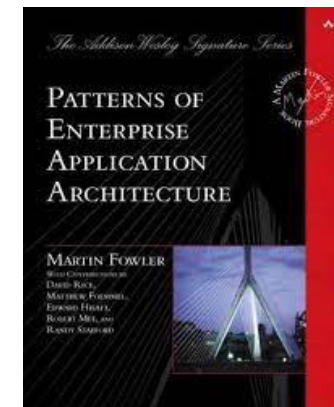
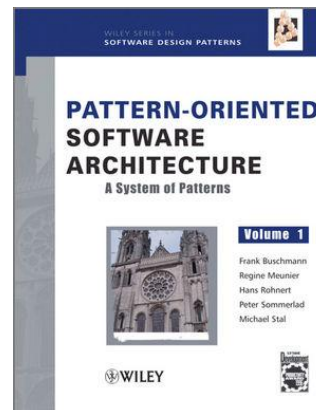
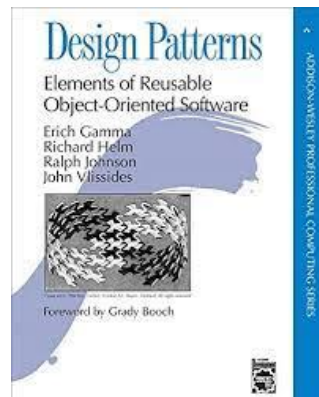
Deployment Patterns

- ❑ Deployment patterns provide **guidance on how to structure the system from a physical standpoint**. Good decisions with respect to the deployment of the software system are essential to achieve quality attributes such as availability.
- ❑ Examples
 - 2, 3, 4 and n-tier deployment, Load balanced cluster, Failover cluster



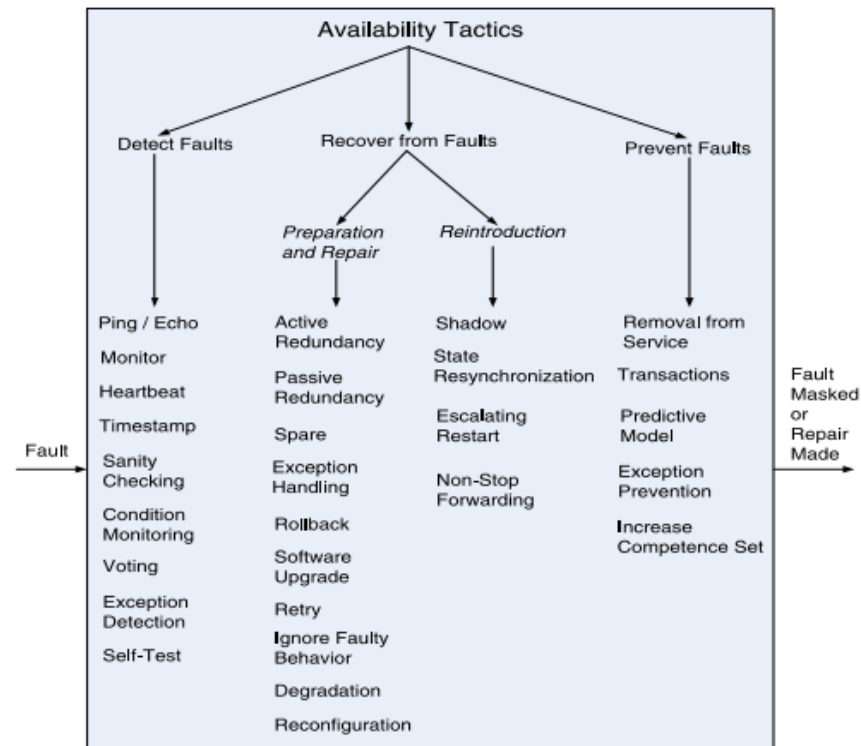
Architectural/Design Patterns

- ❑ Patterns are proven (conceptual) solutions to recurring design problems. Originated in building architecture.
- ❑ Many patterns exist (thousands), and they are documented across several pattern catalogs.
- ❑ It is difficult to draw a clear boundary between “design” and “architectural” patterns



Tactics

- ❑ Design decisions that influence the control of a quality attribute response.
- ❑ There are tactics categorizations for the quality attributes of:
 - Availability
 - Interoperability
 - Modifiability
 - Performance
 - Security
 - Testability
 - Usability



Externally Developed Component

- ❑ These are reusable code solutions
 - E.g. Middleware, frameworks
- ❑ A Framework is a reusable software element that provides generic functionality, addressing recurring concerns across a range of applications.
 - Examples for Java:

Concern	Framework	Usage
Local user interface	Swing	Inheritance
Web UI	Java Server Faces (JSF)	XML, Annotations
Component connection	Spring	XML, Annotations

Identifying Design Concepts

- ❑ In order to make appropriate design decisions during architectural design, you need to consider different alternatives (not just the first thing that comes to mind) that satisfy an iteration goal.
- ❑ Several ways of identifying design candidates

method	description
<i>Leverage existing best practices</i>	You can make use of catalogs that are available in printed or online form.
<i>Leverage your own knowledge and experience</i>	If the system is similar to other systems you have designed in the past, you can begin with some of the design concepts that you have used before.
<i>Leverage the knowledge and experience of others.</i>	You can identify design concepts with some of your peers through brainstorming

Selecting Design Concepts

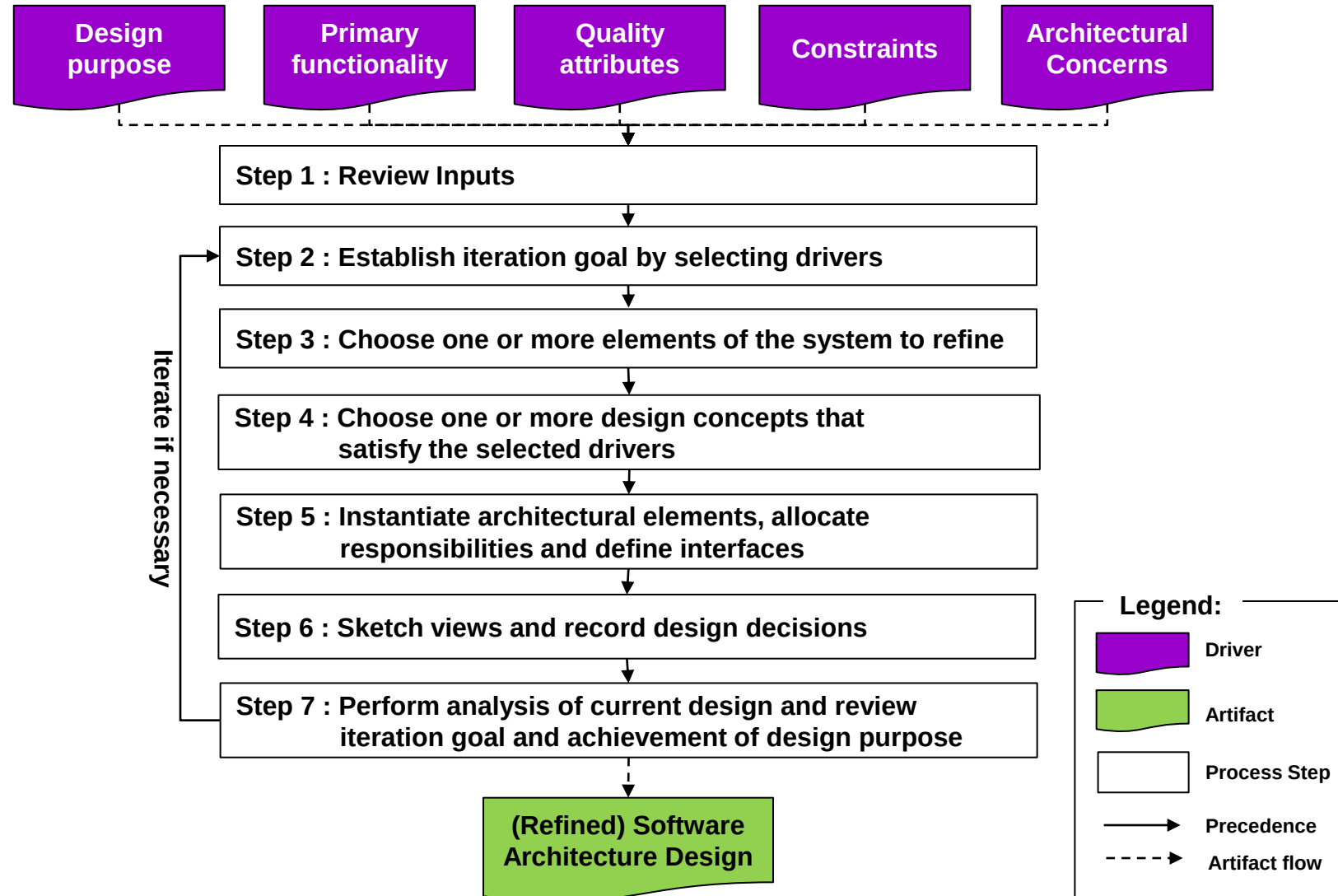
- ❑ Once you have identified a list of alternative design concepts, you need to select one to solve the design problem at hand.
- ❑ You can achieve this by creating a table that lists the pros and cons associated with each alternative

Name of Alternative	Pros	Cons	Cost
Web application	Can be accessed from a variety of platforms using a standard web browser Fast page loading Simple deployment	Does not support “rich” interaction	Low
Rich Internet application	Supports “rich” user interaction Simple deployment and updating	Longer page loading times Requires a runtime environment to be installed on the client browser	Medium
Mobile application	Supports “rich” user interaction	Less portability Screen limitations	High

Attribute-Driven Design (ADD)

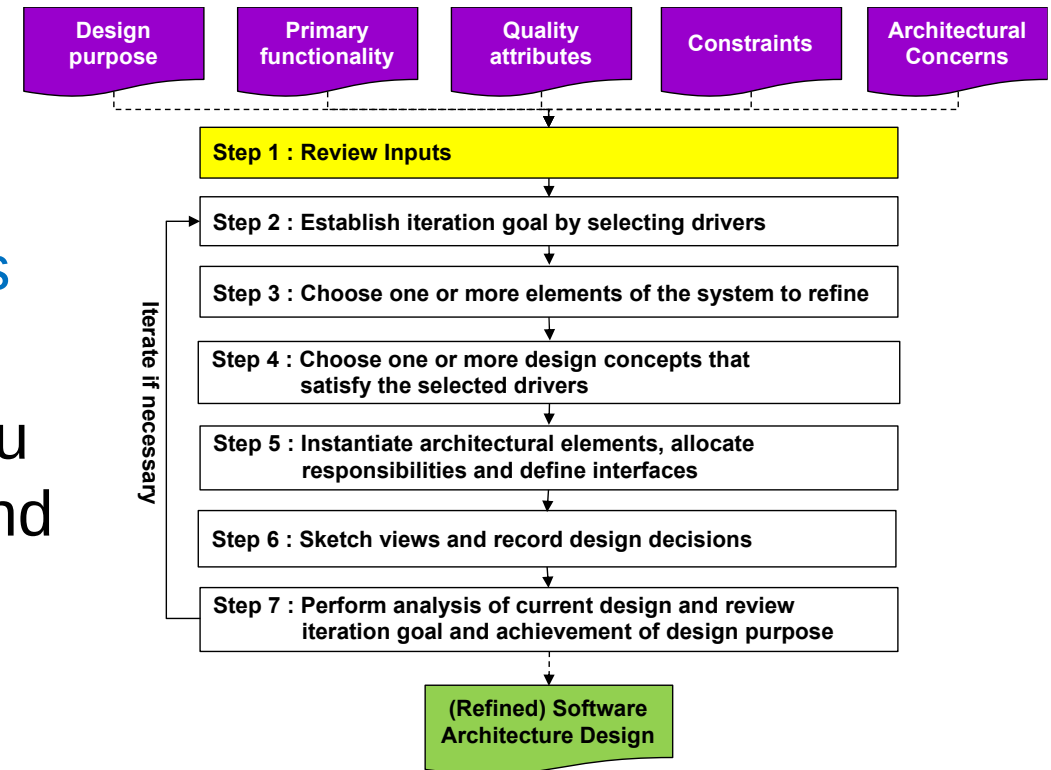
- ❑ **Attribute-Driven Design (ADD) is a method for designing an architecture to satisfy both quality requirements and functional requirements as well as constraints.**
 - Version 2.0 released November 2006
 - Version 3.0 released May 2016
- ❑ **The method promotes an iterative approach to design**
- ❑ **It provides a detailed set of steps for architecture design**

Steps and Artifacts of ADD 3.0



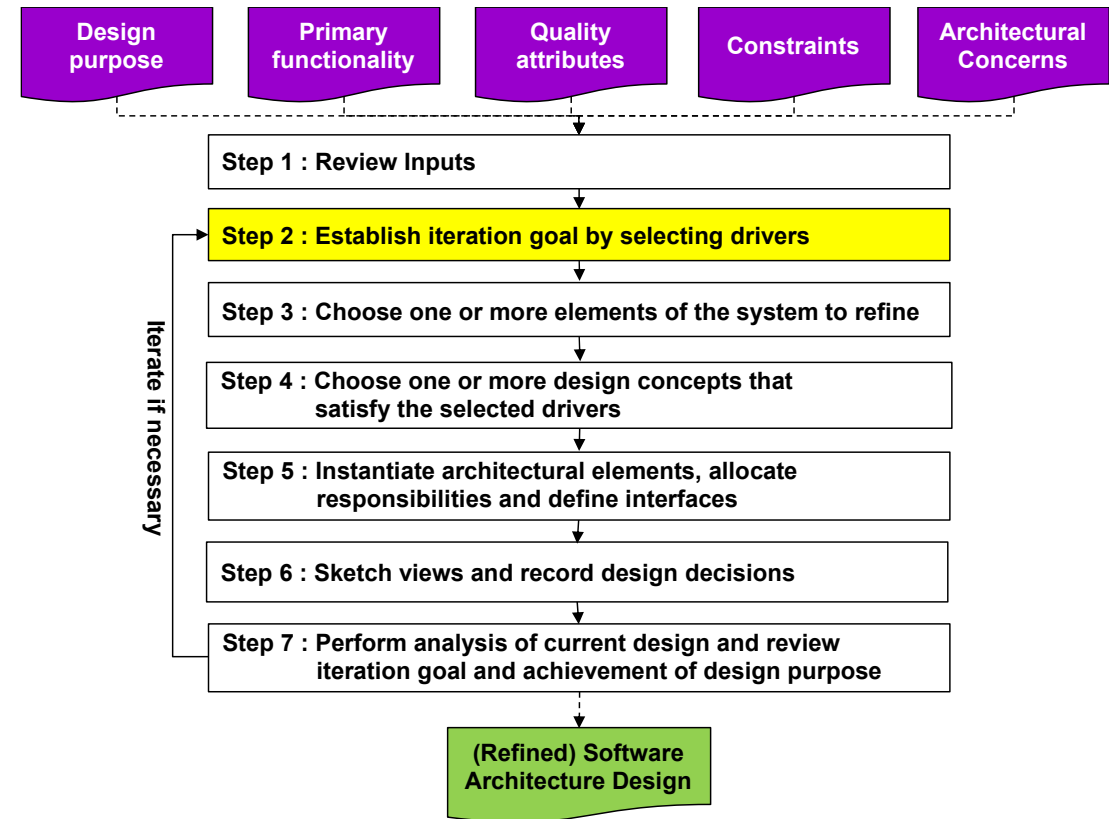
Step1: Review inputs

- ❑ Before starting a design round, you need make sure that the inputs to the design process are available and correct.
- ❑ At this point, we assume that primary functionality and quality attribute scenarios have been prioritized.
- ❑ You should be able to start designing if you have established the primary use cases and the most important quality attribute scenarios



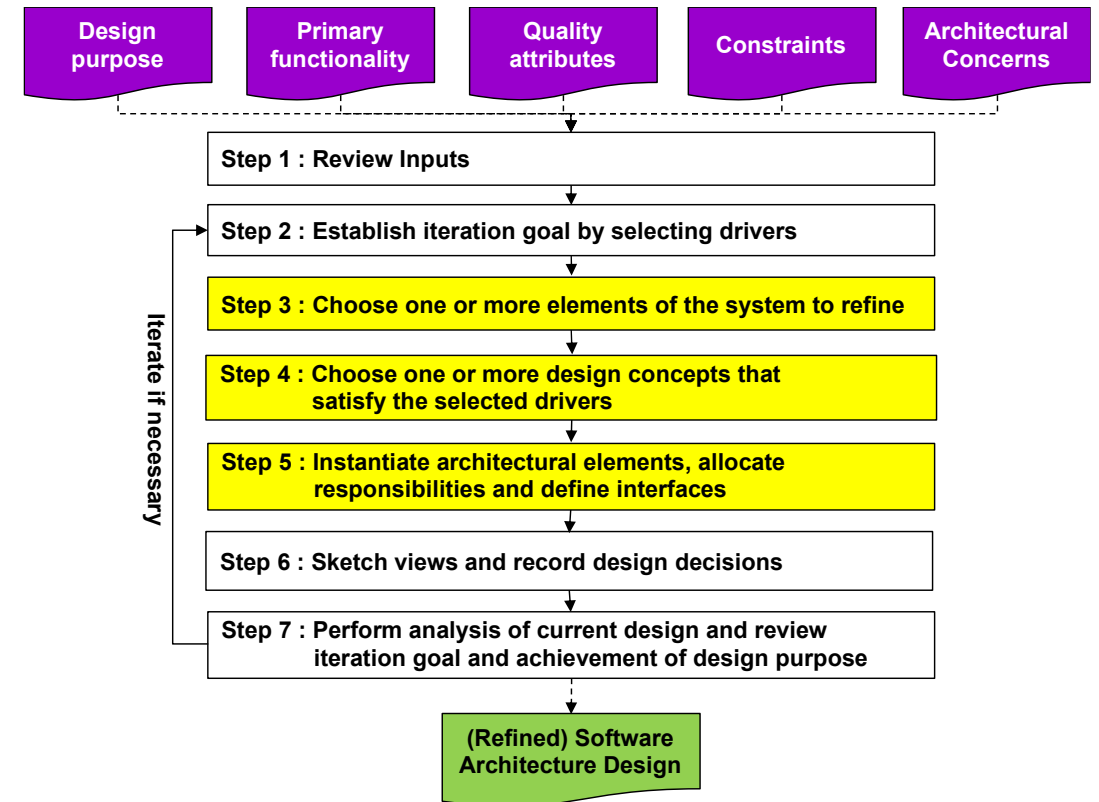
Step 2: Establish iteration goal

- A design round is generally performed in a series of design iterations, where **each iteration focuses on achieving a particular goal.**
- Depending on the type of system whose architecture is being designed, **there may be a “best” ordering of the iteration goals that need to be addressed**



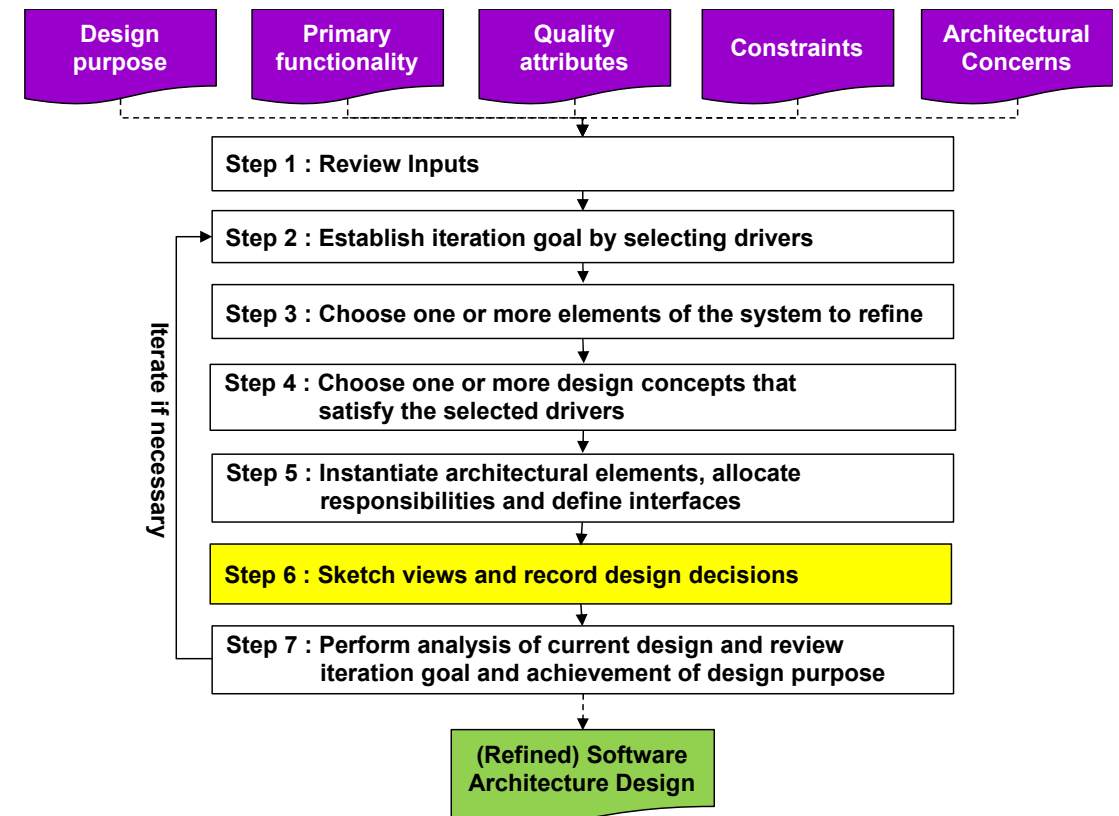
Step 3/4/5: Make design decisions

- Types of decisions are made to address the iteration goal
 - Selection of the parts that need to be decomposed
 - Identification and selection of existing solution that support the decomposition
 - Instantiating elements out of the existing solution
 - Allocating responsibilities to each of the elements
 - Establishment of interfaces between collaborating elements



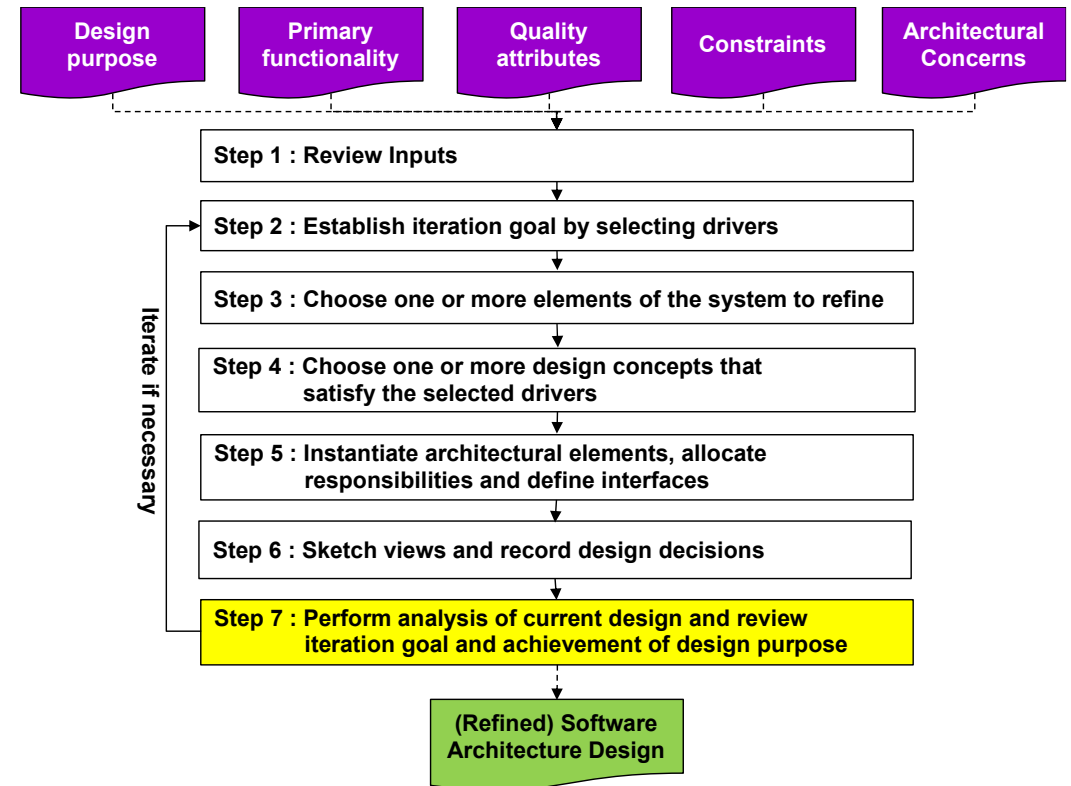
Step 6: Sketch views and Record design decisions

- ❑ This step may be done in parallel with step 5
- ❑ The views – the representation of the structures you created – need to be captured
 - The views are almost certainly incomplete, so these diagrams may need to be revisited and refined in a subsequent iteration.
- ❑ You should record the significant decisions and the reasons that led to these decisions (i.e., the rationale), to facilitate later analysis and understanding of the decisions.



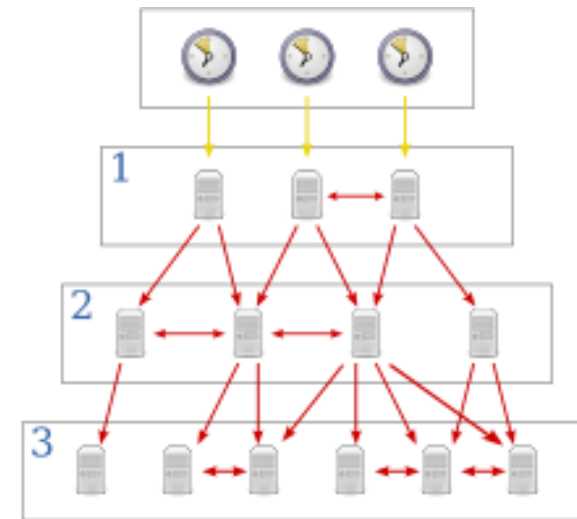
Step7: Perform analysis of current design and review iteration goal and achievement of design purpose

- ❑ You can perform the analysis yourself by reviewing the sketches of the views and design decisions that you recorded, but an even better idea is to have someone else help you review this design.
- ❑ Once the design performed in the iteration has been analyzed, you should review the state of your architecture in terms of the established design purpose.



Case Study: FCAPS System

- ❑ **A management system for a network of time servers**
 - A **time server** is a server computer that reads the actual time from a reference clock and distributes this information to its clients using a computer network.
- ❑ **FCAPS** stand for:
 - Fault management
 - Configuration management
 - Accounting
 - Performance management
 - Security management

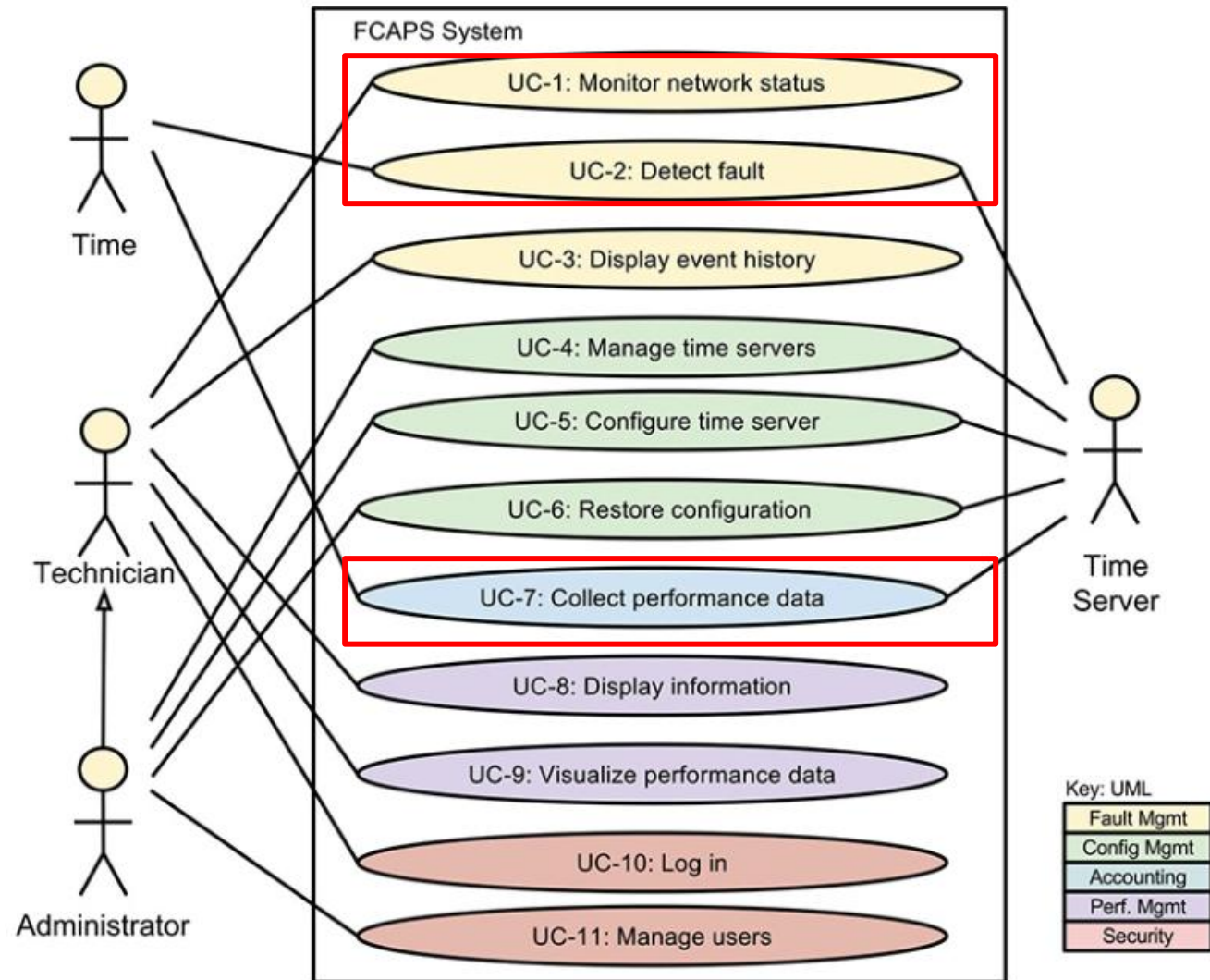


Requirements

- ❑ Requirement elicitation activities had been previously performed, and the following is a summary of the most relevant requirements collected.
 - Use Case Model
 - Quality Attribute Scenarios
 - Constraints
 - Architectural Concerns

All figures and tables after this slide are taken from the following book:
H Cervantes, R Kazman, Designing Software Architectures, A Practical Approach, Addison Wesley, 2016

Use Case Model



Quality Attribute Scenarios

ID	Quality Attribute	Scenario	Associated Use Case
QA-1	Performance	Several time servers send traps to the management system at peak load; 100% of the traps are successfully processed and stored.	UC-2
QA-2	Modifiability	A new time server management protocol is introduced to the system as part of an update. The protocol is added successfully without any changes to the core components of the system.	UC-5
QA-3	Availability	A failure occurs in the management system during normal operation. The management system resumes operation in less than 30 seconds.	All
QA-4	Performance	The management system collects performance data from a time server during peak load. The management system collects all performance data within 5 minutes, while processing all user requests, to ensure no loss of data due to CON-5.	UC-7
QA-5	Performance, usability	A user displays the event history of a particular time server during normal operation. The list of events from the last 24 hours is displayed within 1 second.	UC-3
QA-6	Security	A user performs a change in the system during normal operation. It is possible to know who performed the operation and when it was performed 100% of the time.	All

Constraints

ID	Constraint
CON-1	A minimum of 50 simultaneous users must be supported.
CON-2	The system must be accessed through a web browser (Chrome V3.0+, Firefox V4+, IE8+) in different platforms: Windows, OSX, and Linux.
CON-3	An existing relational database server must be used. This server cannot be used for other purposes than hosting the database.
CON-4	The network connection to user workstations can have low bandwidth but is generally reliable.
CON-5	Performance data needs to be collected in intervals of no more than 5 minutes, as higher intervals result in time servers discarding data.
CON-6	Events from the last 30 days must be stored.

Architectural Concerns

ID	Concern
CRN-1	Establishing an overall initial system structure.
CRN-2	Leverage the team's knowledge about Java technologies, including Spring, JSF, Swing, Hibernate, Java Web Start and JMS frameworks, and the Java language.
CRN-3	Allocate work to members of the development team.

Step 1 : Review Inputs

Category	Details																					
Design purpose	The purpose is to produce a sufficiently detailed design to support the construction of the system																					
Primary functional requirements	From the use case presented earlier, the primary ones were determined to be : UC-1, UC-2, UC-7																					
Quality attribute scenarios	The scenarios presented earlier have been prioritized as follows: <table><tr><th>Scenario ID</th><th>Importance to the Customer</th><th>Difficulty of Implementation According to the Architect</th></tr><tr><td>QA-1</td><td>High</td><td>High</td></tr><tr><td>QA-2</td><td>High</td><td>Medium</td></tr><tr><td>QA-3</td><td>High</td><td>High</td></tr><tr><td>QA-4</td><td>High</td><td>High</td></tr><tr><td>QA-5</td><td>Medium</td><td>Medium</td></tr><tr><td>QA-6</td><td>Medium</td><td>Low</td></tr></table> QA-1, QA-2, QA-3, QA-4 are selected as drivers.	Scenario ID	Importance to the Customer	Difficulty of Implementation According to the Architect	QA-1	High	High	QA-2	High	Medium	QA-3	High	High	QA-4	High	High	QA-5	Medium	Medium	QA-6	Medium	Low
Scenario ID	Importance to the Customer	Difficulty of Implementation According to the Architect																				
QA-1	High	High																				
QA-2	High	Medium																				
QA-3	High	High																				
QA-4	High	High																				
QA-5	Medium	Medium																				
QA-6	Medium	Low																				
Constraints	All of them presented earlier are included as drivers																					
Architectural concerns																						

Iterations

❑ Iteration1

- Establishing an Overall System Structure

❑ Iteration2

- Identifying Structures to Support Primary Functionality

❑ Iteration3

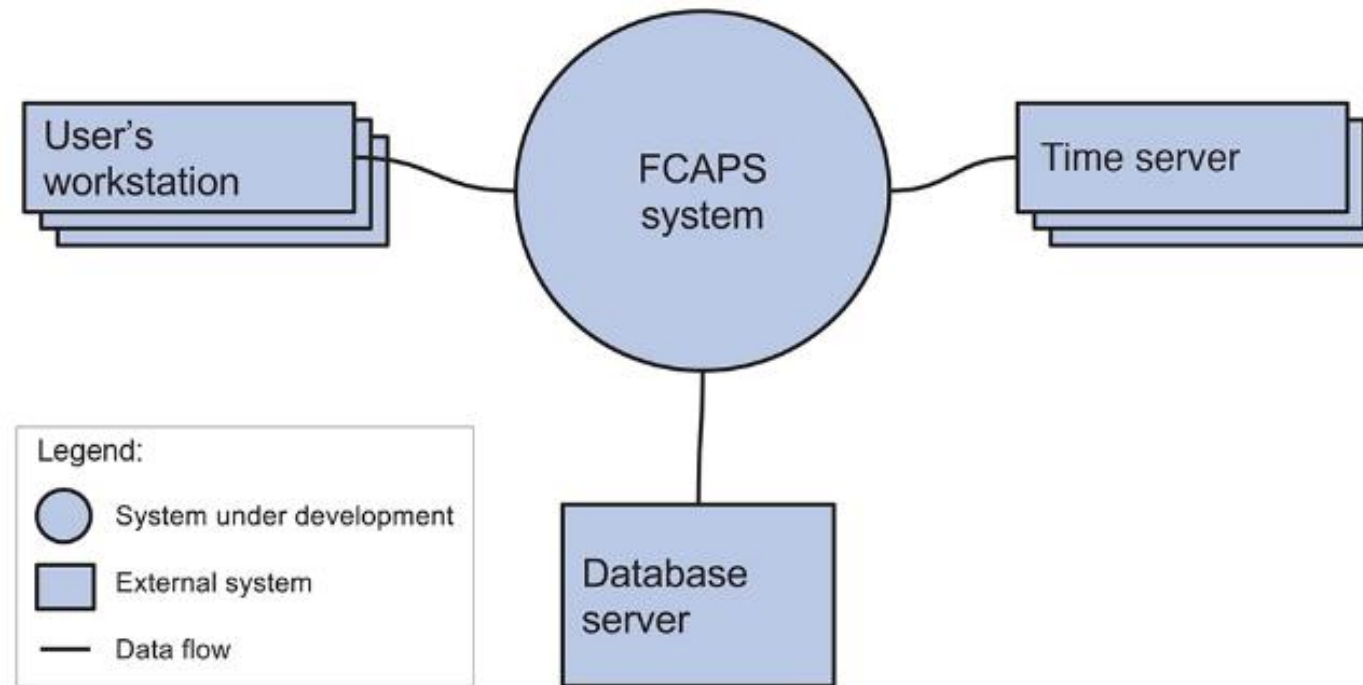
- Addressing Quality Attribute Scenario Driver (QA3)

Step 2 : Establish Iteration Goal

- This is the first iteration in the design of a greenfield system, so the iteration goal is to achieve the architectural concern CRN-1
 - *establishing an overall system structure*

Step 3 : Choose One or More Elements of the System to Refine

- This is a greenfield development effort, so in this case the element to refine is the entire FCAPS system, which is shown below:



Step 4 : Choose One or More Design Concepts

- In this initial iteration, given the goal of structuring the entire system, design concepts are selected

Design Decisions	Rationale	
Logically structure the client part of the system using the Rich Client Application reference architecture	This reference architecture supports the development of applications that are installed in the user PC. These applications support rich user interface capabilities that are needed for UC-1 . Although these types of applications do not run in a web browser (CON-2), they can be installed from a web browser using Java Web Start	
	Alternative	Reason of Discarding
	Rich internet applications (RIA)	This option supports the development of applications with a rich user interface that runs inside a web browser. This option was discarded because plugins for executing RIA were less broadly available than the JVM
	Web applications	This option was discarded because it is difficult to provide a rich user interface experience.

Step 4 : Choose One or More Design Concepts

Design Decisions	Rationale
Logically structure the server part of the system using the Service Application reference architecture	Service applications do not provide a user interface but rather expose services that are consumed by other applications. No other alternatives were considered and discarded, as the architect was familiar with this reference architecture and considered it fully adequate to meet the requirements
Physically structure the application using the three-tier deployment pattern	Since the system must be accessed from a web browser (CON-2) and existing database server must also be used (CON-3), a three-tier deployment is appropriate. Discarding alternatives include other n-tier patterns with $n \neq 3$
Build the user interface of the client application using the Swing Java framework	The standard framework for building Java Rich Client ensure portability (CON-2) and it is what the developers were already familiar with (CRN-2). Discarded alternatives: The Eclipse SWT framework was considered, but the developers were not as familiar with it.

Step 5 : Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

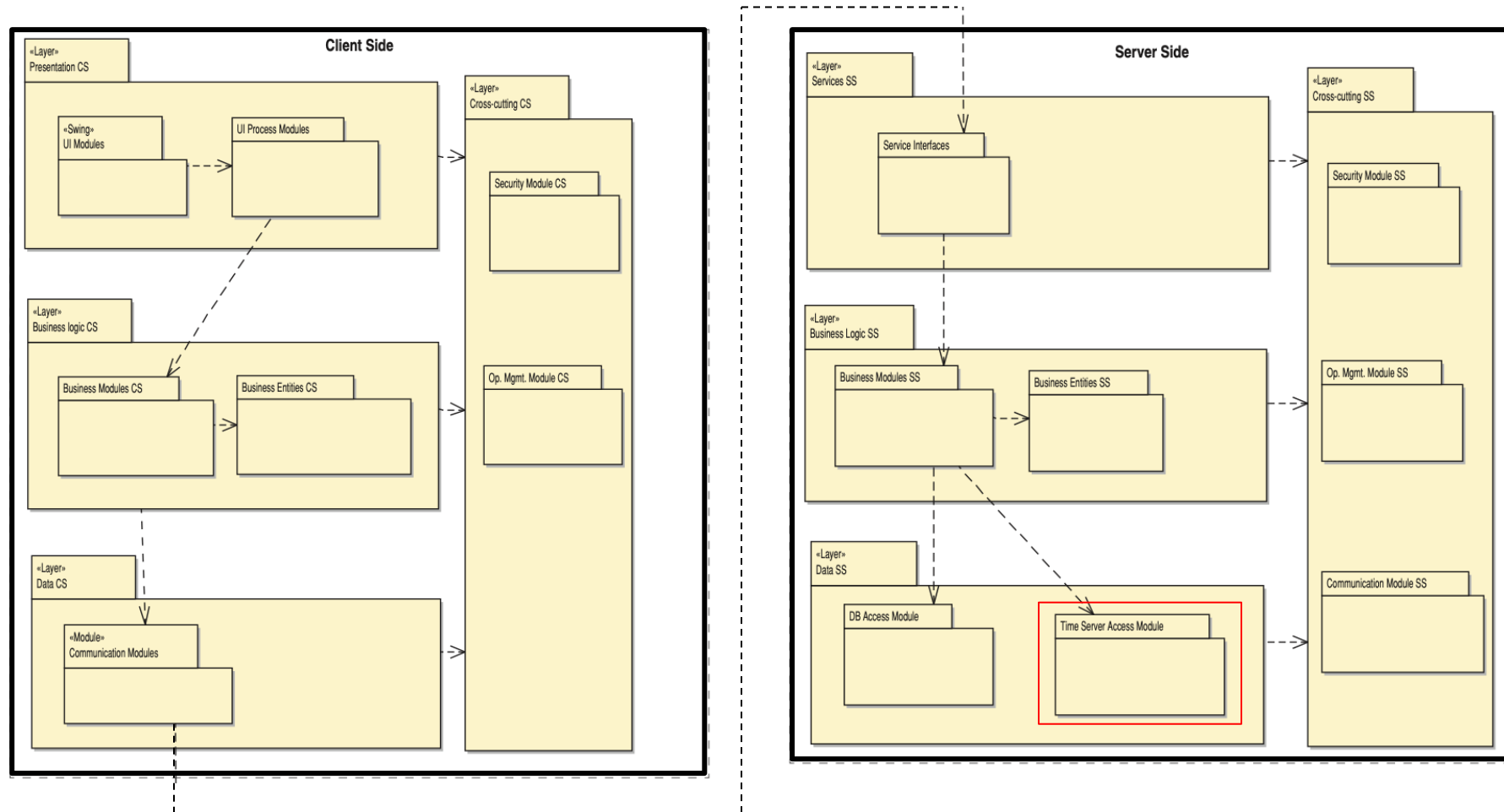
- The instantiation design decisions considered and made are summarized in the following table:

Design Decision and Location	Rationale
Remove local data sources in the rich client application	It is believed that there is no need to store data locally, as the network connection is generally reliable. Also, communication with the server is handled in the data layer. Internal communication between components in the client is managed through local method calls and does not need particular support.
Create a module dedicated to accessing the time servers in the data layer of the Service Application reference architecture	The service agents component from the reference architecture is adapted to abstract the access to the time servers. This will further facilitate the achievement of QA-2 and will play a critical role in the achievement of UC-2 and UC-7.

- In this initial iteration, it is typically **too early to precisely define functionality and interfaces.**

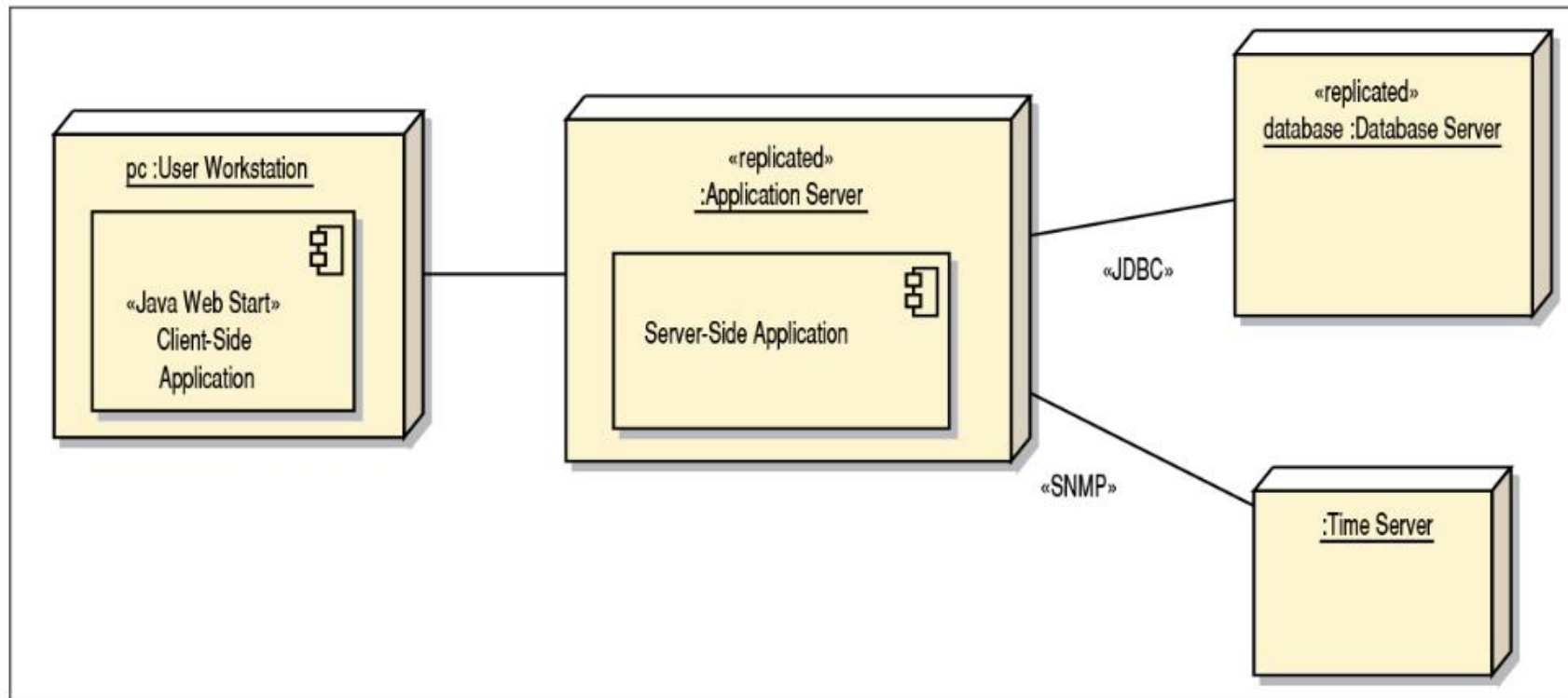
Step 6 : Sketch Views and Record Design Decisions

- The diagram shows the sketch of a module view



Step 6 : Sketch Views and Record Design Decisions

- ❑ The deployment diagram sketches an allocation view that illustrates where the components associated with the modules in the previous diagram will be deployed



Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	UC-1		Selected reference architecture establishes the modules that will support this functionality.
	UC-2		Selected reference architecture
			
QA-1			No relevant decisions made, as it is necessary to identify the elements that participate in the use case that is associated with the scenario.
	QA-2		Introduction of a time server access module in the data layer on the server application that encapsulates communication with the time servers. The details of this component and its interfaces have not been defined yet.
	QA-3		Identification of the elements derived from the deployment pattern that will need to be replicated.

Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	CON-1		Structuring the system using 3 tiers will allow multiple clients to connect to the application server. Decisions regarding concurrent access have not been made yet.
		CON-2	Use of Java Web Start technology allows access through a web browser to download the Rich Client. Since the Rich Client is being programmed in Java, this supports execution under Windows, OSX, and Linux.
		CON-3	Physically structure the application using the 3-tier deployment pattern, and isolate the database by providing database access components in the data layer of the application server.
	CON-4		Use of Java Web Start technology



Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
		CRN-1	Selection of reference architectures and deployment pattern.
	CRN-2		Technologies that have been considered up to this point take into account the knowledge of the developers. Other technologies still need to be selected (e.g., communication with the time servers).
CRN-3			No relevant decisions made.

Iteration 2

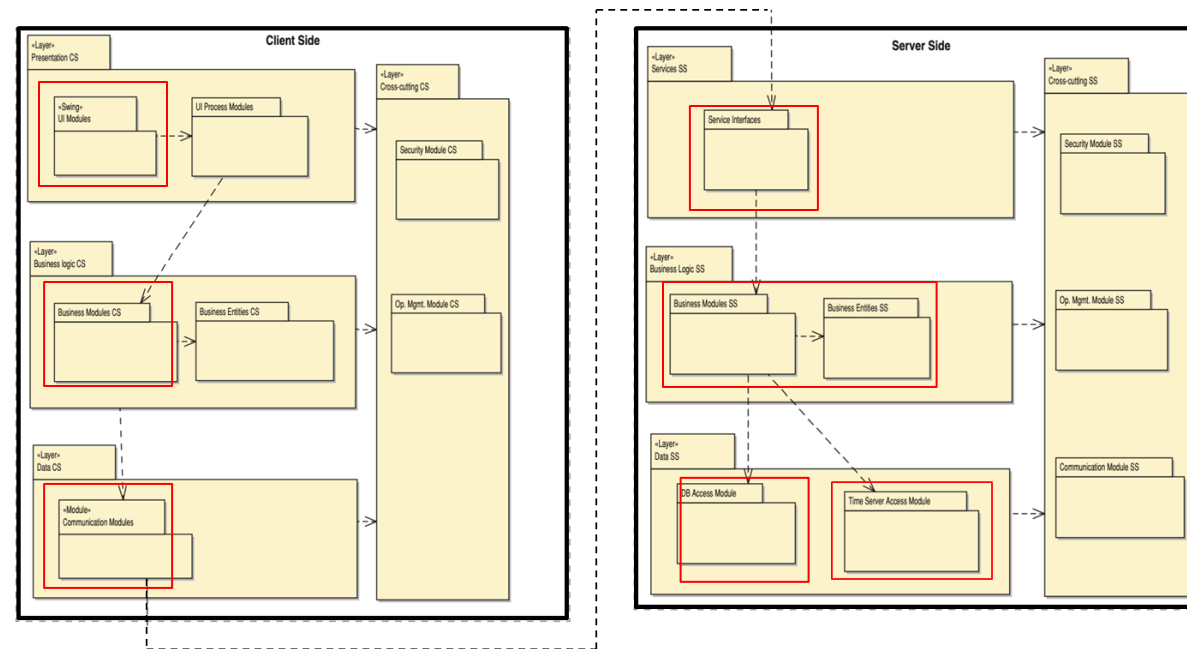
- ❑ In this iteration, we move from the generic and coarse-grained descriptions of functionality used in iteration 1 to more detailed decisions that will drive implementation and hence the formation of development teams.

Step 2 : Establish Iteration Goal

- ❑ The goal of this iteration is to address the general architectural concern of *identifying structures to support primary functionality*.
- ❑ In this second iteration, the architect considers the system's primary use cases:
 - UC-1
 - UC-2
 - UC-7

Step 3 : Choose One or More Elements of the System to Refine

- ❑ The elements that will be refined in this iteration are the modules located in the different layers defined by the two reference architectures from the previous iteration.



Step 4 : Choose One or More Design Concepts

Design Decisions	Rationale
Create a Domain Model for the application	Before starting a functional decomposition, it is necessary to create an initial domain model for the system, identifying the major entities in the domain, along with their relationships. There are no good alternatives.
Identify Domain Objects that map to functional requirements	Each distinct functional element of the application needs to be encapsulated in a self-contained building block – a domain object. One possible alternative is to not consider domain object and instead directly decompose layers into modules, but this increase the risk of not considering a requirement.
Decompose Domain Objects into specialized Modules	Domain objects represent complete sets of functionality, but this functionality is supported by finer-grained elements located within the layers. Specialization of modules is associated with the layers where they are located There are no good alternatives

Step 4 : Choose One or More Design Concepts

Design Decisions	Rationale	
Use Spring framework and Hibernate	Spring is a widely used framework to support enterprise application development. Hibernate is an object to relational mapping (ORM) framework that integrates well with Spring.	
	Alternative	Reason of Discarding
	JEE	Spring was selected because it was considered more “lightweight” and the development team was already familiar with it, resulting in greater and earlier productivity.
	Other ORM frameworks	Development team already was familiar with, and happy with the performance of, Hibernate .

Step 5 : Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

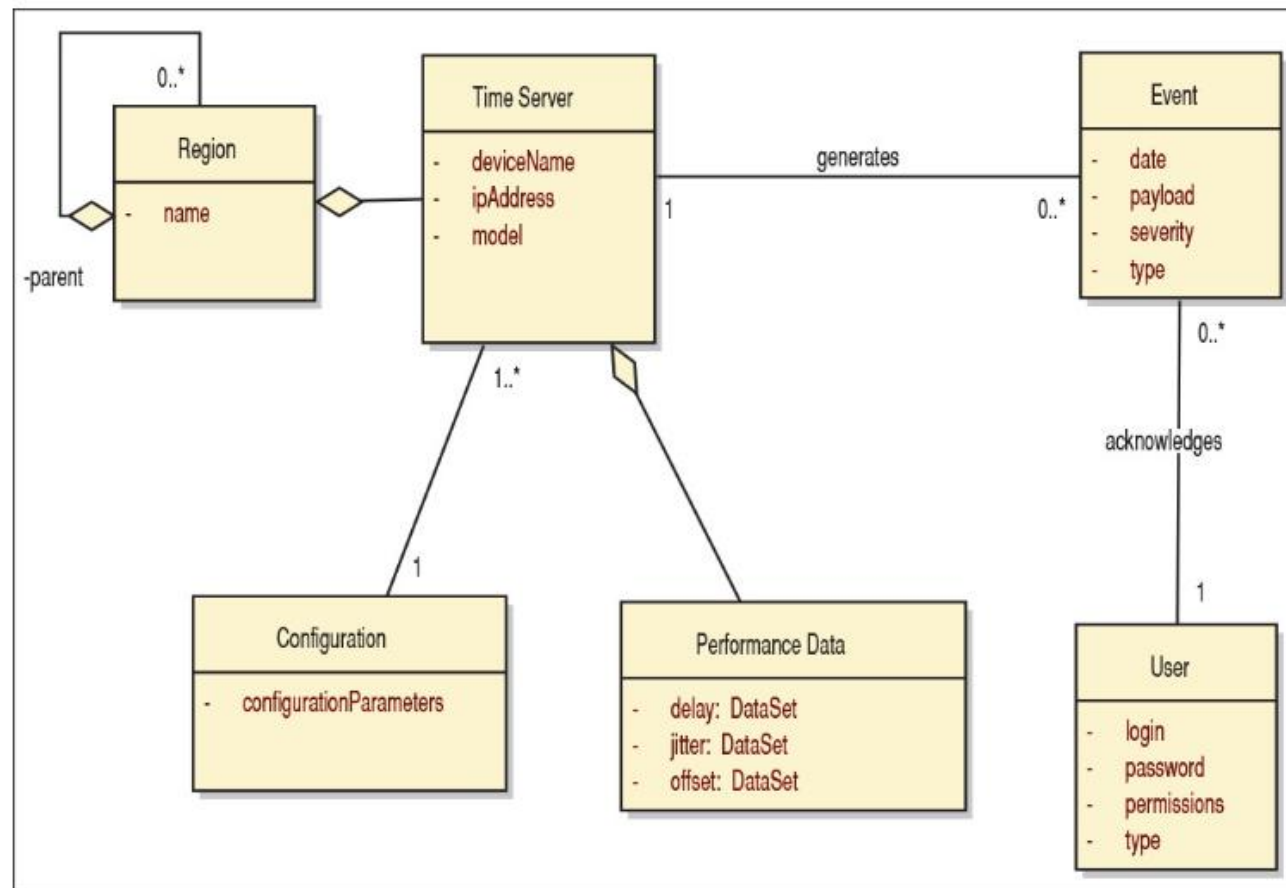
- ❑ The instantiation design decisions considered and made are summarized in the following table:

Design Decisions	Rationale
Create only an initial domain model	
Map the system use cases to domain objects	
Decompose the domain objects across the layer to identify layer-specific modules with an explicit interface	
Connect components associated with modules using Spring	
Associate frameworks with a module in the data layer	

- ❑ While the structures and interfaces are identified in this step of the method, they are captured in the next step.

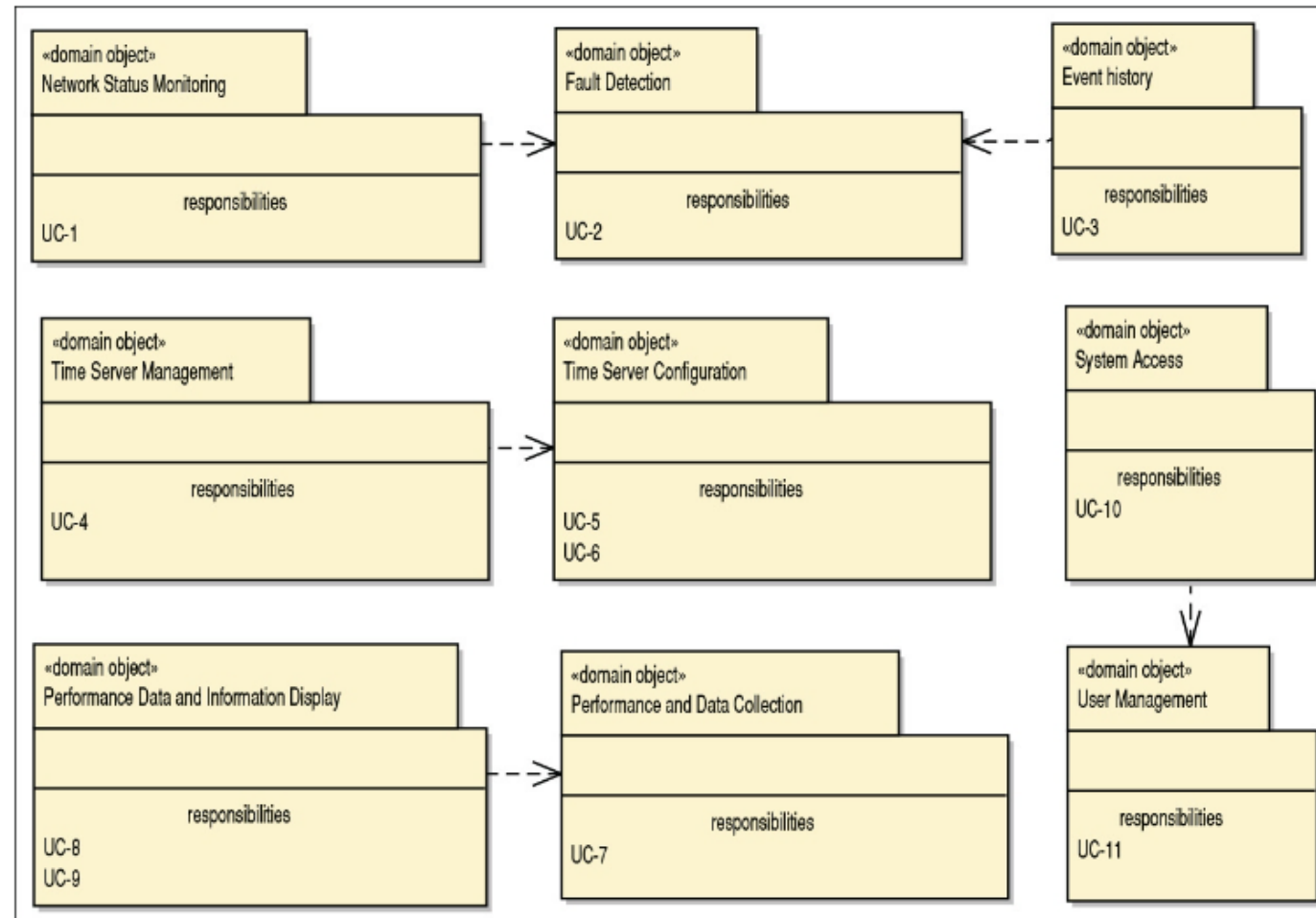
Step 6 : Sketch Views and Record Design Decisions

▣ An initial domain model



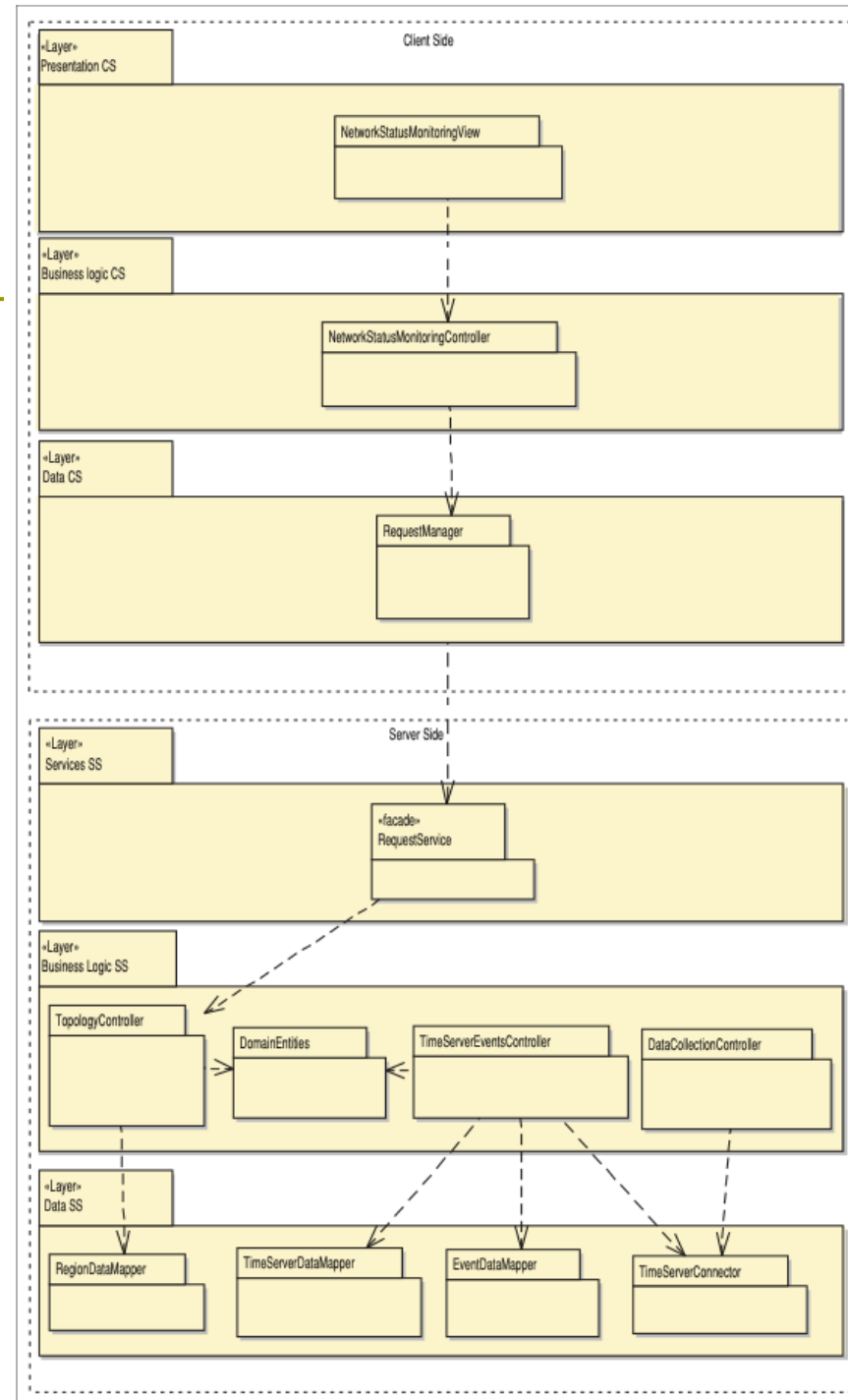
Step 6 : Sketch Views and Record Design Decisions

- the domain objects that are instantiated for the use case model



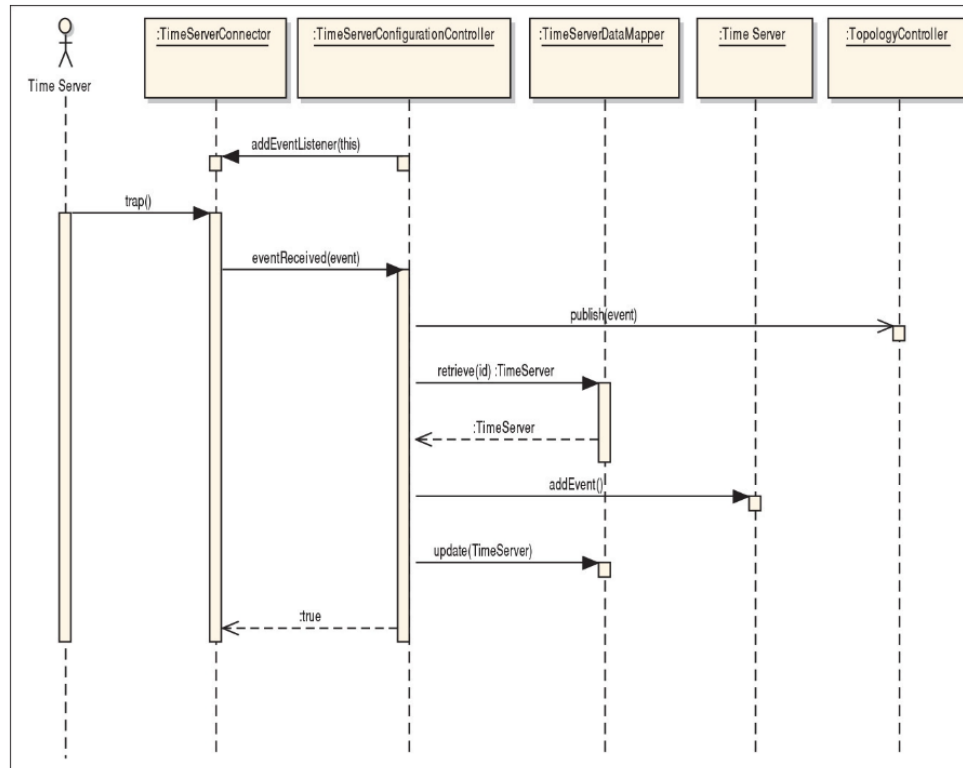
Step 6 : Sketch Views and Record Design Decisions

- a sketch of a module view with modules that are derived from the domain objects and associated with the primary use cases.
 - Note that explicit interfaces are not shown but their existence is assumed.



Step 6 : Sketch Views and Record Design Decisions

- The following sequence diagram for UC-2 was created to define interfaces of the interacting elements



Interfaces of the interacting elements

Method Name	Description
Element: TimeServerConnector	
boolean addEventListener (EventListener el)	This method allows components from the business logic to register themselves as listeners to events that are received from the time servers.
Element: TimeServerConfigurationController	
boolean eventReceived(Event evt)	This callback method is invoked when an event is received.
Element: TopologyController	
publish(Event evt)	This method notifies the clients that a new event has occurred.
Element: TimeServerDataMapper	
TimeServer retrieve(int id)	Retrieves a TimeServer identified by its id.
boolean update(TimeServer ts)	Persists changes in a TimeServer.

Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
		UC-1	Modules across the layers and preliminary interfaces to support this use case have been identified.
		UC-2	Modules across the layers and preliminary interfaces to support this use case have been identified.
		UC-7	Modules across the layers and preliminary interfaces to support this use case have been identified.
	QA-1		The elements that support the associated use case (UC-2) have been identified.
	QA-2		The elements that support the associated use case (UC-5) have been identified.
	QA-3		No relevant decisions made.
	QA-4		The elements that support the associated use case (UC-7) have been identified.
	CON-1		No relevant decisions made.
	CON-4		No relevant decisions made.

Iteration 3

- ▣ Building on the fundamental structural decisions made in iterations 1 and 2, we can now start to reason about the fulfillment of some of the more important quality attributes.
- ▣ This iteration focuses on just one of these quality attribute scenarios

Step 2 : Establish Iteration Goal

- For this iteration, the architect focuses on the **QA-3** quality attribute scenario:
 - A failure occurs in the management system during operation. The management system resumes operation in less than 30 seconds.

Step 3: Choose One or More Elements of the System to Refine

- For this availability scenario, the elements that will be refined are the physical nodes that were identified during the first iteration:
 - Application server
 - Database server

Step 4: Choose One or More Design Concepts

- The design concepts used in this iteration are the following:

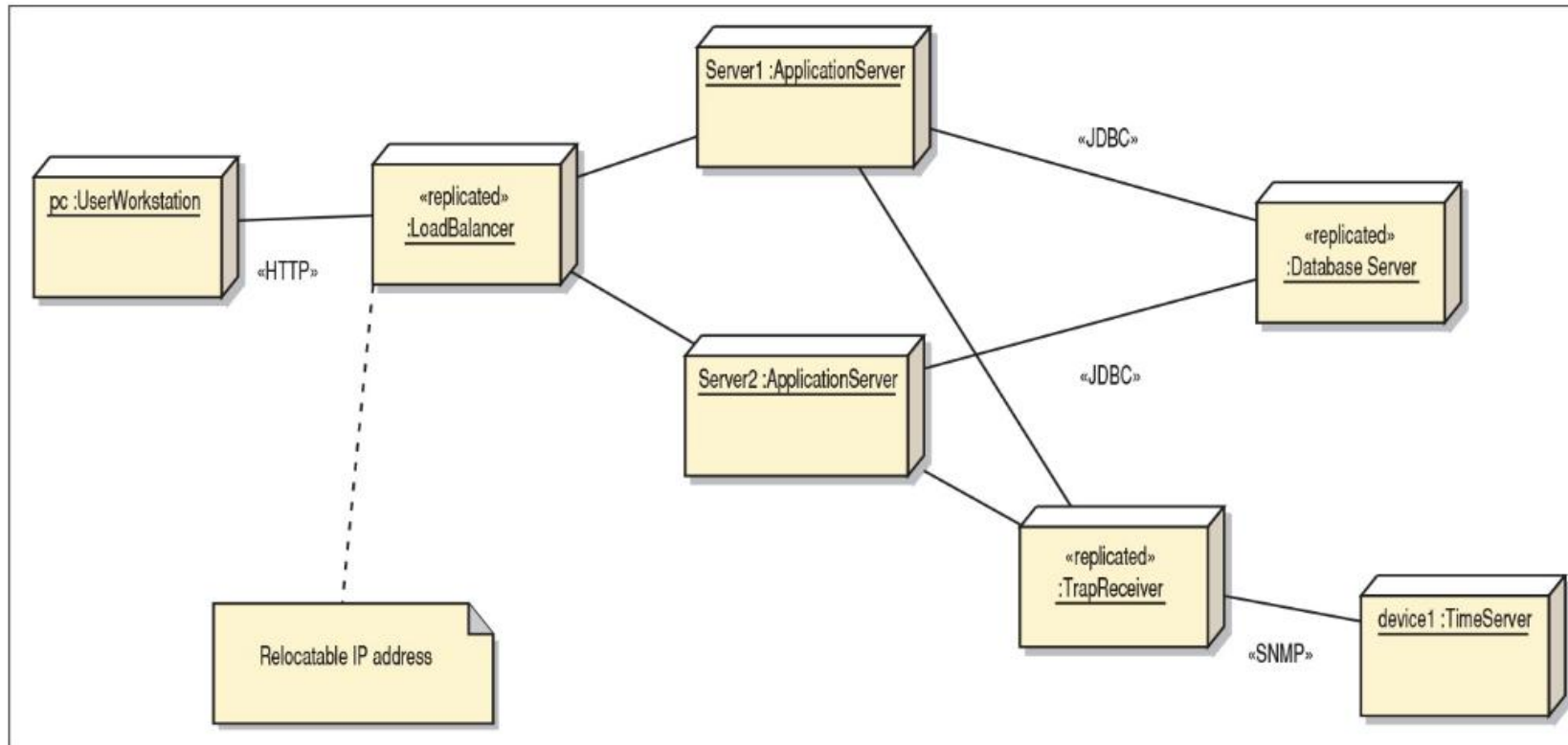
Design Decisions and Location	Rationale and Assumptions
Introduce the active redundancy tactic by replicating the application server and other critical components such as the database	By replicating the critical elements, the system can withstand the failure of one of the replicated elements without affecting functionality.
Introduce an element from the message queue technology family	Traps received from the time servers are placed in the message queue and then retrieved by the application. Use of a queue will guarantee that traps are processed and delivered in order (QA-1).

Step 5 : Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Design Decisions and Location	Rationale
Deploy message queue on a separate node	Deploying the message queue on a separate node will guarantee that no traps are lost in case of application failure. This node is replicated using the tactic of active redundancy, but only one copy receives and treats events coming from the network devices.
Use active redundancy and load balancing in the application server	Because two replicas of the application server are active at any time, it makes sense to distribute and balance the load among the replicas. This tactic can be achieved through the use of the Load-Balanced Cluster pattern (see Section A.2.3). This introduces a new architectural concern, CRN-5: Manage state in replicas.
Implement load balancing and redundancy using technology support	Many technological options for load balancing and redundancy can be implemented without having to develop an ad hoc solution that would be less mature and harder to support.

Step 6: Sketch Views and Record Design Decisions

- ❑ a refined deployment diagram that includes the introduction of redundancy in the system



Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	QA-1		The introduction of a separate replicated trap receiver node can help ensure 100% of the traps are processed, even in the case of a failure of the application server. Furthermore, because trap reception is performed in a separate node, this approach reduces application server processing load, thereby helping performance. Because specific technologies have not been chosen, this driver is marked as “partially addressed”.
	QA-2		No relevant decisions made.
	QA-3		By making the application server redundant, we reduce the probability of failure of the system. Furthermore, if the load balancer fails, a passive replica is activated within the required time period. Because specific technologies have not been chosen (message queue), this driver is marked as “partially addressed”.
	QA-4		No relevant decisions made.

Step 7: Perform Analysis of Current Design and Review Iteration Goal

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	CON-1		Replication of the application server and the use of a load balancer will help in supporting multiple user requests.
	CON-4		No relevant decisions made.
	CON-5		No relevant decisions made.
	CON-6		No relevant decisions made.
	CRN-2		No relevant decisions made.
	CRN-4		No relevant decisions made.
CRN-5			This new architectural concern is introduced in this iteration: manage state in replicas. At this point, no relevant decisions have been made.

Summary

- ❑ **Architectural design** is the activity of creating an architecture by making series of design decisions that satisfy the given requirements
- ❑ During architectural design, possible design options should be explored and evaluated before making decisions.
- ❑ Architectural drivers are requirements that shape the architecture. These include:
 - Primary functionality, QAs, Constraints, Design Purpose, Architectural Concerns
- ❑ Design concepts are reusable building blocks that can be used for creating structures. These include:
 - Reference architectures, Architectural Design Patterns, Deployment Patterns, Tactics, Externally Development Component
- ❑ Attribute-Driven Design (ADD) is an iterative approach to designing an architecture to satisfy both quality requirements and functional requirements as well as constraints.